

Bertrand Meyer

---

**CSC-3TC33-TP : ALGORITHMES ET  
FONDEMENTS DE L'INFORMATIQUE**

---

*Bertrand Meyer*

*Automne 2024*

---

1. Version du 11 novembre 2024.

**CSC-3TC33-TP : ALGORITHMES ET FONDEMENTS DE  
L'INFORMATIQUE**

**Bertrand Meyer**



Ces notes sont en cours d'écriture et contiennent probablement des erreurs ou des coquilles. Tout retour sera apprécié. Une version plus complète (comprenant des corrigés d'exercices) et mise à jour est à disposition sur le site web du cours.

Merci à Irène Charon et Olivier Hudry pour leur autorisation de reprendre de nombreux éléments de [CH19]. Merci de même à David Madore pour son autorisation de reprendre de nombreux éléments de [Mad23].



## TABLE DES MATIÈRES

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| <b>1. Concevoir un programme</b> .....                                    | 9   |
| 1.1. Introduction.....                                                    | 9   |
| 1.2. Spécification, preuve et test.....                                   | 10  |
| 1.3. Ressources, performance et complexité.....                           | 16  |
| 1.4. Stratégies algorithmiques et schémas classiques.....                 | 20  |
| 1.5. Exercices.....                                                       | 39  |
| <b>2. Structures de données</b> .....                                     | 43  |
| 2.1. Induction structurelle.....                                          | 43  |
| 2.2. Type de donnée abstrait.....                                         | 75  |
| 2.3. Types abstraits usuels et implémentations classiques.....            | 80  |
| 2.4. Exercices.....                                                       | 105 |
| <b>3. Algorithmes de graphe</b> .....                                     | 117 |
| 3.1. Éléments de théorie des graphes.....                                 | 117 |
| 3.2. Parcourir un graphe.....                                             | 124 |
| 3.3. Problèmes de plus court chemins.....                                 | 145 |
| 3.4. Arbres couvrants.....                                                | 155 |
| 3.5. Couplages et graphes bipartis.....                                   | 160 |
| 3.6. Exercices.....                                                       | 166 |
| <b>4. Manipulations de chaînes de caractère et langages formels</b> ..... | 191 |
| 4.1. Recherche de motifs.....                                             | 193 |
| 4.2. Automates et langages rationnels.....                                | 209 |
| 4.3. Exercices.....                                                       | 253 |
| <b>5. Décidabilité et complexité</b> .....                                | 259 |
| 5.1. Décidabilité.....                                                    | 259 |
| 5.2. Classes de complexité.....                                           | 269 |
| 5.3. Les problèmes <b>NP</b> -complets.....                               | 273 |
| 5.4. Exercices.....                                                       | 277 |

|                                                  |            |
|--------------------------------------------------|------------|
| <b>6. Éléments de corrigé.....</b>               | <b>285</b> |
| 6.1. Concevoir un programme.....                 | 285        |
| 6.2. Structures de données.....                  | 289        |
| 6.3. Algorithmes de graphes.....                 | 308        |
| 6.4. Langages.....                               | 369        |
| 6.5. Décidabilité et complexité.....             | 390        |
| <b>A. Aide à la programmation en Python.....</b> | <b>401</b> |
| A.1. Les objets en Python.....                   | 401        |
| <b>Index.....</b>                                | <b>405</b> |
| <b>Bibliographie.....</b>                        | <b>407</b> |



# CHAPITRE 1

## CONCEVOIR UN PROGRAMME

### 1.1. Introduction

Ce cours s'intéresse principalement à la résolution de problèmes de calcul par des moyens mécaniques et a pour objet d'offrir des outils théoriques de réflexion sur la conception de systèmes qui produisent des solutions de manière automatique.

**Exemple 1.1.** — Voici quelques exemples de problèmes que nous pourrions vouloir résoudre :

- Préparer la meilleur mousse au chocolat du monde
- Calculer la somme des entiers d'une liste
- Trier des copies
- Amener une voiture automatique à sa destination
- Choisir une liste de lecture musicale conforme aux goûts de son auditeur.

Dans ce contexte, une notion familière est la notion d'algorithme.

**Définition 1.2.** — Un *algorithme* est une suite d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécuté de façon automatique.

**Remarque 1.3.** — Ces dernières années, la langue commune a tendance à mélanger les termes « algorithmes » et « intelligence artificielle » pour désigner de façon floue une boîte noire qui rend des décisions toute seule. Du point de vue d'un informaticien, on réservera le terme IA à toute forme d'algorithme qui essaie de produire un comportement intelligent, aussi subjectif que ce critère puisse être.

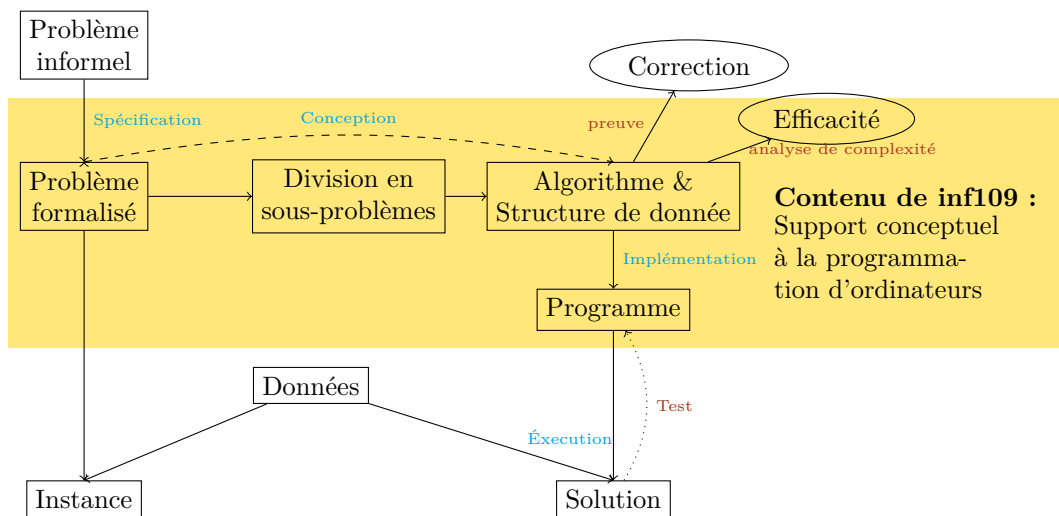
Nous repoussons à la fin du cours (chapitre 5) la question de savoir ce que signifie calculer avec une machine. Nous nous contentons de supposer que nous disposons d'une machine idéale dotée

- d'une mémoire, avec en particulier des types élémentaires prédéfinis (entiers, flottants, booléens, etc) et une façon de les combiner entre eux,
- d'opérations élémentaires (lecture et écriture en mémoire, opérations arithmétiques ou booléennes, etc.

— d’un processeur, qui effectue les opérations en temps constant.

Ce modèle nous fournit une abstraction qui permet de parler d’algorithmes indépendamment du matériel et du langage de programmation. Pour illustrer notre propos, nous nous appuyerons sur des implémentations en langage Python et occasionnellement en langage C.

Avant de nous précipiter dans l’étude des algorithmes, essayons de prendre du recul sur l’ensemble du contexte : nous essayons de résoudre un problème, ou plutôt une classe de problème. Nous parlerons d’*instance d’un problème* pour désigner un cas particulier spécifique de ce problème détaillé avec l’ensemble des données d’entrées.



**Exemple 1.4.** — Disons que l’on cherche le meilleur chemin pour aller de Paris à Rome, ce qui est une façon informelle de poser un problème. Le problème formalisé pourrait être : étant donné un graphe pondéré  $G = (V, E, \rho)$ , avec  $\rho : E \rightarrow \mathbb{R}$  et deux sommets  $s$  et  $t$  de  $V$ , trouver le chemin  $P$  de  $s$  vers  $t$  de poids  $\rho(P)$  le plus léger. Un algorithme permettant de résoudre ce problème pourrait être l’algorithme de Dijkstra. Une instance du problème serait la donnée du graphe de toutes routes européennes avec leur longueur et les villes de Paris et Rome comme sommets particuliers.

## 1.2. Spécification, preuve et test

**1.2.1. Spécification.** — Avant de parler du « comment » résoudre un problème, il convient de préciser le « quoi ».

**Définition 1.5.** — On appelle *spécification d’un problème ou d’un algorithme* la description sans ambiguïté des données fournies en entrées, des effets sur l’environnement et de la valeur de retour.

**Remarque 1.6.** — Pour pouvoir raisonner sur des programmes, il est en général nécessaire de caractériser complètement une spécification dans un langage formel ou

un langage mathématique. On peut par exemple formuler un prédicat logique satisfait par les données manipulées en entrée ou en sortie.

Formellement, une spécification est donc constituée de *préconditions*, c'est-à-dire de propriétés dont jouissent les données d'entrée et l'environnement avant exécution du programme, et de *postconditions*, c'est-à-dire de propriétés dont jouissent les données de sortie et l'environnement après exécution du programme.

Détailler les préconditions ou les postconditions présente de nombreux avantages, selon l'agent qu'on incarne :

- Pour celle ou celui qui conçoit l'algorithme, les préconditions délimitent les entrées que l'on cherche à traiter et fournissent le cadre du problème à résoudre. Les postconditions expliquent ce que l'on cherche à accomplir
- Pour celle ou celui qui raisonne sur l'algorithme, les préconditions et les postconditions deviennent les hypothèses et les conclusions de théorème que l'on cherche à montrer.
- Pour celui ou celle qui utilise l'algorithme, les préconditions et les postconditions fournissent un mode d'emploi.
- Pour celui ou celle qui implémente l'algorithme, les préconditions et les postconditions peuvent devenir des tests vérifiés à l'exécution, qui éventuellement interrompent le programme avec un message d'erreur qui informe l'utilisateur de sa méconduite.

**Exemple 1.7.** — La spécification de l'algorithme de *calcul du plus grand commun diviseur* est la suivante.

- Le programme doit recevoir en entrée deux paramètres, disons  $a$  et  $b$ , qui sont des entiers. Les préconditions sont
  1. On a  $a > 0$ .
  2. On a  $b > 0$ .
- La valeur de retour de l'algorithme est un entier  $d$ . Les postconditions sont les suivantes :
  1. On a  $d > 0$ .
  2. L'entier  $d$  divise  $a$ .
  3. L'entier  $d$  divise  $b$ .
  4. Pour tout entier  $n$  qui divise  $a$  et  $b$ ,  $n$  divise  $d$ .

**Exemple 1.8.** — La spécification de l'algorithme du drapeau hollandais est la suivante.

- Le programme doit recevoir en entrée un tableau  $T$  de longueur  $n$ . Les préconditions sont
  1. Les cases du tableau prennent trois valeurs : rouge, blanc ou bleu.



- Les postconditions sont les suivantes :

1. Le contenu de  $T$  est identique en entrée et en sortie
2. Les cases rouges sont regroupées à gauche.
3. Les cases blanches sont regroupées au milieu.
4. Les cases bleues sont regroupées à droite.



**Définition 1.9.** — On dit qu'un algorithme *se termine* s'il produit un résultat en temps fini quelles que soient les données d'entrée fournies.

**Définition 1.10.** — On dit qu'un algorithme est *partiellement correct* par rapport à une spécification, si pour toute donnée d'entrée vérifiant les préconditions, si le calcul s'arrête, alors les postconditions sont satisfaites.

**Définition 1.11.** — On dit qu'un algorithme *est correct* s'il se termine et est partiellement correct.

Il peut paraître drôle de définir la notion de correction partielle qui semble vaine. Malheureusement, pour certains problèmes, il n'existe que des algorithmes partiellement corrects (cf chapitre 5).

**1.2.2. Preuve.** — Prouver un algorithme ou un programme revient à démontrer qu'il est correct (au moins partiellement) par rapport à sa spécification. Pour y parvenir, on conduit des démonstrations mathématiques qui établissent de proche en proche des postconditions à partir de préconditions.

Deux outils permettent de traiter le cas délicat de la justification du bon fonctionnement d'une boucle (instruction `for` ou `while`).

**Définition 1.12.** — On appelle *variant* toute quantité à valeur dans l'ensemble des entiers naturels (ou plus généralement à valeur dans un ensemble muni d'un ordre bien-fondé, cf. *infra* définition ??) qui décroît strictement à chaque itération de la boucle.

Un variant s'utilise afin de démontrer la terminaison d'un algorithme.

**Définition 1.13.** — On appelle *invariant* toute propriété  $P$  telle que si  $P$  est vérifié en entrée de boucle, alors elle est encore vérifiée en entrée de boucle à l'itération suivante.

Un invariant s'utilise afin de démontrer la correction partielle d'un algorithme.

**Remarque 1.14.** — Il est fréquent de retrouver dans des codes des *assertions*. Par exemple, en python, l'instruction `assert(test)` où `test` s'évalue en un booléen. Une assertion est évaluée au moment de l'exécution du programme et lève une exception si le test s'évalue à Faux.

Les assertions ont deux intérêts. En premier lieu, elles permettent de tester un programme au cours de son développement. Dans certains cas, on préférera désactiver les assertions dans le code mis en production. En second lieu, elles permettent

de documenter les hypothèses (préconditions ou invariants de boucle) faites par le programmeur, ce qui est utile pour maintenir le code à long terme.

**Exemple 1.15.** — Revisitons l'exemple 1.7 du calcul du pgcd, que l'on détermine classiquement avec l'algorithme d'Euclide.

LISTING 1.1. Algorithme d'Euclide

```

1  def gcd(a, b):
2      x, y = a, b
3      while y!=0:
4          temp = y
5          y = x % y
6          x = temp
7      return(x)

```

**Proposition 1.16.** — Le code 1.1 est correct par rapport à la spécification (cf. exemple 1.7).

*Démonstration.* — Pour prouver que l'algorithme se termine, nous pouvons utiliser la valeur  $y \in \mathbb{N}$  comme variant. Notons  $(x_0, y_0)$  les valeurs du couple  $(x, y)$  en entrée de boucle. La division euclidienne de  $x_0$  par  $y_0$  fournit un couple  $q, r$  spécifiée par les conditions :

$$x_0 = qy_0 + r \quad \text{où } 0 \leq r < y_0$$

En sortie de boucle, le couple  $(x, y)$  vaut  $(y_0, r)$ . La valeur  $y$  a bien diminué strictement.

Pour prouver que l'algorithme est correct, nous pouvons utiliser l'invariant suivant :  $\text{pgcd}(x, y) = \text{pgcd}(a, b)$  Cette propriété est vraie à l'initialisation. Écrivons à nouveau l'égalité  $x_0 = qy_0 + r$  où  $(x_0, y_0)$  est la valeur du couple  $(x, y)$  en entrée de boucle. Si un entier  $d$  divise  $x_0$  et  $y_0$ , alors, comme  $r = x_0 - qy_0$ , il divise aussi  $r$ . Donc les diviseurs commun à  $(x_0, y_0)$  sont des diviseurs communs de  $(y_0, r)$ . Inversement, si un entier  $d$  divise  $y_0$  et  $r$ , comme  $x_0 = qy_0 + r$ ,  $d$  est aussi un diviseur de  $x_0$ . Donc les diviseurs commun à  $(y_0, r)$  sont des diviseurs communs de  $(x_0, y_0)$ . En particulier, on a  $\text{pgcd}(y_0, r) = \text{pgcd}(x_0, y_0)$  Comme l'invariant est vérifié en entrée de boucle, on a encore  $\text{pgcd}(y_0, r) = \text{pgcd}(a, b)$  .  $\square$

**Exemple 1.17.** — Revisitons l'exemple 1.8 du drapeau hollandais que l'on peut résoudre par le code suivant

LISTING 1.2. Drapeau hollandais (Dijkstra)

```

8  def dutch_flag(T):
9      n = len(T)
10     r = 0
11     w = 0
12     b = n-1
13     while w <= b:

```

```

14         match T[w]:
15             case "Red":
16                 T[r], T[w] = T[w], T[r]
17                 r += 1
18                 w += 1
19             case "White":
20                 w += 1
21             case "Blue":
22                 T[w], T[b] = T[b], T[w]
23                 b -= 1
24
25     # Quelques essais :
26     from random import choice
27     T = [choice(["Red", "White", "Blue"]) for _ in range(10)]
28     print(T)
29     dutch_flag(T)
30     print(T)

```

**Proposition 1.18.** — *Le code 1.2 est correct par rapport à la spécification (cf. exemple 1.8).*

*Démonstration.* — Montrons que le programme se termine. On adopte comme variant la quantité  $v = b - w$ . Initialement, le variant  $v$  vaut  $n - 1$ . Lors d'un passage dans la boucle « tant que », trois cas peuvent se produire et soit  $w$  est incrémenté, soit  $b$  est décrémenté. Dans tous les cas,  $v$  diminue d'une unité. Ainsi  $v$  est bien un variant et le programme se termine.

Montrons que le programme est correct. Nous introduisons les invariants suivants :

- ( $\boxtimes_1$ ) Le tableau  $T$  contient une permutation du tableau initial.
- ( $\boxtimes_2$ ) En entrée de boucle, les indices  $r$ ,  $w$  et  $b$  satisfont  $r \leq w \leq b$ .
- ( $\boxtimes_3$ ) Les cases du tableau  $T[i]_{i \in [0, r[}$  sont rouges ; les cases du tableau  $T[i]_{i \in [r, w[}$  sont blanches et les cases du tableau  $T[i]_{i \in [b+1, n[}$  sont bleues.

Ces invariants sont satisfaits à l'entrée dans la boucle **while**. Montrons que les trois invariants sont conservés par la boucle « tant que ».

L'invariant ( $\boxtimes_1$ ) est conservé car d'une part, à l'entrée dans la boucle,  $T$  est une permutation du tableau initial et par ailleurs les seules opérations effectuées sur le tableau sont de la forme  $T[a], T[b] = T[b], T[a]$  qui permutent deux cases du tableau si  $a \neq b$  ou conservent le tableau si  $a = b$ .

Pour prouver la conservation de l'invariant ( $\boxtimes_2$ ), nous supposons que  $r \leq w \leq b$  et regardons l'évolution des indices les trois cas. Dans le premier cas, on passe à  $r + 1$ ,  $w + 1$  et  $b$ . Il est vrai que  $r + 1 \leq w + 1$ . Pour la seconde inégalité, il s'agit du test de poursuite de la boucle « tant que ». Dans le second cas, on obtient le triplet  $r$ ,  $w + 1$  et  $b$ . Il est vrai que  $r \leq w + 1$ . Pour la seconde inégalité, il s'agit du test de poursuite de la boucle « tant que ». Dans le dernier cas, on obtient le triplet  $r$ ,  $w$  et  $b - 1$ . Il est

vrai que  $r \leq w$ . Pour la seconde inégalité, il s'agit du test de poursuite de la boucle « tant que ».

Pour prouver la conservation de l'invariant  $(\mathfrak{I}_3)$ , nous raisonnons cas par cas. Le premier est le plus délicat. Soit  $r < w$ , mais alors, à cause de  $(\mathfrak{I}_3)$ , on a en entrée de boucle  $T[r] = \text{White}$ . Après échange, on a donc  $T[w] = \text{White}$ . Par ailleurs, on a en entrée de boucle  $T[w] = \text{Red}$ . Après échange, on a donc  $T[r] = \text{Red}$ . En incrémentant  $r$  et  $w$ , l'invariant  $(\mathfrak{I}_3)$  est bien conservé. Dans le second cas, l'incrément de  $w$  correspond au fait qu'on a détecté que  $T[w] = \text{White}$ . Dans le dernier cas, on a en entrée de boucle  $T[w] = \text{Blue}$ . Après échange, on a donc  $T[b] = \text{Blue}$ . En décrémentant  $b$ , l'invariant  $(\mathfrak{I}_3)$  est bien conservé.  $\square$

**Remarque 1.19.** — Les preuves fournissent une garantie incontestable sur le fond de l'algorithme mais n'éliminent pas les erreurs de programmation. Dans le contexte d'un cours d'algorithmique, elles peuvent paraître comme un gadget de mathématiciens. Elles sont néanmoins utilisées à grande échelle grâce à des outils formels. Il existe des cadres de programmation dans lesquels le programmeur annote son code avec des préconditions, postconditions, variants ou invariants; la machine se charge alors de trouver ou de contrôler des éléments de preuve qui relient les annotations.

**Exemple 1.20.** — Pour aller plus loin (et hors du programme de ce cours), on pourra consulter les liens suivants : <https://toccata.gitlabpages.inria.fr/toccata/gallery/gcd.en.html> ou <https://toccata.gitlabpages.inria.fr/toccata/gallery/flag.en.html> qui formalisent les preuves du calcul du pgcd ou du drapeau hollandais à l'aide de la plateforme Why3. On reconnaîtra les mots clés **variant** et **invariant**. Les préconditions sont introduites par le mot clé **requires**; les postconditions sont introduites par le mot clé **ensures**.

**1.2.3. Test.** — On n'insistera jamais suffisamment sur la nécessité de toujours tester un programme avant de s'en satisfaire et on gardera à l'esprit l'adage célèbre « Program testing can be used to show the presence of bugs, but never to show their absence! » que l'on doit à Dijkstra (EWD249).

Il existe des méthodologies assez avancées de conception de test, qui cherchent à couvrir différents types de dysfonctionnement possibles et qui dépassent le cadre de ce cours. En marge de ce cours, nous pourrions surtout être intéressé par des *tests unitaires*, c'est-à-dire des tests qui contrôlent le bon fonctionnement de petites composantes ou petites parties d'un programme.

La méthode de développement *Test-Driven Development* (ou développement piloté par les tests en langue française) est une méthodologie de développement logiciel qui pousse le recours aux tests à son paroxysme, en imposant de d'abord écrire les tests que le code doit satisfaire avant d'écrire le programme lui-même.

Quelques principes de base s'imposent pour composer des tests :

- Attention à l'excès de confiance. Il vaut mieux être paranoïaque et être persuadé que son code contient des erreurs. Variante de ce conseil : il faut tout tester avec un *a priori* de méfiance, y compris la partie du code qui a l'air facile!

- Un jeu de test comprend toujours à la fois des données d'entrée et la description des valeurs de retour ou des effets attendus, qui ne dépendent que de la spécification. Une erreur méthodologique serait d'exécuter son code sur un exemple, d'observer le résultat et de juger que tout va bien postérieurement. Il est important de séparer la conception de tests et l'analyse des résultats après exécution des tests.
- Les tests méritent d'être écrits en vue d'être conservés et d'être documentés. Lorsqu'on écrit une fonction de quelques lignes seulement, il est tentant de vérifier son code directement en l'exécutant dans un terminal. Il vaut mieux préparer un script chargé de tous les tests déjà écrits, conservé de manière permanente et enrichi au fur et à mesure du développement du code. Ainsi, à chaque modification légère de son code, les tests seront déjà prêts et non pas besoin d'être réinventés.
- Dans un monde idéal, les tests et le code source sont écrits par deux personnes différentes.

Voici quelques conseils pour repérer les bugs les plus courants :

- Tester les *cas de base* d'un programme ou d'une structure récursifs. Dans un programme récursif, tester le cas initial ; si un programme utilise une liste comme paramètre d'entrée, tester la liste vide, la liste à un élément, etc.
- Tester les *cas limites*. Ce conseil s'applique particulièrement aux programmes qui manipulent des indices : vérifier les cas d'égalité dans des inégalités.
- Tester l'*ensemble des branches* du flot de contrôle.

**Exemple 1.21.** — Pour tester le code 1.2 qui résout le problème du drapeau hollandais, on peut prévoir les cas suivants : tableau vide, tableau à un élément (3 cas), tableau de longueur quelconque ayant seulement deux des trois couleurs, tableau quelconque, etc

**Remarque 1.22.** — Les tests valident une implémentation plutôt qu'un algorithme. Ils permettent de repérer et d'éliminer des rapidement des bugs faciles à voir. Ils peuvent être utilisés dès la phase de développement, voire même précéder cette phase. Souvent, écrire des tests permet aussi de se mettre dans la peau du futur utilisateur : cela permet de faire apparaître les limites du programme que l'on cherche à écrire ou de mieux réfléchir à sa conception.

### 1.3. Ressources, performance et complexité

**Définitions 1.23.** — Soient  $\mathbf{u} = (u_n)_{n \in \mathbb{N}} \subseteq \mathbb{R}^{\mathbb{N}}$  et  $\mathbf{v} = (v_n)_{n \in \mathbb{N}} \subseteq \mathbb{R}^{\mathbb{N}}$  deux suites réelles. On définit les notations

- $\mathbf{u} = O(\mathbf{v})$  s'il existe une constante  $A > 0$  et un rang  $n_0 \in \mathbb{N}$  tel que pour tout indice  $n \geq n_0$ ,  $|u_n| \leq A|v_n|$ .
- $\mathbf{u} = \Omega(\mathbf{v})$  s'il existe une constante  $A' > 0$  et un rang  $n_0 \in \mathbb{N}$  tel que pour tout indice  $n \geq n_0$ ,  $|v_n| \leq A'|u_n|$ .



- $\mathbf{u} = \Theta(\mathbf{v})$  s'il existe deux constantes  $A > 0$ ,  $A' > 0$  et un rang  $n_0 \in \mathbb{N}$  tel que pour tout indice  $n \geq n_0$ ,  $|u_n| \leq A|v_n|$  et  $|v_n| \leq A'|u_n|$ .

Les principales ressources consommées par un algorithme sont les suivantes :

1. le *temps d'exécution* nécessaire pour faire aboutir un calcul,
2. l'*espace occupé* en mémoire pour réaliser un calcul,
3. l'énergie consommée par un calcul,
4. la quantité de données échangées dans le cas où le calcul se fait sur des unités distribuées.

Historiquement, l'algorithmique s'intéresse énormément aux deux premières ressources. La question du temps est un enjeu systématique. La question de l'espace a pu dépendre des époques et des systèmes utilisés : historiquement, la mémoire coûtait très cher ; encore aujourd'hui, résoudre des problèmes algorithmiques sur des petits systèmes embarqués ou traiter des données massives demande de surveiller la consommation en espace. La question de l'énergie est désormais regardée avec plus d'attention : indépendamment de toute considération éthique sur le gaspillage, programmer avec sobriété permet aussi d'augmenter l'autonomie de la station de calcul (utile par exemple sur un téléphone) ou simplement de faire diminuer les coûts économiques (dans le cas de *datacenters*). Ces affirmations sont toutefois à nuancer, car souvent, les plus grandes économies proviennent du fait d'éteindre les appareils et de les allumer à la demande plutôt que de les laisser en veille.

Mesurer la *complexité d'un algorithme* revient à étudier comment la consommation évolue quand la taille de l'instance augmente.

Pour mesurer la taille de l'instance  $I$ , de façon générique, nous comptons simplement le nombre  $|I|$  de bits nécessaires à les représenter. Selon les données manipulées, des paramètres plus spécifiques aux données sont employés, du moment qu'ils sont  $\Theta(|I|)$ .

**Exemple 1.24.** — Dans l'algorithme d'Euclide (voir code 1.1), les paramètres d'entrée sont deux entiers  $a$  et  $b$ , que l'on représente par leur décomposition binaire avec  $\lceil \log_2 a \rceil + 1$  et  $\lceil \log_2 b \rceil + 1$  bits respectivement.

**Exemple 1.25.** — Dans le problème du drapeau hollandais (voir code 1.2), le paramètre d'entrée est un tableau  $T$ , qui contient des chaînes de caractères pouvant prendre trois valeurs. On pourrait bien entendu compter précisément le nombre de bits occupés, mais, en ordre de grandeur, il suffit de connaître la longueur du tableau pour se faire une idée de la place qu'il occupe.

**Exemple 1.26.** — Dans un problème portant sur un graphe, on exprime en général la taille du graphe en fonction du nombre de sommets et du nombre d'arêtes.

**1.3.1. Complexité en temps.** — Dans un premier temps, pour déterminer la complexité en temps, nous adoptons un modèle basique de décompte des opérations élémentaires. Nous appelons *opération élémentaire* toutes les opérations que nous considérons faites en temps constant, à savoir notamment :

- l'accès en mémoire pour lecture ou écriture d'une variable simple ou d'une case indicée d'un tableau
- les opérations arithmétiques (somme, soustraction, multiplication, division, reste dans une division) ou booléennes sur des mots machine : entiers, flottant, booléens.
- les comparaisons des contenus de deux variables simples.

Des opérations dérivées d'une combinaison d'un petit nombre borné d'opération énumérées ci dessus peuvent aussi être considérée comme élémentaires. Par exemple, échanger le contenu de deux variables ou de deux cases d'un tableau.

Dans la plupart des analyses d'algorithmes, il est en général considéré que les entiers naturels sont en réalité des entiers bornés : pour cette raison, on considère plutôt que les opérations arithmétiques s'effectuent en temps constant. Ce point de vue est historiquement cohérent avec les implémentations, puisque les premiers langages de programmation identifiaient des entiers à des mots machine. Dans des langages modernes, tels que `Python3`, les entiers peuvent prendre des tailles quelconques. Si un algorithme est amené à manipuler des entiers de taille quelconque, il faut raisonner avec prudence et se poser la question du mode de représentation.

Pour tout algorithme  $\mathcal{A}$ , notons  $\text{coût}(\mathcal{A}, I)$  le nombre d'opérations élémentaires exécutée par l'algorithme  $\mathcal{A}$  sur l'instance  $I$ .

**Définitions 1.27.** — Soit  $\mathcal{A}$  un algorithme. On appelle *complexité en temps dans le pire des cas* tout ordre de grandeur du nombre d'opérations élémentaires en fonction de la taille de l'instance à traiter, c'est-à-dire tout ordre de grandeur de

$$c_{\mathcal{A}}(n) = \sup\{\text{coût}(\mathcal{A}, I); |I| = n\}.$$

en fonction de  $n$ .

Si une loi de probabilité a été définie sur l'ensemble des instances valides de  $\mathcal{A}$ , on appelle *complexité en temps dans le cas moyen* tout ordre de grandeur de l'espérance

$$m_{\mathcal{A}}(n) = \mathbb{E}(\text{coût}(\mathcal{A}, I); |I| = n).$$

en fonction de  $n$ .

On appelle *complexité en temps dans le meilleur des cas* tout ordre de grandeur du nombre d'opérations élémentaires en fonction de la taille de l'instance à traiter, c'est-à-dire tout ordre de grandeur de

$$c_{\mathcal{A}}(n) = \inf\{\text{coût}(\mathcal{A}, I); |I| = n\}.$$

en fonction de  $n$ .

Quand la complexité d'un algorithme est en  $\Theta(f(n))$ , on dit que l'algorithme est en temps

- *constant* si  $f(n) = O(1)$ ,
- *logarithmique* si  $f(n) = O(\log_2 n)$ ,
- *linéaire* si  $f(n) = O(n)$ ,
- *linéarithmique* si  $f(n) = O(n \log_2 n)$ ,
- *quadratique* si  $f(n) = O(n^2)$ ,

— *polynomial* si  $f(n) = O(n^k)$  pour un certain entier  $k$ .

**Exemple 1.28.** — Analysons dans le détail le code 1.1.

Chacune des lignes sauf celle qui introduit le « tant que » correspond à une ou deux opérations élémentaires. Le test du « tant que » se fait aussi en une opération élémentaire. Il reste à compter le nombre d'itérations dans la boucle « tant que ». Or nous pouvons observer le fait suivant

Si  $a \geq b$  et si le calcul du pgcd entre  $a$  et  $b$  vaut  $d$  et passe par  $k$  étapes de la boucle « tant que », alors on a

$$a \geq dF_{k+1} \text{ et } b \geq dF_k$$

où  $F_n$  est la suite de Fibonacci

où  $F_0 = 0$ ,  $F_1 = 1$  et pour tout entier  $k$ ,  $F_{k+2} = F_{k+1} + F_k$ . Ce fait est vérifié pour  $k = 0$  et se montre facilement par récurrence. Mais, un calcul montre que  $F_k = \Omega(\phi^k)$ , où  $\phi = \frac{1+\sqrt{5}}{2}$ , ce qui implique que

$$\log_2 a, \log_2 b = \Omega(\log n)$$

Nous en déduisons que la complexité du code 1.1 est  $O(\log_2 a, \log_2 b)$ , autrement dit, l'algorithme d'Euclide est un algorithme qui fonctionne en temps linéaire.

**Exemple 1.29.** — Analysons dans le détail le code 1.2 relatif au drapeau hollandais.

Chacune des lignes sauf celle qui introduit le « tant que » s'exécutent en temps constant. Pour l'analyse de la terminaison du code, nous avons annoté « tant que » avec un variant  $v = b - w$  qui décroît d'une unité à chaque étape. Il s'ensuit que la boucle « tant que » est répétée  $O(n)$  fois (où  $n$  est la longueur du tableau).

Nous en déduisons que la complexité du code 1.2 est  $O(n)$ , autrement dit, l'algorithme du drapeau hollandais fonctionne en temps linéaire.

Voici quelques cas typiques de complexité.

|                |                                                |
|----------------|------------------------------------------------|
| Logarithmique  | Recherche par dichotomie                       |
| Linéaire       | Boucle simple, recherche séquentielle          |
| Linéarithmique | Diviser pour régner, tris efficaces            |
| Quadratique    | Double boucles imbriquées, tableaux 2D         |
| Cubique        | Triple boucles imbriquées, produit de matrice. |
| Exponentielle  | Recherche exhaustive sur $n$ choix.            |
| Factorielle    | Recherche par énumération de permutations.     |

**Remarque 1.30.** — La complexité asymptotique ne dit rien du temps de calcul réel. On peut néanmoins prendre comme repère le ratio suivant : sur une machine actuelle, on effectue de l'ordre de  $10^9$  opérations par seconde. Cela conduit au tableau

suivant :

|                      | 10        | 100        | 1000       | $10^4$      | $10^6$    | $10^9$     |
|----------------------|-----------|------------|------------|-------------|-----------|------------|
| $\Theta(\log_2 n)$   | 3ns       | 7ns        | 10ns       | 3ns         | 3ns       | 3ns        |
| $\Theta(\sqrt{n})$   | 3ns       | 10ns       | 31ns       | 100ns       | 1 $\mu$ s | 31 $\mu$ s |
| $\Theta(n)$          | 10ns      | 100ns      | 1 $\mu$ s  | 10 $\mu$ s  | 1ms       | 1s         |
| $\Theta(n \log_2 n)$ | 33ns      | 664ns      | 10 $\mu$ s | 133 $\mu$ s | 20ms      | 30s        |
| $\Theta(n^2)$        | 10ns      | 10 $\mu$ s | 1ms        | 100ms       | 17min     | 32ans      |
| $\Theta(n^3)$        | 1 $\mu$ s | 1ms        | 1s         | 17min       | 32ans     | $\infty$   |
| $\Theta(2^n)$        | 1 $\mu$ s | $\infty$   | $\infty$   | $\infty$    | $\infty$  | $\infty$   |
| $\Theta(n!)$         | 4ms       | $\infty$   | $\infty$   | $\infty$    | $\infty$  | $\infty$   |

**Remarque 1.31.** — Il faut aussi garder à l'esprit que la notion de complexité en temps ne répond pas à toutes les problématiques qui peuvent se poser. Une branche de l'informatique s'intéresse aux *systèmes temps réel*, c'est-à-dire aux systèmes chargés de contrôler ou de piloter des procédés physiques avec des garanties de vitesse d'exécution.

**1.3.2. Complexité en espace.** — Les définitions sont analogues à celles concernant le temps.

**Définition 1.32.** — Soit  $\mathcal{A}$  un algorithme. On appelle *complexité en espace dans le pire des cas* tout ordre de grandeur de l'espace mémoire nécessaire au fonctionnement de l'algorithme en fonction de la taille de l'instance à traiter.

On peut aussi définir les notions de *complexité en espace en moyenne* ou dans le *meilleur des cas*.

**Remarque 1.33.** — En général, lorsqu'on parle de complexité en espace, il convient d'indiquer précisément le cadre dans lequel on se place en précisant si l'on compte seulement la mémoire supplémentaire par rapport aux données d'entrée ou si l'on inclut aussi l'espace utilisé pour les entrées et les sorties.

**Exemple 1.34.** — Dans le problème du tri par ordre croissant des éléments d'un tableau, on parle de *tri en place* lorsque le tri ne recopie pas les données mais se contente de les réorganiser en utilisant une quantité constante de mémoire supplémentaire par rapport aux données d'entrée.

## 1.4. Stratégies algorithmiques et schémas classiques

**1.4.1. Diviser les problèmes en sous tâches.** — Une règle de base lorsque l'on s'attaque à un problème algorithmique est de commencer par identifier toutes les sous-tâches qui concourent à la résolution de son problème et qui sont plus simples à réaliser individuellement. D'aucuns considèrent qu'il s'agit de l'essence même de la *pensée computationnelle*.

Il est souvent tentant d'écrire un code tout-en-un qui se charge de tout gérer à la fois. C'est une erreur courante. Un code long est difficilement lisible, difficilement

testable ou difficilement vérifiable. Le temps que l'on croit gagner en écrivant rapidement un petit code compact est vite perdu quand il s'agit de le déboguer ou de maintenir le code.

**Exemple 1.35.** — Résoudre un système d'équation par la méthode du pivot de Gauss est un algorithme que l'on apprend tout petit. Lorsqu'on le programme, il faut résister à l'envie de le coder d'une seule traite et de vouloir décrire toutes les actions les unes après les autres comme le ferait un mathématicien sur sa feuille de papier. À minima, on pourra avoir besoin d'écrire des fonctions spécifiques pour

- chacune des opérations élémentaires sur les lignes,
- chacune des opérations élémentaires sur les colonnes,
- placer un coefficient non nul en position de pivot
- nettoyer les lignes situées en dessous du pivot
- etc.

#### 1.4.2. Problèmes d'exploration arborescente. —

**1.4.2.1. Le retour sur trace.** — La technique du *retour sur trace* (ou *backtracking*) est une technique d'exploration qui consiste à chercher une solution réalisable d'un problème incrémentalement, en s'interrompant dès que l'on peut établir avec certitude que la solution partielle en cours de construction ne pourra pas être complétée. Dès lors, on rebrousse chemin pour changer l'une des décisions prises précédemment.

De façon abstraite, résoudre un problème par retour sur trace revient à considérer un problème sous la forme d'un triplet  $(D, c, v)$  où

- $D$  est un ensemble fini : l'ensemble des décisions à prendre
- $c : D \rightarrow \mathbb{N}$  est une application qui associe toute décision à prendre  $d$  à un nombre de choix  $c(d)$ .
- $v : (D \rightarrow \mathbb{N}) \rightarrow \{\text{Vrai}, \text{Faux}\}$  est un vérificateur qui, pour toute fonction partielle  $s : D \rightarrow \mathbb{N}$  renvoie Faux s'il n'est pas possible d'étendre  $s$  en une solution réalisable du problème et renvoie Vrai sinon.

Rechercher une solution se fait alors comme suit

LISTING 1.3. Retour sur trace

```

31 def solve(s):
32     for d in decisions:
33         if s(d) == 0:
34             for c in range(1, choix(d)+1):
35                 s[d] = c
36                 if check(s) and solve(s):
37                     return True
38                 s[d]=0
39             return False
40     return True

```

**Exemple 1.36 (Sudoku).** — Le *sudoku* est l'exemple iconique de la recherche par retour sur trace dont voici un exemple.

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
|   | 8 |   |   |   |   | 3 |   |
| 2 |   |   | 6 | 7 |   |   | 1 |
|   |   |   |   |   |   |   |   |
|   | 6 |   | 2 | 1 |   | 7 |   |
| 5 |   |   |   |   |   |   | 3 |
| 9 |   |   | 7 | 5 |   |   | 8 |
| 4 |   | 1 | 3 | 9 | 7 |   | 6 |
|   | 2 |   |   |   |   | 1 |   |
| 8 |   | 3 | 1 | 6 | 5 |   | 9 |

L'ensemble des décisions à prendre porte sur les 81 cases du sudoku, que nous numérotions de 0 à 80 :

$$D = \llbracket 0, 80 \rrbracket.$$

et qui coïncident aux cases

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 |
| 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 |
| 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 |
| 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 |
| 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 |
| 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 |

On se donne les fonctions

```

41 def decision(r,c):
42     return 9*r+c
43
44 def row(c):
45     return c//9
46
47 def col(c):
48     return c%9
49
50 def block(c):
51     return decision(3*(row(c)//3), 3*(col(c)//3))

```

Enfin, la contrainte à vérifier peut se coder ainsi

```

52 def check(G, p):
53     r = row(p)
54     c = col(p)
55     # Verification des lignes
56     for cc in range(9):
57         if cc != c and G[p]==G[decision(r,cc)]:

```

```

58         return False
59     # Verification des colonnes
60     for rr in range(9):
61         if rr != r and G[p]==G[decision(rr,c)]:
62             return False
63     # Verification du bloc
64     for z in [0, 1, 2, 9, 10, 11, 18,19, 20]:
65         pp = block(p)+z
66         if pp != p and G[p]==G[pp]:
67             return False
68     return True

```

On obtient finalement le code suivant :

```

69 def solve():
70     for d in decisions:
71         if s[d] == 0:
72             for c in choix(d):
73                 s[d] = c
74                 if (check(s, d) and solve()):
75                     return True
76             s[d]=0
77         return False
78     return True

```

Les données de l'exemple

```

79 s = [
80 0, 8, 0, 0, 0, 0, 0, 3, 0,
81 2, 0, 0, 6, 0, 7, 0, 0, 1,
82 0, 0, 0, 0, 0, 0, 0, 0, 0,
83 0, 6, 0, 2, 0, 1, 0, 7, 0,
84 5, 0, 0, 0, 0, 0, 0, 0, 3,
85 9, 0, 0, 7, 0, 5, 0, 0, 8,
86 4, 0, 1, 3, 0, 9, 7, 0, 6,
87 0, 2, 0, 0, 0, 0, 0, 1, 0,
88 8, 0, 3, 1, 0, 6, 5, 0, 9
89 ]
90
91 solve()

```

conduisent au sudoku final

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 8 | 4 | 9 | 5 | 2 | 6 | 3 | 7 |
| 2 | 3 | 5 | 6 | 8 | 7 | 9 | 4 | 1 |
| 7 | 9 | 6 | 4 | 1 | 3 | 8 | 5 | 2 |
| 3 | 6 | 8 | 2 | 9 | 1 | 4 | 7 | 5 |
| 5 | 1 | 7 | 8 | 6 | 4 | 2 | 9 | 3 |
| 9 | 4 | 2 | 7 | 3 | 5 | 1 | 6 | 8 |
| 4 | 5 | 1 | 3 | 2 | 9 | 7 | 8 | 6 |
| 6 | 2 | 9 | 5 | 7 | 8 | 3 | 1 | 4 |
| 8 | 7 | 3 | 1 | 4 | 6 | 5 | 2 | 9 |

**1.4.2.2.** *L'algorithme min-max.* — Non traité

**1.4.3. Sous structures optimales.** — De nombreux schémas exploitent le fait qu'il est possible de résoudre une instance d'un problème en se ramenant à une instance plus petite.

**1.4.3.1.** *Les algorithmes gloutons.* — La notion d'algorithme glouton est une notion vague que l'on emploie pour décrire le comportement d'un algorithme.

**Définition 1.37.** — On qualifie de *gloutonne* toute stratégie algorithmique qui consiste à rechercher un optimum global par une suite de décisions qui, à chaque étape, réalisent un optimum local.

Il n'y a pas de sens à parler de « l' » algorithme glouton pour résoudre un problème puisque chacun peut avoir sa propre idée sur une manière de concevoir un algorithme selon des principes gloutons. Cependant, sur certains problèmes classiques, il est canonique de nommer certains algorithmes ainsi.

**Remarque 1.38.** — En général, un algorithme ne calcule pas un optimum global. Il est toujours nécessaire de valider l'approche gloutonne par une démonstration mathématique.

**Exemple 1.39 (Rendu de monnaie).** — Dans ce problème, on souhaite minimiser le nombre de pièces de monnaie nécessaire pour atteindre une certaine somme.

Entrées : Entier  $m \in \mathbb{N}$  (le montant à rendre), ensemble fini  $\{s_1, \dots, s_k\}$  avec  $s_1 > s_2 > \dots > s_k > 0$  (les pièces possibles).

Sorties : Décomposition  $(a_1, \dots, a_k) \in \mathbb{N}^k$  où

$$m = a_1 s_1 + \dots + a_k s_k$$

qui minimise la quantité  $\rho = a_1 + \dots + a_k$ .

Principe glouton pour un algorithme : Pour rendre la monnaie sur un montant total  $m$ , on peut chercher la plus grosse pièce de monnaie inférieure au montant  $m$  puis répéter le processus sur le montant restant. Voici un code qui met en œuvre cette idée.

LISTING 1.4. Rendu de monnaie



```

92 def rendu(m: int, S: list[int]) -> list[int]:
93     ans = [0] * len(S)
94     for i,s in enumerate(S):
95         while m>=s:
96             m -= s
97             ans[i] += 1
98     if m!= 0:
99         raise Exception("Desolé, rendu impossible !")
100     return ans

```

Il est important de noter que cet algorithme n'est pas garanti de fonctionner. Par exemple, pour rendre la somme de 6 avec des pièces de valeur faciale 4 et 3, on peut effectuer  $6 = 3 + 3$ . Le code 1.4 ne trouve aucune solution.

Par ailleurs, en toute généralité, cet algorithme ne fournit pas forcément de solution optimale. Dans le cas où  $m = 12$  et  $S = (5, 4, 1)$ , il est facile de voir que la décomposition  $12 = 4+4+4$  est optimale. Le code 1.4 propose plutôt la décomposition  $12 = 5+5+1+1$  avec une pièce de plus.

**Exemple 1.40 (Sélection d'activité).** — Dans ce problème, on dispose d'un ensemble d'activités  $A$  ayant chacune un temps de début et un temps de fin. On cherche à sélectionner un sous-ensemble de cardinal maximum d'activités qui ne se chevauchent pas

Entrées : Ensemble fini de couples  $A = (a_i)_{1 \leq i \leq k} \subseteq \mathbb{Z} \times \mathbb{Z}$  où, pour tout  $i$  compris entre 1 et  $k$ , on a  $a_i = (d_i, f_i)$  avec  $d_i < f_i$ .

Sorties : Sous-ensemble d'activité  $A' = (a_i)_{i \in I}$  avec  $I \subseteq \llbracket 1, k \rrbracket$  de cardinal  $|I|$  maximum où, pour tous indices  $i$  et  $j$  distincts de  $I$ , les intervalles  $[d_i, f_i[$  et  $[d_j, f_j[$  sont disjoints :

$$[d_i, f_i[ \cup [d_j, f_j[ = \emptyset.$$

LISTING 1.5. Sélection d'activité

```

101 def selection(A: list[(int, int)]) -> list[(int,int)]:
102     # Trions les activités par temps final croissant
103     A.sort(key = lambda a : a[1])
104     Output = []
105     last_f = -sys.maxsize
106     for (d,f) in A:
107         if d >= last_f:
108             Output.append((d,f))
109             last_f = f
110     return Output

```

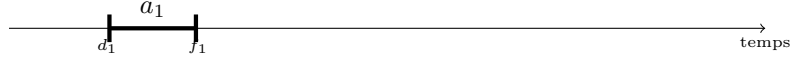
**Proposition 1.41.** — L'algorithme 1.5 est correct par rapport à sa spécification.

*Démonstration.* — Nous démontrons la correction de **selection** par récurrence sur le cardinal du paramètre d'entrée  $A$ . Si  $|A| \leq 1$ , la justification est claire.

Soit  $n \geq 2$  un entier. Supposons que l'algorithme soit correct pour toute entrée  $\hat{A}$  avec  $|\hat{A}| < n$  et donnons nous une liste d'entrée  $A$  de cardinal  $n$ . Soit  $O$  un sous-ensemble d'activité de  $A$  optimal et  $A'$  le sous-ensemble calculé par l'algorithme 1.5. Notons  $o_1$  l'activité se finissant en premier dans  $O$



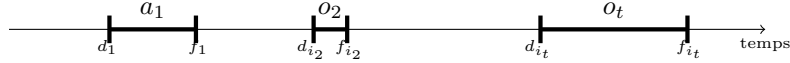
et notons  $a'_1$  l'activité se finissant en premier dans  $A'$ .



Comme  $a'$  est l'activité qui se termine en premier parmi toutes les activités de  $A$ , le sous-ensemble

$$O' = O \setminus \{o_1\} \cup \{a'_1\}$$

est encore admissible et est aussi optimal.



Appelons  $\hat{A}$  l'ensemble des activités qui démarrent après la fin de  $a'_1$ . Nous observons que  $O'' = O' \setminus \{a'_1\}$  est un choix d'activité optimal pour  $\hat{A}$ . Nous observons aussi que  $A'' = A' \setminus \{a'_1\}$  est exactement la sortie l'exécution de **selection** sur l'entrée  $\hat{A}$ . Mais alors, par hypothèse de récurrence  $O''$  et  $A''$  ont même cardinal, ce qui prouve que  $A'$  est de cardinal optimal.  $\square$

**1.4.3.2. Diviser pour régner.** — La stratégie algorithmique *diviser pour régner* consiste à résoudre un problème selon les trois étapes

1. décomposer une instance d'un problème en deux sous-instances du même problème ou plus,
2. résoudre récursivement ces sous-instances
3. combiner les solutions de ces sous-instances pour construire la solution de l'instance initiale.

**Exemple 1.42.** — L'algorithme d'*exponentiation rapide* permet de calculer  $a^n$  dans un monoïde à partir de l'élément  $a$  et de l'entier  $n$ . Il s'appuie sur l'observation suivante : pour tout élément  $a$

$$a^n = \begin{cases} (a^{n'}) \cdot (a^{n'}) & \text{si } n = 2n' \quad (\text{Cas pair}), \\ a \cdot (a^{n'}) \cdot (a^{n'}) & \text{si } n = 2n' + 1 \quad (\text{Cas impair}). \end{cases}$$

En voici une implémentation, pour le produit usuel,

LISTING 1.6. Exponentiation rapide

```

111 def square_and_multiply(a, n):
112     if n == 0:
113         return 1
114     if n==1:
115         return a
116     b = square_and_multiply(a, n//2)
117     if n%2 == 0:
118         return b * b
119     else:
120         return a * b * b

```

**Proposition 1.43.** — En supposant que le produit se calcule toujours en temps constant, la complexité du code 1.6 est  $O(\log_2 n)$ .

*Démonstration.* — En dehors de l'appel récursif, il n'y a qu'une suite finie d'opérations réalisées par `square_and_multiply` qui sont toutes des opérations élémentaires. Ainsi, un appel coûte un temps constant. Soit  $k$  un entier tel que  $n \leq 2^k$ . La fonction est appelée avec une valeur de  $n$  inférieure à  $2^{k-1}$ , puis  $2^{k-2}$ , etc. Elle est donc appelée  $k$  fois au total. Or  $k = \Theta(\log_2 n)$ . La complexité du code 1.6 est donc  $O(\log_2 n)$ .  $\square$

**Remarque 1.44.** — Il est possible de rater complètement l'implémentation de l'algorithme d'exponentiation rapide comme dans l'exemple suivant

LISTING 1.7. Mauvaise implémentation de l'exponentiation rapide

```

121 def square_and_multiply(a, n):
122     if n == 0:
123         return 1
124     if n==1:
125         return a
126     if n%2 == 0:
127         return square_and_multiply(a, n//2) * square_and_multiply(a,
            ↪ n//2)
128     else:
129         return a * square_and_multiply(a, n//2) *
            ↪ square_and_multiply(a, n//2)

```

Sans mémoïsation des résultats, l'appel récursif est joué deux fois, ce qui conduit à une complexité linéaire.

**Exemple 1.45.** — Le *tri fusion* permet de trier un tableau.

Rappelons que trier un tableau consiste concevoir un algorithme ainsi spécifié. Un tableau  $T$  de longueur  $n$  est fourni en entrée. On suppose qu'il existe un ordre total

$\leq$  sur les éléments de  $T$ . En sortie, on demande que le tableau résultant soit une permutation du tableau initial et que, pour tout indice  $i \in \llbracket 0, n-1 \rrbracket$ ,  $T[i] \leq T[i+1]$ .

Le *tri fusion* consiste à trier la moitié gauche puis la moitié droite en appliquant l'algorithme de tri récursivement, puis à fusionner les deux moitiés triées en parcourant chacun des tableaux et en sélectionnant le plus petit élément pour le recopier à sa place définitive.

LISTING 1.8. Tri fusion

```

130 def merge_sort(T):
131     if len(T) >= 2:
132         m = len(T)//2
133         left = T[:m]
134         merge_sort(left)
135         right = T[m:]
136         merge_sort(right)
137         i = j = k = 0
138         while i < len(left) and j < len(right):
139             if left[i] <= right[j]:
140                 T[k] = left[i]
141                 k, i = k+1, i+1
142             else:
143                 T[k] = right[j]
144                 k, j = k+1, j+1
145         while i < len(left):
146             T[k] = left[i]
147             k, i = k+1, i+1
148         while j < len(right):
149             T[k] = right[j]
150             k, j = k+1, j+1

```

Le tri fusion n'est pas un algorithme de tri *en place* (voir exemple 1.34). Il consomme un espace mémoire proportionnel à la taille du tableau à trier.

**Définition 1.46.** — Un algorithme de tri est un *tri comparatif* si l'exécution de l'algorithme ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Le tri fusion est un exemple de tri comparatif.

**Théorème 1.47.** — *Tout algorithme de tri comparatif effectue  $\Omega(n \log_2 n)$  comparaisons dans le pire des cas.*

*Démonstration.* — Selon l'état du tableau d'entrée, un algorithme de tri comparatif est amené à opérer  $n!$  permutations distinctes de l'entrée pour former la sortie. Or, si  $d$  comparaisons ont été effectuées, il y a au plus  $2^d$  comportements de l'algorithme possible. Ainsi tout algorithme correct satisfait  $2^d \geq n!$ .

Mais d'après la formule de Stirling,  $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ , on a

$$d \geq \log_2(n!) = \Omega(n \log_2 n)$$

ce qui prouve le résultat attendu.  $\square$

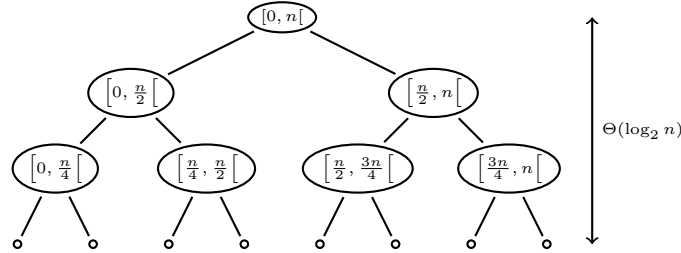
Le théorème 1.47 montre ainsi que le tri fusion admet une complexité  $\Omega(n \log_2 n)$ .

**Proposition 1.48.** — *La complexité du code 1.11 est  $O(n \log_2 n)$ .*

*Démonstration.* — Soit  $T$  un tableau à trier de longueur  $n$  et soit  $k$  un entier tel que  $2^{k-1} < n \leq 2^k$ . Notons déjà que  $k = \Theta(\log_2 n)$ .

En dehors des appels récursifs, toutes les opérations effectuées par un appel de la fonction `merge_sort` sont des opérations élémentaires. Dans chacune des boucles `while`, soit  $i$ , soit  $j$  est incrémenté d'une unité. De plus, nous passons de  $i + j = 0$  à  $i + j = n$ , ce qui montre qu'il y a  $n$  passages au total dans l'un des quatre corps de boucle. Ainsi, en dehors des appels récursifs, l'appel de la fonction `merge_sort` coûte  $\Theta(n)$  opérations.

Par ailleurs, un appel de `merge_sort` sur un tableau de taille  $\leq 2^k$  engendre deux appels sur des tableaux de taille  $\leq 2^{k-1}$  si  $k \geq 0$ . L'arbre des appels est donc le suivant :



La complexité totale est donc proportionnelle à

$$2^k + \underbrace{\frac{2^k}{2} + \frac{2^k}{2}}_{2 \text{ fois}} + \underbrace{\frac{2^k}{4} + \dots + \frac{2^k}{4}}_{4 \text{ fois}} + \dots + \underbrace{\frac{2^k}{2^k} + \frac{2^k}{2^k}}_{2^k \text{ fois}}$$

soit encore

$$k2^k = \Theta(n \log_2 n).$$

$\square$

**1.4.3.3. La programmation dynamique.** — La programmation dynamique est à la fois une technique d'optimisation mathématique et stratégie algorithmique de résolution de problèmes. En informatique, on rencontre cette technique lorsque, dans un problème d'optimisation, on peut exprimer une dépendance entre la solution optimale d'une instance et les solutions optimales d'autres instances plus petites. On parle alors de *principe d'optimalité de Bellman*.

Deux caractéristiques sont nécessaires pour qualifier une mise en œuvre algorithmique de *programmation dynamique* :

- Le problème à résoudre doit présenter la propriété de *sous-structure optimale* : à savoir la solution optimale d’une instance peut se déduire des solutions optimales d’instances plus petites.
- Les sous problèmes (ou plutôt les instances plus petites à considérer pour résoudre une instance donnée) doivent *se chevaucher*.

**Remarque 1.49.** — Lorsque la propriété de sous-structure optimale relie une instance avec une seule sous-instance (et non pas plusieurs), on parle plutôt d’*algorithme glouton* dont on a démontré l’optimalité. Certains qualifie parfois ce type d’approche de *diminuer pour régner* (ou *decrease and conquer*).

Lorsque les sous-problèmes ne se chevauchent pas, on parle plutôt de stratégie *diviser pour régner*.

La mise en œuvre d’une stratégie de programmation dynamique se fait essentiellement de deux manières : par une approche descendante ou par une approche ascendante.

- L’approche descendante est l’approche la plus naturelle une fois que l’on a écrit une équation mathématique reliant un optimum à des optima de sous-problèmes. Elle consiste, en général, à écrire un programme récursif qui se rappelle pour calculer les optima des sous-problèmes. Comme les sous-problèmes se chevauchent, pour éviter que le coût de calcul explose, il est indispensable de *mémoïser* toute solution déjà calculée dans une table pour ne jamais la recalculer.
- L’approche ascendante consiste à anticiper, pour chaque instance, quels sous-instances devront être traitées auparavant et à remplir une table de solutions dans un ordre qui compatible.

**Exemple 1.50.** — Reprenons le problème du rendu de monnaie (exemple 1.39). Rappelons que nous souhaitons exprimer un entier naturel  $m$  (la monnaie à rendre) comme une combinaison des valeurs  $s_1, s_2, \dots, s_k$  (les valeurs faciales des pièces) qui minimise la somme  $\rho$  des coefficients (le nombre de pièces rendues). Nous pouvons écrire les équations suivantes :

$$\rho(m) = \begin{cases} 0 & \text{si } m = 0 \\ 1 + \min_{i \in \llbracket 1, k \rrbracket} \rho(m - s_i) & \text{s'il existe } i \in \llbracket 1, k \rrbracket \text{ tel que } m \geq s_i \text{ et } \rho(m - s_i) \neq \infty \\ \infty & \text{sinon} \end{cases}$$

qui traduisent le principe d’optimalité de Bellman. La deuxième équation se comprend ainsi : pour décomposer la somme  $m$  en une combinaison de pièces minimum, il faut trouver au moins une pièce puis décomposer la somme restante de manière minimale. La troisième équation couvre tous les cas où une décomposition n’est pas possible.

Le code 1.9 ci-dessous retranscrit les relations de récurrence au travers d’un code récursif. Toute valeur  $\rho(m)$  déjà calculée est enregistrée dans le dictionnaire `memo` qui contient les associations  $m \mapsto \rho(m)$ . Lorsqu’une somme monétaire  $m$  ne peut être décomposée avec les pièces de  $S$ , nous utilisons `None` pour le signaler (ce qui correspond au symbole  $\infty$  dans l’équation ci-dessus).

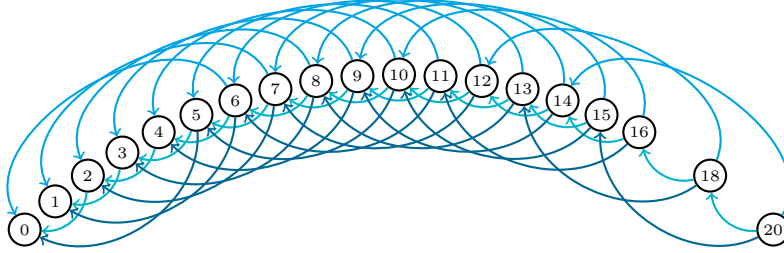
LISTING 1.9. Rendu de monnaie – version descendante

```

151 def coin_change_topdown_dp(m, S, memo = None):
152     if memo == None:
153         memo = dict({0:0})
154     if m not in memo:
155         # we look for the recursive calls
156         rhos = [coin_change_topdown_dp(m-s, S, memo) for s in S if
157                 ↪ m-s >= 0]
158         # we filter out the infinity values
159         rhos = [x for x in rhos if x is not None]
160         if not rhos: # rhos is the empty list
161             memo[m] = None
162         else: # rhos is not the empty list
163             # the recurrence equation applies
164             memo[m] = 1 + min(rhos)
165     return memo[m]

```

Nous pouvons illustrer les enchaînements d'appels récurrents par le diagramme suivant : ici avec un rendu sur la somme  $m = 20$  et un jeu de pièce  $S = (6, 5, 2)$ .



En fin d'exécution, le dictionnaire issu du code 1.9 prend les valeurs suivantes :

|           |   |          |   |          |   |   |   |   |   |   |    |    |    |    |    |    |    |  |    |  |    |
|-----------|---|----------|---|----------|---|---|---|---|---|---|----|----|----|----|----|----|----|--|----|--|----|
| $\rho(m)$ | 0 | 1        | 2 | 3        | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |  | 18 |  | 20 |
| $m$       | 0 | $\infty$ | 1 | $\infty$ | 2 | 1 | 1 | 2 | 2 | 3 | 2  | 2  | 2  | 3  | 3  | 3  | 3  |  | 3  |  | 4  |

L'approche ascendante consiste à constater que l'on peut aussi calculer toutes les valeurs  $(\rho(i))_{0 \leq i \leq m}$  pour des indices croissants. Nous consignons toutes les valeurs calculées dans un tableau. Ici, le point clé est que, dans le code 1.10, chaque accès à une case du tableau `memo` se fait à un moment où sa valeur définitive a déjà été calculée.

LISTING 1.10. Rendu de monnaie – version ascendante

```

165 def coin_change_bottomup_dp(m, S):
166     memo = [None for _ in range(m+1)]
167     memo[0] = 0

```

```

168     for i in range(1,m+1):
169         rhos = [memo[i-s] for s in S if i-s>=0]
170         rhos = [x for x in rhos if x is not None]
171         if not rhos: # rhos is the empty list
172             memo[i] = None
173         else: # rhos is not the empty list
174             memo[i] = 1 + min(rhos)
175     return memo[m]

```

En fin d'exécution sur le même exemple, le dictionnaire issu du code 1.10 prend les valeurs suivantes :

|           |   |          |   |          |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
|-----------|---|----------|---|----------|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| $\rho(m)$ | 0 | 1        | 2 | 3        | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| $m$       | 0 | $\infty$ | 1 | $\infty$ | 2 | 1 | 1 | 2 | 2 | 3 | 2  | 2  | 2  | 3  | 3  | 3  | 3  | 3  | 3  | 4  | 4  |

Cette fois-ci, même les valeurs  $\rho(17)$  et  $\rho(19)$  ont été calculée.

**1.4.4. Recourir à l'aléa.** — Un *algorithme probabiliste* est un algorithme qui a la possibilité de recourir à une opération supplémentaire : le tirage aléatoire et uniforme d'un bit. Par opposition, on parle d'*algorithme déterministe* pour désigner des algorithmes qui produisent toujours la même sortie et passent par les mêmes états.

Générer du « vrai » aléa est une tâche difficile ; en pratique, on se contente de générateurs de nombres pseudo-aléatoire, c'est-à-dire des algorithmes déterministes qui produisent une suite de nombres qui présente des propriétés statistiques proche de la distribution de nombres aléatoires.

Les algorithmes probabilistes sont souvent plus simples à écrire que des algorithmes déterministes de complexité équivalente. Dans certains cas, ils sont plus rapides ou permettent de lisser la complexité en amenant l'algorithme à s'exécuter avec les mêmes performances que le cas moyen.

**Définitions 1.51.** — On dit qu'un algorithme probabiliste est de type *Las Vegas* s'il produit toujours un résultat systématiquement correct mais dont le temps d'exécution peut varier selon les tirages effectués.

On dit qu'un algorithme probabiliste est de type *Monte Carlo* s'il peut produire un résultat parfois incorrect selon les tirages effectués mais dont le temps d'exécution ne varie pas.

**1.4.4.1. Algorithmes de Las Vegas.** — Les algorithmes de type *Las Vegas* sont en général utilisés pour obtenir de bonnes performances temporelles quand le cas moyen est plus intéressant que le pire des cas ou quand le fait que les données soient structurées conduit à de mauvaise performance. Ils sont aussi pertinent dans le cas où un algorithme est exécuté sur des données fournies par un adversaire malin qui souhaiterait nous faire perdre notre temps.



**Exemple 1.52.** — Le problème des  $n$  dames consiste à placer  $n$  dames sur un échiquier de dimension  $n \times n$  dans des positions où elles ne se menacent pas l'une l'autre.

Nous adoptons le format de valeur de retour suivant : les lignes et les colonnes sont numérotées de 0 à  $n - 1$  ; un placement est un tuple  $(r_1, \dots, r_n) \in \llbracket 0, n - 1 \rrbracket^n$  dans lequel les  $n$  dames se trouvent aux positions  $(c, r_c)_{c \in \llbracket 0, n - 1 \rrbracket}$ .

Dans l'illustration ci dessous, le premier échiquier est acceptable mais le second ne l'est pas car des dames peuvent s'attaquer en diagonale

|              | Col 0                                                                             | Col 1                                                                             | Col 2                                                                             | Col 3                                                                             |              | Col 0                                                                             | Col 1                                                                              | Col 2                                                                               | Col 3                                                                               |
|--------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Lig 3        |                                                                                   |  |                                                                                   |                                                                                   | Lig 3        |                                                                                   |                                                                                    |  |                                                                                     |
| Lig 2        |                                                                                   |                                                                                   |                                                                                   |  | Lig 2        |                                                                                   |                                                                                    |                                                                                     |  |
| Lig 1        |  |                                                                                   |                                                                                   |                                                                                   | Lig 1        |  |                                                                                    |                                                                                     |                                                                                     |
| Lig 0        |                                                                                   |                                                                                   |  |                                                                                   | Lig 0        |                                                                                   |  |                                                                                     |                                                                                     |
| [1, 3, 0, 2] |                                                                                   |                                                                                   |                                                                                   |                                                                                   | [1, 0, 3, 2] |                                                                                   |                                                                                    |                                                                                     |                                                                                     |

L'idée naturelle pour résoudre ce problème de manière déterministe serait de procéder par retour sur trace. Il est néanmoins nettement plus rapide de chercher à remplir l'échiquier aléatoirement selon le principe suivant :

---

**Algorithme 1.1 :** Recherche randomisée d'un placement de dames

---

```

1 répéter
2   pour  $c \in \llbracket 0, n - 1 \rrbracket$  faire
3     Calculer l'ensemble  $A$  des positions encore possibles.
4     si  $A$  est vide alors
5       | Interrompre la recherche
6     sinon
7       | Choisir une ligne  $r_c$  aléatoirement parmi les
7       | positions encore possibles.
8 jusqu'à une solution a été trouvée ;

```

---

On écrit le code suivant, qui se distingue d'une recherche par retour sur trace par le fait de s'interrompre s'il ne parvient pas à compléter une solution commencée aléatoirement.

LISTING 1.11. Problème des  $n$  dames - Algorithme de type Las Vegas

```

176 from random import choice
177
178 def extend_board(board, n):

```

```

179     forbidden = set()
180     ncols = len(board)
181     for col in range(ncols):
182         row = board[col]
183         forbidden.add(row)
184         forbidden.add(row + ncols - col)
185         forbidden.add(row - ncols + col)
186     allowed = [x for x in range(n) if x not in forbidden]
187     if allowed:
188         board.append(choice(allowed))
189     return(allowed != [])
190
191
192 def one_try_nqueen(board, n):
193     board.clear()
194     for _ in range(n):
195         if not extend_board(board, n):
196             return False
197     return True
198
199 def iterated_nqueen(n):
200     board = []
201     while not one_try_nqueen(board, n):
202         pass
203     return board

```

L'algorithme 1.1 est un algorithme de type Las Vegas qui fonctionne très rapidement en pratique.

D'aucuns qualifient la situation de « trouver du foin dans un botte de foin » (sic!). On emploie cette idée quand l'objet que l'on cherche à construire est abondant mais que, de temps à autre, sa construction échoue. Ici un résultat mathématique prouve qu'il y a asymptotiquement  $(0.143n)^n$  placements possibles de  $n$  dames.

**Exemple 1.53.** — L'algorithme de *tri rapide* permet de trier un tableau en place. Il fonctionne selon une stratégie *diviser pour régner*.

L'étape *diviser* consiste à réutiliser le principe de réorganisation d'un tableau en place par l'algorithme du drapeau hollandais (voir exemple 1.8). Une valeur pivot  $p$  est choisie dans le tableau ; les éléments rouges sont les éléments du tableau strictement inférieurs à  $p$ , les éléments blancs sont les éléments du tableau égaux à  $p$ , les éléments bleus sont les éléments du tableau strictement supérieurs à  $p$ .

L'étape *régner* consiste à appliquer récursivement l'algorithme de tri aux deux zones formées des éléments strictement inférieur et strictement supérieur au pivot, qui ont été groupés respectivement en tête et en queue du tableau.

L'étape *combinaison* n'est pas nécessaire ici, puisque le tri est effectué en place.

LISTING 1.12. Tri rapide

```

204 from random import choice
205
206 def pivot(T,p,s,t):
207     """
208     Réorganise le tableau T entre les indices s (inclus)
209     et t (exclu) en utilisant le pivot p
210     """
211     r, w, b = s, s, t-1
212     while w <= b:
213         if T[w] < p:
214             T[r], T[w] = T[w], T[r]
215             r += 1
216             w += 1
217         elif T[w] == p:
218             w += 1
219         else:
220             T[w], T[b] = T[b], T[w]
221             b -= 1
222     return(r,w)
223
224 def quick_sort(T, s=0, t=None):
225     if t is None:
226         t = len(T)
227     if t-s > 1:
228         p = choice(T)
229         ss, tt = pivot(T,p,s,t)
230         quick_sort(T, s, ss)
231         quick_sort(T, tt, t)

```

Traditionnellement, la valeur du pivot retenue est la valeur de la première case. Il est cependant judicieux de choisir une valeur aléatoire et triée uniformément parmi les valeurs du tableau. Dans le code 1.12, la ligne qui change tout est la ligne contenant l'instruction `p = choice(T)` et qui rend l'algorithme probabiliste. On obtient ici un algorithme de type *Las Vegas* dont nous admettons la correction.

Dans le cas où le pivot est systématiquement l'élément minimum ou l'élément maximum du tableau, la fonction `pivot` coupe très peu le tableau en deux. Elle est alors appelée récursivement jusqu'à  $n - 1$  fois, où  $n$  est la longueur du tableau, si les valeurs contenues dans le tableau sont toutes distinctes. Comme la complexité en temps de `pivot(T,p,s,t)` est  $O(t - s)$ , ceci montre que l'algorithme de tri rapide peut coûter jusqu'à  $O(n^2)$  opérations élémentaires.

En choisissant un pivot aléatoire, la fonction `pivot` coupe le tableau en deux parties de tailles proches, ce qui conduit à une complexité linéarithmique en moyenne. Nous occultons la preuve mathématique de ce fait.

**Proposition 1.54.** — *La complexité en moyenne du code 1.12 est  $O(n \log_2 n)$ .*

*Démonstration.* — Chaque opération de réorganisation par `pivot(T,p,s,t)` est de complexité proportionnelle à la longueur du tableau traité, c'est-à-dire  $t - s$ . Lors du tout premier appel `pivot`, cette longueur est  $n$ . Lors de tous les autres appels, cette longueur correspond au nombre d'éléments jugés inférieurs ou supérieurs au dernier pivot employé. La somme des longueurs des portions de tableau traitées par `pivot` est donc égale au nombre de paires d'éléments qui ont été comparés à un moment ou un autre au cours de l'exécution de l'algorithme.

Notons  $\tau_0 \leq \tau_1 \leq \dots \leq \tau_{n-1}$  les éléments du tableau. Pour  $i$  et  $j$  fixé avec  $i < j$ , s'il existe un élément  $\tau_k$  choisi comme pivot avec  $i < k < j$  avant que  $\tau_i$  ou que  $\tau_j$  ne soit choisi comme pivot, les éléments  $\tau_i$  et  $\tau_j$  sont définitivement placés dans deux parties distinctes du tableau où ils ne seront plus comparés. Pour que  $\tau_i$  et  $\tau_j$  soient comparés entre eux, il est indispensable que  $\tau_i$  ou  $\tau_j$  soit le premier des éléments  $\{\tau_i, \tau_{i+1}, \dots, \tau_{j-1}, \tau_j\}$ . Or, les pivots étant choisis uniformément, chacun des  $j - i + 1$  éléments de  $\{\tau_i, \tau_{i+1}, \dots, \tau_{j-1}, \tau_j\}$  a autant de chance d'être utilisé comme pivot en premier. Ainsi

$$\mathbb{P}(\tau_i \text{ et } \tau_j \text{ ont été comparés}) = \frac{2}{j - i + 1}.$$

Il ne reste qu'à laisser parler les mathématiques :

$$\begin{aligned} \sum_{0 \leq i < j < n} \frac{2}{j - i + 1} &= \sum_{i=1}^n \sum_{j=i+1}^n \frac{2}{j - i + 1} \\ &= \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} \\ &= \sum_{k=2}^n \sum_{i=1}^{n+1-k} \frac{2}{k} \\ &= \sum_{k=2}^n (n+1-k) \frac{2}{k} \\ &= (n+1) \sum_{k=2}^n \frac{2}{k} - 2(n-1) \\ &= \Theta(n \log_2 n) \end{aligned}$$

Il y a  $\Theta(n \log_2 n)$  comparaisons d'éléments ce qui fournit la complexité du code 1.12 comme attendu.  $\square$

L'avantage d'utiliser un tri rapide randomisé est double : nous seulement nous profitons la plus part du temps d'une complexité raisonnable, mais, de plus, nous déjouons la malédiction de retrier en temps  $O(n^2)$  un tableau qui serait déjà trié.

**Remarque 1.55.** — Notre présentation dévie de la présentation classique du tri rapide : en général, la fonction partition se contente de séparer les éléments inférieur au pivot des éléments strictement supérieurs. Puisque nous avons déjà présenté l'algorithme du drapeau hollandais plus haut, il était plus simple de le reprendre.

**1.4.4.2. Algorithmes de Monte Carlo.** — Les algorithmes de type *Monte Carlo* sont en général utilisés pour confirmer un résultat rapidement avec un bon indice de confiance.

**Exemple 1.56.** — La plupart des tests de primalité utilisés industriellement sont des algorithmes de Monte Carlo. Nous admettons le petit théorème de Fermat suivant

Si  $p$  est un nombre premier et si  $a$  est un entier non divisible par  $p$ , alors  
 $a^{p-1} \equiv 1 \pmod{p}$ .

Nous en déduisons le test suivant.

LISTING 1.13. Test de primalité de Fermat

```

232 from random import randint
233
234 def square_and_multiply_mod_p(a, n, p):
235     if n==0:
236         return 1
237     b = square_and_multiply_mod_p(a, n//2, p) % p
238     if n%2 == 0:
239         return (b * b) % p
240     else:
241         return (a * b * b) % p
242
243 def fermat_test(p):
244     a = randint(2,p-1)
245     x = square_and_multiply_mod_p(a, p-1, p)
246     return ((x-1)%p == 0)

```

En vérité, ce teste reste assez mauvais, car il existe une infinité de nombres composés que ce test de détecte pas. Le test de *Miller-Rabin* fait mieux avec des idées assez proches et reste largement utilisé en pratique.

**Exemple 1.57.** — Le problème de la vérification du produit de deux matrices consiste à vérifier que trois matrices  $\mathbf{A}$ ,  $\mathbf{B}$  et  $\mathbf{C}$  de taille  $n \times n$  satisfont  $\mathbf{AB} = \mathbf{C}$ .

On peut résoudre ce problème en calculant explicitement le produit  $\mathbf{AB}$  et en le comparant à  $\mathbf{C}$ . Si l'on utilise l'algorithme de produit de matrice naïf, la complexité en temps est alors  $O(n^3)$ . Avec des algorithmes plus sophistiqués, on peut même atteindre une complexité  $O(n^\omega)$  où  $\omega \simeq 2,372$ .

**Algorithme 1.2 :** Algorithme de Freivalds**Entrées :** Trois matrices **A**, **B** et **C****Résultat :** Si  $\mathbf{AB} = \mathbf{C}$ , renvoie vrai. Si  $\mathbf{AB} \neq \mathbf{C}$ , renvoie faux avec probabilité  $\geq \frac{1}{2}$ .1 Tirer au hasard un vecteur  $\mathbf{x} \in \{0, 1\}^n$ .2 Calculer  $\mathbf{y} = \mathbf{A}(\mathbf{B}\mathbf{x})$ 3 Calculer  $\mathbf{z} = \mathbf{C}\mathbf{x}$ 4 **si**  $\mathbf{x} = \mathbf{y}$  **alors**5 |   **retourner** *Vrai*6 **sinon**7 |   **retourner** *Faux*

**Lemme 1.58.** — *Le temps d'exécution de l'algorithme 1.2 est déterministe et vaut  $O(n^2)$ .*

**Remarque 1.59.** — Une complexité en  $\Omega(n^2)$  est incontournable pour un algorithme déterministe. En effet, si l'on pouvait se passer de lire l'un des coefficients de **A**, **B** ou **C**, on pourrait imaginer un premier jeu d'entrée tel que  $\mathbf{AB} = \mathbf{C}$ ; puis en modifiant le coefficient non lu, on créerait un second jeu d'entrée tel que  $\mathbf{AB} \neq \mathbf{C}$ . Alors notre algorithme déterministe se tromperait sur l'un des deux cas. Il lui faut donc lire tous les coefficients, ce qui nécessite  $\Omega(n^2)$  opérations.

Il est assez clair que si  $\mathbf{AB} = \mathbf{C}$ , alors, pour tout vecteur  $\mathbf{x}$ , on a  $\mathbf{ABx} = \mathbf{Cx}$ , ce qui montre la correction de l'algorithme dans ce cas.

**Proposition 1.60.** — *Si si  $\mathbf{AB} \neq \mathbf{C}$ , alors l'algorithme 1.2 se trompe avec probabilité  $\leq \frac{1}{2}$ .*

*Démonstration.* — Notons  $\mathbf{D} = \mathbf{AB} - \mathbf{C}$  et supposons  $\mathbf{D} \neq \mathbf{0}$ . Appelons  $((d_{i,j}))_{\substack{1 \leq i \leq n \\ 1 \leq j \leq n}}$  les coefficients de **D**. Appelons  $(x_i)_{1 \leq i \leq n}$  les valeurs du vecteur  $\mathbf{x}$ . Quitte à permuter des lignes et des colonnes, nous pouvons supposer que  $d_{1,1}$  est non nul. Lorsque l'on fixe les  $n - 1$  valeurs  $x_2, \dots, x_n$ , le terme  $d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n$  ne peut s'annuler que pour une valeur de  $x_1$ . Ainsi, quand on tire  $x_1$  uniformément parmi  $\{0, 1\}$ , la probabilité d'annulation est

$$\mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0 | x_2, \dots, x_n) \leq \frac{1}{2}.$$

Mais, en général,

$$\begin{aligned} \mathbb{P}(\mathbf{D}\mathbf{x} = \mathbf{0}) &\leq \mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0) \\ &\leq \sum_{(x_2, \dots, x_n) \in \{0,1\}^n} \frac{1}{2^{n-1}} \mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0 | x_2, \dots, x_n) \\ &\leq \frac{1}{2} \end{aligned}$$

ce qui prouve notre affirmation. □

### 1.5. Exercices

**Exercice 1.61.** — Montrer que l’algorithme de rendu de monnaie (exemple 1.39) ne fournit pas un rendu optimal en général mais que, sous l’hypothèse,

$$\forall i \in \llbracket 2, k \rrbracket, \quad s_1 + \cdots + s_{i-1} < s_i$$

l’algorithme de rendu de monnaie fournit un rendu optimal.

**Exercice 1.62.** — Concevoir un algorithme en temps linéaire qui, étant donné un réel  $x$  et un tableau  $S$  de  $n$  réels triés par ordre croissant, détermine s’il existe deux éléments de  $S$  dont la somme vaut  $x$ .

**Exercice 1.63.** — Romiette se trouve à l’intersection de quatre routes infinies en direction des points cardinaux ; Julio se trouve sur l’une de ces routes à une distance  $d$ . Donner un algorithme permettant à Romiette de retrouver Julio en temps  $O(d)$ .

**Exercice 1.64 (Gnothi seauton).** — On dit qu’une personne est une célébrité si tout le monde la connaît mais elle ne connaît personne. Vous vous rendez en soirée où se trouve  $n$  invités. Pouvez-vous trouver les célébrités en  $O(n)$  questions de la forme “X connaît-il Y ?”

**Exercice 1.65.** — Étant donné un tableau infini, contenant une suite finie de  $n$  chiffres “0” suivie d’une suite infinie de “1”, avec quelle rapidité en fonction de  $n$  peut-on trouver  $n$  ?

**Exercice 1.66.** — On considère le programme Python suivant :

LISTING 1.14. Fonction inconnue

```

247 def f(x, y):
248     if x == 0 or y == 0:
249         return 0
250     elif x==1:
251         return f(x, y-1)
252     else:
253         return f(x-1, y)

```

Donner une spécification de  $f$  et démontrer que le programme est correct (terminaison et correction partielle) par rapport à cette spécification.

**Exercice 1.67.** — On considère les programmes Python suivants :

LISTING 1.15. Fonction inconnue

```

254 def f(x, y):
255     if x == y:
256         return(0)
257     else:
258         return(1 + f(x+1, y+2))
259
260 def g(x, y):
261     if x < y:
262         return(x)
263     else:
264         return(g(f(x, y), y))
265
266 def h(x, y):
267     if y == 0:
268         return(0)
269     else:
270         return h(y, g(x, y))

```

Donner une spécification de  $f$ ,  $g$  et  $h$ . Démontrer que chaque programme est correct (terminaison et correction partielle) par rapport à cette spécification.

**Exercice 1.68.** — On considère le programme Python suivant :

LISTING 1.16. Fonction 91 de MacCarthy

```

271 def macCarthy91(x):
272     if x > 100:
273         return x-10
274     else:
275         return macCarthy91(macCarthy91(x+11))

```

Donner une spécification de `macCarthy91` et démontrer que le programme est correct (terminaison et correction partielle) par rapport à cette spécification.

**Exercice 1.69.** — Montrer qu'il existe une fonction  $\mu : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$  telle que

$$\forall (x, y) \in \mathbb{N}^2, \quad \mu(x, y) = \begin{cases} 0 & \text{si } x = 0 \\ \mu(x-1, \mu(x, y)) & \text{sinon} \end{cases}$$

Que dire de la fonction suivante ?

LISTING 1.17. Fonction 91 de MacCarthy

```

276 def morris(x, y):
277     if x==0:
278         return(0)
279     else:
280         return(morris(x-1,morris(x,y)))

```



**Exercice 1.70.** — Soit  $f$  : une fonction strictement croissante et  $c$  un réel. La notation  $f^{(i)}(n)$  désigne  $f \circ f \circ \dots \circ f(n)$  ( $i$  fois). On définit la notation  $f_c^*$  comme suit :

$$f_c^*(n) = \min\{i \in \mathbb{N}; f^{(i)}(n) \leq c\}.$$

Par exemple, pour le logarithme binaire, avec  $c = 1$ ,

$$\begin{aligned} \log_1^* 2 &= 1 \\ \log_1^* 4 &= 2 \\ \log_1^* 16 &= 3 \\ \log_1^* 65536 &= 4 \\ \log_1^* 2^{65536} &= 5 \end{aligned}$$

Pour chacune des fonctions suivantes, donner une borne sur  $f_c^*(n)$  aussi précise que possible.

| $f(n)$       | $c$ | $f_c^*(n)$ |
|--------------|-----|------------|
| $n - 1$      | 0   |            |
| $n/2$        | 1   |            |
| $n/2$        | 2   |            |
| $\sqrt{n}$   | 2   |            |
| $\sqrt{n}$   | 1   |            |
| $n^{1/3}$    | 2   |            |
| $n/\log_2 n$ | 2   |            |

**Exercice 1.71.** — Classer les fonctions suivantes par ordre de grandeur ( $f = \Omega(g)$ ) en signalant lesquelles sont du même ordre ( $f = \Theta(g)$ ).

|                              |                         |                         |                       |              |                   |
|------------------------------|-------------------------|-------------------------|-----------------------|--------------|-------------------|
| $2^{2^n+1}$                  | $1/n$                   | $(\sqrt{2})^{\log_2 n}$ | $n^2$                 | $n!$         | $(\log_2 n)!$     |
| $\left(\frac{3}{2}\right)^n$ | $n^3$                   | $\log_2^2 n$            | $\log_2(n!)$          | $2^{2^n}$    | $n^{1/\log_2 n}$  |
| $\ln \ln n$                  | $n/\log_2 n$            | $n \cdot 2^n$           | $n^{\log_2 \log_2 n}$ | $\ln n$      | $1$               |
| $2^{\log_2 n}$               | $(\log_2 n)^{\log_2 n}$ | $e^n$                   | $4^{\log_2 n}$        | $(n+1)!$     | $\sqrt{\log_2 n}$ |
| $(\log_2 n)^{1/\log_2 n}$    | $2^{\sqrt{2 \log_2 n}}$ | $n$                     | $2^n$                 | $n \log_2 n$ | $2^{2^{n+1}}$     |

On rappelle que  $n! \simeq \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ .

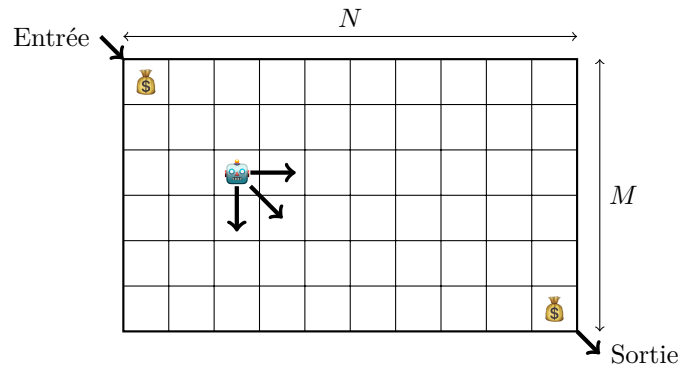
**Exercice 1.72.** — Pour chacune des fonctions  $f(n)$  et des durées  $t$  suivantes, calculer la plus grande taille  $n$  d'un problème que l'on peut résoudre durant le temps  $t$  par un algorithme fonctionnant en  $f(n)$  microsecondes.

| $f \backslash t$ | 1 sec. | 1 min. | 1 h. | 1 jour | 1 mois | 1 an | 1 siècle |
|------------------|--------|--------|------|--------|--------|------|----------|
| $\log_2 n$       |        |        |      |        |        |      |          |
| $\sqrt{n}$       |        |        |      |        |        |      |          |
| $n$              |        |        |      |        |        |      |          |
| $n \log_2 n$     |        |        |      |        |        |      |          |
| $n^2$            |        |        |      |        |        |      |          |
| $n^3$            |        |        |      |        |        |      |          |
| $2^n$            |        |        |      |        |        |      |          |
| $n!$             |        |        |      |        |        |      |          |

**Exercice 1.73.** — Soit  $T$  un tableau de longueur  $n$  ( $n \geq 1$ ) d'entiers relatifs tous distincts. On désire écrire un programme `fixedpoint` dont la valeur de retour est un booléen valant **Vrai** s'il existe un indice  $p \in \llbracket 0, n-1 \rrbracket$  tel que  $T[p] = p$  et valant **Faux** sinon.

1. Écrire une solution `fixedpoint` en Python et en donner la complexité en temps.
2. On suppose désormais que le tableau est ordonné par valeur croissante. Construire une nouvelle version du programme `fixedpoint` dont on fournira le code et la complexité.

**Exercice 1.74.** — *Toto* le robot 🤖 se trouve à l'entrée Nord-Ouest d'un damier rectangulaire de  $M \times N$  cases. Il doit sortir par la sortie Sud-Est en descendant vers le Sud et en allant vers l'Est. Il a le choix à chaque pas (un pas = une case) entre : descendre verticalement vers le Sud ; aller en diagonale vers le Sud-Est ; ou se déplacer horizontalement vers l'Est. Il y a un sac d'or sur chaque case, dont la valeur  $(v_{i,j})_{1 \leq i \leq M, 1 \leq j \leq N}$  est lisible depuis la position initiale de *Toto*. Le but de *Toto* est de ramasser le plus d'or possible durant son trajet.



Écrire un programme Python qui calcule la trajectoire de *Toto*.

## CHAPITRE 2

### STRUCTURES DE DONNÉES

#### 2.1. Induction structurelle

Les définitions récursives sont très présentes lorsqu'on souhaite manipuler certains objets ou programmer certaines fonction en informatique. Cette première section tente de présenter les fondements mathématiques permettant de parler de ces constructions avec rigueur.

Les idées que nous présentons généralisent les constructions par récurrence.

**2.1.1. Langages.** — Dans tout ce cours, nous aurons besoin de désigner et de construire des objets, ce que nous ferons généralement en manipulant des écritures syntaxiques, même si les puristes pourraient nous reprocher de confondre syntaxe et objets eux-mêmes.

**2.1.2. Alphabets, mots et langages.** — Notre point de départ est donc de parler d'alphabet ou de jeu de caractère.

**Définition 2.1.** — Un *alphabet* est un ensemble fini, sans structure particulière, dont les éléments s'appellent *lettres*, ou encore *caractères* dans une terminologie plus informatique, ou parfois aussi *symboles*.

**Exemple 2.2.** — Voici quelques alphabets célèbres :

- L'alphabet  $\Sigma = \{0, 1\}$  s'appelle l'alphabet binaire. Dans le contexte de ce cours, nous donnerons souvent des exemples sur l'alphabet binaire  $\Sigma = \{\mathbf{a}, \mathbf{b}\}$  ou parfois sur l'alphabet ternaire  $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$ .
- L'alphabet latin usuel est  $\Sigma = \{\mathbf{a}, \mathbf{b}, \dots, \mathbf{z}\}$ .
- L'alphabet des chiffres est  $\Sigma = \{0, 1, \dots, 9\}$ .
- Dans un contexte informatique, des jeux de caractères (c'est-à-dire des alphabets) souvent importants sont ASCII, Latin-1 ou Unicode : en plus de former un ensemble, ces jeux de caractère attribuent un numéro à chacun de leurs éléments (par exemple, la lettre A majuscule porte le numéro 65 dans ces trois jeux de caractères), mais cette structure supplémentaire ne nous intéressera pas ici.

**Définition 2.3.** — Un *mot* sur un alphabet  $\Sigma$  est une suite finie, éventuellement vide, de symbole de  $\Sigma$ .

Le mot est désigné en écrivant les lettres les unes à la suite des autres : autrement dit, si  $x_1, \dots, x_n \in \Sigma$  sont des lettres, le mot formé par la suite finie  $x_1, \dots, x_n$  est simplement écrit  $x_1 \cdots x_n$ .

**Définition 2.4.** — L'entier  $n$  est appelé la *longueur* du mot  $w = x_1 \cdots x_n$ . Nous notons  $|w|$  la longueur du mot  $w$ .

**Définition 2.5.** — Quel que soit l'alphabet, il existe un unique mot de longueur 0, c'est-à-dire un unique mot n'ayant aucune lettre, appelé le *mot vide* (ou la *chaîne de caractères vide*).

Étant donné qu'il n'est pas commode de désigner un objet par une absence de symbole, on introduit un symbole spécial, généralement  $\varepsilon$ , pour désigner ce mot vide : on a donc  $|\varepsilon| = 0$ . On souligne que le symbole  $\varepsilon$  *ne fait pas partie* de l'alphabet  $\Sigma$ , c'est un symbole *spécial* qui a été introduit pour désigner le mot vide. (Lorsque les mots sont délimités par des guillemets, comme il est usage pour les chaînes de caractères en informatique, le mot vide n'a pas besoin d'un symbole spécial : il s'écrit juste "" — sans aucun caractère entre les guillemets.)

**Remarque 2.6.** — La notation suivante est souvent utile : si  $w$  est un mot sur un alphabet  $\Sigma$  et si  $z$  est une lettre (i.e. un élément de  $\Sigma$ ), on note  $|w|_z$  le nombre total d'occurrences de la lettre  $z$  dans  $w$ .

Formellement, on peut définir  $|w|_z$  de la façon suivante : si  $w = x_1 \cdots x_n$  où  $x_1, \dots, x_n \in \Sigma$ , alors  $|w|_z$  est le cardinal de l'ensemble  $\{i : x_i = z\}$ . On peut remarquer qu'on a :  $|w| = \sum_{z \in \Sigma} |w|_z$  (i.e., la longueur de  $w$  est la somme des nombres d'occurrences dans celui-ci des différentes lettres de l'alphabet).

**Exemple 2.7.** — À titre d'exemple, sur l'alphabet  $\Sigma = \{a, b, c, d\}$ , le nombre d'occurrences des différentes lettres dans le mot **abbcab** sont :  $|\text{abbcab}|_a = 2$ ,  $|\text{abbcab}|_b = 3$ ,  $|\text{abbcab}|_c = 1$  et  $|\text{abbcab}|_d = 0$ .

L'ensemble de tous les mots sur un alphabet  $\Sigma$  se note  $\Sigma^*$  (cette notion se comprendra mieux après avoir rencontré la définition ?? d'une construction plus générale).

**Remarque 2.8.** — Lorsque l'alphabet  $\Sigma$  est *vide*, c'est-à-dire  $\Sigma = \emptyset$ , alors le mot vide est le seul mot qui existe : on a  $\Sigma^* = \{\varepsilon\}$  dans ce cas. C'est la seule situation où l'ensemble  $\Sigma^*$  des mots est un ensemble fini. Dans la suite, nous négligerons parfois ce cas particulier, qu'on pourra oublier : c'est-à-dire que nous ferons parfois l'hypothèse tacite que  $\Sigma \neq \emptyset$ .

**Définition 2.9.** — On appelle *langage* sur un alphabet  $\Sigma$  tout ensemble de mots, autrement dit toute partie de  $\Sigma^*$ .

**Exemple 2.10.** — L'ensemble  $\{d, dc, dcc, dccc, dcccc, \dots\} = \{dc^r : r \in \mathbb{N}\}$  des mots formés d'un  $d$  suivi d'un nombre quelconque (éventuellement nul) de  $c$  est un langage sur l'alphabet  $\Sigma = \{a, b, c, d\}$ . On verra plus loin que ce langage est « rationnel » (et pourra être désigné par l'expression rationnelle  $dc^*$ ).

**Exemple 2.11.** — Le langage (fini)  $\{foo, bar, baz\}$  constitué des seuls trois mots  $foo, bar, baz$  sur l'alphabet  $\Sigma = \{a, b, f, o, r, z\}$ .

**Exemple 2.12.** — Le langage (fini) constitué des mots de longueur exactement 42 sur l'alphabet  $\Sigma = \{p, q, r\}$ . Cet ensemble a pour cardinal exactement  $3^{42}$ .

**Exemple 2.13.** — Le langage constitué des mots de longueur exactement 1 sur un alphabet  $\Sigma$  (= mots de une seule lettre), qu'on peut identifier à  $\Sigma$  lui-même (en identifiant un mot de une lettre à la lettre en question, cf. ??).

**Exemple 2.14.** — Le langage (fini) constitué du seul mot vide (= mot de longueur exactement 0) sur l'alphabet, disons,  $\Sigma = \{p, q, r\}$ . Ce langage  $\{\varepsilon\}$  a pour cardinal 1 (ou  $3^0$  si on veut). Il ne faut pas le confondre avec le suivant :

**Exemple 2.15.** — Le langage vide, qui ne contient aucun mot (sur un alphabet quelconque). Ce langage a pour cardinal 0.

**Exemple 2.16.** — Le langage sur l'alphabet  $\Sigma = \{a, b\}$  constitué des mots qui commencent par trois  $a$  consécutifs : ou, si on préfère, qui ont le mot  $aaa$  comme préfixe.

**Exemple 2.17.** — Le langage sur l'alphabet  $\Sigma = \{a, b\}$  constitué des mots qui contiennent trois  $a$  consécutifs ; ou, si on préfère, qui ont  $aaa$  comme facteur.

**Exemple 2.18.** — Le langage sur l'alphabet  $\Sigma = \{a, b\}$  constitué des mots qui contiennent au moins trois  $a$ , non nécessairement consécutifs ; ou, si on préfère, qui ont  $aaa$  comme sous-mot.

**Exemple 2.19.** — Le langage sur l'alphabet  $\Sigma = \{a\}$  constitué de tous les mots dont la longueur est un nombre premier ( $L = \{aa, aaa, a^5, a^7, a^{11}, \dots\}$ ). Ce langage est infini.

**Exemple 2.20.** — Le langage sur l'alphabet  $\Sigma = \{0, 1\}$  constitué de tous les mots commençant par un 1 et qui, interprétés comme un nombre écrit en binaire, désignent un nombre premier ( $L = \{10, 11, 101, 111, 1011, \dots\}$ ).

**Exemple 2.21.** — Le langage sur l'alphabet Unicode constitué de tous les mots qui représentent un document XML bien-formé d'après la spécification XML 1.0.

**Remarque 2.22.** — On pourrait aussi considérer un langage (sur l'alphabet  $\Sigma$ ) comme une *propriété* des mots (sur l'alphabet en question). Précisément, si  $P$  est une propriété qu'un mot  $w \in \Sigma^*$  peut ou ne pas avoir, on considère le langage  $L_P = \{w \in \Sigma^* : w \text{ a la propriété } P\}$ , et inversement, si  $L \subseteq \Sigma^*$  est un langage, on considère la propriété « appartenir à  $L$  » : en identifiant la propriété et le langage qu'on vient d'associer l'un à l'autre (par exemple, le langage des mots commençant par  $a$  et la propriété « commencer par  $a$  »), un langage pourrait être considéré comme une propriété des mots.

Ce qui précède n'a rien de spécifique aux langages : une partie d'un ensemble  $E$  quelconque peut être identifiée à une propriété que les éléments de  $E$  peuvent ou ne pas avoir, à savoir, appartenir à la partie en question : nous utiliserons cette confusion

constamment lorsque nous définirons des ensembles par induction structurale dans la partie suivante.

**2.1.2.1. Concaténation.** — Evoquons l'opération centrale pour la manipulation de mots.

**Définition 2.23.** — Si  $u = x_1 \cdots x_m$  et  $v = y_1 \cdots y_n$  sont deux mots, de longueurs respectives  $m$  et  $n$ , sur un même alphabet  $\Sigma$ , alors on définit un mot  $uv = x_1 \cdots x_m y_1 \cdots y_n$  de longueur  $m + n$ , dont les lettres sont obtenues en mettant bout à bout celles de  $u$  puis celles de  $v$  (dans cet ordre), et on l'appelle *concaténation* (ou, si cela ne prête pas à confusion, simplement *produit*) des mots  $u$  et  $v$ . (Dans un contexte informatique, on parle de concaténation de chaînes de caractères.)

**Proposition 2.24.** — Parmi les propriétés de la concaténation, signalons les faits suivants :

- le mot vide  $\varepsilon$  est « neutre » pour la concaténation, ce qui signifie par définition :  $\varepsilon w = w\varepsilon = w$  quel que soit le mot  $w \in \Sigma^*$  ;
- la concaténation est « associative », ce qui signifie par définition :  $u(vw) = (uv)w$  quels que soient les mots  $u, v, w \in \Sigma^*$  (on peut donc noter  $uvw$  sans parenthèse).

On peut traduire de façon savante ces deux propriétés en une phrase : l'ensemble  $\Sigma^*$  est un *monoïde*, d'élément neutre  $\varepsilon$ , pour la concaténation (cela signifie exactement ce qui vient d'être dit).

On a par ailleurs  $|uv| = |u| + |v|$  (la longueur de la concaténation de deux mots est la somme des concaténations), et on rappelle par ailleurs que  $|\varepsilon| = 0$  ; on peut traduire cela de manière savante : la longueur est un *morphisme de monoïdes* entre le monoïde  $\Sigma^*$  des mots (pour la concaténation) et le monoïde  $\mathbb{N}$  des entiers naturels (pour l'addition) (cela signifie exactement ce qui vient d'être dit).

**Remarque 2.25 (Complément).** — Le monoïde  $\Sigma^*$  possède la propriété suivante par rapport à l'ensemble  $\Sigma$  : si  $M$  est un monoïde quelconque (c'est-à-dire un ensemble muni d'une opération binaire associative  $\cdot$  et d'un élément  $e$  neutre pour cette opération), et si  $\psi : \Sigma \rightarrow M$  est une application quelconque, alors il existe un unique morphisme de monoïdes  $\hat{\psi} : \Sigma^* \rightarrow M$  (c'est-à-dire une application préservant le neutre et l'opération binaire) tel que  $\hat{\psi}(x) = \psi(x)$  si  $x \in \Sigma$ . (Démonstration : on a nécessairement  $\hat{\psi}(x_1 \cdots x_n) = \psi(x_1) \cdots \psi(x_n)$ , or ceci définit bien un morphisme comme annoncé.) On dit qu'il s'agit là d'une propriété « universelle », et plus précisément que  $\Sigma^*$  est le *monoïde libre* sur l'ensemble  $\Sigma$ . Par exemple, le morphisme « longueur »  $\ell : \Sigma^* \rightarrow \mathbb{N}$  est le  $\ell = \hat{\psi}$  obtenu en appliquant cette propriété à la fonction (constante)  $\psi(x) = 1$  pour tout  $x \in \Sigma$ .

**Définition 2.26.** — Lorsque  $w \in \Sigma^*$  et  $r \in \mathbb{N}$ , on définit un mot  $w^r$  comme la concaténation de  $r$  mots tous égaux à  $w$ , autrement dit, comme la répétition  $r$  fois du mot  $w$ . Formellement, on définit par récurrence :

- $w^0 = \varepsilon$  (le mot vide),

—  $w^{r+1} = w^r w$ .

(Ces définitions valent, d'ailleurs, dans n'importe quel monoïde. On peut constater que  $w^r w^s = w^{r+s}$  quels que soient  $r, s \in \mathbb{N}$ .) On a bien sûr  $|w^r| = r|w|$ .

Cette définition sert notamment à désigner de façon concise les mots comportant des répétitions d'une même lettre : par exemple, le mot  $aaaaa$  peut s'écrire tout simplement  $a^5$ , et le mot  $aaabb$  peut s'écrire  $a^3b^2$ . (De même que pour le mot vide, il faut souligner que ces exposants *ne font pas partie* de l'alphabet.)

**2.1.3. Définitions inductives.** — Informellement, définir un ensemble par induction revient à le décrire par des règles de constructions. On obtient alors des définitions qui ont l'air de se mordre la queue.

**Définitions 2.27.** — Soit  $n$  un entier naturel. On appelle *règle d'inférence* la donnée de  $n + 1$  prédicats logiques :  $P_1, \dots, P_n$  et  $Q$ . Les prédicats  $P_1, \dots, P_n$  et  $Q$  s'appellent les *prémisses* ; le prédicat  $Q$  s'appelle la *conclusion*.

L'entier  $n$  s'appelle l'*arité* de la règle.

Un *axiome* est une règle d'arité nulle.

**Remarque 2.28.** — On présente une règle d'inférence sous la forme :

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{Q} \text{ (nom de la règle)}$$

**Exemple 2.29.** — Voici deux règles

1. (zéro) l'entier 0 est pair ;
2. (successeur) pour tout entier  $n$ , si  $n$  est pair, alors  $n + 2$  est pair.

Ces deux règles peuvent aussi se présenter sous la forme :

$$\frac{}{0 \in P} \text{ (zéro)} \qquad \frac{n \in P}{n + 2 \in P} \text{ (successeur)}$$

où  $\mathcal{P}$  désigne l'ensemble des nombres pairs. La première des deux règles est même un axiome.

**Définition 2.30.** — On parle de *système de règles d'inférence* pour désigner tout ensemble de règles d'inférences.

Dans ce chapitre, nous définissons par induction des parties  $\mathcal{X}$  d'un ensemble déjà existant si bien que tous nos prédicats prendront la forme  $x \in \mathcal{X}$ .

**Définition 2.31.** — Soit  $E$  un ensemble. Une *définition inductive d'une partie  $\mathcal{X}$  de  $E$*  est la donnée d'un système d'inférence dont chacune des règles est de la forme

$$(1) \quad \frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \dots \quad x_{n_r} \in \mathcal{X} \quad (x_1, \dots, x_{n_r}) \in \text{Dom}(r)}{r(x_1, \dots, x_{n_r}) \in \mathcal{X}} \text{ (r)}$$

pour une certaine fonction  $r : E^{n_r} \rightarrow E$  dont le domaine de définition est noté  $\text{Dom}(r)$ .

**Remarque 2.32.** — Dans la suite du texte, nous omettrons d'écrire systématiquement la dernière prémisses  $(x_1, \dots, x_n) \in \text{Dom}(r)$ . Nous disons que l'arité de la règle ci-dessus est  $n_r$ .

**Remarque 2.33.** — Dans la définition précédente, lorsque l'entier  $n_r$  est nul, il faut comprendre  $E^{n_r}$  comme l'ensemble des tuple de longueur nulle. Il n'y a qu'un seul tel tuple, à savoir le tuple  $()$ . Par conséquent, une fonction  $E^0 \rightarrow E$  est en vérité une fonction constante, ne possédant qu'une seule association  $() \mapsto e$  entre le tuple vide et un certain élément  $e \in E$ . Ainsi, quand  $n_r$  est nul, nous sommes simplement en train d'introduire une règle qui est un axiome.

**Exemple 2.34 (Mise en garde).** — Le système de règles d'inférence de l'exemple 2.29 semble définir les entiers pairs, à savoir l'ensemble des entiers de la forme  $2n$  avec  $n \in \mathbb{N}$ . Est-ce bien le cas ? En vérité, il existe toutes sortes d'ensembles qui satisfont les deux règles du système : l'ensemble que nous cherchons à définir  $\{2n; n \in \mathbb{N}\}$ , mais aussi les ensembles  $\mathbb{N}$ ,  $\mathbb{Z}$ ,  $\mathbb{R}$ ,  $\mathbb{C}$ , l'ensemble des racines de polynômes à coefficients entiers, etc. Le système de règles de l'exemple 2.29 n'est donc pas suffisant tout seul pour définir les entiers de la forme  $2n$  avec  $n \in \mathbb{N}$ .

**Théorème 2.35.** — *Étant donné une définition inductive d'une partie de  $E$ , il existe un ensemble  $\mathcal{X}_0$  qui est le plus petit (au sens de l'inclusion) des ensembles  $\mathcal{X}$  tels que, pour toute règle de la forme (1), on a ait*

- (2) *si les appartenances  $x_1 \in \mathcal{X}$ ,  $x_2 \in \mathcal{X}$ , ...,  $x_n \in \mathcal{X}$  sont vérifiées et si  $r(x_1, x_2, \dots, x_n)$  est défini, alors, on a aussi l'appartenance  $r(x_1, x_2, \dots, x_n) \in \mathcal{X}$ .*

*Démonstration.* — Notons  $\mathcal{F}$  la famille des ensembles  $\mathcal{X}$  tels que pour tout  $x \in E$  et pour toute règle  $r$  du système, si les appartenances  $x_1 \in \mathcal{X}$ ,  $x_2 \in \mathcal{X}$ , ...,  $x_n \in \mathcal{X}$  sont vérifiées, alors on a aussi l'appartenance  $r(x_1, x_2, \dots, x_n) \in \mathcal{X}$ .

Nous pouvons remarquer que la famille  $\mathcal{F}$  contient au moins  $\mathcal{X} = E$ , donc est une famille non-vide. Intéressons nous à la partie de  $\mathcal{X}_0$  définie par

$$\mathcal{X}_0 = \bigcap_{\mathcal{X} \in \mathcal{F}} \mathcal{X}.$$

Nous pouvons observer que  $\mathcal{X}_0$  satisfait la condition (2).

De plus,  $\mathcal{X}_0$  est forcément inclus dans un ensemble vérifiant (2).  $\square$

**Définition 2.36.** — Dans les conditions du théorème 2.35, on dit que le sous-ensemble  $\mathcal{X}_0 \subseteq E$  est défini inductivement.

**Exemple 2.37.** — L'ensemble des entiers naturels pairs, à savoir l'ensemble

$$\{2n; n \in \mathbb{N}\}$$

est défini inductivement par le système de l'exemple 2.29 comme sous-ensemble de  $E = \mathbb{R}$ .

**Exemple 2.38.** — Soit  $\Sigma = \{a, b, c\}$  un alphabet à trois lettres. Le langage  $\mathcal{L}$  des mots de  $\Sigma$  la forme  $a^n b c^n$ , avec  $n \in \mathbb{N}$ , est défini inductivement par le système, contenant deux règles dont un axiome,



$$\frac{}{\mathbf{b} \in \mathcal{L}} \qquad \frac{w \in \mathcal{L}}{\mathbf{awc} \in \mathcal{L}}$$

Ce langage contient

$$\mathcal{X} = \{\mathbf{b}, \mathbf{abc}, \mathbf{aabcc}, \mathbf{aaabccc}, \mathbf{aaaabcccc}, \dots\}.$$

**Exemple 2.39.** — Soit  $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ . La définition inductive d'une partie de  $\Sigma^*$  donnée par les cinq règles d'inférence, dont trois axiomes,

$$\frac{}{\varepsilon \in \mathcal{P}} \qquad \frac{}{\mathbf{a} \in \mathcal{P}} \qquad \frac{}{\mathbf{b} \in \mathcal{P}} \qquad \frac{w \in \mathcal{P}}{\mathbf{awa} \in \mathcal{P}} \qquad \frac{w \in \mathcal{P}}{\mathbf{bwb} \in \mathcal{P}}$$

permet de définir l'ensemble  $\mathcal{P}$  des palindromes.

**2.1.3.1. Les listes.** — En informatique, une liste est définie par le fait qu'elle est soit vide, soit constitué d'un premier élément suivi d'une liste. Cette définition semble se mordre la queue, mais elle est parfaitement formalisée par une définition inductive.

**Définition 2.40.** — Soit  $\Sigma_0$  un alphabet ne contenant pas les trois symboles  $(, : :$  et  $)$ , soit  $E$  un langage sur  $\Sigma_0$  et soit  $\Sigma = E \cup \{\perp, (, : :, )\}$ . On appelle *ensemble des listes à valeur dans  $E$*  la partie de  $\Sigma^*$ , définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e::q) \in \mathcal{L}} \text{ (Liste de tête } e\text{)}$$

où il y a un axiome et autant de règles supplémentaires que d'éléments de  $E$ .

**Remarque 2.41.** — Nous esquivons les difficultés en définissant les listes par la syntaxe de leurs éléments et non leurs éléments eux-même.

**Exemple 2.42.** — La liste formée des entiers « 7 puis 9 puis 4 » correspond formellement au mot

$$(7::(9::(4::\perp))).$$

**Implémentation 2.43 (C).** — Cette définition inductive coïncide naturellement avec la construction classique de liste chaînée, dont on rappelle la syntaxe en C (croisée dans le cours Inf107), et qui permet de profiter de l'allocation de mémoire de taille variable en cours d'exécution du programme :

LISTING 2.1. Liste chaînée en C

```

281  #include <stdio.h>
282  #include <stdlib.h>
283
284  typedef struct node node_t;
285  struct node {
286      int data;
287      node_t *next;
288  };
289
290  int main() {
291      node_t *m1 = malloc(sizeof(node_t));

```

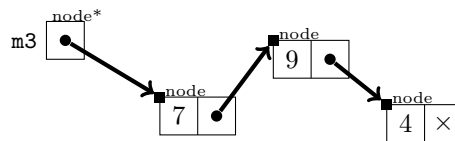
```

292     m1->data = 3;
293     m1->next = NULL;
294     node_t *m2 = malloc(sizeof(node_t));
295     m2->data = 7;
296     m2->next = m1;
297     node_t *m3 = malloc(sizeof(node_t));
298     m3->data = 1;
299     m3->next = m2;
300     for (node_t *p = m3; p!=NULL; p=p->next){
301         printf("%d ",p->data);
302     }
303     return 0;
304 }

```

La liste vide se déclare alors par un simple pointeur nul. Une liste non-vide se déclare par un maillon, dont un champ fait référence à une donnée et l'autre champ fait référence au reste de la liste.

Dans l'exemple que nous avons construit, l'organisation en mémoire correspond à la situation suivante :



(où les trois maillons ont pu être alloué un peu n'importe comment sur le tas).

**Implémentation 2.44.** — Le langage Python étant plus moderne et plus élaboré, il est artificiel de vouloir créer une liste en déclarant explicitement des maillons et en les manipulant directement. On peut cependant déclarer des maillons sous forme d'objets et reproduire le même comportement que le code C ci-dessus.

LISTING 2.2. Liste chaînée en Python

```

305 class Node:
306     def __init__(self, data):
307         self.data = data
308         self.next = None
309
310 m1 = Node(4)
311 m2 = Node(9)
312 m2.next = m1
313 m3 = Node(7)
314 m3.next = m2
315
316 m = m3
317 while True:

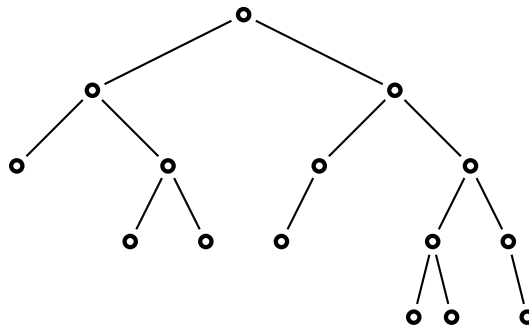
```

```

318     print(m.data)
319     if m.next is not None:
320         m = m.next
321     else:
322         break

```

**2.1.3.2. Les arbres binaires.** — Souvent, nous serons amenés à représenter ou à traiter de l'information organisée hiérarchiquement. Nous voulons alors encoder de l'information qui correspond à un graphique tel que



Notez que seule importe la position relative des sommets les uns par rapport aux autres.

**Définition 2.45.** — On appelle *ensemble des arbres binaires* l'ensemble  $\mathcal{A}$  des mots sur l'alphabet  $\Sigma = \{\perp, (, |, )\}$  définis inductivement par le système

$$\frac{}{\perp \in \mathcal{A}} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}}$$

Lorsqu'un arbre  $a$  est de la forme  $a = (g|d)$ , on dit que  $g$  est le sous-arbre *gauche* de l'arbre  $a$  et que  $d$  est le sous-arbre *droit* de l'arbre  $a$ .

**Implémentation 2.46.** — De façon correcte mais frustrante, nous pouvons représenter un arbre en python en utilisant la valeur `None` pour la première règle et des couples pour la seconde règle.

**Exemple 2.47.** — L'arbre binaire dessiné ci-dessus peut parfaitement se représenter en python comme

```

323 t = (((None, None), ((None, None), (None, None))),
324      (((None, None), None),
325      (((None, None), (None, None)), (None, (None, None)))))

```

C'est un format complètement illisible ou tout du moins pénible à manipuler pour un ou une programmeuse. En effet, il faut systématiquement se souvenir que le sous-arbre gauche correspond à la composante n° 0 du couple et le sous-arbre droit correspond à la composante n° 1 du couple.

**Remarque 2.48.** — Dans l'exemple 2.47, on pourrait se demander comment la variable  $t$  est stockée en mémoire. Un site, tel que <https://pythontutor.com/>, nous renseigne : nous laissons la lectrice ou le lecteur faire l'expérience.

**Définition 2.49.** — Soit  $a = (g|d)$  un arbre. On appelle *racine* de l'arbre  $a$  l'instance de la règle  $(\cdot | \cdot)$  employée pour joindre les sous arbres  $g$  et  $d$ . L'ensemble des racines des sous-arbres s'appellent les *nœuds* ou les *sommets* de l'arbre.

L'arbre vide n'a pas de racine et n'a pas de sommet.

Mathématiquement, la définition de racine n'a pas beaucoup d'intérêt. Informatiquement cependant, elle coïncide avec le fait que pour introduire un arbre, nous avons besoin d'un mode de construction systématique.

**Implémentation 2.50 (C).** — En C, on écrit le code suivant, qui instancie la première règle d'inférence (création de l'arbre vide) par le pointeur nul et la seconde règle grâce à un enregistrement constitué de deux pointeurs vers chacun des fils.

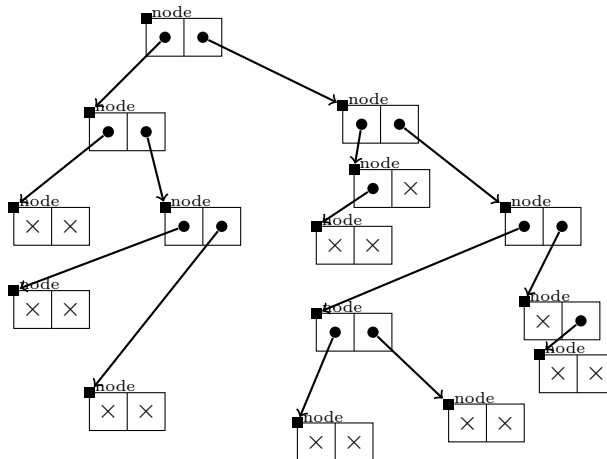
LISTING 2.3. Arbres binaires (version sans étiquette) en C

```

326 typedef struct node node_t;
327 struct node {
328     node_t *left;
329     node_t *right;
330 };

```

L'arbre donné en exemple est alors



où les espaces mémoires ont pu être alloués sans organisation particulière.

**Implémentation 2.51.** — En Python, on peut procéder comme suit si l'on souhaite contrôler la création d'objets qui sont reliés entre eux.

LISTING 2.4. Arbres binaires (version sans étiquette) en Python

```

331 class Node():
332     def __init__(self, a = None, b = None):
333         self.left = a
334         self.right = b
335
336 s = Node(
337     Node(
338         Node(),
339         Node(
340             Node(),
341             Node()))),
342     Node(
343         Node(),
344         Node(
345             Node(
346                 Node(),
347                 Node()),
348             Node()))))

```

**Remarque 2.52.** — En apparence, dans le code 2.4, l'information à stocker est la même (on retrouve la même série de parenthésage que dans l'exemple 2.47!). Néanmoins le choix de représentation fait mieux apparaître qui joue quoi comme rôle. Le fait de passer par le constructeur `Node` marque l'utilisation de la deuxième règle de la définition inductive. Par ailleurs, l'accès aux sous-arbres gauche ou droit se fait de manière nommée en utilisant les attributs de l'objet, plutôt qu'en comptant sur une convention d'utilisation des composantes d'un couple comme nous avons fait dans l'exemple 2.47.

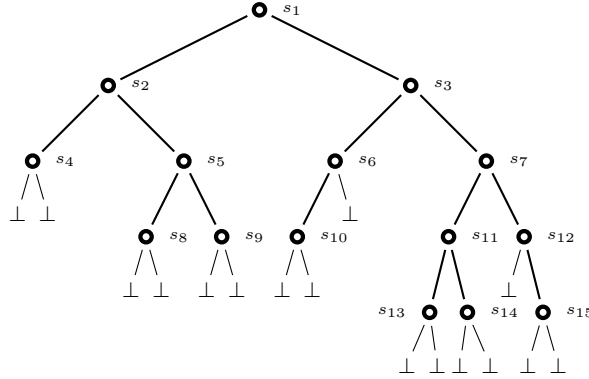
Précisons le vocabulaire des arbres.

**Définitions 2.53.** — Nous parlons de

- *fil gauche* d'un arbre  $a = (g|d)$ , ou de sa racine, pour désigner la racine du sous-arbre gauche  $g$  de ce sommet (si elle existe)
- *fil droit* d'un arbre  $a = (g|d)$ , ou de sa racine, pour désigner la racine du sous-arbre droit  $d$  de ce sommet (si elle existe)
- *père* d'un sommet  $s$  pour désigner l'unique sommet ayant  $s$  comme fils gauche ou droit (la racine d'un arbre ne possède pas de père)
- *sommet interne* pour désigner un sommet ayant au moins un fils (qu'il soit gauche ou droit)
- *feuille* pour désigner un sommet n'ayant pas de fils, autrement dit pour désigner un sommet  $f = (g|d)$  où  $g$  et  $d$  sont l'arbre vide.
- *ancêtre* d'un sommet  $s$  pour désigner tout sommet  $s'$  tel qu'il existe une suite de sommet  $s = s_0, s_1, \dots, s_k = s'$  telle que pour tout indice  $i \in \llbracket 0, k-1 \rrbracket$ , le sommet  $s_{i+1}$  est le père du sommet  $s_i$ .

- *descendant* d'un sommet  $s$  pour désigner tout sommet  $s'$  tel qu'il existe une suite de sommet  $s = s_0, s_1, \dots, s_k = s'$  telle que pour tout indice  $i \in \llbracket 0, k-1 \rrbracket$ , le sommet  $s_{i+1}$  est un fils du sommet  $s_i$ .

**Exemple 2.54.** — Dans l'arbre



la racine est  $s_1$  ; les nœuds internes sont  $s_1, s_2, s_3, s_5, s_6, s_7, s_{11}, s_{12}$  ; les feuilles sont  $s_4, s_8, s_9, s_{10}, s_{13}, s_{14}, s_{15}$ .

Le père de chaque sommet est donné par le tableau suivant

| Sommet | $s_2$ | $s_3$ | $s_4$ | $s_5$ | $s_6$ | $s_7$ | $s_8$ | $s_9$ | $s_{10}$ | $s_{11}$ | $s_{12}$ | $s_{13}$ | $s_{14}$ | $s_{15}$ |
|--------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|
| Père   | $s_1$ | $s_1$ | $s_2$ | $s_2$ | $s_3$ | $s_3$ | $s_5$ | $s_5$ | $s_6$    | $s_7$    | $s_7$    | $s_{10}$ | $s_{10}$ | $s_{11}$ |

Le sommet  $s_7$  a pour ancêtres les sommets  $s_1, s_3$  et  $s_7$  ; le sommet  $s_7$  a pour descendants les sommets  $s_7, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}$ .

**Exemple 2.55.** — Il n'y a qu'un seul arbre binaire dont la racine est une feuille. C'est l'arbre qui ne possède qu'un seul sommet, que l'on écrirait en Python `Node(None, None)` et que l'on dessinerait par



**Remarque 2.56.** — Dans l'implémentation, il peut être intéressant que chaque sommet connaisse son père, autrement dit que l'on adopte des maillons de la forme.

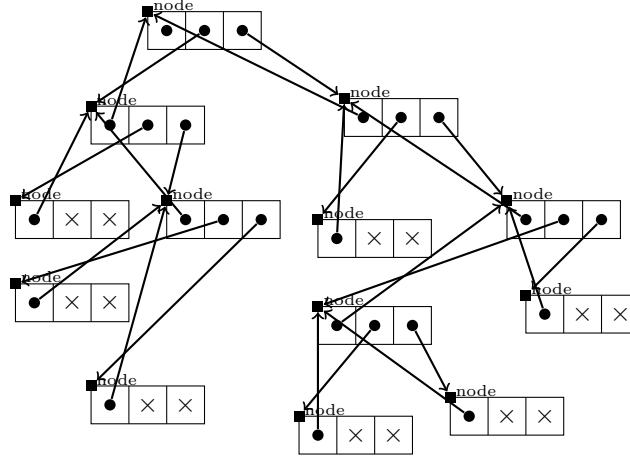
LISTING 2.5. Arbres binaires (version sans étiquette) en C

```

349 typedef struct node node_t;
350 struct node {
351     node_t *parent;
352     node_t *left;
353     node_t *right;
354 };

```

L'arbre donné en exemple est alors

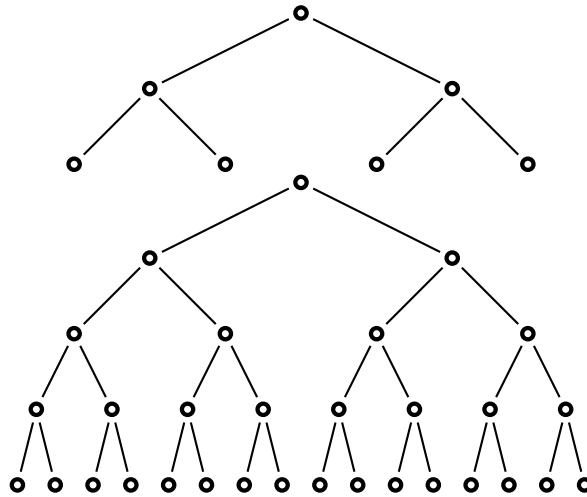


Certains arbres ont une forme particulière.

**Définition 2.57.** — On dit qu'un arbre binaire est *complet* s'il appartient au sous-ensemble  $\mathcal{C}$  des arbres binaires défini par les règles

$$\frac{}{\perp \in \mathcal{C}} \qquad \frac{c \in \mathcal{C}}{(c|c) \in \mathcal{C}}$$

**Exemple 2.58.** — Voici deux exemples (dans les illustrations ci-dessous, seules les occurrences de la règle d'introduction de sommets sont marquées, les copies des arbres vides sont omises).



**2.1.3.3. Arbres binaires étiquetés.** — Les arbres binaires restent malgré tout pauvres. En général, on souhaite y associer de l'information, comme dans l'exemple de la figure 2.1.

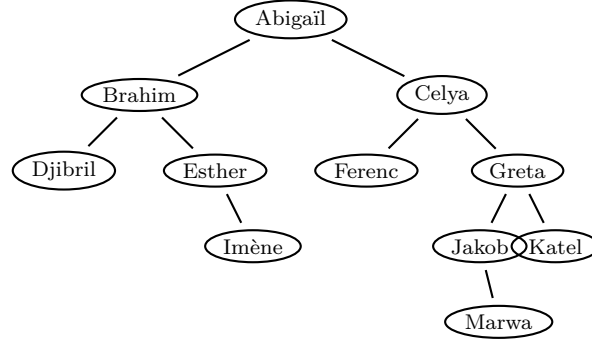


FIGURE 2.1. Un exemple d'arbre binaire étiqueté

**Définition 2.59.** — Soit  $\Sigma_0$  un alphabet ne contenant pas les trois symboles  $\perp$ ,  $($ ,  $)$  et soit  $E$  un langage sur  $\Sigma_0$ . On appelle *ensemble des arbres binaires étiquetés par  $E$*  l'ensemble  $\mathcal{A}$  des mots de  $\Sigma = E \cup \{\perp, (, |, )\}$  définis par le système

$$\frac{}{\perp \in \mathcal{A}} \text{ (Arbre vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine étiquetée par } e\text{)}$$

où  $e$  appartient à  $E$ .

**Implémentation 2.60.** — Pour représenter des arbres étiquetés, nous pouvons utiliser une structure de maillon comme suit en C : la première règle est instanciée à travers le pointeur nul, la seconde règle se construit avec le type suivant.

LISTING 2.6. Arbres binaires (version avec étiquette) en C

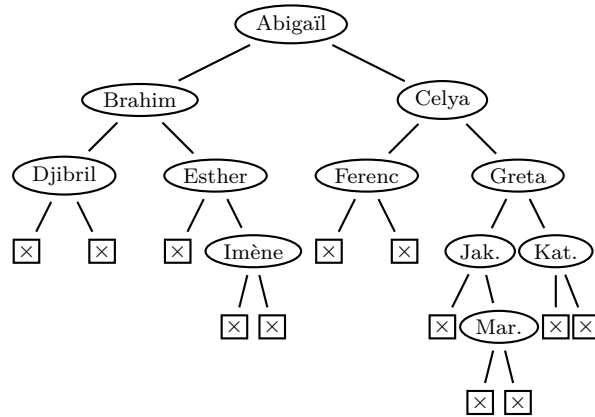
```

355 typedef struct node node_t;
356 struct node {
357     void *data;
358     node_t *left;
359     node_t *right;
360 };
  
```

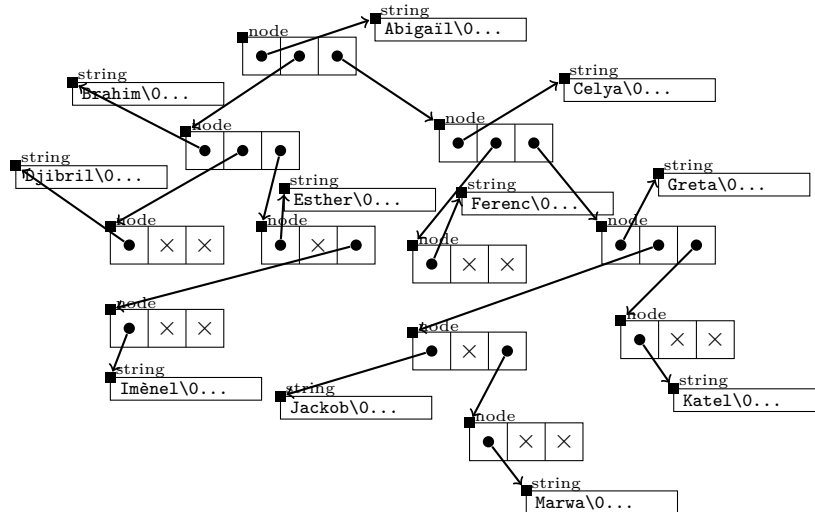
où l'on a supposé que les étiquettes de l'arbre sont toutes entières.

Voici à quoi ressemblerait l'organisation en mémoire de l'exemple ci-dessous. Commençons par noter que, en vérité, nous sommes obligé de stocker des copies de l'arbre vide pour « fermer » chacune des branches.





L'occupation en mémoire de cet arbre devient alors



où les espaces mémoires ont pu être alloués sans organisation particulière.

**Implémentation 2.61.** — En python, il suffit d'enrichir la classe précédente (voir code 2.4). La constante `None` implémente la première règle d'inférence. Le code suivant implémente exactement la seconde règle d'inférence

LISTING 2.7. Arbres binaires (version avec étiquette) en Python

```

361 class Node():
362     def __init__(self, d, a = None, b = None):
363         self.data = d
364         self.left = a
365         self.right = b
366 
```

```

367 arb = Node("Abigael",
368           Node("Brahim",
369               Node("Djibril"),
370               Node("Esther",
371                   None,
372                   Node("Imene")))),
373           Node("Celya",
374               Node("Ferenc"),
375               Node("Greta",
376                   Node("Jakob",
377                       None,
378                       Node("Marwa")),
379                   Node("Katel"))))

```

Le vocabulaire pour les arbres binaires (*cf.* définition 2.53) s'étend aux arbres binaires étiquetés.

**Exemple 2.62.** — Dans l'arbre ci-dessus, la racine est Abigaïl. Le sommet Celya possède un fils gauche qui est Ferenc et un fils droit qui est Greta. Le père de Celya est Abigaïl. L'ensemble des ancêtres de Greta comprend Abigaïl, Celya et Greta; l'ensemble des descendants de Greta comprend Greta, Jakob, Katel et Marwa.

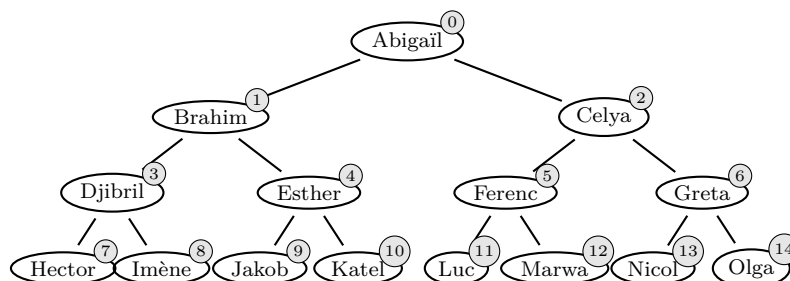
Les feuilles sont Djibril, Imène, Ferenc, Marwa et Katel. Les sommets internes sont Abigaïl, Brahim, Celya, Esther, Greta et Jakob.

**Remarque 2.63.** — Comme dans le cas non étiqueté, on peut définir des *arbres binaires étiqueté complet* (formellement, il faut pour cela avoir défini la hauteur d'un arbre, ce qui sera fait plus loin, *cf.* exemple ??).

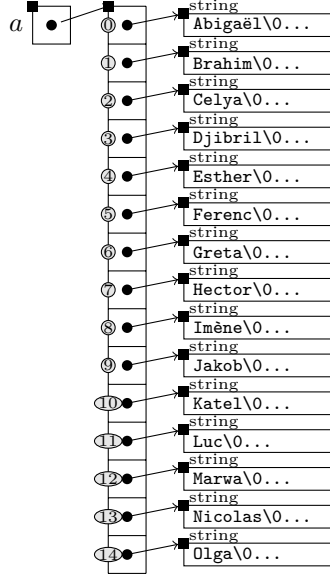
Ce type d'arbre très compact admet une représentation par un tableau : on numérote les sommets par lignes successives entre 0 et  $2^h - 1$  et on inscrit l'étiquette du sommet de n°  $i$  dans la case n°  $i$  d'un tableau.

On constate alors que le fils gauche du sommet n°  $i$  est le sommet n°  $2i + 1$  et le fils droit du sommet n°  $i$  est le sommet n°  $2i + 2$ . Inversement, le père du sommet n°  $i$  est le sommet  $\lfloor (i - 1)/2 \rfloor$ . Il reste donc très facile de naviguer dans l'arbre d'un sommet à ses voisins.

**Exemple 2.64.** — Voici un exemple :



Insistons sur le fait que nous stockons alors l'arbre à l'aide d'un tableau. Ici, nous aurions l'organisation en mémoire suivante.



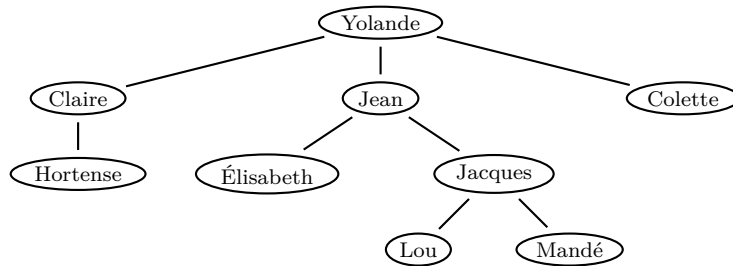
**2.1.3.4. Les arbres généraux.** — Les arbres généraux non vides sont formés d'une racine et d'une liste d'arbres généraux non vides.

**Définition 2.65.** — Soit  $\Sigma_0$  un alphabet ne contenant pas les trois symboles  $\perp$ ,  $($ ,  $|$ ,  $)$  et soit  $E$  un langage sur  $\Sigma_0$ . On appelle *ensemble des arbres binaires étiquetés par  $E$*  l'ensemble  $\mathcal{A}$  des mots de  $\Sigma = E \cup \{\perp, (, |, )\}$  définis par le système

$$\frac{}{(e|\perp) \in \mathcal{A}} \text{ (Singl.)} \qquad \frac{a = (e|\ell) \in \mathcal{A} \quad h \in \mathcal{A}}{(e|h::\ell) \in \mathcal{A}} \text{ (Ext)}$$

où  $e$  appartient à  $E$ .

**Exemple 2.66.** — Voici un arbre général : l'arité de chaque sommet est variable.



**Remarque 2.67.** — Un arbre binaire n'est pas cas particulier d'un arbre général. Dans un arbre binaire, un fils est soit un fils droit, soit un fils gauche. Il ne peut pas être fils unique.

**Implémentation 2.68.** — La représentation en C montre qu'on ne gagne pas grand chose en terme d'utilisation de la mémoire par rapport à un arbre binaire. En effet, il est nécessaire de relier tout nœud à une liste d'arbre de longueur quelconque. Un nœud doit donc pointer non seulement vers son « fils aîné » mais aussi sur la suite de la liste d'arbres, autrement dit sur son « frère cadet ».

LISTING 2.8. Arbres généraux en C

```

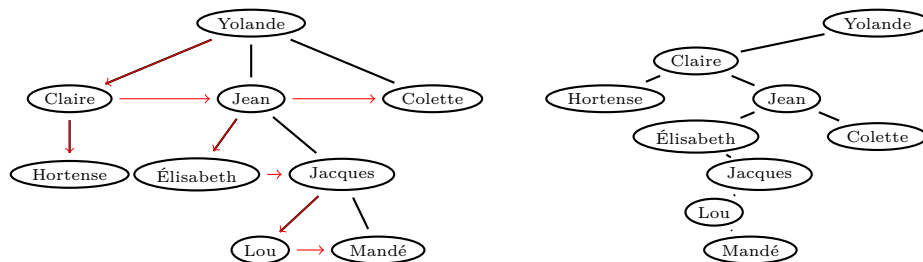
380 typedef struct node node_t;
381 struct node {
382     void *data;
383     node_t *first_child;
384     node_t *next_tree;
385 };

```

**Proposition 2.69.** — Il existe une bijection entre arbres généraux et arbres binaires non vide dont la racine ne possède pas de fils droit.

| Arbres généraux | Arbres binaires |
|-----------------|-----------------|
| Fils aîné       | Fils gauche     |
| Frère cadet     | Fils droit      |

**Exemple 2.70.** — La bijection s'illustre ainsi



**Implémentation 2.71.** — Puisque le langage Python peut gérer des listes de manière native, il est plus facile de manipuler des arbres généraux dans ce langage.

LISTING 2.9. Arbres généraux en Python

```

386 class Node():
387     def __init__(self, d, a = None):
388         self.data = d
389         if a == None:
390             self.children = [] # list of Nodes
391         else:
392             self.children = a
393
394 f1 = Node("Claire", [Node("Hortense")])
395 f2 = Node("Jean", [Node("Élisabeth"),

```

```

396         Node("Jacques", [Node("Lou"),
397                             Node("Mandé"])]))
398 f3 = Node("Colette")
399
400 a = Node("Yolande")
401 a.children.append(f1)
402 a.children.append(f2)
403 a.children.append(f3)

```

**2.1.3.5. Les formules classiques.** — Il est classique de représenter toute sorte de calcul par des arbres. Voici comment.

**Définition 2.72.** — L'ensemble  $\mathcal{E}$  des *expressions arithmétiques dans  $\mathbb{N}$*  est la partie de  $\Sigma^*$ , où  $\Sigma = \mathbb{N} \cup \{ (, -, +, *, ) \}$ , définie inductivement par les règles d'inférence suivante :

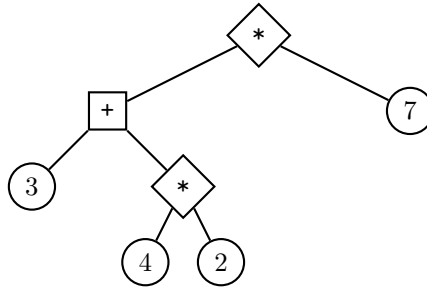
$$\begin{array}{c}
 \frac{}{n \in \mathcal{E}} \text{ (Entier } n) \\
 \frac{y \in \mathcal{E}}{(-y) \in \mathcal{E}} \text{ (Opposé)} \\
 \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y+z) \in \mathcal{E}} \text{ (Somme)} \\
 \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y-z) \in \mathcal{E}} \text{ (Soustraction)} \\
 \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y*z) \in \mathcal{E}} \text{ (Produit)}
 \end{array}$$

où  $n$  appartient à  $\mathbb{N}$ .

**Exemple 2.73.** — L'expression

$$x = ( ( 3 + ( 4 * 2 ) ) * 7 )$$

correspond à un *arbre étiqueté*, expliquant comment les différents types de règles ont été utilisés pour dériver  $x$  depuis les axiomes. Dans cet arbre, les feuilles correspondent à la règle d'introduction des entiers ; les sommets internes correspondent aux règles d'introduction des opérateurs  $-$ ,  $+$  ou  $*$ .



**Implémentation 2.74.** — Pour représenter une formule de logique en Python, nous écrivons un constructeur de classe par règle.

LISTING 2.10. Expressions arithmétiques en Python

```

404 class Neg():
405     def __init__(self, a):
406         self.operand = a

```

```

407 class Opp():
408     def __init__(self, a):
409         self.operand = a
410 class BinaryOperator():
411     def __init__(self, a, b):
412         self.operand1 = a
413         self.operand2 = b
414 class Sum(BinaryOperator):
415     pass
416 class Sub(BinaryOperator):
417     pass
418 class Product(BinaryOperator):
419     pass
420
421 x = Product(Sum(3,Product(4,2)),7)

```

Puisque les codes relatifs à la classe `Sum`, `Sub` et `Product` sont quasiment identiques, nous factorisons le code au sein d'une classe `BinaryOperator` dont héritent `Sum`, `Sub` et `Product`. Concrètement, ceci signifie que la méthode `__init__` de `BinaryOperator` est reprise telle quelle par `Or` et `And` et que nous n'avons pas besoin de l'écrire trois fois.

Fixons un ensemble fini ou dénombrable  $V$  de symboles

$$x_0, x_1, x_2, \dots$$

que nous appelons *variables booléennes*.

**Définition 2.75.** — L'ensemble  $\mathcal{F}$  des *formules de logique propositionnelle* est la partie de  $\Sigma^*$ , où  $\Sigma = V \cup \{\neg, \wedge, \vee, (, )\}$ , définie inductivement par les règles d'inférence suivante :

$$\begin{array}{c}
 \frac{}{x_n \in \mathcal{F}} \text{ (Variable } n) \\
 \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \vee \psi) \in \mathcal{F}} \text{ (Ou)} \\
 \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \wedge \psi) \in \mathcal{F}} \text{ (Et)} \\
 \frac{\varphi \in \mathcal{F}}{\neg \varphi \in \mathcal{F}} \text{ (Negation)}
 \end{array}$$

où  $n$  appartient à  $\mathbb{N}$ .

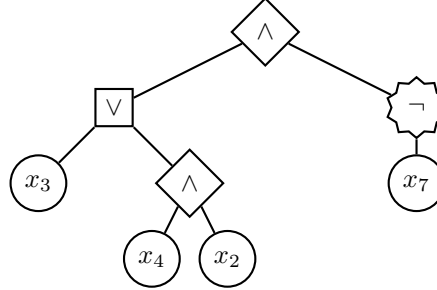
**Remarque 2.76.** — On peut plus largement introduire l'autres opérateurs tels que  $\rightarrow$  ou  $\leftrightarrow$  avec des règles analogues

$$\begin{array}{c}
 \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \rightarrow \psi) \in \mathcal{F}} \text{ (Implication)} \\
 \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \leftrightarrow \psi) \in \mathcal{F}} \text{ (Equivalence)}
 \end{array}$$

**Exemple 2.77.** — L'expression

$$((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$

correspond à un *arbre*, expliquant comment les quatre types de règles ont été utilisés pour dériver  $x$  depuis les axiomes.



**Implémentation 2.78.** — Pour représenter une formule de logique en Python, nous écrivons un constructeur de classe par règle.

LISTING 2.11. Formules de logique en Python

```

422 class Neg():
423     def __init__(self, a):
424         self.operand = a
425 class BinaryOperator():
426     def __init__(self, a, b):
427         self.operand1 = a
428         self.operand2 = b
429 class Or(BinaryOperator):
430     pass
431 class And(BinaryOperator):
432     pass
433
434 y = And(Or(3, And(4, 2)), Neg(7))

```

Puisque les codes relatifs à la classe `Or` et `And` sont quasiment identiques, nous factorisons le code au sein d'une classe `BinaryOperator` dont héritent `Or` et `And`. Concrètement, ceci signifie que la méthode `__init__` de `BinaryOperator` est reprise telle quelle par `Or` et `And` et que nous n'avons pas besoin de l'écrire deux fois.

**2.1.4. La preuve par induction structurelle.** — Décrivons « la » méthode de raisonnement sur les objets définis par induction structurelle.

**Théorème 2.79 (Preuve par induction structurelle)**

Soit  $\mathcal{X}$  une partie d'un ensemble  $E$  définie par un système de règles d'inductions  $S$  et  $\mathcal{P} : E \rightarrow \mathbb{B} = \{V, F\}$  un prédicat portant sur les éléments de  $E$ . Alors les deux assertions sont équivalentes

1. pour tout  $x \in \mathcal{X}$ , on a  $\mathcal{P}(x)$ ,
2. pour toute règle de  $S$  de la forme

$$\frac{x_1 \in \mathcal{X} \quad x_2 \in \mathcal{X} \quad \cdots \quad x_n \in \mathcal{X} \quad (x_1, \dots, x_n) \in \text{Dom}(r)}{r(x_1, \dots, x_n) \in \mathcal{X}} \quad (r)$$

*s'il existe  $x_1, x_2, \dots, x_n$  dans  $E^n$  tels que  $\mathcal{P}(x_1), \mathcal{P}(x_2), \dots, \mathcal{P}(x_n)$  sont tous vérifiés et si  $r(x_1, \dots, x_n)$  est défini, alors  $\mathcal{P}(r(x_1, \dots, x_n))$  est aussi vérifié.*

*Démonstration.* — Nous considérons le sous-ensemble  $Y$  des éléments  $x$  de  $E$  tels que  $\mathcal{P}(x)$  est satisfait. L'ensemble  $Y$  satisfait les conditions de l'équation (2). Mais d'après le théorème (2.35),  $X$  est le plus petit ensemble qui satisfait les conditions de l'équation (2). Donc  $X$  est inclus dans  $Y$ . □

**Remarque 2.80.** — Ce principe généralise le principe de preuve par récurrence. Quand on l'utilise, il convient de distinguer deux cas : les conclusions d'axiomes doivent être prouvés inconditionnellement ; les conclusions des autres règles doivent être prouvés sous l'hypothèse des prémisses.

**Exemple 2.81.** — Soient  $\Sigma = \{a, b\}$  et  $\mathcal{L} \subseteq \Sigma^*$  le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{L}} \text{ (ax)} \qquad \frac{w \in \mathcal{L}}{awb \in \mathcal{L}} \text{ (r}_1\text{)} \qquad \frac{w \in \mathcal{L}}{bwa \in \mathcal{L}} \text{ (r}_2\text{)}$$

et  $\mathcal{P}(w) : \Sigma^* \rightarrow \{V, F\}$  le prédicat « il y a autant de  $a$  et  $b$  dans  $w$  ».

Montrons que  $\mathcal{P}$  est vrai pour tout mot de  $\mathcal{L}$ . Une telle preuve s'écrit comme une preuve par récurrence.

*Démonstration.* — Axiome : Soit  $w$  le mot  $w = \varepsilon$ . Alors on a les égalités  $|w|_a = 0$  et  $|w|_b = 0$ . Mais on peut conclure que  $|w|_a = |w|_b$ . Ainsi  $\mathcal{P}$  est vérifié pour tous les axiomes.

Hérédité pour la règle  $(r_1)$  : Soit un mot  $w$  tel que  $\mathcal{P}(w)$  est vrai, c'est-à-dire tel que l'on ait  $|w|_a = |w|_b$ . Intéressons nous au mot  $w' = awb$ . On a  $|w'|_a = |w|_a + 1$  d'une part et  $|w'|_b = |w|_b + 1$  d'autre part. Mais alors  $|w'|_a = |w'|_b$ . Ainsi  $\mathcal{P}(w')$  est vrai.

Hérédité pour la règle  $(r_2)$  : la justification analogue.

Nous avons vérifié que  $\mathcal{P}$  est vrai pour les axiomes et que  $\mathcal{P}$  se transmet par chacune des règles. D'après le théorème 2.79,  $\mathcal{P}$  est vrai pour tout mot de  $\mathcal{L}$ . □

**Définition 2.82.** — On dit qu'un mot  $w = x_1x_2 \dots x_t$  est *bien parenthésé* si, pour tout préfixe  $u = x_1x_2 \dots x_s$  (avec  $1 \leq s \leq t$ ) de  $w$ , le mot  $u$  contient plus de parenthèses ouvrantes que de parenthèses fermantes.

**Exemple 2.83.** — Soient  $\Sigma = \{<, >, a\}$  et  $\mathcal{L} \subseteq \Sigma^*$  le langage défini inductivement par

$$\frac{}{a \in \mathcal{L}} \text{ (ax)} \qquad \frac{w \in \mathcal{L}}{<w> \in \mathcal{L}} \text{ (r}_1\text{)} \qquad \frac{w_1 \in \mathcal{L} \quad w_2 \in \mathcal{L}}{w_1w_2 \in \mathcal{L}} \text{ (r}_2\text{)}$$

et  $\mathcal{P}(w) : \Sigma^* \rightarrow \{V, F\}$  le prédicat « le mot  $w$  est bien parenthésé », ce qui équivaut plus formellement à « pour tout préfixe  $u$  de  $w$ , on a  $|u|_< \leq |u|_>$  ».

Montrons que  $\mathcal{P}$  est vrai pour tout mot de  $\mathcal{L}$ . Une telle preuve s'écrit comme une preuve par récurrence.



*Démonstration.* — Axiome : Soit  $w$  le mot  $w = \mathbf{a}$ . Le mot  $w$  possède deux préfixes :  $u = \varepsilon$  et  $u = \mathbf{a}$ . Dans les deux cas, nous avons  $|u|_< = |u|_> = 0$

Hérédité pour la règle  $(r_1)$  : Soit un mot  $w$  tel que  $\mathcal{P}(w)$  est vrai, c'est-à-dire tel que l'on ait  $|u|_< \leq |u|_>$  pour tout préfixe  $u$  de  $w$ . Intéressons nous à un préfixe  $u'$  du mot  $w' = \langle w \rangle$ . Si  $u' = \varepsilon$ , on a  $|u'|_< = |u'|_> = 0$ . Si  $u' = <$ , on a  $|u'|_< = 1$  et  $|u'|_> = 0$ . Si  $u'$  est de la forme  $\langle u \rangle$  où  $u$  est un préfixe de  $w$ , on a  $|u'|_< = 1 + |u|_<$  et  $|u'|_> = |u|_>$ . Mais comme,  $|u|_< \leq |u|_>$ , il est clair que  $|u'|_< \leq |u'|_>$ . Si  $u' = \langle w \rangle$ , on a  $|u'|_< = 1 + |w|_<$  et  $|u'|_> = |w|_> + 1$ . Mais comme,  $|w|_< \leq |w|_>$ , il est clair que  $|u'|_< \leq |u'|_>$ . Tous les cas de préfixes sont couverts, nous pouvons conclure que  $\mathcal{P}(w')$  est satisfait.

Hérédité pour la règle  $(r_2)$  : Soient deux mots  $w_1$  et  $w_2$  tels que  $\mathcal{P}(w_1)$  et  $\mathcal{P}(w_2)$  sont vrais. Intéressons nous à un préfixe  $u$  du mot  $w = w_1 w_2$ . Soit  $u$  est un préfixe de  $w_1$  et par hypothèse  $|u|_< \leq |u|_>$ . Soit  $u$  est de la forme  $u = w u'$  où  $u'$  est un préfixe de  $w_2$  et par hypothèse  $|w|_< \leq |w|_>$  et  $|u'|_< \leq |u'|_>$ , mais alors, en sommant  $|u|_< \leq |u|_>$ . Tous les cas de préfixes sont couverts, nous pouvons conclure que  $\mathcal{P}(w)$  est satisfait.

Nous avons vérifié que  $\mathcal{P}$  est vrai pour les axiomes et que  $\mathcal{P}$  se transmet par chacune des règles. D'après le théorème 2.79,  $\mathcal{P}$  est vrai pour tout mot de  $\mathcal{L}$ .  $\square$

**Remarque 2.84.** — La notion de document bien parenthésé est emblématique de tous les langages à balise tels que le HTML ou le XML dans lesquels chaque donnée est encadrée par une balise ouvrante et une balise fermante.

### 2.1.5. Dérivations et fonctions définies par induction structurelle. —

#### 2.1.5.1. Dérivations et arbres de dérivations. —

**Définition 2.85.** — On dit qu'une assertion est *dérivable* dans un système d'inférence s'il s'agit de la conclusion d'un axiome ou bien de la conclusion d'une règle dont les prémisses sont dérivables.

Précisément, l'ensemble des éléments dérivable est l'ensemble

$$\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m.$$

où la suite  $(\mathcal{D}_m)_{m \in \mathbb{N}}$  est définie par  $\mathcal{D}_0$  est formé des éléments appartenant à  $X$  par axiome et

$$\mathcal{D}_{m+1} = \mathcal{D}_m \cup \bigcup_{r: \text{r\`egle}} \{r(x_1, \dots, x_n) \in E; (x_1, \dots, x_n) \in \mathcal{D}_m^n\}.$$

**Exemple 2.86.** — Soient  $\Sigma = \{\mathbf{a}, \mathbf{b}\}$  et  $\mathcal{L} \subseteq \Sigma^*$  le langage défini inductivement par

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (ax)} \qquad \frac{w \in \mathcal{X}}{\mathbf{a}w\mathbf{b} \in \mathcal{X}} \text{ (r}_1\text{)} \qquad \frac{w \in \mathcal{X}}{\mathbf{b}w\mathbf{a} \in \mathcal{X}} \text{ (r}_2\text{)}$$

On a

$$\begin{aligned}
 \mathcal{D}_0 &= \{\varepsilon\} \\
 \mathcal{D}_1 &= \{\varepsilon, ab, ba\} \\
 \mathcal{D}_2 &= \{\varepsilon, ab, ba, aabb, abab, baba, bbaa\} \\
 \mathcal{D}_3 &= \{\varepsilon, ab, ba, aabb, abab, baba, bbaa, \\
 &\quad aaabbb, aababb, ababab, abbaab, \\
 &\quad baabba, bababa, bbabaa, bbbaaa\} \\
 \mathcal{D}_4 &= \dots
 \end{aligned}$$

En fait, les mots dérivables sont les mots de  $\Sigma^*$  avec autant de **a** que de **b**.

Quand on peut dériver un élément  $x$  à partir des axiomes, on présente la dérivation sous forme d'*arbre de dérivation* :

- Les feuilles sont étiquetées par les axiomes ;
- les sommets internes sont étiquetés par des instanciations des règles.
- L'élément  $x$  est dans la conclusion de la racine ;
- les prémisses d'un sommet interne sont les conclusions de ses fils.

**Exemple 2.87.** — La formule logique

$$((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7)$$

définie selon les règles du système de la définition 2.75 admet pour arbre de dérivation

$$\frac{\frac{\frac{}{x_3 \in \mathcal{F}} \text{ (Var } x_3)}{\frac{\frac{\frac{}{x_4 \in \mathcal{F}} \text{ (Var } x_4)}{\frac{\frac{}{x_2 \in \mathcal{F}} \text{ (Var } x_2)}{(x_4 \wedge x_2) \in \mathcal{F}} \text{ (Et)}}}{(x_3 \vee (x_4 \wedge x_2)) \in \mathcal{F}} \text{ (Ou)}}}{((x_3 \vee (x_4 \wedge x_2)) \wedge \neg x_7) \in \mathcal{F}} \text{ (Et)}$$

**Théorème 2.88 (Représentation).** — Soit  $\mathcal{X}$  une partie d'un ensemble  $E$  définie inductivement (cf. théorème 2.35), alors l'ensemble des éléments de  $\mathcal{X}$  coïncident avec l'ensemble  $\mathcal{D}$  des éléments dérivables.

*Démonstration.* — Soit  $\mathcal{P}(x)$  le prédicat  $x \in \mathcal{D}$ . Ce prédicat est vrai pour les axiomes (à cause de  $\mathcal{D}_0$ ) et est stable pour chacune des règles de la définition de  $\mathcal{X}$ . D'après le théorème de preuve par induction structurelle,  $\mathcal{P}(x)$  est vrai pour tout élément de  $\mathcal{X}$ . Autrement dit  $\mathcal{X} \subseteq \mathcal{D}$ .

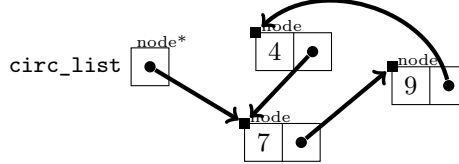
Pour l'inclusion inverse, nous raisonnons par récurrence.

On doit avoir  $\mathcal{D}_0 \subseteq \mathcal{X}$  car il s'agit des axiomes.

Par ailleurs, supposons que  $\mathcal{D}_m \subseteq \mathcal{X}$ . Un élément  $y$  de  $\mathcal{D}_{m+1}$  est soit dans  $\mathcal{D}_m$ , soit de la forme  $r(x_1, \dots, x_n)$  où  $(x_1, \dots, x_n) \in \mathcal{D}_m \subseteq \mathcal{X}$ . Mais comme  $\mathcal{X}$  est stable pour la règle  $r$ ,  $y \in \mathcal{X}$ . Ainsi, dans les deux cas,  $y$  appartient à  $\mathcal{D}_{m+1}$  et on a  $\mathcal{D}_{m+1} \subseteq \mathcal{X}$ . Mais alors, on a aussi  $\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m \subseteq \mathcal{X}$ .  $\square$

**Remarque 2.89.** — Nous sommes rassurés par le théorème de représentation (théorème 2.88) : il prouve que tous nos objets définis par *induction structurelle*, tels que les listes, arbres binaires, formule sont bien *représentables informatiquement* par les constructeurs.

**Remarque 2.90.** — L'inverse n'est pas vrai, nous pouvons même représenter plus de choses : par exemple des listes circulaires.



**Définition 2.91.** — On dit que la définition par induction d'un sous-ensemble  $\mathcal{X}$  est non ambiguë dès lors qu'il n'existe qu'une seule manière de produire chaque élément de  $\mathcal{X}$  (autrement dit, s'il n'existe, pour tout  $x \in \mathcal{X}$ , qu'un seul arbre de dérivation).

**Contre-exemple 2.92.** — On peut définir l'ensemble  $\mathbb{N}^2$  par les trois règles, dont un axiome, suivantes

$$\frac{}{(0,0) \in \mathbb{N}^2} \text{ (ax)} \quad \frac{(m,n) \in \mathbb{N}^2}{(\text{succ}(m),n) \in \mathbb{N}^2} \text{ (r}_1\text{)} \quad \frac{(m,n) \in \mathbb{N}^2}{(m,\text{succ}(n)) \in \mathbb{N}^2} \text{ (r}_1\text{)}$$

Nous observons que l'élément  $(\text{succ}(0), \text{succ}(0))$  est un élément dérivable, qui admet deux arbres de dérivation.

$$\frac{\frac{\frac{}{(0,0) \in \mathbb{N}^2} \text{ (ax)}}{(\text{succ}(0),0) \in \mathbb{N}^2} \text{ (r}_1\text{)}}{(\text{succ}(0),\text{succ}(0)) \in \mathbb{N}^2} \text{ (r}_2\text{)} \quad \frac{\frac{\frac{}{(0,0) \in \mathbb{N}^2} \text{ (ax)}}{(0,\text{succ}(0)) \in \mathbb{N}^2} \text{ (r}_2\text{)}}{(\text{succ}(0),\text{succ}(0)) \in \mathbb{N}^2} \text{ (r}_1\text{)}$$

Par conséquent, la définition de  $\mathbb{N}^2$  que nous avons donnée est ambiguë.

**Remarque 2.93.** — Le caractère ambigu porte sur la manière de définir l'ensemble, pas sur l'ensemble lui-même

**2.1.5.2. Constructions inductives de fonctions.** — Le théorème suivant permet de définir sans ambiguïté une fonction sur un ensemble défini par induction, simplement en considérant les cas d'induction : autrement dit, on a bien le droit de programmer une fonction récursivement.

**Théorème 2.94.** — Soient  $E$  et  $F$  deux ensembles et  $\mathcal{X}$  un sous-ensemble de  $E$  défini par une définition inductive  $D$  non ambiguë. Soit, pour chacune des règles  $r : E^{n_r} \rightarrow E$  de la définition  $D$ , une application  $g_r : E^{n_r} \times F^{n_r} \rightarrow F$ . Alors il existe une unique application  $f : \mathcal{X} \rightarrow F$  telle que, pour toute règle  $r$ ,

$$\forall (x_1, \dots, x_n) \in \mathcal{X}^{n_r} \cap \text{Dom}(r),$$

$$f(r(x_1, \dots, x_{n_r})) = g_r(x_1, \dots, x_{n_r}, f(x_1), \dots, f(x_{n_r}))$$

**Remarque 2.95.** — Dans la théorème précédent, lorsque l'entier  $n_r$  est nul, il faut comprendre  $E^0 \times F^0$  comme l'ensemble des couples de tuples de longueur nulle. Or, il n'y a qu'un seul tel tuple, à savoir le tuple  $()$ . Par conséquent, une fonction  $g_r : E^0 \times F^0 \rightarrow F$  est en vérité une fonction constante, ne possédant qu'une seule association

$(( ), ( )) \mapsto g$  où  $g$  est un certain élément de  $F$ . Ainsi, quand  $n_r$  est nul et  $x$  est la conclusion d'un axiome, nous disons simplement que  $f(x) = g$ .

*Démonstration.* — Soient  $f_1 : \mathcal{X} \rightarrow F$  et  $f_2 : \mathcal{X} \rightarrow F$  deux applications répondant au théorème. Nous introduisons le prédicat  $\mathcal{P}(x) : \ll f_1(x) = f_2(x) \gg$ . Soit  $r$  une règle, donnons-nous  $(x_1, \dots, x_{n_r}) \in \mathcal{X}^{n_r} \cap \text{Dom}(r)$ . Supposons  $\mathcal{P}(x_1), \dots, \mathcal{P}(x_{n_r})$ . Alors

$$\begin{aligned} f_1(r(x_1, \dots, x_{n_r})) &= g_r(x_1, \dots, x_{n_r}, f_1(x_1), \dots, f_1(x_{n_r})) \\ &= g_r(x_1, \dots, x_{n_r}, f_2(x_1), \dots, f_2(x_{n_r})) \\ &= f_2(r(x_1, \dots, x_{n_r})) \end{aligned}$$

Mais alors on a  $\mathcal{P}(r(x_1, \dots, x_{n_r}))$ . D'après le théorème de preuve par induction structurale  $\mathcal{P}(x)$  est toujours vrai sur  $\mathcal{X}$ . Ainsi les applications  $f_1$  et  $f_2$  sont égales.

D'après le théorème de représentation (théorème 2.88), tout élément de  $x$  est dérivable. De plus, l'ensemble des éléments dérivable est l'ensemble

$$\mathcal{D} = \bigcup_{m \in \mathbb{N}} \mathcal{D}_m.$$

où  $\mathcal{D}_0$  est formé des éléments des axiomes et

$$\mathcal{D}_{m+1} = \mathcal{D}_m \cup \bigcup_{r: \text{r\`egle}} \{r(x_1, \dots, x_n) \in E; (x_1, \dots, x_n) \in \mathcal{D}_m^n\}.$$

Nous commençons par définir  $f(x)$  pour  $x \in \mathcal{D}$  en utilisant les constantes  $g_r$  pour chaque axiome ( $r$ ). Puis par récurrence sur  $m$ , nous étendons la définition de  $f$  de  $\mathcal{D}_m$  à  $\mathcal{D}_{m+1}$  en utilisant les fonctions  $g_r$  pour les autres règles  $r$ . Nous savons déjà qu'il n'y a au plus une manière de définir  $f(x)$ . Aussi, ce procédé se déroule sans problème de définition.  $\square$

**Exemple 2.96.** — La définition inductive des entiers  $\mathbb{N}$  donnée par

$$\frac{}{0 \in \mathbb{N}} \text{ (zéro)} \qquad \frac{n \in \mathbb{N}}{n+1 \in \mathbb{N}} \text{ (succ.)}$$

est non ambiguë.

En conséquence, il est licite de définir l'application *factorielle* par les deux règles suivantes :

1.  $0! = 1$
2.  $(n+1)! = (n+1) \cdot n! = 1$

soit encore

$$\frac{}{0! = 1} \text{ (zéro)} \qquad \frac{n \in \mathbb{N}}{(n+1)! = (n+1) \times (n!)} \text{ (succ.)}$$

LISTING 2.12. Factorielle

```

435 def fact(n):
436     if n==0:
437         return 1
438     else:
439         return n * fact(n-1)

```

**Exemple 2.97.** — La définition inductive des couples d'entiers  $\mathbb{N}^2$  donnée par le contre exemple 2.92 est ambiguë.

Ceci correspond au fait bien connu que les programmes **f1** et **f2** suivants

```

440 def f1(x,y):
441     if (x,y) == (0,0):
442         return 1
443     if x > 0:
444         return(f1(x-1,y)+1)
445     if y > 0:
446         return(2*f1(x,y-1))
447
448 def f2(x,y):
449     if (x,y) == (0,0):
450         return 1
451     if y > 0:
452         return(2*f2(x,y-1))
453     if x > 0:
454         return(f2(x-1,y)+1)

```

ne calculent pas le même résultat alors que le code semble indiquer les mêmes règles. La fonction  $f_1$  calcule  $f_1(x, y) = x + 2^y$  ; la fonction  $f_2$  calcule  $f_2(x, y) = (x + 1) \cdot 2^y$ . À l'exécution, le flot de contrôle est déterminé par l'ordre du code : la première branche dont le test est satisfait est exécutée.

**Exemple 2.98.** — La définition inductive des listes  $\mathcal{L}$  à valeur dans  $E$  donnée par

$$\frac{}{\perp \in \mathcal{L}} \text{ (Liste vide)} \qquad \frac{q \in \mathcal{L}}{(e :: q) \in \mathcal{L}} \text{ (Liste de tête } e)$$

est non ambiguë.

Il est licite de définir l'application *longueur de liste* par

$$\frac{}{\text{Long}(\perp) = 0} \qquad \frac{q \in \mathcal{L}}{\text{Long}(e :: q) = \text{Long}(q) + 1}$$

LISTING 2.13. Longueur d'une liste

```

455 class Node:
456     def __init__(self, dt, nxt = None):
457         self.data = dt
458         self.next = nxt
459
460 def length(l):
461     if l is None:
462         return(0)
463     else:
464         return(1+length(l.next))

```

Soit  $\ell_2$  une liste que l'on suppose fixée, il est possible de définir la fonction *concaténation avec la liste  $\ell_2$*  par

$$\frac{}{\text{concat}_{\ell_2}(\perp) = \ell_2} \qquad \frac{q \in \mathcal{L}}{\text{concat}_{\ell_2}(e :: q) = (e :: \text{concat}_{\ell_2}(q))}$$

LISTING 2.14. Concaténation de listes

```

465 def concat(l1, l2):
466     if l1 is None:
467         return(l2)
468     else:
469         return(Node(l1.data, concat(l1.next, l2)))

```

**Remarque 2.99.** — L'exemple 2.98 de la concaténation est très illustratif d'un fait. Lorsqu'on définit une application sur des objets définis par induction structurée, l'application à définir ne peut porter que sur un seul argument. En effet, nous ne sommes capables de déstructurer qu'un seul élément à la fois. Naturellement, nous serions tentés de définir la concaténation comme une application  $(\mathcal{L} \times \mathcal{L}) \rightarrow \mathcal{L}$ . Or nous ne connaissons pas de définition inductive de  $\mathcal{L} \times \mathcal{L}$ . Ainsi, nous avons d'abord défini l'application partielle

$$\text{concat}_{\ell_2} : \begin{cases} \mathcal{L} & \rightarrow \mathcal{L} \\ \ell_1 & \mapsto \text{concat}_{\ell_2}(\ell_1). \end{cases}$$

à partir de laquelle nous définissons la concaténation comme

$$\text{concat} : \begin{cases} \mathcal{L} & \rightarrow (\mathcal{L} \rightarrow \mathcal{L}) \\ \ell_2 & \mapsto \text{concat}_{\ell_2}. \end{cases}$$

Cette transformation s'appelle la *curryfication*.

**Exemple 2.100.** — La définition inductive des arbres binaires donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(g|d) \in \mathcal{A}} \text{ (cons.)}$$

est non ambiguë.

Il est licite de définir l'application *hauteur* par

$$\frac{}{\text{haut}(\perp) = -1} \text{ (vide)} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\text{haut}(e|g|d) = 1 + \max_{\mathbb{Z}}(\text{haut}(g), \text{haut}(d))} \text{ (cons.)}$$

LISTING 2.15. Hauteur d'un arbre

```

470 class Node():
471     def __init__(self, d, a = None, b = None):
472         self.data = d
473         self.left = a
474         self.right = b
475
476 def height(a):

```

```

477     if a is None:
478         return(-1)
479     else:
480         return(1 + max(height(a.left), height(a.right)))

```

**Remarque 2.101.** — La profondeur d'un sommet, qui est le nombre d'ancêtres stricts, ne peut pas être définie inductivement (la profondeur d'une feuille n'est pas constante).

**Exemple 2.102.** — La définition inductive des arbres binaire étiquetés dans  $E$  donnée par

$$\frac{}{\perp \in \mathcal{A}} \text{ (vide)} \quad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}} \text{ (Arbre de racine } e \in E)$$

est non ambiguë. On suppose que  $E$  possède un ordre total.

Il est licite de définir l'application *maximum* par

$$\frac{\frac{}{\max_{\mathcal{A}}(\perp) = -\infty} \text{ (vide)} \quad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{\max_{\mathcal{A}}((e|g|d)) = \max_E(e, \max_{\mathcal{A}}(g), \max_{\mathcal{A}}(d))} \text{ (Arbre de racine } e \in E)$$

LISTING 2.16. Maximum des étiquettes d'un arbre

```

481 class Node():
482     def __init__(self, d, a = None, b = None):
483         self.data = d
484         self.left = a
485         self.right = b
486
487 def tree_max(a):
488     if a is None:
489         return(-sys.maxsize)
490     else:
491         return(max(a.data, tree_max(a.left), tree_max(a.right)))

```

**Exemple 2.103.** — La définition inductive des expressions arithmétiques donnée par

$$\frac{}{n \in \mathcal{E}} \text{ (Entier } n) \quad \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y+z) \in \mathcal{E}} \text{ (Somme)} \quad \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{(y*z) \in \mathcal{E}} \text{ (Produit)}$$

est non ambiguë (grâce au parenthésage).

Il est licite de définir l'application *évaluation* par

$$\frac{}{\text{eval}(n) = n} \text{ (Entier } n) \quad \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{\text{eval}((y+z)) = \text{eval}(y) +_{\mathbb{N}} \text{eval}(z)} \quad \frac{y \in \mathcal{E} \quad z \in \mathcal{E}}{\text{eval}((y*z)) = \text{eval}(y) \times_{\mathbb{N}} \text{eval}(z)}$$

Dans le code ci-dessous, nous nous appuyons sur le fait que des constructeurs distincts ont été introduits pour chaque opération pour la reconnaître.

LISTING 2.17. Évaluation d'une expression arithmétique

```

492 class BinaryOperator():
493     def __init__(self, a, b):
494         self.operand1 = a
495         self.operand2 = b
496 class Sum(BinaryOperator):
497     pass
498 class Product(BinaryOperator):
499     pass
500
501 def eval(x):
502     match x:
503         case int() | float():
504             return x
505         case Sum():
506             return(eval(x.operand1) + eval(x.operand2))
507         case Product():
508             return(eval(x.operand1) * eval(x.operand2))

```

**Définition 2.104.** — L'ensemble des *valeurs de vérité* est un ensemble à deux éléments que l'on note

$$\mathbb{B} = \{\text{Vrai}, \text{Faux}\} = \{\text{True}, \text{False}\} = \{V, F\}.$$

**Définition 2.105.** — Une *valuation* de l'ensemble des variables booléennes  $\mathcal{V} = \{x_1, x_2, \dots\}$  est une distribution des valeurs de vérité sur l'ensemble des variables propositionnelles  $\mathcal{V}$ , autrement dit une fonction

$$\alpha : \mathcal{V} \rightarrow \mathbb{B}.$$

Nous définissons exhaustivement trois fonctions booléennes  $f_{\neg} : \mathbb{B} \rightarrow \mathbb{B}$ ,  $f_{\vee} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ ,  $f_{\wedge} : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$  par les tables :

| $x$ | $f_{\neg}(x)$ | $x$ | $y$ | $f_{\vee}(x, y)$ | $x$ | $y$ | $f_{\wedge}(x, y)$ |
|-----|---------------|-----|-----|------------------|-----|-----|--------------------|
| V   | F             | V   | V   | V                | V   | V   | V                  |
| F   | V             | V   | F   | V                | V   | F   | F                  |
|     |               | F   | V   | V                | F   | V   | F                  |
|     |               | F   | F   | F                | F   | F   | F                  |

L'évaluation d'une formule logique  $\varphi$  d'après la valuation  $\alpha$  est la valeur de vérité, notée  $\llbracket \varphi \rrbracket_{\alpha}$ , définie inductivement par



$$\begin{array}{c}
\frac{}{\llbracket x_n \rrbracket_\alpha = \alpha(x_n)} \text{ (Variable } n) \qquad \frac{\varphi \in \mathcal{F}}{\llbracket \neg \varphi \rrbracket_\alpha = f_\neg(\llbracket \varphi \rrbracket_\alpha)} \text{ (Negation)} \\
\frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\llbracket \varphi \vee \psi \rrbracket_\alpha = f_\vee(\llbracket \varphi \rrbracket_\alpha, \llbracket \psi \rrbracket_\alpha)} \text{ (Ou)} \qquad \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\llbracket \varphi \wedge \psi \rrbracket_\alpha = f_\wedge(\llbracket \varphi \rrbracket_\alpha, \llbracket \psi \rrbracket_\alpha)} \text{ (Et)}
\end{array}$$

LISTING 2.18. Évaluation d'une formule de logique

```

509 def eval(x, alpha):
510     match x:
511         case int():
512             return alpha[x]
513         case Neg():
514             return not eval(x.operand, alpha)
515         case Or():
516             return (eval(x.operand1, alpha) or eval(x.operand2,
517                 ↪ alpha))
518         case And():
519             return (eval(x.operand1, alpha) and eval(x.operand2,
520                 ↪ alpha))

```

**2.1.5.3. Parcours d'arbre.** — Nous pouvons aussi définir récursivement comment explorer un arbre.

**Définition 2.106.** — On appelle *parcours d'arbre* tout algorithme qui « visite » tous les sommets de l'arbre et fait subir un traitement à chaque sommet rencontré.

**Remarque 2.107.** — Derrière le terme « parcours d'arbre » se cache l'idée qu'on ne dispose pas de l'ensemble des sommets, mais qu'on ne connaît (initialement) que la racine de l'arbre. En conséquence, un sommet quelconque ne peut être découvert que au moment où l'on s'intéresse à son père.

**Définitions 2.108.** — Trois parcours peuvent se définir facilement par induction structurelle :

- On appelle *parcours en ordre préfixe* le parcours qui consiste à
  - examiner la racine (et lui faire subir un traitement)
  - parcourir le sous-arbre gauche en ordre préfixe
  - parcourir le sous-arbre droit en ordre préfixe
- On appelle *parcours ordre infixe* le parcours qui consiste à
  - parcourir le sous-arbre gauche en ordre infixe
  - examiner la racine (et lui faire subir un traitement)
  - parcourir le sous-arbre droit en ordre infixe
- On appelle *parcours ordre postfixe* le parcours qui consiste à
  - parcourir le sous-arbre gauche en ordre postfixe
  - parcourir le sous-arbre droit en ordre postfixe
  - examiner la racine (et lui faire subir un traitement)

**Remarque 2.109.** — On parle aussi de parcours préordre pour désigner le parcours préfixe ; on parle aussi de parcours dans l'ordre symétrique pour désigner le parcours infixe ; enfin, on parle aussi de parcours en postordre ou de parcours en ordre suffixe pour désigner le parcours ordre postfixe.

Voici une implémentation du parcours en ordre préfixe, dans laquelle le traitement de chaque sommet consiste simplement à afficher son étiquette sur la sortie standard.

LISTING 2.19. Parcours en ordre préfixe

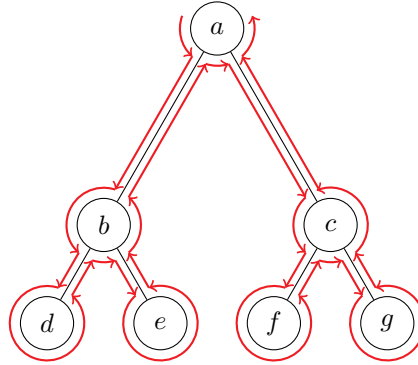
```
519 class Node():
520     def __init__(self, d, a = None, b = None):
521         self.data = d
522         self.left = a
523         self.right = b
524
525 def prefix_traversal(a, visit):
526     if isinstance(a, Node):
527         visit(a)
528         prefix_traversal(a.left, visit)
529         prefix_traversal(a.right, visit)
530
531 def root_print(a):
532     print(a.data)
533
534 prefix_traversal(arbre, root_print)
```

**Exemple 2.110.** — Avec l'arbre binaire de la figure 2.1.

- Le parcours en ordre préfixe est Abigael, Brahim, Djibril, Esther, Imene, Celya, Ferenc, Greta, Jakob, Marwa, Katel.
- Le parcours en ordre infixe est Djibril, Brahim, Esther, Imene, Abigael, Ferenc, Celya, Jakob, Marwa, Greta, Katel.
- Le parcours en ordre suffixe est Djibril, Imene, Esther, Brahim, Ferenc, Marwa, Jakob, Katel, Greta, Celya, Abigael.

**Remarque 2.111.** — On peut voir les ordres obtenus par les parcours en préordre et en postordre sur l'ensemble des noeuds d'une autre façon en considérant ce qu'on appelle « un parcours à main gauche » de l'arbre. On part de la racine et on suit le tracé de l'arbre comme l'indique la figure ci-dessous. On s'aperçoit que l'ordre préfixe sur les sommets correspond à l'examen des noeuds lors de la première rencontre des sommets ; l'ordre infixe correspond au passage à la verticale sous les sommets ; l'ordre postfixe correspond à la dernière rencontre avec un sommet.

On peut définir de manière analogue la notion de parcours en ordre préfixe ou de parcours en ordre postfixe d'arbres généraux.



## 2.2. Type de donnée abstrait

En informatique, on parle de *barrière d'abstraction* pour désigner le fait que l'on doit pouvoir manipuler des abstractions de haut niveau sans se soucier de leurs détails d'implémentation (voire même sans les connaître!).

De manière générale, on dit qu'un code est *modulaire* s'il est composé d'éléments les plus indépendants possibles les uns des autres, et qui fonctionnent ensemble simplement. Quand les modules sont bien indépendants, il est possible de les développer et les tester indépendamment et de les réutiliser simplement dans d'autres projets.

Dans cette section, nous étudions comme ces idées peuvent se traduire au niveau de la représentation et de la manipulation de données.

**2.2.1. Types abstraits.** — Commençons par l'idée mathématique et pure d'une donnée.

**Définition 2.112.** — Un *type abstrait de donnée* est un modèle mathématique qui décrit des données ainsi qu'un ensemble d'opérations que l'on peut effectuer sur elles.

On peut voir un type abstrait comme un contrat qui spécifie un comportement du point de vue de l'utilisateur notamment en termes de valeurs possibles, d'opérations possibles sur les données de ce type et de comportement de ces opérations.

De manière importante, un type abstrait est indépendant de son implémentation concrète. Plusieurs implémentations d'un même type abstrait sont possibles. Selon les possibilités offertes par le langage ou paradigmes de programmation, les mécanismes d'écriture de l'abstraction peuvent prendre des formes variées.

**Exemple 2.113.** — Le type abstrait `Rectangle` est la donnée de deux réels positifs, `length` et `width`, ainsi que deux fonctions :

- `create`, de type `Int → Int → Rectangle`, dont les paramètres d'entrée sont une longueur et une largeur et la valeur de retour est un rectangle
- `area`, de type `Rectangle → Int`, dont la valeur de retour est l'aire du rectangle

**Remarque 2.114.** — Toutes les notions de complexité (en temps ou en espace) sont attachées aux détails d’implémentations, c’est-à-dire aux structures de données et non pas aux types abstraits.

**2.2.2. Structure de données.** — Envisageons les implémentations concrètes.

**Définition 2.115.** — Une *structure de donnée* est une construction explicite qui met en œuvre un type abstrait à partir des possibilités offertes par le langage.

La plupart des langages de programmation permettent d’attribuer et d’utiliser la mémoire disponible de différentes façons :

- sous la forme d’une variable isolée ; une variable est en général d’un certain type prédéfini par le langage (le type entier, le type flottant, le type caractère, etc.)
- sous la forme de tableaux statique, c’est-à-dire d’une suite de variables de même type associées à des « cases » consécutives dans la mémoire ; cette consécutivité permet à l’ordinateur de savoir où sont rangés les éléments du tableau à partir du premier, ce qui permet d’avoir accès directement (c’est-à-dire en  $O(1)$ ) à une variable du tableau à partir d’un indice donné ;
- à l’aide de pointeurs, indiquant la localisation dans la mémoire de l’objet auquel on s’intéresse.

De plus, un mécanisme permet de construire des données agrégées qui combinent une quantité finie de données de types déjà définis.

En langage C, il s’agit de constructions introduites par le mot clé **struct** que l’on combine en général avec le mot clé **typedef** pour définir la structure une fois pour toute.

**Exemple 2.116.** — En langage C, nous pourrions implémenter le type abstrait rectangle (voir exemple 2.113) par les instructions suivantes : Le fichier d’en-tête vient déterminer comment est constitué notre structure de donnée et quelles fonctions s’y rapportent.

LISTING 2.20. Fichier .h codant un type abstrait *Rectangle*

```

535 #fichier.h
536 typedef struct rect rect_t;
537 struct rect {
538     int length;
539     int width;
540 };
541
542 rect_t *create(int l, int w)
543
544 int area(rect_t *r)

```

Le fichier source vient définir les fonctions.

LISTING 2.21. Fichier .c codant un type abstrait *Rectangle*

```

545 #fichier.c
546
547 rect_t *create(int l, int w) {
548     rect_t *new_rect = malloc(sizeof(rect_t));
549     new_rect->length = l;
550     new_rect->width = w;
551     return new_rect;
552 }
553
554 int area(rect_t *r) {
555     return (r->length * r->width);
556 }

```

Dans des langages incluant le paradigme de *programmation orientée objet*, il est possible également définir des *objets* (voir aussi définition A.1) qui sont des briques de base qui encapsulent l'ensemble des données et qui définissent simultanément l'ensemble des fonctions pouvant s'y rattacher. L'usage est de parler alors :

- des *attributs*, pour désigner les données qui précisent les caractéristiques de l'objet.
- des *méthodes*, pour désigner les fonctions qui permettent d'interagir avec l'objet et qui en précisent le comportement.

À cet égard, les classes constituent une très bonne abstraction pour implémenter des types abstraits.

**Exemple 2.117.** — En langage Python et en adoptant un point de vue de programmation orientée objet, nous pourrions implémenter le type abstrait rectangle (voir exemple 2.113) par les instructions suivantes :

LISTING 2.22. Classe Rectangle en Python

```

557 class Rectangle():
558     def __init__(self, a, b):
559         self.length = a
560         self.width = b
561
562     def area(self):
563         return self.length * self.width
564
565 r = Rectangle(3,4)
566 r.area()

```

### 2.2.3. Structures mutables et structures persistentes. —

**Définition 2.118.** — On dit qu'une structure de donnée est *mutable* si son contenu est modifiable au cours du temps.

**Exemple 2.119.** — Parmi les types de données intégrés (*built-in data types*) du langage Python qui sont mutables, on peut citer les listes (type `list`), les ensembles (type `set`) ou les dictionnaires (type `dict`).

**Définition 2.120.** — On dit qu'une structure de donnée est *immuable* s'il n'est pas autorisé de modifier la structure après sa création.

**Exemple 2.121.** — Parmi les types de données intégrés (*built-in data types*) du langage Python qui sont immuables, on peut citer les entiers (type `int`), les booléens (type `bool`), les chaînes de caractères (type `string`) ou les tuples (type `tuple`).

En général, dire que les chaînes de caractères ou les tuples sont immuables est bien compris, puisque le programmeur se rend bien compte que l'accès `s[i]`, où `s` est de type `string` ou `tuple` est accepté par le langage, tandis que la réécriture `s[i] = 'a'` est refusée.

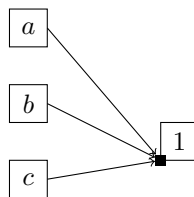
Dire que les entiers sont immuables peut paraître plus déroutant puisqu'il semble possible d'écrire successivement `a = 1` puis `a = 2`. En vérité, il faut comprendre que en Python, la notion de variable n'a pas de sens. Il y a d'une part des noms et d'autre part des objets. Ainsi, dans l'instruction `a = 1`, le nom `a` est initialement associé à l'objet 1 puis l'association est redéfinie. Il est à noter d'ailleurs que l'objet 1 n'est créé qu'une seule fois. L'exemple

```

567 a = 1
568 b = 1
569 c = 1
570 assert(id(b) == id(a)) # a, b, c sont associés à un unique objet
571 assert(id(c) == id(a)) # dont l'identifiant est le toujours même

```

correspond à une organisation



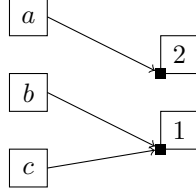
Si l'on exécute ensuite

```

572 a = 2
573 assert(id(b) != id(a)) # b, c restent associées au même identifiant
    ↪ qu'avant
574 assert(id(b) == id(c)) # mais a est associé à un nouvel objet

```

correspond à une organisation



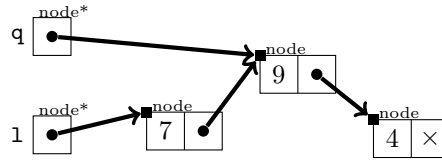
**Remarque 2.122.** — Le choix de manipuler exclusivement des données immuables est caractéristique du paradigme de *programmation fonctionnelle*, qui est un type de programmation très proche des mathématiques : on déclare des variables une fois pour toute et les calculs sont considérés surtout en tant que résultat de l'évaluation d'une fonction, si bien qu'il n'y a pas lieu de faire évoluer la valeur d'une variable.

**Définition 2.123.** — On dit qu'une structure de données est *persistante* si elle préserve ses versions antérieures lorsqu'elle est modifiée ; une telle structure est *im-muable*.

**Exemple 2.124.** — Lorsqu'une liste chaînée est immuable, il s'agit même d'une structure persistante. En effet, lorsqu'on ajoute un élément en tête d'une liste existante  $q$  pour faire une nouvelle liste  $\ell$ , il n'est pas nécessaire de recopier  $q$ . Les listes  $h$  et  $\ell$  peuvent cohabiter en mémoire.

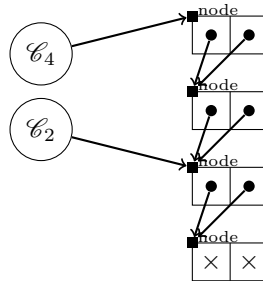
$$h = (9 :: (4 :: \perp))$$

$$\ell = (7 :: (9 :: (4 :: \perp)))$$



Dans certains cas, manipuler des structures persistantes peut même conduire à des économies de mémoire. À partir du moment où un objet est immuable, il n'est pas nécessaire de le recréer à chaque fois qu'on en a besoin.

**Exemple 2.125.** — Les arbres binaires complets non étiquetés (voir définition 2.57 et exemple 2.58) peuvent se représenter avec économie comme suit :



Le lecteur curieux de comprendre comment détecter la possibilité d'effectuer du partage en mémoire entre structures persistantes pourra démarrer ses recherches à partir du mot clé *hash consing*.

### 2.3. Types abstraits usuels et implémentations classiques

Les langages de programmation modernes offrent pour la plupart des bibliothèques dans lesquelles sont implémentées les structures de données les plus courantes. Une difficulté survient toutefois car le vocabulaire n'est pas toujours stabilisé ou encore quand il faut naviguer entre le nom de la structure de données vs le nom du type abstrait.

#### 2.3.1. Les types abstraits linéaires. —

**2.3.1.1. Les piles.** — Une pile est une collection qui permet de stocker et restituer des objets selon une politique « dernier arrivé, premier servi ».

A minima, les opérations souhaitées sont les suivantes. Nous commençons par en donner les signatures :

- $\text{PileVide} : () \rightarrow \text{Pile}$
- $\text{EstVide} : \text{Pile} \rightarrow \text{Booléen}$
- $\text{Empiler} : \text{Pile}, \text{Element} \rightarrow \text{Pile}$
- $\text{Depiler} : \text{Pile} \rightarrow \text{Pile}, \text{Element}$

Ces quatre opérations obéissent aux spécifications suivantes :

- La fonction  $\text{EstVide}$  doit satisfaire à

$$\text{EstVide}(p) = \begin{cases} \text{Vrai} & \text{si } p = \text{PileVide}() \\ \text{Faux} & \text{si } p \text{ est de la forme } \text{Empiler}(p', x) \end{cases}$$

- La fonction  $\text{Depiler}$  n'est valide que si  $\text{EstVide}(p)$  vaut Faux et dans ce cas, on doit avoir

$$\text{Depiler}(p) = (p', x) \text{ où } p \text{ est de la forme } \text{Empiler}(p', x)$$

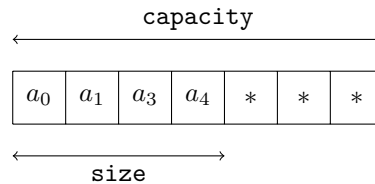
**Remarque 2.126.** — Vouloir définir à tout prix quelles primitives relèvent ou non du type abstrait  $\text{Pile}$  n'a pas vraiment de sens. Chaque programmeur peut avoir ses besoins particuliers, chaque langage (au travers de ses bibliothèques) peut avoir sa propre opinion. Toutefois, l'idée généralement partagée est qu'on ne va pas chercher un élément au milieu de la pile et qu'on se contente de celui du dessus.

**Application 2.127.** — Les piles sont essentielles au bon fonctionnement de l'informatique mais sont souvent cachées. Elles servent notamment aux compilateurs, à gérer les piles d'adresse de retour (appels récursifs de fonctions), à gérer les variables locales.

Elles peuvent aussi servir à l'évaluation d'expressions arithmétiques ou pour les parcours de graphe en profondeur.



**2.3.1.1.1. Implémentations par un tableau statique.** — Si la capacité de la pile est bornée, on peut réaliser directement une pile à travers un tableau statique. Les éléments sont placés au début du tableau. Il est nécessaire de maintenir la taille de la pile pour savoir quelles cases du tableau sont utilisées.



Le détail du code est donnée en C.

LISTING 2.23. Structure de pile en C

```

575 typedef struct mystack mystack_t;
576 struct mystack {
577     int capacity;
578     int size;      // 0 <= size <= capacity
579     int *data;     // tableau de taille capacity
580 };
581
582 mystack_t *empty_stack(int c) {
583     mystack_t *s = malloc(sizeof(mystack_t));
584     s->capacity = c;
585     s->size = 0;
586     s->data = calloc(c, sizeof(int));
587     return s;
588 }
589
590 bool is_empty(mystack_t *s) {
591     return (s->size == 0);
592 }
593
594 void push(mystack_t *s, int x) {
595     if (s->size < s->capacity) {
596         s->data[s->size] = x;
597         s->size ++;
598     }
599 }
600
601 int pop(mystack_t *s) {
602     int x = 0;
603     if (s->size > 0) {
604         s->size --;
605         x = s->data[s->size];

```

```

606     }
607     return x;
608 }
```

En Python, on peut utiliser simplement les méthodes `append` et `pop` avec une liste.

**2.3.1.1.2. Implémentations par une liste chaînée.** — Les listes chaînées, introduite en 2.1.3.1, conviennent parfaitement pour implémenter des piles.

**Remarque 2.128.** — L'implémentation d'une pile par un tableau a un inconvénient : il est nécessaire de fixer une capacité à priori et de deviner quelle taille sera nécessaire pour ne pas commettre de débordement. Cependant, elle est très facile à mettre en œuvre et ne demande pas de mécanismes sophistiqués tels que des pointeurs.

**2.3.1.2. Les files.** — Une file est une collection qui permet de stocker et restituer des objets selon une politique « premier arrivé, premier servi ».

A minima, les opérations souhaitées sont les suivantes. Nous commençons par en donner les signatures :

- `FileVide` :  $() \rightarrow \text{File}$
- `EstVide` :  $\text{File} \rightarrow \text{Booléen}$
- `Enfiler` :  $\text{File}, \text{Element} \rightarrow \text{File}$
- `Defiler` :  $\text{File} \rightarrow \text{File}, \text{Element}$

Ces quatre opérations obéissent aux spécifications suivantes :

- La fonction `EstVide` doit satisfaire à

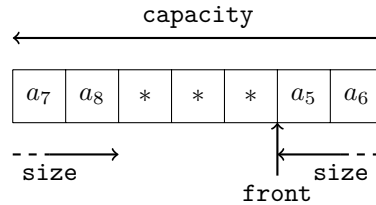
$$\text{EstVide}(f) = \begin{cases} \text{Vrai} & \text{si } f = \text{FileVide}() \\ \text{Faux} & \text{si } f \text{ est de la forme } \text{Enfiler}(f', x) \end{cases}$$

- La fonction `Defiler` n'est valide que si `EstVide(f)` vaut Faux et dans ce cas, on doit avoir selon les cas :

$$\text{Defiler}(\text{Enfiler}(f, x)) = \begin{cases} \text{FileVide}(), x & \text{si } f = \text{FileVide}() \\ \text{Enfiler}(g, x), y & \text{sinon, où } (g, y) = \text{Defiler}(f). \end{cases}$$

**Application 2.129.** — Les files interviennent dès qu'il y a un accès limité à des ressources : routeurs, gestion d'impressions, tampons (flux de données à traiter). Elles sont aussi utiles pour définir le parcours en largeur dans un graphe.

Comme les piles, les files peuvent s'implémenter avec un grand tableau. Il faut cette fois-ci gérer deux indices : `front` et `size`. Les éléments dans la pile sont stockés du plus ancien au plus récent entre l'indice `front` inclus et l'indice `front+size` exclu. Il est nécessaire de compter les indices modulo `capacity` pour que la file puisse durer plus longtemps.



(Dans l'illustration, le premier élément à être défilé est  $a_5$ ). Voici un début de code

LISTING 2.24. Structure de file en C

```

609 typedef struct mystack mystack_t;
610 struct mystack {
611     int capacity;
612     int front;
613     int size;
614     int *data;
615 };

```

On peut également implémenter des files par des listes (simplement) chaînées

**2.3.1.3. Les listes.** — Mise en garde. Le terme *liste* est un terme hautement ambigu en informatique. Dans quasiment tous les langages de programmation, il existe une structure de *list* qui désigne presque systématiquement une autre réalité du point de vu de l'algorithmique.

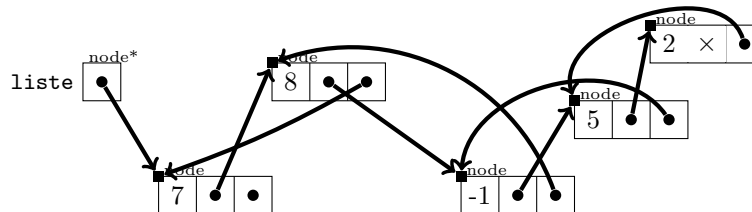
Nous considérerons la définition suivante : Une *liste* est un type abstrait qui permet de ranger les éléments séquentiellement et qui permet d'accéder librement à tous les éléments. Il offre les opérations suivante :

- ListeVide :  $() \rightarrow \text{Lile}$ . Permet de créer une liste vide.
- EstVide :  $() \rightarrow \text{Booléen}$ . Teste si la liste est la liste vide.
- Insérer : Liste, Element  $\rightarrow$  Liste. Ajoute un élément à la liste (quelque part)
- Supprimer : Liste  $\rightarrow$  Liste, Element. Extraie un élément (quelque part) de la liste.
- Premier, Dernier : Liste  $\rightarrow$  Element. Permet de consulter l'élément en tête ou en fin de la liste.
- Queue : Liste  $\rightarrow$  Liste. Renvoie la liste privée de son élément de tête.
- Suivant, Précédent : Liste, Adresse ou Indice  $\rightarrow$  Element. Permet de consulter l'élément juste après ou avant l'élément désigné par une adresse (un pointeur vers un élément de la liste) ou un indice.
- Supprimer : Liste, Adresse ou Indice  $\rightarrow$  Liste. Permet de supprimer l'élément désigné par une adresse (un pointeur vers un élément de la liste) ou un indice.
- Rechercher : Liste  $\rightarrow$  Adresse ou Indice. Permet de repérer la position d'une valeur dans une liste.
- Concatener : Liste, Liste  $\rightarrow$  Liste. Permet de fusionner deux listes dans l'ordre des éléments de la première puis de la seconde.

**Remarque 2.130.** — Cette simple énumération peut paraître longue mais les bibliothèques modernes sont même beaucoup plus riches que cela.

Comme pour les piles et les files que nous avons déjà vues, il y a deux grands modes d'implémentation des listes : par des tableaux ou par des maillons chaînés.

**2.3.1.3.1. Listes doublement chaînées.** — Signalons une variante des listes chaînée, qui consiste à munir chaque maillon d'un pointeur vers le maillon suivant et vers le maillon précédent. On obtient alors des listes doublement chaînées, qui peuvent facilement se parcourir dans les deux sens.



Le langage Python dispose d'une structure de donnée qui implémente les listes doublement chaînées, qui sont appelées **deque** (pour l'anglais *double ended queue*)<sup>(1)</sup>. Les **deque** font partie du module **collections**. Leur utilisation par rapport aux listes native Python est conseillée dans le cas où l'on doit ajouter beaucoup d'éléments à la fois en tête et en queue de liste. En effet, c'est dans ce scénario que l'on profite de la complexité  $O(1)$  pour les opérations d'ajout ou suppression en tête et en queue de liste.

Voici un exemple montrant les fonctions principales.

LISTING 2.25. Les listes chaînées s'appelle deque en Python

```

616 from collections import deque
617 d = deque()
618 d.append(1)
619 d.appendleft(2)
620 d.append(3)
621 d.popleft()
622 d.appendleft(4)
623 d.pop()
624 d.appendleft(5)
625 d.append(6)

```

Après execution, la variable *d* contient la liste [5, 4, 1, 6].

1. Wikipedia considère au contraire que les deque sont des types abstraits qui s'implémentent usuellement à l'aide de listes doublement chaînées !

**2.3.1.3.2. Tableaux redimensionnables ou vecteurs.** — Après toutes ces variantes, il est temps de se demander quelle structure de donnée est utilisée par Python derrière le type natif `list`. Python utilise des *tableaux redimensionnables* (appelés aussi parfois *vecteurs* dans le contexte de l’informatique).

Pour déclarer un *tableau redimensionnable* en C, nous partons de la même définition que nous avons utilisé tout du long.

LISTING 2.26. Structure de pile en C

```

626 typedef struct vector vector_t;
627 struct vector {
628     int capacity;
629     int size;      // 0 <= size <= capacity
630     int *data;     // tableau de taille capacity
631 };

```

Cependant, lorsque la capacité du tableau est atteinte, nous réallouons un nouveau tableau de taille `2*capacity` et nous recopions tous les éléments dans le nouveau tableau.

On pourrait être effrayé par la complexité de l’opération. Cependant, il faut noter que chaque redimensionnement a lieu à chaque fois que la longueur de la liste dépasse une certaine valeur de la forme  $2^k$ . Le prochain redimensionnement ayant lieu à la longueur  $2^{k+1}$ , on peut considérer que les  $2^k = 2^{k+1} - 2^k$  éléments ajoutés entre temps payent pour la copie complète d’un tableau de longueur  $2^{k+1}$ . En moyenne, cela revient à un coût constant par élément. On parle de *complexité amortie* pour qualifier ce mode de calcul.

**2.3.1.3.3. Bilan.** — Voyons les principaux avantages et inconvénients entre les deux modes d’implémentations des listes (au sens d’un type abstrait).

| Primitive                      | Tableau                          | Liste chaînée                                                     |
|--------------------------------|----------------------------------|-------------------------------------------------------------------|
| Création                       | Taille bornée, espace perdu      |                                                                   |
| Insertion                      | En fin du tableau                | Ajout d’un maillon                                                |
| Accès au $k$ -ième terme       | Immédiat                         | Parcours de la liste nécessaire                                   |
| Suppression du $k$ -ième terme | Facile seulement en fin de liste | Facile n’importe où si l’adresse du maillon précédent est connue. |

**Remarque 2.131.** — Dans un langage de programmation comme Java, les deux modèles de liste coexistent et s’appellent respectivement `LinkedList` et `ArrayList`. Les deux collections implémentent l’interface `List`, si bien qu’un usager simple a l’impression de disposer de deux structures redondantes. Pour choisir laquelle utiliser, il convient de bien analyser ses besoins.

**2.3.2. Stocker et retrouver un élément.** —

**2.3.2.1. Types abstraits ensemble et dictionnaires.** — En informatique, un *ensemble* est une collection qui permet de stocker des objets avec les opérations d'ajout, suppression et un test d'appartenance.

A minima, les opérations souhaitées sont les suivantes.

- EnsembleVide :  $() \rightarrow \text{Ensemble}$ .
- EstVide :  $\text{Ensemble} \rightarrow \text{Booléen}$
- Insérer :  $\text{Ensemble}, \text{Element} \rightarrow \text{Ensemble}$
- Supprimer :  $\text{Ensemble}, \text{Element} \rightarrow \text{Ensemble}$
- Appartient :  $\text{Ensemble}, \text{Element} \rightarrow \text{Booléen}$ .

Ces quatre opérations obéissent aux spécifications suivantes :

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(e) = \begin{cases} \text{Vrai} & \text{si } e = \text{EnsembleVide}() \\ \text{Faux} & \text{si } e \text{ est de la forme } \text{Insérer}(e', y) \end{cases}$$

- La fonction Appartient satisfait :

$$\text{Appartient}(e, x) = \begin{cases} \text{Faux} & \text{si } e = \text{EnsembleVide}() \\ \text{Vrai} & \text{si } e \text{ est de la forme } \text{Insérer}(e', x) \\ \text{Appartient}(e', x) & \text{si } e \text{ est de la forme } \text{Insérer}(e', y) \end{cases}$$

- La fonction Supprimer( $e, x$ ) n'est valide que si EstVide( $e$ ) vaut Faux et alors, en notant  $e' = \text{Supprimer}(e, x)$  :

$$\text{Supprimer}(\text{Insérer}(e, y), x) = \begin{cases} e' & \text{si } x = y \\ \text{Insérer}(e', y) & \text{sinon} \end{cases}$$

- etc.

Une *table d'association*, ou *tableau associatif*, ou *dictionnaire* est une collection qui permet d'associer des valeurs à des clés, à raison d'au plus une valeur par clé. Cette notion informatique correspond à la notion de fonction en mathématique (possédant un domaine de définition fini, pour être précis).

A minima, les opérations souhaitées sont les suivantes.

- DictionnaireVide :  $() \rightarrow \text{Dictionnaire}$ .
- EstVide :  $\text{Dictionnaire} \rightarrow \text{Booléen}$
- Insérer :  $\text{Dictionnaire}, \text{Clé}, \text{Valeur} \rightarrow \text{Dictionnaire}$
- Supprimer :  $\text{Dictionnaire}, \text{Element} \rightarrow \text{Dictionnaire}$
- Rechercher :  $\text{Dictionnaire}, \text{Clé} \rightarrow \text{Valeur} \cup \text{Erreur}$ .

Voici quelques axiomes et règles s'appliquant à ce type :

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(d) = \begin{cases} \text{Vrai} & \text{si } d = \text{DicoVide}() \\ \text{Faux} & \text{si } d \text{ est de la forme } \text{Insérer}(d', x, v) \end{cases}$$

- La fonction `Rechercher` satisfait :

$$\text{Rech.}(d, x) = \begin{cases} \text{Erreur} & \text{si } e = \text{DicoVide}() \\ v & \text{si } d \text{ est de la forme } \text{Insérer}(d', x, v) \\ \text{Rech.}(d', x) & \text{si } d \text{ est de la forme } \text{Insérer}(d', y) \\ & \text{où } x \neq y \end{cases}$$

- La fonction `Supprimer(e, x)` n'est valide que si `EstVide(e)` vaut Faux et alors :

$$\text{Suppr.}(\text{Insérer}(d, y, v), x) = \begin{cases} d & \text{si } x = y \\ \text{Insérer}(d', y, v) & \text{où } d' = \text{Suppr.}(d, x) \end{cases}$$

Nous observons qu'il y a une grande parenté entre les deux notions. En effet, si l'on sait définir une structure de donnée pour définir un dictionnaire, qui à n'y stocker que des associations entre un élément comme clé et une valeur joker comme valeur associée, on a quasiment défini une structure de donnée qui implémente le type abstrait ensemble.

Inversement, dans les constructions d'ensembles, pour peu que l'on stocke un couple d'une clé et d'une valeur, on retrouve presque la notion de dictionnaire.

Cette parenté est rappelée par la syntaxe Python de deux types proches : le type `set`, qui se construit avec des accolades, et le type `dict`, qui se construit avec des accolades et un double point pour séparer la clé de la valeur.

Par ailleurs, on retrouve en Python l'idée des types abstraits ensemble et dictionnaire derrière les types émulant des conteneurs Python (ou *containers*). En quelque mot, il s'agit de construire de munir toute classe ayant vocation à se comporter comme un *containers* d'une série de méthodes dont le nom est canonique et qui permettent à Python de retrouver le sucre syntaxique habituel. Par exemple

- `object.__contains__(self, item)` test l'appartenance d'un objet et permet à la syntaxe Python `x in X` de fonctionner.
- `object.__len__(self)` doit renvoyer le nombre d'éléments.
- les trois méthodes `object.__getitem__(self)`, `object.__setitem__(self)`, `object.__delitem__(self)` permettent de gérer des accès et écritures directes d'un élément par son indice.
- etc.

Nous renvoyons au §3.37 de la documentation Python pour des détails à ce sujet : <https://docs.python.org/3/reference/datamodel.html#emulating-container-types>

**2.3.2.2. Solution naïves.** — Commençons par énumérer quelques solutions naïves pour stocker des ensembles.

- On peut bien entendu utiliser une liste dont maintient l'invariant « la liste est sans doublon ». C'est en général une mauvaise idée. Chaque recherche d'un élément peut prendre un temps linéaire en la taille de la liste ; chaque insertion de même, puisqu'il faut vérifier qu'un élément ne s'y trouve pas avant de l'insérer.
- On peut aussi utiliser une liste dont maintient l'invariant « la liste est sans doublon et elle est triée ». Dans ce cas, la recherche d'un élément peut se faire par une recherche dichotomique : dans le cas où l'accès aux éléments est direct,

le coût est logarithmique en la taille de la liste. L'insertion reste une opération coûteuse.

Ces deux solutions sont rarement optimales lorsque l'ensemble à stocker est de grande taille. Pourtant, de nombreux programmeurs débutants se contentent souvent de ne programmer qu'avec des listes, quitte à être très inefficaces. Inversement, lorsque l'ensemble à stocker est de petite taille, il est souvent suffisant et plus efficace de le stocker dans un tableau plutôt que d'utiliser des structures élaborées que nous verrons plus loin.

Signalons une autre manière astucieuse de représenter des ensembles, spécifique à la représentation d'ensembles d'entiers naturels. On suppose ici que l'on souhaite stocker un sous-ensemble  $S$  de l'intervalle entier  $\llbracket 0, M - 1 \rrbracket$ . On utilise pour cela un tableau  $t$  de booléens de longueur  $M$  dont on fixe les valeurs par

$$t[k] = \begin{cases} \text{Vrai} & \text{si } k \in S \\ \text{Faux} & \text{sinon} \end{cases}.$$

L'intersection et l'union de deux ensembles se traduisent, respectivement, par les opérations logique ET et OU calculées composante par composante. Cette manière de procéder est particulièrement habile si  $M$  est une puissance de 2. En effet, on peut directement utiliser des mots machines de  $M$  bits pour stocker le tableau. Par exemple, avec des mots de 64 bits, on peut représenter des ensembles d'entiers compris entre 0 et 63. Lorsque l'ensemble d'entiers à représenter est plus grand, on peut souvent s'appuyer sur une structure de tableau de booléens issue de la bibliothèque standard du langage. Par exemple, `std::vector<bool>` en langage C++ ou bien dans la bibliothèque `numpy` de Python. Il s'agit en général d'une réécriture de la notion de tableau pour que chaque booléen soit réellement codé sur un bit (et non pas un mot machine). Les opérations booléens sont alors hautement optimisées pour être efficace. Les tableaux de booléens sont notamment utilisés pour optimiser des recherches dans des tables de bases de données : le tableau représente alors les indices des lignes à conserver.

### 2.3.2.3. Structure d'arbres binaires de recherche. —

**Définition 2.132.** — Soit  $(E, \leq)$  un ensemble ordonné. Un *arbre binaire de recherche* est un arbre binaire étiqueté dans  $E$  qui jouit de l'invariant suivant : pour tout sommet interne  $s$ , d'étiquette  $e \in E$ , et pour tout sommet  $s'$  descendant de  $s$ , d'étiquette  $e' \in E$ , si le sommet  $s'$  appartient au sous-arbre gauche du sommet  $s$ , alors on a

$$e' < e;$$

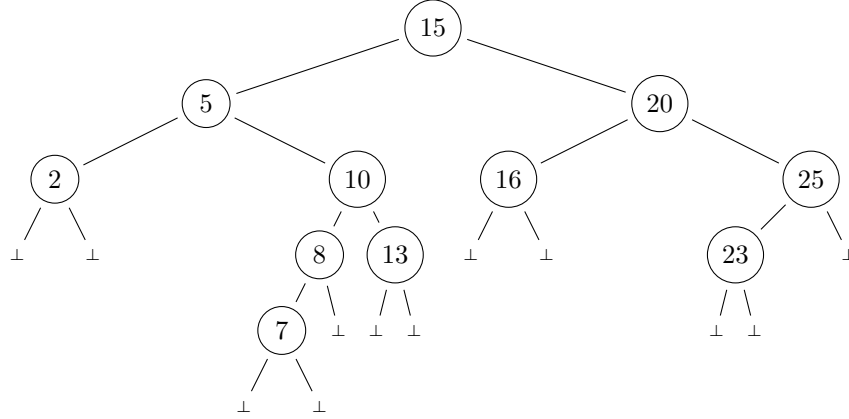
en revanche, si le sommet  $s'$  appartient au sous-arbre droit de  $s$ , alors on a

$$e \leq e'.$$

**Remarque 2.133.** — Dans certaines définitions, il est demandé que les inégalités soient strictes des deux côtés, ce qui implique que toutes les étiquettes sont distinctes. Cela permet alors de représenter des ensembles plutôt que des multi-ensembles.

**Exemple 2.134.** — Voici un exemple d'arbre





Nous disons qu'un élément  $x \in E$  appartient à un arbre binaire de recherche  $a$  et nous notons  $x \in a$  si  $x$  est l'étiquette de l'un des sommets de  $a$ .

Soient  $x \in E$  et  $a$  un arbre binaire de recherche. Quatre cas peuvent se présenter si nous souhaitons trouver l'élément  $x$  dans un arbre  $a$ .

$$\begin{cases} \text{Si } a = (e|g|d) \text{ et } x < e, & \text{alors } x \in a \Leftrightarrow x \in g \\ \text{Si } a = (e|g|d) \text{ et } x = e, & \text{alors } x \in a \\ \text{Si } a = (e|g|d) \text{ et } x > E, & \text{alors } x \in a \Leftrightarrow x \in d \\ \text{Si } a = \perp, & \text{alors } x \in a \Leftrightarrow x \in d \end{cases}$$

où  $e$  désigne l'étiquette de la racine,  $g$  son fils gauche et  $d$  son fils droit quand ils existent.

LISTING 2.27. Arbres binaires de recherche

```

632 class ABR():
633     def __init__(self, d, a = None, b = None):
634         self.data = d
635         self.left = a
636         self.right = b
637
638     def find(root, key):
639         if root is None or root.data == key:
640             return root
641         if key < root.data:
642             return find(root.left, key)
643         else:
644             return find(root.right, key)
645
646     def insert(root, key):
647         if root is None:
648             return ABR(key)
649         if key < root.data:

```

```

650     root.left = insert(root.left, key)
651     elif key > root.data:
652         root.right = insert(root.right, key)
653     return(root)

```

Le code ainsi présenté permet d'implémenter facilement une structure d'ensemble. Pour implémenter une structure de dictionnaire, on peut remplacer l'attribut unique `self.data` par deux attributs, disons `self.data` et `self.data`, et faire porter la recherche sur le premier des deux. Pour être rigoureux, la structure présentée autorise la représentation de multi-ensemble (plusieurs éléments identiques peuvent être ajoutés). Pour représenter uniquement des ensembles, nous pourrions imposer que les étiquettes soient toutes distinctes.

**Remarque 2.135.** — Pour raisonner par induction structurelle sur les graphes, nous pouvons voir qu'ils peuvent s'obtenir comme sous-ensemble des arbres binaires étiquetés avec les règles

$$\frac{}{\perp \in \mathcal{ABR}} \qquad \frac{g \in \mathcal{ABR} \quad d \in \mathcal{ABR} \quad \begin{cases} g=\perp \text{ ou } \max(g) < e \text{ et} \\ d=\perp \text{ ou } e \leq \min(d) \end{cases}}{(e|g|d) \in \mathcal{ABR}}$$

pour tout  $e \in E$  et où les fonction min et max sont celles de l'exemple 2.102.

**Proposition 2.136.** — Soit  $(E, \leq)$  un ensemble ordonné et  $A$  un arbre binaire étiqueté dans  $E$ . L'arbre  $A$  est un arbre binaire de recherche si et seulement si la liste des étiquettes obtenue suivant le parcours en ordre infixe (cf. définitions 2.108) forme une suite croissante pour l'ordre  $\leq$ .

*Démonstration.* — Dans le cas direct, nous raisonnons par induction structurelle sur l'arbre. Supposons que  $A$  est un arbre binaire de recherche. Si  $A$  est de la forme  $\perp$ , la liste obtenue est la liste vide, qui est ordonnée. Sinon, d'après la remarque 2.135,  $A$  est de la forme  $(e|g|d)$  avec  $\max(g) \leq e$  si  $g$  n'est pas vide et  $e \leq \min(d)$  si  $d$  n'est pas vide. Par définition du parcours infixe, la liste des éléments de  $A$  est la liste  $L = [L_g, e, L_d]$  où  $L_g$  est la liste obtenue à partir de l'arbre  $g$  et  $L_d$  est la liste obtenue à partir de l'arbre  $d$ . Mais par hypothèse d'induction structurelle, les listes  $L_g$  et  $L_d$  sont croissantes pour l'ordre  $\leq$ . Par ailleurs, les inégalités  $\max(g) \leq e$  et  $e \leq \min(d)$  assurent que  $L$  est trié par valeurs croissantes.

Dans le cas indirect, nous raisonnons par récurrence sur la taille de la liste. Réciproquement, supposons que l'arbre  $A$  est un arbre binaire étiqueté quelconque dont la liste des étiquettes dans l'ordre infixe  $L$  est croissante. Si  $L$  est la liste vide, alors  $A$  est l'arbre vide et il n'y a rien à dire, c'est forcément un arbre binaire de recherche. Si  $L$  n'est pas la liste vide, alors  $A$  est de la forme  $(e|g|d)$ , alors, la liste des étiquettes  $L$  se décompose en trois parties :  $[L_g, e, L_d]$  où  $L_g$  est la liste des étiquettes de  $g$  et  $L_d$  est la liste des étiquettes de  $d$ . Comme  $L$  est trié par ordre croissant, on a forcément  $\max(L_g) \leq e$  et  $e \leq \min(L_d)$ . De plus, par hypothèse de récurrence sur les listes  $L_g$  et  $L_d$  (qui contiennent au moins un élément de moins que  $L$ ), les arbres  $g$  et  $d$  sont des arbres binaires de recherche. Ainsi, d'après la remarque 2.135,  $A = (e|g|d)$  est bien un arbre binaire de recherche.  $\square$

**2.3.2.3.1. Supprimer un élément.** — Nous déduisons immédiatement de la remarque qui précède.

**Proposition 2.137.** — *Le sommet de plus petite, respectivement de plus grande, étiquette d'un arbre binaire de recherche s'obtient, s'il est défini, par un parcours depuis la racine empruntant toujours le fils gauche, respectivement toujours le fils droite et s'arrêtant quand il n'y a plus de fils gauche, respectivement plus de fils droit.*

*Démonstration.* — Démontrons cette propriété pour la recherche de l'élément maximum. Nous raisonnons par induction structurelle. Si l'arbre est vide, il n'y a rien à dire. Si l'arbre est non vide et de la forme  $a = (e, g, d)$ , il a été construit à partir deux arbres tels que

$$(g = \perp \text{ ou } \max(g) < e) \text{ et } (d = \perp \text{ ou } e \leq \min(d))$$

Observons que dans tous les cas, le maximum ne peut pas provenir de  $g$ . Il s'agit donc soit du sommet  $e$ , soit du sommet maximum dans  $d$ . Deux cas peuvent survenir. Si  $d$  est l'arbre vide, alors le maximum de  $a$  vaut  $e$ . Mais alors le parcours à droite depuis la racine s'arrête immédiatement après la racine comme souhaité. Si  $d$  n'est pas l'arbre vide, comme  $e \leq \min(d)$ , le sommet maximum de  $a$  est aussi le sommet maximum de  $d$ . Par hypothèse d'induction, le maximum de  $d$  se calcule à partir de la racine de  $d$  par un parcours qui emprunte le fils toujours à droite. Mais alors le parcours qui emprunte le fils toujours à droite depuis la racine de  $a$  termine au même sommet, qui est bien le maximum de  $a$  dans ce cas.  $\square$

Nous pouvons à présent parler de la suppression d'un sommet dans un arbre binaire de recherche. Supprimer un élément d'un arbre binaire de recherche se fait par induction sur la structure : il est facile de vérifier que les deux règles

$$\text{Supprimer}(\perp, e') = \perp$$

$$\text{Supprimer}((e|g|d), e') = \begin{cases} (e|\text{Supprimer}(g, e')|d) & \text{si } e' < e \\ (e|g|\text{Supprimer}(d, e')) & \text{si } e' > e \end{cases}$$

sont compatibles avec l'invariant de bonne formation d'un arbre binaire de recherche. Le cas  $\text{Supprimer}((e, g, d), e)$  où l'élément à supprimer est à la racine est le plus délicat. Si  $d$  est l'arbre vide, renvoyer  $g$  convient. Remarquons que symétriquement, si  $g$  est l'arbre vide, renvoyer  $d$  convient. Si  $d$  est non-vide, nous aimerions trouver comme nouvelle racine une étiquette  $x$  de  $g$  ou de  $d$  telle que pour toute étiquette  $\hat{e} \in g$ ,  $\hat{e} < x$  et pour toute étiquette  $\hat{e} \in d$ ,  $x \leq \hat{e}$  et faire le moins de transformations possibles sur  $g$  et  $d$ . Avec nos contraintes, notre choix de  $x$  porte alors sur le minimum de  $d$ .

$$\text{Supprimer}((e, g, d), e) = \begin{cases} g & \text{si } d = \perp \\ d & \text{si } g = \perp \\ (\max(g)|\text{Supprimer}(g, \max(d))|d) & \text{sinon} \end{cases}$$

On peut remarquer que les appels récursifs ne s'enchaînent pas, puisque  $\max(d)$  est un sommet qui n'a pas de fils gauche.

LISTING 2.28. Suppression dans un arbre binaire de recherche

```

654 def find_min(root):
655     if root is None or root.left is None:
656         return root
657     else:
658         return find_min(root.left)
659
660 def remove(root, key):
661     if root is None:
662         pass
663     elif root.data > key:
664         root.left = remove(root.left, key)
665     elif root.data < key:
666         root.right = remove(root.right, key)
667     else:
668         if root.left is None:
669             root = root.right
670         elif root.right is None:
671             root = root.left
672         else:
673             mnode = find_min(root.right)
674             root.data = mnode.data
675             root.right = remove(root.right, root.data)
676     return root

```

**2.3.2.3.2. Complexité.** — Toutes les opérations que nous venons de décrire se basent sur la recherche d'un élément, dont la complexité en temps est linéaire en la profondeur d'un élément recherché, autrement dit, dans le pire des cas, linéaire en la hauteur de l'arbre.

En notant  $n$  le nombre de sommets d'un arbre, on peut constater que la hauteur d'un arbre est linéaire en  $n$  dans le pire des cas et logarithmique en  $n$  dans le cas moyen. En conséquence, les algorithmes de recherche d'un élément, d'ajout ou de suppression sont linéaires en  $n$  dans le pire des cas et logarithmiques en  $n$  dans le cas moyen.

**Remarque 2.138.** — La littérature regorge de variantes d'arbres binaires de recherche, dit *équilibrés*. En général, il s'agit de sous-ensemble de d'arbres binaires de recherche qui se définissent avec une règle supplémentaire sur la hauteur des sous-arbres.

$$\frac{}{\perp \in \mathcal{ABR}_{\text{éq}}} \quad \frac{g \in \mathcal{ABR}_{\text{éq}} \quad d \in \mathcal{ABR}_{\text{éq}} \quad -1 \leq h(d) - h(g) \leq 1}{(e, g, d) \in \mathcal{ABR}_{\text{éq}}} \text{ Si}$$

l'on note  $N(h)$  le nombre minimum de sommets appartenant à un arbre équilibré de hauteur  $h$ , on observe que  $N(h) = 1 + N(h-1) + N(h-2)$ . En effet, un arbre minimal de hauteur  $h$  contient : sa racine, un sous-arbre de hauteur  $N(h-1)$  et un second

arbre de hauteur comprise entre  $h - 2$  et  $h - 1$  (pour des raisons d'équilibre). Une preuve par récurrence montre que  $N(h) = \Omega(\varphi^h)$  où  $\varphi = \frac{1+\sqrt{5}}{2}$  est le nombre d'or.

Parmi les exemples d'arbres équilibrés, on peut citer les *arbres AVL* ou les *arbres rouges et noirs*.

**2.3.2.4. Structure de tables de hachage.** — Mentionnons pour être complet que les types abstraits ensembles et dictionnaire peuvent être implémentés par des *tables de hachage*.

Les types `set` et `dict` du langage Python sont implémentés selon un principe de table de hachage. Du point de vue de l'utilisateur, ces types natifs s'utilisent comme un mathématicien écrirait des ensembles et des fonctions (au sens mathématique d'une association entre clés du domaine de définition de la fonction et valeurs dans le domaine d'arrivée de la fonction).

LISTING 2.29. Exemple d'utilisation des types `set` et `dict` en Python

```

677 s = set() # construit l'ensemble vide
678 s.add(3) # ajoute un élément à l'ensemble s
679 len(s) # renvoie le cardinal de S
680 3 in s # teste l'appartenance à S
681
682 d = dict() # construit la fonction vide
683 d['a'] = 97 # construit l'association de la clé 'a' à la valeur 97
684 'a' in d # teste l'appartenance de 'a' au domaine de définition de d
685 d['a'] # renvoie la valeur de

```

**2.3.2.4.1. Principes de fonctionnement.** — Une table de hachage consiste à stocker un ensemble de  $t$  éléments  $(e_i)_{0 \leq i < t}$  ou de  $t$  associations  $(k_i \mapsto v_i)_{0 \leq i < t}$  formés d'une clé  $k_i$  et d'une valeur  $v_i$  dans un tableau  $T$  ayant une taille  $n$  supérieure mais proche de  $t$ .

Dans la suite de ce paragraphe, nous nous concentrons sur le cas d'associations, c'est-à-dire que nous décrivons une structure de données permettant de réaliser un dictionnaire. Dans ce cas, le tableau  $T$  est en fait un tableau de couples qui permet de stocker dans chaque case l'une des associations  $(k_i \mapsto v_i)_{0 \leq i < t}$ .

Pour parvenir à faire rentrer une série hétéroclite de clés dans un tableau et déterminer l'indice de quelle case utiliser, nous supposons définie une certaine fonction  $h : K \rightarrow \mathbb{N}$ , dite *fonction de hachage*, qui transforme toute clé en un entier.

**Définition 2.139.** — On appelle *fonction de hachage* (non cryptographique) toute fonction  $h : K \rightarrow \mathbb{N}$  qui associe des entiers de taille fixe à des données de taille quelconque (issues de l'ensemble  $K$ ). Les valeurs renvoyées par une fonction de hachage s'appellent les *résumés*, ou les *signatures* ou simplement les *hachés*.

Une bonne fonction de hachage est une fonction qui se calcule rapidement et qui donne une impression de distribution aléatoire et uniforme de ses hachés. Dans des

contextes cryptographiques, des propriétés supplémentaires sont requises, essentiellement liées à l'impossibilité pratique de construire une clé correspondant à un haché donné.

**Exemple 2.140.** — La fonction de hachage utilisée par python est la fonction *SipHash* qui combine des opérations de XOR bit à bit et de multiplication (cf PEP 456, <https://peps.python.org/pep-0456/>). Son code correspond à

LISTING 2.30. Fonction de hachage *SipHash* écrite en python

```

686 def fnv(p):
687     if len(p) == 0:
688         return 0
689
690     # bit mask, 2**32-1 or 2**64-1
691     mask = 2 * sys.maxsize + 1
692
693     x = hashsecret.prefix
694     x = (x ^ (ord(p[0]) << 7)) & mask
695     for c in p:
696         x = ((1000003 * x) ^ ord(c)) & mask
697     x = (x ^ len(p)) & mask
698     x = (x ^ hashsecret.suffix) & mask
699
700     if x == -1:
701         x = -2
702
703     return x

```

- La *création* d'une table de hachage se fait en initialisant un tableau de longueur  $n$ . L'usage est d'utiliser une puissance de 2 comme valeur de  $n$ . À la création de la table, on utilise une valeur sentinelle pour indiquer que chaque alvéole est vide.
- L'*insertion* d'une association  $k \mapsto v$  se fait en calculant le haché  $h(k)$  de la clé  $k$ , puis d'enregistrer le couple  $(k, v)$  dans l'alvéole  $T[i]$  d'indice  $i = h(k) \bmod n$ . Lorsque  $n$  est une puissance de 2, calculer l'indice  $i$  est extrêmement simple puisqu'il suffit de ne conserver que les bits de poids faible de  $h(k)$ . Dans la section 2.3.2.4.3, nous verrons comment traiter le cas où la case est déjà occupée.
- La *recherche* d'une valeur  $v$  associée à la clé  $k$  se fait en calculant d'abord  $i = h(k) \bmod n$ , puis en vérifiant la valeur  $T[i]$ . Si  $T[i]$  contient un couple de la forme  $(k', v)$  avec  $k' = k$ , on peut renvoyer la valeur  $v$ . Sinon, on lève une erreur pour signaler que la table de hachage ne contient pas la clé  $k$ .

**Exemple 2.141.** — Nous souhaitons stocker des associations de prénoms et de numéros de téléphone. Nous utilisons comme fonction de hachage le numéro de première lettre du prénom dans l'ordre alphabétique. Avec les données

| Clé     | Valeur         | Haché | Haché mod. 4 |
|---------|----------------|-------|--------------|
| Amine   | 06.34.56.01.89 | 1     | 1            |
| Sharon  | 06.59.12.71.30 | 19    | 3            |
| Félicie | 07.75.23.29.94 | 6     | 2            |

et un tableau  $T$  de longueur  $n = 4$ , on obtient le contenu suivant :

|   |                |                |                |
|---|----------------|----------------|----------------|
| 0 | Amine          | Félicie        | Sharon         |
| 0 | 06.34.56.01.89 | 07.75.23.29.94 | 06.59.12.71.30 |

$T$

**Remarque 2.142.** — Pour accéder au haché d'un objet Python  $x$ , il suffit de demander `hash(x)`, qui fait partie des fonctions intégrées (*built-in*) de Python. Il n'y a pas de sens à définir le haché d'un objet mutable; seuls les objets immuables sont hachables. Ceci explique pourquoi ni un ensemble de type `set`, ni un dictionnaire de type `dict` ne peut contenir de liste comme élément ou comme clé. Par contre, il est parfaitement possible d'utiliser une liste comme valeur associée à une clé.

Dans le cas où un programmeur voudrait fournir sa propre méthode de hachage lorsqu'il définit une nouvelle classe, il est nécessaire de surcharger la méthode `__hash__`.

**2.3.2.4.2. Facteur de charge.** — Que se passe-t-il si la table est pleine ?

**Définition 2.143.** — On appelle *facteur de charge* le ratio  $t/n$  où  $t$  est le nombre d'associations insérées dans la table et  $n$  est la longueur du tableau  $T$ .

**Exemple 2.144.** — Le facteur de charge du tableau suivant est  $3/4$ .

|   |                |                |                |
|---|----------------|----------------|----------------|
| 0 | Amine          | Félicie        | Sharon         |
| 0 | 06.34.56.01.89 | 07.75.23.29.94 | 06.59.12.71.30 |

$T$

Pour assurer le bon fonctionnement de la table de hachage, on calcule le facteur de charge après chaque insertion. Dans le cas où ce facteur dépasse un certain seuil, on double la taille de la table de hachage, ce qui implique de recalculer la position de chaque association déjà insérée.

Dans le cas de Python, il semble que la valeur de seuil soit  $2/3$ .

**Exemple 2.145.** — Dans l'exemple ci-dessus, on obtient après redimensionnement le tableau suivant :

|   |                |   |                |   |   |                |   |
|---|----------------|---|----------------|---|---|----------------|---|
| 0 | Amine          | 0 | Sharon         | 0 | 0 | Félicie        | 0 |
| 0 | 06.34.56.01.89 | 0 | 06.59.12.71.30 | 0 | 0 | 07.75.23.29.94 | 0 |

**Remarque 2.146.** — En complexité amortie, le coût du redimensionnement de la table de hachage est  $\Theta(1)$ .

**2.3.2.4.3. Gestion des collisions.** — Jusqu'à présent, nous avons supposé que toute case était systématiquement disponible. Que se passe-t-il si on souhaite insérer une association  $k \mapsto v$  dans  $T[i]$  alors que la case  $T[i]$  est déjà occupée ?

**Définition 2.147.** — On parle de *collision* dans une table de hachage de longueur  $n$  et de fonction de hachage  $h$  lorsqu'on souhaite insérer deux associations de clé  $k$  et  $k'$  telles que leur haché satisfait

$$h(k) \equiv h(k') \pmod{n}.$$

**Exemple 2.148.** — Dans le cas où l'on souhaite insérer les données suivantes

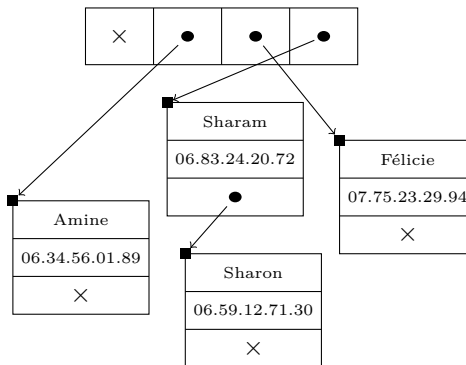
| Clé     | Valeur         | Haché | Haché mod. 4 |
|---------|----------------|-------|--------------|
| Amine   | 06.34.56.01.89 | 1     | 1            |
| Sharon  | 06.59.12.71.30 | 19    | 3            |
| Félicie | 07.75.23.29.94 | 6     | 2            |
| Sharam  | 06.83.24.20.72 | 19    | 3            |

il y a collision entre la clé **Sharon** et **Sharam**.

Une première technique de résolution des collisions consiste à organiser l'ensemble des données enregistrées à l'indice  $i$  sous forme d'une liste chaînée dont l'adresse du premier maillon est enregistrée dans  $T[i]$ . L'insertion se fait par insertion dans la liste désignée par  $T[i]$  ; la recherche d'une association de clé  $k$  se fait par un parcours de la liste désignée par  $T[i]$ . Avec cette technique, il est relativement facile de retirer une association : il suffit de la retirer de la liste chaînée.

**Définition 2.149.** — La résolution des collisions par recours à des listes chaînées est qualifiée de résolution par *chaînage externe*.

**Exemple 2.150.** — Nous poursuivons l'exemple avec les mêmes données :



Une seconde technique pour insérer l'association  $k \mapsto v$  dans la table consiste à exploiter au mieux les cases vides dans  $T$ . Dans le cas où la case d'indice  $i$  pressentie pour enregistrer le couple  $(k, v)$  est déjà occupée, on recalcule une nouvelle position en fonction de  $i$  jusqu'à obtenir une case libre. La recherche d'une association de clé  $k$  se fait alors par un parcours du tableau en démarrant à la case d'indice  $i$  et en poursuivant la recherche en se déplaçant dans  $T$  de la même manière que pour



une insertion jusqu'à trouver la clé recherchée ou bien trouver une case vide. La suppression d'un élément de clé  $k$  est plus délicate : il faut veiller à « reboucher » d'éventuels trous laissés par la suppression afin que les recherches d'éléments soient toujours possibles.

**Définition 2.151.** — La résolution des collisions par recours aux cases encore vides est qualifiée de résolution par *adressage ouvert*.

L'idée la plus simple pour se déplacer dans le tableau à partir de la case  $i$  est de s'intéresser à la case immédiatement après, à savoir la case d'indice  $(i + 1) \bmod n$ .

**Définition 2.152.** — On parle de *sondage linéaire* lorsqu'on recherche une case vide en se déplaçant dans le tableau en se décalant d'un pas fixe.

**Exemple 2.153.** — Avec les mêmes données et en utilisant le sondage linéaire (avec pas de 1), les positions sondées deviennent :

| Clé     | Valeur         | Haché mod 4 | Haché $2^{\text{aire}}$ mod 4 | Haché $3^{\text{aire}}$ mod 4 | Etc |
|---------|----------------|-------------|-------------------------------|-------------------------------|-----|
| Amine   | 06.34.56.01.89 | 1           | 2                             | 3                             |     |
| Sharon  | 06.59.12.71.30 | 3           | 0                             | 1                             |     |
| Félicie | 07.75.23.29.94 | 2           | 3                             | 0                             |     |
| Sharam  | 06.83.24.20.72 | 3           | 0                             | 1                             |     |

**Remarque 2.154.** — L'implémentation des ensembles (de type `set`) et dictionnaires (de type `dict`) par Python se fait selon un principe d'*adressage ouvert* avec un *double sondage* pour résoudre les collisions. Dans le cas où l'alvéole  $T[i]$  est occupée, on s'intéresse à l'alvéole  $T[i']$  où  $i' = 5i + 1 + p$ , où  $p$  est un terme de perturbation, et on répète le calcul d'un nouvel indice jusqu'à trouver une case vide. Avec une telle relation de récurrence, les indices sondés paraissent aléatoires. L'idée sous-jacente est qu'il est parfois difficile de garantir que la fonction de hachage étale bien les hachés des clés : il est donc intéressant que l'implémentation des tables de hachage elle-même tente de distribuer au mieux les hachés des clés à ranger.

Nous renvoyons les curieux directement au code source <https://github.com/python/cpython/blob/main/Objects/dictobject.c> pour mieux explorer les détails d'implémentation.

**Remarque 2.155.** — Dans la bibliothèque standard du langage C++, les deux manières de représenter des ensembles et des dictionnaires coexistent. Les structures `set` et `map` sont réalisées à travers des arbres binaires de recherche autoéquilibrant ; les structures `unordered_set` et `unordered_map` sont réalisées à travers des tables de hachage.

### 2.3.3. Ranger des éléments avec des priorités. —

**2.3.3.1. Le type abstrait : file à priorité.** — Une *file à priorité* est une collection qui permet de stocker des objets avec une priorité et de les restituer par ordre de priorité, selon une politique « plus important, premier servi ».

A minima, les opérations souhaitées sont les suivantes. Nous commençons par en donner les signatures :

- FileAPrioritéVide :  $() \rightarrow \text{FileAPriorité}$
- EstVide :  $\text{FileAPriorité} \rightarrow \text{Booléen}$
- Enfiler :  $\text{FileAPriorité}, \text{Element}, \text{Priorité} \rightarrow \text{FileAPriorité}$
- Defiler :  $\text{FileAPriorité} \rightarrow \text{FileAPriorité}, \text{Element}, \text{Priorité}$

L'élément de priorité est à valeur dans un ensemble ordonné. La fonction Defiler restitue l'élément présent dans la file de plus grande priorité. Dans certain cas, on admet de confondre l'élément avec sa priorité.

Ces quatre opérations obéissent aux spécifications suivantes :

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(f) = \begin{cases} \text{Vrai} & \text{si } f = \text{FileAPrioritéVide}() \\ \text{Faux} & \text{si } f \text{ est de la forme Enfiler}(f', x, p) \end{cases}$$

- La fonction Defiler n'est valide que si EstVide( $f$ ) vaut Faux et dans ce cas, on doit avoir, en notant  $(g, y, q) = \text{Defiler}(f)$ , selon les cas :

$$\text{Defiler}(\text{Enfiler}(f, x, p)) = \begin{cases} (f, x, p) & \text{si } p > q \\ \text{Enfiler}(g, x, p), y, q & \text{sinon.} \end{cases}$$

**Remarque 2.156.** — Dans de nombreux algorithmes, il est souhaité d'examiner un ensemble d'éléments  $E$  par ordre croissant selon un certain critère. Souvent, ces algorithmes sont présentés en commençant par trier l'ensemble des éléments. Il peut être plus élégant de simplement ranger ces éléments dans une file à priorité.

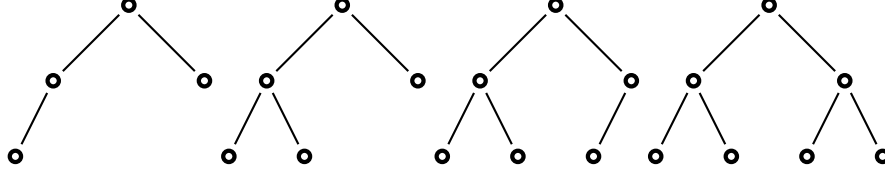
**2.3.3.2. La structure de tas.** — Un arbre binaire est dit parfait si toutes ses lignes sont remplies, sauf la dernière qui doit être remplie de gauche à droite. Formellement, on peut adopter la définition suivante

**Définition 2.157.** — Notons  $\mathcal{C}$  l'ensemble des arbres binaires complets étiquetés dans  $E$ . L'ensemble  $\mathcal{P}$  des *arbres binaires parfaits étiquetés dans  $E$*  est défini inductivement comme le sous-ensemble des arbres binaires étiquetés dans  $E$  définis par les règles :

$$\frac{}{\perp \in \mathcal{P}} \qquad \frac{g \in \mathcal{C} \quad d \in \mathcal{P} \quad h(g) = h(d)}{(e|g|d) \in \mathcal{P}} \\ \frac{g \in \mathcal{P} \quad d \in \mathcal{C} \quad h(g) = h(d) + 1}{(e|g|d) \in \mathcal{P}}$$

où  $h$  désigne la fonction hauteur (voir exemple 2.100) et où  $e \in E$ .

**Exemple 2.158.** — Voici les quatre arbres parfaits de hauteur 2, sans leurs étiquettes,



**Lemme 2.159.** — La hauteur d'un arbre parfait contenant  $n$  sommets est  $\Theta(\log_2 n)$ .

*Démonstration.* — Un arbre parfait de hauteur  $h$  possède entre  $\sum_{k=0}^{h-1} 2^k + 1 = 2^h$ , quand la dernière ligne ne possède qu'un sommet, et  $\sum_{k=0}^h 2^k = 2^{h+1} - 1$  sommets, quand la dernière ligne est pleine. On a donc l'encadrement

$$2^h \leq n < 2^{h+1}$$

ce qui équivaut à

$$h \leq \log_2(n) < h + 1$$

et montre que  $h = \Theta(\log_2 n)$ .  $\square$

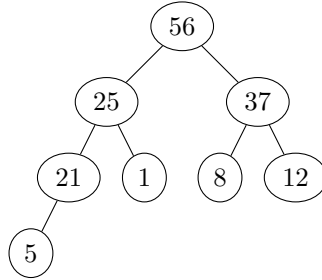
**Définition 2.160.** — Soit  $(E, \leq)$  un ensemble ordonné. On appelle *tas* (ou *tas max*) tout élément du sous-ensemble  $\mathcal{T} \subseteq \mathcal{P}$  des arbres parfaits étiquetés dans  $E$  définis inductivement par

$$\frac{\perp \in \mathcal{T} \quad g \in \mathcal{T} \quad d \in \mathcal{T} \quad \begin{cases} g=\perp \text{ ou } g=(e_g \mid g_g \mid d_g) \text{ avec } e_g \leq e \text{ et} \\ d=\perp \text{ ou } d=(e_d \mid g_d \mid d_d) \text{ avec } e_d \leq e \end{cases} \quad (e \mid g \mid d) \in \mathcal{P}}{(e \mid g \mid d) \in \mathcal{T}}$$

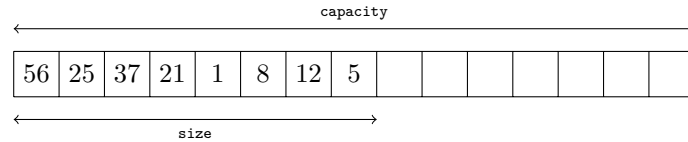
pour tout  $e \in E$ .

La structure de donnée de tas désigne l'implémentation des arbres étiquetés jouissant de la propriété de tas (définition 2.160) implémentés par un tableau (cf. 2.63).

**Exemple 2.161.** — Voici un exemple de tas (où les éléments et les priorités sont mélangés)



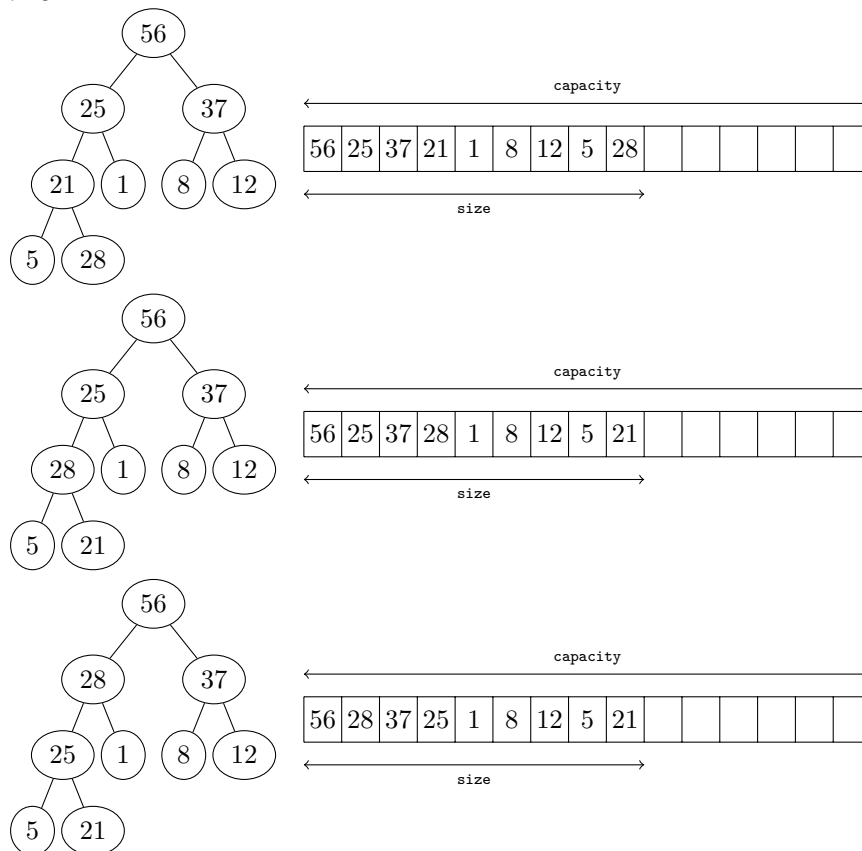
Il se représente dans le tableau suivant, où les dernières cases ne sont pas utilisées à ce stade.



**Remarque 2.162.** — De manière analogue à la construction de tas max, on peut définir la notion de *tas min* dans lequel le plus petit élément est placé à la racine (au lieu du plus grand élément).

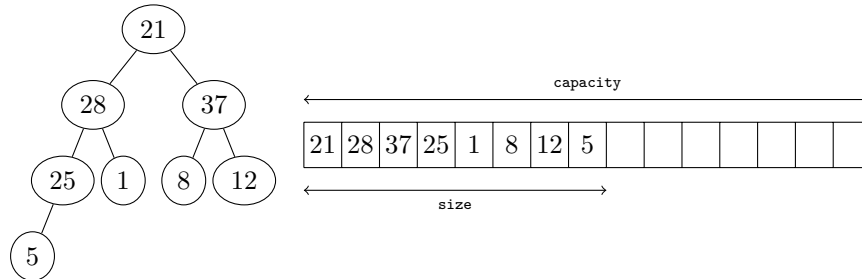
Pour *insérer un élément* dans le tas, on suit la démarche suivante : On commence par ajouter l'élément dans la première alvéole disponible (en fin de tableau). Puis, tant que la propriété de tas est violée, on échange l'élément inséré avec son père.

**Exemple 2.163.** — Voici par quels états passe la structure quand on insère l'élément 28

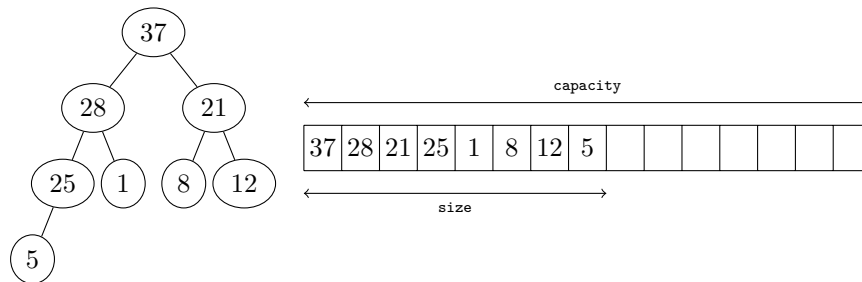


Pour *extraire un élément* du tas, on suit la démarche suivante : On commence par retirer l'élément de racine et on complète (temporairement) la place laissée vacante par le dernier élément du tableau. Puis, on restaure la propriété de tas de la racine vers la feuille en échangeant le père avec le plus grand de ses fils.

**Exemple 2.164.** — Voici par quels états passe la structure quand on extrait l'élément de plus haute priorité. Il s'agit de l'élément 56 qui est retiré et remplacé dans l'immédiat par l'élément 21.



Nous comparons 21 à 28 et à 37. Le plus grand des éléments est 37.



Puis on compare 21 à 8 et à 12. On constate qu'il n'y a plus besoin de faire d'échange.

LISTING 2.31. Structure de tas

```

704 class Heap():
705     def __init__(self, n):
706         self.capacity = n
707         self.size = 0
708         self.tree = [None for _ in range(n)]
709
710     def _swap(self, i, j):
711         self.tree[i], self.tree[j] = self.tree[j], self.tree[i]
712
713     def _restore_heap_from_bot_(self, i):
714         if i > 0:
715             p = (i-1)//2
716             if self.tree[p] < self.tree[i]:
717                 self._swap(p, i)
718                 self._restore_heap_from_bot_(p)
719
720     def push(self, x):
721         self.tree[self.size] = x
722         self._restore_heap_from_bot_(self.size)
723         self.size += 1
724
725     def _restore_heap_from_top_(self, i):

```

```

726         d = {i:self.tree[i]}
727         if 2*i+1 < self.size:
728             d[2*i+1] = self.tree[2*i+1]
729         if 2*i+2 < self.size:
730             d[2*i+2] = self.tree[2*i+2]
731         j = max(d, key=d.get)
732         if j != i:
733             self._swap_(i,j)
734             self._restore_heap_from_top_(j)
735
736     def pop(self):
737         self._swap_(0,self.size-1)
738         self.size -= 1
739         self._restore_heap_from_top_(0)

```

**2.3.3.3. Complexité et application aux tris.** — La complexité des opérations sur un tas est proportionnelle à la hauteur de l'arbre.

**Proposition 2.165.** — Dans un tas contenant  $n$  élément, l'opération d'insertion et de suppression d'un élément a une complexité en temps qui est  $O(\log_2 n)$ .

**Application 2.166.** — Pour trier un tableau, on peut successivement insérer puis retirer les éléments du tas. On appelle cette méthode le *tri tas*. Comme l'illustre le code ci-dessus, ces opérations peuvent même se faire en place et ne nécessitent pas de créer de nouveau tableau. La complexité du tri tas est  $O(n \log_2 n)$ .

**2.3.4. Partitions d'ensembles.** — Le type abstrait *partition* permet de représenter la division d'un ensemble en parties disjointes ou encore l'ensemble des classes d'équivalence d'une *relation d'équivalence*.

À minima, les primitives souhaitées pour le type abstrait partition sont les suivantes. Nous commençons par en donner les signatures :

- Créer : Ensemble  $\rightarrow$  Partition
- Unir : Partition, Element, Element  $\rightarrow$  Partition
- Trouver : Partition, Element  $\rightarrow$  Identifiant

La fonction *Trouver* renvoie un identifiant qui permet de repérer à quel part ou à quelle classe d'équivalence appartient un élément. Nous pourrions chercher à numéroter les parts. Mais, en pratique, il est plus simple d'utiliser les éléments eux-mêmes comme identifiants des composantes. La fonction *Unir* doit permettre de fusionner les deux parts ou les deux classes d'équivalence qui contiennent deux éléments donnés. Dans certains cas, il peut être intéressant que la fonction *Unir* renvoie également un booléen qui indique si une réunion a réellement eu lieu ou si les deux éléments appartenaient déjà à la même part. Enfin, la fonction *Créer* est une fonction d'initialisation chargée de diviser un ensemble en une partition formée d'un singleton par élément.

Plus formellement, la fonction `Trouver` répond aux spécifications suivantes : pour tout  $a, b, x \in E$ ,

$$\begin{aligned} \text{Trouver}(\text{Créer}(E), x) &= x \\ \text{Trouver}(\text{Unir}(P, a, b), a) &= \text{Trouver}(\text{Unir}(P, a, b), b) \\ \text{Trouver}(\text{Unir}(P, a, b), x) &= \begin{cases} \text{Trouver}(\text{Unir}(P, a, b), a) & \text{si } \text{Trouver}(P, x) = \text{Trouver}(P, a) \\ \text{Trouver}(P, x) & \text{ou si } \text{Trouver}(P, x) = \text{Trouver}(P, b) \\ \text{Trouver}(P, x) & \text{sinon} \end{cases} \end{aligned}$$

**Application 2.167.** — L'application principale du type abstrait « partition » est la représentation de la partition des sommets d'un graphe sous forme de composantes connexes. Dans ce cadre, l'opération `Trouver` permet de vérifier si deux sommets sont reliés entre eux par un chemin : l'opération `Unir` permet de fusionner deux composantes connexes.

**2.3.4.1. Implémentation naïve avec un tableau.** — Dans cette implémentation, nous supposons que les éléments de l'ensemble  $E$  sont numérotés. Nous raisonnons donc sur l'ensemble  $E = \llbracket 0, n-1 \rrbracket$ .

Nous stockons toute l'information dans un tableau à  $n$  cases. L'opération `Créer` revient à initialiser un tableau  $T$  avec  $T[x] = x$  pour tout  $x \in \llbracket 0, n-1 \rrbracket$ . L'opération `Trouver(x)` revient à renvoyer  $T[x]$ . Pour l'opération `Unir(a, b)` se fait en parcourant le tableau  $T$  et en remplaçant  $T[x]$  par  $T[a]$  si  $T[x] = T[b]$ .

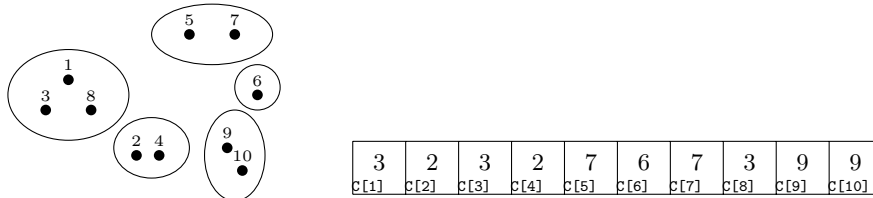
LISTING 2.32. Implémentation naïve du type abstrait partition

```

740 class NaivePartition():
741     def __init__(self, n):
742         self.T = [x for x in range(n)]
743     def find(self, x):
744         return self.T[x]
745     def union(self, a, b):
746         for x in range(len(self.T)):
747             if self.T[x] == self.T[b]:
748                 self.T[x] = self.T[a]
```

**Proposition 2.168.** — La complexité en temps de l'opération `find` est  $O(1)$  ; la complexité en temps de l'opération `union` est  $O(n)$ .

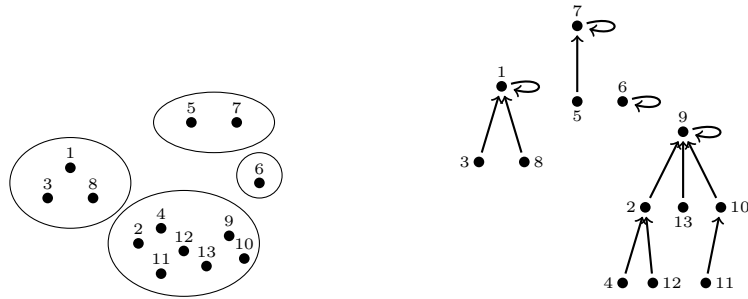
**Exemple 2.169.** — Considérons la partition suivante et sa représentation par un tableau.



**2.3.4.2. Implémentation par la structure « unir et trouver ».** — La structure *union find* consiste à organiser les éléments de la partition sous forme d’une forêt, dans laquelle les éléments pointent vers leur père. La racine de chaque arbre pointe vers elle-même.

L’opération **Créer** revient à initialiser un tableau de parents avec  $P[x] = x$  pour tout  $x \in \llbracket 0, n - 1 \rrbracket$ . L’opération **Trouver**( $x$ ) revient à remonter à la racine de  $x$ . Pour l’opération **Unir**( $a, b$ ), on récupère la racine de chaque arbre contenant  $a$  et  $b$  et on fait pointer l’une sur l’autre. Afin de maîtriser la forme des arbres, on retient la hauteur de chaque arbre et on branche le plus petit sur le plus grand.

**Exemple 2.170.** — Considérons la partition suivante et sa représentation par un tableau.



LISTING 2.33. Implémentation naïve du type abstrait partition

```

749 class UnionFind():
750     def __init__(self, n):
751         self.parent = [i for i in range(n)]
752         self.height = [0 for i in range(n)]
753     def find(self, u):
754         if u == self.parent[u]:
755             return(u)
756         return self.find(self.parent[u])
757     def union(self, u, v):
758         p, q = self.find(u), self.find(v)
759         if p==q:
760             return
761         if self.height[p] > self.height[q]:
762             self.parent[q] = p
763         elif self.height[p] == self.height[q]:
764             self.parent[q] = p
765             self.height[p] += 1
766         else:
767             self.parent[p] = q

```



**Proposition 2.171.** — La complexité en temps de l'opération `find` et de l'opération `union` du code 2.33 est  $O(\log_2 n)$ .

**Remarque 2.172.** — L'amélioration suivante consiste à écraser l'arbre en stockant un

```

768 def improved_find(self, u):
769     if u != self.parent[u]:
770         self.parent[u] = self.find(self.parent[u])
771     return self.parent[u]
```

## 2.4. Exercices

**Exercice 2.173.** — Soit  $(E, \leq)$  un ensemble ordonné. Montrer que l'ordre  $\leq$  est bien fondé si et seulement si il n'existe pas de suite infinie  $\mathbf{u} = (u_n)_{n \in \mathbb{N}} \subseteq E$  satisfaisant

$$\text{pour tout } n \in \mathbb{N}, \quad u_{n+1} < u_n.$$

**Exercice 2.174.** — Nous proposons la définition inductive suivante d'arbres binaires étiquetés non vides, avec une règle pour les feuilles (qui sont étiquetées) et une règle pour les nœuds internes.

Soit  $\Sigma_0$  un alphabet ne contenant pas les trois symboles  $(, |, )$  et soit  $E$  un langage sur  $\Sigma_0$ . On définit l'ensemble  $\mathcal{A}$  des mots sur l'alphabet  $\Sigma = E \cup \{(, |, )\}$  par le système

$$\frac{}{(e) \in \mathcal{A}} \qquad \frac{g \in \mathcal{A} \quad d \in \mathcal{A}}{(e|g|d) \in \mathcal{A}}$$

où  $e$  appartient à  $E$ .

Cette définition permet-elle de représenter les « mêmes » arbres que ceux définis par la définition 2.59? (Pour rendre précis le mot « même » : existe-t-il une bijection qui commute aux constructeurs?)

**Exercice 2.175.** — Soit  $\Sigma$  un alphabet. Donner une définition inductive du langage  $\mathcal{L}$  des mots de longueur paire de  $\Sigma^*$

**Exercice 2.176.** — Soit  $\Sigma$  l'alphabet binaire  $\Sigma = \{0, 1\}$  et  $\mathcal{R}$  la partie de  $\Sigma^*$  formée des mots binaires qui représentent un entier naturel en base 2. Autrement dit, l'ensemble  $\mathcal{R}$  est constitué du mot 0 et des mots binaires non vides commençant par le symbole 1. Décrire  $\mathcal{R}$  par une définition inductive d'une partie de  $\Sigma^*$ .

**Exercice 2.177.** — Soit  $\Sigma$  l'alphabet binaire  $\Sigma = \{0, 1\}$  et  $\mathcal{Z}$  la partie de  $\Sigma^*$  formée des mots binaires ne comportant pas deux zéros consécutifs.

1. Énumérer les éléments de  $\mathcal{Z}$  de longueur inférieure à 3.
2. Proposer une définition inductive de  $\mathcal{Z}$ .
3. Dessiner un arbre de dérivation pour chacun des mots de  $\mathcal{Z}$  de longueur inférieure à 3.
4. Dire si la définition donnée à la question 2. est ambiguë.

5. Proposer une définition inductive non ambiguë de  $\mathcal{L}$ .

**Exercice 2.178.** — Compléter le code 2.24 afin de coder entièrement une file en C

**Exercice 2.179.** — Dessiner tous les arbres binaires ayant exactement deux sommets internes.

**Exercice 2.180.** — Soit  $\Sigma$  l'alphabet  $\Sigma = \{\mathbf{z}, \mathbf{b}, \mathbf{t}, (, ), |\}$  et  $\mathcal{G}$  la partie de  $\Sigma^*$  formée des mots définis par les trois règles suivantes, dont un axiome,

$$\frac{}{\mathbf{z} \in \mathcal{G}} \text{ (Z)} \quad \frac{w_1 \in \mathcal{G} \quad w_2 \in \mathcal{G}}{\mathbf{b}(w_1 | w_2) \in \mathcal{G}} \text{ (B)} \quad \frac{w_1 \in \mathcal{G} \quad w_2 \in \mathcal{G} \quad w_3 \in \mathcal{G}}{\mathbf{t}(w_1 | w_2 | w_3) \in \mathcal{G}} \text{ (T)}$$

Démontrer que pour tout mot  $e \in \mathcal{G}$ , on a la relation

$$|e|_{\mathbf{z}} = 2 \cdot |e|_{\mathbf{t}} + |e|_{\mathbf{b}} + 1.$$

**Exercice 2.181.** — Cet exercice utilise les fonctions **Long** et **Concat** définies dans l'exemple 2.98. Démontrer que pour toutes listes  $\ell_1$  et  $\ell_2$  au sens de la définition 2.40, on a

$$\text{Long}(\text{Concat}_{\ell_2}(\ell_1)) = \text{Long}(\ell_1) + \text{Long}(\ell_2).$$

**Exercice 2.182.** — Quelle relation y-a-t-il entre le nombre de feuilles et le nombre de sommets internes dans un arbre binaire. La démontrer.

**Exercice 2.183.** — On appelle *cardinal* d'un arbre binaire le nombre de sommets de l'arbre.

1. Donner une définition inductive de la fonction cardinal.
2. Quelles relations peut-on établir entre le nombre de sommets d'un arbre binaire et sa hauteur. Démontrer les relations proposées.

**Exercice 2.184.** — Écrire un programme Python qui compte le nombre d'arbres binaires de hauteur  $h$ .

**Exercice 2.185.** — Proposer un exemple de définition inductive d'une partie  $\mathcal{S}$  d'un ensemble  $E$ , constituée de plusieurs axiomes et d'une seule règle d'arité non nulle, qui soit une définition ambiguë.

**Exercice 2.186.** — Une portion de la mémoire est représentée ci-dessous (contenu et adresse de chaque case mémoire).

|    |    |    |    |    |    |    |    |    |    |     |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|-----|----|----|----|----|----|
| 36 | 80 | 82 | 72 | 83 | 81 | 65 | 98 | 21 | 57 | 147 | 69 | 70 | 55 | 19 | 84 |
| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58  | 59 | 60 | 61 | 62 | 63 |

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 34 | 47 | 80 | 43 | 87 | 80 | 97 | 67 | 76 | 91 | 23 | 20 | 95 | 26 | 0  | 77 |
| 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 |

|    |    |    |    |    |    |    |    |     |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|-----|----|----|----|----|----|----|----|
| 28 | 27 | 57 | 34 | 79 | 1  | 56 | 40 | 173 | 75 | 76 | 20 | 84 | 72 | 80 | 55 |
| 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88  | 89 | 90 | 91 | 92 | 93 | 94 | 95 |

|    |    |    |    |     |     |     |     |     |     |     |     |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 9  | 95 | 0  | 7  | 24  | 95  | 28  | 9   | 68  | 3   | 5   | 49  | 8   | 32  | 20  | 92  |
| 96 | 97 | 98 | 99 | 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 |

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 93  | 16  | 2   | 20  | 91  | 91  | 67  | 12  | 91  | 47  | 51  | 55  | 28  | 65  | 36  | 31  |
| 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 |

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 76  | 4   | 0   | 52  | 2   | 41  | 27  | 90  | 18  | 29  | 20  | 7   | 63  | 70  | 46  | 28  |
| 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 |

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 23  | 34  | 26  | 24  | 97  | 23  | 28  | 80  | 3   | 11  | 18  | 27  | 98  | 56  | 27  | 14  |
| 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 |

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 54  | 52  | 88  | 0   | 14  | 93  | 46  | 23  | 39  | 62  | 41  | 45  | 64  | 70  | 81  | 55  |
| 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 |

1. Retrouver le contenu d'une liste implémentée par un tableau enregistrée à l'adresse 139. Par convention, la première case représente la taille de la liste et un tableau de longueur 12 cases démarre à la seconde case.
2. Retrouver le contenu d'une liste simplement chaînée enregistrée à l'adresse 117. Par convention, chaque maillon occupe deux cases mémoires contigües, la première contenant l'élément de la liste, la seconde un pointeur. Le pointeur nul est codé par 0.
3. Y-a-t-il des conflits de mémoire entre les deux objets ?

**Exercice 2.187.** — On suppose que les listes sont implémentées par des maillons chaînés. Décrire puis implémenter dans le langage de votre choix un algorithme inversant l'ordre d'une liste, de complexité  $O(n)$  en temps et de complexité  $O(1)$  en espace.

**Exercice 2.188.** — Une liste a été déclarée en utilisant  $n$  maillons simplement chaînés. L'entier  $n$  n'est pas connu; seul l'adresse du premier maillon est connue. La liste s'achève soit par une adresse nulle, soit par l'adresse de l'un des maillons

précédents de la liste. Déterminer en temps  $O(n)$  si la liste se termine ou si elle boucle sur elle même. Vous disposez d'une mémoire de taille constante.

**Exercice 2.189.** — Une liste est implémentée par des maillons simplement chaînés. Trouver le milieu de la liste.

(Source : question d'entretien d'embauche Yahoo)

**Exercice 2.190.** — Comment afficher une liste simplement chaînée en sens inverse en temps linéaire et mémoire constante ? (Source : question d'entretien d'embauche Facebook)

**Exercice 2.191.** — Comment supprimer un maillon d'une liste simplement chaînée ? (Vous ne connaissez qu'un pointeur vers ce maillon et non pas les pointeurs de début ou fin de la liste)

**Exercice 2.192.** — Écrire un algorithme qui dans une liste retourne sur place les éléments  $k$  par  $k$ . Exemple : pour  $k = 2$

$$(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11) \mapsto (2, 1, 4, 3, 6, 5, 8, 7, 10, 9, 11),$$

et pour  $k = 3$ ,

$$(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11) \mapsto (3, 2, 1, 6, 5, 4, 9, 8, 7, 10, 11).$$

(Source : question d'entretien d'embauche Microsoft)

**Exercice 2.193.** — Etant donné deux listes  $L_1$  et  $L_2$  simplement chaînées (vous ne connaissez donc qu'un pointeur vers le premier maillon de chaque liste),

1. Comment décider si les deux listes possèdent une partie commune ?
2. Comment trouver le point de fusion ?
3. Même question si vous êtes autorisé à modifier les listes.

(Source : question d'entretien d'embauche Qualcomm)

**Exercice 2.194 (Pile).** — On dispose d'une pile (*stack*)  $P$  implémentée par un tableau (*array*) dans l'état initial suivant :

| 0  | 1  | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|----|----|---|---|---|---|---|---|---|---|
| 17 | -2 | 3 |   |   |   |   |   |   |   |


  
P.hauteur

Dans quels états successifs se trouve la pile après les instructions suivantes ?

1. Push(P,8)
2. Push(P,Pop(P)+Pop(P))
3. Pop(P)
4. Push(P,5)

**Exercice 2.195 (Modélisation de files par des piles)**

Peut-on modéliser une file à l'aide de deux piles ? Si oui, écrire les algorithmes correspondants. On veillera à fournir des algorithmes qui conservent le temps de calcul : autrement dit, si  $O(n)$  opérations sont été effectuées sur une file classique initialement vide, alors  $O(n)$  opérations sont effectuées dans le nouveau dispositif.

**Exercice 2.196 (Permutations par des piles et des files)**

On considère une machine très simple qui ne possède qu'une pile et qui peut uniquement effectuer les opérations suivantes : **Push**, empiler un caractère de l'entrée ou **Pop**, dépiler un caractère et l'ajouter à la sortie. Un algorithme sur une telle machine se limite donc à une suite de **Push** et de **Pop**

1. Quelle est la sortie lorsque l'entrée est (1, 2, 3, 4, 5) et que la suite d'instructions est

**Push, Push, Push, Pop, Push, Pop, Pop, Push, Pop, Pop?**

2. Existe-t-il une suite d'instructions telle que à partir de l'entrée (1, 2, 3, 4, 5) la sortie soit
  - (a) (1, 2, 3, 5, 4)
  - (b) (2, 3, 5, 4, 1)
  - (c) (3, 1, 2, 5, 4)
3. Montrer qu'il est possible d'obtenir en sortie la permutation  $(p_1, \dots, p_n)$  de l'entrée  $(1, \dots, n)$  s'il n'existe pas d'indices  $i < j < k$  tels que  $p_j < p_k < p_i$ .
4. On remplace la pile par une file. Quelles permutations peut-on désormais obtenir ?

**Exercice 2.197.** — Concevoir une pile munie des opérations suivantes en temps  $O(1)$  : **push**, **pop** et **get\_min**. Les opérations **push** et **pop** empilent et dépilent des éléments issus d'un ensemble totalement ordonné tandis que **get\_min** renvoie l'élément minimum de la pile.

**Exercice 2.198.** — Concevoir une file munie des opérations suivantes en temps : **push\_rear**, **pop\_front** et **get\_min**. Les opérations **push\_rear** et **pop\_front** empilent et dépilent des éléments issus d'un ensemble totalement ordonné tandis que **get\_min** renvoie l'élément minimum de la file. Les opérations **push\_rear** et **get\_min** se feront en temps  $O(1)$ , l'opération **pop\_front** se fera en temps  $O(1)$  en moyenne (i.e. une suite de  $n$  opérations coûte  $O(n)$ ).

**Exercice 2.199.** — Construire une structure de donnée permettant d'insérer, rechercher et d'effacer des entiers en temps  $O(1)$ , sachant qu'il y a moins de  $m$  entiers, qui sont tous compris entre 0 et  $n - 1$ . Vous disposez de  $m + n + 1$  cases mémoires. *Indication : hormis un nombre  $O(1)$  de cases, vous n'avez pas la possibilité d'initialiser la mémoire. Vous devez trouver un mécanisme qui décide si une case a été écrite par vous ou contient n'importe quoi.*

(Question classique d'entretien d'embauche)

**Exercice 2.200.** — Une portion de la mémoire est représentée ci-dessous (contenu et adresse de chaque case mémoire).

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 36  | 80  | 82  | 72  | 83  | 81  | 65  | 98  | 0   | 0   | 147 | 69  | 70  | 55  | 19  | 84  |
| 48  | 49  | 50  | 51  | 52  | 53  | 54  | 55  | 56  | 57  | 58  | 59  | 60  | 61  | 62  | 63  |
| 34  | 47  | 80  | 43  | 87  | 80  | 97  | 67  | 76  | 151 | 0   | 20  | 95  | 0   | 26  | 77  |
| 64  | 65  | 66  | 67  | 68  | 69  | 70  | 71  | 72  | 73  | 74  | 75  | 76  | 77  | 78  | 79  |
| 28  | 27  | 57  | 0   | 0   | 1   | 56  | 40  | 173 | 75  | 76  | 20  | 84  | 72  | 80  | 55  |
| 80  | 81  | 82  | 83  | 84  | 85  | 86  | 87  | 88  | 89  | 90  | 91  | 92  | 93  | 94  | 95  |
| 0   | 0   | 143 | 7   | 24  | 95  | 28  | 9   | 68  | 3   | 5   | 49  | 8   | 32  | 20  | 92  |
| 96  | 97  | 98  | 99  | 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 |
| 93  | 16  | 87  | 49  | 91  | 91  | 67  | 12  | 91  | 47  | 51  | 55  | 28  | 65  | 36  | 31  |
| 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 |
| 76  | 4   | 0   | 52  | 2   | 41  | 27  | 90  | 18  | 29  | 20  | 54  | 63  | 70  | 46  | 28  |
| 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 |
| 23  | 34  | 26  | 24  | 97  | 23  | 28  | 80  | 0   | 0   | 18  | 27  | 98  | 56  | 27  | 14  |
| 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 |
| 54  | 52  | 88  | 0   | 14  | 93  | 46  | 23  | 39  | 62  | 41  | 45  | 64  | 70  | 0   | 55  |
| 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 |

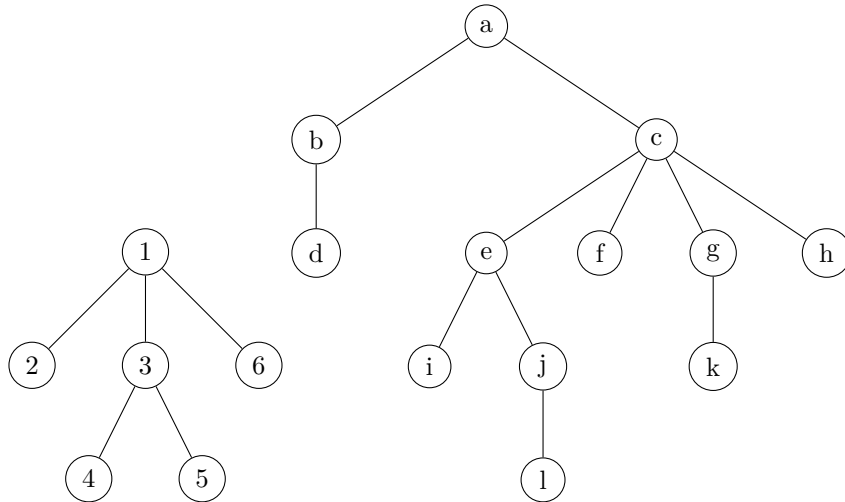
Dans chacun des cas suivants, dessiner l'arbre et son contenu.

1. Un arbre binaire complet est stocké dans un tableau de longueur 15 débutant à l'adresse 119, *i.e.* le fils gauche du nœud  $i$  est en position  $2i + 1$ , le fils droit en position  $2i + 2$ .
2. Un arbre binaire est stocké via des pointeurs à l'adresse 113. Chaque maillon est formé de trois cases mémoires contiguës contenant une donnée, l'adresse du fils gauche, l'adresse du fils droit. Le pointeur nul est noté 0.

**Exercice 2.201.** — Donner le rapport entre l'espace occupé par les données et l'espace total pour chacune des implémentations suivantes d'un arbre binaire à  $n$  sommets.

1. Tout nœud stocke une donnée, deux pointeurs fils et un pointeur père. Une donnée occupe 4 octets, chaque pointeur occupe 4 octets.
2. Seules les feuilles stockent des données. Les nœuds internes sont formés de deux pointeurs fils. Une donnée occupe 4 octets, un pointeur deux octets.

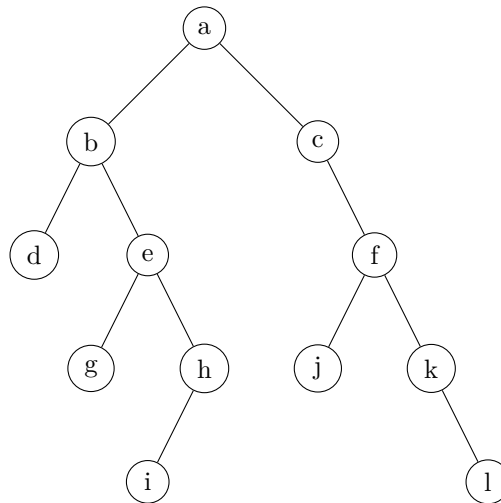
**Exercice 2.202.** — 1. Pour chacun des arbres suivants, donner la suite des nœuds obtenus par un parcours dans l'ordre préfixe et dans l'ordre postfixe



2. Représenter les deux arbres ci-dessus par des arbres binaires dans lequel le fils gauche représente le premier des fils et le fils droit le “frère cadet”.

**Exercice 2.203.** — Représenter l’expression  $(8 \times -7) + ((9 + 6)/5)$  par un arbre utilisant les constructions de la section 2.1.3.5.

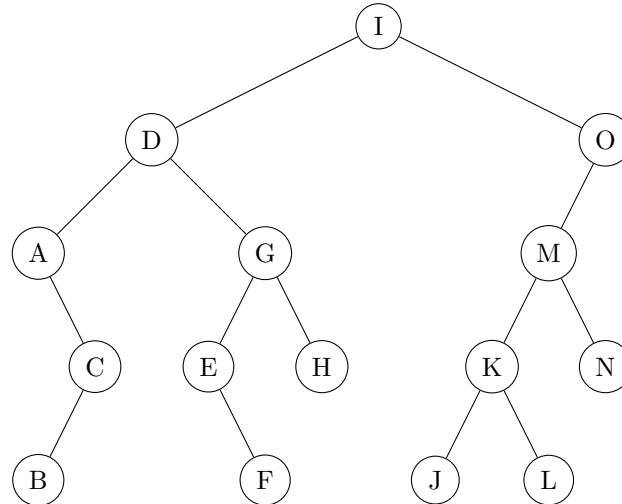
**Exercice 2.204.** — On donne l’arbre binaire suivant.



1. Donner la suite des nœuds obtenus par parcours préfixe, suffixe et infixe.
2. Donner la représentation de l’arbre par un tableau.

**Exercice 2.205.** — Le *parcours en largeur* d’un arbre consiste à énumérer les sommets d’un arbre « ligne par ligne » de gauche à droite. Écrire un algorithme qui, étant donné un arbre, construit un nouvel arbre de même forme et rajoute à chaque étiquette le numéro du sommet dans l’ordre du parcours en largeur.

**Exercice 2.206.** — On donne l'arbre binaire suivant.



1. Donner la suite des nœuds obtenus par parcours préfixe, suffixe et infixe.
2. Montrer que pour tout sommet  $x$ , les sommets du sous-arbre gauche portent une valeur inférieure à celle de  $x$  et les sommets du sous-arbre de droite portent une valeur supérieure à celle de  $x$ .

**Exercice 2.207.** — Soit  $T$  un arbre binaire à  $n$  sommets. Quelle est la plus petite et la plus grande hauteur que peut avoir  $T$  ?

**Exercice 2.208.** — Construire une structure de données permettant de stocker  $n$  données indicées  $a_0, \dots, a_{n-1}$  et munie des opérations

- i. `add( $x, i$ )`, qui additionne  $x$  à  $a_i$ .
- ii. `report( $i, j$ )` qui renvoie  $\sum_{k=i}^j a_k$ .

dont la complexité soit  $O(\log n)$  opérations élémentaires.

**Exercice 2.209 (Arbres cartésiens).** — Une suite  $x_1, x_2, \dots, x_n$  d'entiers étant donnée, on cherche à répondre rapidement à des requêtes répétées de la forme : trouver le minimum de  $x_i, x_{i+1}, \dots, x_j$  pour  $1 \leq i \leq j \leq n$  donnés.

1. concevoir une structure de données d'espace  $O(n^2)$  qui permette de répondre aux requêtes en temps  $O(1)$ .
2. concevoir une structure de données d'espace  $O(n)$  qui permette de répondre aux requêtes en temps  $O(\log n)$ .

(Question classique d'entretien d'embauche)

**Exercice 2.210.** — Dans cet exercice, on appelle *taille* d'un arbre binaire le nombre de sommets qu'il contient. On dit qu'un sommet est *lourd* si son sous-arbre gauche est de taille strictement supérieure à celle de son sommet droit. Enfin, on appelle *poids* d'un arbre le nombre de sommets lourds. Comment calculer efficacement le poids d'un arbre ?



**Exercice 2.211.** — Proposer une structure de données qui permette de gérer un dictionnaire de téléphone portable. Décrire les opérations suivantes **insérer** qui insère un nouveau mot au dictionnaire, **premier** qui renvoie le premier mot associé à une suite de chiffres donnée, **suivant** qui renvoie de le textonyme suivant d'un mot dans le dictionnaire.

**Exercice 2.212.** — Construire un arbre binaire de recherche (ABR) équilibré à partir d'une liste simplement chaînée et triée.

(Source : entretien d'embauche Google)

**Exercice 2.213.** — Une opération *concaténer* prend en entrée deux ensembles  $S_1$  et  $S_2$  stockés sous la forme d'arbres binaires de recherche et tels que pour tout élément de  $s_1 \in S_1$  et tout élément  $s_2 \in S_2$ , on a  $s_1 < s_2$ . Décrire un algorithme qui concatène  $S_1$  et  $S_2$  en un seul arbre, dont la complexité dans le pire des cas sera  $O(h)$ , où  $h$  est une borne sur la hauteur des deux arbres.

**Exercice 2.214.** — On considère un tas formé de  $n$  nœuds et dont la racine contient le plus grand élément. Étant donné un entier  $k < n$  et un élément  $x$ , suggérer un algorithme qui décide si le  $k$ -ième plus grand élément du tas est supérieur à  $x$  ou non et fonctionne en temps  $O(k)$ . Vous disposez de  $O(k)$  espaces supplémentaires en mémoire.

**Exercice 2.215.** — Que se passe-t-il quand on insère successivement les éléments de clé

5, 28, 19, 15, 20, 33, 12, 17, 10

dans une table de hachage de taille  $m = 9$  dont les collisions sont résolues par chaînage (externe), avec la fonction de hachage  $h : n \mapsto (n \bmod 9)$ .

**Exercice 2.216.** — On considère l'insertion des éléments de clé

10, 22, 31, 4, 15, 28, 17, 88, 59

dans une table de hachage de taille  $m = 11$  dont les collisions sont résolues par adressage ouvert. Que se passe-t-il pour chacune des méthodes de sondage suivantes ?

1. (sondage linéaire) Pour placer  $k$ , on teste successivement les emplacements

$$(k + i) \bmod m$$

avec  $i = 0, 1, 2, \dots$

2. (sondage quadratique) Pour placer  $k$ , on teste successivement les emplacements

$$(k + i + 3i^2) \bmod m$$

avec  $i = 0, 1, 2, \dots$

3. (sondage double) Pour placer  $k$ , on teste successivement les emplacements

$$(k + i \cdot (1 + (k \bmod m - 1))) \bmod m$$

avec  $i = 0, 1, 2, \dots$

**Exercice 2.217.** — On souhaite enregistrer des noms dans une table de hachage de longueur  $n = 26$ . Pour ce faire, on identifie les lettres de l'alphabet selon la correspondance  $A = 0, B = 1, C = 2, \dots, Z = 25$ . Nous disposons de deux fonctions de hachages :

$$h_1 : \{A, B, \dots, Z\}^+ \rightarrow \{0, \dots, 25\}$$

$$w = a_1 a_2 \dots a_r \mapsto h_1(w) = a_1$$

et

$$h_2 : \{A, B, \dots, Z\}^+ \rightarrow \{0, \dots, 25\}$$

$$w = a_1 a_2 \dots a_r \mapsto h_2(w) = a_1 + a_3 \pmod{26}$$

1. On donne la liste des noms suivantes. Compléter le tableau

| Nom           | $h_1$ | $h_2$ | Nom          | $h_1$ | $h_2$ |
|---------------|-------|-------|--------------|-------|-------|
| Alfio         | 0     | 5     | Jean-Marie   |       |       |
| Amel          | 0     | 4     | Joachim      |       |       |
| Aminata       | 0     | 8     | John         |       |       |
| Anthony       | 0     | 19    | Josée        |       |       |
| Antoinette    | 0     | 19    | Klaus-Dieter |       |       |
| Bernardine    | 1     | 18    | Manuela      | 12    | 25    |
| Boris         | 1     | 18    | Marco        | 12    | 3     |
| Charles       | 2     | 2     | Michèle      | 12    | 14    |
| Denise        | 3     | 16    | Otto         | 14    | 7     |
| Diego         | 3     | 7     | Petra        | 15    | 8     |
| Dominique     | 3     | 15    | Philippe     | 15    | 23    |
| Erik          | 4     | 12    | Roberta      | 17    | 18    |
| Eva           | 4     | 4     | Sacha        | 18    | 20    |
| Gérard        | 6     | 23    | Stefan       | 18    | 22    |
| Hichem        | 7     | 9     | Sylvaine     | 18    | 3     |
| Ian           | 8     | 21    | Thomas       | 19    | 7     |
| Jacques       | 9     | 11    | Todor        | 19    | 22    |
| Jean-François | 9     | 9     |              |       |       |

2. Peut-on utiliser l'adressage ouvert pour résoudre les collisions ?
3. Déterminer la table de hachage de la liste en fonction de  $h_1$  et de  $h_2$ . La résolution des collisions se fera par chaînage externe.
4. Y-a-t-il une table de hachage qui paraît meilleure que l'autre ? En quel sens ?
5. Est-ce que la différence est due uniquement au hasard ou voyez-vous une raison qui rend l'une des fonctions de hachage préférable pour stocker des prénoms ?

**Exercice 2.218.** — On suppose que l'on stocke un ensemble  $E$  de  $t$  clés, à valeur dans  $U$ , dans une table de hachage de longueur  $n$ . Montrer que si  $|U| \geq tn$ , alors, il existe  $E$  tel que toutes les éléments de  $E$  sont envoyés dans la même case par la fonction de hachage, autrement dit, la recherche d'un élément de  $E$  sera en  $\Theta(t)$  malgré le recours à une table de hachage.

**Exercice 2.219.** — On souhaite ranger des entiers dans une table de hachage de longueur  $n$ , où  $n$  est une puissance de 2. Lorsque une collision se produit, on se déplace

circulairement dans le tableau d'une cellule, puis de deux, puis de trois, etc. jusqu'à trouver un emplacement vide.

1. Calculer la position dans le tableau après  $i$  étapes. Expliquer le terme « sondage quadratique ».
2. Montrer que dans le pire des cas, toutes les cellules du tableau sont examinées afin de déterminer un emplacement vide.

**Exercice 2.220.** — Un multi-ensemble permet de stocker une collection de données où l'ordre des éléments n'a pas d'importance mais où un élément peut avoir une occurrence multiple.

1. Proposer un jeu de primitives pour le type abstrait `MULTIENSEMBLE` ainsi qu'une spécification formelle des fonctions.
2. Comment construire une structure de `MULTIENSEMBLE` à partir du type abstrait `DICTIONNAIRE` ?

**Exercice 2.221.** — La boutique du BDE vend  $n$  articles. Elle reçoit des commandes de la forme [produit, quantité, nom du client] et effectue les livraisons dans l'ordre d'arrivée des commandes. Il n'y a qu'un type de produit par commande. Le BDE souhaite également accéder à tout moment à la quantité commandée mais non livrée par article. On suppose qu'il n'y a jamais plus de  $m$  articles en attente de livraison avec  $m \leq n$ . Concevez une structure de donnée  $S$  de coût en mémoire  $O(m)$  possédant les opérations :

- `commande()` qui ajoute à  $S$  les champs [produit, quantité, nom du client] d'une nouvelle commande en temps moyen  $O(1)$ .
- `livraison()` qui retire de  $S$  et renvoie la commande la plus ancienne en temps moyen  $O(1)$ .
- `achalandage( $p$ )` qui renvoie la quantité de produit  $p$  commandée mais non livrée en temps moyen  $O(1)$ .

En écrire le code et l'organiser sous la forme d'une classe Python.



## CHAPITRE 3

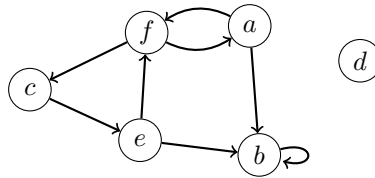
### ALGORITHMES DE GRAPHE

#### 3.1. Éléments de théorie des graphes

##### 3.1.1. Définitions. —

**Définition 3.1.** — Un *graphe orienté* est couple  $(V, E)$  où  $V$  est un ensemble fini et  $E$  est une partie de  $V \times V$ . Les éléments de  $V$  s'appellent les *sommets*; les éléments de  $E$  s'appellent les *arcs*.

**Exemple 3.2.** —  $V = \{a, b, c, d, e, f\}$   
 $E = \{ab, af, bb, ce, eb, ef, fa, fc\}$ .



**Vocabulaire 3.3.** — Soit  $G = (V, E)$  un graphe orienté.

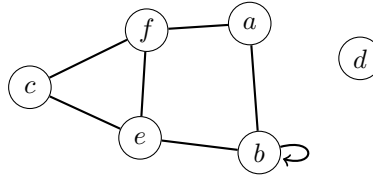
- On dit que le sommet  $u \in V$  est un *prédécesseur* du sommet  $v \in V$  si le couple  $uv$  est un arc de  $G$ .
- Pour tout sommet  $v \in V$ , on appelle *degré entrant* et on note  $d^-(v)$  le nombre de prédécesseurs de  $v$ .
- On dit que le sommet  $u \in V$  est un *successeur* du sommet  $v \in V$  si le couple  $vu$  est un arc de  $G$ .
- Pour tout sommet  $v \in V$ , on appelle *degré sortant* et on note  $d^+(v)$  le nombre de successeurs de  $v$ .
- Une suite de sommets  $x_0x_1 \cdots x_k$  est un *chemin* de longueur  $k$  si pour tout  $i \in \llbracket 0, k-1 \rrbracket$ , le couple  $x_i x_{i+1}$  est un arc de  $G$ .
- On parle de *chemin élémentaire* si, de plus, les sommets  $x_0, x_1, \dots, x_k$  sont tous distincts.

- On dit qu'un sommet  $v \in V$  est *accessible* depuis un sommet  $u$ , et on note  $u \rightsquigarrow v$ , s'il existe un chemin  $x_0, x_1, \dots, x_k$  avec  $x_0 = u$  et  $x_k = v$ .
- On parle de circuit si  $x_0x_1 \cdots x_k$  est un chemin avec  $x_k = x_0$
- On parle de circuit élémentaire si, de plus, tous les sommets  $x_0, x_1, \dots, x_{k-1}$  sont distincts.

**Définition 3.4.** — Un *graphe non orienté* est un graphe orienté  $(V, E)$  tel que pour tout arc  $uv \in E$ , le couple  $vu$  est aussi un arc.

Les éléments de  $V$  s'appellent les *sommets*; les classes d'équivalence de  $E$  pour la relation  $uv \equiv vu$  s'appellent les *arêtes*.

**Exemple 3.5.** —  $V = \{a, b, c, d, e, f\}$   
 $E = \{ab, ba, af, fa, bb, be, eb, ce, ce, cf, fc, ef, fe\}$ .



**Vocabulaire 3.6.** — Soit  $G = (V, E)$  un graphe orienté.

- Deux sommets  $u$  et  $v \in V$  sont *voisins* si la paire  $uv$  est une arête.
- Pour tout sommet  $v \in V$ , on appelle *degré* et on note  $d(v)$  le nombre de voisins de  $v$ .
- Une suite de sommets  $x_0x_1 \cdots x_k$  est une *chaîne* de longueur  $k$  si pour tout  $i \in \llbracket 0, t-1 \rrbracket$ , la paire  $x_i x_{i+1}$  est une arête de  $G$  et pour tout  $i \in \llbracket 0, t-2 \rrbracket$ , les sommets  $x_i$  et  $x_{i+2}$  sont distincts.
- On parle de *chaîne élémentaire* si, de plus, les sommets  $x_0, x_1, \dots, x_t$  sont tous distincts.
- On parle de *cycle* si  $x_0x_1 \cdots x_k$  est une chaîne avec  $x_k = x_0$
- On parle de *cycle élémentaire* si, de plus, les sommets  $x_0, x_1, \dots, x_{k-1}$  sont tous distincts.
- On dit qu'un graphe est *acyclique* s'il ne contient pas de cycle.

**Remarque 3.7.** — Dans les cas pratiques, il est utile de considérer des graphes étiquetés, les étiquettes pouvant être attachées aux sommets ou aux arcs et arêtes. En particulier, lorsque des étiquettes du graphe  $G = (V, E)$  portent des valeurs numériques  $\rho : E \rightarrow \mathbb{R}$ , on parle de *graphe pondéré* et on note  $G = (V, E, \rho)$ .

#### 3.1.1.1. Connexité. —

**Lemme 3.8.** — Dans un graphe non-orienté, la relation d'accessibilité  $\rightsquigarrow$  est une relation d'équivalence.

*Démonstration.* — En effet, soient  $u, v$  et  $w$  trois sommets de  $V$ .

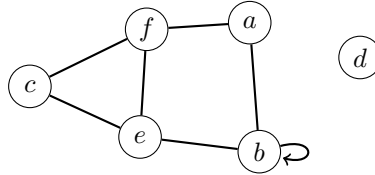
- On a  $v \rightsquigarrow v$  (il suffit de prendre un chemin de longueur nulle). Ceci montre que la relation  $\rightsquigarrow$  est *réflexive*.
- Si  $u \rightsquigarrow v$ , alors il existe une suite de sommets  $x_0x_1 \cdots x_k$  avec  $x_0 = u$ ,  $x_k = v$  et pour tout  $i \in \llbracket 0, k-1 \rrbracket$ , le couple  $x_i x_{i+1}$  est un arc de  $G$ . Mais comme  $G$  est non orienté, on a aussi que pour tout  $i \in \llbracket 0, k-1 \rrbracket$ , le couple  $x_{i+1} x_i$  est un arc de  $G$ . Aussi la suite  $x_k x_{k-1} \cdots x_1 x_0$  forme un chemin de  $v$  vers  $u$ . Donc  $u \rightsquigarrow v$ . Ceci montre que la relation  $\rightsquigarrow$  est *symétrique*.
- Si  $u \rightsquigarrow v$  et si  $v \rightsquigarrow w$ , alors il existe deux suites de sommets  $x_0x_1 \cdots x_k$  et  $y_0y_1 \cdots y_\ell$  avec d'une part  $x_0 = u$ ,  $x_k = v$  et pour tout  $i \in \llbracket 0, k-1 \rrbracket$ , le couple  $x_i x_{i+1}$  est un arc de  $G$  et d'autre part  $y_0 = v$ ,  $y_\ell = w$  et pour tout  $i \in \llbracket 0, \ell-1 \rrbracket$ , le couple  $y_i y_{i+1}$  est un arc de  $G$ . Mais alors, en concaténant les deux suites, on obtient un chemin de  $u$  vers  $w$ . Ceci montre que la relation  $\rightsquigarrow$  est *transitive*.

□

**Définition 3.9.** — Dans un graphe non orienté, les classes d'équivalence pour la relation  $\rightsquigarrow$  s'appellent les *composantes connexes*.

Un graphe non orienté est dit *connexe* s'il ne possède qu'une seule classe d'équivalence.

**Exemple 3.10.** — Dans le graphe où  $V = \{a, b, c, d, e, f\}$  et  $E = \{ab, af, bb, be, ce, cf, ef\}$ .



la relation  $\rightsquigarrow$  est satisfaite pour les sommets suivants :

| $\rightsquigarrow$ | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|--------------------|-----|-----|-----|-----|-----|-----|
| $a$                | Oui | Oui | Oui | Non | Oui | Oui |
| $b$                | Oui | Oui | Oui | Non | Oui | Oui |
| $c$                | Oui | Oui | Oui | Non | Oui | Oui |
| $d$                | Non | Non | Non | Oui | Non | Non |
| $e$                | Oui | Oui | Oui | Non | Oui | Oui |
| $f$                | Oui | Oui | Oui | Non | Oui | Oui |

Il s'ensuit qu'il y a deux composantes connexes : à savoir

$$\{a, b, c, e, f\} \quad \text{et} \quad \{d\}.$$

**Remarque 3.11.** — Pour représenter les composantes connexes d'un graphe, on peut utiliser le type « partition » que nous avons rencontré à la section ???. Nous commençons par créer une partition dans laquelle tous les sommets sont distincts.

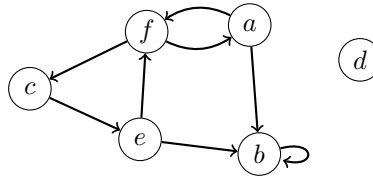
Puis, pour chaque arête  $uv$  du graphe, nous appliquons l'opération « unir » à  $u$  et à  $v$ .

Si  $G = (V, E)$  est un graphe non-orienté, la relation  $\rightsquigarrow$  n'est pas une relation d'équivalence sur  $V$  en général. En effet, la symétrie de  $\rightsquigarrow$  n'est pas garantie. On peut toutefois introduire la relation  $\longleftrightarrow$  définie par  $u \longleftrightarrow v$  s'il existe un chemin de  $u$  vers  $v$  et il existe un chemin de  $v$  vers  $u$ . La relation  $\longleftrightarrow$  est bien une relation d'équivalence.

**Définitions 3.12.** — Dans un graphe orienté, les classes d'équivalence pour la relation  $\longleftrightarrow$  s'appellent les *composantes fortement connexes*.

Un graphe non orienté est dit *fortement connexe* s'il ne possède qu'une seule classe d'équivalence.

**Exemple 3.13.** — Dans le graphe où  $V = \{a, b, c, d, e, f\}$  et  $E = \{ab, af, bb, ce, eb, ef, fa, fc\}$ ,



la relation  $\longleftrightarrow$  est satisfaite pour les sommets suivants :

| $\rightsquigarrow$ | a   | b   | c   | d   | e   | f   |
|--------------------|-----|-----|-----|-----|-----|-----|
| a                  | Oui | Oui | Oui | Non | Oui | Oui |
| b                  | Oui | Oui | Oui | Non | Oui | Oui |
| c                  | Oui | Oui | Oui | Non | Oui | Oui |
| d                  | Non | Non | Non | Oui | Non | Non |
| e                  | Oui | Oui | Oui | Non | Oui | Oui |
| f                  | Oui | Oui | Oui | Non | Oui | Oui |

Il s'ensuit qu'il y a trois composantes fortement connexes : à savoir

$$\{a, c, e, f\}, \quad \{b\} \quad \text{et} \quad \{d\}.$$

**3.1.1.2. Arbres au sens de la théorie des graphes.** — La notion d'*arbre* apparaît également en théorie des graphes. Les deux notions sont proches. Cependant, dans une structure de donnée en arbre, il y a toujours une racine ; en théorie des graphes, il n'y a pas de racine.

**Proposition 3.14.** — Soit  $G = (V, E)$  un graphe non orienté. On note  $n = |V|$  le nombre de sommets et  $m = |E|$  le nombre d'arêtes. Les affirmations suivantes sont équivalentes :

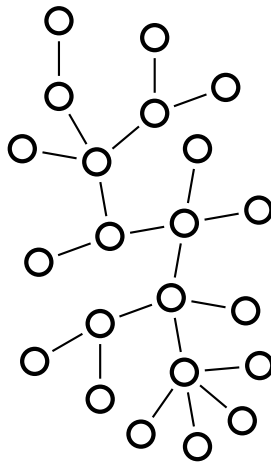
1. Le graphe  $G$  est acyclique et connexe.
2. Le graphe  $G$  est connexe et  $m = n - 1$ .



3. Le graphe  $G$  est *acyclique* et  $m = n - 1$ .
4. Le graphe  $G$  est *connexe*, mais toute suppression d'une arête à  $E$  crée deux composantes connexes.
5. Le graphe  $G$  est *acyclique*, mais tout ajout d'une arête à  $E$  crée un cycle.
6. Pour tout couple de sommets  $uv$ , il existe une et une seule chaîne entre le sommet  $u$  et le sommet  $v$ .

*Démonstration.* — TODO

□



**Définitions 3.15.** — On appelle *arbre*, au sens de la théorie des graphes, tout graphe (non orienté) connexe acyclique.

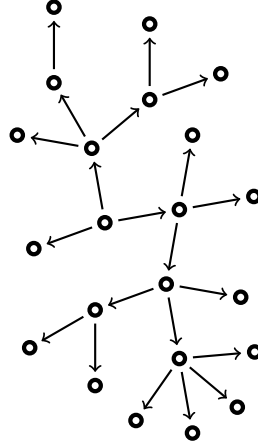
On appelle *forêt* tout graphe (non orienté) acyclique.

**Remarque 3.16.** — Chaque composante connexe est un arbre : une forêt est donc la réunion disjointe d'arbres.

On peut également s'intéresser au cas orienté. On parle d'*arborescence* lorsqu'on oriente les arêtes d'un arbre à partir d'un seul sommet (appelé *racine*).

**Définition 3.17.** — On appelle *arborescence* de racine  $r$  tout graphe orienté  $G = (V, E)$  où  $r$  est un sommet particulier de  $V$ , tel que tout sommet de  $G$  est accessible depuis  $r$  et tel que, pour tout sommet  $v \in V$ ,

$$\deg^-(v) = \begin{cases} 1 & \text{si } v \in V \setminus \{r\} \\ 0 & \text{si } v = r \end{cases}$$



Ainsi, pour tout sommet  $v \in V \setminus \{r\}$ , il existe un unique arc  $uv$  dont l'extrémité finale est  $v$  : on appelle *père de  $v$*  le sommet  $u$ . Il est facile de représenter une arborescence par la structure de donnée d'arbre.

**Lemme 3.18.** — Dans une arborescence  $G = (V, E)$  de racine  $r$ , pour tout sommet  $v \in V$ , il existe un unique chemin de  $r$  vers  $v$ .

*Démonstration.* — Si  $v$  est distinct de  $r$ ,  $v$  ne possède qu'un seul prédécesseur. On peut initialiser la construction d'un chemin se terminant en  $v$ . Mais répétant le procédé jusqu'à rencontrer le sommet  $r$ , ce même fait permet de construire un unique chemin de longueur donnée et de destination  $v$ . Or il existe au moins un chemin de  $r$  vers  $v$  dans l'arborescence. Donc ce chemin est unique.  $\square$

**3.1.2. Représentations par des structures de donnée.** — Il existe trois grandes manières de représenter les graphes orientés, qui portent toutes sur la manière de stocker les arêtes et qui se distinguent selon la finesse avec laquelle sont regroupés les arcs. Les graphes non orientés se stockent en général comme les graphes orientés : les arêtes sont alors encodées sous la forme de deux arcs, un arc par direction.

*Ensemble d'arcs.* — Stocker l'ensemble des arcs dans une seule collection : ensemble d'arcs (pour des graphes simples) ou dictionnaire associant arcs et leur étiquette (pour des graphes dont les arêtes sont étiquetées).

*Liste d'adjacence.* — Associer à chaque sommet  $v$  une collection qui stocke l'ensemble des arcs ayant pour origine le sommet  $v$  ou un dictionnaire associant arcs et leur étiquette.

En général, on parle dans ce cas de *représentation par liste d'adjacence*. En effet, il est souvent commode de numérotter les sommets entre 0 et  $n - 1$ , où  $n$  est le nombre de sommets, et de regrouper les successeurs de chaque sommet dans une liste chaînée.

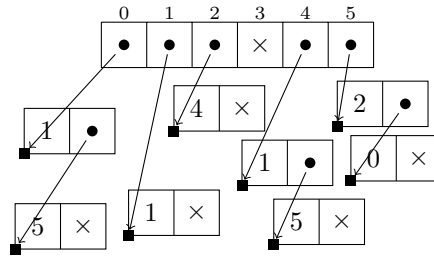
LISTING 3.1. Graphes avec listes d'adjacence en C

```

772 typedef struct node node_t;
773 struct node {
774     int head;
775     int weight;
776     node_t *next;
777 };
778
779 typedef struct graph graph_t;
780 struct graph {
781     int N;
782     node_t **neighbours;
783 };

```

Pour le graphe de l'exemple 3.2, on obtient la configuration suivante



*Matrice d'adjacence.* — Associer à chaque couple de sommets  $u$  et  $v$  l'arc et ses caractéristiques (s'il existe).

En général, on parle dans ce cas de *représentation par matrice d'adjacence*. En effet, il est souvent commode de numéroter les sommets entre 0 et  $n - 1$  et d'indiquer dans un tableau bidimensionnel la matrice d'adjacence du graphe

**Définition 3.19.** — On appelle *matrice d'adjacence* d'un graphe  $G = (V, E)$ , où  $n$  est le nombre de sommets et

$$V = \{v_0, v_1, \dots, v_{n-1}\}$$

la matrice  $((M_{i,j}))_{0 \leq i,j < n}$  telle que

$$M_{i,j} = \begin{cases} 1 & \text{si } v_i v_j \in E \\ 0 & \text{sinon} \end{cases}$$

Pour le graphe de l'exemple 3.2, on obtient la matrice suivante

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

La représentation par des listes d'adjacence est avantageuse lorsque le graphe n'est pas très dense. En effet, la complexité en espace est  $O(n + m)$ . La représentation par des matrices d'adjacence est plus lourde en mémoire, avec une complexité en espace en  $O(n^2)$ . Elle est avantageuse lorsque l'opération de vérification de l'appartenance d'une arête est effectuée souvent. Par contre, pour itérer sur l'ensemble des voisins d'un sommet, une complexité en  $O(n)$  est requise, même s'il n'y a qu'un faible nombre de voisins.

**Remarque 3.20.** — Traditionnellement, la plupart des manuels d'algorithmiques font l'hypothèse que les sommets sont numérotés de 0 à  $n - 1$ . C'est une hypothèse raisonnable à faire quand on programme dans un langage de programmation où la bibliothèque standard offre peu de possibilités. Dans les langages modernes, les listes, ensembles et dictionnaires sont suffisamment faciles à manipuler et sont en général itérables, si bien qu'il serait dommage de toujours se ramener à des indices entiers dans des tableaux statiques qu'on gère soi-même à la main.

**Remarque 3.21.** — Tout au long de ce chapitre, nous faisons l'hypothèse qu'une entité centrale connaît l'ensemble du graphe et que ce graphe n'évolue pas. Cette hypothèse est suffisante dans de nombreux cas. Néanmoins il faut garder à l'esprit que, dans certaines applications, la connaissance d'un graphe est décentralisée et partielle : typiquement, dans les applications à des problèmes de réseau, un sommet du graphe ne connaît que ses voisins immédiats. Par ailleurs, en fonction des défaillances de machines ou des serveurs éteints, le graphe ne reste jamais le même.

### 3.2. Parcourir un graphe

À l'image de la définition 2.106 relative au parcours d'arbres, parcourir un graphe désigne le fait de visiter l'ensemble des sommets du graphe, en vue de leur faire éventuellement subir un traitement, en se déplaçant de voisins en voisins au départ d'un premier sommet initial.

En général, dans un parcours de graphe, il est nécessaire de noter quels sommets ont déjà été rencontrés pour ne pas les ré-examiner plusieurs fois. On utilise l'expression *traverser l'arc  $uv$*  pour parler du fait que l'on s'intéresse au sommet  $v$  à partir du sommet  $u$ . Il est parfois même utile d'appeler, pour tout sommet  $v$  distinct du sommet initial,  $\text{père}(v)$  le premier sommet par lequel le sommet  $v$  a été découvert.

**Définition 3.22.** — On appelle *arborescence d'un parcours de graphe*  $G = (V, E)$  au départ du sommet  $r$  le graphe  $(V', E')$  où  $V'$  est l'ensemble des sommets qui ont été examinés au cours du parcours de graphe et  $E'$  est l'ensemble des arcs  $uv \in E$  tels que  $u = \text{père}(v)$ .

**Proposition 3.23.** — Soit  $(V', E')$  l'arborescence d'un parcours du graphe  $G = (V, E)$  au départ du sommet  $r$ , alors  $V'$  coïncide avec l'ensemble des sommets accessibles dans  $G$  depuis le sommet  $r$ .

*Démonstration.* —

□

**Application 3.24.** — Soient  $G = (V, E)$  un graphe,  $u$  et  $v$  deux sommets de  $G$ . Pour vérifier que  $v$  est accessible depuis  $u$ , il suffit de calculer un parcours de graphe depuis  $u$  et vérifier si  $v$  apparaît au cours du parcours. Pour calculer la composante connexe de  $u$ , il suffit de calculer un parcours de graphe depuis  $u$ .

**Définition 3.25.** — Étant donné un parcours de graphe au départ du sommet  $r$ , on appelle *numérotation préfixe* des sommets accessibles la numérotation obtenue lorsqu'on attribue un numéro à chaque sommet rencontré lors de sa première rencontre.

**3.2.1. Le parcours en profondeur.** — Le parcours en profondeur, en anglais *depth first search* (ou *DFS*), est l'un des deux grands modes d'exploration d'un graphe. Il consiste à se déplacer systématiquement vers un voisin non encore rencontré.

**3.2.1.1. Présentation récursive.** — La présentation la plus simple du parcours en profondeur s'appuie sur un programme récursif.

---

**Algorithme 3.1 : Parcours en profondeur (DFS)**

---

**Entrées :** Graphe  $G = (V, E)$ , sommet  $s$

**Résultat :** Visite des sommets accessibles depuis  $s$

```

1 Marquer  $s$ 
2 pour  $w$  voisin de  $s$  non encore marqué faire
3   |   père( $w$ )  $\leftarrow s$ 
4   |   DFS( $G, w$ )

```

---

Cet algorithme peut s'implémenter comme suit.

LISTING 3.2. Parcours en profondeur récursif

```

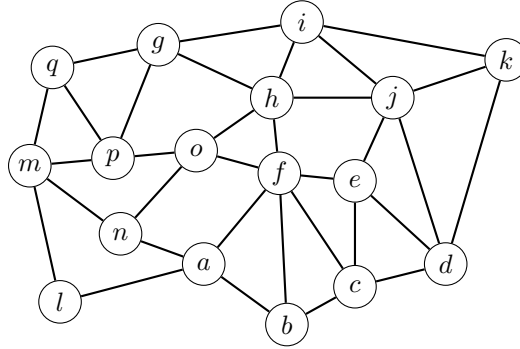
784 class Graph():
785     # Les sommets sont numérotés de 0 à n-1
786     # Les arcs sont stockés par listes de voisins
787     def __init__(self, n):
788         self.neighbors = [[] for _ in range(n)]
789     def add_edge(self, u, v):
790         self.neighbors[u].append(v)
791         self.neighbors[v].append(u)
792     def dfs(self, r, parent=None):
793         if parent is None:
794             parent = dict({r:-1})
795         for u in self.neighbors[r]:
796             if u not in parent:
797                 parent[u] = r
798                 self.dfs(u, parent)
799         return parent

```

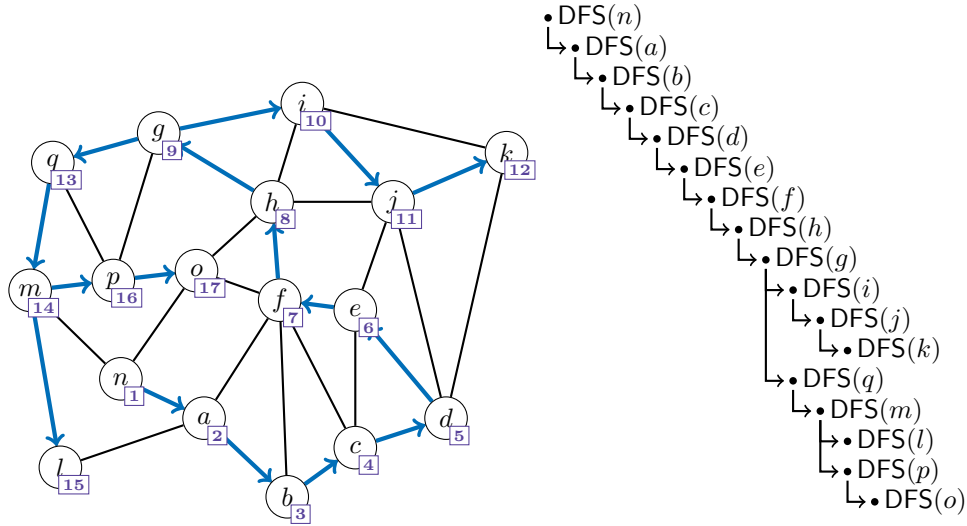
**Remarque 3.26.** — Cette implémentation fait le choix de construire l'arborescence parent sous la forme d'un dictionnaire global. Nous pourrions aussi décentraliser cette information et la stocker sous forme d'attribut au niveau de chaque sommet.

Cette présentation récursive de l'algorithme dissimule la gestion de la mise en attente de sommets dans la pile d'appel des fonctions.

**Exemple 3.27.** — Voici un graphe que l'on souhaite parcourir au départ du sommet  $n$ .



Le parcours en profondeur au départ du sommet  $n$  conduit à l'arborescence suivante. Les numéros correspondent à la numérotation préfixe, c'est-à-dire l'ordre dans lequel les sommets ont été découverts. Nous représentons également comment les appels récursifs s'enchaînent.



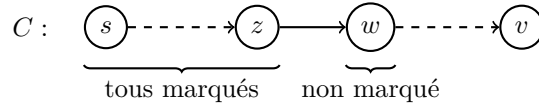
**Proposition 3.28.** — À l'issue de l'exécution de  $\text{DFS}(G, s)$ , l'ensemble des sommets marqués  $M$  est l'ensemble des sommets accessibles depuis  $s$  dans  $G$ . De plus, l'ensemble  $A$  des arcs de la forme père( $v$ ) $v$  forme une arborescence  $(M, A)$  de racine  $s$ .

*Démonstration.* — On observe qu'un sommet n'est marqué qu'une seule fois au plus et que l'affectation  $\text{père}(v)$  n'est fait qu'au plus une fois par sommet  $v$ . En outre, sauf pour l'appel initial, quand un sommet  $v$  est marqué au début de  $\text{DFS}(G, v)$ , la fonction appelante vient d'attribuer une valeur à  $\text{père}(v)$ , qui est un sommet déjà marqué. Ainsi, on est bien dans la situation où, dans  $(M, A)$ , tous les sommets sont accessibles depuis la racine et

$$\deg_{(M,A)}^-(v) = \begin{cases} 1 & \text{si } v \in M \setminus \{s\} \\ 0 & \text{si } v = s \end{cases}$$

ce qui caractérise le fait que  $(M, A)$  est une arborescence. D'après le lemme 3.18, il existe un chemin entre  $s$  et tout sommet de  $M$ , donc  $M$  est bien un ensemble de sommets accessibles depuis  $s$ .

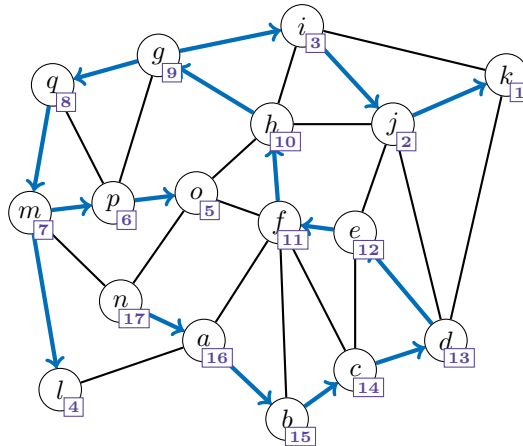
Montrons enfin que tout sommet accessible est dans  $M$ . Supposons, par l'absurde, qu'il existe un sommet  $v$  accessible depuis  $s$  non marqué par  $\text{DFS}(G, v)$ . Notons  $C$  un chemin du sommet  $s$  vers le sommet  $v$ . Appelons  $w$  le premier sommet non marqué que l'on rencontre le long de  $C$  (ce sommet existe car  $v$  n'est pas marqué et ne peut être  $s$ ). Notons encore  $z$  son prédécesseur dans  $C$ .



Mais alors, comme  $z$  est marqué,  $\text{DFS}(G, v)$  aurait dû s'intéresser à  $w$  comme voisin et  $w$  aurait dû être marqué. Contradiction!  $\square$

**Définition 3.29.** — Dans le parcours de graphe en profondeur au départ du sommet  $r$ , on appelle *numérotation postfixe* des sommets accessibles la numérotation obtenue lorsqu'on attribue un numéro à chaque sommet rencontré lors de sa dernière rencontre, c'est-à-dire qu'on numérote le sommet  $v$  au moment où l'appel  $\text{DFS}(v)$  se termine.

**Exemple 3.30.** — Nous poursuivons l'exemple 3.27 et attribuons les numéros postfixes.



**3.2.1.2. Présentation itérative.** — Certains langages de programmation fonctionnent mal avec la programmation récursive. C'est le cas de Python. Il est alors nécessaire de gérer soi-même la pile contenant les sommets à traiter.

Voici un exemple d'implémentation. Il faut noter que, pour garantir que l'ordre des sommets soit le même, nous empilons les voisins d'un sommet du dernier au premier.

LISTING 3.3. Parcours en profondeur itératif (suite de ??)

```

800 class Graph():
801     # [...] Reprendre le code de la classe existante !
802     def iterative_dfs(self, r):
803         stack = [(-1,r)]
804         # La pile contient les arcs
805         parent = dict()
806         while stack:
807             u,v = stack.pop()
808             if v not in parent:
809                 parent[v] = u
810                 for w in reversed(self.neighbors[v]):
811                     stack.append((v,w))
812         return parent

```

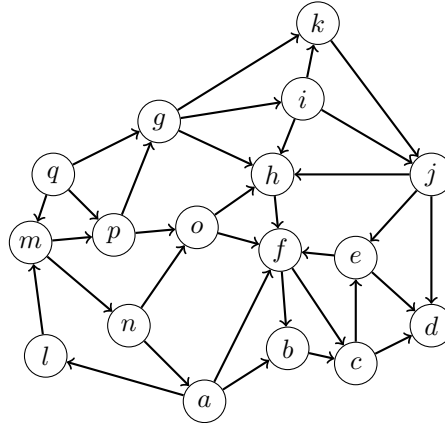
**3.2.1.3. Application à la détection de circuit.** — Le parcours en profondeur permet de repérer l'existence de circuits dans un graphe orienté.

**Définition 3.31.** — Soit  $G = (V, E)$  un graphe orienté et  $s$  un sommet de  $V$ . On suppose que tous les sommets de  $V$  sont accessibles depuis  $s$  et on note  $(V, A)$  l'arborescence du parcours DFS( $G, s$ ). On distingue alors quatre types d'arcs :

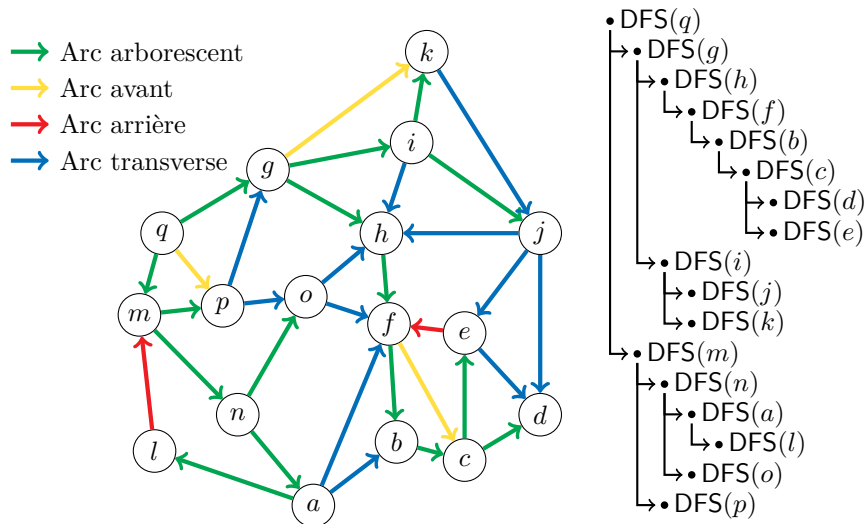
- les arcs de l'arborescence de parcours : ce sont les arcs de  $A$ , de la forme père( $v$ ) $v$ .
- les arcs avant : ce sont les arcs de la forme  $uv$  où  $v$  est un descendant de  $u$  dans  $(V, A)$
- les arcs arrière : ce sont les arcs de la forme  $uv$  où  $u$  est un descendant de  $v$  dans  $(V, A)$
- les arcs transverses : ce sont les arcs restants.

**Exemple 3.32.** — Établissons la classification sur le graphe  $G$  suivant dans le parcours DFS( $G, q$ ) :





On obtient les parcours et la classification suivante.



**Proposition 3.33.** — Un graphe orienté  $G = (V, E)$ ,  $s$  un sommet de  $V$ . On suppose que tout sommet de  $V$  est accessible depuis  $s$ . Alors le graphe  $G$  est acyclique si et seulement si le parcours  $\text{DFS}(G, s)$  ne rencontre pas d'arc arrière.

*Démonstration.* — Nous raisonnons sur les contraposées. Si  $\text{DFS}(G, s)$  rencontre un arc arrière, disons l'arc  $uv$ , alors, d'une part  $u$  est accessible depuis  $v$  dans l'arborescence de parcours, donc il existe un chemin  $v \rightsquigarrow u$ . Mais, ce chemin peut être fermé avec l'arc  $uv$  pour former un cycle.

Réciproquement, supposons que  $G$  possède un cycle  $C$ . Notons  $x_1, x_2, \dots, x_t$  les sommets de ce cycle, où  $x_1$  est le premier sommet détecté par le parcours. Nous notons que le sommet  $x_t$  est accessible depuis  $x_1$ . Donc  $\text{DFS}(G, x_1)$  doit rencontrer  $x_t$ . De plus, quand  $x_t$  est rencontré, l'arc  $x_t x_1$  sera testé et sera déclaré arc arrière.  $\square$

Le fait qu'un arc  $uv$  est un arc arrière peut se caractériser comme suit : au moment où l'arc  $uv$  est testé par le parcours en profondeur, le sommet  $u$  a déjà été rencontré, mais l'appel  $\text{DFS}(u)$  n'est pas terminé. Ceci conduit au code suivant

LISTING 3.4. Détection de circuit

```

813 # [...] Reprendre le code de la classe Graph existante !
814 def hascycle(self, r, seen=None, done=None):
815     if seen is None:
816         seen = set({r})
817         done = set()
818     for u in self.neighbors[r]:
819         if u not in seen:
820             seen.add(u)
821             if self.hascycle(u, seen, done):
822                 return True
823         elif u not in done:
824             return True
825     done.add(u)
826     return False

```

**Remarque 3.34.** — Dans le cas orienté, la notion d'arc avant ou d'arc transverse n'a plus lieu d'être. Tester l'acyclicité revient simplement à vérifier que le parcours de graphe ne retrouve pas un sommet déjà vu.

**3.2.1.4. Application au tri topologique.** — Nous cherchons à numéroter les sommets d'un graphe orienté tout en respectant le sens des arcs.

**Proposition 3.35.** — Soit  $G = (V, E)$  un graphe orienté. La relation d'accessibilité  $\rightsquigarrow$  est une relation ordre si et seulement si le graphe  $G$  ne contient pas de circuit.

*Démonstration.* — Nous avons déjà démontré que la relation  $\rightsquigarrow$  est toujours réflexive et transitive (cf lemme 3.8). Il nous suffit donc de démontrer que la relation  $\rightsquigarrow$  est antisymétrique si et seulement si le graphe  $G$  ne contient pas de circuit, ce que nous faisons dans les deux paragraphes suivants.

Supposons que la relation  $\rightsquigarrow$  est antisymétrique. S'il existait un circuit dans  $G$ , on pourrait considérer deux sommets  $u$  et  $v$  distincts le long de ce circuit. Mais alors on aurait  $u \rightsquigarrow v$  et  $v \rightsquigarrow u$  (en utilisant des morceaux du circuit pour construire des chemins), ce qui n'est pas possible.

Supposons que le graphe  $G$  ne contient pas de circuit. Si  $u \rightsquigarrow v$  et  $v \rightsquigarrow u$  avec  $u \neq v$ , alors en concaténant les chemins, on obtiendrait un circuit non trivial, ce qui n'est pas possible.  $\square$

**Lemme 3.36.** — Dans un graphe orienté acyclique, il existe toujours un sommet de degré entrant nul et un sommet de degré sortant nul.

*Démonstration.* — Soit  $v$  un sommet quelconque d'un graphe orienté acyclique  $G$ . Supposons avoir construit un chemin élémentaire  $x_k x_{k-1} \cdots x_1 x_0$  de longueur  $k$  se

terminant en  $x_0 = v$ , alors  $x_k$  est de degré entrant nul, soit il possède un prédécesseur, disons  $x_{k+1}$ . Comme le graphe est acyclique, le chemin  $x_{k+1}x_k \cdots x_1x_0$  est aussi élémentaire. Si  $G$  ne possède aucun sommet de degré entrant nul, nous pourrions construire à partir du chemin  $v$  de longueur 0 un chemin élémentaire de longueur aussi grande que souhaitée, ce qui est impossible car l'ensemble de sommets  $V$  est fini. Aussi, le graphe  $G$  possède un sommet de degré entrant nul.

La preuve est identique pour le degré sortant.  $\square$

**Proposition 3.37.** — Soit  $G = (V, E)$  un graphe orienté acyclique avec  $n$  sommets. Il existe une manière de numéroté les sommets

$$V = \{v_1, v_2, \dots, v_n\}$$

de sorte que pour tout couple d'indices  $i$  et  $j \in \llbracket 1, n \rrbracket$  distincts, si  $v_i v_j$  est un arc de  $E$ , alors  $i < j$ .

*Démonstration.* — Nous raisonnons par récurrence. Le résultat est vrai pour les graphes à un seul sommet. Si un graphe  $G = (V, E)$  possède plusieurs sommets, d'après, le lemme 3.36, il contient un sommet, disons  $v_n$  de degré sortant nul. Soit  $G' = (V', E')$  le graphe obtenu à partir de  $G$  en supprimant  $v_n$  et les arcs qui se terminent en  $v_n$ . Par hypothèse de récurrence, nous pouvons numéroté les sommets de  $G'$

$$V' = \{v_1, v_2, \dots, v_{n-1}\}$$

de sorte que pour tout couple d'indices  $i$  et  $j \in \llbracket 1, n \rrbracket$  distincts, si  $v_i v_j$  est un arc de  $E'$ , alors  $i < j$ . Mais alors, en conservant la même numérotation, on a aussi dans  $G$  que pour tout couple d'indices  $i$  et  $j \in \llbracket 1, n \rrbracket$  distincts, si  $v_i v_j$  est un arc de  $E$ , alors  $i < j$ .  $\square$

**Remarque 3.38.** — Nous pourrions convertir la preuve de cette proposition en un algorithme : on obtient l'algorithme de Kahn.

LISTING 3.5. Algorithme de Kahn

```

827 from collections import deque
828
829 class Graph():
830     def __init__(self):
831         self.successors = dict({})
832     def add_edge(self, u, v):
833         if u in self.successors:
834             self.successors[u].append(v)
835         else:
836             self.successors[u] = [v]
837         if v not in self.successors:
838             self.successors[v] = []
839     def degree(self):
840         deg = dict()
841         for u in self.successors:

```

```

842         deg[u] = 0
843     for u in self.successors:
844         for v in self.successors[u]:
845             deg[v] += 1
846     return deg
847 def kahn(self):
848     count = 0
849     numbering = []
850     deg = self.degree()
851     waiting = deque()
852     for u in deg:
853         if deg[u] == 0:
854             waiting.append(u)
855     while waiting:
856         u = waiting.popleft()
857         numbering.append(u)
858         for v in self.successors[u]:
859             deg[v] -= 1
860             if deg[v] == 0:
861                 waiting.append(v)
862     return numbering

```

**Définition 3.39.** — Soit  $G = (V, E)$  un graphe orienté acyclique avec  $n$  sommets. On appelle *numérotation topologique* ou *tri topologique* des sommets de  $G$  toute numérotation

$$V = \{v_1, v_2, \dots, v_n\}$$

telle que pour tout couple d'indices  $i$  et  $j \in \llbracket 1, n \rrbracket$  distincts, si  $v_i v_j$  est un arc de  $E$ , alors  $i < j$ .

**Remarque 3.40.** — Plus généralement, on peut observer que pour tout couple de sommets distincts  $v_i v_j$ , si  $v_i \rightsquigarrow v_j$ , alors  $i < j$ . Effectuer un tri topologique revient donc à calculer un raffinement de l'ordre donné par la relation d'accessibilité  $\rightsquigarrow$  en un ordre total. En mathématique, on parle aussi de *linéarisation de l'ordre*  $\rightsquigarrow$ .

L'algorithme de Tarjan permet de construire une numérotation topologique à partir de plusieurs parcours en profondeur. Il consiste à démarrer un DFS au départ de tout sommet non encore numéroté et à attribuer le numéro postfixe aux sommets rencontrés, sans réinitialiser le compteur d'un DFS au suivant. Enfin, l'ordre des numéros est renversé.

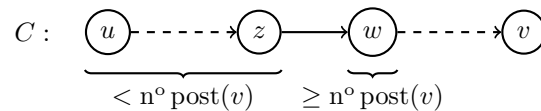
**Algorithme 3.2 :** Tri topologique (Algorithme de Tarjan)**Entrées :** Graphe  $G(V, E)$ **Sorties :** Numérotation topologique des sommets

- 1 Initialiser un compteur
- 2 **tant que**  $\exists$  sommet  $v$  non numéroté **faire**
- 3     Lancer un DFS au départ de  $v$
- 4     Attribuer le numéro postfixe aux sommets rencontrés  
       (sans réinitialiser le compteur d'un DFS au suivant)
- 5 Renverser l'ordre des numéros

**Proposition 3.41.** — *L'algorithme de Tarjan est correct par rapport à sa spécification.*

*Démonstration.* — Soient  $u$  et  $v$  deux sommets du graphe. Supposons que  $v$  est un sommet accessible depuis le sommet  $u$ , il nous faut démontrer que le numéro attribué à  $u$  est plus petit que celui de  $v$ , ou encore que le numéro postfixe de  $u$  est plus grand que celui de  $v$ .

Raisonnons par l'absurde : supposons que  $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$ . Notons  $C$  un chemin du sommet  $u$  au sommet  $v$ . Appelons  $w$  le premier sommet rencontré sur  $C$  tel que le numéro postfixe de  $w$  est plus grand que celui de  $v$ . Notons encore  $z$  son prédécesseur dans  $C$ .



On doit avoir  $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(v) \leq \text{n}^\circ \text{post}(w)$ . Mais alors, comme  $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(w)$ , lorsque le numéro postfixe est donné à  $z$ , l'algorithme n'a pas encore pu considérer le sommet  $w$ . Or  $w$  est un successeur de  $z$  et aurait dû être considéré dans l'appel  $\text{DFS}(z)$  avant que le numéro  $z$  soit donné, ce qui est une contradiction!  $\square$

LISTING 3.6. Tri topologique (Algorithme de Tarjan)

```

863 # [...] Reprendre le code de la classe Graph existante !
864 def _numbering_dfs(self, r, counter, visited, numbering):
865     if r in visited:
866         return counter
867     visited.add(r)
868     for u in self.neighbors[r]:
869         if u not in visited:
870             counter = self._numbering_dfs(u, counter, visited,
871                                         numbering)
872     numbering[r] = counter
873     return (counter - 1)
874 def topological_sort(self):

```

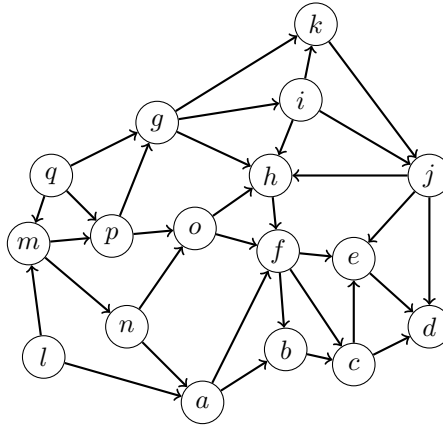
```

874     visited = set()
875     counter = len(self.neighbors) - 1
876     numbering = dict()
877     for v in range(len(self.neighbors)):
878         counter = self._numbering_dfs(v, counter, visited,
879                                     ↪ numbering)
880     return numbering

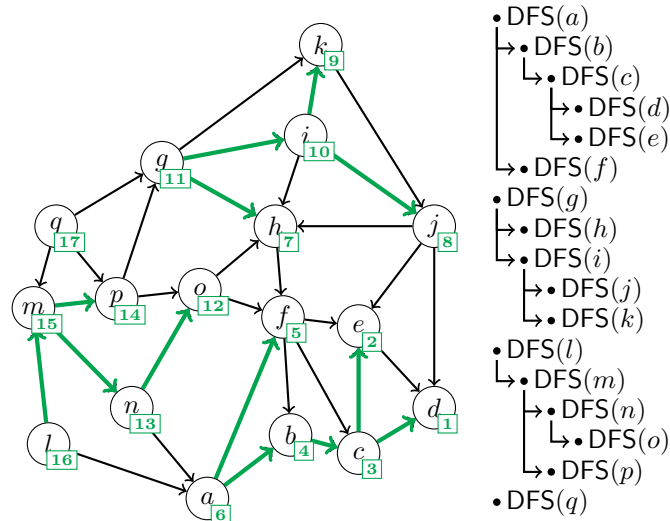
```

La complexité de l'algorithme de Tarjan est  $O(|V| + |E|)$  si le graphe est représenté par des listes d'adjacences. Elle est  $O(n^2)$  si le graphe est représenté par des matrices d'adjacence.

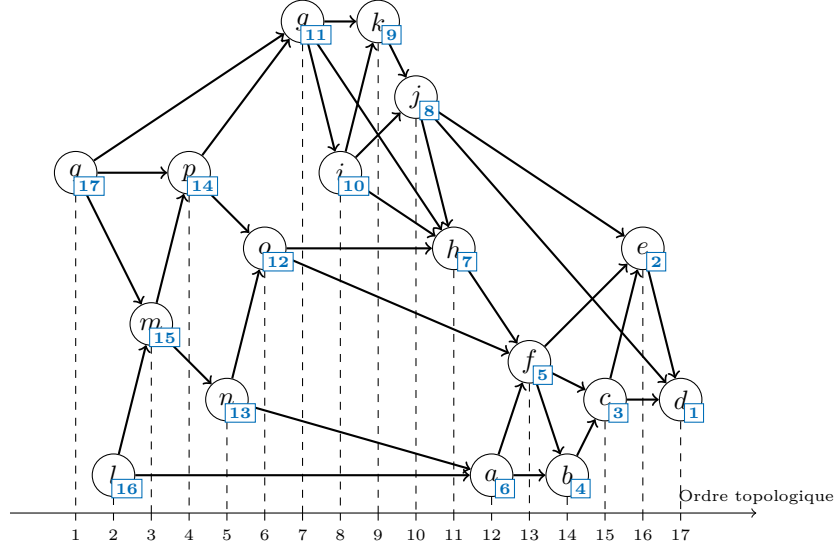
**Exemple 3.42.** — Voici un exemple d'exécution sur le graphe



On obtient les parcours et les numéros postfixes suivants



L'ordre topologique que nous avons calculé est alors le suivant.



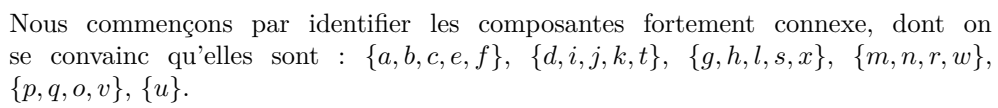
**3.2.1.5. Composantes fortement connexes.** — La notion de composante fortement connexe a été introduite par la définition 3.12 à partir de la relation  $\rightsquigarrow$  qui est une relation d'équivalence. Voyons comment calculer ces composantes fortement connexes.

**Définition 3.43.** — On appelle *graphe des composantes fortement connexes* le graphe quotient, qui a pour sommets l'ensemble des composantes fortement connexes, c'est-à-dire l'ensemble des classes d'équivalence  $\bar{V} = V / \rightsquigarrow$  et qui a pour arcs l'ensemble

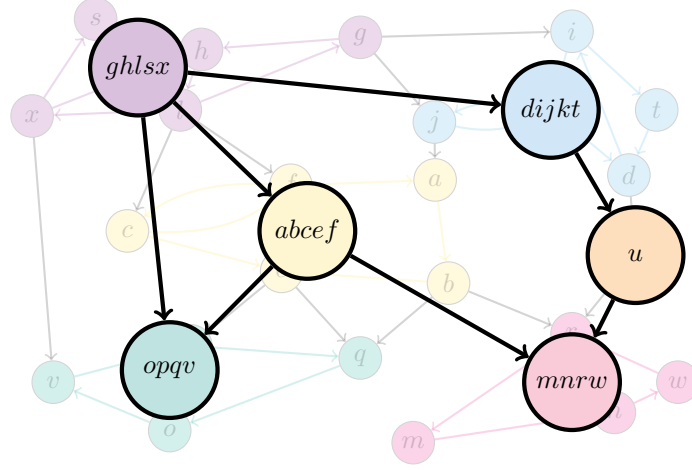
$$\bar{E} = \{(\bar{u}, \bar{v}) \in \bar{V} \times \bar{V}; (u, v) \in E\}.$$

Autrement, on obtient le graphe des composantes fortement connexes en fusionnant les sommets qui appartiennent à la même composante et en laissant les arcs aller d'une composante à une autre.

**Exemple 3.44.** — Considérons le graphe







**Théorème 3.45.** — *Le graphe des composantes fortement connexe est un graphe orienté acyclique.*

*Démonstration.* — Supposons qu'il existe un circuit dans  $\overline{G} = (\overline{V}, \overline{E})$ , que nous notons  $X_0 X_1 \cdots X_k$  avec  $X_i \in \overline{V}$  pour tout  $i$  compris entre 0 et  $k$  et  $X_0 = X_k$ . Par définition de  $\overline{G}$ , il existe un arc  $x_i y_{i+1}$  dans  $G$  avec  $x_i \in X_i$  et  $y_{i+1} \in X_{i+1}$ . De plus, comme  $y_i$  et  $x_i$  appartiennent à la même composante fortement connexe, il existe un chemin  $y_i \rightsquigarrow x_i$  dans  $G$ . Mais alors, on peut fabriquer dans  $G$  le cycle suivant :

$$x_0 \underbrace{y_1 \rightsquigarrow x_1}_{\in X_1} \underbrace{y_2 \rightsquigarrow x_2}_{\in X_2} y_3 \cdots x_{k-2} \underbrace{y_{k-1} \rightsquigarrow x_{k-1}}_{\in X_{k-1}} \underbrace{y_k \rightsquigarrow x_0}_{\in X_0 = X_k}.$$

Mais ceci montre que tous les sommets sont dans la même composante connexe, donc que  $X_0 = X_1 = \cdots = X_k$ , autrement dit que le circuit initial n'en est pas un.  $\square$

**Définition 3.46.** — On appelle *graphe transposé* et on note  $G^\top$  le graphe  $G^\top = (V, E^\top)$  où

$$E^\top = \{vu; uv \in E\}.$$

L'algorithme suivant calcule les composantes fortement connexes. Il commence par numéroter les sommets d'une manière proche de l'algorithme que nous avons vu pour le tri topologique.

---

**Algorithme 3.3 :** Algorithme de Kosaraju - Sharir
 

---

**Entrées :** Graphe orienté  $G = (V, E)$ 
**Résultat :** Composantes fortement connexes

- 1 **tant que**  $\exists$  sommet non numéroté **faire**
  - 2     Lancer un DFS depuis un sommet quelconque et  
       numéroter les sommets en ordre postfixe en conservant  
       le même compteur.
  - 3 **tant que**  $\exists$  sommet dans  $G^\top$  **faire**
  - 4     Lancer un DFS dans  $G^\top$  depuis le sommet de plus haut  
       numéro et supprimer les sommets visités de  $G^\top$ .
  - 5 Chaque DFS de la phase 2 fournit une composante.
- 

Pour analyser cet algorithme, nous notons, pour toute composante fortement connexe  $C$ ,

$$n^\circ(C) = \max_{u \in C} n^\circ(u)$$

où  $n^\circ(u)$  est le numéro attribué par l'algorithme 3.3 dans la première phase.

**Lemme 3.47.** — Soient  $C$  et  $C'$  deux composantes fortement connexes, si  $C'$  est accessible depuis  $C$ , alors  $n^\circ(C) > n^\circ(C')$ .

*Démonstration.* — La preuve du lemme est identique à la preuve de correction de l'algorithme de Tarjan pour la numérotation topologique. On raisonne dans le graphe quotient et on note que le numéro attribué à une composante  $C$  est donné par le premier des appels DFS( $v$ ) où  $v \in C$ .  $\square$

**Théorème 3.48.** — L'algorithme 3.3 est correct par rapport à sa spécification.

*Démonstration.* — Il est clair que l'algorithme fournit une partition des sommets. Il suffit de voir que cette partition coïncide avec les composantes fortement connexes.

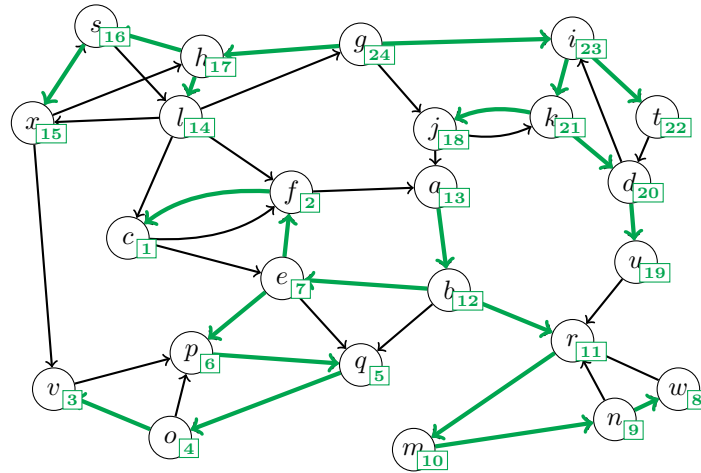
Nous raisonnons par récurrence sur le nombre de DFS lancés lors de la phase 2.

Soit  $u_0$  le sommet de plus grand numéro. Montrons que  $DFS_{G^\top}(u_0)$  renvoie la composante fortement connexe  $\overline{u_0}$  de  $u_0$ . Si le sommet  $v$  appartient à la composante fortement connexe  $\overline{u_0}$ , alors  $u_0$  est accessible depuis  $v$  dans  $G$  donc  $v$  est accessible depuis  $u_0$  dans  $G^\top$  et apparaît dans  $DFS_{G^\top}(u_0)$ .

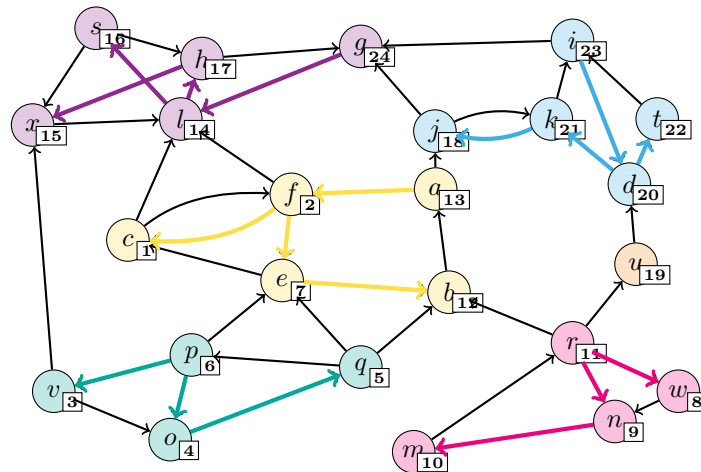
En revanche, si le sommet  $v$  n'appartient pas à la composante  $\overline{u_0}$  et  $v$  apparaît dans  $DFS_{G^\top}(u_0)$ . Alors, tout la composante  $\bar{v}$  du sommet  $v$  apparaît dans  $DFS_{G^\top}(u_0)$ , y compris un certain sommet  $v_0$  de plus haut  $n^\circ$ . Mais alors le sommet  $u_0$  serait accessible depuis  $v_0$ , ce qui contredirait le caractère topologique des numéros donnés aux composantes fortement connexe.

Après suppression des sommets de la composante  $\overline{u_0}$ , soit il ne reste plus aucun sommet et nous avons prouvé le cas des graphes qui ne nécessitent qu'un seul DFS en phase 2, soit nous pouvons appliquer l'hypothèse de récurrence pour conclure.  $\square$

**Exemple 3.49.** — Reprenons le graphe de l'exemple 3.44. La phase 1 de l'algorithme conduit aux parcours et aux numéros suivants.



Lors de la phase 2, les parcours en profondeur sont les suivants :



LISTING 3.7. Composantes fortement connexes (Algorithme de Kosaraju)

```

880 class DiGraph:
881     def __init__(self, n):
882         self.n = n
883         self.succ = [ [] for _ in range(n)]
884         self.pred = [ [] for _ in range(n)]
885     def add_edge(self, u, v):
886         self.succ[u].append(v)
887         self.pred[v].append(u)
888     def phase_one_dfs(self, v, visited, stack):
889         visited.add(v)
890         for w in self.succ[v]:

```

```

891         if w not in visited:
892             self._phase_one_dfs_(w, visited, stack)
893     stack = stack.append(v)
894     def _phase_one_(self):
895         stack = []
896         visited = set()
897         for u in range(self.n):
898             if u not in visited:
899                 self._phase_one_dfs_(u, visited, stack)
900         return stack
901     def _phase_two_dfs_(self, v, visited, cc):
902         visited.add(v)
903         cc.add(v)
904         for u in self.pred[v]:
905             if u not in visited:
906                 self._phase_two_dfs_(u, visited, cc)
907         return(cc)
908     def scc(self):
909         stack = self._phase_one_()
910         visited = set()
911         sccs = []
912         while stack:
913             v = stack.pop()
914             if v not in visited:
915                 sccs.append(self._phase_two_dfs_(v, visited, set()))
916         return sccs

```

**Proposition 3.50.** — La complexité en temps de l'algorithme de Kosaraju est  $O(n+m)$  où  $n$  est le nombre de sommets et  $m$  est le nombre d'arêtes.

**3.2.2. Le parcours en largeur.** — Le parcours en largeur, en anglais *breadth first search* (ou *BFS*), est l'autre des deux grands modes d'exploration d'un graphe. Il consiste à se déplacer par couches successives autour du sommet initial.

**Algorithme 3.4 :** Parcours en largeur**Entrées :** Graphe  $G = (V, E)$ , sommet  $s$ **Résultat :** Visite des sommets accessibles depuis  $s$ 


---

```

1  $T \leftarrow \{s\}$  (file servant de collection d'attente)
2 Marquer  $s$ 
3 tant que  $T$  non vide faire
4   Extraire  $v$  de  $T$ 
5   Visiter  $v$ 
6   pour  $w$  voisin de  $v$  non marqué faire
7     père( $w$ )  $\leftarrow v$ 
8      $T \leftarrow T \cup \{w\}$ 
9     Marquer  $w$ 

```

---

**Proposition 3.51.** — *L'algorithme de parcours en largeur 3.5 est correct par rapport à sa spécification.*

*Démonstration.* — Commençons par la preuve de terminaison. Notons  $m$  le nombre de sommets non-marqués et  $t$  le nombre d'éléments dans la file  $T$ . Pour prouver la terminaison de l'algorithme, montrons que la quantité  $(m, t)$  est un variant. Deux cas se produisent lors du passage dans la boucle « tant que ». Si le sommet  $v$  n'a aucun voisin non marqué, alors  $(m, t)$  devient  $(m, t - 1)$  ; si le sommet  $v$  possède au moins un voisin non marqué, ce voisin sera marqué, alors  $(m, t)$  devient  $(m', t')$  avec  $m' < m$ . Dans les deux cas, la quantité  $(m, t)$  diminue strictement au sens de l'ordre lexicographique. Comme l'ensemble  $\mathbb{N}^2$  est bien fondé pour cet ordre, l'algorithme se termine.

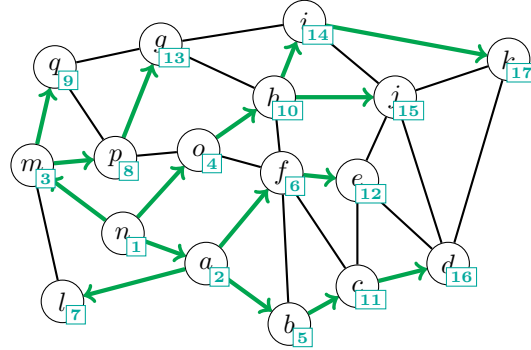
Montrons que l'algorithme est correct par rapport à sa spécification. Commençons par prouver que tout sommet visité est un sommet accessible depuis  $s$ . Démontrons l'invariant : « l'ensemble des arcs père( $v$ ) $v$  forme une arborescence de l'ensemble des sommets marqués ». Initialement, seul le sommet  $s$  est marqué et il n'y a aucun arc de la forme père( $v$ ) $v$ . L'invariant est donc vérifié avant d'entrer dans la boucle « tant que ». Supposons que l'invariant est vérifié à l'entrée dans la boucle « tant que ». À chaque fois qu'un sommet  $w$  supplémentaire est marqué, un arc (père( $w$ ),  $w$ ) est créé, où père( $w$ ) est déjà un sommet marqué. Ainsi l'invariant reste vérifié en sortie de boucle. Ceci prouve que les sommets marqués forment une arborescence. Mais tout sommet marqué entre dans la collection  $T$  et doit en sortir avant la fin de l'algorithme : il est donc visité. Par ailleurs, seuls les sommets passés par  $T$  sont visités. Ainsi l'ensemble des sommets visités forme une arborescence de racine  $s$ . D'après le lemme 3.18, ils sont tous accessibles depuis  $s$ .

Montrons enfin que tout sommet  $v$  accessible depuis  $r$  est visité. Soit  $C$  un chemin du sommet  $r$  vers le sommet  $v$ . Nous raisonnons par l'absurde. Supposons que  $v$  n'est pas visité par l'algorithme 3.5. Notons  $w$  le premier sommet non visité appartenant au chemin  $C$  et  $z$  son prédécesseur.



pas par les états suivants :

- $[n]$
- $[a, m, o]$
- $[m, o, b, f, l]$
- $[o, b, f, l, p, q]$
- $[b, f, l, p, q, h]$
- $[f, l, p, q, h, c]$
- $[l, p, q, h, c, e]$
- $[p, q, h, c, e]$
- $[q, h, c, e, g]$
- $[h, c, e, g]$
- $[c, e, g, i, j]$
- $[e, g, i, j, d]$
- $[g, i, j, d]$
- $[i, j, d]$
- $[j, d, k]$
- $[d, k]$
- $[k]$
- $\emptyset$



**Théorème 3.53.** — Soit  $A$  l'arborescence du parcours en largeur d'un graphe  $G = (V, E)$  depuis le sommet  $r$ . Alors, pour tout sommet  $v \in V$ , l'unique chemin de  $r$  vers  $v$  dans  $A$  est un plus court chemin de  $r$  vers  $v$  dans le graphe  $G$ . De plus, la longueur de ce chemin se calcule par l'algorithme suivant.

LISTING 3.9. Calcul de la distance à la racine

```

931 # [...] Compléter le code de la classe existante !
932 def distance(self, r):
933     distance = dict({r:0})
934     waiting = collections.deque([r])
935     while waiting:
936         v = waiting.popleft()
937         for u in self.neighbors[v]:
938             if u not in distance:
939                 distance[u] = distance[v]+1
940                 waiting.append(u)
941     return distance

```

*Démonstration.* — Chaque sommet passe par trois états et dans cet ordre de *non vu*, *en attente* (quand il entre dans la file et que  $d(v)$  a été calculé) et traité (quand il sort de la file). Nous numérotions l'ensemble des sommets dans l'ordre dans lesquels ils

entrent (et sortent) de la file  $T$

$$r = v_0, v_1, \dots, v_{n-1}$$

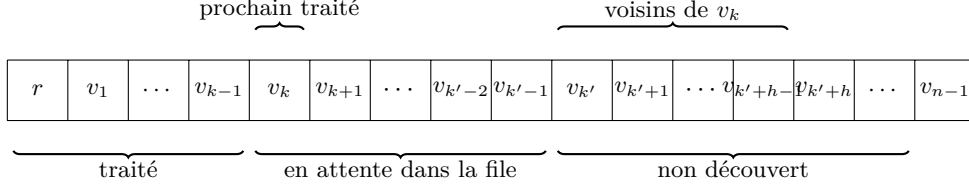
Nous pouvons démontrer que cet ensemble de sommet est exactement l'ensemble des sommets accessibles depuis  $r$ .

Nous observons que **distance**[ $u$ ] est affecté une fois pour toute (la valeur ne change pas). Nous notons  $d(u)$  la valeur calculée et  $\delta(u)$  la longueur d'un plus court chemin de  $r$  à  $v$  dans  $G$ . Le paragraphe précédent montre que  $\delta(u) \leq d(u)$ .

Nous commençons par démontrer par récurrence le dictionnaire **distance** contient bien, pour tout sommet  $v$ , la longueur  $d(v)$  de l'unique chemin de  $r$  à  $v$  dans l'arborescence  $A$  du parcours. En particulier,  $d(v)$  représente la longueur d'un chemin, qui n'est peut-être pas de longueur optimale, et majore la « vraie » distance  $\delta(v)$  de  $r$  à  $v$ .

Soient  $k$  et  $k'$  les indices avec  $k \leq k'$  tels que  $v_0, v_1, \dots, v_{k-1}$  ont été traités et  $v_k, v_{k+1}, \dots, v_{k'-1}$  sont encore en attente dans la file.

- ( $\clubsuit_1$ ) Pour tout  $i$  avec  $0 \leq i < k' - 1$ ,  $d(v_i) \leq d(v_{i+1})$ ;
- ( $\clubsuit_2$ ) On a  $d(v_{k'-1}) - d(v_k) \leq 1$ ;
- ( $\clubsuit_3$ ) Pour tout  $i$  avec  $0 \leq i < k'$ ,  $d(v_i) = \delta(v_i)$ ;
- ( $\clubsuit_4$ ) Si un sommet  $v_j$  quelconque vérifie  $\delta(v_j) < d(v_k)$ , alors  $j < k$  et si  $\delta(v_j) = d(v_k)$ , alors  $k \leq j < k'$  (autrement dit, aucun sommet n'est oublié dans la suite croissant de ( $\clubsuit_1$ )).



Initialement, ces invariants sont bien vérifiés : on a  $k = 0$ ,  $k' = 1$ , seul le sommet  $r$  est en attente avec  $d(r) = 0$ .

Supposons les invariants vérifiés à l'entrée de la boucle tant que. Le sommet défilé est le sommet  $v_k$  ; les sommets enfilés sont les sommets  $v'_k, v_{k'+1}, \dots, v_{k'+h-1}$  avec  $d(v'_k) = \dots = d(v_{k'+h-1}) = d(v_k) + 1$ . Comme  $d(v_{k'-1}) \leq d(v_k) + 1$  (à cause de ( $\clubsuit_2$ )), l'invariant ( $\clubsuit_1$ ) reste satisfait. De plus, puisque  $d(v_{k+1}) \geq d(v_k)$  (à cause de ( $\clubsuit_1$ )), l'invariant ( $\clubsuit_2$ ) reste vrai en sortie de la boucle « tant que ».

Pour conserver ( $\clubsuit_3$ ), montrons que  $d(v'_k) = \delta(v'_k)$ ,  $d(v_{k'+1}) = \delta(v_{k'+1})$ ,  $\dots$ ,  $d(v_{k'+h-1}) = \delta(v_{k'+h-1})$ . Soit  $j$  avec  $0 \leq j < h$ . Nous savons déjà que  $\delta(v_{k'+j}) \leq d(v_{k'+j})$ . Pour avoir  $\delta(v_{k'+j}) < d(v_{k'+j})$ , nous devrions avoir un plus court chemin se terminant par un certain arc  $(y, v_{k'+j})$  où  $\delta(y) < d(v_k)$ . Mais alors, à cause de l'invariant ( $\clubsuit_4$ ), nous aurions traité le sommet  $y$  avant le sommet  $v_k$  et aurions déjà détecté un chemin de longueur  $\delta(v_{k'+j})$  se terminant par l'arc  $(y, v_{k'+j})$ , ce qui n'est pas le cas. Donc  $\delta(v_{k'+j}) = d(v_{k'+j})$ .

Soit enfin un sommet  $v_j$  avec  $\delta(v_j) \leq d(v_{k+1})$ . Si  $d(v_{k+1}) = d(v_k)$ , l'invariant ( $\clubsuit_4$ ) est automatiquement vrai. Sinon, on a  $d(v_{k+1}) = d(v_k) + 1$ , à cause de ( $\clubsuit_2$ ). Si



$\delta(v_j) < d(v_{k+1})$ , l'invariant  $(\mathfrak{H}_4)$  montre que  $v_j$  a déjà été vu. De plus, à cause de  $(\mathfrak{H}_1)$ , les sommets sont rangés par valeur croissante de  $d$ . Donc le sommet  $v_j$  apparaît bien avant le sommet  $v_{k+1}$ . Si  $\delta(v_j) = d(v_{k+1})$ , le prédécesseur  $y$  de  $v_j$  sur un plus court chemin satisfait  $\delta(y) = d(v_k)$ . À cause de  $(\mathfrak{H}_4)$ ,  $y$  apparaît déjà dans la file. Mais comme les sommets sont rangés par valeur croissante de  $d$ ,  $y$  a été traité avant  $v_{k+1}$ . Comme  $v_j$  est un voisin de  $y$ , il a déjà été ajouté à la file quand  $y$  a été traité. Dans tous les cas, nous constatons que l'invariant  $(\mathfrak{H}_4)$  est préservé.  $\square$

### 3.3. Problèmes de plus court chemins

#### 3.3.1. Considérations générales. —

**Définition 3.54.** — Soit  $G = (V, E, \rho)$  un graphe pondéré. Le *poids d'un chemin*  $C = x_0 \cdots x_k$  est la quantité

$$\rho(C) = \sum_{i=1}^{k-1} \rho(x_i x_{i+1}).$$

Autrement dit, le poids d'un chemin est la somme des poids des arcs qui le compose.

**Définition 3.55.** — Soit  $G = (V, E, \rho)$  un graphe pondéré dépourvu de circuits de poids strictement négatif. La *distance* entre un sommet  $u \in V$  et un sommet  $v \in V$  accessible depuis  $u$  est le minimum

$$d(u, v) = \min\{\rho(C); C \text{ chemin de } u \text{ vers } v\}.$$

On parle de *plus court chemin* pour désigner tout chemin de  $u$  vers  $v$  de poids  $d(u, v)$ .

Lorsque  $v$  n'est pas accessible depuis  $u$ , on pose parfois  $d(u, v) = +\infty$ .

**Remarque 3.56.** — Cette définition est licite : en effet, s'il existe un chemin de  $u$  vers  $v$  comprenant un circuit, supprimer ce circuit du chemin conduit à définir un nouveau chemin de poids plus petit. Ainsi, le minimum des poids peut porter simplement sur les poids de chemins élémentaires. Or il n'y a qu'un nombre fini de chemins élémentaires dans un graphe.

En général, on s'intéresse à trois problèmes d'optimisation :

- Étant donné un graphe  $G$  et deux sommets  $s$  et  $t$  de  $G$ , calculer la distance entre  $s$  et  $t$  et, éventuellement, un plus court chemin.
- Étant donné un graphe  $G$  et un sommet  $s$  de  $G$ , calculer la distance entre  $s$  et n'importe quel sommet  $t$  du graphe  $G$ , éventuellement, un plus court chemin à destination de chaque sommet.
- Étant donné un graphe  $G$ , calculer la distance tout couple de sommet et, éventuellement, un plus court chemin entre eux..

**Lemme 3.57.** — Si  $x_0 x_1 \cdots x_k$  est un plus court chemin entre le sommet  $x_0 = s$  et le sommet  $x_k = t$ , alors, pour tout indice  $i$  compris entre 0 et  $k$ , le chemin  $x_0 x_1 \cdots x_i$  est un plus court chemin de  $s$  vers  $x_i$ .

**Lemme 3.58.** — *Il est toujours possible de représenter l'ensemble des plus courts chemins de  $s$  vers n'importe quel sommet sous la forme d'une arborescence.*

*Démonstration.* — Pour tout sommet  $v$ , notons  $C_v$  un plus court chemin de  $s$  vers  $v$ . Nous construisons une arborescence  $A$  comme suit. Initialement,  $A$  est l'arborescence restreinte au sommet  $s$ . Pour tout sommet  $v \in V$ , nous identifions le dernier sommet  $w$  du chemin  $C_v$  qui appartient à  $A$  et ajoutons à  $A$  les sommets et les arcs de  $C_v$  compris entre  $w$  et  $v$ . A cause du lemme précédent, les chemins dans  $A$  issus de  $s$  sont tous des plus courts chemins entre  $s$  et leur extrémité finale. De plus,  $A$  contient bien tous les sommets en fin de procédure.  $\square$

### 3.3.2. Source unique et graphes sans circuits. —

**Lemme 3.59.** — *Soit  $G = (V, E, \rho)$  un graphe orienté pondéré sans circuit de poids négatif et  $s$  un sommet de  $V$ . Alors, pour tout sommet  $v$ , on a*

$$d(s, v) = \begin{cases} 0 & \text{si } v = s, \\ \min_{\substack{u \in V; \\ uv \in E}} d(s, u) + \rho(uv) & \text{sinon} \end{cases}$$

*Démonstration.* — Si  $v = 0$ , l'égalité est claire.

Sinon, commençons par observer que pour tout prédécesseur  $u$  du sommet  $v$ , la quantité  $d(s, u) + \rho(uv)$  représente la longueur d'un chemin de  $s$  vers  $v$ , à savoir le chemin le plus court de  $s$  vers  $u$  suivi de l'arc  $uv$ . D'autre part, un plus court chemin de  $s$  vers  $v$  est forcément de la forme  $x_0 \cdots x_{k-1}x_k = v$  et a pour longueur  $d(s, u) + \rho(uv)$  avec  $u = x_{k-1}$ .  $\square$

Peut-on exploiter ces relations de récurrence ? Si le graphe possède un circuit, c'est peine perdue ! Pour calculer une valeur sur le circuit, il faut connaître celles de tous les prédécesseurs. En revanche, dans le cas où le graphe  $G$  est acyclique, les sommets de  $G$  peuvent être numérotés dans un ordre topologique et les formules du lemme 3.59 peuvent être utilisées les unes après les autres pour déterminer toutes les distances.

---

#### Algorithme 3.5 : Algorithme de Bellman

---

**Entrées :** Graphe  $G = (V, E, \rho)$  acyclique orienté, sommet  $s$

**Résultat :** Distance du sommet  $s$  à tout sommet accessible, arborescence des plus courts chemins

- 1 Trier les sommets accessibles dans l'ordre topologique :

$$v_1 = s, v_2, \dots, v_n.$$

- 2 Poser  $\text{dist}(s) = 0$ .

- 3 **pour**  $i \in \llbracket 2, n \rrbracket$  **faire**

- 4     Calculer

$$\text{dist}(v_i) = \min_{u \text{ préd. de } v_i} \text{dist}(u) + \rho(uv_i).$$

$$\text{père}(v_i) = \text{argmin}_{u \text{ préd. de } v_i} \text{dist}(u) + \rho(uv_i).$$


---

**Théorème 3.60.** — *L'algorithme de Bellman est correct par rapport à sa spécification.*

*Démonstration.* — Nous introduisons l'invariant suivant : après l'itération  $i$ , quand le sommet  $v_i$  est traité, on a, pour tout  $j \leq i$ , que le dictionnaire `dist` contient la distance  $d(s, v_j)$  et le dictionnaire `père` contient une arborescence de plus courts chemins dans  $\{v_1, \dots, v_i\}$ . Il est clair que cet invariant est initialement satisfait. Le lemme 3.59 montre qu'il reste vrai après chaque passage dans la boucle.  $\square$

LISTING 3.10. Plus court chemin avec source unique dans un graphe orienté acyclique - Algorithme de Bellman

```

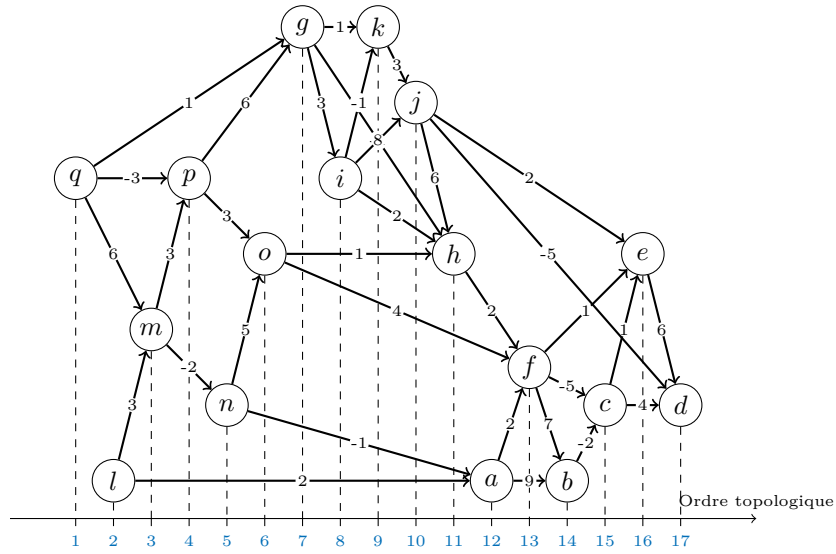
942 class WeightedGraph():
943     def __init__(self, n):
944         self.succ = [[] for _ in range(n)]
945         self.pred = [[] for _ in range(n)]
946         self.weight = dict()
947     def add_edge(self, u, v, rho):
948         self.succ[u].append(v)
949         self.pred[v].append(u)
950         self.weight[(u,v)] = rho
951     def _top_sort_(self, r, visited, stack):
952         if r in visited:
953             return stack
954         visited.add(r)
955         for u in self.succ[r]:
956             if u not in visited:
957                 stack = self._top_sort_(u, visited, stack)
958         stack.append(r)
959         return stack
960     def dag_sssp(self, r):
961         stack = self._top_sort_(r, set(), [])
962         dist = {r:0}
963         parent = {r:-1}
964         stack.pop()
965         for v in reversed(stack):
966             d = { u:dist[u]+self.weight[(u,v)] for u in self.pred[v]
967                  ↪ }
968             u0, d0 = min(d.items(), key=lambda x: x[1])
969             dist[v] = d0
970             parent[v] = u0
971         return (dist, parent)

```

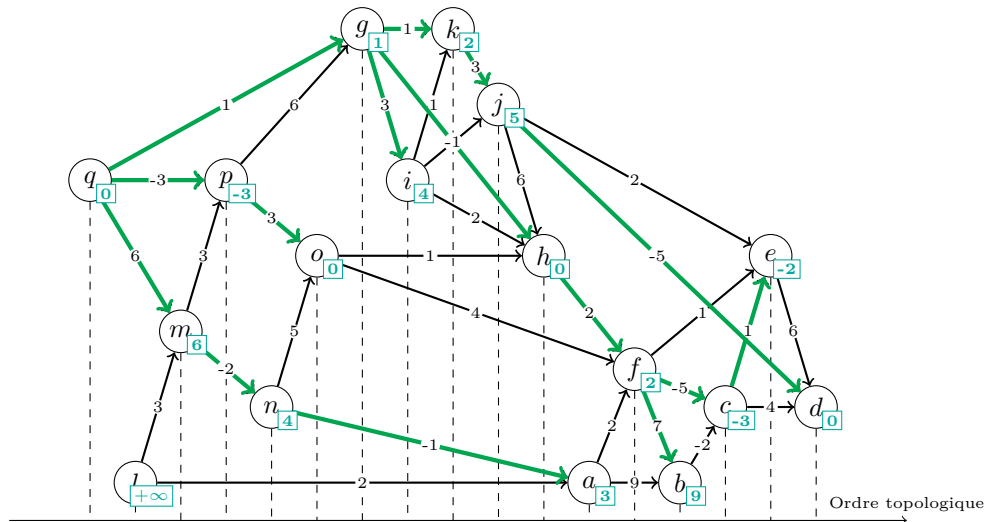
**Proposition 3.61.** — Lorsque les graphes sont représentés par des listes d'adjacence, la complexité en temps de l'algorithme de Bellman est  $O(n + m)$ , où  $n$  est le nombre de sommets et  $m$  est le nombre d'arcs.

*Démonstration.* — □

**Exemple 3.62.** — Voici un graphe orienté acyclique pondéré. Nous cherchons l'ensemble des distances et des plus courts chemins de source  $q$ . Nous supposons donnée une numérotation topologique des sommets.



Après avoir considéré chacun des sommets, nous obtenons :



**3.3.3. Source unique et graphes à poids positifs.** — Dans le cas de graphes orientés quelconques mais dont les poids sont positifs, l'*algorithme de Dijkstra* vient faire la synthèse des deux algorithmes croisés jusqu'ici.

À l'image du parcours en largeur, il construit les distances et l'arborescence de plus courts chemin par un parcours de graphe à l'aide d'une file, la différence étant que le parcours se fait avec un file à priorité (voir section 2.3.3.1) plutôt qu'une file classique. À l'image de l'algorithme de Bellman, les valeurs des distances sont déduites de l'équation du lemme 3.57.

---

**Algorithme 3.6 :** Algorithme de Dijkstra

---

**Entrées :** Graphe orienté pondéré  $G = (V, E, \rho)$ , sommet  $s$

**Résultat :** Distance du sommet  $s$  à tout sommet accessible,  
arborescence des plus courts chemins

```

1 pour  $v \in V \setminus \{s\}$  faire
2    $\pi(v) \leftarrow +\infty$ 
3  $\pi(s) \leftarrow 0$ 
4  $F \leftarrow \{s\}$  : file à priorité, où la priorité du sommet  $v$  est  $\pi(v)$ 
5 tant que  $F \neq \emptyset$  faire
6    $v \leftarrow$  Extraire de  $F$  le sommet de plus petite priorité  $\pi(v)$ 
7   Marquer  $v$ 
8   pour  $w$  successeur de  $v$  non marqué faire
9     si  $\pi(v) + \rho(vw) < \pi(w)$  alors
10      si  $w \in F$  alors
11        Diminuer la priorité  $\pi(w) \leftarrow \pi(v) + \rho(vw)$  dans  $F$ 
12      sinon
13        Insérer  $w$  à  $F$  avec la priorité  $\pi(w) \leftarrow \pi(v) + \rho(vw)$ 
14       $\text{père}(w) \leftarrow v$ 
15 retourner  $\pi$  et père

```

---

**Théorème 3.63.** — *L'algorithme de Dijkstra est correct par rapport à sa spécification.*

*Démonstration.* — Nous observons que le nombre de sommets non marqués augmente d'une unité à chaque passage dans la boucle « tant que ». Par conséquent, l'algorithme se termine.

Appelons  $\Pi$  l'ensemble des sommets  $v$  tels que  $\pi(v) < +\infty$  et  $M$  l'ensemble des sommets marqués. Nous pouvons adapter la preuve faite sur le parcours en largeur pour montrer que tout sommet accessible est bien marqué par l'algorithme et apparaît dans l'arborescence finale. Comme pour le parcours en largeur, les sommets entrent dans  $\Pi$ , puis dans  $M$ .

Nous nous intéressons aux invariants suivants :

( $\boxtimes_1$ ) Le dictionnaire père représente une arborescence de racine  $s$  portant sur les sommets de  $\Pi$ . De plus l'unique chemin de  $s$  vers  $v$  dans cette arborescence est de poids  $\pi(v)$

( $\boxtimes_2$ ) Pour tout sommet  $v \in \Pi \setminus \{s\}$ ,

$$\pi(v) = \min_{u \in M} (\pi(u) + \rho(uv)),$$

( $\boxtimes_3$ ) Pour tout sommet  $v \in M$ ,  $\pi(v)$  est égal à la distance de  $s$  à  $v$ .

L'invariant ( $\boxtimes_1$ ) se justifie facilement : père et  $\pi$  sont modifiés simultanément. Lorsqu'ils sont modifiés, par hypothèse,  $v$  est marqué,  $\pi(v)$  est le poids de l'unique chemin de  $s$  à  $v$ . Or en rattachant  $w$  à  $v$ , tout chemin de  $s$  à  $w$  doit passer par  $v$ . Il n'y en a qu'un seul, qui est alors de longueur  $\pi(v) + \rho(vw)$ .

L'invariant ( $\boxtimes_2$ ) se justifie également par récurrence. Avant de marquer le sommet  $v$ , on a pour tout sommet  $w \in \Pi \setminus \{s\}$ ,  $\pi(w) = \min_{u \in M} (\pi(u) + \rho(uw))$ . Quand  $v$  est marqué, les lignes 9 et 10 du code modifient  $\pi$  pour que cette propriété soit conservée.

Nous avons déjà démontré que  $\pi(v)$  représente la longueur d'un chemin. Supposons que pour un certain sommet  $v$ , nous marquons  $v$  alors que  $\pi(v)$  est strictement supérieur à  $d(s, v)$ . Soit  $C$  un chemin de  $s$  vers  $v$  de poids  $d(s, v)$ . Le prédécesseur  $u$  de  $v$  sur  $C$  ne peut pas être marqué. En effet, à cause de l'invariant ( $\boxtimes_3$ ), on aurait  $\pi(u) = d(s, u)$ , et à cause de l'invariant ( $\boxtimes_2$ ), on aurait  $\pi(v) \leq \pi(u) + \rho(uv) = d(s, v)$ . Considérons le premier sommet  $w$  sur le chemin  $C$  tel que  $w$  est non marqué. Appelons  $z$  son prédécesseur, qui lui est marqué, ce qui implique notamment que

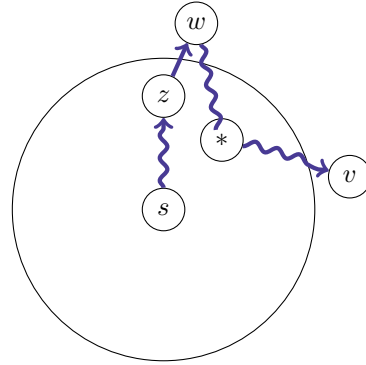
$$\pi(w) \leq \pi(z) + \rho(zw).$$

On a alors

$$\begin{aligned} \pi(v) &> d(s, v) \\ &\geq d(s, w) \quad (\text{puisque les poids sont positifs.}) \\ &= d(s, z) + \rho(zw) \quad (\text{puisque } C \text{ est un plus court chemin.}) \\ &= \pi(z) + \rho(zw) \quad (\text{puisque } z \in M.) \\ &\geq \pi(w) \quad (\text{comme vu plus haut.}) \end{aligned}$$

Nous voyons que  $\pi(v) > \pi(w)$ , ce qui contredit le fonctionnement de la file de priorité. Ainsi l'invariant ( $\boxtimes_3$ ) est lui aussi préservé.  $\square$

**Remarque 3.64.** — Nous avons présenté les files de priorité implémentées avec des tas. Cette structure de donnée ne supporte pas l'opération de diminution de la priorité d'un élément déjà présent. Des structures plus avancées telles que des *tas de Fibonacci* autorisent une telle opération. Nous contournons la difficulté ici en utilisant un tas classique et en insérant plusieurs fois le même sommet avec des priorités de plus en plus petites. Par ailleurs, nous testons si un élément est déjà marqué au moment de le défiler, pour éviter de le traiter plusieurs fois.



LISTING 3.11. Plus court chemin avec source unique dans un graphe orienté à poids positifs - Algorithme de Dijkstra

```

971 class WeightedGraph():
972     # Classe ainsi complétée
973     def _sssp_init_(self, r):
974         distance = [sys.maxsize//2 for _ in self.succ]
975         parent = [-1 for _ in self.succ]
976         distance[r] = 0
977         return(distance, parent)
978     def dijkstra(self, r):
979         distance, parent = self._sssp_init_(r)
980         queue = [(0,r)]
981         visited = set()
982         while queue:
983             _,v = hp.heappop(queue)
984             if v not in visited:
985                 visited.add(v)
986                 for w in self.succ[v]:
987                     d = distance[v] + self.weight[(v,w)]
988                     if d < distance[w]:
989                         distance[w] = d
990                         parent[w] = v
991                         hp.heappush(queue, (d, w))
992         return(distance, parent)

```

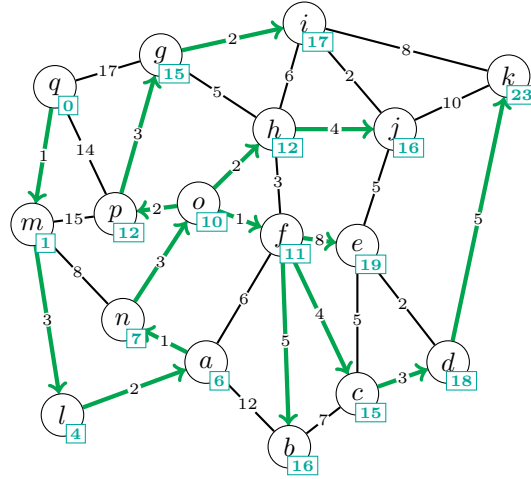
**Proposition 3.65.** — Lorsque les graphes sont représentés par des listes d'adjacence, la complexité en temps de l'algorithme de Bellman est  $O(n + m \log_2 m)$ , où  $n$  est le nombre de sommets et  $m$  est le nombre d'arcs.

*Démonstration.* — Les opérations de création des variable `distance` et `parent` nécessitent un temps linéaire en  $n$ . La file de priorité peut contenir jusqu'à  $m$  éléments. Ainsi, la boucle `parent` peut être répétée jusqu'à  $O(m)$  fois. Les opérations d'insertions ou de suppression coûtent par ailleurs  $O(\log_2 m)$ . Le test d'appartenance d'un sommet à `visited` peut être fait en  $O(1)$  si `parent` est implémenté par une table de hachage. Les opérations dans la boucle `for` ne sont faite qu'au plus une fois par sommet et coûtent un temps proportionnel au nombre de successeur de chaque sommet. Au total, elle représentent un coût en  $O(m)$ .  $\square$

**Remarque 3.66.** — La proposition ci-dessus correspond à l'analyse du code 3.11. Avec une structure de tas de Fibonacci, nous pourrions abaisser la complexité à  $O(n \log_2 n + m)$ , ce qui est la meilleure complexité connue. Inversement, nous pourrions nous contenter de placer les priorités  $\pi$  dans un tableau et de chercher le minimum par un parcours du tableau : la complexité est alors  $O(n^2)$ .

**Exemple 3.67.** — Nous cherchons les plus courts chemins issus du sommet  $q$ . La file de priorité passe par les états suivants :

- $[q]$
- $[m, p, g]$
- $[l, n, p, g]$
- $[a, n, p, g]$
- $[n, f, p, g, b]$
- $[o, f, p, g, b]$
- $[f, p, h, g, b]$
- $[p, h, c, g, b, e]$
- $[h, c, g, b, e]$
- $[g, c, j, b, i, e]$
- $[c, j, b, i, e]$
- $[j, b, i, d, e]$
- $[b, i, d, e, k]$
- $[i, d, e, k]$
- $[d, e, k]$
- $[e, k]$
- $[k]$
- $\emptyset$



**3.3.4. Source unique et graphes à poids quelconques.** — Traitons à présent le cas de graphes dont les poids sont de signe quelconque et qui ne contiennent pas de circuit de poids négatif.

L'algorithme de Bellman Ford itère sur les équations du lemme 3.59.

---

**Algorithme 3.7 :** Algorithme de Bellman Ford

---

**Entrées :** Graphe orienté pondéré  $G = (V, E, \rho)$ , sommet  $s$

**Résultat :** Distance du sommet  $s$  à tout sommet accessible,  
arborescence des plus courts chemins

```

1 pour  $v \in V \setminus \{s\}$  faire
2    $\pi(v) \leftarrow +\infty$ 
3  $\pi(s) \leftarrow 0$ 
4 tant que il existe un arc  $(v, w)$  tel que  $\pi(w) > \pi(v) + \rho(vw)$  faire
5   pour tout arc  $(v, w)$  faire
6     si  $\pi(w) > \pi(v) + \rho(vw)$  alors
7        $\pi(w) \leftarrow \pi(v) + \rho(vw)$ 
8        $\text{père}(w) \leftarrow v$ 
9 retourner  $\pi$  et père

```

---

**Définition 3.68.** — Soit  $G = (V, E, w)$  un graphe pondéré dépourvu de circuits de poids strictement négatif. La *distance en moins de  $k$  arcs* entre un sommet  $u \in V$  et



un sommet  $v \in V$  accessible depuis  $u$  est le minimum

$$d_{\leq k}(u, v) = \min\{w(C); C \text{ chemin de } u \text{ vers } v \text{ de longueur inférieure à } k\}.$$

**Proposition 3.69.** — Pour tout sommet  $v$ , après  $k$  itérations de l'algorithme de Bellman Ford, on a

$$\pi(k) \leq d_{\leq k}(s, v).$$

*Démonstration.* — □

L'implémentation suivante comprend une légère optimisation, due à Moore : plutôt que de tester chacun des arcs à chaque itération, on peut se contenter de garder trace des sommets que l'on a modifié

LISTING 3.12. Plus court chemin avec source unique dans un graphe orienté - Algorithme de Bellman Ford Moore

```

993 class WeightedGraph():
994     # Classe ainsi complétée
995     def BellmanFord(self, r):
996         distance, parent = self._sssp_init_(r)
997         waiting = deque([r])
998         while waiting:
999             v = waiting.popleft()
1000             for w in self.succ[v]:
1001                 d = distance[v] + self.weight[(v,w)]
1002                 if d < distance[w]:
1003                     distance[w] = d
1004                     parent[w] = v
1005                     if w not in waiting:
1006                         waiting.append(w)
1007         return(distance, parent)

```

**3.3.5. Source multiple et graphes à poids quelconques.** — Nous terminons la présentation d'algorithmes de plus courts chemins par le cas où l'on souhaite calculer les distances entre tout couple de sommet  $u$  et  $v$ .

Soit  $G = (V, E, \rho)$  un graphe pondéré, dont nous numérotions les sommets

$$v_1, v_2, \dots, v_n.$$

Nous supposons que  $G$  ne contient pas de circuit de poids négatif.

Pour tout couple de sommets  $(v_i, v_j)$  et tout entier  $r$ , nous notons  $\pi(v_i, v_j, r)$  le poids, s'il existe, d'un plus court chemin allant de  $v_i$  à  $v_j$  en n'utilisant que les sommets  $\{v_1, \dots, v_r\}$  comme sommets intermédiaires. Quand un tel chemin n'existe pas, nous posons simplement  $\pi(v_i, v_j, r) = +\infty$ .

Nous faisons les constats suivants :

- Si  $r = 0$ , les seuls chemins possibles entre le sommet  $v_i$  et le sommet  $v_j$  qui ne passent que par les  $r$  premiers sommets sont des chemins directs, formés d'un seul arc  $v_i v_j$ . Ainsi

$$\pi(v_i, v_j, 0) = \begin{cases} 0 & \text{si } i = j \\ \rho(v_i v_j) & \text{si } v_i v_j \in E \\ +\infty & \text{sinon} \end{cases}$$

- Si  $r > 0$ , un chemin du sommet  $v_i$  au sommet  $v_j$  peut soit passer, soit ne pas passer par le sommet  $v_r$ . Dans le premier cas, un chemin optimal est alors formé de deux chemins  $v_i \rightsquigarrow v_r$  et  $v_r \rightsquigarrow v_j$  qui doivent chacun être optimaux et l'utiliser que les sommets  $\{v_1, \dots, v_{r-1}\}$  comme sommets intermédiaires. Dans le second cas, nous observons que le chemin optimal n'utilise que les sommets  $\{v_1, \dots, v_{r-1}\}$  comme sommets intermédiaires. Aussi

$$\pi(v_i, v_j, r) = \min \left\{ \begin{array}{c} \pi(v_i, v_r, r-1) + \pi(v_r, v_j, r-1) \\ \pi(v_i, v_j, r) \end{array} \right\}.$$

Ces équations nous permettent d'écrire un algorithme de programmation dynamique comme suit.

LISTING 3.13. Plus court chemin avec source unique dans un graphe orienté - Algorithme de Floyd Warshall

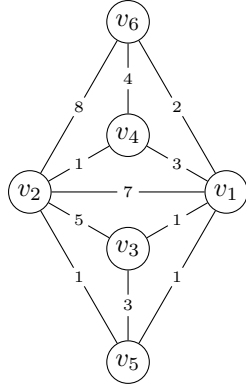
```

1008 import sys
1009 from copy import deepcopy
1010
1011 class WeightedGraph():
1012     def __init__(self, n):
1013         self.n = n
1014         self.adjacency = [[sys.maxsize//2 for _ in range(n)] for _ in
1015                             range(n)]
1016     def add_edge(self, u, v, rho):
1017         self.adjacency[u][v] = rho
1018     def floydwarshall(self):
1019         D = deepcopy(self.adjacency)
1020         for u in range(self.n):
1021             D[u][u] = 0
1022         for r in range(self.n):
1023             for u in range(self.n):
1024                 for v in range(self.n):
1025                     D[u][v] = min(D[u][v], D[u][r] + D[r][v])
1026         return(D)

```

**Proposition 3.70.** — La complexité en temps de l'algorithme de Floyd Warshall est  $O(n^3)$  où  $n$  est le nombre de sommets du graphe.

**Exemple 3.71.** — Considérons le graphe suivant et observons l'évolution de la matrice  $D$ .



Pour  $r = 0$  :

|       | $v_1$ | $v_2$ | $v_3$    | $v_4$    | $v_5$    | $v_6$    |
|-------|-------|-------|----------|----------|----------|----------|
| $v_1$ | 0     | 7     | 1        | 3        | 1        | 2        |
| $v_2$ | 7     | 0     | 5        | 1        | 1        | 8        |
| $v_3$ | 1     | 5     | 0        | $\infty$ | 3        | $\infty$ |
| $v_4$ | 3     | 1     | $\infty$ | 0        | $\infty$ | 4        |
| $v_5$ | 1     | 1     | 3        | $\infty$ | 0        | $\infty$ |
| $v_6$ | 2     | 8     | $\infty$ | 4        | $\infty$ | 0        |

Pour  $r = 1$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 7     | 1     | 3     | 1     | 2     |
| $v_2$ | 7     | 0     | 5     | 1     | 1     | 8     |
| $v_3$ | 1     | 5     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 4     | 4     |
| $v_5$ | 1     | 1     | 2     | 4     | 0     | 3     |
| $v_6$ | 2     | 8     | 3     | 4     | 3     | 0     |

Pour  $r = 2$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 7     | 1     | 3     | 1     | 2     |
| $v_2$ | 7     | 0     | 5     | 1     | 1     | 8     |
| $v_3$ | 1     | 5     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 2     | 4     |
| $v_5$ | 1     | 1     | 2     | 2     | 0     | 3     |
| $v_6$ | 2     | 8     | 3     | 4     | 3     | 0     |

Pour  $r = 3$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 6     | 1     | 3     | 1     | 2     |
| $v_2$ | 6     | 0     | 5     | 1     | 1     | 8     |
| $v_3$ | 1     | 5     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 2     | 4     |
| $v_5$ | 1     | 1     | 2     | 2     | 0     | 3     |
| $v_6$ | 2     | 8     | 3     | 4     | 3     | 0     |

Pour  $r = 4$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 4     | 1     | 3     | 1     | 2     |
| $v_2$ | 4     | 0     | 5     | 1     | 1     | 5     |
| $v_3$ | 1     | 5     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 2     | 4     |
| $v_5$ | 1     | 1     | 2     | 2     | 0     | 3     |
| $v_6$ | 2     | 5     | 3     | 4     | 3     | 0     |

Pour  $r = 5$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 2     | 1     | 3     | 1     | 2     |
| $v_2$ | 2     | 0     | 3     | 1     | 1     | 4     |
| $v_3$ | 1     | 3     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 2     | 4     |
| $v_5$ | 1     | 1     | 2     | 2     | 0     | 3     |
| $v_6$ | 2     | 4     | 3     | 4     | 3     | 0     |

Pour  $r = 6$  :

|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 2     | 1     | 3     | 1     | 2     |
| $v_2$ | 2     | 0     | 3     | 1     | 1     | 4     |
| $v_3$ | 1     | 3     | 0     | 4     | 2     | 3     |
| $v_4$ | 3     | 1     | 4     | 0     | 2     | 4     |
| $v_5$ | 1     | 1     | 2     | 2     | 0     | 3     |
| $v_6$ | 2     | 4     | 3     | 4     | 3     | 0     |

### 3.4. Arbres couvrants

Étant donné un graphe non orienté pondéré connexe  $G = (V, E, \rho)$ , on souhaite connecter l'ensemble des sommets par une collection d'arêtes dont le poids est minimal. Si les poids sont tous positifs, on obtient naturellement un sous-graphe connexe et acyclique, autrement dit un arbre.

**Définitions 3.72.** — On appelle *arbre couvrant* dans un graphe non orienté connexe  $G = (V, E)$  tout arbre de la forme  $(V, A)$  avec  $A \subseteq E$ .

Si  $G$  est graphe pondéré  $G = (V, E, \rho)$  et  $(V, T)$  est un arbre, on appelle *poids* de l'arbre la somme

$$\rho(T) = \sum_{e \in T} \rho(e).$$

Un *arbre couvrant de poids minimum* est un arbre  $(V, T)$  qui minimise la quantité  $\rho(T)$  parmi tous les arbres couvrants possibles.

**3.4.1. Sélection gloutonne des arêtes : l'algorithme de Kruskal.** — Nous présentons un premier algorithme dont le principe est de sélectionner des arêtes par ordre de poids croissant en maintenant l'invariant « l'ensemble des arêtes sélectionné est acyclique ».

---

**Algorithme 3.8 :** Algorithme de Kruskal

---

**Entrées :** Graphe non orienté pondéré connexe  $G = (V, E, \rho)$

**Résultat :** Arbre couvrant de poids minimal  $(V, T)$

- 1 Ranger les arêtes  $E$  par poids croissant dans  $F$  (file à priorité)
  - 2  $T \leftarrow \emptyset$
  - 3 **tant que**  $|T| < |V| - 1$  **faire**
  - 4     Extraire de  $F$  l'arête  $e$  de poids minimum
  - 5     **si**  $T \cup \{e\}$  *est acyclique* **alors**
  - 6          $T \leftarrow T \cup \{e\}$
- 

Cette description en pseudo-code cache une difficulté : comment déterminer si un graphe est acyclique. Nous pourrions bien sûr détecter l'existence de cycle avec l'un des algorithmes vus plus haut. Cependant, nous observons que, tout du long de l'exécution de l'algorithme,  $(V, T)$  est une forêt. Or, lorsqu'on ajoute une arête à une forêt, deux cas sont possibles : on crée un cycle au sein d'une composante connexe quand les deux extrémités appartiennent à la même composante connexe ; on ne crée pas de cycle mais on fusionne deux composantes connexes quand les deux extrémités appartiennent à deux composantes connexes distinctes.

Pour tester la création d'un cycle dans l'algorithme de Kruskal, il est utile de conserver la partition de l'ensemble des sommets en les composantes connexes. C'est exactement ce que fait le type abstrait « partition » (cf. ??) qui s'implémente généralement avec la structure « unir et trouver » (ou *union-find* en anglais).

**Théorème 3.73.** — *L'algorithme de Kruskal est correct par rapport à sa spécification.*

*Démonstration.* — La terminaison de l'algorithme est claire puisque la collection  $F$  perd un élément à chaque étape. Nous notons que la valeur de retour  $(V, T)$  est forcément acyclique. Nous observons qu'elle est forcément connexe. En effet, si elle ne l'était pas, nous pourrions trouver un chemin dans  $G$  entre deux sommets appartenant à deux composantes distinctes de  $(V, T)$ . Sur ce chemin, il y aurait une arête  $e$  dont les extrémités seraient dans deux composantes distinctes. Mais alors, au moment de

considérer cette arête dans la boucle « tant que », l'algorithme aurait dû inclure  $e$  à  $T$ . Ainsi l'algorithme se termine et fournit un arbre couvrant.

Supposons que pour un certain graphe  $G = (V, E, \rho)$ , l'algorithme de Kruskal ne renvoie pas un arbre optimal. Soit  $T = \{e_1, \dots, e_{n-1}\}$  l'ensemble d'arêtes qui résulte de l'algorithme de Kruskal

$$\rho(e_1) \leq \rho(e_2) \leq \dots \leq \rho(e_{n-1}).$$

Soit  $T_0 = \{f_1, \dots, f_{n-1}\}$  l'ensemble d'arêtes d'un arbre couvrant  $(V, T_0)$  de poids strictement inférieur à  $\rho(T)$ . quitte à réordonner les arêtes de  $T_0$ , nous supposons aussi que

$$\rho(f_1) \leq \rho(f_2) \leq \dots \leq \rho(f_{n-1}).$$

Comme  $\rho(T_0) < \rho(T)$ , on ne peut avoir  $\rho(e_i) \leq \rho(f_i)$  pour toute valeur de  $i$ . Soit  $\ell$  le plus petit indice tel que  $\rho(e_i) > \rho(f_i)$ .

Notons  $A$  l'ensemble d'arête  $A = \{e_1, \dots, e_{\ell-1}\}$ ; alors,  $(V, A)$  forme une forêt et possède  $n - \ell + 1$  composantes connexes  $C_1, C_2, \dots, C_{n-\ell+1} \subseteq V$ . Notons  $B$  l'ensemble d'arête  $B = \{f_1, \dots, f_\ell\}$ ; alors  $(V, B)$  forme une forêt avec  $n - \ell$  composantes connexes.

Il y a donc forcément une arête  $f$  de  $B$  qui relie deux composantes distinctes de  $A$  et  $A \cup \{f\}$  est encore une forêt. En effet, si tel n'est pas le cas, chaque arête de  $B$  possède ses deux extrémités dans une seule composante  $C_i$ . Mais comme  $B$  est une forêt, il ne peut y avoir qu'au plus  $|C_i| - 1$  arêtes dont les extrémités sont dans  $C_i$ . Au total,  $B$  ne peut compter que moins de  $\sum_{i=1}^{n-\ell+1} |C_i| - 1 = \ell - 1$  arêtes, ce qui n'est pas le cas.

Par ailleurs, on doit avoir

$$\rho(f) \leq \rho(f_\ell) < \rho(e_\ell).$$

Mais alors, l'algorithme de Kruskal aurait pu intégrer l'arête  $f$  avant d'intégrer l'arête  $e_\ell$  à  $T$  et s'est trompé : ceci est une contradiction. Ainsi l'algorithme de Kruskal calcule bien un arbre de poids minimum.  $\square$

LISTING 3.14. Arbre couvrant de poids minimum - Algorithme de Kruskal

```

1026 import heapq as hp
1027
1028 class UnionFind():
1029     def __init__(self, n):
1030         self.parent = [i for i in range(n)]
1031         self.height = [0 for i in range(n)]
1032     def find(self, u):
1033         if u != self.parent[u]:
1034             return self.find(self.parent[u])
1035         else:
1036             return u
1037     def union(self, u, v):
1038         p = self.find(u)
1039         q = self.find(v)

```

```

1040         if p==q:
1041             return False
1042         if self.height[p] > self.height[q]:
1043             self.parent[q] = p
1044         elif self.height[p] == self.height[q]:
1045             self.parent[q] = p
1046             self.height[p] += 1
1047         else:
1048             self.parent[p] = q
1049         return(True)
1050
1051     class WeightedGraph():
1052         def __init__(self,n):
1053             self.n = n
1054             self.edges = []
1055         def add_edge(self, u, v, rho):
1056             self.edges.append((u,v,rho))
1057         def kruskal(self):
1058             E = [(rho, u, v) for (u,v,rho) in self.edges]
1059             hp.heapify(E)
1060             T = set()
1061             connected_comp = UnionFind(self.n)
1062             while len(T) < self.n-1:
1063                 _, u, v = hp.heappop(E)
1064                 if connected_comp.union(u,v):
1065                     T.add((u,v))
1066             return(T)

```

**Proposition 3.74.** — La complexité de l'algorithme de Kruskal est  $O(m \log_2 m)$  où  $m$  est le nombre d'arêtes.

*Démonstration.* — Il y a  $m$  arêtes. Donc, les opérations de gestion du tas (insertion et suppression) coûtent  $O(\log_2 m)$  opérations élémentaires pour chaque arête et  $O(m \log_2 m)$  pour toutes les insertions et suppressions. La création de la structure « unir et trouver » coûte un temps  $O(n)$ . Les opérations de recherche et d'union coûtent  $O(\log_2 n)$  (puisque nous veillons à équilibrer les arbres). Enfin, l'ajout d'une arête à  $T$  se fait en temps constante (si  $T$  est géré comme une table de hachage). En conséquence, le coût par boucle « tant que » est en  $O(\log_2 n + \log_2 m)$ . Comme  $n = O(m)$ , on a bien un coût total en  $O(m \log_2 m)$ .  $\square$

### 3.4.2. Extension gloutonne d'une composante : l'algorithme de Prim. —

Le second algorithme que nous présentons fonctionne également sur un principe glouton. Cette fois-ci nous conservons une seule composante que nous faisons grossir en lui adjoignant l'arête incidente de plus petit poids.

**Algorithme 3.9 :** Algorithme de Prim**Entrées :** Graphe non orienté pondéré connexe  $G = (V, E, \rho)$ **Résultat :** Arbre couvrant de poids minimal  $(V, T)$ 

```

1 Ranger les sommets de  $V$  dans une file à priorité  $F$ , avec
  initialement la priorité  $\sigma(v) = +\infty$  pour tout sommet  $v$ .
2 tant que  $|T| < |V| - 1$  faire
3   Extraire le sommet  $v$  de  $F$  de priorité minimum
4   Marquer  $v$ 
5    $T \leftarrow T \cup \{\text{proche}(v)v\}$  (sauf à la première itération)
6   pour  $u$  voisin de  $v$  non marqué faire
7     si  $\rho(uv) < \sigma(u)$  alors
8       Diminuer la priorité  $\sigma(u) \leftarrow \rho(uv)$ 
9        $\text{proche}(u) \leftarrow v$ 
10 retourner arbre  $(V, T)$ 

```

**Théorème 3.75.** — *L'algorithme de Prim est correct par rapport à sa spécification.*

*Démonstration.* — La terminaison de l'algorithme est claire puisque un sommet est marché à chaque étape et  $T$  gagne un élément.

Notons  $v_0$  le tout premier sommet extrait de la file.

On peut facilement montrer les invariants suivants

- ( $\star_1$ ) Le graphe  $(M, T)$  est un arbre couvrant de la restriction du graphe  $G$  aux sommets marqués  $M$ .
- ( $\star_2$ ) Pour tout sommet  $v \in V \setminus M$ ,

$$\begin{cases} \sigma(v) &= \min_{u \in M} \rho(uv). \\ \text{proche}(v) &= \operatorname{argmin}_{u \in M} \rho(uv) \quad \text{si } v \in \Sigma. \end{cases}$$

Il reste à vérifier que l'arbre construit est un arbre de poids minimum. Notons  $e_1, e_2, \dots, e_{n-1}$  les arêtes de  $T$ , où  $T$  est l'arbre construit par l'algorithme. Soit  $T_0$  un arbre couvrant de poids minimum. Supposons que  $\rho(T) > \rho(T_0)$ . Comme  $T$  et  $T_0$  sont distincts, il existe un indice  $k$ , que nous prenons aussi petit que possible, que  $e_k \notin T_0$ . Si plusieurs arbres de poids minimum existent, nous choisissons celui qui coïncide le plus longtemps avec  $T$ , c'est-à-dire celui pour lequel  $k$  est le plus grand. Soient  $u$  et  $v$  les extrémités de  $e_k$ , avec disons  $u$  qui est marqué au moment  $e_k$  est ajouté à  $T$  et  $v$  non marqué. Comme  $T_0$  est un arbre couvrant, il existe un chemin  $C$  de  $u$  vers  $v$  dans  $T_0$ . Nous considérons la première arête  $f$  de ce chemin  $C$  qui relie un sommet marqué à un sommet non marqué en partant de  $u$ . Ajouter  $f$  à  $T_0$  crée un cycle passant  $e_k$ , que nous déconnectons en supprimant l'arête  $e_k$ . Aussi  $T_1 = T_0 \cup \{e_k\} \setminus \{f\}$  est un nouvel arbre couvrant. Il doit satisfaire  $\rho(T_1) \geq \rho(T_0)$  puisque  $T_0$  est minimal. Mais alors, on doit avoir  $\rho(f) \geq \rho(e_k)$ . Mais l'arête  $e_k$  vérifiait ( $\star_2$ ), donc  $\rho(e_k) \geq \rho(f)$ . Ainsi  $\rho(f) = \rho(e_k)$ . Mais alors  $T_1$  est un arbre minimal qui coïncide avec  $T$  sur une arête de plus : ceci est une contradiction.  $\square$

LISTING 3.15. Arbre couvrant de poids minimum - Algorithme de Prim

```

1067 class WeightedGraph():
1068     def __init__(self, n):
1069         self.n = n
1070         self.succ = [[] for _ in range(n)]
1071         self.pred = [[] for _ in range(n)]
1072         self.weight = dict()
1073     def add_edge(self, u, v, rho):
1074         self.succ[u].append(v)
1075         self.pred[v].append(u)
1076         self.weight[(u,v)] = rho
1077     def prim(self):
1078         visited = {0}
1079         T = set()
1080         d = dict({v:self.weight[(0,v)] for v in self.succ[0]})
1081         nearest = dict({v:0 for v in self.succ[0]})
1082         for _ in range(self.n - 1):
1083             v = min(d, key = d.get)
1084             visited.add(v)
1085             T.add((v, nearest[v]))
1086             d.pop(v)
1087             for u in self.succ[v]:
1088                 if u not in visited:
1089                     if u not in d or d[u] > self.weight[(v,u)]:
1090                         d[u] = self.weight[(v,u)]
1091                         nearest[u] = v
1092         return(T)

```

**Proposition 3.76.** — *L'implémentation 3.15 de l'algorithme de Prim a une complexité en temps  $O(n^2)$  où  $n$  est le nombre de sommets du graphe.*

*Démonstration.* — L'initialisation se fait en temps proportionnel au nombre de voisins du sommet 0. La boucle est répétée  $n - 1$  fois. La recherche du minimum est linéaire en  $n$ . La mise à jour de **d** et de **nearest** est proportionnel au nombre de voisins du sommet  $v$ . Au total, la complexité est donc  $O(m + n^2)$ . Mais  $m = O(n^2)$ . La complexité est donc  $O(n^2)$ .  $\square$

Avec des tas de Fibonacci, qui implémentent des files à priorité permettant d'abaisser des priorités en temps amorti constant, on obtiendrait une complexité  $O(m)$ .

### 3.5. Couplages et graphes bipartis

Dans cette section, nous nous intéressons à des problèmes d'appariement.



**Définitions 3.77.** — Un *couplage* dans un graphe non orienté  $G = (V, E)$  est un ensemble d'arête  $M \subseteq E$  dont les extrémités sont toutes distinctes.

Un couplage est dit *maximal* s'il est maximal pour l'inclusion, c'est-à-dire qu'il n'existe pas d'arête  $e \in E$  telle que  $M \cup \{e\}$  est encore un couplage.

Un couplage est dit *maximum* s'il est maximum pour le cardinal, c'est-à-dire qu'il n'existe pas de couplage  $M'$  avec  $|M| < |M'|$ .

Nous notons  $\nu(G)$  le cardinal d'un couplage maximum.

**3.5.1. Un algorithme approché glouton.** — Nous commençons par nous intéresser à un algorithme naïf de recherche de couplage.

---

**Algorithme 3.10 :** Recherche gloutonne d'un couplage

---

**Entrées :** Graphe non orienté  $G = (V, E)$

**Résultat :** Couplage  $M$  maximal

```

1  $M \leftarrow \emptyset, C \leftarrow \emptyset$ 
2 pour toute arête  $uv \in E$  faire
3   si  $u \notin C$  et  $v \notin C$  alors
4      $\quad$  Ajouter  $uv$  à  $M$ . Ajouter  $u$  et  $v$  à  $C$ .
5 retourner  $M$ 
```

---

**Théorème 3.78.** — L'algorithme 3.11 renvoie un couplage  $M$  dont le cardinal satisfait

$$\frac{\nu(G)}{2} \leq |M| \leq \nu(G).$$

*Démonstration.* — Il est assez clair que  $M$  est un couplage tout au long de l'exécution de l'algorithme 3.11. De plus, la condition d'arrêt de l'algorithme implique que ce couplage est finalement maximal. Comme  $M$  est un couplage, on a forcément

$$|M| \leq \nu(G).$$

Soit  $M_0$  un couplage de cardinal maximum, autrement dit un couplage tel que  $|M_0| = \nu(G)$ . Supprimons de  $M_0$  toute arête qui possède un sommet en commun avec une arête de  $M$ . Pour chaque arête de  $M$ , nous supprimons donc jusqu'à 2 arêtes de  $M_0$ . Au total, il reste donc au moins  $\nu(G) - 2|M|$  arêtes dans  $M$ . Ainsi, si  $|M| < \frac{\nu(G)}{2}$ , il reste au moins une arête  $e$  qui n'a aucune extrémité en commun avec  $M$ . Mais alors  $M \cup \{e\}$  est un couplage et  $M$  n'est pas maximal. En passant à la contraposée, on obtient l'inégalité  $\frac{\nu(G)}{2} \leq |M|$  attendue.  $\square$

**Remarque 3.79.** — Pour chacune des deux bornes du théorème 3.78, il existe des instances de graphe les atteignant. Ainsi, l'algorithme 3.11 n'est pas un algorithme qui calcul un couplage maximum. Cependant, on dit qu'il s'agit d'un *algorithme d'approximation* de facteur  $\frac{1}{2}$ , puisqu'il fournit une réponse qui, dans le pire des cas, s'écarte de l'optimum d'un facteur  $\frac{1}{2}$ .

**3.5.2. Un algorithme exact dans le cas biparti.** — Nous nous attaquons à un cas simple : le cas des graphes où les sommets peuvent être de deux couleurs et où les arêtes relient toujours des sommets de couleurs opposées.

**Définition 3.80.** — Un graphe non orienté  $G = (V, E)$  est dit *biparti* s'il existe une partition de l'ensemble des sommets en deux parts  $V = V_0 \sqcup V_1$  telle que l'ensemble des arêtes vérifie  $E \subseteq V_0 \times V_1 \cup V_1 \times V_0$ .

**3.5.2.1. Augmentation.** — Nous savons déjà qu'il est peine perdue de chercher un couplage maximum par une sélection gloutonne d'arêtes. Nous nous proposons d'organiser un troc plus large.

**Définition 3.81.** — Soit  $G = (V, E)$  un graphe biparti et  $M$  un couplage dans  $G$ . Nous appelons  $V_0$  et  $V_1$  les deux parts de la partition de l'ensemble des sommets  $V$ . Un *chemin augmentant* dans  $G$  par rapport à  $M$  est un chemin  $x_0x_1x_2 \cdots x_{2k}x_{2k+1}$  de  $G$  de longueur impaire entre un sommet  $x_0 \in V_0$  non apparié dans  $M$  et un sommet  $x_{2k+1} \in V_1$  non apparié dans  $M$  tel que pour tout indice  $i$  compris entre 0 et  $2k$ ,

$$\begin{cases} x_i x_{i+1} \notin M & \text{où } i \text{ est pair} \\ x_i x_{i+1} \in M & \text{où } i \text{ est impair.} \end{cases}$$

Comme  $x_0$  appartient à  $V_0$ , on a plus généralement que le sommet  $x_i$  appartient à  $V_0$  si l'indice  $i$  est pair et le sommet  $x_i$  appartient à  $V_1$  si l'indice  $i$  est impair.

Rechercher un chemin augmentant se fait par un parcours de graphe, dans lequel il est important de contrôler que l'arête employée est tantôt déjà dans le couplage et tantôt non encore dans le couplage. Dans le code suivant, la valeur  $-1$  est utilisée pour désigner un sommet factice qui n'existe pas.

LISTING 3.16. Recherche d'un chemin augmentant

```

1093 class Matching():
1094     def __init__(self):
1095         self.vertices0 = set()
1096         self.vertices1 = set()
1097         self.edges = set()
1098     def add_edge(self, u, v):
1099         self.vertices0.add(u)
1100         self.vertices1.add(v)
1101         self.edges.add((u,v))
1102     def remove_edge(self, u, v):
1103         self.edges.remove((u,v))
1104
1105 class BipartiteGraph():
1106     def __init__(self, n0, n1):
1107         self.n0 = n0
1108         self.n1 = n1
1109         self.neighbour0 = [[] for _ in range(n0)]
1110         self.neighbour1 = [[] for _ in range(n1)]

```

```

1111     self.weight = dict()
1112 def add_edge(self, u, v):
1113     self.neighbour0[u].append(v)
1114     self.neighbour1[v].append(u)
1115 def _find_aug_path_(self, M, queue, parent0, parent1):
1116     nextqueue = []
1117     for v0 in queue:
1118         for v1 in self.neighbour0[v0]:
1119             if (v0,v1) not in M.edges and v1 not in parent1:
1120                 parent1[v1]=v0
1121                 if v1 not in M.vertices1:
1122                     return(v1)
1123                 else:
1124                     for v2 in self.neighbour0[v1]:
1125                         if (v1,v2) in M.edges and v2 not in
1126                             ↪ parent0:
1127                             parent0[v2] = v1
1128                             nextqueue.append(v2)
1129     if not nextqueue:
1130         return -1
1131     return self._find_aug_path_(M,nextqueue, parent0, parent1)

```

**Lemme 3.82.** — Soit  $G = (V, E)$  un graphe biparti,  $M$  un couplage dans  $G$  et  $P$  un chemin augmentant de longueur  $2k + 1$ . Alors l'ensemble d'arêtes

$$M \triangle P = \begin{cases} M \cup \{x_{2i}x_{2i+1}; i \in \llbracket 0, k \rrbracket\} \\ \setminus \{x_{2i-1}x_{2i}; i \in \llbracket 1, k \rrbracket\} \end{cases}$$

est un couplage de  $G$ , contenant une arête de plus que  $M$ .

Nous traduisons ce lemme en code, qui calcule  $M \triangle P$  à partir d'un couplage  $M$  et d'un chemin augmentant  $P$

LISTING 3.17. Augmentation d'un couplage par un chemin augmentant

```

1131 class BipartiteGraph():
1132     # On complète ainsi la classe
1133     def _augment_(self, M):
1134         queue = [v0 for v0 in range(self.n0) if v0 not in
1135             ↪ M.vertices0]
1136         parent0 = dict({v0:-1 for v0 in queue})
1137         parent1 = dict()
1138         t = self._find_aug_path_(M, queue, parent0, parent1)
1139         if t == -1:
1140             return False
1141         else:
1142             while t != -1:

```

```

1142         u = parent1[t]
1143         M.add_edge(u,t)
1144         t = parent0[u]
1145         if t!=-1:
1146             M.remove_edge(u,t)
1147     return True

```

**Proposition 3.83 (Lemme de Berge).** — Soit  $G = (V, E)$  un graphe biparti et  $M$  un couplage dans  $G$ . Alors  $M$  est un couplage maximum si et seulement s'il n'existe pas de chemin augmentant dans le graphe  $G$  par rapport à  $M$ .

*Démonstration.* — Le lemme 3.82 montre que l'existence d'un chemin augmentant permet de construire un nouveau couplage  $M'$  avec  $|M'| = |M| + 1$ , ce qui signifie que  $M$  n'est pas un couplage maximum. Nous venons de prouver le sens direct par la contraposée : si  $M$  est un couplage maximum, alors il n'existe pas de chemin augmentant dans le graphe  $G$  par rapport à  $M$ .

Supposons que  $M$  n'est pas un couplage maximum et considérons un couplage  $M_0$  tel que  $|M_0| > |M|$ . Autour de chaque sommet  $v \in V$ , différents cas peuvent se produire :

- il existe une arête d'extrémité  $v$  appartenant à la fois à  $M$  et à  $M_0$ .
- il existe deux arêtes d'extrémité  $v$ , l'un appartenant à  $M$  et l'autre à  $M_0$ .
- il existe une arête d'extrémité  $v$  appartenant à  $M$
- il existe une arête d'extrémité  $v$  appartenant à  $M_0$
- il n'existe pas d'arête d'extrémité  $v$  ni dans  $M$  ni dans  $M_0$ .

Ainsi, la réunion  $M \cup M_0$  est formé d'arêtes doubles et de chemins disjoints, qui alternent entre une arête de  $M$  et une arête de  $M_0$ . Nous supprimons de  $M \cup M_0$  les arêtes doubles et les chemins de longueur paire, ce qui nous conduit à supprimer autant d'arêtes de  $M$  que de  $M_0$ . Il nous reste un ensemble de chemins disjoints, de longueur impaire, avec globalement plus d'arêtes issues de  $M_0$  que de  $M$ . L'un d'entre eux doit commencer et se terminer par des arêtes de  $M_0$  et constitue un chemin augmentant dans  $G$  par rapport à  $M$ . Ainsi si  $M$  est un couplage non maximum, alors existe un chemin augmentant dans le graphe  $G$  par rapport à  $M$ .  $\square$

La proposition justifie l'algorithme suivant.

---

**Algorithme 3.11 :** Recherche d'un couplage dans un graphe biparti

---

**Entrées :** Graphe biparti  $G = (V_1 \sqcup V_2, E)$

**Résultat :** Couplage  $M$  maximum

---

1  $M \leftarrow \emptyset$

2 **tant que** il existe un chemin augmentant  $P$  **faire**

3      $M \leftarrow M \triangle P$

4 **retourner**  $M$

---

LISTING 3.18. Construction d'un couplage maximum

```

1148 class BipartiteGraph():
1149     # On complète ainsi la classe
1150     def maximum_matching(self):
1151         M = Matching()
1152         while self._augment_(M):
1153             pass
1154         return(M)

```

**3.5.2.2. Certificat d'optimalité.** — Voyons comment justifier qu'un couplage est maximal sans avoir à le calculer nous-même.

**Définition 3.84.** — Soit  $G = (V, E)$  un graphe. On dit qu'un ensemble de sommets  $K \subseteq V$  est une *couverture* de  $G$  si toute arête de  $E$  est incidente à un sommet de  $K$ .

**Lemme 3.85.** — Soit  $M$  un couplage et  $K$  une couverture dans un graphe quelconque, alors on a l'inégalité

$$|M| \leq |K|.$$

*Démonstration.* — Comme  $K$  est une couverture, au moins une des extrémités des arêtes de  $M$  appartient à  $K$ . Pour toute arête  $e \in M$ , nous notons  $v_e$  l'une d'entre elle. Chacune des extrémités  $(v_e)_{e \in M}$  est distinctes, car  $M$  est un couplage. Donc l'ensemble  $V_0 = \{v_m; m \in M\}$  est un ensemble de cardinal  $|M|$ . Mais alors, comme  $V_0 \subseteq K$ , on a

$$|M| \leq |K|$$

comme attendu. □

**Théorème 3.86 (König, Egervary).** — Soit  $G = (V, E)$  un graphe biparti. Le cardinal maximum d'un couplage est égal au cardinal minimum d'une couverture.

*Démonstration.* — Soit  $Z$  l'ensemble des sommets accessibles dans la dernière recherche de chemin augmentant. Posons

$$K = (V_0 \setminus Z) \cup (V_1 \cap Z).$$

Commençons par démontrer que  $K$  est bien une couverture. Soit  $e = (v_0, v_1) \in E$  avec  $v_0 \in V_0$  et  $v_1 \in V_1$ . Si  $v_0 \notin Z$ , il n'y a rien à démontrer : l'arête  $e$  est couverte par  $v_0 \in K$ . Supposons que  $v_0 \in Z$ .

- Cas où  $e \in M$ . Le sommet  $v_0$  étant dans  $Z$ , soit le sommet  $v_0$  est la racine d'une des arborescences de parcours par lesquels on a cherché un chemin augmentant pour la dernière fois, soit le sommet  $v_0$  a été accédé en cours de parcours depuis une arête de  $M$ . Dans le premier cas, aucune arête de  $M$  n'est incidente à  $v_0$  (car  $v_0$  serait apparié et nous nous n'aurions pas pu partir de  $v_0$ ) : c'est donc un cas impossible. Dans le second cas, nous prenons acte qu'il y a une arête de  $M$  incidente à  $v_0$ . Mais comme  $M$  est un couplage, cela ne peut être que l'arête  $e$  que nous voulions couvrir.

- Cas où  $e \notin M$ . Comme  $v_0$  appartient à  $Z$ ,  $v_0$  est l'un des sommets auquel on accède quand on cherche un chemin augmentant pour la dernière fois. Mais dans cette recherche, on continue ensuite l'exploration en traversant toute arête n'appartenant pas à  $M$ . En particulier, on traverse l'arête  $e = (v_0, v_1)$ . Ainsi  $v_1$  est forcément visité et appartient à  $Z$ .

Dans les deux cas, l'arête  $e$  est couverte par le sommet  $v_1 \in K$ .

Il reste à voir que l'on a l'inégalité  $|K| \leq |M|$ . Pour cela, montrons d'abord que tout sommet de  $K$  est une extrémité d'une arête de  $M$ .

Pour tout sommet  $v_0 \in V_0$ , s'il n'existe pas d'arête de  $M$  incidente à  $v_0$ , nous démarrons un parcours depuis  $v_0$ . Par conséquent  $v_0$  appartient à  $Z$ . Donc tout sommet de  $V_0 \setminus Z$  est incident à une arête de  $M$ . Par ailleurs, si  $v_1 \in V_1 \cap Z$ , c'est qu'une arête de  $M$  arrive en  $v_1$ . Enfin, aucune arête de  $M$  ne peut avoir que l'une de ses deux extrémités dans  $Z$ . Donc  $K$  ne contient que l'un des deux extrémités de  $M$  au plus. Ainsi  $|K| \leq |M|$ .

Mais alors, à cause du lemme 3.85, on a même

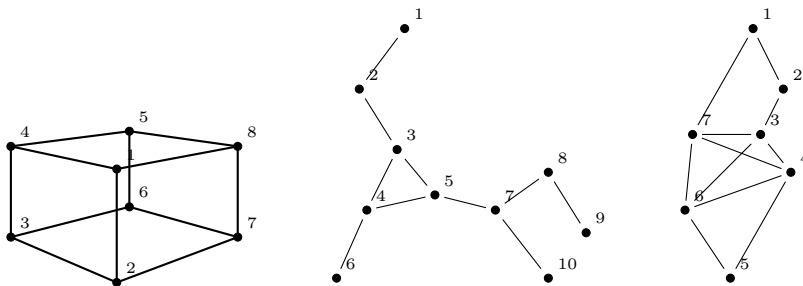
$$|K| = |M|.$$

comme attendu. □

**Remarque 3.87.** — On peut tirer du théorème 3.86 une observation importante. Il est possible de convaincre un tiers qu'un calcul de couplage optimum est correct sans lui demander de réexécuter tout l'algorithme de recherche d'un couplage maximum. Il suffit de fournir une couverture en guise de garantie. Le tiers voulant contrôler la solution peut se contenter de vérifier qu'on lui a bien fourni un couplage, une couverture et que les deux éléments sont de même cardinal.

### 3.6. Exercices

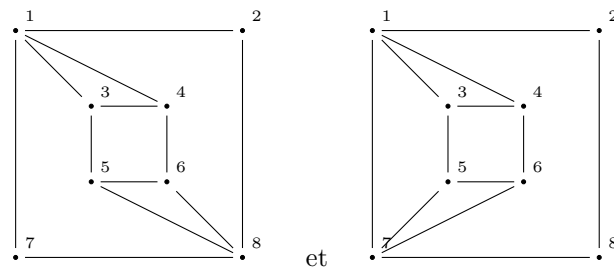
**Exercice 3.88.** — On donne les graphes suivants .



1. Représenter chaque graphe par une matrice puis par un tableau de liste d'adjacences.
2. Que représente la somme des éléments d'une ligne et la somme des éléments d'une colonne des matrices d'adjacence ?

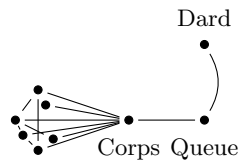
3. Soit  $A$  la matrice d'adjacence de l'un des graphes ci-dessus et  $B = A^2$ , quelle relation y-a-t-il entre le coefficient  $(B_{i,j})$  et le nombre de chaînes de longueur 2 entre les sommets  $i$  et  $j$  ?

**Exercice 3.89.** — Les deux graphes suivants sont-ils isomorphes ?



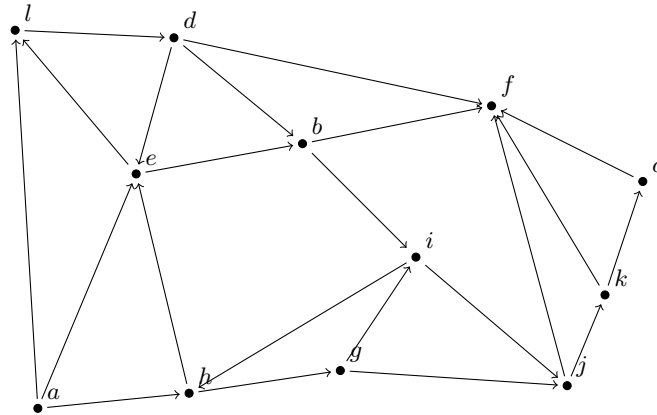
**Exercice 3.90.** — On dit qu'un graphe non orienté est  $d$ -régulier si chaque sommet est de degré  $d$ . Montrer que si  $G$  est un graphe  $d$ -régulier avec un nombre impair  $n$  de sommets,  $d$  est forcément pair.

**Exercice 3.91.** — Un *scorpion* est un graphe non-orienté à  $n$  sommets qui comprend trois sommets spéciaux appelés dard, queue et corps. Le dard est de degré 1 et est relié à la queue ; la queue est de degré 2 et est relié au dard et au corps ; le corps est de degré  $n - 2$  et est relié au corps et aux  $n - 3$  sommets restants. Des arêtes peuvent exister entre certains des  $n - 3$  sommets restants.



Suggérer un algorithme décidant en temps  $O(n)$  si un graphe représenté par sa matrice d'adjacence  $A$  est un scorpion.

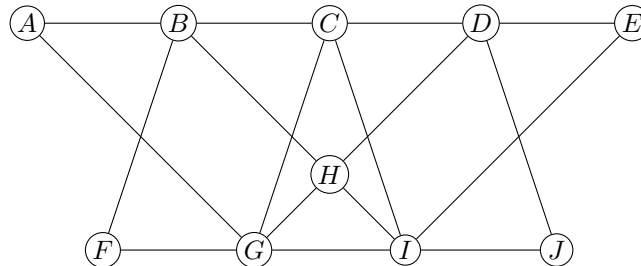
**Exercice 3.92.** — Voici un graphe orienté :



1. Donner l'ordre des sommets rencontrés au départ de  $a$  par un parcours DFS (parcours en profondeur).
2. Donner l'ordre des sommets rencontrés au départ de  $a$  par un parcours BFS (parcours en largeur)
3. Reprenez l'exercice avec le graphe induit par  $G$  si l'on oublie l'orientation des arêtes.

Les voisins d'un sommet seront étudiés dans l'ordre croissant de leur identifiant.

**Exercice 3.93.** — On considère le graphe suivant :

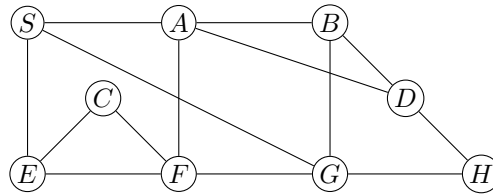


Est-il possible que l'algorithme de parcours en largeur étudie les sommets dans l'ordre suivant ?

1. H, B, D, G, I, C, A, F, J, E ?
2. F, B, A, G, C, H, I, D, E, J ?
3. J, D, I, C, E, G, H, A, F, B ?
4. A, B, G, C, H, F, I, D, E, J ?

**Exercice 3.94.** — On considère le graphe suivant





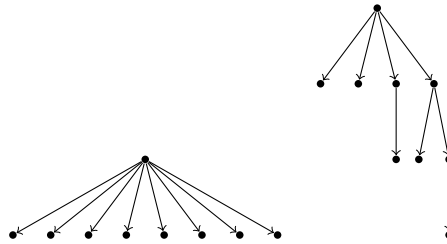
1. Compléter la suite de sommet suivante pour qu'elle corresponde à l'ordre des sommets rencontrés en effectuant un parcours de graphe en largeur.

$S, \square, \square, \square, H, \square, F, C, \square.$

Dessiner l'arborescence de parcours correspondante.

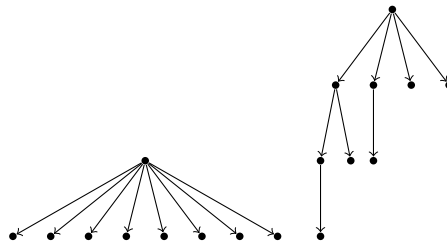
2. Est-il possible que l'arc  $(D, H)$  appartienne à l'arborescence de parcours d'un BFS issu du sommet  $S$ ?

**Exercice 3.95.** — Est-il possible que, en effectuant des parcours en largeur au départ de deux sommets distincts d'un même graphe connexe, on obtienne comme arborescence de parcours les arborescences suivante ?



**Exercice 3.96.** — Soient  $G = (V, E)$  un graphe non orienté et  $r, s$  deux sommets distincts de  $V$ . On suppose que dans un parcours en largeur démarré au sommet  $r$ , le sommet  $s$  a été le cinquième sommet rencontré. Existe-t-il forcément une exécution d'un parcours en largeur démarré au sommet  $s$  telle que  $r$  soit le cinquième sommet rencontré ?

**Exercice 3.97.** — Est-il possible que, en effectuant des parcours en profondeur au départ de deux sommets distincts d'un même graphe connexe, on obtienne comme arborescence de parcours les arborescences suivante ?

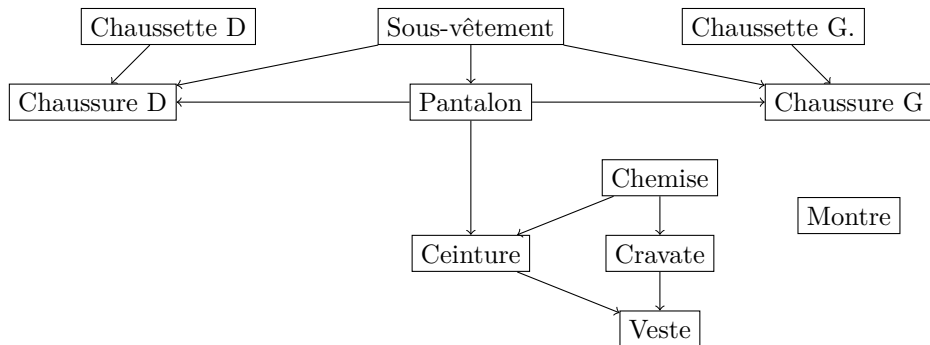


**Exercice 3.98.** — Modifier l'un des algorithmes de parcours de graphe afin qu'il retourne pour chaque sommet sa distance à un sommet  $s$  fixé.

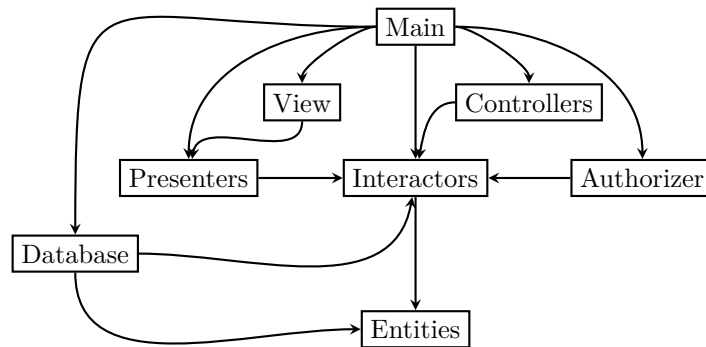
**Exercice 3.99.** — Proposer un algorithme qui calcule le nombre de sommets atteignables depuis un sommet donné.

**Exercice 3.100.** — Vous vous trouvez dans un labyrinthe. Quel algorithme entre DFS et BFS utilisez-vous pour retrouver la sortie ?

**Exercice 3.101.** — Votre ami doit se vêtir pour un rendez-vous important. Dans quel ordre l'habillez-vous avec un algorithme du cours ?

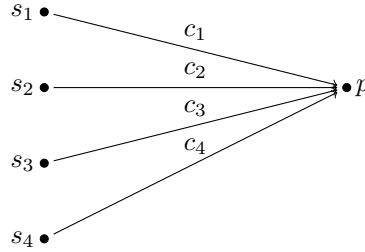


**Exercice 3.102.** — Une application logicielle est formée de plusieurs composants, dont les dépendances sont exprimées par le graphe suivant.



Lorsqu'une nouvelle version de l'application est publiée, chaque composant est compilé, testé puis publié. Dans quel ordre suggérez-vous de procéder pour construire l'ensemble du système ?

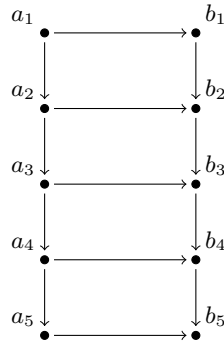
**Exercice 3.103.** — Vous travaillez pour le compte d'une grande industrie. Vous êtes chargé de concevoir un système permettant de retrouver le nombre de composants élémentaires utilisés pour la production d'un certain produit. On représente le fait que  $c_1$  pièces  $s_1$ ,  $c_2$  pièces  $s_2$ ,  $\dots$ ,  $c_k$  pièces  $s_k$  sont nécessaires à la production de la pièce  $p$  par un graphe orienté



Par exemple, pour faire une bicyclette, il faut 2 roues, 1 cadre, 1 guidon, etc. Pour faire 1 guidon, il faut 1 barre de guidon, 2 freins, etc. Une même pièce peut être utilisée pour produire des éléments différents

1. Préciser (et justifier) la structure de donnée  $G$  (et son implémentation) que vous utilisez pour représenter l'ensemble des informations.
2. Détailler un algorithme qui renvoie la liste des composants élémentaires utilisés pour la production d'un certain produit avec leur nombre d'occurrence total.
3. Donner le nombre de composants élémentaires nécessaires à la fabrication d'un avion Airbus (si vous ne connaissez pas la réponse, remplacer Airbus par Boeing).

**Exercice 3.104.** — Un parenthésage valide est une suite de  $n$  parenthèses ouvrantes et  $n$  fermantes telles que pour tout  $k$ , les  $k$  premières parenthèses contiennent plus de parenthèses ouvrantes que fermantes. Par exemple,  $((()))$  est valide mais  $((()))()$  n'est pas valide. Montrer que les parenthésages valides sont en bijection avec les tris topologiques du graphe suivant :



**Exercice 3.105.** — Le graphe orienté suivant (cf. fig 3.1), adapté d'une publication aux PNAS<sup>(1)</sup>, représente par des sommets un ensemble de gènes et par des arcs des mécanismes de régulation d'un gène par un autre. Retrouver les composantes fortement connexes de ce graphe et tracer le graphe orienté acyclique induit par ses composantes fortement connexes.

1. Q. Zhou, H. Chipperfield, D. A. Melton, and W. H. Wong. *A gene regulatory network in mouse embryonic stem cells*. Proceedings of the National Academy of Sciences of the United States of America, 104(42):16438–16443, 2007.

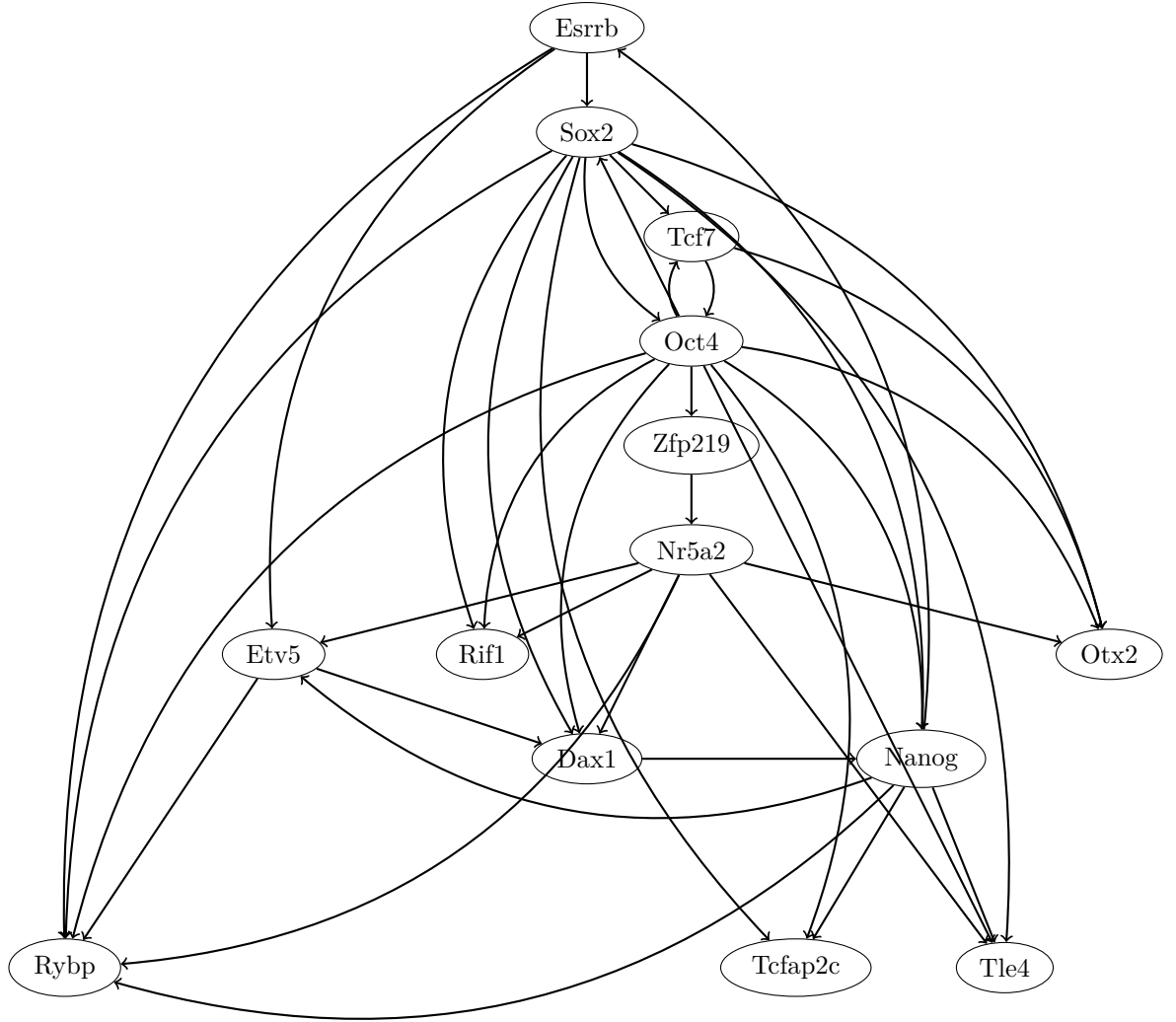


FIGURE 3.1. Graphe de l'exercice 3.105

**Exercice 3.106.** — On donne le graphe orienté  $G = (V, E)$  suivant par un tableau de listes d'adjacence.

|             |     |        |           |     |        |     |        |        |        |     |
|-------------|-----|--------|-----------|-----|--------|-----|--------|--------|--------|-----|
| Sommet      | $A$ | $B$    | $C$       | $D$ | $E$    | $F$ | $G$    | $H$    | $I$    | $J$ |
| Successeurs | $Q$ | $H, O$ | $E, L, S$ | $M$ | $B, S$ | $R$ | $F, Q$ | $I, N$ | $B, O$ | $K$ |

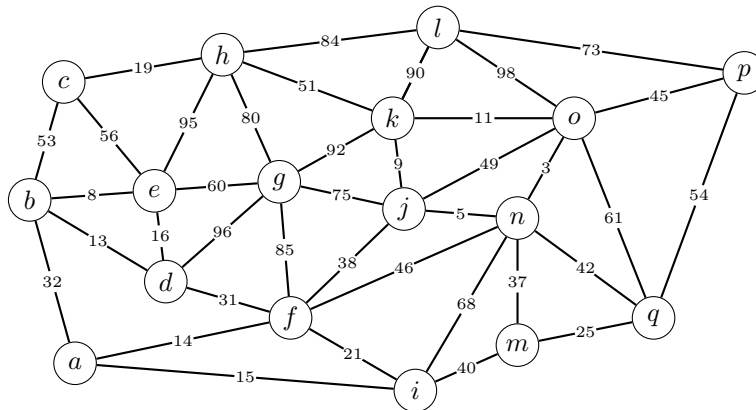
|             |        |        |        |           |     |        |        |           |        |
|-------------|--------|--------|--------|-----------|-----|--------|--------|-----------|--------|
| Sommet      | $K$    | $L$    | $M$    | $N$       | $O$ | $P$    | $Q$    | $R$       | $S$    |
| Successeurs | $J, N$ | $E, R$ | $F, P$ | $B, H, I$ | $N$ | $D, L$ | $A, H$ | $A, F, G$ | $C, J$ |

Calculer les composantes fortement connexes de  $G$  et dessiner le graphe orienté acyclique formé par les composantes fortement connexes. (Lors de l'application de l'algorithme, privilégier les sommets selon leur ordre alphabétique)

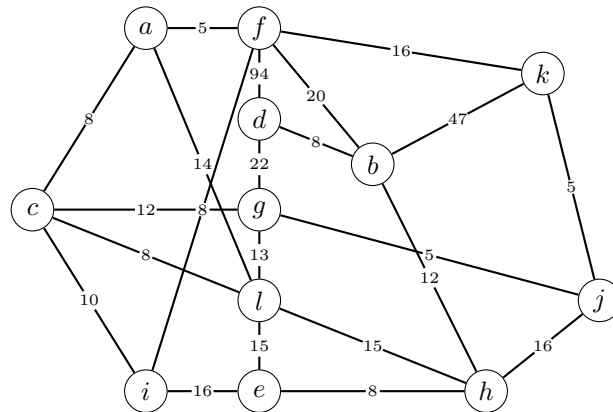
**Exercice 3.107.** — Vous dînez chez M. l'ambassadeur qui reçoit  $n$  invités dont vous ne connaissez pas l'opinion sur un sujet d'importance et à qui il ne serait pas diplomatique de la réclamer. Vous surprenez à  $m$  reprises une conversation entre deux quelconques des invités. De chaque conversation, vous en déduisez que les deux personnes ont les mêmes opinions, des opinions opposées ou vous ne pouvez pas conclure. Votre chancellerie vous demande de vérifier l'hypothèse selon laquelle les opinions sont partagées en (moins de) deux camps. Vous répondrez par un algorithme de complexité  $O(n + m)$ .

**Exercice 3.108.** — Donner une CNS pour qu'un graphe orienté acyclique n'admette qu'une seule numérotation topologique.

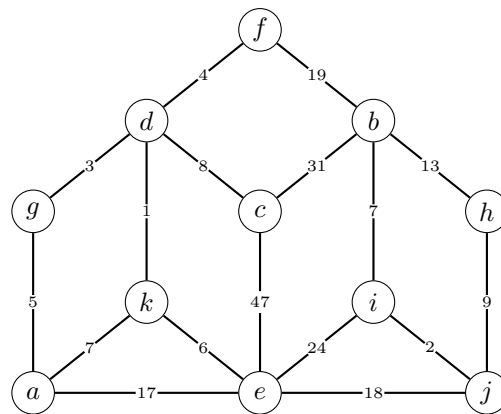
**Exercice 3.109.** — Déterminer un arbre couvrant de poids minimum dans le graphe suivant



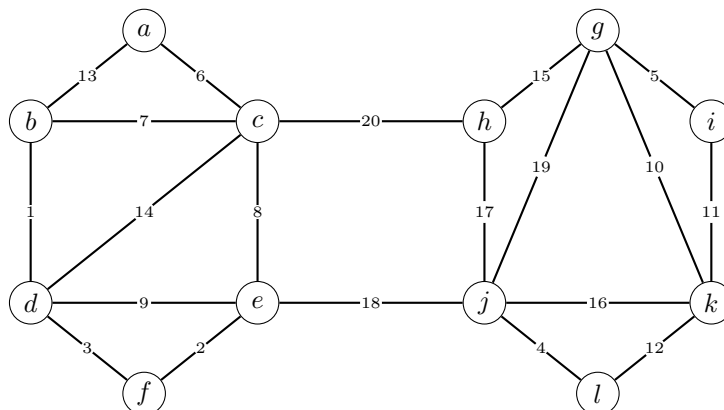
**Exercice 3.110.** — Déterminer un arbre couvrant de poids minimum dans le graphe suivant



**Exercice 3.111.** — Déterminer un arbre couvrant de poids minimum dans le graphe suivant



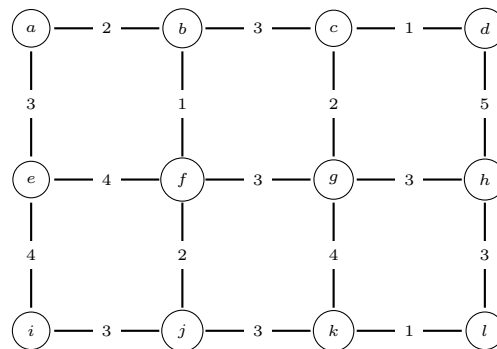
**Exercice 3.112.** — Déterminer un arbre couvrant de poids minimum dans le graphe suivant



**Exercice 3.113.** — Un opérateur de télécom souhaite cabler cinq villes entre elles. Le coût de cablage entre deux villes est donné par la table suivante. Quel réseau proposez-vous de construire ?

$$\begin{pmatrix} 0 & 3 & 5 & 11 & 9 \\ 3 & 0 & 3 & 9 & 8 \\ 5 & 3 & 0 & +\infty & 10 \\ 11 & 9 & +\infty & 0 & 7 \\ 9 & 8 & 10 & 7 & 0 \end{pmatrix}$$

**Exercice 3.114.** — On donne le graphe pondéré suivant.



Trouver un arbre couvrant minimal en appliquant successivement l'algorithme de Kruskal et l'algorithme de Prim.

**Exercice 3.115.** — Soit  $G = (V, E)$  un graphe et  $T$  un arbre couvrant de poids minimum.

1. Si on augmente le poids de chaque arête de  $G$  d'une constante  $c$ , est-ce que l'arbre  $T$  reste minimal ?
2. Si on diminue le poids d'une arête de  $T$ ,  $T$  est-il encore un arbre couvrant minimal ?

**Exercice 3.116.** — Soit  $G = (V, E)$  un graphe connexe où chaque arête  $e \in E$  porte un poids  $w(e) \in \mathbb{N}$ .

1. Montrer que si les poids des arêtes sont tous différents, alors il existe un unique arbre couvrant de poids minimum.
2. Vous disposez d'un algorithme de calcul d'un arbre couvrant de poids minimal, mais qui ne fonctionne que lorsque les poids des arêtes sont tous différents. Trouver un moyen de l'utiliser pour effectuer la calcul d'un arbre optimum quand les arêtes peuvent avoir des poids identiques.

**Exercice 3.117.** — Soit  $G = (V, E)$  un graphe et  $S \subseteq E$  un ensemble d'arêtes ne comptant pas de cycle. Modifier l'algorithme de Prim afin de trouver un arbre couvrant de poids minimal contenant les arêtes  $S$ .

**Exercice 3.118.** — Considérons un ensemble  $V$  de  $n$  objets  $v_1, v_2, \dots, v_n$  distincts et une fonction pseudo-distance  $d : V \times V \rightarrow \mathbb{Q}_+$  telle que  $d(v_i, v_j) = 0$  ssi  $i = j$  (séparation) et  $d(v_i, v_j) = d(v_j, v_i)$  (symétrie).

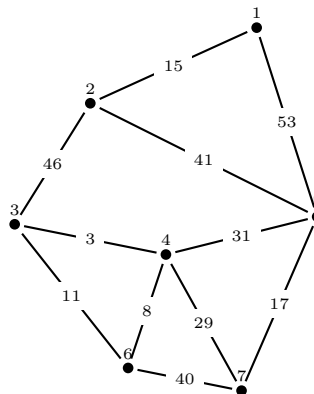
Un  $k$ -partitionnement de données (ou  $k$ -clustering) consiste à répartir les objets  $v_i$  en  $k$  sous-ensembles disjoints  $V_1, \dots, V_k$  de  $V$ , appelés *paquets* ou *clusters*. L'*espacement* entre deux paquets  $V_i$  et  $V_j$  est la pseudo-distance minimale entre les éléments de  $V_i$  et les éléments de  $V_j$ .

Proposer un algorithme qui calcule un  $k$ -partitionnement de données d'espacement maximal. Justifier sa correction et estimer sa complexité.

**Exercice 3.119.** — Donner un exemple de graphe tel que l'arbre couvrant de poids minimum coïncide avec l'arborescence des plus courts chemins au départ d'un certain sommet.

**Exercice 3.120.** — 1. Étant donné un graphe non-orienté  $G = (V, E)$  et un chemin entre deux sommets  $s$  et  $t$ , on appelle *longueur d'étranglement* (ou *bottleneck length*) le poids maximum des arêtes rencontrées sur ce chemin. La *distance d'étranglement* (ou *bottleneck distance*) entre  $s$  et  $t$  est la plus petite longueur d'étranglement des chemins entre  $s$  et  $t$ . Décrire un algorithme qui calcule la distance d'étranglement entre tout couple de sommets  $(s, t) \in V^2$ . On supposera que toutes les arêtes ont des poids distincts.

2. Le graphe ci-dessous représente la bande passante disponible entre différents routeurs connectés entre eux. Comment calculer la bande passante disponible entre deux routeurs quelconques (problème du *chemin le plus large*) ?



De quelle bande passante dispose-t-on entre les sommets 3 et 6 ?

**Exercice 3.121.** — On considère le graphe complet à 1000 sommets  $G = (V, E)$  où  $V = \{v_i; i \in \llbracket 1, 1000 \rrbracket\}$ . Pour toute arête  $e = (v_i, v_j) \in E$ , on fixe le poids de  $e$  à  $w(e) = \max(i, j)$ . Quel est le poids d'un arbre couvrant de poids minimum ? Combien y a-t-il d'arbres de ce poids ?

**Exercice 3.122.** — On considère le graphe complet à 1000 sommets  $G = (V, E)$  où  $V = \{v_i; i \in \llbracket 1, 1000 \rrbracket\}$ . Pour toute arête  $e = (v_i, v_j) \in E$ , le poids de l'arête  $e$  est fixé



à

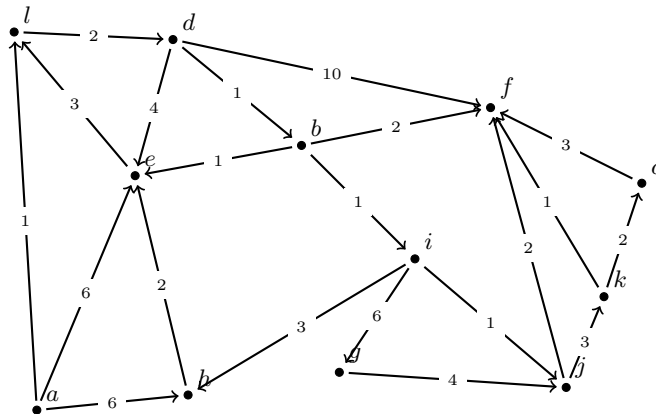
$$w(e) = \begin{cases} 1 & \text{si } i \leq 500 \text{ et } j \leq 500 \\ 2 & \text{si } i \geq 501 \text{ et } j \geq 501 \\ 3 & \text{sinon} \end{cases}$$

Quel est le poids d'un arbre couvrant de poids minimum ? Combien y a-t-il d'arbres de ce poids ?

**Exercice 3.123.** — Un étudiant de Centrale a inventé un nouvel algorithme, de type « diviser pour régner », pour résoudre le problème de l'arbre couvrant minimal. L'algorithme est le suivant. Soit un graphe  $G = (V, E)$ . Diviser l'ensemble de sommets  $V$  en deux parties disjointes  $V_1$  et  $V_2$  telles que la différence entre les cardinaux  $|V_1|$  et  $|V_2|$  est inférieure à 1. On pose  $E_1 = \{(v, v') \in E \cap V_1 \times V_1\}$  et  $E_2 = \{(v, v') \in E \cap V_2 \times V_2\}$ . Calculer un arbre couvrant  $T_1$  de poids minimum dans le sous-graphe  $G_1 = (V_1, E_1)$  en appliquant récursivement l'algorithme. Calculer de même un arbre  $T_2$  qui couvre le sous-graphe  $G_2 = (V_2, E_2)$ . Enfin, rechercher l'arête  $e = (v_1, v_2)$  de plus petit poids telle que  $v_1 \in V_1$  et  $v_2 \in V_2$ . Renvoyer l'arbre  $T_1 \cup T_2 \cup \{e\}$ .

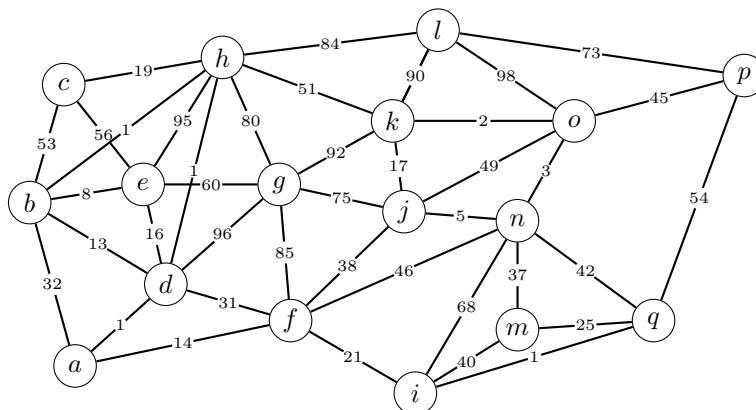
Que pensez-vous de cet algorithme ?

**Exercice 3.124.** — Voici un graphe orienté  $G = (V, E)$  :

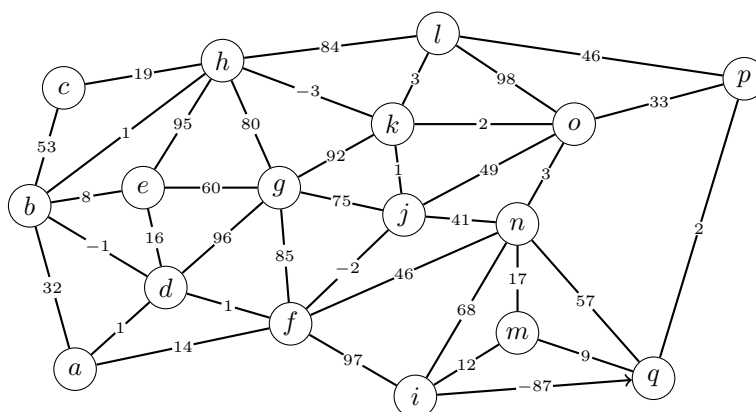


1. Appliquer l'algorithme DIJKSTRA( $a$ ) à  $G$ .
2. Appliquer l'algorithme DIJKSTRA( $a$ ) au graphe non-orienté obtenu en oubliant l'orientation des arcs, autrement dit le graphe  $G' = (V, E')$  avec  $E' = \{(u, v); (u, v) \in E\} \cup \{(v, u); (u, v) \in E\}$ .
3. Appliquer l'algorithme FORD( $a$ ) à  $G$ .

**Exercice 3.125.** — Calculer l'ensemble des plus courts chemins issus du sommet  $j$  dans le graphe non orienté suivant en appliquant l'algorithme de Dijkstra puis l'algorithme de Bellman-Ford.



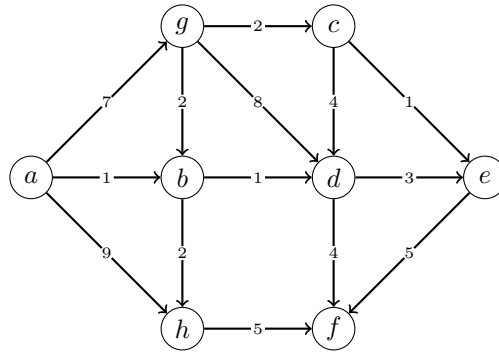
**Exercice 3.126.** — On s'intéresse au graphe orienté ci-dessous.



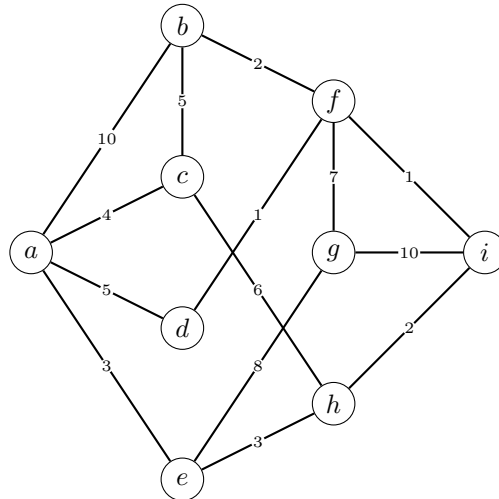
(L'arc  $(i, q)$  est orienté, les autres arêtes sont des arcs dans les deux sens.)

1. Calculer l'arborescence de parcours obtenu en appliquant l'algorithme de Dijkstra au départ du sommet  $o$ .
2. Constaté qu'il existe des sommets  $v$  pour lesquels la longueur de chemin obtenue n'est pas la distance entre le sommet  $o$  et le sommet  $v$ .
3. Repérer une arête qui, ajouté à l'arborescence comme il est fait dans l'algorithme de Ford-Dantzig, conduit à une correction de l'arborescence.
4. Repérer une arête qui, ajouté à l'arborescence comme il est fait dans l'algorithme de Ford-Dantzig, permet de détecter un cycle absorbant.

**Exercice 3.127.** — Déterminer la distance entre le sommet  $a$  et tous les autres sommets du graphe suivant. Calculer des plus courts chemins associés.



**Exercice 3.128.** — Déterminer la distance entre le sommet  $a$  et tous les autres sommets du graphe suivant. Calculer des plus courts chemins associés.

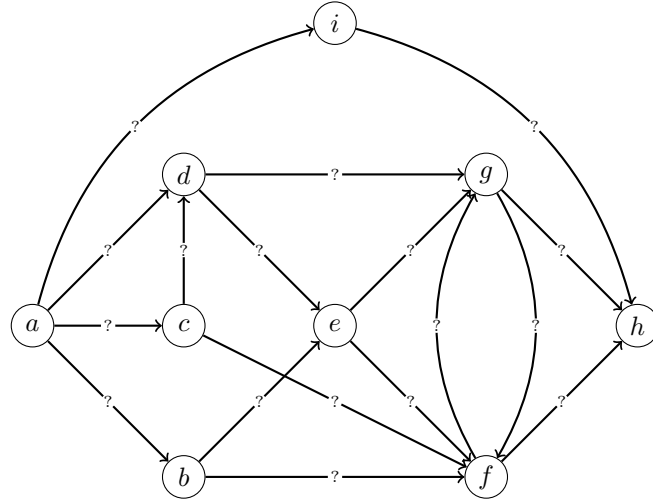


**Exercice 3.129.** — Soit  $G$  un graphe orienté complet dont l'ensemble des sommets est  $\{a, b, c, d, e, f\}$  et dont les poids sont donnés par le tableau suivant

|     | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|-----|-----|-----|-----|-----|-----|-----|
| $a$ | 0   | 12  | 8   | 43  | 79  | 35  |
| $b$ | 93  | 0   | 18  | 21  | 60  | 33  |
| $c$ | 17  | 3   | 0   | 37  | 50  | 30  |
| $d$ | 85  | 10  | 91  | 0   | 17  | 7   |
| $e$ | 28  | 47  | 39  | 14  | 0   | 108 |
| $f$ | 31  | 7   | 29  | 73  | 20  | 0   |

Appliquer l'algorithme de Dijkstra au départ du sommet  $a$ .

**Exercice 3.130.** — On a exécuté l'algorithme de Dijkstra sur le graphe  $G = (V, E)$  suivant



et on a obtenu la trace d'exécution suivante.

| Étape | <i>a</i> | <i>b</i>  | <i>c</i>  | <i>d</i>  | <i>e</i>  | <i>f</i>  | <i>g</i>  | <i>h</i>  | <i>i</i>  |
|-------|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0     | 0        | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| 1     | 0        | 3         | 2         | 4         | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | 34        |
| 2     | 0        | 3         | 2         | 4         | $+\infty$ | 14        | $+\infty$ | $+\infty$ | 34        |
| 3     | 0        | 3         | 2         | 4         | 12        | 13        | $+\infty$ | $+\infty$ | 34        |
| 4     | 0        | 3         | 2         | 4         | 6         | 13        | 8         | $+\infty$ | 34        |
| 5     | 0        | 3         | 2         | 4         | 6         | 12        | 7         | $+\infty$ | 34        |
| 6     | 0        | 3         | 2         | 4         | 6         | 8         | 7         | 37        | 34        |
| 7     | 0        | 3         | 2         | 4         | 6         | 8         | 7         | 36        | 34        |
| 8     | 0        | 3         | 2         | 4         | 6         | 8         | 7         | 35        | 34        |
| 9     | 0        | 3         | 2         | 4         | 6         | 8         | 7         | 35        | 34        |

Retrouver un encadrement des pondérations des arêtes de  $E$  aussi précis que possible.

**Exercice 3.131.** — Avec une source d'eau intarissable, un seau de 3 litres et un seau de 5 litres, mesurez 4 litres l'eau.

Source : question d'entretien d'embauche Microsoft (d'après [www.mycareerstack.com](http://www.mycareerstack.com)).

**Exercice 3.132.** — On considère un graphe pondéré  $G = (V, E, p)$  et deux sommets  $s$  et  $t$ .

1. Si l'on multiplie les poids  $p$  par une constante, le plus court chemin entre  $s$  et  $t$  reste-t-il le même ?
2. Si l'on ajoute une constante  $c > 0$  à tous les poids, le plus court chemin entre  $s$  et  $t$  reste-t-il le même ?

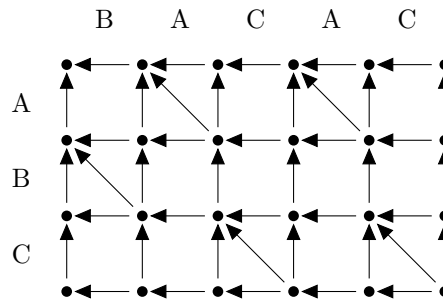
**Exercice 3.133.** — Étant donné un graphe pondéré  $G = (V, E, p)$  dont les poids sont tous strictement positifs et deux sommets  $s$  et  $t$ , quel algorithme préconisez-vous pour calculer un plus court chemin entre  $s$  et  $t$  entre les algorithmes de Dijkstra

et Bellman-Ford? Donnez un exemple de graphe à  $n$  sommets où l'algorithme de Dijkstra coûte  $O(1)$  et celui de Bellman-Ford  $O(n^3)$ .

**Exercice 3.134.** — Étant donnée deux suites finies  $x = x_1x_2\dots x_m$  et  $y = y_1y_2\dots y_n$ , le problème PLUSLONGUESOUSSUITECOMMUNE (ou LCS d'après *longest common subsequence*) est le problème de déterminer une suite  $z = z_1z_2\dots z_k$  aussi longue que possible telle qu'il existe des indices  $1 \leq i_1 < i_2 < \dots < i_k \leq m$  et  $1 \leq j_1 < j_2 < \dots < j_k \leq n$  satisfaisant

$$\forall 1 \leq \ell \leq k, \quad z_\ell = x_{i_\ell} = y_{j_\ell}.$$

1. Donner l'ensemble des plus longues sous-suites communes à ABC et ACB.
2. On construit un graphe à  $(m+1) \cdot (n+1)$  sommets en forme de quadrillage  $(m+1) \times (n+1)$ . Si  $x_i = y_j$ , on ajoute une arête diagonale dans la  $i$ -ième ligne et  $j$ -ième colonne. Voici un exemple avec les suites ABC et BACAC.



Montrer que LCS peut être exprimé comme un problème de plus court chemin dans le graphe  $G$  en choisissant bien les sommets initial et final ainsi que les poids à donner aux arêtes.

3. Quel algorithme est le plus approprié pour résoudre le problème du plus court chemin? Quel est son coût?

**Exercice 3.135.** — Vous travaillez pour une entreprise de BTP et êtes chargé de l'affectation de travailleurs à des chantiers. Un chantier particulier nécessite le nombre d'ouvriers intérimaires suivant par mois

| Mois     | Mars | Avril | Mai | Juin | Juillet | Août |
|----------|------|-------|-----|------|---------|------|
| Ouvriers | 4    | 6     | 7   | 4    | 6       | 2    |

Les mouvements de personnel ont lieu en début de chaque mois. Il y a exactement 3 ouvriers en février et en septembre. Vos contraintes sont les suivantes :

- (Coût des transferts) affecter un nouveau travailleur au chantier coûte 5 (recrutement, formation), en retirer un coûte 8 (indemnité de fin de mission).
- (Règles de transfert) vous ne pouvez affecter plus de 3 nouveaux ouvriers par mois, ni retirer plus du tiers de l'équipe.
- (Flexibilité) tout travailleur présent superflu vous coûte 10; tout ouvrier manquant vous coûte 20, le travail d'ouvriers manquants pouvant être rattrapé pendant la même période par d'autres présents dans la limite de 25% de leur temps de travail.

Formuler ce problème en terme de plus court chemin dans un graphe et le résoudre avec l'algorithme le plus approprié.

**Exercice 3.136.** — (Le problème du sac à dos) Vous souhaitez transporter des articles  $(a_i)_{i \leq k}$  dont vous connaissez le volume  $c_i$  et la valeur  $v_i$ . Votre sac à dos est de capacité  $\bar{C}$ , trop petite pour emporter tous les objets. Vous souhaitez maximiser la valeur totale de ceux que vous choisissez d'emporter.

- On donne le graphe orienté  $G = (V, E)$  avec  $V = \{(i, x); 0 \leq i \leq k+1, 0 \leq x \leq C\}$ . On donne comme arc les couples (s'ils sont possibles)
  - $(i-1, x) \rightarrow (i, x)$  de poids 0,
  - $(i-1, x) \rightarrow (i, x+c_i)$  de poids  $-v_i$ ,
  - $(k, x) \rightarrow (k+1, C)$

Que représente un plus court chemin dans  $G$  entre deux sommets bien choisis ?

- On donne un sac à dos de capacité 8 et les objets suivants :

| objet | n° | volume | valeur |
|-------|----|--------|--------|
|       | 1  | 5      | 4      |
|       | 2  | 3      | 7      |
|       | 3  | 2      | 3      |
|       | 4  | 2      | 5      |
|       | 5  | 1      | 4      |

**Exercice 3.137.** — Le graphe suivant modélise la formation d'un brin d'ARN messager à partir de l'ARN transcrite depuis un gène, il est basé sur ENSG00000081052 de la base de donnée <http://statgen.ncsu.edu/asg/>. Dans notre modèle, les nœuds  $1 \leq i \leq 42$  du graphe représentent des portions d'ADN ; le brin d'ARN messager est obtenu comme le raccord entre différents segments (nœuds du graphe). Une arête  $(u, v)$  du graphe est pondérée par la probabilité  $p$  de continuer le segment  $u$  par le segment  $v$ . Les nœuds 0 et 43 indiquent le début et la fin d'un brin mais n'ont pas de signification biologique. Nous représentons le graphe dans la table 1 sous forme de liste de successeurs  $(v : p)$  où  $v$  est le successeur et  $p$  sa probabilité. Quel brin d'ARN messager est-il le plus probable ?

**Exercice 3.138.** — Dans un monde post-apocalyptique, les monnaies ont disparu et les produits, que nous appelons  $(p_i)_{i \leq n}$ , s'échangent à nouveau sous forme de troc. Après un tour rapide au marché, vous disposez de l'information suivante pour certains  $i, j \leq n$  :  $n_i$  produits  $p_i$  s'échangent contre  $n_j$  produits  $p_j$  auprès du marchand  $m_{i,j}$ , où les  $n_i$  sont des nombres entiers.

- Comment détecter si le système de prix est incohérent. Par exemple,  $\{1 \text{ pomme} = 2 \text{ navets}, 1 \text{ navet} = 3 \text{ carottes et } 4 \text{ carottes} = 5 \text{ pommes}\}$  est incohérent.
- En supposant que le système de prix est cohérent, comment déterminer la suite des marchands  $m_{i,j}$  la plus rapide pour obtenir un produit  $p_t$  à partir d'un produit  $p_s$  par échange ?

**Exercice 3.139.** — On considère la liste des tâches suivantes à effectuer afin de construire une maison.

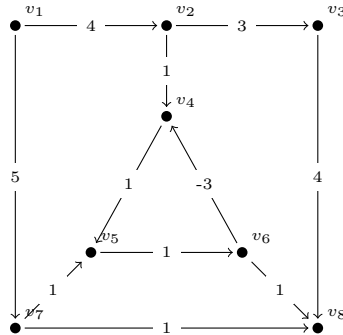
TABLE 1. Graphe de l'exercice 3.137

|    |            |                                             |                      |    |                                  |
|----|------------|---------------------------------------------|----------------------|----|----------------------------------|
| 0  | (1 : .5)   | (3 : .25)                                   | (24 : .25)           | 22 | (23 : 1)                         |
| 1  | (2 : .5)   | (4 : .5)                                    |                      | 23 | (25 : .25) (26 : .5) (32 : .25)  |
| 2  | (5 : 1)    |                                             |                      | 24 | (25 : 1)                         |
| 3  | (4 : 1)    |                                             |                      | 25 | (26 : .5) (28 : .5)              |
| 5  | (6 : .25)  | (7 : .25)                                   | (8 : .25) (11 : .25) | 26 | (27 : .34) (31 : .33) (38 : .33) |
| 6  | (7 : 1)    |                                             |                      | 27 | (28 : 1)                         |
| 7  | (8 : 1)    |                                             |                      | 28 | (29 : 1)                         |
| 8  | (9 : .75)  | (11 : .25)                                  |                      | 29 | (30 : .66) (31 : .34)            |
| 9  | (10 : .5)  | (11 : .5)                                   |                      | 30 | (31 : 1)                         |
| 10 | (11 : 1)   |                                             |                      | 31 | (32 : 1)                         |
| 11 | (12 : .66) | (13 : .17) (16 : .17)                       |                      | 32 | (33 : .34) (34 : .33) (35 : .33) |
| 12 | (13 : .5)  | (14 : .25) (15 : .25)                       |                      | 33 | (35 : 1)                         |
| 13 | (14 : 1)   |                                             |                      | 34 | (35 : 1)                         |
| 14 | (15 : 1)   |                                             |                      | 35 | (36 : .34) (37 : .33) (39 : .33) |
| 15 | (16 : 1)   |                                             |                      | 36 | (37 : 1)                         |
| 16 | (17 : .43) | (18 : .43) (20 : .14)                       |                      | 37 | (38 : 1)                         |
| 17 | (18 : 1)   |                                             |                      | 38 | (39 : 1)                         |
| 18 | (19 : .32) | (20 : .17) (21 : .17) (22 : .17) (39 : .17) |                      | 39 | (40 : 1)                         |
| 19 | (20 : 1)   |                                             |                      | 40 | (41 : .28) (42 : .72)            |
| 20 | (21 : 1)   |                                             |                      | 41 | (42 : 1)                         |
| 21 | (22 : .6)  | (23 : .2) (29 : .2)                         |                      | 42 | (43 : 1)                         |
|    |            |                                             |                      | 43 |                                  |

| N° | Tâche                  | Temps | À faire avant n° |
|----|------------------------|-------|------------------|
| 1  | Préparation du terrain | 2     | 2                |
| 2  | Fondations             | 4     | 3                |
| 3  | Monter les murs        | 10    | 4,6,7            |
| 4  | Acheminement eau       | 4     | 5,9              |
| 5  | Plomberie              | 5     | 10               |
| 6  | Electricité            | 7     | 10               |
| 7  | Toit                   | 6     | 8                |
| 8  | Finitions murs ext.    | 7     | 9                |
| 9  | Peinture ext.          | 9     | 14               |
| 10 | Finition cloisons      | 8     | 11,12            |
| 11 | Sols                   | 4     | 13               |
| 12 | Peinture int.          | 5     | 13               |
| 13 | Décoration intérieure  | 6     |                  |
| 14 | Aménagement extérieur  | 2     |                  |

On cherche à connaître le temps minimal de déroulement du chantier. Organiser le chantier au sein d'un graphe orienté où les sommets représentent les tâches et de sorte qu'un certain plus court chemin réponde au problème. Quel est le temps minimal du chantier donné en exemple ?

**Exercice 3.140.** — Appliquer l'algorithme de Bellman-Ford au graphe suivant au départ du sommet  $v_1$ .

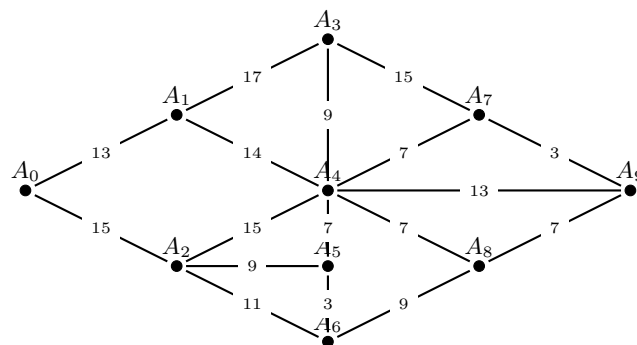


**Exercice 3.141.** — Calculer les longueurs des chemins les plus courts au départ de  $A$  vers l'ensemble des sommets à partir de la table des distances :

| Vers | A        | B        | C        | D        | E        | F        | G        |
|------|----------|----------|----------|----------|----------|----------|----------|
| De A | 0        | 1        | $\infty$ | $\infty$ | $\infty$ | 2        | 12       |
| B    | $\infty$ | 0        | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| C    | $\infty$ | -15      | 0        | 4        | 8        | $\infty$ | $\infty$ |
| D    | $\infty$ | $\infty$ | 4        | 0        | $\infty$ | $\infty$ | -2       |
| E    | $\infty$ | $\infty$ | $\infty$ | 4        | 0        | $\infty$ | $\infty$ |
| F    | $\infty$ | $\infty$ | $\infty$ | 9        | 3        | 0        | 12       |
| G    | $\infty$ | -12      | 2        | 3        | -1       | -4       | 0        |

**Exercice 3.142.** — La carte suivante représente les distances en kilomètres entre différentes villes.

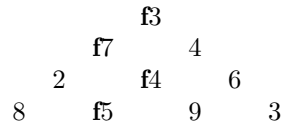
1. Calculer avec l'algorithme de DIJKSTRA le chemin le plus court entre  $A_0$  et  $A_9$ . (Expliquer chaque étape).



2. Un chauffeur roule à 60km/h en moyenne. Suite à un accident, on annonce un retard de 10 min au passage à travers de la ville  $A_4$ . Décrire un algorithme de recherche du plus court chemin tenant compte de cette information. Quelle est désormais la route la plus courte ?



**Exercice 3.143.** — Dans le dessin ci-dessous, en partant du sommet supérieur et en se déplaçant vers le bas par des nombres adjacents,



la plus grande somme du haut vers le bas possible est  $3 + 7 + 4 + 9 = 23$ . Comment résoudre ce problème en général sans énumérer toutes les possibilités.

[Source : <http://projecteuler.net> : Problème 67]

**Exercice 3.144.** — On considère la variante suivante de l'algorithme de Dijkstra dans laquelle les sommets ne sont plus marqués et peuvent ré-entrer dans la file.

---

**Algorithme 3.12 :** Variante de l'algorithme de Dijkstra

---

**Entrées :** Graphe orienté pondéré  $G = (V, E, \rho)$ , sommet  $s$

**Résultat :** Distance du sommet  $s$  à tout sommet accessible,  
arborescence des plus courts chemins

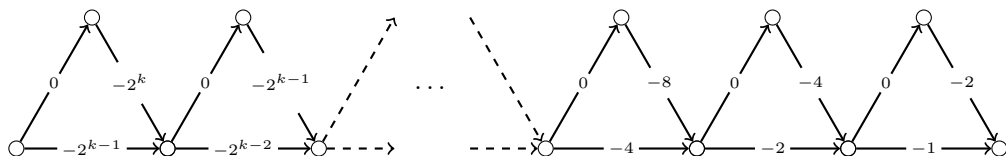
```

1  $\pi(s) \leftarrow 0$ 
2  $F \leftarrow \{s\}$  : file à priorité, où la priorité du sommet  $v$  est  $\pi(v)$ 
3 tant que  $F \neq \emptyset$  faire
4    $v \leftarrow$  Extraire de  $F$  le sommet de plus petite priorité  $\pi(v)$ 
5   pour  $w$  successeur de  $v$  faire
6     si  $\pi(v) + \rho(vw) < \pi(w)$  alors
7       si  $w \in F$  alors
8         Diminuer la priorité  $\pi(w) \leftarrow \pi(v) + \rho(vw)$  dans
           $F$ 
9       sinon
10        Insérer  $w$  à  $F$  avec la priorité
           $\pi(w) \leftarrow \pi(v) + \rho(vw)$ 
11       $\text{père}(w) \leftarrow v$ 
12 retourner  $\pi$  et père

```

---

1. L'algorithme ci-dessus est-il correct lorsque le graphe ne possède pas de circuit de poids négatif?
2. Estimer la complexité en temps lorsque cet algorithme est exécuté sur le graphe suivant



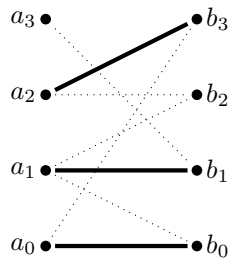
**Exercice 3.145.** — Soit  $G = (V, E, \rho)$  un graphe orienté pondéré. On note

$$V = \{v_1, v_2, \dots, v_n\}$$

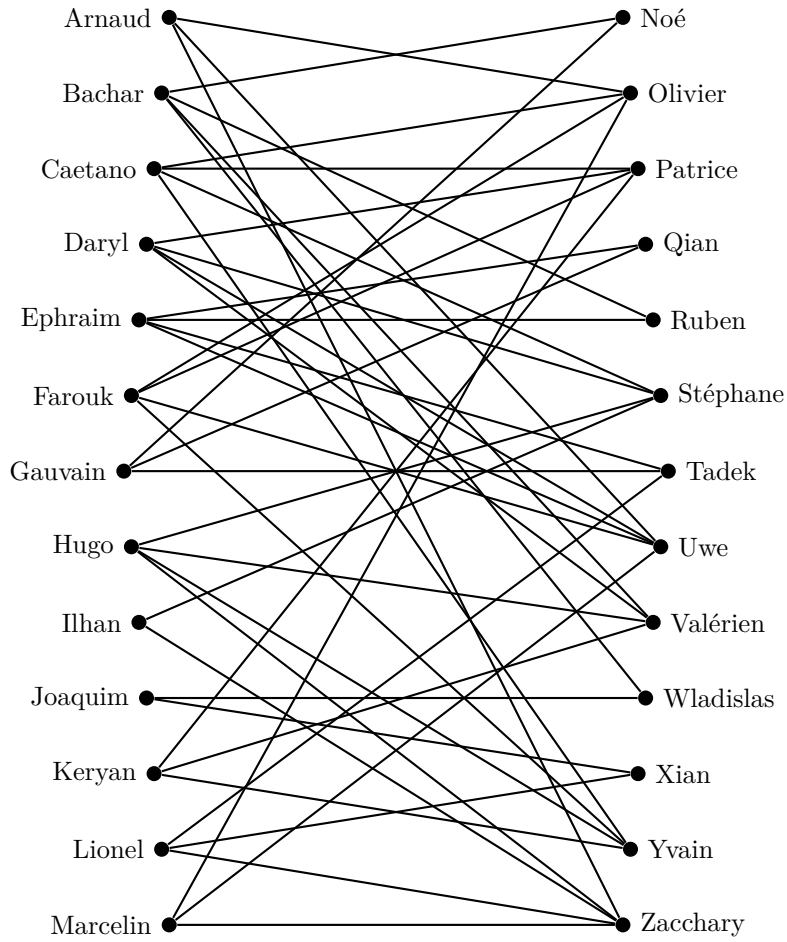
les sommets de  $G$ . Proposer une variante de l'algorithme de Floyd-Warshall (voir code 3.13) dans laquelle  $\pi(v_i, v_j, r)$  représente le poids d'un plus court chemin allant du sommet  $v_i$  vers le sommet  $v_j$  qui n'utilise que les sommets  $\{v_1, v_2, \dots, v_r\}$ . Dans cette variante,  $\pi(v_i, v_j, r)$  vaut  $+\infty$  si l'un des indices  $i$  ou  $j$  est strictement supérieur à  $r$ .

**Exercice 3.146.** — Combien y a-t-il d'arcs au maximum dans un graphe orienté acyclique à  $n$  sommets ?

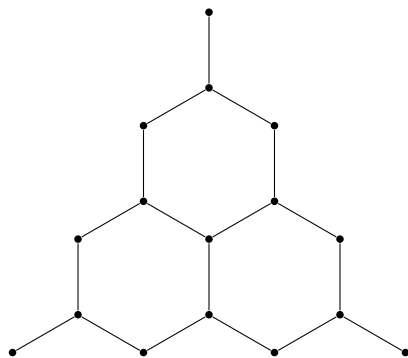
**Exercice 3.147.** — On propose le couplage suivant. Est-il maximal (pour l'inclusion) ? Est-il maximum (pour le cardinal) ?



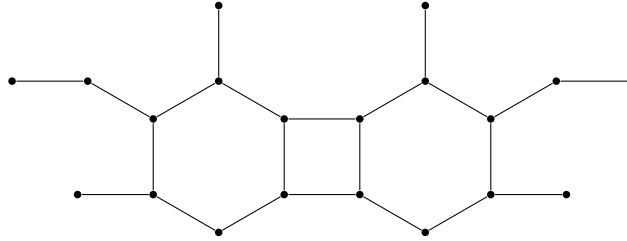
**Exercice 3.148.** — Déterminer un couplage de cardinal maximum et justifier qu'il est bien de cardinal maximum.



**Exercice 3.149.** — Déterminer un couplage de cardinal maximum dans le graphe ci-dessus.



**Exercice 3.150.** — Déterminer un couplage de cardinal maximum dans le graphe ci-dessus.



**Exercice 3.151.** — 1. Soit  $G = (V = V_1 \sqcup V_2, E)$  un graphe biparti satisfaisant  $|V_1| = |V_2| = n$ . Nous allons démontrer que  $G$  possède un couplage parfait (couplage à  $n$  couples) si et seulement si pour tout  $S \subseteq V_1$ , l'ensemble des voisins  $N(S)$  des sommets de  $S$  vérifie

$$(3) \quad |S| \leq |N(S)|$$

Ce résultat s'appelle le lemme des mariages.

- (a) Montrer que l'équation 3 est bien une condition nécessaire.
- (b) On ajoute deux sommets  $s$  et  $t$  à  $G$ . On considère le réseau basé sur les sommets  $V \cup \{s, t\}$ , d'arcs  $(s, v_1)$  de capacité 1 pour tout  $v_1 \in V_1$ ,  $(v_1, v_2)$  de capacité  $+\infty$  pour tout  $(v_1, v_2) \in E \cap (V_1 \times V_2)$  et  $(v_2, t)$  de capacité 1 pour tout  $v_2 \in V_2$ .

Soit  $S$  une coupe de capacité minimale. Montrer que  $c(S, \bar{S}) \leq n$ . On pose  $X = S \cap V_1$ , montrer que  $N(X) \subseteq S \cap V_2$ . En déduire que  $c(S, \bar{S}) \geq n - |X| + |N(X)|$ . Conclure.

2. On divise un jeu de cartes en 13 tas de 4 cartes. Montrer qu'il est possible de reconstituer la suite Roi, Dame, Valet, ..., Deux, As complète en sélectionnant une carte par pile de quatre.

**Exercice 3.152.** — On considère le problème de loger quinze étudiants au sein du palaisél (le palais des étudiants) de l'école de magie Padlourd. Six d'entre eux étudient la science des sortilèges, les neuf autres la science des potions. Le palaisél possède seulement 6 chambres, dont deux doubles ( $C_1$  et  $C_2$ ) et quatre simples ( $C_3$  à  $C_6$ ). Par convenance personnelle, chaque étudiant n'accepte que certaines chambres selon la table suivante. Comment attribuer un nombre maximal de chambres acceptables sachant qu'une chambre double ne doit pas contenir deux étudiants de la même

discipline.

| Étudiant | Chambre acceptable        |
|----------|---------------------------|
| $P_1$    | $C_1, C_3$                |
| $P_2$    | $C_1, C_2, C_3, C_4, C_5$ |
| $P_3$    | $C_4, C_5$                |
| $P_4$    | $C_2$                     |
| $P_5$    | $C_3, C_6$                |
| $P_6$    | $C_1, C_2$                |
| $P_7$    | $C_3, C_4, C_5, C_6$      |
| $P_8$    | $C_5, C_6$                |
| $P_9$    | $C_3, C_5$                |
| $S_1$    | $C_2, C_3, C_4$           |
| $S_2$    | $C_1, C_6$                |
| $S_3$    | $C_2, C_5$                |
| $S_4$    | $C_3, C_4, C_6$           |
| $S_5$    | $C_1$                     |
| $S_6$    | $C_2, C_5$                |

**Exercice 3.153.** — Soit  $\mathcal{A} = \{A_1, \dots, A_n\}$  une famille d'ensembles. Un *système de représentants distincts* est une suite  $x_1, \dots, x_n$  d'éléments tous distincts tels que  $x_i \in A_i$  pour tout  $i$ . Indiquer si les familles suivantes admettent un SDR.

1.  $\{3, 4, 5\}, \{2, 5, 6\}, \{1, 2, 5\}, \{1, 2, 3\}, \{1, 3, 6\}$ .
2.  $\{1, 2, 3, 4, 5, 6\}, \{1, 3, 4\}, \{1, 4, 7\}, \{2, 3, 5, 6\}, \{3, 4, 7\}, \{1, 3, 4, 7\}, \{1, 3, 7\}$ .

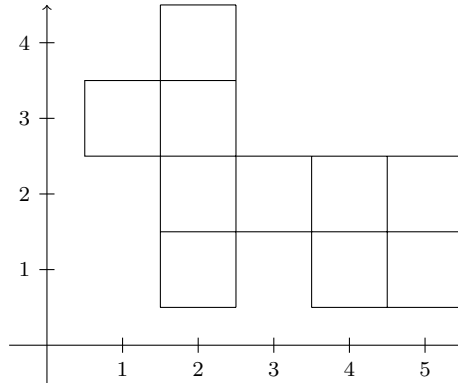
**Exercice 3.154.** — On dit qu'un graphe est  $k$  régulier si tout sommet est de degré  $k$ . Montrer que tout graphe biparti  $k$  régulier sur  $2n$  sommets possède un couplage parfait (i.e. un couplage à  $n$  arêtes).

**Exercice 3.155.** — Deux joueurs  $A$  et  $B$  s'opposent dans un jeu. Ils jouent à tour de rôle en citant un nom inédit d'un acteur pour  $A$  et d'une actrice pour  $B$ . Le joueur  $A$  commence par citer le nom d'un acteur. À chaque tour suivant, le joueur répond par un nom d'acteur/actrice ayant joué dans le même film que le précédent acteur cité. Quand un joueur ne peut pas donner de nom, il perd la partie.

On dit qu'un joueur possède une stratégie gagnante s'il dispose d'une manière de jouer qui l'assure de gagner quelque soit le jeu de son adversaire.

On suppose qu'il y a  $n$  acteurs et  $n$  actrices et que sont connus l'ensemble des acteurs et les films dans lesquels ils ont joué. Donner un algorithme qui décide en temps polynomial lequel des joueurs possède une stratégie gagnante.

**Exercice 3.156.** — On s'intéresse au problème suivant : la figure ci-dessous peut-elle être recouverte par des pièces de dominos (i.e. des rectangles de taille  $2 \times 1$ ) ?



1. Soit  $(i, j)$  et  $(i', j')$  les coordonnées des centres des carrés formant un domino. Que peut-on dire de la parité de  $i + j$  par rapport à la parité de  $i' + j'$ .
2. Modéliser le problème de recouvrement par un problème de graphe vu en cours.
3. Expliquer comment déterminer s'il admet une solution.
4. Appliquer votre modélisation à l'exemple et conclure.

**Exercice 3.157.** — Soit  $G = (V, E)$  un graphe biparti où  $V_1$  et  $V_2$  désignent les deux parts de l'ensemble des sommets  $V$ . Soit  $R_1$  un sous-ensemble de sommets de  $V_1$  et  $R_2$  un sous-ensemble de sommets de  $V_2$ . On suppose qu'il existe un couplage  $C_1$  qui couvre tous les sommets de  $R_1$  (i.e. tout sommet de  $R_1$  est l'extrémité d'une arête de  $C_1$ ) et qu'il existe un couplage  $C_2$  qui couvre tous les sommets de  $R_2$ . Montrer qu'il existe un couplage  $C$  qui couvre  $R_1 \cup R_2$ .

**Exercice 3.158.** — Un paquet de  $m \cdot n$  cartes de  $m$  valeurs et  $n$  couleurs est formé d'exactly une carte de chaque valeur et de chaque couleur. On a disposé l'ensemble des cartes en un rectangle de dimension  $m$  lignes et  $n$  colonnes. Montrer qu'il existe un ensemble de  $m$  cartes, à raison d'une par ligne, tel qu'il possède toutes les valeurs distinctes.

**Exercice 3.159.** — Soit  $S$  un ensemble fini et  $k$  un entier tel que  $2k + 1 \leq |S|$ . Pour tout entier  $t$ , on note  $\binom{S}{t}$  l'ensemble des sous-ensembles de  $S$  qui possèdent exactement  $t$  éléments. Montrer qu'il existe une application injective  $f : \binom{S}{k} \rightarrow \binom{S}{k+1}$ .

Autrement dit, on peut compléter des ensembles à  $k$  éléments avec un élément supplémentaire pour chacun d'entre eux, de sorte que les ensembles à  $k + 1$  éléments résultant soient tous distincts.

## CHAPITRE 4

### MANIPULATIONS DE CHAÎNES DE CARACTÈRE ET LANGAGES FORMELS

Avant de lire ce chapitre, il pourra être utile de relire les définitions d'*alphabet*, de *mot* et de *langage* rencontrée au chapitre 2.1.1 que nous complétons par les définitions suivantes.

**Définition 4.1.** — Soit  $\Sigma$  un alphabet. Lorsque  $u, v, w \in \Sigma^*$  vérifient  $w = uv$ , autrement dit lorsque le mot  $w$  est la concaténation des deux mots  $u$  et  $v$ , on dit également :

- que  $u$  est un *préfixe* de  $w$ , ou
- que  $v$  est un *suffixe* de  $w$ .

De façon équivalente, si  $w = x_1 \cdots x_n$  (où  $x_1, \dots, x_n \in \Sigma$ ) est un mot de longueur  $n$ , et si  $0 \leq k \leq n$  est un entier quelconque compris entre 0 et  $n$ , on dit que  $u = x_1 \cdots x_k$  (c'est-à-dire, le mot formé des  $k$  premières lettres de  $w$ , dans le même ordre) est le *préfixe de longueur  $k$*  de  $w$ , et que  $v = x_{k+1} \cdots x_n$  (mot formé des  $n - k$  dernières lettres de  $w$ , dans le même ordre) est le *suffixe de longueur  $n - k$*  de  $w$ . Il est clair qu'il s'agit bien là de l'unique façon d'écrire  $w = uv$  avec  $|u| = k$  et  $|v| = n - k$ , ce qui fait le lien avec la définition donnée au paragraphe précédent ; parfois on dit que  $v$  est le suffixe *correspondant* à  $u$  ou que  $u$  est le préfixe correspondant à  $v$  (dans le mot  $w$ ).

**Remarque 4.2.** — Le mot vide est préfixe et suffixe de n'importe quel mot. Le mot  $w$  lui-même est aussi un préfixe et un suffixe de lui-même. Entre les deux, pour n'importe quelle longueur  $k$  donnée, il existe un unique préfixe et un unique suffixe de longueur  $k$ . (Il peut tout à fait se produire que le préfixe et le suffixe de longueur  $k$  soient égaux pour d'autres  $k$  que 0 et  $|w|$ , comme le montre l'exemple qui suit.)

**Exemple 4.3.** — Le mot **abbcab** sur l'alphabet  $\Sigma = \{a, b, c, d\}$  a les sept préfixes suivants, rangés par ordre croissant de longueur :  $\varepsilon$  (le mot vide), **a**, **ab**, **abb**, **abbc**, **abbca** et **abbcab** lui-même ; il a les sept suffixes suivants, rangés par ordre croissant de longueur :  $\varepsilon$  (le mot vide), **b**, **ab**, **cab**, **bcab**, **bbcab** et **abbcab** lui-même. Le suffixe correspondant au préfixe **abb** est **cab** puisque **abbcab** = (**abb**)(**cab**).

Comme généralisation à la fois de la notion de préfixe et de celle de suffixe, on a la notion de facteur.

**Définition 4.4.** — Si  $u_0, v, u_1 \in \Sigma^*$  sont trois mots quelconques sur un même alphabet  $\Sigma$ , et si  $w = u_0vu_1$  est leur concaténation, on dit que  $v$  est un *facteur* de  $w$ .

Alors qu'un préfixe ou suffixe du mot  $w$  est déterminé simplement par sa longueur, un facteur est déterminé par sa longueur et l'emplacement à partir duquel il commence.

**Exemple 4.5.** — Les facteurs du mot **abbcab** sont :  $\varepsilon$  (le mot vide), **a**, **b**, **c**, **ab**, **bb**, **bc**, **ca**, **abb**, **bbc**, **bca**, **cab**, **abbc**, **bbca**, **bcab**, **abbca**, **bbcab** et **abbcab** lui-même.

Dans un contexte informatique, ce que nous appelons ici « facteur » est souvent appelé « sous-chaîne de caractères ». Il ne faut cependant pas confondre ce concept avec celui de sous-mot défini ci-dessous.

**Définition 4.6.** — Si  $w = x_1 \cdots x_n$  est un mot de longueur  $n$ , on appelle *sous-mot* de  $w$  un mot de la forme  $x_{i_1} \cdots x_{i_k}$  où  $1 \leq i_1 < \cdots < i_k \leq n$ . En plus clair, cela signifie que  $v$  est obtenu en ne gardant que certaines lettres du mot  $w$ , dans le même ordre, mais en effaçant d'autres ; à la différence du concept de facteur, celui de sous-mot n'exige pas que les lettres gardées soient consécutives.

**Exemple 4.7.** — Le mot **acb** est un sous-mot du mot **abbcab** (obtenu en gardant les lettres soulignées ici : a**b**bc**a**b) ; pour se rattacher à la définition ci-dessus, on pourra prendre  $(i_1, i_2, i_3) = (1, 4, 6)$ .

**Remarque 4.8.** — Les remarques de dénombrement suivantes peuvent aider à mieux comprendre les notations de préfixe, suffixe, facteur et sous-mot : si  $w$  est un mot de longueur  $n$ , alors il a

- exactement  $n + 1$  préfixes (car un préfixe est déterminé par sa longueur  $k$  entre 0 et  $n$ ),
- exactement  $n + 1$  suffixes (raison analogue),
- au plus  $\sum_{k=1}^n (n+1-k) + 1 = \frac{1}{2}(n^2 + n + 2)$  facteurs (car un facteur est déterminé par sa longueur  $k$  et son point de départ qui peut être choisi parmi  $n + 1 - k$  possibilités, le +1 final étant mis pour le facteur vide),
- au plus  $2^n$  sous-mots (car un sous-mot est déterminé en choisissant, pour chacune des  $n$  lettres, si on l'efface ou la conserve).

Le nombre exact peut être plus petit en cas de coïncidences entre certains choix (par exemple, **aaa** n'a que 4 facteurs,  $\varepsilon$ , **a**, **aa**, **aaa** alors que **abc** en a bien  $\frac{1}{2}(3^2 + 3 + 2) = 7$ ) ; mais les bornes ci-dessus sont effectivement atteintes pour certains mots.



#### 4.1. Recherche de motifs

Étant donnés deux mots  $s$  et  $p$ , *rechercher le motif  $p$  dans la chaîne de caractères  $s$*  consiste à déterminer si  $p$  est un facteur de  $s$  et, éventuellement, calculer la ou les positions où se trouve le motif  $p$ .

Selon les situations, on peut vouloir ne rechercher qu'un seul motif, plusieurs motifs ou même une famille de motifs précisée par des règles de construction.

**Remarque 4.9.** — En python, vérifier qu'un mot  $p$  est un facteur d'un autre mot  $s$  se fait par l'instruction `p in s`, qui renvoie un booléen. Pour connaître le premier indice où  $p$  apparaît, on peut utiliser l'instruction `s.find(p)` ou `s.index(p)`. Dans le cas où  $p$  n'est pas un facteur de  $s$ , `s.find(p)` renvoie l'indice  $-1$  tandis que `s.index(p)` lève une exception `ValueError`.

##### 4.1.1. Recherche d'un motif unique. —

**4.1.1.1. Par force brute.** — Soient  $p$  et  $s$  deux mots sur un alphabet  $\Sigma$ . Rechercher le motif  $p$  dans la chaîne de caractères  $s$  peut se faire naïvement par force brute en faisant glisser le motif  $p$  le long de la chaîne  $s$  et en comparant lettre par lettre les symboles composant  $p$  avec les symboles composant  $s$ .

LISTING 4.1. Recherche d'une sous-chaîne par force brute

```

1155 def bruteforce(s, p):
1156     n = len(s)
1157     m = len(p)
1158     for i in range(n-m+1):
1159         j = 0
1160         while s[i+j] == p[j] and j < m:
1161             j += 1
1162             if j == m:
1163                 return(i)
1164     return("No match")

```

**Proposition 4.10.** — La complexité de la recherche d'une sous-chaîne de caractères par force brute a une complexité en temps  $O(mn)$  où  $n$  est la longueur du mot  $s$  et où  $m$  est la longueur du mot  $p$ .

**Remarque 4.11.** — La recherche par force brute est assez inefficace : à chaque fois que l'on décale la position présumée du motif  $p$  par rapport à la chaîne  $s$ , on oublie l'ensemble des comparaisons réussies entre des lettres de  $s$  et des lettres de  $p$ . Cette perte d'information explique la complexité élevée.

**4.1.1.2. Par un parcours linéaire de la chaîne  $s$ .** — Nous nous plaçons dans l'hypothèse où la chaîne  $s$  est de longueur grande par rapport au motif  $p$ . Il est alors légitime de se permettre un prétraitement de  $p$  avant de rechercher réellement  $p$  dans  $s$ . Idéalement, nous souhaiterions étudier  $s$  en une seule passe.

**Définition 4.12.** — Un *bord* d'un mot  $w \in \Sigma^+$  tout facteur propre de  $w$  qui est à la fois préfixe et suffixe de  $w$ .

**Remarque 4.13.** — Quand un mot  $w$  n'est pas le mot vide, le mot vide  $\varepsilon$  est forcément un bord de  $w$ . Par ailleurs, deux bords de même longueur sont forcément égaux, puisqu'ils sont préfixes de  $w$ . Nous sommes donc certains que le plus long bord de  $w$  est bien défini.

**Exemple 4.14.** — Le mot  $w = \mathbf{aba}$  a pour ensemble de bords  $\{\mathbf{a}, \varepsilon\}$ ; le mot  $w = \mathbf{ababab}$  a pour ensemble de bords  $\{\mathbf{abab}, \mathbf{ab}, \varepsilon\}$ .

**Définition 4.15.** — On note  $\text{bord}(w)$  le *plus long bord* de  $w$ .

**Lemme 4.16.** — L'ensemble des bords d'un mot  $w \in \Sigma^+$  est de la forme

$$\{\text{bord}(w), \text{bord}(\text{bord}(w)), \dots, \text{bord}^k(w)\}$$

où  $k$  est le plus petit entier tel que  $\text{bord}^k(w)$  est le mot vide.

*Démonstration.* — Il suffit d'observer qu'un bord de longueur quelconque est à la fois un préfixe et un suffixe du bord le plus long, ce qui en fait un bord de ce bord le plus long.  $\square$

Dans la mesure où un bord de  $w$  est notamment un préfixe de  $w$ , il suffit d'en connaître la longueur pour le recalculer. Pour tout mot  $p \in \Sigma^*$ , nous appelons  $s_p(i)$  la longueur du plus long bord du préfixe de longueur de  $i$  de  $p$ . La quantité  $s_p$  est définie pour  $i \in \llbracket 1, |p| \rrbracket$  et vérifie

$$0 \leq s_p(i) < i.$$

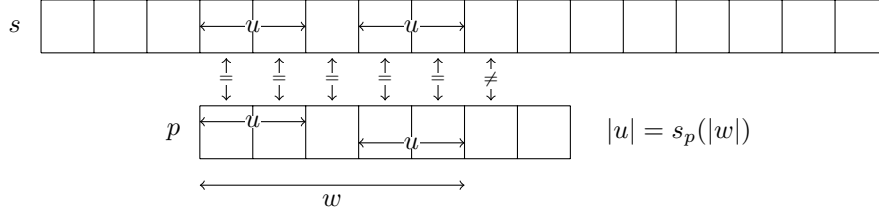
Nous étendons  $s_p$  en posant  $s_p(0) = -1$ .

Pour calculer  $s_p$ , nous pouvons faire l'observation suivante. Soit  $w = x_0x_1 \cdots x_i$  le préfixe de longueur  $i+1$  du mot  $p$ . Un bord non vide de  $w$  est forcément de la forme  $ux_i$  où  $u$  est un préfixe de  $w$  suivi du symbole  $x_i$ . Mais alors  $u$  est un bord du mot  $w' = x_0x_1 \cdots x_{i-1}$ . Pour calculer  $s_p(i+1)$ , il suffit donc d'énumérer les bords de  $w'$  par longueur décroissante, ce qui peut être fait grâce au lemme 4.16, et trouver le premier qui est suivi du symbole  $x_i$ .

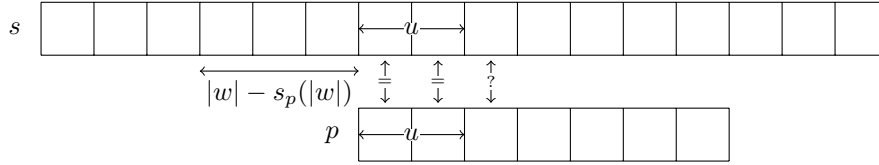
**Proposition 4.17.** — On a donc

$$s_p(i+1) = \begin{cases} 1 + \underbrace{s_p \circ s_p \circ \cdots \circ s_p(i)}_{k \text{ fois}} & \text{où } k \text{ plus petit entier t.q. } x_{s_p^k(i)} = x_i \\ 0 & \text{si } k \text{ n'existe pas} \end{cases}$$

Nous pouvons améliorer une première fois l'algorithme de recherche de sous-chaîne par force brute. Supposons avoir détecté que le préfixe  $w$  de longueur  $i$  du motif  $p$  coïncide avec  $i$  lettres consécutives de la chaîne  $s$ , mais que la lettre suivant ne coïncide pas



Alors, on peut décaler  $p$  de  $|w| - s_p(|w|)$  positions pour continuer la recherche.



On pourra se convaincre que l'observation que l'on vient de faire est valide même lorsque  $w$  est le mot vide, ce qui justifie que l'on pose  $s_p(0) = -1$ .

**Remarque 4.18.** — Nous observons encore une source d'inefficacité : il est possible que les deux occurrences préfixe et suffixe de  $u$  soient suivies de la même lettre. On est alors amené à comparer un symbole de  $s$  plusieurs fois avec la même valeur (à des positions différentes) dans  $p$ .

**Définition 4.19.** — Soient  $w$  un préfixe propre d'un mot  $p$  et  $b$  un bord de  $w$ . On note  $x$  la lettre suivant  $w$  dans  $p$  et  $y$  la lettre suivant  $b$  dans  $w$ , *i.e.*

$$p = wx \cdots \quad \text{et} \quad w = by \cdots$$

La bord  $b$  est *compatible* si  $x \neq y$ .

**Exemple 4.20.** — Le mot  $w = \text{aba}$ , préfixe de  $p = \text{abababc}$ , n'a pas de bord compatible le mot  $w = \text{ababab}$ , préfixe de  $p = \text{abababc}$ , a pour ensemble de bords compatible  $\{\text{abab}, \text{ab}, \varepsilon\}$ .

Soit  $w = x_0x_1 \cdots x_{i-1}$  et  $x_i$  la lettre suivante dans  $p$ . On note  $r_p(i)$  la *longueur du plus long bord compatible* de  $w$ . On pose  $r_p(i) = -1$  s'il n'y a pas de bord compatible (en particulier,  $r_p(0) = -1$ ).

$$r_p(i) = \begin{cases} s_p(i) & \text{si } x_{s_p(i)} \neq x_i \\ r_p(s_p(i)) & \text{sinon} \end{cases}$$

FINME : terminer l'analyse

LISTING 4.2. Calcul de la longueur du plus long bord compatible

```

1165 def pluslongbordcomp(p):
1166     r = [0] * len(p)
1167     r[0] = i = -1
1168     for j in range(1, len(p)):
1169         while i >= 0 and p[i] != p[j-1]:
1170             i = r[i]
```

```

1171         i += 1
1172     if p[i] != p[j]:
1173         r[j] = i
1174     else:
1175         r[j] = r[i]
1176 return r

```

**Théorème 4.21.** — La complexité de l'algorithme de calcul des longueurs du plus long bord compatible  $r_p$  est  $O(m)$  où  $m$  est la longueur du motif  $p$ .

*Démonstration.* — La phase d'initialisation coûte  $O(m)$  opérations.

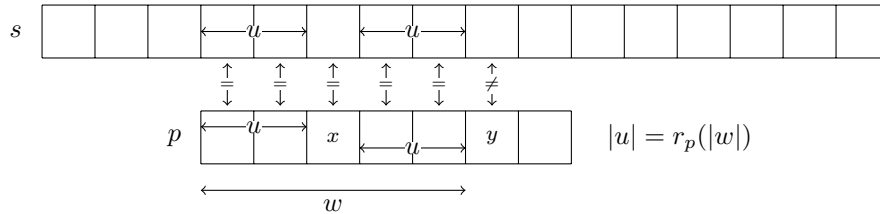
Toutes les opérations dans les deux boucles s'effectuent en temps constant pour chaque passage. Il suffit donc de compter le nombre de passage dans les boucles. Mais ici, l'analyse de la combinaison de la boucle **for** et **while** est délicate.

Nous observons que la valeur  $i$  est incrémentée une fois par passage dans la boucle **for**. Au total, la valeur  $i$  est incrémentée  $|p|$  fois. Inversement, à chaque fois que l'on passe dans la boucle **while**, à cause de l'instruction  $i = r[i]$ , la valeur de  $i$  décroît strictement. Or la variable  $i$  est invariablement positive tout au long de l'exécution de l'algorithme. On peut donc voir les  $|p|$  incréments de  $i$  comme un crédit positif dans lequel on vient puiser pour « payer » un passage dans la boucle **while**. Il y a donc au maximum  $|p|$  passages possibles dans la boucle **while**.

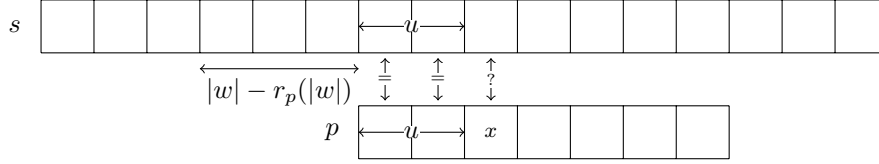
Nous en déduisons que la complexité de l'algorithme est  $O(m)$ .  $\square$

**Remarque 4.22.** — La preuve ci-dessus met en œuvre une technique classique de calcul de complexité : il s'agit de la *méthode comptable*. Certaines opérations nous fournissent un « crédit » que l'on peut consommer pour réaliser d'autres opérations.

Nous pouvons améliorer une seconde fois l'algorithme de recherche de sous-chaîne par force brute. Supposons, comme précédemment, avoir détecté que le préfixe  $w$  de longueur  $i$  du motif  $p$  coïncide avec  $i$  lettres consécutives de la chaîne  $s$ , mais que la lettre suivant ne coïncide pas



Nous identifions le plus long bord compatible  $u$  ; nous notons  $x$  et  $y$  les lettres respectives qui suivent  $u$  en tant que préfixe et suffixe de  $x$ . Les lettres  $x$  et  $y$  sont distinctes. Alors, on peut décaler  $p$  de  $|w| - r(|w|)$  positions pour continuer la recherche.



**Lemme 4.23.** — Dans les conditions énoncées ci-avant, pour tout décalage du motif  $p$  de strictement moins de  $|w| - r_p(|w|)$  cases, le motif  $p$  ne coïncide pas avec les lettres correspondantes de la chaîne  $s$

*Démonstration.* — Supposons que l'on puisse décaler  $p$  de  $\delta$  cases, avec  $\delta < |w| - r_p(|w|)$  et obtenir une concordance parfaite entre les lettres de  $p$  et celles de  $s$ . Alors, les  $|w| - \delta$  premières et dernières case de  $p$  seraient identiques, donc formeraient un bord. Notons  $x$  et  $y$  la lettre suivante (comme dans le schéma). On devrait avoir  $x \neq y$ , car  $x$  ne concorde pas avec  $s$  avant le décalage par  $\delta$  et  $y$  concorde après décalage. Mais alors, nous aurions repéré un bord compatible de longueur strictement supérieure à  $r_p(|w|)$  ce qui n'est pas possible.  $\square$

Il est temps de présenter l'algorithme de Knuth Morris Pratt qui incorpore toutes les améliorations que nous avons discutées.

LISTING 4.3. Algorithme de Knuth Morris Pratt

```

1177 def knuthMorrisPratt(s, p):
1178     r = pluslongbordcomp(p)
1179     i = j = 0
1180     while i < len(p) and j < len(s):
1181         if p[i] == s[j]:
1182             i += 1
1183             j += 1
1184         else:
1185             if r[i] == -1:
1186                 i = 0
1187                 j += 1
1188             else:
1189                 i = r[i]
1190     if i == len(p):
1191         return j-i
1192     else:
1193         return -1

```

**Théorème 4.24.** — La complexité de l'algorithme de Knuth Morris Pratt est  $O(m+n)$  où  $m$  est la longueur du motif  $p$  et  $n$  est la longueur de la chaîne  $s$ .

*Démonstration.* — La phase d'initialisation du tableau  $r$  coûte  $O(m)$  comme déjà vu plus haut.

Ici encore, nous utilisons la méthode comptable pour calculer la complexité du corps de l'algorithme. Nous observons les faits suivants. Lors du passage dans la boucle **while**, soit  $j$  est incrémenté, ce qui peut se produire au plus  $|s|$  fois, soit  $i$  est décrémenté. La variable  $i$  est parfois incrémentée en même temps que  $j$  mais doit rester tout le temps supérieure à  $-1$ . Comme  $j$  varie entre 0 et  $|s|$ , la variable  $i$  peut être vue comme un compte à dépenser qui n'est pas crédité de plus de  $|s|$  fois. Mais alors, cela implique que l'on ne peut pas passer par la branche  $i = r[i]$  plus de  $|s|$  fois non plus.

Ainsi la complexité de l'algorithme de Knuth Morris Pratt est  $O(m + n)$ .  $\square$

**Définition 4.25.** — Un *algorithme en ligne* est un algorithme qui reçoit ses données d'entrée sous forme d'un flux et qui doit prendre des décisions au fur et à mesure sans connaître l'argument d'entrée dans sa totalité.

**Exemple 4.26.** — Le motif  $p$  étant fixé, la recherche de  $p$  comme facteur du mot  $s$  par l'*algorithme de KMP* est un algorithme en ligne.

#### 4.1.2. Recherche d'une famille de mots. —

##### 4.1.2.1. La structure d'arbre préfixe ou trie. —

**Définition 4.27.** — Soit  $\Sigma = \{z_1, z_2, \dots, z_\sigma\}$  un alphabet de cardinal  $\sigma$ . On appelle *trie* (ou *arbre préfixe*) tout arbre étiqueté dont les nœuds internes sont d'arité  $\sigma$ .

**Remarque 4.28.** — Le point important de la définition est que les symboles de l'alphabet  $\Sigma$  ont été énumérés. En conséquence, tout nœud interne possède un sous-arbre directement associé à une lettre de l'alphabet.

La structure de donnée « trie » permet de réaliser le type abstrait « ensemble » quand les éléments de l'ensemble sont des mots de  $\Sigma^*$  ou dictionnaire quand les clés du dictionnaire sont des mots de  $\Sigma^*$ .

Pour insérer ou rechercher l'association de clé  $w = z_{i_1} z_{i_2} \dots z_{i_n}$  et la valeur  $v$ , on construit la suite de sommets

$$(s_0, s_1, \dots, s_n)$$

telle que  $s_0$  est la racine de trie et pour tout  $j$  compris entre 1 et  $n$ , le sommet  $s_j$  est la racine du  $i_j$ -ième sous-arbre de  $s_{j-1}$ . Dans le cas de l'insertion, on inscrit la valeur  $v$  comme étiquette du sommet  $s_n$ ; dans le cas de la recherche de la valeur associée à  $w$ , on renvoie la valeur de l'étiquette  $s_n$ .

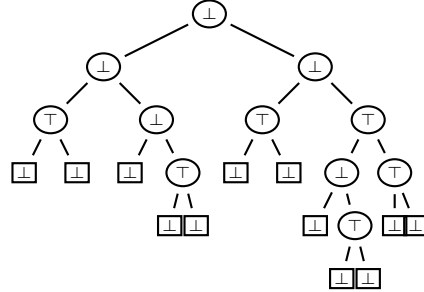
Pour faire fonctionner la structure de trie, il est nécessaire de disposer d'une valeur sentinelle, que nous notons ici  $\perp$ , qui indique qu'un sommet ne possède pas d'étiquette.

Dans le cas où nous utilisons un trie pour représenter un ensemble fini de mots, il est nécessaire de disposer d'une seconde valeur sentinelle, que nous notons ici  $\top$ , qui marque qu'un élément appartient à l'ensemble.

**Exemple 4.29.** — Sur l'alphabet binaire  $\Sigma = \{a, b\}$ , le sous-ensemble fini  $\mathcal{L} \subseteq \Sigma^* \rightarrow \mathbb{N}$  contenant les mots

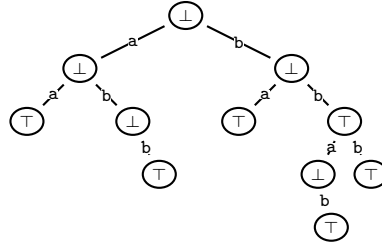
aa   abb   ba   bb   bbab   bbb

se représente par



où nous avons adopté la convention que le sous-arbre gauche correspond à la lettre **a** et le sous-arbre droit correspond à la lettre **b**.

Pour plus de clarté, nous pourrions choisir d'ajouter à nos dessins une information concernant la lettre utilisée sur chaque arête et nous nous abstenons de dessiner les arbres vides qui viennent fermer la structure :

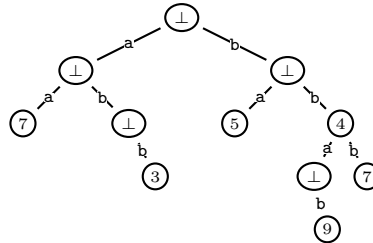


Dans cet exemple, nous pourrions chercher si l'élément **bba** appartient à l'ensemble représenté. La réponse est non, car en suivant le chemin indiqué par **bba**, nous obtenons un sommet d'étiquette non défini. Nous pourrions aussi chercher si l'élément **aabbb** appartient à l'ensemble représenté. La réponse est non, mais cette fois-ci parce que la construction du chemin n'aboutit pas

**Exemple 4.30.** — Sur l'alphabet binaire  $\Sigma = \{a, b\}$ , le dictionnaire de type  $\Sigma^* \rightarrow \mathbb{N}$  contenant les associations

|      |           |   |
|------|-----------|---|
| aa   | $\mapsto$ | 7 |
| abb  | $\mapsto$ | 3 |
| ba   | $\mapsto$ | 5 |
| bb   | $\mapsto$ | 4 |
| bbab | $\mapsto$ | 9 |
| bbb  | $\mapsto$ | 7 |

se représente par le trie dont le schéma suit :



En pratique, il existe plusieurs options pour représenter un trie en mémoire. On peut utiliser une structure disposant d'un tableau statique pour stocker les sous-arbres avec autant de cases que de symboles dans l'alphabet  $\Sigma$ . Quand la taille de  $\Sigma$  grandit, ce mode de représentation peut se révéler couteux : en effet, de nombreux champs restent vides. On peut aussi préférer de stocker le tableau des sous-arbres sous la forme d'un dictionnaire associant des symboles de  $\Sigma$  à des tries.

LISTING 4.4. Représentation des tries en Python

```

1194 class Trie():
1195     def __init__(self):
1196         self.children = dict()
1197         self.value = None
1198     def __setitem__(self, key, value):
1199         if key == "":
1200             self.value = value
1201         else:
1202             x = key[0]
1203             if x not in self.children:
1204                 self.children[x] = Trie()
1205             self.children[x].__setitem__(key[1:], value)
1206     def __getitem__(self, key):
1207         if key == "":
1208             return(self.value)
1209         else:
1210             x = key[0]
1211             if x not in self.children:
1212                 return None
1213             else:
1214                 return self.children[x].__getitem__(key[1:])

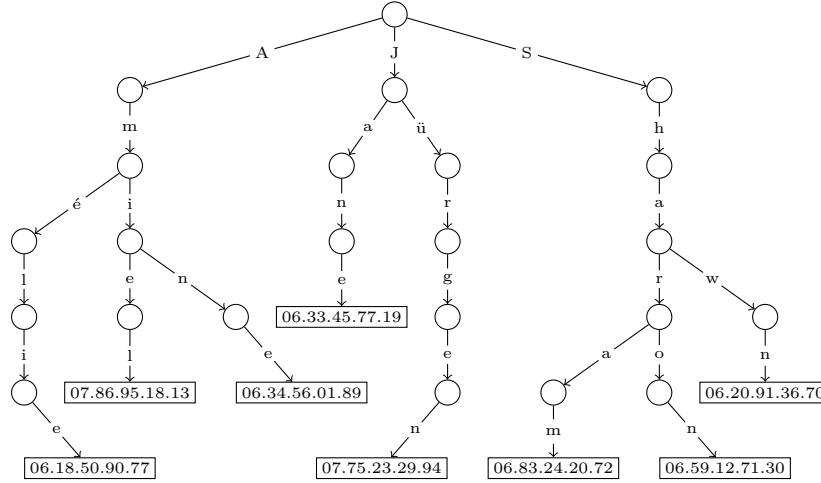
```

**Exemple 4.31.** — Dans cet exemple, nous souhaitons associer des chaînes de caractères à des numéros.

| Clé    | Valeur         |
|--------|----------------|
| Amélie | 06.18.50.90.77 |
| Amiel  | 07.86.95.18.13 |
| Amine  | 06.34.56.01.89 |
| Jane   | 06.33.45.77.19 |
| Jürgen | 07.75.23.29.94 |
| Sharam | 06.83.24.20.72 |
| Sharon | 06.59.12.71.30 |
| Shawn  | 06.20.91.36.70 |

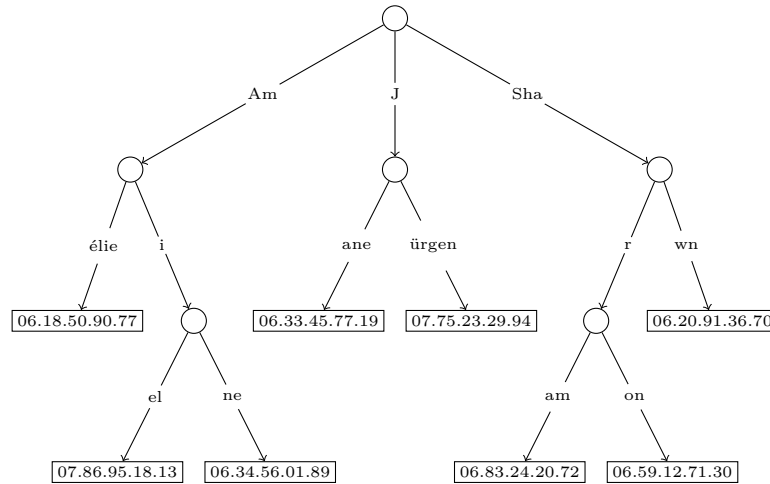
Nous utilisons un trie organisé comme suit.





**Remarque 4.32.** — Lorsque le trie est creux (beaucoup d'arêtes et peu d'associations), on regroupe plusieurs arêtes formant un chemin. On parle alors d'arbres radix ou d'arbres Patricia (abréviation pour *Practical Algorithm To Retrieve Information Coded In Alphanumeric*).

**Exemple 4.33.** — Nous revisitons l'exemple précédent sous le format d'un arbre :

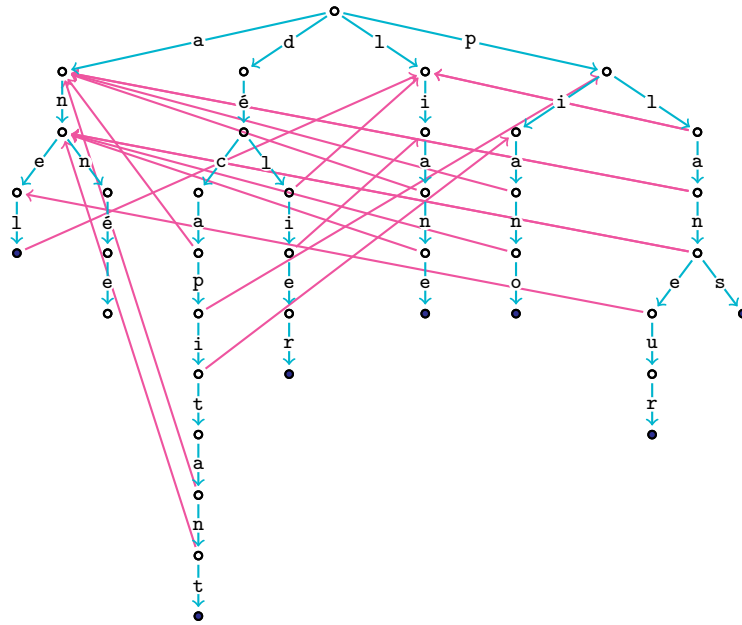


**4.1.2.2. L'algorithme d'Aho-Corasick.** — Étant donné deux mots  $s$  et une famille finie de mots  $\{p_1, p_2, \dots, p_k\}$  sur un alphabet  $\Sigma$ , le problème de la *recherche de plusieurs sous-chaînes* consiste à déterminer si l'un des motifs  $p_i$  est un *facteur* de  $s$  et, éventuellement, localiser  $p_i$  dans  $s$  (voire toutes les occurrences de tous les  $p_i$ ).

L'algorithme d'Aho-Corasick généralise les idées de Morris Pratt en organisant la famille de motifs  $\{p_1, p_2, \dots, p_k\}$  sous forme de trie dans lequel, pour chaque sommet, il a été identifié un lien vers le plus long suffixe existant dans le trie.

La recherche de motif se fait alors comme suit : pour chaque lettre de  $s$ , on essaie de progresser de la racine vers les feuilles. S'il n'est pas possible de trouver dans le trie une branche correspondant à la lettre lue, on utilise les liens secondaires pour se recalculer sur le plus long suffixe.

**Exemple 4.34.** — Considérons la recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans**. Le précalcul conduit à la figure suivante.



où les liens (en rose) n'ont pas été dessinés s'ils pointent vers la racine

**4.1.2.3. Le codage de Huffman.** — L'une des applications importantes des tries concerne la manipulation de codes binaires.

**4.1.2.3.1. Notion de code binaire.** — Un code binaire permet de représenter de l'information issue d'un alphabet de cardinal quelconque.

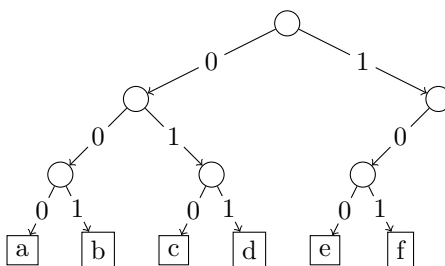
**Définition 4.35.** — Un *code binaire* pour l'alphabet  $\Sigma$  est une application injective  $E : \Sigma \rightarrow \{0, 1\}^+$ .

**Exemple 4.36.** — Avec  $\Sigma = \{a, b, c, d, e, f\}$ , on fixe le code  $E_{\clubsuit}$

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

**Exemple 4.37.** — Le code ASCII est une norme permettant de représenter les caractères les plus fréquents (notamment les chiffres arabes, les 26 lettres de l'alphabet en majuscule et en minuscule, quelques symboles mathématiques, quelques symboles de ponctuation, etc) sur 7 bits.

*Exemple 4.39.* — Le trie associé au code  $E_{\clubsuit}$  est le trie suivant



*Exemple 4.41.* — Avec le code

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

$$c a f e \mapsto 010000101100 \in \{0, 1\}^*.$$

**Exemple 4.44.** — Le  $\Sigma = \{z_1, z_2, \dots, z_\sigma\} \rightarrow \{0, 1\}^*$  donné par

$$z_i \mapsto \underbrace{0 \dots 0}_i 1$$

**Proposition 4.46.** — *Un code sans préfixe est uniquement décodable.*

*Démonstration.* — Nous observons que, dans le *trie* d'un code sans préfixe, toutes les feuilles représentent un codon et seules les feuilles représentent un codon.

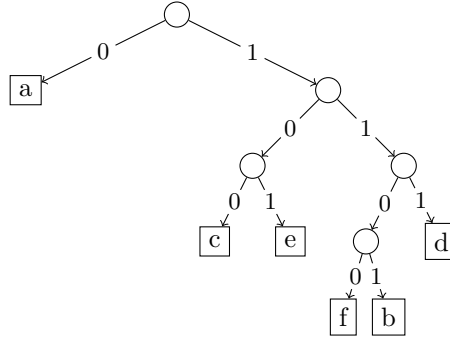
Pour décoder les premiers bits d'une suite de  $\{0, 1\}^*$ , on effectue une recherche dans le *trie*. Compte tenu de notre observation, cette recherche conduit nécessairement à une feuille, ce qui permet de commencer le décodage. De plus, comme il s'agit d'une feuille, nous ne risquons pas de nous tromper en rattachant les premiers bits lus au codon détecté.

Ceci fournit un algorithme de décodage linéaire et montre qu'un code sans préfixe est toujours *uniquement décodable*.  $\square$

**Exemple 4.47.** — Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



**4.1.2.3.2. Codage optimal.** — Dans ce paragraphe, nous cherchons à encoder un mot  $w \in \Sigma^*$ . Nous supposons avoir calculé le tableau des nombres d'occurrence de chaque caractère :  $(|w|_z)_{z \in \Sigma}$ .

**Définition 4.48.** — Nous appelons *longueur de codage* du mot  $w$  par rapport au code  $E$  la quantité

$$|E(w)| = \sum_{z \in \Sigma} |w|_z \cdot |E(z)|,$$

c'est-à-dire le nombre de bits utilisés pour encoder le mot  $w$ .

**Exemple 4.49.** — Calculons la longueur de codage sur un exemple

|                         | $a$    | $b$   | $c$    | $d$    | $e$    | $f$   | Total   |
|-------------------------|--------|-------|--------|--------|--------|-------|---------|
| Occurrences             | 45 000 | 5 000 | 12 000 | 16 000 | 13 000 | 9 000 | 100 000 |
| Codage $E_{\clubsuit}$  | 3      | 3     | 3      | 3      | 3      | 3     | 300 000 |
| Codage $E_{\spadesuit}$ | 1      | 4     | 3      | 3      | 3      | 4     | 224 000 |

$$45 + 4 \cdot 5 + 3 \cdot 12 + 3 \cdot 16 + 3 \cdot 13 + 4 \cdot 9 = 224.$$

Nous aurons besoin plus loin du code suivant :

LISTING 4.5. Dénombrement des occurrences de lettre

```

1215 def occurrences(s):
1216     occ = dict()
1217     for x in s:
1218         if x not in occ:
1219             occ[x] = 0
1220             occ[x] += 1
1221     return occ

```

Nous nous intéressons désormais au problème suivant : étant donné un mot  $w$  dont les nombres d'occurrences (ou des fréquences) des symboles de  $\Sigma$  sont connus, déterminer un code binaire de *longueur de codage minimum*.

**Théorème 4.50 (Inégalité de Kraft).** — Soit  $\Sigma$  un alphabet et  $E : \Sigma \rightarrow \{0, 1\}^*$  un code binaire uniquement décodable. Alors

$$\sum_{z \in \Sigma} 2^{-|E(z)|} \leq 1.$$

*Démonstration.* — Soit  $\Sigma$  un alphabet et  $E : \Sigma \rightarrow \{0, 1\}^*$  un code binaire uniquement décodable. Notons  $\ell_z$  l'entier  $\ell_z = |E(z)|$ . Appelons  $e = \max_{z \in \Sigma} |E(z)|$ . Appelons  $C$  la quantité

$$C = \sum_{z \in \Sigma} 2^{-|E(z)|}$$

Pour tout entier naturel  $m$ , nous avons

$$\begin{aligned}
 C^m &= \left( \sum_{z \in \Sigma} 2^{-|E(z)|} \right)^m \\
 &= \sum_{z_1 \in \Sigma} \sum_{z_2 \in \Sigma} \dots \sum_{z_m \in \Sigma} 2^{-\ell_{z_1} - \ell_{z_2} - \dots - \ell_{z_m}}
 \end{aligned}$$

Intéressons nous par ailleurs à l'ensemble

$$P = \{E(w); w \in \Sigma^m\}$$

des encodages de mots de longueur  $m$  sur  $\Sigma$ . Comme le code  $E$  est uniquement décodable, tous les éléments de  $P$  sont distincts et  $P$  est de cardinal  $\sigma^m$  (où  $\sigma = |\Sigma|$ ). Mais alors, chacun des termes dans la somme ci-dessus correspond à un élément de  $P$ .

Appelons  $q_\ell = \{w \in P; |w| = \ell\}$  le nombre d'encodage dans  $P$  qui sont de longueur  $\ell$ . On a donc

$$C^m = \sum_{\ell=1}^{me} q_\ell 2^{-\ell}$$

où  $e$  est la longueur du plus grand codon. Nous pouvons observer par ailleurs que  $q_\ell \leq 2^\ell$ . En conséquence

$$C^m \leq \sum_{\ell=1}^{me} 1 = me$$

Mais alors, pour tout entier  $m$ , on a

$$C \leq (me)^m.$$

En particulier, quand  $m$  tend vers  $+\infty$ , on obtient la majoration

$$C \leq 1.$$

□

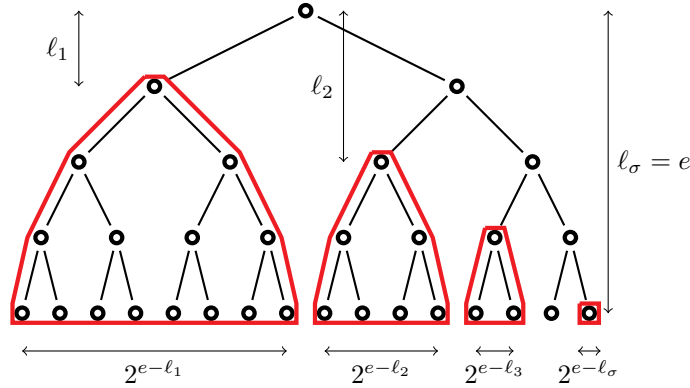
**Théorème 4.51.** — *Réciproquement, s'il existe une famille finie d'entiers naturels non nuls  $(\ell_z)_{z \in \Sigma}$  tels que  $\sum_{z \in \Sigma} 2^{-\ell_z} \leq 1$ , alors il existe un code préfixe de  $E : \Sigma \rightarrow \{0, 1\}^*$  dont les codons satisfont  $|E(z)| = \ell_z$  pour tout symbole  $z \in \Sigma$ .*

*Démonstration.* — Nous notons comme précédemment  $e = \max_{z \in \Sigma} \ell_z$ .

Nous démarrons avec un arbre binaire complet de hauteur  $e$  : il possède  $2^e$  feuilles. Nous répétons le processus suivant pour chacune des lettres  $z$  de  $\Sigma$  par ordre croissant de la valeur  $\ell_z$  : nous choisissons un sommet de l'arbre de profondeur  $\ell_z$  non encore marqué, nous associons la lettre  $z$  au sommet choisi, nous supprimons tous les descendants de ce sommet.

Comme les lettres sont considérées par ordre croissant de  $\ell_z$ , nous ne choisissons jamais comme sommet associé à une lettre un sommet qui serait sur le chemin entre la racine et le sommet choisi pour une autre lettre.

Nous observons que, initialement, il y a  $2^e$  feuilles. À chaque étape, nous supprimons  $2^{e-\ell_z}$  feuilles. Du fait de l'inégalité de Kraft, on a  $\sum_{z \in \Sigma} 2^{e-\ell_z} \leq 2^e$ . Ainsi, tout au long de la procédure, il existe au moins une feuille à profondeur  $e$  qui n'a pas encore été supprimée, qui implique qu'il existe bien un sommet à profondeur  $\ell_z$ .



□

**Corollaire 4.52.** — *La compression optimale de données sans perte est toujours possible en utilisant un code sans préfixes.*

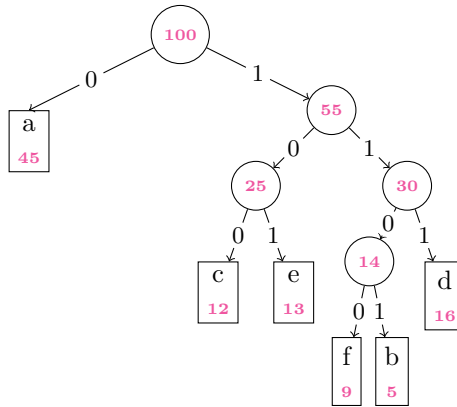
**4.1.2.3.3. Trie pondéré et algorithme de Huffman.** — Voyons comment construire un code optimal.

**Définitions 4.53.** — Étant donné un code  $E : \Sigma \rightarrow \{0,1\}^*$  et des nombres d'occurrence, l'*arbre de compression* de  $E$  est le trie pondéré qui représente  $E$  où le poids d'une feuille est son nombre d'occurrence et le poids d'un nœud interne est la somme des poids de ses fils.

La longueur de codage d'un arbre de compression est le poids porté par la racine.

Nous disons qu'un arbre de compression est optimum si la longueur de codage de cet arbre est minimum parmi tous les arbres de compressions de  $E$ .

**Exemple 4.54.** — Nous poursuivons l'exemple 4.49



Nous implémentons un arbre de compression comme suit :

LISTING 4.6. Arbre de compression en Python

```

1222 class HuffmanNode():
1223     def __init__(self, a, b):
1224         if isinstance(a, str) and isinstance(b, int):
1225             self.letter = a
1226             self.weight = b
1227         if isinstance(a, HuffmanNode) and isinstance(b, HuffmanNode):
1228             self.weight = a.weight + b.weight
1229             self.zero = a
1230             self.one = b
1231     def __lt__(self, other):
1232         return self.weight < other.weight
1233     def show_code(self):
1234         if hasattr(self, "letter"):
1235             return [":" + self.letter]
1236         else:
1237             c0 = self.zero.show_code()
1238             c1 = self.one.show_code()

```

```

1239         return ["0"+c for c in c0] + ["1"+c for c in c1]
1240     def get_symbol(self, s):
1241         if hasattr(self, "letter"):
1242             return (self.letter,s)
1243         elif s[0] == '0':
1244             return self.zero.get_symbol(s[1:])
1245         else:
1246             return self.one.get_symbol(s[1:])
1247     def decode(self, s):
1248         if s == "":
1249             return ""
1250         else:
1251             (x, t) = self.get_symbol(s)
1252             return x + self.decode(t)

```

**Lemme 4.55.** — Dans un arbre optimum, tout nœud interne possède deux fils.

*Démonstration.* —

□

**Lemme 4.56.** — Dans un arbre optimum, les deux symboles de plus petite occurrence sont à la profondeur de l'arbre.

*Démonstration.* —

□

**Lemme 4.57.** — Il existe un arbre optimum dans lequel les deux plus petites occurrences sont « frères ».

*Démonstration.* —

□

**Lemme 4.58.** — Soit  $A$  un arbre dans lequel les deux plus petites occurrences  $x$  et  $y$  sont « frères ». Soit  $A'$  l'arbre où le père de  $x$  et  $y$  est remplacé par un symbole  $x+y$  poids somme.

Alors  $A$  est un arbre de compression optimal ssi  $A'$  est un arbre de compression optimal

*Démonstration.* —

□

Nous pouvons à présent proposer l'algorithme de construction d'un codage suivant



**Algorithme 4.1 : Algorithme de Huffman****Entrées :** Occurences des symboles  $(o_\sigma)_{\sigma \in \Sigma}$ **Sorties :** Arbre de compression optimal

```

1  $F \leftarrow$  file à priorité vide
2 pour  $\sigma \in \Sigma$  faire
3   | Enfiler un arbre feuille d'étiquette  $(\sigma, o_\sigma)$ .
4 tant que  $F$  contient  $\geq 2$  arbres faire
5   | Extraire les deux arbres  $A_1$  et  $A_2$  de plus petit
   | poids  $o_1$  et  $o_2$ .
6   | Enfiler l'arbre Nœud( $A_1, A_2$ ) de poids  $o_1 + o_2$ .
7 retourner Sommet de la file

```

**Proposition 4.59.** — *L'algorithme de Huffman calcul un arbre optimal.*

*Démonstration.* —

□

Voici une proposition d'implémentation.

LISTING 4.7. Représentation des tries en Python

```

1253 import heapq as hp
1254
1255 def build_huffman(s):
1256     occ = occurrences(s)
1257     L = [HuffmanNode(x, occ[x]) for x in occ]
1258     hp.heapify(L)
1259     while len(L) >= 2:
1260         h1 = hp.heappop(L)
1261         h2 = hp.heappop(L)
1262         h = HuffmanNode(h1, h2)
1263         hp.heappush(L, h)
1264     return L[0]

```

## 4.2. Automates et langages rationnels

Les notes qui suivent sont tirées essentiellement de [Mad23].

### 4.2.1. Manipulations de langages. —

**4.2.1.1. Opérations sur les langages.** — Les langages étant des ensembles, nous rappelons les opérations usuelles sur les ensembles qui s'appliquent à des langages. Nous construisons également de nouvelles opérations

**Définition 4.60.** — Si  $L_1$  et  $L_2$  sont deux langages sur un même alphabet  $\Sigma$  (autrement dit,  $L_1, L_2 \subseteq \Sigma^*$ ), on peut former les langages *union*  $L_1 \cup L_2$  et *intersection*  $L_1 \cap L_2$  qui sont simplement les opérations ensemblistes usuelles (entre parties de  $\Sigma^*$ ).

Les opérations correspondantes sur les propriétés de mots sont respectivement le « ou logique » (=disjonction) et le « et logique » (=conjonction) : à titre d'exemple, sur  $\Sigma = \{a, b\}$  si  $L_1$  est le langage des mots commençant par  $a$  et  $L_2$  le langage des mots finissant par  $b$ , alors  $L_1 \cup L_2$  est le langage des mots commençant par  $a$  *ou bien* finissant par  $b$ , tandis que  $L_1 \cap L_2$  est le langage des mots commençant par  $a$  *et* finissant par  $b$ .

**Définition 4.61.** — Si  $L$  est un langage sur l'alphabet  $\Sigma$ , autrement dit  $L \subseteq \Sigma^*$ , on peut former le langage  $\Sigma^* \setminus L$ , parfois noté simplement  $\bar{L}$  si ce n'est pas ambigu, dit *langage complémentaire* de  $L$ , et qui est simplement l'ensemble des mots sur  $\Sigma$  n'appartenant pas à  $L$ .

L'opération correspondante sur les propriétés de mots est la négation logique.

**Exemple 4.62.** — Sur  $\Sigma = \{a, b\}$ , si  $L$  est le langage des mots commençant par  $a$ , alors  $\bar{L}$  est le langage des mots ne commençant pas par  $a$ , c'est-à-dire, la réunion de  $\{\varepsilon\}$  et du langage des mots commençant par  $b$  (car sur  $\Sigma = \{a, b\}$ , un mot ne commençant pas par  $a$  est vide ou bien commence par  $b$ ).

**Définition 4.63.** — Si  $L_1$  et  $L_2$  sont deux langages sur un même alphabet  $\Sigma$  (autrement dit,  $L_1, L_2 \subseteq \Sigma^*$ ), on peut former le langage *concaténation*  $L_1 L_2$  : il est défini comme l'ensemble des mots  $w$  qui peuvent s'écrire comme concaténation d'un mot  $w_1$  de  $L_1$  et d'un mot  $w_2$  de  $L_2$ , soit

$$\begin{aligned} L_1 L_2 &:= \{w_1 w_2 : w_1 \in L_1, w_2 \in L_2\} \\ &= \{w \in \Sigma^* : \exists w_1 \in L_1 \exists w_2 \in L_2 (w = w_1 w_2)\} \end{aligned}$$

**Exemple 4.64.** — Sur l'alphabet  $\Sigma = \{a, b, c, d\}$ , si on a  $L_1 = \{a, bb\}$  et  $L_2 = \{bc, cd\}$  alors  $L_1 L_2 = \{abc, acd, bbcb, bbcd\}$ .

**Définition 4.65.** — Si  $L$  est un langage sur l'alphabet  $\Sigma$ , autrement dit  $L \subseteq \Sigma^*$ , et si  $r \in \mathbb{N}$ , on peut définir un langage  $L^r$  comme le langage  $L^r = \{w_1 \cdots w_r : w_1, \dots, w_r \in L\}$  constitué des concaténation de  $r$  mots appartenant à  $L$ , ou si on préfère, par récurrence :

- $L^0 = \{\varepsilon\}$ ,
- $L^{r+1} = L^r L$ .

**Exemple 4.66.** — Sur l'alphabet  $\Sigma = \{a, b, c, d\}$ , si on a  $L = \{a, bb\}$ , alors  $L^2 = \{aa, abb, bba, bbbb\}$  et  $L^3 = \{aaa, aabb, abba, abbbb, bbaa, bbabb, bbbba, bbbbbb\}$ .

**Remarque 4.67 (Attention).** —  $L^r$  n'est pas le langage  $\{w^r : w \in L\}$  constitué des répétitions  $r$  fois ( $w^r$ ) des mots  $w$  de  $L$  : c'est le langage des concaténations de  $r$  mots appartenant à  $L$  mais ces mots peuvent être différents. À titre d'exemple, si  $L = \{a, b\}$  alors  $L^r$  est le langage constitué des  $2^r$  mots de longueur exactement  $r$  sur  $\{a, b\}$ , ce n'est pas l'ensemble à deux éléments  $\{a^r, b^r\}$  constitué des seuls deux mots  $a^r = aaa \cdots a$  et  $b^r = bbb \cdots b$ .

**Définition 4.68.** — Si  $L$  est un langage sur l'alphabet  $\Sigma$ , on définit enfin l'*étoile de Kleene*  $L^*$  de  $L$  comme le langage constitué des concaténations d'un nombre quelconque de mots appartenant à  $L$ , c'est-à-dire la réunion de tous les langages  $L^r$  mentionnés ci-dessus :

$$\begin{aligned} L^* &:= \bigcup_{r=0}^{+\infty} L^r = \bigcup_{r \in \mathbb{N}} L^r \\ &= \{w_1 \cdots w_r : r \in \mathbb{N}, w_1, \dots, w_r \in L\} \end{aligned}$$

**Exemple 4.69.** — Sur l'alphabet  $\Sigma = \{a, b, c, d\}$ , si on a  $L = \{a, bb\}$ , alors on a  $L^* = \{\varepsilon, a, bb, aa, abb, bba, bbbb, aaa, aabb, abba, abbbb, bbaa, bbabb, bbbba, bbbbbb, \dots\}$ .

**Remarque 4.70.** — Comme ci-dessus, il faut souligner que les mots  $w_1, \dots, w_r$  concaténés n'ont pas à être égaux : notamment,  $\{a, b\}^*$  est le langage constitué de tous les mots sur l'alphabet  $\{a, b\}$ , pas le langage  $\{a\}^* \cup \{b\}^*$  constitué des mots obtenus en répétant la lettre  $a$  ou en répétant la lettre  $b$ .

On remarquera que la définition de  $L^*$  ci-dessus redonne bien, lorsqu'on l'applique à l'alphabet  $\Sigma$  lui-même (considéré comme langage des mots de longueur 1), l'ensemble  $\Sigma^*$  de tous les mots : la notation  $\Sigma^*$  est donc justifiée *a posteriori*.

Le mot vide appartient toujours à  $L^*$  (quel que soit  $L$ ) puisque  $L^0 = \{\varepsilon\}$  et qu'on peut prendre  $r = 0$  ci-dessus (autrement dit, le mot vide est la concaténation de zéro mots de  $L$ ).

On peut par ailleurs montrer que  $L^*$  est le plus petit (pour l'inclusion) langage  $M$  tel que  $M = \{\varepsilon\} \cup LM$ .

On introduit parfois la notation  $L^+ := \bigcup_{r=1}^{+\infty} L^r = \{w_1 \cdots w_r : r > 0, w_1, \dots, w_r \in L\}$  pour l'ensemble des mots formés par concaténation d'un nombre *non nul* de mots de  $L$ . Lorsque le mot vide  $\varepsilon$  n'appartient pas déjà à  $L$ , ce langage  $L^+$  diffère de  $L^*$  seulement en ce qu'il ne contient pas  $\varepsilon$  ; tandis que si  $\varepsilon$  appartient déjà à  $L$ , alors  $L^+$  est égal à  $L^*$ . En toute généralité, on a  $L^+ = LL^*$ .

Il sera marginalement question dans ces notes de « classes de langages » : une classe de langages est un ensemble de langages (c'est-à-dire une partie de  $\mathcal{P}(\Sigma^*)$ , ou si on préfère un élément de  $\mathcal{P}(\mathcal{P}(\Sigma^*))$ ). On ne développera pas de théorie générale des classes de langages et on n'en parlera pas de façon systématique, mais on parlera de certaines classes de langages importantes : la classe des langages rationnels ou reconnaissables, la classe des langages décidables, la classe des langages semi-décidable, la classe des langages reconnaissable en temps polynomial par un algorithme déterministe ou encore la classe des langages vérifiable en temps polynomial par un algorithme déterministe.

#### 4.2.1.2. Langages rationnels. —

**4.2.1.2.1. Définition et exemples.** — Les langages qui vont jouer le rôle le plus important dans cette section sont les langages dits « rationnels » ci-dessous, et qui s'avéreront être les mêmes que les langages « reconnaissables » par automates finis.

L'importance des langages rationnels, et des expressions régulières (=régulières) qui les décrivent, vient :

- du point de vue théorique : de ce qu'ils forment une classe à la fois suffisamment simple pour pouvoir être étudiée, suffisamment riche pour contenir toutes sortes de langages intéressants, suffisamment naturelle pour être définie de plusieurs manières différentes, et suffisamment flexible pour être stable un certain nombre d'opérations ;
- du point de vue pratique et informatique : de ce qu'ils sont algorithmiquement commodes à manier (grâce à la théorie) et suffisent à représenter beaucoup de « recherches » qu'on a naturellement envie de faire dans un texte, de « motifs » qu'on peut vouloir trouver, ou de « contraintes » qu'on peut vouloir imposer à une chaîne de caractères.

**Définition 4.71.** — Soit  $\Sigma$  un alphabet. L'ensemble des *langages rationnels*  $Rat(\Sigma)$  est le sous-ensemble de l'ensemble des langages défini inductivement par les axiomes :

$$\begin{array}{c} \frac{}{\emptyset \in Rat(\Sigma)} \text{ (Vide)} \quad \frac{}{\{\varepsilon\} \in Rat(\Sigma)} \text{ (Mot vide)} \quad \frac{}{\{x\} \in Rat(\Sigma)} \text{ (Symbole)} \\ \text{où } x \text{ est un symbole de } \Sigma \text{ et par les règles :} \\ \frac{\mathcal{L}_1 \in Rat(\Sigma) \quad \mathcal{L}_2 \in Rat(\Sigma)}{\mathcal{L}_1 \cup \mathcal{L}_2 \in Rat(\Sigma)} \text{ (union)} \quad \frac{\mathcal{L}_1 \in Rat(\Sigma) \quad \mathcal{L}_2 \in Rat(\Sigma)}{\mathcal{L}_1 \mathcal{L}_2 \in Rat(\Sigma)} \text{ (concat)} \\ \frac{\mathcal{L} \in Rat(\Sigma)}{\mathcal{L}^* \in Rat(\Sigma)} \text{ (étoile)} \end{array}$$

**Exemple 4.72.** — Sur l'alphabet  $\{a, b, c, d\}$ , comme le langage  $\{c\}$  (constitué du seul mot  $c$ ) est rationnel, son étoile de Kleene, c'est-à-dire  $\{c\}^* = \{\varepsilon, c, cc, ccc, cccc, \dots\}$ , est rationnel, et comme  $\{d\}$  l'est aussi, la concaténation  $\{d\}(\{c\}^*) = \{d, dc, dcc, dccc, \dots\}$  est encore un langage rationnel. Nous pouvons résumer cette argumentation à travers un arbre de dérivation comme suit :

$$\frac{\frac{}{\{c\} \in Rat(\Sigma)} \text{ (Symbole)} \quad \frac{\frac{}{\{d\} \in Rat(\Sigma)} \text{ (Symbole)}}{\{d\}^* \in Rat(\Sigma)} \text{ (étoile)}}{\{c\}\{d\}^* \in Rat(\Sigma)} \text{ (concat)}$$

**Exemple 4.73.** — Soit  $\Sigma = \{a, b\}$  un alphabet et  $\mathcal{L}$  le langage des mots sur  $\Sigma$  de longueur impaire

$$\mathcal{L} = \{w \in \Sigma^*; |w| \text{ impaire}\}.$$

On a

$$\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^*$$

et la dérivation

$$\frac{\frac{}{\{a\} \in Rat(\Sigma)} \text{ (ax.)} \quad \frac{}{\{b\} \in Rat(\Sigma)} \text{ (ax.)}}{\{a, b\} \in Rat(\Sigma)} \text{ (union)} \quad \frac{\frac{}{\{a\} \in Rat(\Sigma)} \text{ (ax.)} \quad \frac{}{\{b\} \in Rat(\Sigma)} \text{ (ax.)}}{\{aa, ab, ba, bb\} \in Rat(\Sigma)} \text{ (union)} \quad \frac{\frac{}{\{a\} \in Rat(\Sigma)} \text{ (ax.)} \quad \frac{}{\{b\} \in Rat(\Sigma)} \text{ (ax.)}}{\{aa, ab, ba, bb\}^* \in Rat(\Sigma)} \text{ (étoile)}}{\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^* \in Rat(\Sigma)} \text{ (concat)}$$

qui prouve que  $\mathcal{L}$  est bien un langage rationnel.

**Remarque 4.74 (Attention).** — Le fait que la classe  $Rat(\Sigma)$  des langages rationnels soit stable par concaténation signifie que si  $L_1$  et  $L_2$  sont rationnels alors le langage  $L_1 L_2$  (constitué de tous les mots concaténés d'un mot de  $L_1$  et d'un mot de  $L_2$ ) est rationnel ; *cela ne signifie pas* qu'un langage rationnel donné soit stable par concaténation (un langage stable  $L$  par concaténation est un langage tel que si  $w_1, w_2 \in L$  alors  $w_1 w_2 \in L$ ).

**Exemple 4.75.** — Sur l'alphabet  $\Sigma = \{a, b, c, d\}$ , les langages suivants sont rationnels :

- le langage constitué des mots contenant au moins un  $d$  ;
- le langage constitué des mots ne contenant aucun  $d$  ;
- le langage constitué des mots commençant par un  $d$  ;
- le langage constitué des mots commençant par un  $d$  et finissant par un  $c$  ;
- le langage constitué des mots commençant par un  $d$  et finissant par un  $c$ , et dont toutes les lettres intermédiaires sont des  $a$  ou des  $b$ , c'est-à-dire, le langage constitué des mots de la forme suivante : un  $d$ , puis un nombre quelconque de  $a$  et de  $b$ , et enfin un  $c$  ;

Dans tous ces exemples, nous sentons qu'il serait utile de disposer d'une syntaxe qui permette de dénoter la construction qui désigne le langage en tant que langage rationnel.

**4.2.1.2.2. Expressions régulières.** — Pour décrire la manière dont un langage rationnel est fabriqué (à partir des langages de base par les opérations rationnelles), comme il est malcommode d'écrire quelque chose comme  $\{d\}(\{c\}^*)$ , on introduit un nouvel objet, les expressions régulières, qui sont des expressions servant à dénoter un langage rationnel. Par exemple, plutôt que d'écrire «  $\{d\}(\{c\}^*)$  », on parlera du langage « dénoté par l'expression régulière  $dc^*$  ». Ceci fournit du même coup une nouvelle définition des langages rationnels : ce sont les langages dénotés par une expression régulières.

Une expression régulière (sur un alphabet  $\Sigma$ ) est un mot sur l'alphabet  $\Sigma \cup \{\perp, \varepsilon, (, ), |, *\}$ , où  $\perp, \varepsilon, (, ), |, *$  sont de nouveaux caractères *n'appartenant pas* à l'alphabet  $\Sigma$ , appelés *métacaractères*, et qui servent à marquer la manière dont est formée l'expression régulière.

**Définition 4.76.** — Soit  $\Sigma$  un alphabet. L'ensemble des *expressions régulières*  $RegExp(\Sigma)$  est le langage sur  $\Sigma \cup \{\perp, \varepsilon, (, ), |, *\}$  défini inductivement par les axiomes :

$$\begin{array}{c}
 \frac{}{\perp \in RegExp(\Sigma)} \text{ (Vide)} \quad \frac{}{\varepsilon \in RegExp(\Sigma)} \text{ (Mot vide)} \quad \frac{}{x \in RegExp(\Sigma)} \text{ (Symbole)} \\
 \text{où } x \text{ est un symbole de } \Sigma \text{ et par les règles} \\
 \frac{r_1 \in RegExp(\Sigma) \quad r_2 \in RegExp(\Sigma)}{(r_1 | r_2) \in RegExp(\Sigma)} \text{ (union)} \quad \frac{r_1 \in RegExp(\Sigma) \quad r_2 \in RegExp(\Sigma)}{r_1 r_2 \in RegExp(\Sigma)} \text{ (concat)} \\
 \frac{r \in RegExp(\Sigma)}{(r)^* \in RegExp(\Sigma)} \text{ (étoile)}
 \end{array}$$

Compte tenu de la structure inductive des expressions régulières et des langages rationnels, il est facile de l'une à l'autre.

**Définition 4.77.** — Soit  $r$  une expression régulière, on appelle *langage rationnel* dénoté par l'expression régulière  $r$  le langage  $\mathcal{L}(r)$  défini par

$$\begin{array}{c} \frac{}{\mathcal{L}(\perp) = \emptyset} \text{ (Vide)} \quad \frac{}{\mathcal{L}(\varepsilon) = \{\varepsilon\}} \text{ (Mot vide)} \quad \frac{}{\mathcal{L}(x) = \{x\}} \text{ (Symbole)} \\ \text{où } x \text{ est un symbole de } \Sigma \text{ et par} \\ \frac{r_1 \in \text{RegExp}(\Sigma) \quad r_2 \in \text{RegExp}(\Sigma)}{\mathcal{L}(r_1 | r_2) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)} \text{ (union)} \quad \frac{r_1 \in \text{RegExp}(\Sigma) \quad r_2 \in \text{RegExp}(\Sigma)}{\mathcal{L}(r_1 r_2) = \mathcal{L}(r_1) \mathcal{L}(r_2)} \text{ (concat)} \\ \frac{r \in \text{RegExp}(\Sigma)}{\mathcal{L}(r^*) = \mathcal{L}(r)^*} \text{ (étoile)} \end{array}$$

Un langage rationnel est par construction la même chose qu'un langage pour lequel il existe une expression régulière qui le dénote. Les expressions régulières sont une syntaxe qui permet de parler d'ensembles de mots.

**Exemple 4.78.** — Sur l'alphabet  $\Sigma = \{a, b, c, d\}$ ,  $c$  est une expression régulière qui dénote le langage  $\{c\}$ , donc  $(c)^*$  en est une qui dénote le langage  $\{c\}^* = \{\varepsilon, c, cc, ccc, \dots\}$ , et enfin  $d(c)^*$  en est une qui dénote le langage  $\{d, dc, dcc, \dots\}$  des mots formés d'un  $d$  et d'une succession quelconques de  $c$ .

$$\frac{\frac{}{c \in \text{RegExp}(\Sigma)} \text{ (Symbole)} \quad \frac{\frac{}{d \in \text{RegExp}(\Sigma)} \text{ (Symbole)}}{(d)^* \in \text{RegExp}(\Sigma)} \text{ (étoile)}}{c(d)^* \in \text{RegExp}(\Sigma)} \text{ (concat)}$$

**Exemple 4.79.** — Voici quelques autres exemples, toujours sur  $\Sigma = \{a, b, c, d\}$  :

- l'expression régulière  $(a|b)$  dénote le langage  $\{a\} \cup \{b\} = \{a, b\}$  constitué des deux mots d'une seule lettre  $a$  et  $b$  ;
- l'expression régulière  $(a|b)c$  dénote le langage  $\{a, b\}\{c\} = \{ac, bc\}$ , de même que  $(ac|bc)$  ;
- l'expression régulière  $(bc)^*$  dénote le langage  $\{bc\}^* = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$  ;
- l'expression régulière  $a|(bc)^*$  dénote le langage  $\{a\} \cup \{bc\}^* = \{a, \varepsilon, bc, bcbc, bcbcbc, \dots\}$  ;
- l'expression régulière  $a|(bc)^*d$  dénote le langage  $\{ad, d, bcd, bcbcd, bcbcbcd, \dots\}$  ;
- l'expression régulière  $\perp d$  dénote le langage vide  $\emptyset$  (car il n'y a pas de mot dans le langage vide, donc pas non plus de mot dans sa concaténation avec le langage  $\{d\}$ ) ;
- l'expression régulière  $\varepsilon d$  dénote le langage  $\{d\}$  ;
- l'expression régulière  $\perp | c$  dénote le langage  $\{c\}$  ;
- l'expression régulière  $\varepsilon | c$  dénote le langage  $\{\varepsilon, c\}$ .

**Définition 4.80.** — On dira qu'un mot  $w$  *vérifie* une expression régulière  $r$  lorsque ce mot appartient au langage qu'elle dénote (i.e.,  $w \in L(r)$ ). Par exemple,  $dccc$  vérifie l'expression régulière  $d(c)^*$ .

**Définition 4.81.** — Deux expressions régulières  $r_1, r_2$  sont dites *équivalentes* lorsqu'elles dénotent le même langage. À titre d'exemple, sur l'alphabet  $\{a, b\}$ , les deux expressions régulières  $(ab)^*a$  et  $a(ba)^*$  sont équivalentes (toutes les deux dénotent le langage  $\{a, aba, ababa, \dots\}$  constitué des mots commençant et finissant par un  $a$  et dans lesquels chaque paire de  $a$  est séparée par un unique  $b$ ).

On verra plus loin qu'on dispose d'un algorithme permettant de décider si deux expressions régulières sont équivalentes.

**Exemple 4.82.** — Voici quelques exemples d'équivalences d'expressions régulières (valables en remplaçant les différents  $r$  qui interviennent dedans par des expressions régulières quelconques) :

- identités « triviales » :  $(r|\perp) \equiv r$ ,  $(\perp|r) \equiv r$ ,  $r\perp \equiv \perp$ ,  $\perp r \equiv \perp$ ,  $r\varepsilon \equiv r$ ,  $\varepsilon r \equiv r$ ,  $(\perp)^* \equiv \varepsilon$ ;
- identité d'associativité :  $((r_1|r_2)|r_3) \equiv (r_1|(r_2|r_3))$ ;
- identités de distributivité :  $r(r_1|r_2) \equiv (rr_1|rr_2)$ ,  $(r_1|r_2)r \equiv (r_1r|r_2r)$ ;
- identité de commutativité :  $(r_1|r_2) \equiv (r_2|r_1)$ ;
- identités « apériodiques » :  $((r_1|r_2))^* \equiv (r_1)^*(r_2(r_1)^*)^*$ ,  $((r_1|r_2))^* \equiv ((r_1)^*r_2)^*(r_1)^*$  et  $(r_1r_2)^* \equiv (\varepsilon|r_1(r_2r_1)^*r_2)$ ;
- identités « cycliques » :  $(r)^* \equiv (\varepsilon|r)(rr)^* \equiv (\varepsilon|r|rr)(rrr)^* \equiv (\varepsilon|r|rr|rrr)(rrrr)^* \equiv \dots$ ;
- identités d'« idempotence » :  $(r|r) \equiv r$ ,  $((r)^*)^* \equiv (r)^*$ ;

**Remarque 4.83.** — Signalons au passage que toutes ces identités *ne suffisent pas* à produire toutes les équivalences entre expressions régulières. Par exemple, elles ne permettent pas de démontrer l'équivalence suivante :  $(a|b)^* \equiv ((a|b)(b|ab^*ab^*ab^*a))^*(\varepsilon|(a|b)(|ab^*|ab^*ab^*|ab^*ab^*ab^*))$ ; la question d'arriver à trouver un système d'axiomes qui permet de déduire toutes les équivalences entre expressions régulières est un problème délicat (il n'y en a notamment pas qui soit fini).

**Remarque 4.84.** — La convention de parenthésage introduite ci-dessus est inambiguë mais parfois inutilement lourde : on se permettra parfois de l'alléger, par exemple d'écrire  $(r_1|r_2|r_3)$  pour  $((r_1|r_2)|r_3)$  (ou pour  $(r_1|(r_2|r_3))$ ), ce qui n'a guère d'importance vu qu'elles dénotent le même langage), ou encore  $x^*$  pour  $(x)^*$  lorsque  $x$  est formé d'un seul caractère.

Pour retirer des parenthèses, la convention sur la priorité des opérations est la suivante : l'opération d'étoile  $*$  est la plus prioritaire (c'est-à-dire que  $ab^*$  se lit comme  $a(b)^*$  et non pas comme  $(ab)^*$ ), la concaténation est de priorité intermédiaire, et la barre de disjonction  $|$  est la moins prioritaire (c'est-à-dire que  $ab|cd$  se lit comme  $(ab|cd)$  et pas comme  $a(b|c)d$ ).

**4.2.2. Automates finis.** — Les automates finis sont un modèle de calcul particulièrement simple et particulièrement appropriés à l'étude et à l'analyse des langages rationnels. Il faut imaginer un automate fini comme une machine disposant d'une quantité finie (et sous-entendu, très limitée) de mémoire : la configuration complète de cette mémoire est décrite par un *état*, qui appartient à un ensemble fini d'états possibles. On fournit un mot à l'automate en le lui faisant consommer lettre par lettre (de la première à la dernière), et à chaque lettre qu'il reçoit, l'automate prend une décision (à savoir, passer dans un nouvel état) en fonction de son état actuel et de la lettre qu'on lui donne à consommer ; l'automate commence par un état dit *initial* ; lorsque le mot a été entièrement consommé, l'état dans lequel l'automate se trouve a pour conséquence soit que l'automate *accepte* le mot (s'il se trouve dans un état *final*), soit qu'il le *rejette*. L'ensemble des mots acceptés par l'automate est un langage dit *reconnaissable*. On va voir en 4.149 que ces langages reconnaissables sont en fait les mêmes que les langages rationnels définis en ??.

Une subtilité est qu'il existe plusieurs sortes d'automates finis. Nous allons en définir quatre ou cinq, de plus en plus généraux, mais nous allons voir qu'ils sont, finalement, tous équivalents, c'est-à-dire que leur puissance de calcul, ou les langages qu'ils sont capables de reconnaître, sont toujours les mêmes (en revanche, cette équivalence peut avoir un coût algorithmique, car la conversion d'une sorte d'automate en une autre n'est pas gratuite). Du plus particulier au plus général, les automates que nous allons définir sont informellement décrits ainsi :

- les automates finis déterministes (complets) (cf. ??) : ceux-ci partent d'un état initial bien défini, suivent à chaque lettre consommée une transition bien définie vers un nouvel état, et leur comportement est donc entièrement déterministe (d'où le nom) ;
- les automates finis déterministes à spécification incomplète (cf. ??) : la différence est qu'ici l'automate n'a pas forcément d'instruction sur quoi faire quand il est dans un certain état et reçoit un certain symbole, il peut manquer des transitions, auquel cas l'automate cesse de fonctionner et rejette le mot ;
- les automates non-déterministes (cf. 4.109) : ceux-ci peuvent admettre *plusieurs* possibilités de nouvel état dans lequel transitionner lorsqu'on leur donne une lettre donnée à consommer depuis un état donné, et l'automate accepte le mot s'il y a une manière de faire les transitions qui résulte en ce que le mot soit accepté ;
- les automates non-déterministes à transitions spontanées (cf. 4.119) : ceux-ci ont, en plus des précédents, le pouvoir d'évoluer spontanément, sans consommer de lettre ;
- on introduira aussi des automates encore plus généraux, à transitions étiquetées par des expressions régulières (cf. 4.143) pour montrer que tout langage reconnaissable est rationnel.

Dans tous les cas, les automates se représenteront comme des graphes orientés (admettant des boucles et arêtes multiples ; cf. ?? ci-dessous pour un exemple simple), dont les sommets sont les états de l'automate (on leur donne généralement des noms



pour les reconnaître, mais ces noms sont arbitraires), dont les arêtes sont étiquetées par des symboles (des lettres de l'alphabet, ou éventuellement le mot vide  $\varepsilon$  ou une expression régulière sur l'alphabet), et dont certains sommets (=états) sont distingués comme « initiaux » et/ou « finaux » (par une flèche pointant depuis nulle part vers l'état en question ou depuis l'état en question vers nulle part).

Un mot sera accepté par l'automate toujours selon la même logique : s'il y a moyen de relier un état initial à un état final par un chemin orienté dans le graphe tel que le mot corresponde à la lecture des étiquettes des arêtes du chemin dans l'ordre.

#### 4.2.2.1. Automates finis déterministes complets (=DFA). —

**Définition 4.85.** — Un *automate fini déterministe* (complet), ou en abrégé DFA (pour *deterministic finite automaton*), sur un alphabet  $\Sigma$  est la donnée

- d'un ensemble fini  $Q$  dont les éléments sont appelés *états*,
- d'un état  $q_0 \in Q$  appelé *état initial* (un DFA a un unique état initial),
- d'un ensemble  $F \subseteq Q$  d'états appelés états *finiaux*<sup>(1)</sup> ou *acceptants*,
- d'une fonction  $\delta: Q \times \Sigma \rightarrow Q$  appelée *fonction de transition*.

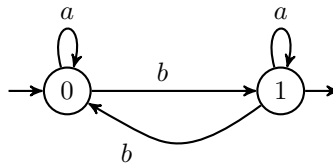
Graphiquement, on représente un DFA comme un graphe orienté aux arêtes étiquetées par des éléments de  $\Sigma$  : plus exactement, on trace un nud pour chaque élément  $q \in Q$ , et lorsque  $\delta(q, x) = q'$  on introduit une flèche  $q \rightarrow q'$  étiquetée par la lettre  $x$ .

La condition cruciale (pour être un DFA) est que, pour chaque état  $q \in Q$  et chaque lettre  $x \in \Sigma$ , *il existe une unique* arête partant de  $q$  et étiquetée par  $x$  (c'est essentiellement une reformulation du fait que  $\delta$  est une fonction).

En outre, on introduit une flèche pointant de nulle part vers  $q_0$  (pour marquer celui-ci comme l'état initial), et pour chaque  $q \in F$  une flèche pointant de  $q$  vers nulle part<sup>(2)</sup> (pour marquer ceux-ci comme états finaux).

Pour abréger la représentation graphique, lorsque plusieurs arêtes étiquetées par des lettres  $x, y$  différentes relient les mêmes sommets  $q, q'$  (i.e., lorsqu'on a à la fois  $\delta(q, x) = q'$  et  $\delta(q, y) = q'$ ), on pourra écrire «  $x, y$  », voire «  $x|y$  », sur l'arête en question et ne la tracer qu'une seule fois.

**Exemple 4.86.** — Pour donner un exemple simple, l'automate sur  $\Sigma = \{a, b\}$  représenté ci-dessous a  $Q = \{0, 1\}$  et  $q_0 = 0$  et  $F = \{1\}$  et la fonction de transition  $\delta$  donnée par  $\delta(0, a) = 0$  et  $\delta(0, b) = 1$  et  $\delta(1, a) = 1$  et  $\delta(1, b) = 0$ . On pourra se convaincre (une fois lues les définitions plus loin) que cet automate accepte les mots dont le nombre de  $b$  est impair.



1. Le pluriel de « final » est indifféremment « finaux » ou « finals ».

2. Certains auteurs préfèrent d'autres conventions, par exemple celle consistant à entourer deux fois les états finaux.

Il faut comprendre le fonctionnement d'un DFA de la manière suivante : initialement, l'automate est dans l'état initial  $q_0$ . On va lui présenter un mot  $w \in \Sigma^*$ , lettre par lettre, de la gauche vers la droite : i.e., si  $w = x_1 \cdots x_n$  on va faire consommer à l'automate les lettres  $x_1, x_2, \dots, x_n$  dans cet ordre. Le fait de consommer une lettre  $x$  fait passer l'automate de l'état  $q$  à l'état  $\delta(q, x)$  ; autrement dit, l'automate passe successivement dans les états  $q_0$  puis  $q_1 := \delta(q_0, x_1)$  puis  $q_2 := \delta(q_1, x_2)$ , et ainsi de suite jusqu'à  $q_n := \delta(q_{n-1}, x_n)$  : on dit que l'automate effectue les transitions  $q_0 \rightarrow q_1$  (en consommant  $x_1$ ) puis  $q_1 \rightarrow q_2$  (en consommant  $x_2$ ) et ainsi de suite. Si  $q_n$  est l'état dans lequel se trouve l'automate une fois qu'il a consommé le mot  $w$ , on dira que l'automate *accepte* ou *rejette* le mot selon que  $q_n \in F$  ou que  $q_n \notin F$ .

Graphiquement, on peut présenter la procédure de la manière suivante : on part de l'état  $q_0$  (sommet du graphe représentant l'automate) indiqué par la flèche entrante (pointant de nulle part), et pour chaque lettre du mot  $w = x_1 \cdots x_n$  considéré, on suit l'arête portant cette lettre pour étiquette (et partant de l'état où on se trouve actuellement). Si à la fin l'état  $q_n$  est acceptant (représenté par une flèche pointant vers nulle part), le mot  $w$  est accepté, sinon il est rejeté.

**Remarque 4.87.** — Cela revient encore à dire que le mot  $w$  est accepté lorsqu'il existe un chemin orienté dans l'automate, reliant l'état  $q_0$  initial à un état  $q_n$  final, et tel que le mot  $w = x_1 \cdots x_n$  soit obtenu en lisant dans l'ordre les étiquettes  $x_i$  des différentes arêtes  $q_{i-1} \rightarrow q_i$  de ce chemin. Cette définition pourra resservir presque à l'identique pour les autres sortes d'automates, la spécificité des DFA est que l'état initial  $q_0$  est unique et que connaissant l'étiquette  $x_i$  il n'y a qu'une seule transition  $q_{i-1} \rightarrow q_i$  possible (il n'y a pas de choix à faire, pas de non-déterminisme).

Pour donner de façon plus formelle la définition du langage accepté par un automate, il sera utile d'introduire une fonction  $\delta^*$  de transition multiple (une généralisation de  $\delta$ ), qui définit l'état  $\delta^*(q, w)$  dans lequel aboutit un DFA si à partir de l'état  $q$  on lui fait consommer successivement les lettres d'un mot  $w$ .

**Définition 4.88.** — Formellement : si  $A = (Q, q_0, F, \delta)$  est un DFA sur l'alphabet  $\Sigma$ , on définit une fonction  $\delta^*: Q \times \Sigma^* \rightarrow Q$  par  $\delta^*(q, x_1 \cdots x_n) = \delta(\cdots \delta(\delta(q, x_1), x_2) \cdots, x_n)$  ou, ce qui revient au même (par récurrence sur la longueur du second argument) :

- $\delta^*(q, \varepsilon) = q$  quel que soit  $q \in Q$  (où  $\varepsilon$  désigne le mot vide),
- $\delta^*(q, wx) = \delta(\delta^*(q, w), x)$  quels que soient  $q \in Q$ ,  $w \in \Sigma^*$  et  $x \in \Sigma$ ,

**Remarque 4.89.** — En particulier,  $\delta^*(q, x) = \delta(q, x)$  si  $x \in \Sigma$ , donc avec la convention faite en ??, on peut dire que  $\delta^*$  prolonge  $\delta$  ; il sera par ailleurs utile de remarquer que  $\delta^*(q, ww') = \delta^*(\delta^*(q, w), w')$ , ce qui se démontre par récurrence).

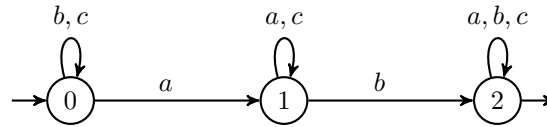
**Définition 4.90.** — Cette fonction  $\delta^*$  étant définie, on dira que l'automate  $A$  *accepte* ou *reconnaît* un mot  $w$  lorsque  $\delta^*(q_0, w) \in F$  ; dans le cas contraire, on dira qu'il *rejette* le mot  $w$ .

**Définition 4.91.** — L'ensemble  $L(A)$  (ou  $L_A$ ) des mots acceptés par l'automate  $A$  s'appelle *langage accepté*, ou *reconnu*, ou *défini*, par l'automate  $A$ .

**Définition 4.92.** — Un langage  $L \subseteq \Sigma^*$  qui peut s'écrire sous la forme du langage  $L(A)$  accepté par un DFA  $A$  est dit *reconnaissable* (sous-entendu : par automate déterministe fini).

**Définition 4.93.** — On dit que deux DFA  $A, A'$  sont *équivalents* lorsqu'ils reconnaissent le même langage, i.e.,  $L(A) = L(A')$ .

**Exemple 4.94.** — l'automate fini ci-dessous sur  $\Sigma := \{a, b, c\}$  a trois états,  $Q = \{0, 1, 2\}$ . On peut en faire la description informelle suivante : l'automate commence dans l'état 0, où il reste jusqu'à rencontrer un  $a$  qui le fait passer dans l'état 1, où il reste ensuite jusqu'à rencontrer un  $b$  qui le fait passer dans l'état 2, où il reste définitivement et qui constitue le seul état acceptant.



Cette description rend clair le fait que l'automate en question accepte exactement les mots contenant un  $a$  suivi, pas forcément immédiatement, d'un  $b$  ; autrement dit, les mots dont  $ab$  est un sous-mot (cf. ??). Ce langage est donc reconnaissable. (Il est aussi rationnel puisque dénoté par l'expression régulière  $(b|c)^*a(a|c)^*b(a|b|c)^*$ .)

LISTING 4.8. Implémentation Python d'un DFA

```

1265 class DFA():
1266     def __init__(self, q0):
1267         self.initial = q0
1268         self.transitions = dict()
1269         self.finals = set()
1270     def add_final_state(self, q):
1271         self.finals.add(q)
1272     def add_transition(self, q, x, qq):
1273         if q not in self.transitions:
1274             self.transitions[q] = dict()
1275         self.transitions[q][x] = qq
1276     def accepts_from(self, q, w):
1277         if w == "":
1278             return q in self.finals
1279         else:
1280             return self.accepts_from(self.transitions[q][w[0]],
1281                                     ↪ w[1:])
1281     def accepts(self, w):
1282         return self.accepts_from(self.initial, w)

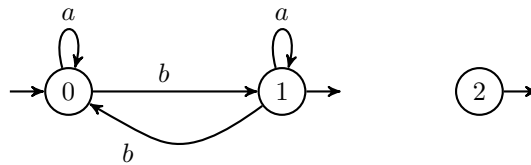
```

**Définition 4.95.** — Un état  $q$  d'un DFA est dit *accessible* lorsqu'il existe un mot  $w \in \Sigma^*$  tel que  $q = \delta(q_0, w)$ , autrement dit, graphiquement, lorsqu'il existe un chemin

orienté  $q_0, q_1, \dots, q_n = q$  reliant l'état initial  $q_0$  à l'état  $q$  considéré : bref, cela correspond à un état auquel il est possible que l'automate arrive (en partant de l'état initial et en consommant un certain mot). Dans le cas contraire, l'état est dit *inaccessible*. Il est évident qu'ajouter ou supprimer (ou modifier) les états accessibles dans un DFA ne change rien au langage reconnu (on obtient des automates équivalents).

**Remarque 4.96.** — La détermination de l'ensemble des états accessibles se fait grâce à un simple parcours de graphe (cf section ??).

**Exemple 4.97.** — Par exemple, dans le DFA qui suit, l'état 2 est inaccessible (l'automate est donc équivalent à celui représenté en ??). On remarquera qu'il ne change rien que cet état soit final ou non.



On va maintenant introduire différentes variations sur le thème des automates finis, c'est-à-dire différentes généralisations de la définition faite en ?? correspondant à des types d'automates finis plus puissants que les DFA mais dont on va montrer, à chaque fois, qu'ils peuvent se ramener à des DFA au sens où pour chacun de ces automates généralisés on pourra construire algorithmiquement un DFA qui reconnaît le même langage (si bien que la classe des langages reconnaissables par n'importe laquelle de ces sortes d'automates sera toujours la même). Les plus simples sont les automates déterministes finis incomplets et on va donc commencer par eux.

#### 4.2.2.2. Automates finis déterministes à spécification incomplète (=DFAi). —

**Définition 4.98.** — Un *automate fini déterministe à spécification incomplète* ou *...partielle*, ou simplement *automate fini déterministe incomplet*<sup>(3)</sup>, en abrégé *DFAi*, sur un alphabet  $\Sigma$  est la donnée

- d'un ensemble fini  $Q$  d'états,
- d'un état initial  $q_0 \in Q$ ,
- d'un ensemble  $F \subseteq Q$  d'états finaux,
- d'une fonction de transition *partielle*<sup>(4)</sup>  $\delta: Q \times \Sigma \dashrightarrow Q$ ,

Autrement dit, la seule différence avec la définition faite en ?? est que la fonction  $\delta$  est partielle, ce qui signifie qu'elle n'est pas obligatoirement définie sur tout couple  $(q, x) \in Q \times \Sigma$ .

3. Le mot « incomplet » signifie en fait « non nécessairement complet », i.e., l'automate a le droit de manquer certaines transitions, mais il peut très bien être complet (un DFA est un DFAi particulier, pas le contraire).

4. Une « fonction partielle »  $f: X \dashrightarrow Y$ , où  $X, Y$  sont deux ensembles est, par définition, la même chose qu'une fonction  $f: D \rightarrow Y$  où  $D \subseteq X$  est un sous-ensemble de  $X$  appelé *ensemble de définition* de  $f$ . (Lorsque en fait  $D = X$ , la fonction est dite « totale ».)

**Remarque 4.99.** — Un DFA est considéré comme un DFAi particulier où la fonction de transition  $\delta$  se trouve être définie partout.

Graphiquement, on représente un DFAi comme un DFA, à la différence près que pour chaque  $q \in Q$  et chaque  $x \in \Sigma$ , il y a maintenant *au plus une* (et non plus exactement une) arête partant de  $q$  et étiquetée par  $x$ .

Le fonctionnement d'un DFAi est le même que celui d'un DFA, à la modification suivante près : si on donne à consommer à l'automate une lettre pour laquelle la transition n'est pas définie, i.e., s'il rencontre un  $x$  pendant qu'il se trouve dans un état  $q$  pour lequel  $\delta(q, x)$  n'est pas défini, alors l'automate cesse de fonctionner : l'automate n'a plus d'état, n'effectue plus de transition, et n'acceptera pas le mot quelles que soient les lettres ultérieures.

Cela revient une fois de plus à dire que le mot  $w$  est accepté lorsqu'il existe un chemin orienté dans l'automate, reliant l'état  $q_0$  initial à un état  $q_n$  final, et tel que le mot  $w = x_1 \cdots x_n$  soit obtenu en lisant dans l'ordre les étiquettes  $x_i$  des différentes arêtes  $q_{i-1} \rightarrow q_i$  de ce chemin. La différence des DFAi par rapport aux DFA est que, cette fois, la tentative de chemin pourrait s'arrêter prématurément (on ne peut plus consommer un symbole et passer dans un nouvel état, et à plus forte raison, aboutir à un état final).

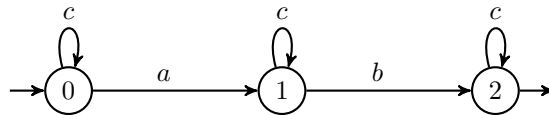
**Définitions 4.100.** — Formellement : si  $A = (Q, q_0, F, \delta)$  est un DFAi sur l'alphabet  $\Sigma$ , on définit une fonction  $\delta^*: Q \times \Sigma^* \dashrightarrow Q$  par  $\delta^*(q, x_1 \cdots x_n) = \delta(\cdots \delta(\delta(q, x_1), x_2) \cdots, x_n)$  avec la convention que dès qu'une sous-expression n'est pas définie, toute l'expression n'est pas définie, ou, ce qui revient au même (par récurrence sur la longueur du second argument) :

- $\delta^*(q, \varepsilon) = q$  quel que soit  $q \in Q$  (où  $\varepsilon$  désigne le mot vide),
- $\delta^*(q, wx) = \delta(\delta^*(q, w), x)$  à condition que  $q' := \delta^*(q, w)$  soit défini et que  $\delta(q', x)$  le soit (et si ces deux conditions ne sont pas satisfaites,  $\delta^*(q, wx)$  n'est pas défini).

Enfin, l'automate  $A$  accepte un mot  $w$  lorsque  $\delta^*(q_0, w)$  est défini et appartient à  $F$ ; dans le cas contraire (que ce soit parce que  $\delta^*(q_0, w)$  n'est pas défini ou parce qu'étant défini il n'appartient pas à  $F$ ), l'automate rejette le mot.

Le langage accepté  $L(A)$  et l'équivalence de deux automates sont définis de façon analogue aux DFA (cf. ??).

**Exemple 4.101.** — Voici un exemple de DFAi sur l'alphabet  $\Sigma = \{a, b, c\}$ . Cet automate reconnaît exactement les mots formés d'un nombre quelconque de  $c$ , suivis d'un  $a$ , suivis d'un nombre quelconque de  $c$ , suivis d'un  $b$ , suivis d'un nombre quelconque de  $c$ .



Ce langage est aussi dénoté par l'expression régulière  $c^*ac^*bc^*$ .

**Remarque 4.102.** — L'intérêt informatique des DFAi est de ne pas s'obliger à stocker inutilement des transitions et des états inutiles au sens où ils ne permettraient jamais d'accepter le mot (voir la notion d'automate « émondé » en 4.106 ci-dessous). C'est la raison pour laquelle, même si les DFA complets sont théoriquement plus satisfaisants à manier (pour la même raison qu'une fonction totale est plus satisfaisante qu'une fonction partielle), il est souvent algorithmiquement plus judicieux de travailler sur des DFAi.

Le code 4.9 suffit d'ailleurs à représenter des DFAi. La seule modification qu'il faut désormais apporter est de tenir compte du fait que la recherche de transitions peut échouer.

LISTING 4.9. Variante d'implémentation Python d'un DFAi

```

1283 # Reprendre le code de la classe DFA() et changer
1284 def accepte_from(self, q, w):
1285     if w == "":
1286         return q in self.finals
1287     else:
1288         try:
1289             return self.accepte_from(self.transitions[q][w[0]],
1290                                     ↪ w[1:])
1291         except KeyError:
1292             return False

```

**Proposition 4.103.** — Soit  $A = (Q, q_0, F, \delta)$  un DFAi sur un alphabet  $\Sigma$ . Alors il existe un DFA  $A' = (Q', q'_0, F', \delta')$  (sur le même alphabet  $\Sigma$ ) qui soit équivalent à  $A$  au sens où il reconnaît le même langage  $L(A') = L(A)$ . De plus,  $A'$  se déduit algorithmiquement de  $A$  en ajoutant au plus un état « puits » à  $A$  : on a  $\#Q' \leq \#Q + 1$ .

*Démonstration.* — On définit  $Q' = Q \cup \{q_\perp\}$  où  $q_\perp$  est un nouvel état (n'appartenant pas à  $Q$ ), qu'on appellera « puits ». On garde l'état initial  $q'_0 = q_0$ . On garde l'ensemble  $F' = F$  d'états finaux, c'est-à-dire notamment que le puits n'est pas acceptant. Enfin, on définit  $\delta'(q, x)$  pour  $q \in Q'$  et  $x \in \Sigma$  par

$$\delta'(q, x) = \delta(q, x) \text{ si } \delta(q, x) \text{ est défini}$$

$$\delta'(q, x) = q_\perp \text{ sinon}$$

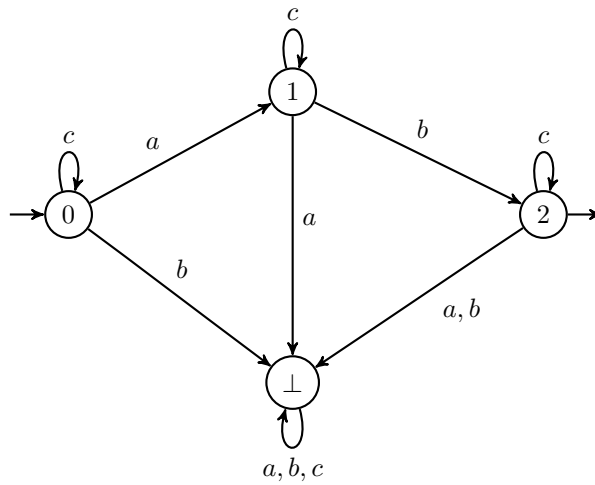
(notamment,  $\delta'(q_\perp, x) = q_\perp$  quel que soit  $x$ ).

Il est alors facile de voir que  $A'$  a le même comportement que  $A$  au sens où  $\delta'^*(q, w) = \delta^*(q, w)$  lorsque le terme de droite est défini et  $\delta'^*(q, w) = q_\perp$  sinon (le DFA  $A'$  « tombe dans le puits » lorsque le DFAi  $A$  cesse de fonctionner). En particulier, ils reconnaissent les mêmes langages.  $\square$

**Définition 4.104.** — De façon générale, un état *puits* dans un DFA est un état  $q_\perp$  tel que  $\delta(q_\perp, x) = q_\perp$  pour toute lettre  $x$ .

On dit que le DFA  $A'$  est obtenu en *complétant* le DFAi  $A$  lorsqu'il est obtenu par la procédure décrite dans la démonstration de cette proposition, c'est-à-dire par l'addition d'un état puits vers lequel on fait pointer toutes les transitions manquantes (sauf si  $A$  est déjà complet, auquel cas on convient qu'il est son propre complété, i.e., on n'ajoute un puits que quand c'est réellement nécessaire).

**Exemple 4.105.** — le DFA suivant représente la complétion du DFAi représenté en ?? :



**Définitions 4.106.** — On définit un état accessible d'un DFAi comme pour un DFA (cf. ??).

On dira en outre d'un état  $q$  d'un DFAi qu'il est *co-accessible* lorsqu'il existe un mot  $w \in \Sigma^*$  tel que  $\delta^*(q, w)$  soit défini et soit final, autrement dit, graphiquement, lorsqu'il existe un chemin orienté reliant l'état  $q$  considéré à un état final (remarquer que les états finaux eux-mêmes sont co-accessibles : prendre  $w = \varepsilon$  dans ce qu'on vient de dire).

Un état non co-accessible est donc un état à partir duquel il est impossible de faire accepter le mot. Cette définition pourrait également être faite pour les DFA, mais pour les DFAi elle présente l'intérêt qu'on peut supprimer les états non co-accessibles (ainsi, bien sûr, que toutes les transitions qui y conduisent) et obtenir de nouveau un DFAi.

**Définition 4.107.** — Un DFAi dont tous les états sont à la fois accessibles et co-accessibles (on les dit aussi *utiles*) est parfois appelé *émondé*.

On peut émonder un DFAi en ne conservant que ses états utiles<sup>(5)</sup> : ainsi, tout DFAi est équivalent à un DFAi émondé.

**Remarque 4.108.** — Il faut prendre garde au fait que certains auteurs définissent les « automates finis déterministes » (sans précision supplémentaire) comme étant complets par défaut, d'autres comme étant incomplets par défaut. Le plus prudent est de préciser systématiquement « complet » ou « incomplet » (en se rappelant qu'« incomplet » signifie « non nécessairement complet ») dès qu'il est important de ne pas confondre.

**4.2.2.3. Automates finis non-déterministes (=NFA).** — Le principe directeur général du non-déterminisme en informatique est, grossièrement parlant, le suivant : on a un mécanisme de calcul (ici, un automate fini) qui doit accepter ou rejeter une certaine donnée (ici, un mot qui lui est présenté), mais plutôt que d'évoluer de façon déterministe d'une configuration à une autre (ici, d'un état à un autre), il se peut qu'il existe plusieurs possibilités différentes, autrement dit, une configuration peut évoluer de plusieurs manières différentes, ce qui donne lieu à plusieurs chemins de calcul différents. Selon le point de vue qu'on adopte, et éventuellement la stratégie algorithmique pour modéliser le non-déterminisme, on peut considérer que ces chemins sont empruntés tous simultanément (et qu'on est dans tous les états possibles à la fois, c'est d'ailleurs l'approche qui servira en 4.115), ou que tous les chemins possibles sont testés successivement, ou encore que l'appareil non-déterministe « devine » quel est le bon ; mais en tout cas la règle fondamentale du non-déterminisme est toujours la suivante : *la donnée est acceptée dès lors qu'il existe au moins un chemin de calcul possible qui conduit à l'accepter* (dans notre cas, cela signifiera au moins un chemin acceptant le mot).

**Définition 4.109.** — Un *automate fini non-déterministe*, en abrégé *NFA*, sur un alphabet  $\Sigma$  est la donnée

- d'un ensemble fini  $Q$  d'états,
- d'un ensemble  $I \subseteq Q$  d'états dits initiaux,
- d'un ensemble  $F \subseteq Q$  d'états dits finaux,
- d'une *relation* de transition  $\delta \subseteq Q \times \Sigma \times Q$  (c'est-à-dire une partie du produit cartésien  $Q \times \Sigma \times Q$ , i.e., un ensemble de triplets  $(q, x, q') \in Q \times \Sigma \times Q$ ) ; lorsque  $(q, x, q') \in \delta$ , on dira qu'il existe une transition étiquetée par  $x$  de  $q$  vers  $q'$ .

Autrement dit, on autorise maintenant un ensemble quelconque d'états initiaux, et de même, au lieu qu'un état  $q$  et une lettre  $x$  déterminent un unique état  $q' = \delta(q, x)$ , on a maintenant affaire à un ensemble  $\delta$  quelconque de triplets  $(q, x, q')$  pour les transitions possibles.

---

5. Si on aime pinailler, il y a un petit problème pour émonder un DFAi n'ayant aucun état final (donc aucun état co-accessible, donc aucun état utile) ; pour résoudre ce problème, on peut modifier légèrement la définition d'un DFAi et autoriser que l'état initial ne soit pas défini (auquel cas l'automate n'accepte évidemment aucun mot, i.e., reconnaît le langage  $\emptyset$ ), si bien que l'automate émondé d'un DFAi sans état final est l'automate vide (sans aucun état).



**Remarque 4.110.** — Un DFAi (ou *a fortiori* un DFA) est considéré comme un NFA particulier en définissant l'ensemble des états initiaux du NFA comme un singleton,  $I_{\text{NFA}} = \{q_{0,\text{DFAi}}\}$ , et en définissant la relation de transition du NFA comme le graphe de la fonction de transition du DFAi, c'est-à-dire  $(q, x, q') \in \delta_{\text{NFA}}$  lorsque  $\delta_{\text{DFAi}}(q, x)$  est défini et vaut  $q'$ .

Graphiquement, on représente un NFA comme un DFA : comme un graphe orienté dont les nuds sont les éléments de  $Q$ , et où on place une arête étiquetée  $x$  de  $q$  vers  $q'$  pour chaque triplet  $(q, x, q') \in \delta$  ; comme précédemment, on marque les états initiaux par une flèche entrante (i.e., pointant de nulle part) et les états finaux par une flèche sortante (i.e., pointant vers nulle part).

Il faut comprendre le fonctionnement d'un NFA de la manière suivante : un mot  $w$  est accepté par l'automate lorsqu'il existe un chemin orienté conduisant d'un état initial  $q_0$  à un état final  $q_n$  et tel que le mot  $w = x_1 \cdots x_n$  soit obtenu en lisant dans l'ordre les étiquettes  $x_i$  des différentes arêtes  $q_{i-1} \rightarrow q_i$  de ce chemin ; autrement dit,  $w$  est accepté lorsqu'il existe  $q_0, \dots, q_n \in Q$  tels que  $q_0 \in I$  et  $q_n \in F$  et  $(q_{i-1}, x_i, q_i) \in \delta$  pour chaque  $1 \leq i \leq n$ .

**Remarque 4.111.** — La différence cruciale avec les DFAi est donc que, maintenant, il pourrait exister plusieurs chemins possibles partant d'un état initial dont les transitions sont étiquetées par les lettres du même mot. Insistons bien sur le fait que le mot est accepté dès lors que l'un au moins de ces chemins relie un état initial à un état final.

De même que dans le cadre des DFA et DFAi on a étendu la fonction de transition  $\delta$  à une fonction  $\delta^*$  qui décrit le comportement de l'automate quand on consomme plusieurs caractères (cf. ??), on va étendre la relation de transition  $\delta$  d'un NFA à la situation où on consomme plusieurs caractères. On a  $(q, w, q') \in \delta^*$  lorsqu'il existe un chemin orienté conduisant de  $q$  à  $q'$  et tel que le mot  $w$  soit obtenu en lisant dans l'ordre les étiquettes des différentes arêtes de ce chemin.

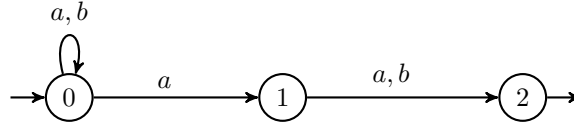
**Définitions 4.112.** — Formellement : si  $A = (Q, I, F, \delta)$  est un NFA sur l'alphabet  $\Sigma$ , on définit une relation  $\delta^* \subseteq Q \times \Sigma^* \times Q$  par  $(q, w, q') \in \delta^*$  lorsque  $w = x_1 \cdots x_n$  et qu'il existe  $(q_0, \dots, q_n)$  tels que  $q_0 = q$  et  $q_n = q'$  et  $(q_{i-1}, x_i, q_i) \in \delta$  pour chaque  $1 \leq i \leq n$  ; ou, ce qui revient au même (par récurrence sur la longueur de  $w$ ) :

- $(q, \varepsilon, q') \in \delta^*$  si et seulement si  $q' = q$ ,
- $(q, wx, q') \in \delta^*$  si et seulement si il existe  $q^\#$  tel que  $(q, w, q^\#) \in \delta^*$  et  $(q^\#, x, q') \in \delta$ .

Enfin, l'automate  $A$  accepte un mot  $w$  lorsqu'il existe  $q_0 \in I$  et  $q_\infty \in F$  tels que  $(q_0, w, q_\infty) \in \delta^*$ .

**Définition 4.113.** — Le langage accepté  $L(A)$  et l'équivalence de deux automates sont définis de façon analogue aux DFA (cf. ??).

**Exemple 4.114.** — Pour illustrer le fonctionnement des NFA, considérons l'automate à trois états sur  $\Sigma = \{a, b\}$  représenté par la figure suivante : on a  $Q = \{0, 1, 2\}$  avec  $I = \{0\}$  et  $F = \{2\}$  et  $\delta = \{(0, a, 0), (0, b, 0), (0, a, 1), (1, a, 2), (1, b, 2)\}$ .



Cet automate n'est pas déterministe car il existe deux transitions étiquetées  $a$  partant de l'état 0. En considérant les différents chemins possibles entre 0 et 2 dans ce graphe, on comprend que le langage qu'il reconnaît est le langage des mots sur  $\{a, b\}$  dont l'avant-dernière lettre est un  $a$  (c'est aussi le langage dénoté par l'expression régulière  $(a|b)^*a(a|b)$ ). Une façon de présenter le non-déterminisme est que l'automate « devine », quand il est dans l'état 0 et qu'on lui fait consommer un  $a$ , si ce  $a$  sera l'avant-dernière lettre, et, dans ce cas, passe dans l'état 1 pour pouvoir accepter le mot.

Comme les DFAi avant eux, les NFA sont en fait équivalents aux DFA ; mais cette fois-ci, le coût algorithmique de la transformation peut être bien plus important :

**Proposition 4.115.** — Soit  $A = (Q, I, F, \delta)$  un NFA sur un alphabet  $\Sigma$ . Alors il existe un DFA  $A' = (Q', q'_0, F', \delta')$  (sur le même alphabet  $\Sigma$ ) qui soit équivalent à  $A$  au sens où il reconnaît le même langage  $L(A') = L(A)$ . De plus,  $A'$  se déduit algorithmiquement de  $A$  avec une augmentation au plus exponentielle du nombre d'états :  $\#Q' \leq 2^{\#Q}$ .

*Démonstration.* — On définit  $Q' = \mathcal{P}(Q) = \{\mathbf{q} \subseteq Q\}$  l'ensemble des parties de  $Q$  : c'est ce qui servira d'ensemble d'états du DFA  $A'$  qu'on construit (i.e., un état de  $A'$  est un ensemble d'états de  $A$  intuitivement, c'est l'ensemble des états dans lesquels on se trouve simultanément). On pose  $q'_0 = I$  (l'état initial de  $A'$  sera l'ensemble de tous les états initiaux de  $A$ , vu comme un élément de  $Q'$ ) ; et on pose  $F' = \{\mathbf{q} \subseteq Q : \mathbf{q} \cap F \neq \emptyset\}$  : les états finaux de  $A'$  sont les états  $\mathbf{q}$  qui, vus comme des ensembles d'états de  $A$ , contiennent au moins un état final. Enfin, pour  $\mathbf{q} \subseteq Q$  et  $x \in \Sigma$ , on définit  $\delta'(\mathbf{q}, x) = \{q_1 \in Q : \exists q_0 \in \mathbf{q} ((q_0, x, q_1) \in \delta)\}$  comme l'ensemble de tous les états  $q_1$  de  $A$  auxquels on peut accéder depuis un état  $q_0$  dans  $\mathbf{q}$  par une transition  $(q_0, x, q_1)$  (étiquetée par  $x$ ) de  $A$ .

Il est alors facile de voir (par récurrence sur  $|w|$ ) que  $\delta'^*(\mathbf{q}, w)$  est l'ensemble de tous les états  $q_1 \in Q$  tels que  $(q_0, w, q_1) \in \delta^*$ , i.e., auxquels on peut accéder depuis un état  $q_0$  dans  $\mathbf{q}$  par une suite de transitions de  $A$  étiquetées par les lettres de  $w$ . En particulier,  $\delta'^*(I, w)$  est l'ensemble de tous les états de  $A$  auxquels on peut accéder depuis un état initial de  $A$  par une suite de transitions de  $A$  étiquetées par les lettres de  $w$  : le mot  $w$  appartient à  $L(A)$  si et seulement si cet ensemble contient un élément de  $F$ , ce qui par définition de  $F'$  signifie exactement  $\delta'^*(I, w) \in F'$ . On a bien prouvé  $L(A') = L(A)$ .

Enfin,  $\#Q' = \#\mathcal{P}(Q) = 2^{\#Q}$  (car une partie de  $Q$  peut se voir à travers sa fonction indicatrice, qui est une fonction  $Q \rightarrow \{0, 1\}$ ).  $\square$

**Définition 4.116.** — On dit que le DFA  $A'$  est obtenu en *déterminisant* le NFA  $A$  lorsqu'il est obtenu par la procédure décrite dans la démonstration de cette proposition en ne gardant que les états accessibles.

Algorithmiquement, la détermination de  $A$  s'obtient par la procédure suivante :

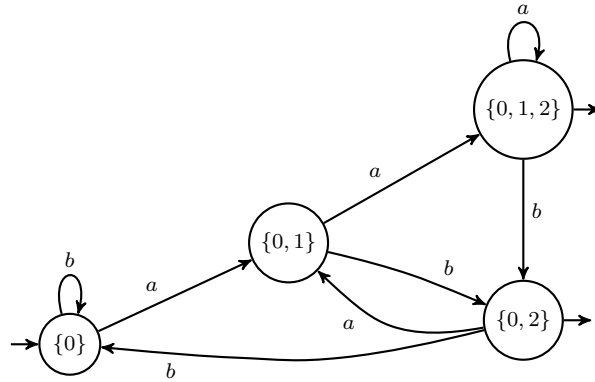
- créer une file (ou une pile) d'ensembles d'états de  $A$  ; initialiser cette file avec le seul élément  $I$  (vu comme un sous-ensemble de  $Q$ ) ; et créer l'automate  $A'$  avec initialement l'unique état  $I$ , marqué comme état initial, et aussi comme final s'il contient un état final de  $A$  ;
- tant que la file/pile n'est pas vide : en extraire un élément  $\mathbf{q}$ , et, pour chaque lettre  $x \in \Sigma$ ,
  - calculer l'ensemble  $\mathbf{q}' = \{q_1 \in Q : \exists q_0 \in \mathbf{q} ((q_0, x, q_1) \in \delta)\}$  (en listant tous les triplets  $(q_0, x, q_1) \in \delta$  dont le premier élément est dans  $\mathbf{q}$  et le second élément est  $x$ ),
  - si  $\mathbf{q}'$  n'existe pas déjà comme état de  $A'$ , l'y ajouter, et dans ce cas, l'ajouter à la file/pile, et de plus, si  $\mathbf{q}'$  contient un état final de  $A$ , marquer cet état comme final pour  $A'$ ,
  - et ajouter à  $A'$  la transition  $\delta'(\mathbf{q}, x) = \mathbf{q}'$ .

La file/pile sert à stocker les états de  $A'$  qui ont été créés mais pour lesquels les transitions sortantes n'ont pas encore été calculées. L'algorithme se termine quand la file/pile se vide, ce qui se produit toujours en au plus  $2^{\#Q}$  étapes car chaque  $\mathbf{q} \subseteq Q$  ne peut apparaître qu'une seule fois dans la file/pile.

Il se peut que l'état  $\emptyset$  soit créé : cet état servira effectivement de puits, au sens où on aura  $\delta'(\emptyset, x) = \emptyset$  quel que soit  $x$  (et l'état n'est pas acceptant).

Il arrive souvent que l'automate déterminisé soit plus petit que les  $2^{\#Q}$  états qu'il a dans le pire cas.

**Exemple 4.117.** — déterminisons le NFA  $A$  présenté en ?? : on commence par construire un état  $\{0\}$  pour  $A'$  car le NFA a  $\{0\}$  pour ensemble d'états initiaux ; on a  $\delta'(\{0\}, a) = \{0, 1\}$  car 0 et 1 sont les deux états auxquels on peut arriver dans  $A$  par une transition partant de 0 et étiquetée par  $a$ , tandis que  $\delta'(\{0\}, b) = \{0\}$  ; ensuite,  $\delta'(\{0, 1\}, a) = \{0, 1, 2\}$  car 0, 1, 2 sont les trois états auxquels on peut arriver dans  $A$  par une transition partant de 0 ou 1 et étiquetée par  $a$  ; et ainsi de suite. En procédant ainsi, on construit l'automate à 4 états qui suit :



On remarquera qu'on a construit moins que les  $2^3 = 8$  états qu'on pouvait craindre.

Il est par ailleurs instructif de regarder comment fonctionne l'automate  $A'$  ci-dessus pour déterminer si l'avant-dernière lettre d'un mot est un  $a$  : essentiellement, il retient deux bits d'information, à savoir « est-ce que la dernière lettre consommée était un  $a$  ? » et « est-ce que l'avant-dernière lettre consommée était un  $a$  ? » ; chacune des quatre combinaisons de ces deux bits correspond à un état de  $A'$  : non/non est l'état  $\{0\}$ , oui/non est l'état  $\{0, 1\}$ , non/oui est l'état  $\{0, 2\}$ , et oui/oui est l'état  $\{0, 1, 2\}$ .

**Remarque 4.118.** — On vient donc de voir que les NFA sont équivalents aux DFA en ce sens qu'ils permettent de définir les mêmes langages. Il est néanmoins intéressant de les définir et de les étudier pour plusieurs raisons : d'une part, l'équivalence qu'on vient de voir, c'est-à-dire la déterminisation d'un NFA en DFA, a un coût algorithmique important (exponentiel en nombre d'états dans le pire cas), d'autre part, certaines opérations sur les automates se définissent plus naturellement sur les NFA que sur les DFA, et certaines constructions conduisent naturellement à un NFA. On verra en particulier en ?? que pour transformer une expression régulière en automate, on passera naturellement par un NFA, même si on peut souhaiter le déterminer ensuite.

#### 4.2.2.4. Automates finis non-déterministes à transitions spontanées ( $=\varepsilon$ NFA). —

**Définition 4.119.** — Un *automate fini non-déterministe à transitions spontanées* ou ...à  $\varepsilon$ -transitions, en abrégé  $\varepsilon$ NFA, sur un alphabet  $\Sigma$  est la donnée

- d'un ensemble fini  $Q$  d'états,
- d'un ensemble  $I \subseteq Q$  d'états dits initiaux,
- d'un ensemble  $F \subseteq Q$  d'états dits finaux,
- d'une relation de transition  $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$ .

Autrement dit, on autorise maintenant des transitions étiquetées par le mot vide  $\varepsilon$  plutôt que par une lettre  $x \in \Sigma$  : ces transitions sont dites *spontanées*, ou  $\varepsilon$ -transitions.

**Remarque 4.120.** — Soulignons qu'on ne définit les  $\varepsilon$ -transitions que pour les automates non-déterministes : ou, pour dire les choses autrement, *un automate qui possède des  $\varepsilon$ -transitions est par nature même non-déterministe.*

La représentation graphique des  $\varepsilon$ NFA est évidente (on utilisera le symbole «  $\varepsilon$  » pour étiqueter les transitions spontanées). Un NFA est considéré comme un  $\varepsilon$ NFA particulier pour lequel il n'y a pas de  $\varepsilon$ -transition.

**Remarque 4.121.** — Intuitivement, les transitions spontanées signifient que l'automate a la possibilité de passer spontanément, c'est-à-dire *sans consommer de lettre*, d'un état  $q$  à un état  $q'$ , lorsque ces états sont reliés par une  $\varepsilon$ -transition. (On comprend, du coup, pourquoi un automate à transition spontanée est forcément non-déterministe : ces transitions spontanées ne sont qu'une *possibilité*, ce qui sous-entend le non-déterminisme.)

De façon plus précise, un  $\varepsilon$ NFA accepte un mot  $w$  lorsqu'il existe un chemin orienté conduisant d'un état initial  $q_0$  à un état final  $q_n$  et tel que  $w$  coïncide avec le mot obtenu en lisant dans l'ordre les étiquettes  $t_i$  des différentes arêtes  $q_{i-1} \rightarrow q_i$  de ce chemin, quitte à ignorer les  $\varepsilon$ .

Autrement dit,  $w$  est accepté lorsqu'il existe  $q_0, \dots, q_n \in Q$  et  $t_1, \dots, t_n \in (\Sigma \cup \{\varepsilon\})$  tels que  $q_0 \in I$  et  $q_n \in F$  et  $(q_{i-1}, t_i, q_i) \in \delta$  pour chaque  $1 \leq i \leq n$  et  $w = t_1 \dots t_n$  (*attention* : dans cette écriture,  $t_1, \dots, t_n$  ne sont pas forcément les lettres de  $w$ , certains des  $t_i$  peuvent être le mot vide  $\varepsilon$ , les lettres de  $w$  sont obtenues en ignorant ces symboles).

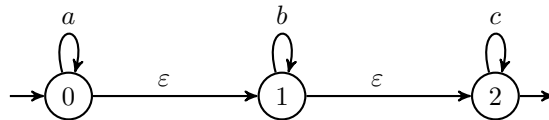
On veut de nouveau définir une relation  $\delta^*$  telle que  $(q, w, q') \in \delta^*$  lorsqu'il existe un chemin orienté conduisant de  $q$  à  $q'$  et tel que le mot  $w$  soit obtenu en lisant dans l'ordre les étiquettes des différentes arêtes de ce chemin.

**Définition 4.122.** — Voici une formalisation possible : si  $A = (Q, I, F, \delta)$  est un  $\varepsilon$ NFA sur l'alphabet  $\Sigma$ , on définit une relation  $\delta^* \subseteq Q \times \Sigma^* \times Q$  par  $(q, w, q') \in \delta^*$  lorsqu'il existe  $q_0, \dots, q_n \in Q$  et  $t_1, \dots, t_n \in (\Sigma \cup \{\varepsilon\})$  tels que  $q_0 = q$  et  $q_n = q'$  et  $(q_{i-1}, t_i, q_i) \in \delta$  pour chaque  $1 \leq i \leq n$ , et enfin  $w = t_1 \dots t_n$ .

Enfin, l'automate  $A$  accepte un mot  $w$  lorsqu'il existe  $q_0 \in I$  et  $q_\infty \in F$  tels que  $(q_0, w, q_\infty) \in \delta^*$ .

**Définition 4.123.** — Le langage accepté  $L(A)$  et l'équivalence de deux automates sont définis de façon analogue aux DFA (cf. ??).

**Exemple 4.124.** — Voici un exemple de  $\varepsilon$ NFA particulièrement simple sur  $\Sigma = \{a, b, c\}$  :



En considérant les différents chemins possibles entre 0 et 2 sur ce graphe, on comprend que le langage qu'il reconnaît est celui des mots sur  $\{a, b, c\}$  formés d'un nombre

quelconque de  $a$  suivi d'un nombre quelconque de  $b$  suivi d'un nombre quelconque de  $c$ , ou, si on préfère, le langage dénoté par l'expression régulière  $a*b*c*$ .

**Définition 4.125.** — Si  $q$  est un état d'un  $\varepsilon$ NFA, on appelle  $\varepsilon$ -fermeture de  $q$  l'ensemble des états  $q'$  (y compris  $q$  lui-même) accessibles depuis  $q$  par une succession quelconque de  $\varepsilon$ -transitions, c'est-à-dire, si on veut,  $\{q' \in Q : (q, \varepsilon, q') \in \delta^*\}$ . On notera temporairement  $C(q)$  cet ensemble. (Par exemple, dans l'exemple ?? ci-dessus, on a  $C(0) = \{0, 1, 2\}$  et  $C(1) = \{1, 2\}$  et  $C(2) = \{2\}$ . Dans tout NFA sans  $\varepsilon$ -transitions, on a  $C(q) = \{q\}$  pour tout état  $q$ .)

Il est clair qu'on peut calculer algorithmiquement  $C(q)$  (par exemple par un parcours en largeur, sur le graphe orienté dont l'ensemble des sommets est  $Q$  et l'ensemble des arêtes est l'ensemble des  $\varepsilon$ -transitions de  $A$  : la  $\varepsilon$ -fermeture  $C(q)$  est simplement l'ensemble des sommets accessibles depuis  $q$  dans ce graphe).

**Proposition 4.126.** — Soit  $A = (Q, I, F, \delta)$  un  $\varepsilon$ NFA sur un alphabet  $\Sigma$ . Alors il existe un NFA  $A^\S = (Q, I^\S, F^\S, \delta^\S)$  (sur le même alphabet  $\Sigma$ ) ayant le même ensemble d'états  $Q$  que  $A$  et qui soit équivalent à  $A$  au sens où il reconnaît le même langage  $L(A^\S) = L(A)$ . De plus,  $A^\S$  se déduit algorithmiquement de  $A$ .

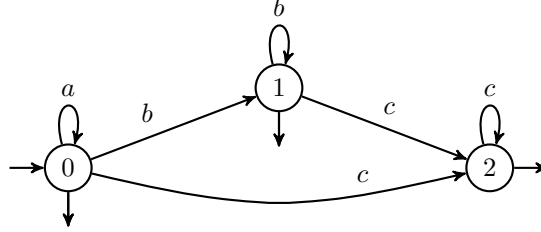
*Démonstration.* — On pose  $I^\S = I$  (mêmes états initiaux). L'idée est maintenant de faire une transition  $(q, x, q') \in \delta^\S$  à chaque fois qu'on peut atteindre  $q'$  à partir de  $q$  dans  $A$  par une suite quelconque de  $\varepsilon$ -transitions suivie d'une unique transition étiquetée par  $x$ , autrement dit,  $(q, \varepsilon, q^\#) \in \delta^*$  (c'est-à-dire  $q^\# \in C(q)$ ) et  $(q^\#, x, q') \in \delta$ .

On définit donc  $\delta^\S \subseteq Q \times \Sigma \times Q$  par  $(q, x, q') \in \delta^\S$  lorsqu'il existe  $q^\# \in C(q)$  tel que  $(q^\#, x, q') \in \delta$  : autrement dit, pour créer les transitions  $q \rightarrow q'$  dans  $A^\S$ , on parcourt tous les  $q^\# \in C(q)$ , et on crée une transition  $q \rightarrow q'$  étiquetée par  $x \in \Sigma$  dans  $A^\S$  lorsqu'il existe une transition  $q^\# \rightarrow q'$  étiquetée par ce  $x$  dans  $A$ . De même, on définit  $F^\S \subseteq Q$  comme l'ensemble des  $q \in Q$  tels que  $C(q) \cap F \neq \emptyset$ , c'est-à-dire, depuis lesquels peut atteindre un état final par une succession de  $\varepsilon$ -transitions.

Si on a un chemin  $q_0 \rightarrow q_1 \rightarrow \dots \rightarrow q_n$  dans  $A$  menant d'un état initial  $q_0 \in I$  à un état final  $q_n \in F$  et étiquetées par  $t_1, \dots, t_n \in (\Sigma \cup \{\varepsilon\})$  (c'est-à-dire  $(q_{i-1}, t_i, q_i) \in \delta$ ), appelons  $j_1 < \dots < j_m$  les indices tels que  $t_{j_i} \in \Sigma$ , autrement dit, tels que la transition  $q_{j_i-1} \rightarrow q_{j_i}$  ne soit pas spontanée, et posons  $j_0 = 0$ . Alors on passe de  $q_{j_{i-1}}$  à  $q_{j_i}$  par une succession de  $\varepsilon$ -transitions (de  $q_{j_{i-1}}$  à  $q_{(j_i)-1}$ ) suivie par une unique transition non spontanée : on a  $q_{(j_i)-1} \in C(q_{j_{i-1}})$  et  $(q_{(j_i)-1}, t_{j_i}, q_{j_i}) \in \delta$ , autrement dit  $(q_{j_{i-1}}, t_{j_i}, q_{j_i}) \in \delta^\S$  ; et comme le mot  $w = t_1 \dots t_n$  s'écrit aussi  $t_{j_1} \dots t_{j_m}$ , on a un chemin reliant  $q_{j_0} = q_0 \in I$  à  $q_{j_m} \in F^\S$  (puisque  $q_n \in C(q_{j_m}) \cap F$ ). Le mot  $w$  supposé accepté par  $A$  est donc accepté par  $A^\S$ . La réciproque est analogue.  $\square$

**Définition 4.127.** — On dit que le NFA  $A^\S$  est obtenu en éliminant les  $\varepsilon$ -transitions dans le  $\varepsilon$ NFA  $A$  lorsqu'il est obtenu par la procédure décrite dans la démonstration de cette proposition, et en supprimant tous les états non-initiaux de  $A^\S$  auxquels n'aboutissent dans  $A$  que des  $\varepsilon$ -transitions (ces états sont devenus inaccessibles dans  $A^\S$ ). Algorithmiquement, il s'agit donc, pour chaque état  $q \in Q$  et chaque  $q^\#$  dans la  $\varepsilon$ -fermeture  $C(q)$  de  $q$ , de créer une transition  $q \rightarrow q'$  étiquetée par  $x$  dans  $A^\S$  pour chaque transition  $q^\# \rightarrow q'$  étiquetée par  $x$  dans  $A$ .

**Exemple 4.128.** — Éliminons les  $\varepsilon$ -transitions du  $\varepsilon$ NFA  $A$  présenté en ?? : comme  $C(0) = \{0, 1, 2\}$ , on fait partir de 0 toutes les transitions partant d'un des états 0, 1, 2 et étiquetées par une lettre, et de même, comme  $C(1) = \{1, 2\}$ , on fait partir de 1 toutes les transitions partant d'un des états 1, 2 et étiquetées par une lettre. On obtient finalement l'automate suivant :



(Sur cet exemple précis, on obtient un automate déterministe incomplet, mais ce n'est pas un phénomène général : en général il faut s'attendre à obtenir un NFA.)

**Remarque 4.129.** — La manière dont on a éliminé les  $\varepsilon$ -transitions ci-dessus consiste à remplacer une succession de  $\varepsilon$ -transitions suivie d'une unique transition étiquetée  $x$  par une transition étiquetée  $x$  (et de même, on modifie les états finaux, mais pas les états initiaux). Il existait un moyen « dual » d'éliminer les  $\varepsilon$ -transitions, à savoir remplacer une unique transition étiquetée  $x$  suivie d'une succession de  $\varepsilon$ -transitions par une transition étiquetée  $x$  (et de même, on modifie les états initiaux, mais pas les états finaux). Autrement dit, la construction  $A \mapsto A^\S$  décrite en 4.126 définit  $(q, x, q') \in \delta^\S$  lorsqu'il existe  $q^\# \in C(q)$  tel que  $(q^\#, x, q') \in \delta$ , et  $I^\S = I$  et  $F^\S$  comme l'ensemble des états  $q$  tels que  $C(q) \cap F \neq \emptyset$  ; la construction « duale »  $A \mapsto A^\P$  consiste à poser  $(q, x, q') \in \delta^\P$  lorsqu'il existe  $q^{b'}$  tel que  $(q, x, q^{b'}) \in \delta$  et  $q' \in C(q^{b'})$ , et  $F^\P = F$  et  $I^\P$  comme la réunion des  $C(q)$  pour tout  $q \in I$ .

Ces deux manières d'éliminer les  $\varepsilon$ -transitions donnent des NFA équivalents. En fait, on peut définir l'une en termes de l'autre en utilisant la définition de l'automate transposé  $A^R$  présentée en ?? plus bas (et qui consiste simplement à inverser le sens de toutes les flèches de l'automate) : si on inverse les flèches, qu'on élimine les  $\varepsilon$ -transitions à la manière décrite en 4.126, et qu'on inverse de nouveau les flèches, on obtient l'élimination « duale », autrement dit,  $A^\P = ((A^R)^\S)^R$ .

L'une ou l'autre manière d'éliminer les  $\varepsilon$ -transitions était possible, mais il vaut mieux ne pas les mélanger. C'est pour cette raison qu'on a fait un choix en 4.126 ; la présente remarque a principalement pour objectif d'expliquer la raison d'une perte de symétrie (notamment entre états initiaux et finaux) dans ce choix.

**Remarque 4.130.** — L'intérêt des  $\varepsilon$ NFA, même s'ils sont finalement équivalents aux NFA, est que certaines constructions sur les automates sont plus simples ou plus transparentes lorsqu'on s'autorise à fabriquer des  $\varepsilon$ NFA (à titre d'exemple, l'automate présenté en ?? est beaucoup plus transparent à lire que le NFA équivalent donné en 4.128). On verra notamment en ?? qu'il est facile (quoique inefficace) de construire un  $\varepsilon$ NFA reconnaissant le langage dénoté par une expression régulière quelconque ; et en ?? que, même si on fabrique directement un NFA, il peut être utile d'introduire

temporairement des transitions spontanées dans la construction de ce NFA, quitte à les éliminer immédiatement (cela simplifie la description).

**4.2.3. Identité entre les langages rationnels et reconnaissables.** — Notre objectif est de montrer que la classe des langages rationnels et la classe des langages reconnaissable coïncident.

**4.2.3.1. Compilation d'une expression régulière en un automate : construction de Glushkov.** — Nous allons maintenant montrer que la classe des langages reconnaissables contient la classe des langages rationnels. Pour cela, nous devons montrer qu'elle contient les constructions de base et qu'elle est stable par les opérations rationnelles (union, concaténation et étoile de Kleene).

Pour établir ces stabilités, on va travailler sur les NFA et utiliser la construction parfois appelée « de Glushkov » ou « automate standard » ; ceci fournira un « automate de Glushkov » pour chaque expression régulière  $r$ . Cette construction travaille, en fait, sur des NFA vérifiant une propriété supplémentaire facile à assurer, et on va commencer par un lemme dans ce sens :

**Définition 4.131.** — On dit qu'un NFA est *standard* s'il ne possède qu'un unique état initial qui n'est la cible d'aucune transition.

**Lemme 4.132.** — Soit  $A$  un NFA. Alors il existe un NFA  $A'$  (sur le même alphabet  $\Sigma$ ) qui soit équivalent à  $A$  et qui possède la propriété supplémentaire d'avoir un unique état initial  $q_0$  et qu'aucune transition n'aboutit à  $q_0$ . De plus,  $A'$  se déduit algorithmiquement de  $A$  et a au plus un état de plus que  $A$ .

L'affirmation du lemme est donc que tout NFA est équivalent à un NFA *standard* qui s'en déduit algorithmiquement par l'ajout d'au plus un état.)

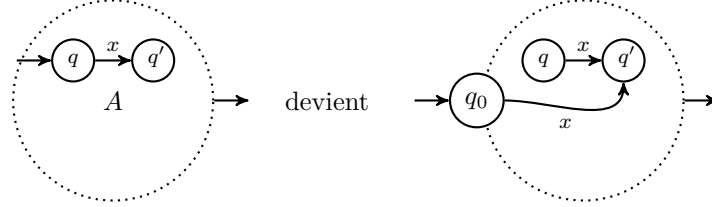
*Démonstration.* — On fabrique  $A'$  en reprenant le même ensemble d'états  $Q$  que dans  $A$  auquel on ajoute un unique nouvel état  $q_0$  qui sera le seul état initial de  $A'$  ; pour chaque transition partant d'un état initial de  $A$ , on ajoute dans  $A'$  une transition identiquement étiquetée, et de même cible, partant de  $q_0$ .

Formellement : soit  $A = (Q, I, F, \delta)$ . On définit alors  $A' = (Q', \{q_0\}, F', \delta')$  de la manière suivante :  $Q' = Q \uplus \{q_0\}$  (où  $\uplus$  désigne une réunion disjointe<sup>(6)</sup>),  $\delta'$  est la réunion de l'ensemble des transitions  $(q, x, q')$  qui étaient déjà dans  $\delta$  et de l'ensemble des  $(q_0, x, q')$  telles qu'il existe une transition  $(q, x, q') \in \delta$  avec  $q \in I$ , et enfin  $F' = F$  ou  $F' = F \cup \{q_0\}$  selon que  $I \cap F = \emptyset$  ou  $I \cap F \neq \emptyset$  respectivement (i.e.,  $q_0$  est final lorsqu'il existait déjà un état à la fois initial et final).

La figure suivante illustre la transformation en question :

6. C'est-à-dire qu'on exige que  $q_0$  n'appartienne pas à  $Q$  (c'est un nouvel état).



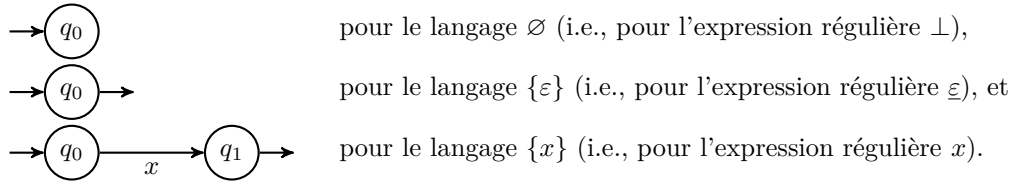


□

**Remarque 4.133.** — De façon équivalente, on peut fabriquer  $A'$  en ajoutant d'abord un unique état initial  $q_0$  et des  $\varepsilon$ -transitions de  $q_0$  vers chacun des états qui étaient initiaux dans  $A$ , puis en éliminant les  $\varepsilon$ -transitions qu'on vient d'ajouter (cf. 4.126). Cela donne le même résultat que ce qui vient d'être dit.

Commençons par fixer des NFA (« standards » au sens de 4.132) reconnaissant les langages de base triviaux  $\emptyset$ ,  $\{\varepsilon\}$  et  $\{x\}$  (pour chaque  $x \in \Sigma$ ), c'est-à-dire ceux dénotés par les expressions régulières  $\perp$ ,  $\underline{\varepsilon}$  et  $x$  respectivement.

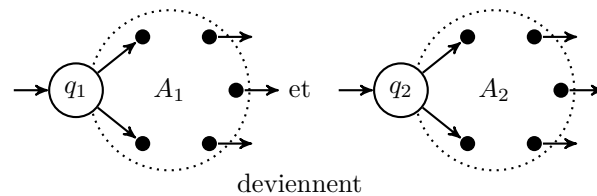
**Définition 4.134.** — On prendra les suivants :

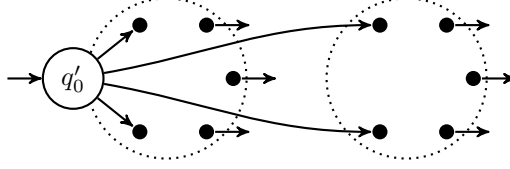


**Proposition 4.135.** — Si  $L_1, L_2$  sont des langages reconnaissables (sur un même alphabet  $\Sigma$ ), alors la réunion  $L_1 \cup L_2$  est reconnaissable ; de plus, un NFA la reconnaissant se déduit algorithmiquement de NFA reconnaissant  $L_1$  et  $L_2$ .

*Démonstration.* — Par hypothèse, il existe des NFA reconnaissant  $L_1$  et  $L_2$  : d'après 4.132, on peut supposer qu'ils sont *standards* en ce sens qu'ils ont un unique état initial qui n'est la cible d'aucune transition. Disons que  $A_1 = (Q_1, \{q_1\}, F_1, \delta_1)$  et  $A_2 = (Q_2, \{q_2\}, F_2, \delta_2)$  sont des NFA standards tels que  $L_1 = L(A_1)$  et  $L_2 = L(A_2)$ .

L'automate  $A'$  s'obtient réunissant  $A_1$  et  $A_2$  mais en « fusionnant » les états initiaux  $q_1$  et  $q_2$  de  $A_1$  et  $A_2$  en un unique état initial  $q'_0$ , d'où partent les mêmes transitions (avec les mêmes étiquettes) que depuis l'un ou l'autre de  $q_1$  ou  $q_2$ . Graphiquement :





De façon plus formelle, considérons un nouvel ensemble d'états  $Q' = (Q_1 \uplus Q_2) \setminus \{q_1, q_2\} \uplus \{q'_0\}$  où  $\uplus$  désigne la réunion disjointe (autrement dit, on prend la réunion disjointe des états non-initiaux de  $A_1$  et  $A_2$  et on ajoute un nouvel état  $q'_0$ ), et la fonction  $\varphi_1: Q_1 \rightarrow Q'$  qui envoie  $q_1$  sur  $q'_0$  et tout autre état de  $Q_1$  sur lui-même, et  $\varphi_2: Q_2 \rightarrow Q'$  de façon analogue. On définit alors l'automate  $A'$  dont l'ensemble d'états est  $Q'$ , l'état initial est  $q'_0$ , les états finaux  $F' = \varphi_1(F_1) \cup \varphi_2(F_2)$ , et la relation de transition  $\delta'$  est formée des triplets  $(\varphi_1(q), x, q')$  où  $(q, x, q') \in \delta_1$  et des  $(\varphi_2(q), x, q')$  où  $(q, x, q') \in \delta_2$ .

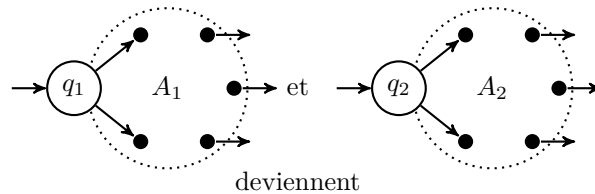
Il est alors clair qu'un chemin de l'état initial à un état final dans cet automate  $A'$  consiste soit en un chemin d'un état initial à un état final dans  $A_1$  soit en un tel chemin dans  $A_2$ . On a donc bien  $L(A') = L_1 \cup L_2$ .  $\square$

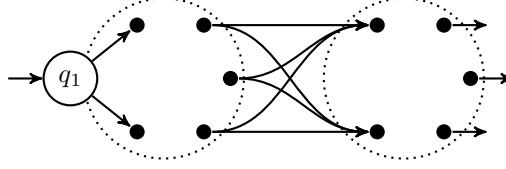
**Remarque 4.136.** — De façon équivalente, on peut fabriquer  $A'$  en ajoutant d'abord un unique état initial  $q'_0$  à la réunion disjointe de  $A_1$  et  $A_2$  et des  $\varepsilon$ -transitions de  $q'_0$  vers  $q_1$  et  $q_2$  (qui cessent d'être initiaux), puis en éliminant les  $\varepsilon$ -transitions qu'on vient d'ajouter (cf. 4.126) ainsi que les états  $q_1$  et  $q_2$  devenus inutiles. Cela donne le même résultat que ce qui vient d'être dit.

**Proposition 4.137.** — Si  $L_1, L_2$  sont des langages reconnaissables (sur un même alphabet  $\Sigma$ ), alors la concaténation  $L_1 L_2$  est reconnaissable ; de plus, un NFA la reconnaissant se déduit algorithmiquement de NFA reconnaissant  $L_1$  et  $L_2$ .

*Démonstration.* — Par hypothèse, il existe des NFA reconnaissant  $L_1$  et  $L_2$  : d'après 4.132, on peut supposer qu'ils sont *standards* en ce sens qu'ils ont un unique état initial qui n'est la cible d'aucune transition. Disons que  $A_1 = (Q_1, \{q_1\}, F_1, \delta_1)$  et  $A_2 = (Q_2, \{q_2\}, F_2, \delta_2)$  sont des NFA standards tels que  $L_1 = L(A_1)$  et  $L_2 = L(A_2)$ .

L'automate  $A'$  s'obtient en réunissant  $A_1$  et  $A_2$ , en ne gardant que les états finaux de  $A_2$ , en supprimant  $q_2$  et en remplaçant chaque transition sortant de  $q_2$  par une transition identiquement étiquetée, et de même cible, partant de *chaque* état final de  $A_1$  (ces derniers seront marqués finaux si  $q_2$  était final dans  $A_2$ ). Graphiquement :





De façon plus formelle, considérons un nouvel ensemble d'états  $Q' = (Q_1 \uplus Q_2) \setminus \{q_2\}$  où  $\uplus$  désigne la réunion disjointe. On définit alors l'automate  $A'$  dont l'ensemble d'états est  $Q'$ , l'état initial est  $q_1$ , l'ensemble  $F'$  des états finaux est  $F_2$  si  $q_2$  n'était pas final dans  $A_2$  et  $F_1 \cup F_2$  si  $q_2$  était final dans  $A_2$ , la relation de transition  $\delta'$  est la réunion de  $\delta_1$ , de l'ensemble des triplets  $(q, x, q') \in \delta_2$  tels que  $q \neq q_2$ , et enfin de l'ensemble des triplets  $(q, x, q')$  tels que  $(q_2, x, q') \in \delta_2$  et que  $q \in F_1$ .

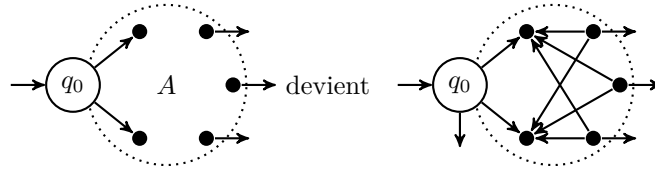
Il est alors clair qu'un chemin de l'état initial  $q_1$  à un état final dans cet automate  $A'$  consiste en un chemin de  $q_1$  à un état final dans  $A_1$  suivi d'un chemin de  $q_2$  à un état final dans  $A_2$  (moins  $q_2$  lui-même). On a donc bien  $L(A') = L_1 L_2$ .  $\square$

**Remarque 4.138.** — De façon équivalente, on peut fabriquer  $A'$  en ajoutant d'abord à la réunion disjointe de  $A_1$  et  $A_2$  une  $\varepsilon$ -transition de chaque état final de  $A_1$  vers  $q_2$  (qui cesse d'être initial), puis en éliminant les  $\varepsilon$ -transitions qu'on vient d'ajouter (cf. 4.126) ainsi que l'état  $q_2$  devenu inutile. Cela donne le même résultat que ce qui vient d'être dit.

**Proposition 4.139.** — Si  $L$  est un langage reconnaissable (sur un alphabet  $\Sigma$ ), alors l'étoile de Kleene  $L^*$  est reconnaissable ; de plus, un NFA la reconnaissant se déduit algorithmiquement de NFA reconnaissant  $L$ .

*Démonstration.* — Par hypothèse, il existe un NFA reconnaissant  $L$  : d'après 4.132, on peut supposer qu'ils est *standard* en ce sens qu'il a un unique état initial qui n'est la cible d'aucune transition. Disons que  $A = (Q, \{q_0\}, F, \delta)$  est un NFA standard tel que  $L = L(A)$ .

L'automate  $A'$  s'obtient en ajoutant à  $A$ , pour chaque transition sortant de  $q_0$ , une transition identiquement étiquetée, et de même cible, partant de chaque état final de  $A$ , et en rendant  $q_0$  final s'il ne l'était pas déjà :



De façon plus formelle, on considère l'automate  $A'$  dont l'ensemble d'états est  $Q' := Q$ , l'état initial est  $q_0$ , les états finaux  $F' := F \cup \{q_0\}$ , et la relation de transition  $\delta'$  est la réunion de  $\delta$  et de l'ensemble des triplets  $(q, x, q')$  tels que  $(q_0, x, q') \in \delta$  et que  $q \in F$ .

Il est alors clair qu'un chemin de l'état initial  $q_0$  à un état final dans cet automate  $A'$  consiste en un nombre quelconque (éventuellement nul) de chemins de l'état initial à un état final dans  $A$ . On a donc bien  $L(A') = L^*$ .  $\square$

**Remarque 4.140.** — De façon équivalente, on peut fabriquer  $A'$  en ajoutant d'abord à  $A$  une  $\varepsilon$ -transition de chaque état final de  $A$  vers  $q_0$ , puis en éliminant les  $\varepsilon$ -transitions qu'on vient d'ajouter (cf. 4.126), et enfin en marquant  $q_0$  comme final. Cela donne le même résultat que ce qui vient d'être dit.

Récapitulons le contenu essentiel de ce que nous avons montré :

**Corollaire 4.141.** — *Tout langage rationnel est reconnaissable ; de plus, un NFA le reconnaissant se déduit algorithmiquement d'une expression régulière le dénotant.*

*En particulier, il existe un algorithme qui, donnée une expression régulière  $r$  (sur un alphabet  $\Sigma$ ) et un mot  $w \in \Sigma^*$ , décide si  $w \in L(r)$ , c'est-à-dire si  $w$  vérifie  $r$ .*

*Démonstration.* — L'affirmation du premier paragraphe se démontre par une preuve par induction structurelle dont nous avons vu les cas de base et les cas d'induction dans les propositions 4.135, 4.137 et 4.139.

Pour décider si un mot  $w$  vérifie une expression régulière  $r$ , on commence par transformer cette expression régulière en NFA comme on vient de l'expliquer (c'est-à-dire construire un NFA  $A(r)$  reconnaissant le langage  $L(r)$  dénoté par  $r$ ), puis on détermine cet automate (cf. 4.115), après quoi il est facile de tester si le DFA résultant de la détermination accepte le mot  $w$  considéré (il suffit de suivre l'unique chemin dans l'automate partant de l'état initial et étiqueté par  $w$ , et de voir si l'état auquel il aboutit est final).  $\square$

Les constructions que nous avons décrites dans cette section associent naturellement un NFA standard à chaque expression régulière : il s'obtient en partant des automates de base décrits en 4.134 et en appliquant les constructions décrites dans les démonstrations de 4.135, 4.137 et 4.139.

Plus exactement, on associe à chaque expression régulière  $r$  (sur un alphabet  $\Sigma$  fixé) un automate  $A(r)$  (ou  $A_{\text{Glushkov}}(r)$ ) standard, appelé *automate de Glushkov*, qui reconnaît le langage  $L(r)$  dénoté par  $r$ , de la manière suivante (par induction sur la complexité de l'expression régulière telle que définie en ??) :

- les automates de Glushkov de  $\perp$ ,  $\emptyset$  et  $x$  (pour  $x \in \Sigma$ ) sont définis comme ceux décrits et illustrés en 4.134,
- connaissant les automates de Glushkov  $A_1$  et  $A_2$  de  $r_1$  et  $r_2$  respectivement, celui de  $(r_1|r_2)$  est celui décrit et illustré dans la démonstration de 4.135,
- connaissant les automates de Glushkov  $A_1$  et  $A_2$  de  $r_1$  et  $r_2$  respectivement, celui de  $r_1r_2$  est celui décrit et illustré dans la démonstration de 4.137,
- connaissant l'automate de Glushkov  $A$  de  $r$ , celui de  $(r)^*$  est celui décrit et illustré dans la démonstration de 4.139.

Cet automate de Glushkov  $A_{\text{Glushkov}}(r)$  possède les propriétés suivantes :

- c'est un NFA reconnaissant le langage  $L(r)$  dénoté par l'expression régulière  $r$  dont on est parti,
- il est standard au sens de 4.132, c'est-à-dire qu'il possède un unique état initial auquel n'aboutit aucune transition,

- les transitions aboutissant à n'importe quel état donné sont toutes étiquetées par la même lettre,
- son nombre d'états est égal à 1 plus le nombre de lettres (à l'exclusion des métacaractères) contenues dans l'expression  $r$ .

Les deux dernières propriétés se vérifient inductivement, c'est-à-dire qu'on observe qu'elles sont satisfaites sur les automates de base décrits en 4.134 qu'elles sont préservées par les constructions de 4.135, 4.137 et 4.139. En fait, on peut être un peu plus précis : chaque état, autre que l'état initial, de l'automate de Glushkov associé à l'expression régulière  $r$  correspond à une lettre  $x$  (à l'exclusion des métacaractères) de  $r$ , l'état en question provient de l'état  $q_1$  de l'automate décrit en 4.134 pour le langage  $\{x\}$ , et toutes les transitions menant à cet état sont étiquetées par  $x$ .

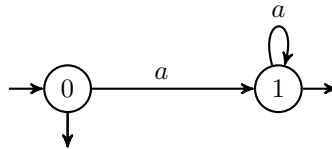
Ces observations sont utiles pour détecter des erreurs lors de la construction de l'automate.

**Exemple 4.142.** — Construisons l'automate qu'elle associe à l'expression régulière  $((a^*|b)c)^*$  sur l'alphabet  $\Sigma = \{a, b, c\}$ . On doit obtenir un automate ayant 4 états.

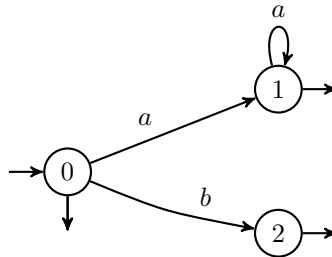
Nous démarrons par l'arbre de dérivation permettant de construire l'expression régulière  $((a^*|b)c)^*$ .

$$\begin{array}{c}
 \frac{}{a} \text{ (Symbole)} \quad \frac{}{b} \text{ (Symbole)} \\
 \frac{}{a^*} \text{ (étoile)} \quad \frac{}{b} \text{ (Symbole)} \\
 \hline
 a^* | b \text{ (union)} \quad \frac{}{c} \text{ (Symbole)} \\
 \hline
 (a^* | b) c \text{ (concat)} \\
 \hline
 ((a^* | b) c)^* \text{ (étoile)}
 \end{array}$$

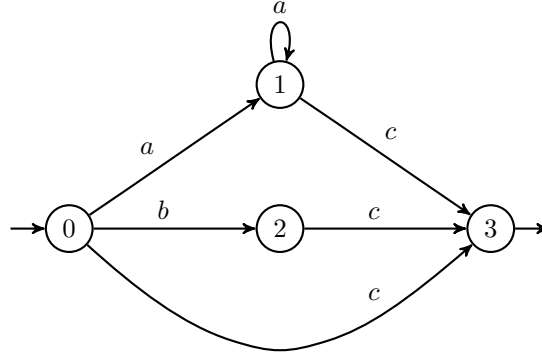
L'automate de Glushkov de  $a^*$  est le suivant, obtenu en appliquant la construction de 4.139 à l'automate trivial pour le langage  $\{a\}$  (donné en 4.134) :



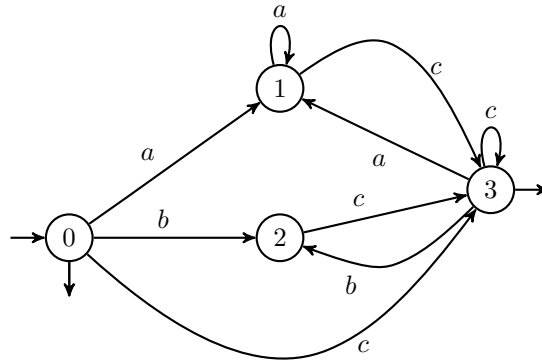
On en déduit au moyen de 4.135 l'automate suivant pour  $a^*|b$  :



On en déduit au moyen de 4.137 l'automate suivant pour  $(a^*|b)c$  :



Et enfin, de nouveau par 4.139 l'automate suivant pour  $((a^*|b)c)^*$  :



Toutes les transitions aboutissant à l'état 1 sont étiquetées  $a$ , toutes celles aboutissant à l'état 2 sont étiquetées  $b$ , et toutes celles aboutissant à 3 sont étiquetées  $c$ .

**4.2.3.2. Automates à transitions étiquetées par des expressions régulières (=RNFA), algorithme d'élimination des états.** — On cherche dans cette section à montrer la réciproque de 4.141, c'est-à-dire, que les langages rationnels sont reconnaissables. On va pour cela donner un algorithme (très coûteux !) qui transforme un automate en expression régulière (dénnotant le langage qu'il reconnaît). Cette algorithme « d'élimination des états » fonctionne naturellement sur une sorte d'automate encore plus générale que tous ceux que nous avons définis jusqu'à présent : on va donc commencer par définir ces automates, même si leur intérêt réside presque uniquement en la preuve de 4.2.2 ci-dessous.

**Définition 4.143.** — Un *automate fini (non-déterministe) à transitions étiquetées par des expressions régulières*, en abrégé *RNFA*, sur un alphabet  $\Sigma$  est la donnée

- d'un ensemble fini  $Q$  d'états,
- d'un ensemble  $I \subseteq Q$  d'états dits initiaux,
- d'un ensemble  $F \subseteq Q$  d'états dits finaux,

- d'un ensemble *fini* de transitions  $\delta \subseteq Q \times (\text{regexp}(\Sigma)) \times Q$  où  $(\text{regexp}(\Sigma))$  désigne l'ensemble des expressions régulières sur  $\Sigma$ .

Autrement dit, on autorise maintenant des transitions étiquetées par des expressions régulières quelconques sur  $\Sigma$ . Remarquons qu'on doit maintenant demander *explicitement* que l'ensemble  $\delta$  des transitions permises soit fini car l'ensemble  $Q \times (\text{regexp}(\Sigma)) \times Q$ , lui, ne l'est pas.

**Définition 4.144.** — Pour un tel automate, on définit une relation  $\delta^* \subseteq Q \times \Sigma^* \times Q$  par  $(q, w, q') \in \delta^*$  lorsqu'il existe  $q_0, \dots, q_n \in Q$  et  $r_1, \dots, r_n \in \text{regexp}(\Sigma)$  tels que  $q_0 = q$  et  $q_n = q'$  et  $(q_{i-1}, r_i, q_i) \in \delta$  pour chaque  $1 \leq i \leq n$ , et enfin  $w \in L(r_1 \cdots r_n)$ .

Concrètement,  $(q, w, q') \in \delta^*$  signifie que le RNFA peut passer de l'état  $q$  à l'état  $q'$  en effectuant des transitions  $(q_0 \rightarrow q_1 \rightarrow \cdots \rightarrow q_n)$  étiquetées par  $r_1, \dots, r_n \in \text{regexp}(\Sigma)$  et en consommant le mot  $w$  au sens où ce dernier se décompose comme concaténation d'autant de facteurs que de transitions ( $w = v_1 \cdots v_n$ ), chacun vérifiant l'expression régulière qui étiquette la transition (soit  $v_i \in L(r_i)$ ).

**Définitions 4.145.** — Enfin, l'automate  $A$  accepte un mot  $w$  lorsqu'il existe  $q_0 \in I$  et  $q_\infty \in F$  tels que  $(q_0, w, q_\infty) \in \delta^*$ .

Le langage accepté  $L(A)$  et l'équivalence de deux automates sont définis de façon analogue aux DFA (cf. ??).

**Remarque 4.146.** — Un  $\varepsilon$ NFA (ou *a fortiori* un NFA, DFAi ou DFA) est considéré comme un RNFA particulier dont les transitions sont étiquetées soit par une unique lettre (considérée comme expression régulière) soit par le symbole  $\varepsilon$  (dénotant le langage  $\{\varepsilon\}$ ) dans le cas des transitions spontanées.

Une expression régulière  $r$  peut aussi être considérée comme un RNFA particulier comportant un unique état initial, un unique état final, et une unique transition de l'un vers l'autre, étiquetée par l'expression  $r$  elle-même. Il est évident que ce RNFA reconnaît exactement le langage dénoté par  $r$ .

La représentation graphique des RNFA ne pose pas de problème particulier (voir en ?? pour différents exemples).

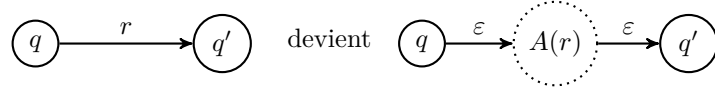
On peut toujours modifier un RNFA de manière à ce qu'il y ait au plus une, ou même si on le souhaite, exactement une, transition entre deux états  $q$  et  $q'$  donnés. En effet, s'il existe plusieurs transitions  $(q, r_1, q'), \dots, (q, r_k, q') \in \delta$  possibles entre  $q$  et  $q'$ , on peut les remplacer par une unique transition  $(q, (r_1 | \cdots | r_k), q')$ , cela ne change visiblement rien au fonctionnement de l'automate (et notamment pas le langage reconnu). S'il n'y a *aucune* transition de  $q$  vers  $q'$ , on peut toujours choisir d'en ajouter une  $(q, \perp, q')$  (qui ne peut pas être empruntée!) si c'est commode.

Comme les  $\varepsilon$ NFA, les NFA et les DFAi avant eux, les RNFA peuvent se ramener aux automates précédemment définis :

**Proposition 4.2.1.** — Soit  $A = (Q, I, F, \delta)$  un RNFA sur un alphabet  $\Sigma$ . Alors il existe un  $\varepsilon$ NFA  $A' = (Q', I', F', \delta')$  (sur le même alphabet  $\Sigma$ ) et qui soit équivalent

à  $A$  au sens où il reconnaît le même langage  $L(A') = L(A)$ . De plus,  $A'$  se déduit algorithmiquement de  $A$ .

*Démonstration.* — On a vu en 4.141 que pour chaque expression régulière  $r$  on peut trouver (algorithmiquement) un  $\varepsilon$ NFA  $A(r)$  (par exemple l'automate de Glushkov ou l'automate de Thompson) qui reconnaît le langage dénoté par  $r$ . On peut donc construire  $A'$  en remplaçant chaque transition  $(q, r, q')$  de  $A$  par une copie de l'automate  $A(r)$  placée entre les états  $q$  et  $q'$ . Symboliquement :



Plus précisément, si  $\{(q_j, r_j, q'_j) : 1 \leq j \leq M\}$  est une énumération de  $\delta$ , on construit  $A'$  en lui donnant pour ensemble d'états  $Q \uplus \biguplus_{j=1}^M Q_{r_j}$  où  $Q_{r_j}$  est l'ensemble d'états de l'automate  $A(r_j)$  construit pour reconnaître  $r_j$ , les ensembles d'états initiaux et finaux sont  $I' = I$  et  $F' = F$  comme dans  $A$ , et la relation de transition  $\delta'$  est la réunion de chacune  $\delta_{r_j}$  de celle des  $\varepsilon$ NFA  $A(r_j)$  à quoi on ajoute encore des transitions spontanées  $(q_j, \varepsilon, q^\sharp)$  pour tout état initial  $q^\sharp \in I_{r_j}$  de  $A(r_j)$  et des transitions spontanées  $(q^\flat, \varepsilon, q'_j)$  pour tout état final  $q^\flat \in F_{r_j}$  de  $A(r_j)$ . Il est clair que faire un chemin dans  $A'$  revient à un faire un chemin dans  $A$  où, à chaque fois qu'on fait la transition  $q_j \rightarrow q'_j$  étiquetée par  $r_j$ , on la remplace par un chemin  $q_j \rightarrow q^\sharp \rightarrow \dots \rightarrow q^\flat \rightarrow q'_j$  formé d'une transition spontanée vers un état initial de  $A(r_j)$  suivi d'un chemin dans ce dernier, suivi d'une transition spontanée depuis un état final de  $A(r_j)$ .  $\square$

Mais la surprise des RNFA est qu'ils peuvent aussi se ramener à des expressions régulières !

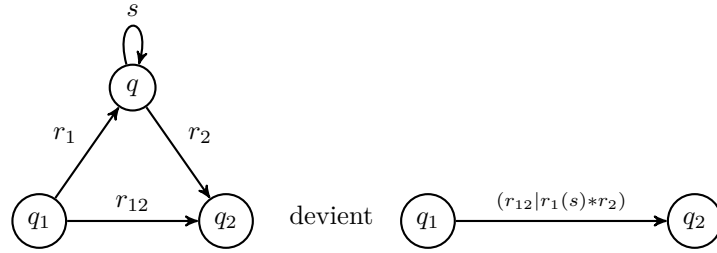
**Proposition 4.2.2.** — Soit  $A = (Q, I, F, \delta)$  un RNFA (ou, en particulier, un NFA ou DFA(i)) sur un alphabet  $\Sigma$ . Alors il existe une expression régulière  $r$  sur  $\Sigma$  qui dénote le langage reconnu par  $A$ , soit  $L(r) = L(A)$ . De plus,  $r$  se déduit algorithmiquement de  $A$ .

*Démonstration.* — On a expliqué qu'on pouvait considérer une expression régulière comme un RNFA ayant un unique état initial, un unique état final, et une unique transition de l'un vers l'autre (étiquetée par l'expression régulière en question). On va montrer que  $A$  est équivalent à un RNFA de cette nature, ce qui montrera bien qu'il est équivalent à une expression régulière.

Remarquons tout d'abord qu'on peut supposer que  $A$  a un unique état initial  $q_0$ , qui ne soit pas final, et qui n'ait aucune transition qui y aboutisse (si ce n'est pas le cas, il suffit de créer un nouvel état  $q_0$ , d'en faire le seul état initial, et de le munir de transitions spontanées c'est-à-dire étiquetées par  $\varepsilon$  vers tous les états précédemment initiaux). De même (symétriquement), on peut supposer que  $A$  a un unique état final  $q_\infty$ , qui ne soit pas initial, et sans aucune transition qui en part. On fera l'hypothèse que  $A$  a ces propriétés, et on s'arrangera pour les préserver dans ce qui suit.



Soient maintenant  $q$  un état de  $A$  qui n'est ni l'état initial  $q_0$  ni l'état final  $q_\infty$ . On va montrer qu'on peut *éliminer*  $q$ , c'est-à-dire, quitte à ajouter des transitions, remplacer  $A$  par un automate équivalent  $A'$  qui n'a pas cet état. Pour cela, soient  $q_1, q_2$  deux états quelconques de  $A$ , autres que  $q$  mais possiblement égaux entre eux, où  $q_1$  peut être l'état initial (mais pas l'état final) et  $q_2$  peut être l'état final (mais pas l'état initial). On a vu en 4.2.3.2 qu'on pouvait supposer qu'il existait une unique transition  $(q_1, r_{12}, q_2)$  et de même  $(q_1, r_1, q)$  et  $(q, r_2, q_2)$  et  $(q, s, q)$  (transition de  $q$  vers lui-même). En même temps qu'on élimine  $q$ , on met dans  $A'$  la transition  $(r_{12}|r_1(s)*r_2)$  entre  $q_1$  et  $q_2$ . Symboliquement :



Pour éliminer l'état  $q$ , cette transformation doit être effectuée *simultanément pour toute paire*  $(q_1, q_2)$  d'états de  $A$  (avec  $q_1 \notin \{q, q_\infty\}$  et  $q_2 \notin \{q, q_0\}$ ) pour laquelle il existe une transition de  $q_1$  vers  $q$  et une transition de  $q$  vers  $q_2$ , *y compris* s'il n'y avait pas déjà une transition de  $q_1$  vers  $q_2$ , et *y compris* si  $q_1 = q_2$  : pour *chaque* telle paire, on remplace l'étiquette de la transition  $r_{12}$  entre eux par  $(r_{12}|r_1(s)*r_2)$ . (On prendra garde aux cas particuliers suivants : si la transition de  $q$  vers lui-même n'existait pas, ce qui revient à prendre  $s = \perp$ , alors on remplace la transition  $r_{12}$  par  $(r_{12}|r_1r_2)$  en vertu du fait que  $(\perp)*$  est équivalente à  $\varepsilon$  ; et si la transition de  $q_1$  vers  $q_2$  n'existait pas, ce qui revient à prendre  $r_{12} = \perp$ , alors on la crée avec l'étiquette  $r_1(s)*r_2$  ; et bien sûr, en combinant ces deux cas particuliers, s'il n'y avait ni transition de  $q$  vers lui-même ni de  $q_1$  vers  $q_2$ , on crée cette dernière avec l'étiquette  $r_1r_2$ .)

La transformation qui vient d'être décrite ne change pas le fonctionnement de l'automate, car tout chemin dans  $A$  peut être remplacé par un chemin dans  $A'$  en effaçant simplement les  $q$  (si on considère  $q_1$  et  $q_2$  les états avant un bloc de  $q$  dans le chemin, on voit que le chemin  $q_1 \rightarrow q \rightarrow q \rightarrow \dots \rightarrow q \rightarrow q_2$  peut se transformer en  $q_1 \rightarrow q_2$  en consommant un mot qui vérifie l'expression régulière  $(r_{12}|r_1(s)*r_2)$ ).

En éliminant (successivement dans n'importe quel ordre) tous les états autres que  $q_0$  et  $q_\infty$ , on aboutit ainsi à un automate ayant une unique transition  $(q_0, r, q_\infty)$ , qui est donc essentiellement l'expression régulière  $r$ .  $\square$

La procédure qu'on a décrite dans la démonstration de cette proposition s'appelle l'algorithme d'*élimination des états* ou *algorithme de Kleene*<sup>(7)</sup>.

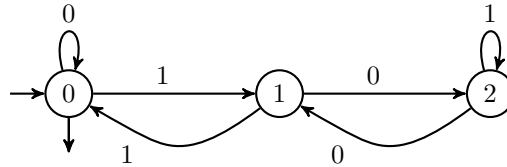
Il va de soi qu'on peut la simplifier un petit peu : s'il n'y a pas de transition de  $q_1$  vers  $q$  ou qu'il n'y en a pas de  $q$  vers  $q_2$  (c'est-à-dire que soit  $r_1$  soit  $r_2$  doit être

7. Peut-être abusivement (le théorème 4.149 est indubitablement dû à Kleene, mais le contenu algorithmique ne l'est peut-être pas) ; il est peut-être plus correct d'attribuer l'algorithme à Brzozowski et McCluskey.

considéré comme valant  $\perp$ ), on ne touche simplement pas à  $r_{12}$  (et si la transition de  $q_1$  vers  $q_2$  n'existait pas non plus, il n'y a pas besoin de la créer); de même, s'il n'y a pas de transition de  $q$  vers lui-même, on ignore la partie  $s^*$ . En revanche, il faut bien penser à créer une transition de  $q_1$  vers  $q_2$ , même si elle n'existait pas au départ, lorsqu'on peut arriver de l'un vers l'autre en passant par  $q$ . Et il faut se souvenir que le cas  $q_2 = q_1$  est à traiter aussi.

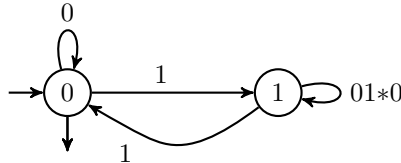
En général, l'élimination des états conduit à une expression extrêmement compliquée (exponentielle dans le nombre d'états de l'automate, au moins dans le pire cas, mais aussi dans beaucoup de cas « typiques »).

**Exemple 4.147.** — Considérons le DFA suivant sur l'alphabet  $\{0, 1\}$ , qui reconnaît les suites binaires qui représentent un nombre multiple de 3 écrit en binaire<sup>(8)</sup> (en convenant que le mot vide est une représentation binaire du nombre 0, ce qui est logique) :



On commence par ajouter un état initial  $q_0$  et un état final  $q_\infty$ , avec des  $\varepsilon$ -transitions  $q_0 \rightarrow 0$  et  $0 \rightarrow q_\infty$ . Pour gagner de la place, nous ne figurerons pas ces deux états sur les dessins qui suivent, mais il faut s'imaginer qu'ils sont toujours là.

L'élimination de l'état 2 conduit à l'automate suivant :

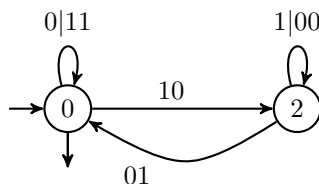


(Répétons qu'on n'a pas figuré l'état initial  $q_0$  ni l'état final  $q_\infty$  : les flèches vers et depuis l'état 0 doivent se comprendre comme des  $\varepsilon$ -transitions  $q_0 \rightarrow 0$  et  $0 \rightarrow q_\infty$ .)

L'élimination de l'état 1 conduit alors à l'automate ayant un unique état 0, avec une transition vers lui-même étiquetée  $0|1(01^*0)^*1$ . Enfin, en éliminant l'état 0, il ne reste qu'une transition de l'état initial vers l'état final, étiquetée par  $(0|1(01^*0)^*1)^*$  : le langage reconnu par l'automate de départ est donc celui dénoté par l'expression régulière  $(0|1(01^*0)^*1)^*$ .

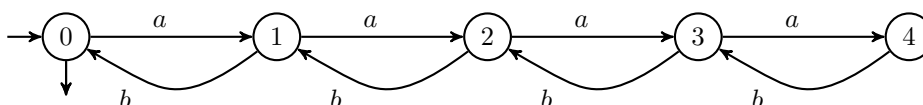
On pouvait aussi choisir d'éliminer l'état 1 en premier, ce qui conduit à l'automate suivant :

8. Les états 0, 1, 2 représentent respectivement les états dans lesquels le nombre binaire lu jusqu'à présent par l'automate est congru à 0, 1, 2 modulo 3.



et finalement à l'expression régulière  $(0|11|10(1|00)*01)^*$ , qui est équivalente à la précédente.

**Exemple 4.148.** — Donnons encore l'exemple du DFAi suivant :



Il s'agit d'un automate « compteur limité », qui ne sait compter que de 0 à 4, incrémentant son compteur quand il reçoit un  $a$  et le décrémentant quand il reçoit un  $b$  (et cessant de fonctionner si le compteur passe au-dessus du maximum ou en-dessous du minimum), et qui accepte finalement les mots dont le nombre de  $b$  égale le nombre de  $a$  sans qu'il y ait jamais eu plus de  $b$  que de  $a$  ni plus de quatre  $a$  de plus que de  $b$ . (On peut dire aussi qu'il s'agit d'une *approximation* du langage des expressions bien-parenthésées, où  $a$  joue le rôle de parenthèse ouvrante et  $b$  de parenthèse fermante ; l'approximation est due au fait qu'on n'accepte que quatre niveaux d'imbrication des « parenthèses ».)

Si on élimine les états dans l'ordre 4, 3, 2, 1, 0, on montre que le langage reconnu par cet automate est décrit par l'expression régulière  $(a(a(ab)*b)*b)*$ . Si on les élimine dans l'ordre 0, 1, 2, 3, 4, en revanche, on obtient une expression de taille considérable<sup>(9)</sup> et beaucoup moins transparente.

Récapitulons le contenu essentiel à retenir comme conséquence immédiate de 4.141 et 4.2.2 :

**Théorème 4.149 (Kleene).** — *La classe des langages rationnels et celle des langages reconnaissables coïncident. (On pourra donc considérer ces termes comme synonymes.)*

Il faut cependant retenir que s'il y a, mathématiquement, équivalence entre ces deux classes de langages, cette équivalence a un coût algorithmique, c'est-à-dire que la conversion d'une expression régulière en automate (surtout si on souhaite un DFA), ou à plus forte raison d'un automate en expression régulière, a une complexité exponentielle dans le pire cas. Il est donc pertinent, en informatique, de ne pas considérer les descriptions d'un langage par une expression régulière, un DFA, et un NFA, comme interchangeables.

9. À savoir :  $\varepsilon \mid a(ba)*b \mid a(ba)*a(b(ba))*b(ba)*b \mid$   
 $a(ba)*a(b(ba)*a)*a(b(b(ba))*a)*a*b(b(ba))*b(ba)*b \mid a(ba)*a(b(ba))*a(a(b(b(ba))*a))*a(b(b(b(ba))*a))*a*b(b(b(ba))*a))*b(b(ba))*a)*b(b(ba))*a)$   
(mais si on la regarde d'assez près, on peut comprendre comment elle fonctionne).

#### 4.2.4. Stabilité des langages reconnaissables par opérations booléennes. —

En complément des constructions de Glushkov, nous allons maintenant montrer que les langages reconnaissables sont également stables par différentes opérations booléennes telles le complémentaire, l'union, l'intersection ou encore la différence symétrique.

**Proposition 4.2.3.** — *Si  $L$  est un langage reconnaissable sur un alphabet  $\Sigma$ , alors le complémentaire  $\Sigma^* \setminus L$  de  $L$  est reconnaissable ; de plus, un DFA reconnaissant l'un se déduit algorithmiquement d'un DFA reconnaissant l'autre.*

*Démonstration.* — Par hypothèse, il existe un DFA (complet !)  $A = (Q, q_0, F, \delta)$  tel que  $L = L(A)$ . Considérons le DFA  $A'$  défini par l'ensemble d'états  $Q' = Q$ , l'état initial  $q'_0 = q_0$ , la fonction de transition  $\delta' = \delta$  et pour seul changement l'ensemble d'états finaux  $F' = Q \setminus F$  complémentaire de  $F$ .

Pour  $w \in \Sigma^*$ , on a  $w \in L(A')$  si et seulement si  $\delta'^*(q_0, w) \in F'$ , c'est-à-dire  $\delta^*(q_0, w) \in F'$  (puisque  $\delta' = \delta$ ), c'est-à-dire  $\delta^*(q_0, w) \notin F$  (par définition du complémentaire), c'est-à-dire  $w \notin L(A)$ . Ceci montre bien que  $L(A')$  est le complémentaire de  $L(A)$ .  $\square$

**Remarque 4.150.** — Cette démonstration a utilisé la caractérisation des langages reconnaissables par les DFA : il était crucial de le faire, et les autres sortes d'automates définis plus haut n'auraient pas permis d'arriver (simplement) à la même conclusion. Il est intéressant de réfléchir à pourquoi. Essentiellement, dans un NFA, un mot est accepté dès qu'il existe un chemin qui l'accepte, or l'existence d'un chemin aboutissant à un état non-final n'est pas la même chose que l'inexistence d'un chemin aboutissant à un état final.

**Proposition 4.2.4.** — *Si  $L_1, L_2$  sont des langages reconnaissables (sur un même alphabet  $\Sigma$ ), alors la réunion  $L_1 \cup L_2$ , l'intersection  $L_1 \cap L_2$  et la différence symétrique  $L_1 \triangle L_2$  sont reconnaissables ; de plus, un DFA reconnaissant l'un comme l'autre se déduit algorithmiquement de DFA reconnaissant  $L_1$  et  $L_2$ .*

*Démonstration.* — Traitons le cas de l'intersection. Par hypothèse, il existe des DFA (complets !)  $A_1 = (Q_1, q_{0,1}, F_1, \delta_1)$  et  $A_2 = (Q_2, q_{0,2}, F_2, \delta_2)$  tels que  $L_1 = L(A_1)$  et  $L_2 = L(A_2)$ . Considérons le DFA  $A'$  défini par l'ensemble d'états  $Q' = Q_1 \times Q_2$  (c'est-à-dire l'ensemble des couples formés d'un état de  $A_1$  et d'un état de  $A_2$ ), l'état initial  $q'_0 = (q_{0,1}, q_{0,2})$ , la fonction de transition  $\delta' : ((p_1, p_2), x) \mapsto (\delta_1(p_1, x), \delta_2(p_2, x))$  et pour ensemble d'états finaux  $F' = F_1 \times F_2$ . Remarquons que  $(p_1, p_2) \in Q'$  appartient à  $F' = F_1 \times F_2$  si et seulement si  $p_1 \in F_1$  et  $p_2 \in F_2$  (i.e.,  $F' \subseteq Q'$  est l'ensemble des couples dont les deux composantes sont finales). Par ailleurs, si  $w \in \Sigma^*$ , on a  $\delta'^*(q'_0, w) = (\delta_1^*(q_{0,1}, w), \delta_2^*(q_{0,2}, w))$ , et par ce qui vient d'être dit, ceci appartient à  $F'$  si et seulement si  $\delta_1^*(q_{0,1}, w) \in F_1$  et  $\delta_2^*(q_{0,2}, w) \in F_2$ . On voit donc qu'un mot  $w$  appartient à  $L(A')$  si et seulement si il appartient à la fois à  $L_1$  et à  $L_2$ , ce qu'il fallait démontrer.

Pour la réunion, on peut invoquer le fait que la réunion est le complémentaire de l'intersection des complémentaires, et utiliser 4.2.3 ; si on déroule cette démonstration, on voit qu'on construit un DFA  $A''$  exactement égal à  $A'$  construit ci-dessus, à la seule

différence près que son ensemble d'états finaux est  $F'' = (F_1 \times Q_2) \cup (Q_1 \times F_2)$ , qui est le sous-ensemble de  $Q'' = Q_1 \times Q_2$  formé des couples dont *l'une au moins* des deux composantes est finale.

Pour la différence symétrique, on peut invoquer le fait que la différence symétrique peut se construire à partir des opérations de réunion, d'intersection et de complémentaire. On peut aussi construire un DFA  $A'''$  exactement égal à  $A'$  construit ci-dessus, à la seule différence près que son ensemble d'états finaux est  $F'' = (F_1 \times F_2^c) \cup (F_1^c \times F_2)$ , qui est le sous-ensemble de  $Q'' = Q_1 \times Q_2$  formé des couples dont *exactement l'une* des deux composantes est finale.  $\square$

**Remarque 4.151.** — La construction  $A'$  ci-dessus est parfois appelée automate *produit* des DFA  $A_1$  et  $A_2$ . Intuitivement, il faut comprendre que faire fonctionner l'automate  $A'$  revient à faire fonctionner les automates  $A_1$  et  $A_2$  simultanément (en parallèle), en leur donnant à consommer les mêmes symboles.

Une conséquence de 4.149 est, par exemple, que le complémentaire d'un langage rationnel est rationnel (cf. 4.2.3) ou que l'intersection de deux langages rationnels est rationnelle (cf. 4.2.4), et que ces opérations se calculent algorithmiquement. Il est donc possible, en principe, à partir d'une expression régulière  $r$  (sur un alphabet  $\Sigma$ ), de fabriquer algorithmiquement une expression régulière  $r'$  (« négation » de  $r$ ) qui dénote le langage complémentaire de celui dénoté par  $r$ ; et de même, à partir d'expressions régulières  $r_1, r_2$ , de fabriquer algorithmiquement une expression régulière  $r''$  (« conjonction » de  $r_1$  et  $r_2$ ) qui dénote le langage intersection de ceux dénotés par  $r_1$  et  $r_2$ .

Le coût de ces opérations, cependant, est astronomique : doublement exponentiel, puisqu'il s'agit de convertir l'expression en NFA, de déterminer le NFA en DFA (à un premier coût exponentiel), d'effectuer l'opération sur un DFA, et de reconvertir le DFA en expression régulière (à un deuxième coût exponentiel). Ce n'est pas faute d'avoir les bons algorithmes : on peut montrer qu'il existe effectivement des familles d'expressions régulières pour lesquelles la négation ou la conjonction produisent une augmentation de taille doublement exponentielle.

**Corollaire 4.152.** — *Il existe un algorithme qui, étant donnés deux automates finis (ou deux expressions régulières), décide si les deux langages reconnus (ou les deux langages dénotés) sont égaux.*

*Démonstration.* — Vérifier que deux langages  $L_1$  et  $L_2$  sont égaux revient à vérifier si le langage  $L = L_1 \triangle L_2$  est le langage vide. La proposition 4.2.4 montre que l'on peut construire un DFA reconnaissant  $L$  à partir de DFA reconnaissant  $L_1$  et  $L_2$ . Par ailleurs, vérifier que le langage reconnu d'un automate est le langage vide se fait en calculant l'intersection entre l'ensemble des états accessibles (ce qui se fait par un parcours de graphe) et l'ensemble des états finaux. Le langage est vide si et seulement s'il n'existe pas d'état final accessible.  $\square$

**4.2.5. Détection du caractère non rationnel d'un langage.** — On ne dispose à ce stade-là d'aucun moyen pour montrer qu'un langage *n'est pas* rationnel. La proposition 4.2.5 qui va suivre, et qui s'appelle couramment « lemme de pompage »

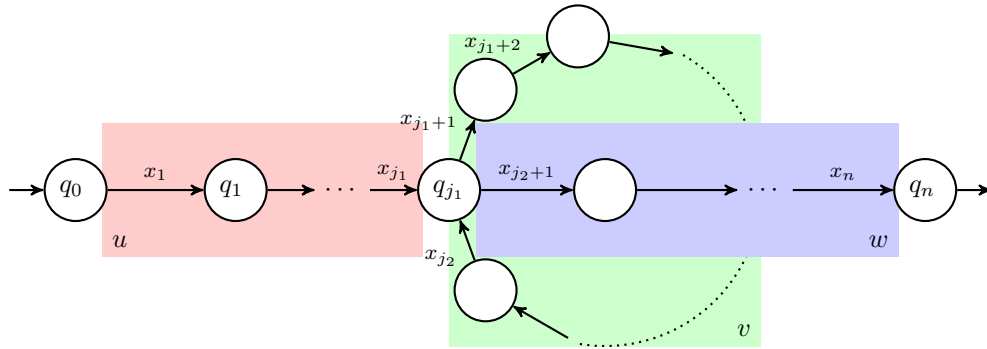
(une traduction abusive de l'anglais « pumping lemma ») constitue le moyen le plus fréquent permettant d'y arriver : il énonce une condition *nécessaire* pour qu'un langage soit rationnel, si bien qu'on peut arriver à montrer qu'un langage n'est pas rationnel en invalidant cette condition (généralement en procédant par l'absurde).

**Proposition 4.2.5 (lemme de pompage pour les langages rationnels)**

Soit  $L$  un langage rationnel. Il existe alors un entier  $k$  tel que tout mot de  $t \in L$  de longueur  $|t| \geq k$  admette une factorisation  $t = uvw$  en trois facteurs  $u, v, w \in \Sigma^*$  où :

- (i)  $|v| \geq 1$  (c'est-à-dire  $v \neq \varepsilon$ ),
- (ii)  $|uv| \leq k$ ,
- (iii) pour tout  $i \geq 0$  on a  $uv^i w \in L$ .

*Démonstration.* — Soit  $A$  un DFA (complet) qui reconnaît  $L$ , et soit  $k$  son nombre d'états : on va montrer que  $k$  vérifie les propriétés énoncées. Pour cela, soit  $t = x_1 \cdots x_n$  un mot de  $L$  de longueur  $n \geq k$ , et soient  $q_0, \dots, q_n$  les états traversés par  $A$  pendant la consommation de  $t$ , autrement dit,  $q_0$  est l'état initial, et  $q_j = \delta(q_{j-1}, x_j)$  pour chaque  $1 \leq j \leq n$  ; l'état  $q_n = \delta^*(q_0, t)$  est final puisque  $t \in L$ . Comme  $n+1 > k$  et comme l'automate  $A$  a  $k$  états, par le principe des tiroirs, il existe  $j_1 \neq j_2$  tels que  $q_{j_1} = q_{j_2}$  : pour être plus précis, soit  $j_2$  le plus petit possible tel que les états  $q_0, \dots, q_{j_2}$  ne soient pas tous distincts, autrement dit, le premier état répété, et soit  $j_1$  la précédente occurrence (forcément unique) de cet état, c'est-à-dire l'indice tel que  $j_1 < j_2$  et  $q_{j_1} = q_{j_2}$ .



Posons  $u = x_1 \cdots x_{j_1}$  (le préfixe de  $t$  de longueur  $j_1$ , qui est le mot vide si  $j_1 = 0$ ) et  $v = x_{j_1+1} \cdots x_{j_2}$  (de longueur  $j_2 - j_1$ ), et enfin  $w = x_{j_2+1} \cdots x_n$  (le suffixe de  $t$  de longueur  $n - j_2$ , avec la convention  $w = \varepsilon$  si  $j_2 = n$ ). Ceci définit bien une factorisation  $t = uvw$ .

On a bien (i)  $|v| \geq 1$  puisque  $j_2 > j_1$ . On a par ailleurs (ii)  $|uv| \leq k$  puisque  $|uv| = j_2$  et que les  $j_2$  états  $q_0, \dots, q_{j_2-1}$  sont distincts (c'est la minimalité de  $j_2$ ) de sorte que  $j_2 \leq k$  (toujours par le principe des tiroirs).

Montrons enfin (iii). On rappelle tout d'abord que  $\delta^*(q, x_1 \cdots x_j) = \delta(\cdots \delta(\delta(q, x_1), x_2) \cdots, x_j)$ . Remarquons que  $\delta^*(q_0, u) = \delta^*(q_0, x_1 \cdots x_{j_1}) = q_{j_1}$ , et

que  $\delta^*(q_{j_1}, v) = \delta^*(q_{j_1}, x_{j_1+1} \cdots x_{j_2}) = q_{j_2} = q_{j_1}$ . De cette dernière égalité, on tire  $\delta^*(q_{j_1}, v^i) = q_{j_1}$  pour tout  $i \geq 0$  (par récurrence sur  $i$ ). Enfin,  $\delta^*(q_{j_1}, w) = \delta^*(q_{j_2}, w) = \delta^*(q_{j_2}, x_{j_2+1} \cdots x_n) = q_n$  (qui est un état final). En mettant ces faits ensemble, on a  $\delta^*(q_0, uv^i w) = \delta^*(q_{j_1}, v^i w) = \delta^*(q_{j_1}, w) = q_n$ , et puisque  $q_n$  est final, ceci montre que le mot  $uv^i w$  est accepté par  $A$ , i.e.,  $uv^i w \in L$ .  $\square$

On attire l'attention sur l'alternation des quantificateurs. Le lemme de pompage énonce le fait que :

- pour tout langage rationnel  $L$ ,
- il existe un entier  $k \geq 0$  tel que
- pour tout mot  $t \in L$  de longueur  $|t| \geq k$ ,
- il existe une factorisation  $t = uvw$  vérifiant les propriétés (i)  $|v| \geq 1$ , (ii)  $|uv| \leq k$  et (iii) qui suit :
- pour tout  $i \geq 0$  on a  $uv^i w \in L$ .

La complexité logique d'un énoncé étant justement mesurée par le nombre d'alternations de quantificateurs (passages entre « pour tout » et « il existe » ou vice versa), celui-ci mérite une attention particulière. Rappelons donc, du point de vue logique, que, quand on veut *appliquer* un résultat de ce genre, on *choisit librement* les objets introduits par un quantificateur universel (« pour tout »), mais *on ne choisit pas* ceux qui sont introduits par un quantificateur existentiel (« il existe ») (ces derniers sont, si on veut, choisis par l'énoncé qu'on applique : on ne fait que recevoir leur existence) ; les choses sont inversées quand on doit démontrer un tel énoncé, mais la démonstration a été faite ci-dessus et il est donc plus fréquent de devoir appliquer le lemme de pompage.

Le modèle d'une démonstration par l'absurde pour montrer qu'un langage  $L$  n'est pas rationnel est donc quelque chose comme ceci :

- on entame un raisonnement par l'absurde en supposant que  $L$  est rationnel, et on choisit  $L$  pour appliquer le lemme de pompage (parfois on l'applique à autre chose, comme l'intersection de  $L$  avec un langage connu pour être rationnel, mais en général ce sera  $L$ ),
- le lemme de pompage fournit un  $k$  (on *ne choisit donc pas* ce  $k$ , il est donné par le lemme),
- on choisit alors un mot  $t \in L$  de longueur  $\geq k$ , et c'est là que réside la difficulté principale de la démonstration,
- le lemme de pompage fournit une factorisation  $t = uvw$ , qu'on *ne choisit pas* non plus mais qu'on peut analyser, souvent en utilisant (i) et (ii),
- et on cherche à appliquer la propriété (iii), ce qui implique de choisir un  $i$ , pour arriver à une contradiction (typiquement : le mot  $uv^i w$  n'est pas dans le langage alors qu'il est censé y être).

Donnons maintenant un exemple d'utilisation du lemme :

**Proposition 4.153.** — Soit  $\Sigma = \{a, b\}$ . Le langage  $L = \{a^n b^n : n \in \mathbb{N}\} = \{\varepsilon, ab, aabb, aaabbb, \dots\}$  constitué des mots formés d'un certain nombre ( $n$ ) de  $a$  suivis du même nombre de  $b$  n'est pas rationnel.

*Démonstration.* — Appliquons la proposition 4.2.5 au langage  $L$  considéré : appelons  $k$  l'entier dont le lemme de pompage garantit l'existence. Considérons le mot  $t := a^k b^k$  : il doit alors exister une factorisation  $t = uvw$  pour laquelle on a (i)  $|v| \geq 1$ , (ii)  $|uv| \leq k$  et (iii)  $uv^i w \in L$  pour tout  $i \geq 0$ . La propriété (ii) assure que  $uv$  est formé d'un certain nombre de répétitions de la lettre  $a$  (car tout préfixe de longueur  $\leq k$  de  $a^k b^k$  est de cette forme) ; disons  $u = a^\ell$  et  $v = a^m$ , si bien que  $w = a^{k-\ell-m} b^k$ . La propriété (i) donne  $m \geq 1$ . Enfin, la propriété (iii) affirme que le mot  $uv^i w = a^{k+(i-1)m} b^k$  appartient à  $L$  ; mais dès que  $i \neq 1$ , ceci est faux : il suffit donc de prendre  $i = 0$  pour avoir une contradiction.  $\square$

L'idée intuitive derrière la démonstration qu'on vient de faire est la suivante : un automate fini ne dispose que d'une quantité finie (bornée) de mémoire, donc ne peut « compter » que jusqu'à un nombre borné (au-delà, il va retomber sur un état déjà atteint par un plus petit nombre de  $a$ , et sera incapable de vérifier si le nombre de  $b$  est égal). C'est ce type de raisonnement que le lemme de pompage permet de formaliser. Généralement parlant, on doit garder à l'esprit le fait que toutes sortes de langages ne sont pas rationnels pour la raison informelle que les identifier demande une quantité de mémoire qui pourrait être arbitrairement grande, et que pour le montrer rigoureusement, le lemme de pompage sera souvent utile.

**4.2.6. L'automate minimal, et la minimisation.** — On sait maintenant convertir une expression régulière en un automate équivalent, et réciproquement convertir un automate en expression régulière équivalente. Il reste un problème auquel nous n'avons pas donné de réponse : existe-t-il une manière canonique de représenter un langage rationnel ?

Pour cela, on va introduire un DFA particulier, *canonique*, associé à un langage rationnel, qu'on pourra calculer algorithmiquement, et qui sera véritablement associé au langage, c'est-à-dire que deux descriptions équivalentes du même langage donneront exactement le même automate canonique ; par conséquent, pour identifier un langage, quelle que soit sa description, il suffira de calculer son automate canonique. En fait, cet automate canonique est simplement le DFA ayant le plus petit nombre d'états reconnaissant le langage en question ; ce qui est remarquable, et qui n'est pas du tout évident *a priori*, c'est qu'il est effectivement canonique (c'est-à-dire qu'il n'existe qu'un seul DFA ayant un nombre minimal d'états reconnaissant le langage) et qu'on peut le calculer algorithmiquement.

**Théorème 4.154 (Myhill-Nerode).** — Soit  $L$  un langage. Pour  $w \in \Sigma^*$ , notons  $w^{-1}L := \{t \in \Sigma^* : wt \in L\}$  (autrement dit, l'ensemble des mots qu'on peut concaténer à  $w$  pour obtenir un mot de  $L$ ). Considérons la relation d'équivalence  $\equiv$  sur  $\Sigma^*$  définie par  $u \equiv v$  si et seulement si  $u^{-1}L = v^{-1}L$  ; ce qui signifie, si on préfère, que  $\forall t \in \Sigma^* ((ut \in L) \iff (vt \in L))$ .

Alors :

- le langage  $L$  est rationnel si et seulement si la relation d'équivalence  $\equiv$  possède un nombre fini  $k$  de classes d'équivalence,



- lorsque c'est le cas, il existe un DFA (complet) ayant  $k$  états qui reconnaît  $L$ , il est unique à renommage des états<sup>(10)</sup> près, et il n'existe pas de DFA (complet) ayant  $< k$  états qui reconnaisse  $L$ .

*Démonstration.* — Supposons d'abord que l'ensemble  $\Sigma^*/\equiv$  des classes d'équivalence pour  $\equiv$  soit fini : appelons-le  $Q$ , et expliquons comment on peut construire un DFA reconnaissant  $L$  et dont l'ensemble des états soit  $Q$ . Notons  $[u] := \{v \in \Sigma^* : u \equiv v\}$  pour la classe d'équivalence de  $u$  pour  $\equiv$ . Posons  $q_0 := [\varepsilon]$  la classe du mot vide. Remarquons que si  $u \equiv v$  (pour  $u, v \in \Sigma^*$ ), alors on a  $u \in L$  si et seulement si  $v \in L$  (en effet, la définition de  $\equiv$  est que  $(ut \in L) \iff (vt \in L)$  pour tout  $t$ , et on applique ça à  $t = \varepsilon$ ) : autrement dit, une classe  $[u] \in Q$  est soit entièrement incluse dans  $L$  soit disjointe de  $L$  ; appelons  $F$  l'ensemble  $\{[u] : u \in L\}$  des classes incluses dans  $L$ . Enfin remarquons que si  $u \equiv v$  (pour  $u, v \in \Sigma^*$ ) et  $x \in \Sigma$ , on a encore  $ux \equiv vx$  (en effet,  $uxt \in L \iff vxt \in L$  pour tout  $t \in L$ ) : il y a donc un sens à définir  $\delta([u], x) = [ux]$ . On a ainsi fabriqué un DFA  $A = (Q, q_0, F, \delta)$ . Vues les définitions de  $q_0$  et  $\delta$ , il est clair que  $\delta^*(q_0, w) = [w]$  pour cet automate, et vue la définition de  $F$ , on a  $\delta^*(q_0, w) \in F$  si et seulement si  $w \in L$ . Ceci montre bien que  $A$  reconnaît le langage  $L$ . On a donc prouvé que si  $\Sigma^*/\equiv$  est fini, le langage  $L$  est rationnel et même il existe un DFA ayant  $k := \#(\Sigma^*/\equiv)$  états qui le reconnaît.

Supposons maintenant que  $B = (Q_B, q_{0,B}, F_B, \delta_B)$  soit un DFA reconnaissant  $L$ . Définissons une nouvelle relation d'équivalence  $\equiv_B$  sur  $\Sigma^*$  par :  $u \equiv_B v$  si et seulement si  $\delta_B^*(q_{0,B}, u) = \delta_B^*(q_{0,B}, v)$  (autrement dit, les mots  $u$  et  $v$  mettent l'automate  $B$  dans le même état). Si on a  $u \equiv_B v$ , on a aussi  $u \equiv v$  : en effet, pour tout  $t \in L$  on a  $\delta_B^*(q_{0,B}, ut) = \delta_B^*(\delta_B^*(q_{0,B}, u), t) = \delta_B^*(\delta_B^*(q_{0,B}, v), t) = \delta_B^*(q_{0,B}, vt)$ , et notamment le membre de gauche appartient à  $F_B$  si et seulement si le membre de droite y appartient, c'est-à-dire que  $ut \in L \iff vt \in L$ . On vient donc de montrer que  $u \equiv_B v$  implique  $u \equiv v$  (la relation  $\equiv_B$  est *plus fine* que  $\equiv$ ) : si on préfère, chaque classe d'équivalence pour  $\equiv$  est donc une réunion de classes d'équivalences pour  $\equiv_B$ . Notamment, comme  $\equiv_B$  a un nombre fini de classes d'équivalence (puisque  $\delta_B^*(q_{0,B}, u)$  ne peut prendre qu'un nombre fini de valeurs, celles dans  $Q_B$ ), il en va de même de  $\equiv$ , et plus précisément, comme  $\equiv_B$  a au plus  $\#Q_B$  classes d'équivalence, il en va de même de  $\equiv$ , c'est-à-dire  $k \leq \#Q_B$ . Il n'existe donc pas de DFA ayant  $< k$  états reconnaissant  $L$ .

Enfin, si  $B$  a  $k$  états et reconnaît  $L$ , cela signifie que les deux relations  $\equiv$  et  $\equiv_B$  coïncident, et on définit une bijection  $\psi$  entre le  $Q = Q_A$  du paragraphe précédent et  $Q_B$  en associant à une classe  $[w] \in Q$  l'état  $\psi([w]) := \delta_B^*(q_{0,B}, w)$  (qui ne dépend que de la classe de  $w$  pour  $\equiv_B$  et on vient de voir que c'est la classe de  $w$  modulo  $\equiv$ , c'est-à-dire  $[w]$  : ceci est donc bien défini). Cette bijection  $\psi$  vérifie  $\psi(q_0) = \psi([\varepsilon]) = \delta_B^*(q_{0,B}, \varepsilon) = q_{0,B}$  ; on a  $[w] \in F$  si et seulement si  $w \in L$ , c'est-à-dire si et seulement si  $\delta_B^*(q_{0,B}, w) \in F_B$  autrement dit  $\psi([w]) \in F_B$ . Et enfin, pour  $w \in \Sigma^*$  et  $x \in \Sigma$ , on a  $\psi(\delta([w], x)) = \psi([wx]) = \delta_B^*(q_{0,B}, wx) = \delta_B(\delta_B^*(q_{0,B}, w), x) = \delta_B(\psi([w]), x)$ . Bref,  $\psi$  préserve l'état initial, les états finaux, et la relation de transition : c'est donc bien un isomorphisme d'automates (un renommage des états).  $\square$

10. C'est-à-dire, si on préfère ce terme, isomorphisme d'automates.

Ce théorème affirme donc qu'il existe (à renommage des états près) un unique DFA (complet) ayant un nombre minimal d'états parmi ceux qui reconnaissent le langage rationnel  $L$  : on l'appelle *automate minimal* ou *automate canonique* du langage  $L$ . La démonstration ci-dessus en donne une construction à partir d'une relation d'équivalence, mais cette démonstration n'est pas algorithmique : on va voir comment on peut le construire de façon algorithmique à partir d'un DFA quelconque qui reconnaît  $L$ .

**Proposition 4.155.** — *Soit  $B = (Q_B, q_{0,B}, F_B, \delta_B)$  un DFA (complet !) reconnaissant un langage  $L$ , et dont tous les états sont accessibles (cf. ??). Alors l'automate minimal  $A$  de  $L$  peut s'obtenir en fusionnant dans  $B$  chaque classe d'équivalence pour la relation d'équivalence  $\equiv$  définie sur  $Q_B$  par*

$$q \equiv q' \iff \forall t \in \Sigma^* (\delta_B^*(q, t) \in F_B \iff \delta_B^*(q', t) \in F_B)$$

(Fusionner chaque classe d'équivalence signifie qu'on construit l'automate  $A$  dont l'ensemble d'états est l'ensemble  $Q_A := Q_B / \equiv$  des classes d'équivalence, l'état initial  $q_{0,A}$  est la classe  $[q_{0,B}]$  de  $q_{0,B}$  pour  $\equiv$ , les états finaux sont les classes contenant au moins un état final et qui d'ailleurs sont entièrement constituées d'états finaux, et la relation de transition est donnée par  $\delta_A([q], x) = [\delta_B(q, x)]$  en notant  $[q]$  la classe de  $q$  pour  $\equiv$ .)

De plus, cet automate  $A$  (ou de façon équivalente, la relation  $\equiv$ ) peut se déduire algorithmiquement de  $B$ .

*Démonstration.* — On a vu dans le cours de la démonstration de 4.154 que l'automate minimal a pour ensemble d'états  $\Sigma^* / \equiv_L$  où  $\equiv_L$  désigne la relation d'équivalence (alors notée  $\equiv$ ) définie par  $u \equiv_L v$  lorsque  $\forall t \in \Sigma^* ((ut \in L) \iff (vt \in L))$ . Si  $q, q' \in Q_B$ , comme  $q, q'$  sont accessibles, il existe  $w, w' \in \Sigma^*$  tels que  $\delta_B^*(q_{0,B}, w) = q$  et  $\delta_B^*(q_{0,B}, w') = q'$  ; et on a  $q \equiv q'$  si et seulement si  $\delta_B^*(q, t) \in F_B \iff \delta_B^*(q', t) \in F_B$  pour tout  $t \in \Sigma^*$ , c'est-à-dire  $\delta_B^*(q_{0,B}, wt) \in F_B \iff \delta_B^*(q_{0,B}, w't) \in F_B$ , c'est-à-dire  $wt \in L \iff w't \in L$ , autrement dit,  $w \equiv_L w'$ . L'application  $\varphi$  qui à un état  $[w]_L$  de l'automate minimal associe la classe de  $\delta_B^*(q_{0,B}, w)$  pour  $\equiv$  est donc une bijection, et comme dans la démonstration de 4.154 on vérifie qu'elle préserve l'état initial, les états finaux, et la relation de transition. On obtient donc bien l'automate minimal en fusionnant chaque classe d'équivalence pour  $\equiv$ .

Montrons maintenant comment on peut construire  $\equiv$  algorithmiquement. Pour cela, on va définir des relations d'équivalence  $\equiv_i$  pour  $i \in \mathbb{N}$  par

$$q \equiv_i q' \iff \forall t (|t| \leq i \implies (\delta_B^*(q, t) \in F_B \iff \delta_B^*(q', t) \in F_B))$$

C'est-à-dire par récurrence

$$\begin{aligned} q \equiv_0 q' &\iff (q \in F_B \iff q' \in F_B) \\ q \equiv_{i+1} q' &\iff (q \equiv_i q' \text{ et } \forall x \in \Sigma (\delta_B(q, x) \equiv_i \delta_B(q', x))) \end{aligned}$$

(la première équivalence vient de ce que  $\delta_B^*(q, \varepsilon) = q$ , et la seconde de ce que  $\delta_B^*(q, xt) = \delta_B^*(\delta_B(q, x), t)$ ).

Il est trivial que  $q \equiv_{i+1} q'$  implique  $q \equiv_i q'$ , c'est-à-dire que  $\equiv_{i+1}$  est plus fine que  $\equiv_i$ , mais  $\equiv$  est plus fine que toutes, et comme elle n'a qu'un nombre fini de classes d'équivalence, le nombre de classes ne peut croître strictement qu'un nombre

fini de fois, et il doit donc stationner : il existe  $i$  tel que  $(\equiv_{i+1}) = (\equiv_i)$ , et à ce moment-là, la seconde équivalence de la récurrence montre que  $(\equiv_j) = (\equiv_i)$  pour tout  $j \geq i$  et donc  $(\equiv) = (\equiv_i)$ .

On peut donc calculer  $\equiv$  selon l'algorithme suivant : calculer  $\equiv_0$ , et par récurrence calculer les  $\equiv_i$  jusqu'à ce que la relation ne change plus,  $(\equiv_{i+1}) = (\equiv_i)$ , auquel cas la dernière relation calculée est la relation recherchée  $(\equiv) = (\equiv_i)$ , et l'automate minimal  $A$  s'obtient en fusionnant chaque classe d'équivalence pour  $\equiv$ .  $\square$

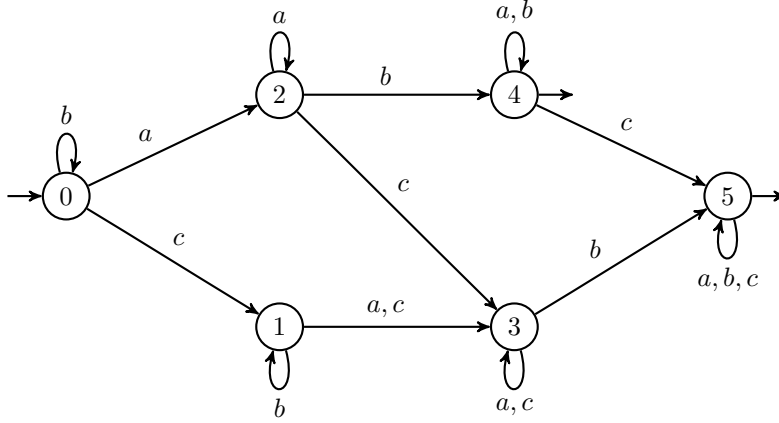
L'algorithme décrit par la proposition 4.155 porte le nom d'algorithme de Moore ou de minimisation ou de réduction.

Voici comment on peut le mettre en œuvre de façon plus concrète :

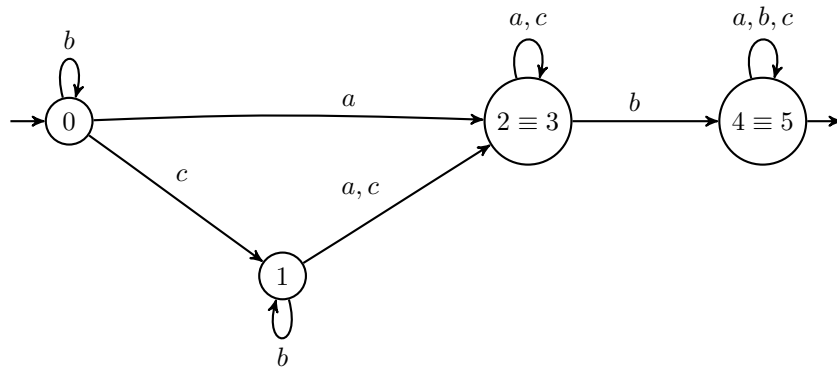
- s'assurer qu'on a affaire à un DFA *complet sans état inaccessible* (si nécessaire, déterminer l'automate s'il n'est pas déterministe, le compléter s'il est incomplet, et supprimer les états inaccessibles s'il y en a) ;
- appeler  $\Pi$  la partition des états en deux classes : les états finaux d'un côté, et les non-finaux de l'autre ;
- répéter l'opération suivante tant que la partition  $\Pi$  change : pour chaque classe  $C$  de  $\Pi$  et chaque lettre  $x \in \Sigma$ , s'il existe deux états  $q, q' \in C$  tels que  $\delta(q, x)$  et  $\delta(q', x)$  ne tombent pas dans la même classe de  $\Pi$ , séparer cette classe  $C$  selon la classe de  $\delta(\cdot, x)$  (autrement dit,  $q, q'$  restent dans la même classe pour la nouvelle partition lorsqu'ils sont dans la même classe pour l'ancienne et que pour chaque lettre  $x$  leurs images  $\delta(q, x)$  et  $\delta(q', x)$  sont aussi dans la même classe) ;
- si  $\Pi$  est la (dernière) partition ainsi obtenue, l'ensemble des états de l'automate construit est l'ensemble des classes de  $\Pi$ , l'état initial est la classe de l'état initial, les états finaux sont les classes qui contiennent un état final (ils sont alors forcément tous finaux), et la fonction de transition est obtenue en prenant la fonction de transition sur un représentant quelconque de la classe (la classe ne doit pas dépendre du représentant).

La dernière étape (construction de l'automate) permet de vérifier qu'on a correctement terminé l'étape précédente (raffinement de la partition) : si deux états dans la même classe ont une transition sortante d'étiquette  $x$  et qui mènent vers des classes différentes, c'est que ces états auraient dû être séparés. Il est donc utile de refaire un contrôle à ce niveau.

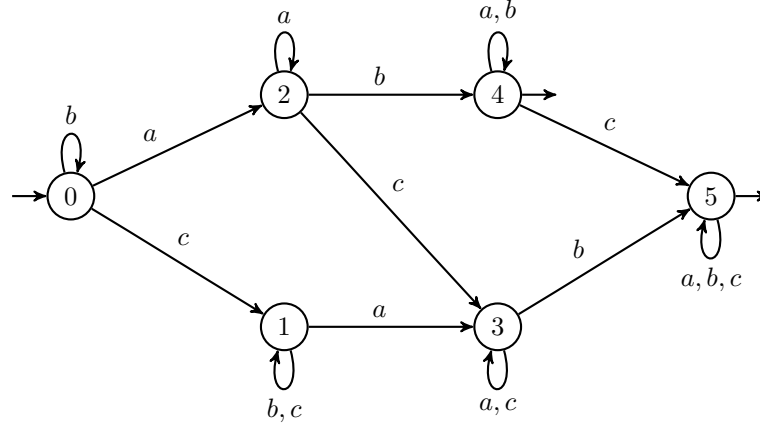
**Exemple 4.156.** — Considérons l'automate suivant :



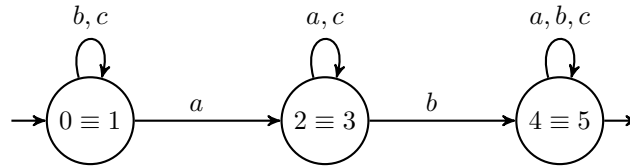
Cet automate est bien déterministe, complet et sans état inaccessible. Dans un premier temps, on partitionne les états en états non-finaux et finaux, soit  $\{0, 1, 2, 3\}$  d'un côté et  $\{4, 5\}$  de l'autre. Ensuite, on doit séparer la classe  $\{0, 1, 2, 3\}$  en deux selon que la transition étiquetée par  $b$  tombe dans la classe  $\{0, 1, 2, 3\}$  elle-même ou bien dans la classe  $\{4, 5\}$  : on arrive donc à trois classes,  $\{0, 1\}$ ,  $\{2, 3\}$  et  $\{4, 5\}$ . Enfin, on doit séparer la classe  $\{0, 1\}$  en deux selon que la transition étiquetée par  $c$  tombe dans la classe  $\{0, 1\}$  elle-même ou bien dans la classe  $\{2, 3\}$  : on arrive alors à quatre classes,  $\{0\}$ ,  $\{1\}$ ,  $\{2, 3\}$  et  $\{4, 5\}$ , et à l'automate minimal suivant :



Il est intéressant de voir comment des petits changements sur l'automate initial modifient la minimisation. Si on fait pointer la transition étiquetée par  $c$  de l'état 1 vers lui-même (au lieu d'aller vers l'état 3), c'est-à-dire :



alors la minimisation ne sépare pas les états 0 et 1, et on obtient (le même automate qu'en ??, à savoir) :



En revanche, si on change l'automate pour rendre l'état 4 non-final (avec ou sans la modification précédemment évoquée), la minimisation aboutit sur la partition triviale en six états, c'est-à-dire que l'automate est, en fait, déjà minimal.

Si aucun état d'un DFA n'est final (c'est-à-dire s'il reconnaît le langage vide  $\emptyset$ ), alors l'algorithme de minimisation termine immédiatement avec une seule classe d'équivalence, et donne donc un automate minimal ayant un unique état, initial mais non final, et des transitions étiquetées par toutes les lettres de cet état vers lui-même. Si *tous* les états d'un DFA sont finaux (il reconnaît le langage  $\Sigma^*$  de tous les mots), de même, l'algorithme de minimisation termine immédiatement avec une seule classe d'équivalence, et donne donc un automate minimal ayant un unique état, à la fois initial et final (et toujours des transitions étiquetées par toutes les lettres de cet état vers lui-même).

#### 4.3. Exercices

**Exercice 4.157.** — On s'intéresse à la recherche du motif  $p = \text{baabaaab}$  dans la chaîne  $s = \text{abaabaabaaabaabbbbbb}$ .

1. Calculer la table  $s_p$  de longueur des plus long bords de  $p$ .
2. Compter le nombre de comparaisons de symboles effectuées par l'algorithme de Morris et Pratt pour déterminer l'ensemble des occurrences de  $p$  dans  $s$ .
3. Calculer la table  $r_p$  de longueur des plus long bords compatibles de  $p$ .

- Compter le nombre de comparaisons de symboles effectuées par l'algorithme de Knuth Morris et Pratt pour déterminer l'ensemble des occurrences de  $p$  dans  $s$ .

**Exercice 4.158.** — Dans cet exercice, on suppose que l'alphabet  $\Sigma$  est un alphabet binaire.

- Soit  $p$  un mot sur  $\Sigma$  et  $w$  un préfixe strict de  $p$ . On suppose que  $b$  est un bord compatible de  $w$ . Que peut-on dire des lettres suivant  $b$  dans son occurrence préfixe et suffixe dans  $w$  ?
- Quel amélioration peut-on apporter à l'algorithme de Knuth Morris Pratt dans ce cas ?

**Exercice 4.159.** — Lesquels des codes suivants ne sont pas des codes de Huffman (pour aucune fréquence de lettres) ?

$(0, 10, 11), \quad (00, 01, 10, 110) \quad (01, 10)$

**Exercice 4.160.** — On donne les symboles suivants ainsi que la probabilité avec laquelle ils apparaissent dans un texte. Détailler le code de Huffman associé. Quel est le taux de compression obtenu en comparaison avec un encodage en code ASCII (7 bits par caractère) ?

- 

| $a$ | $b$ | $c$ | $d$ | $e$ |
|-----|-----|-----|-----|-----|
| .25 | .25 | .2  | .15 | .15 |

- 

| $a$ | $b$ | $c$ | $d$ | $e$ | $f$ | $g$ |
|-----|-----|-----|-----|-----|-----|-----|
| .01 | .24 | .05 | .20 | .47 | .01 | .02 |

**Exercice 4.161.** — On dispose d'un texte écrit à l'aide de l'alphabet  $\Sigma = \{a, b, c, d, e, f, g, h\}$ . Les proportions de chaque symbole dans le texte sont données en pourcents dans le tableau suivant

| Symbole        | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ | $g$ | $h$ |
|----------------|-----|-----|-----|-----|-----|-----|-----|-----|
| Proportion (%) | 2   | 10  | 4   | 18  | 12  | 16  | 32  | 6   |

Construire le code de Huffman associé à ce texte. Comparer le taux de compression entre un code à longueur fixe et le code de Huffman

**Exercice 4.162.** — Concevoir un code de Huffman pour un texte qui ne comprend que les mots du tableau suivant avec des fréquences égales et séparés par une espace, notée  $\sqcup$ .

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| bac  | bas  | bec  | cas  | sac  | sec  |
| aces | bacs | base | becs | cabs | case |

**Exercice 4.163.** — Étant donné un alphabet  $\{A, B, C, D, E, F, G, H\}$  et leurs occurrences 1, 1, 1, 2, 3, 5, 5, 12 dans un mot à compresser, dessiner trois arbres de compression de Huffman non-isomorphes de hauteur 5 ou 6 et calculer la longueur du texte compressé par chacun des arbres.

**Exercice 4.164.** — Quel code de Huffman obtient-on avec un alphabet  $\Sigma = \{\sigma_1, \dots, \sigma_k\}$  de  $k$  lettres et les fréquences  $[F_1, F_2, \dots, F_k]$  où  $F_n$  est le  $n$ -ième terme de la suite de Fibonacci :

$$F_1 = 1 \quad F_2 = 1, \quad \forall n \in \mathbb{N}, F_{n+2} = F_{n+1} + F_n.$$

**Exercice 4.165.** — Encoder un texte par un codage préfixe adapté permet certes de réduire la taille de la représentation de ce texte, mais on peut se demander combien de place il faut pour mémoriser un tel codage. Montrer que l'on peut représenter le code préfixe optimal d'un alphabet de  $k$  lettres en utilisant uniquement  $2k - 1 + k \lceil \log_2 k \rceil$  bits. [Indication : utiliser  $2k - 1$  bits pour spécifier la structure de l'arbre tel qu'identifié par un certain parcours.]

**Exercice 4.166.** — Soit  $\Sigma = \{0, 1\}$ . On appelle *mot binaire* un mot sur l'alphabet  $\Sigma$ , et mot binaire *normalisé* un mot binaire qui *soit* commence par 1, *soit* est exactement égal à 0.

(1) Montrer que le langage  $L_n = \{0, 1, 10, 11, 100, 101, \dots\}$  des mots binaires normalisés est rationnel en exhibant directement une expression régulière qui le dénote, et montrer qu'il est reconnaissable en exhibant directement un automate fini qui le reconnaît.

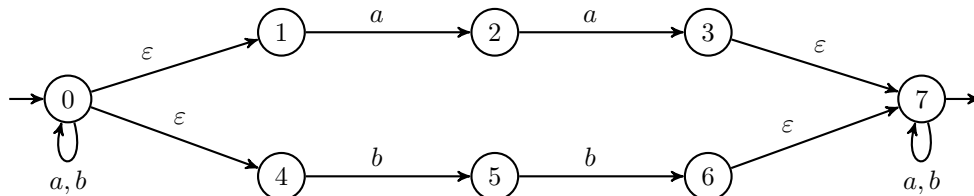
(2) On définit la *valeur numérique* d'un mot binaire  $x_{n-1} \dots x_0$  comme  $\sum_{i=0}^{n-1} x_i 2^i$  (où  $x_i$  vaut 0 ou 1 et est numéroté de 0 pour le chiffre le plus à droite à  $n - 1$  pour le plus à gauche) ; la valeur numérique du mot vide  $\varepsilon$  est 0.

Parmi les langages suivants, certains sont rationnels. Dire lesquels et justifier brièvement pourquoi ils le sont (on ne demande pas de justifier pourquoi ceux qui ne sont pas rationnels ne le sont pas) :

- (a) le langage  $L_a$  des mots binaires dont la valeur numérique est paire,
- (b) le langage  $L_b$  des mots binaires *normalisés* dont la valeur numérique est paire,
- (c) le langage  $L_c$  des mots binaires dont la valeur numérique est multiple de 3 (indication : selon que  $n$  est congru à 0, 1 ou 2 modulo 3, et selon que  $x$  vaut 0 ou 1, à quoi est congru  $2n + x$  modulo 3 ?),
- (d) le langage  $L_d$  des mots binaires dont la valeur numérique est un nombre premier,
- (e) le langage  $L_e$  des mots binaires dont la valeur numérique est une puissance de 2, i.e., de la forme  $2^i$  pour  $i \in \mathbb{N}$ ,

**Exercice 4.167.** — Soit  $\Sigma = \{a\}$ . Montrer que le langage  $L = \{a^2, a^3, a^5, a^7, a^{11}, a^{13} \dots\}$  constitué des mots ayant un nombre *premier* de  $a$ , n'est pas rationnel.

**Exercice 4.168.** — On considère l'automate suivant :

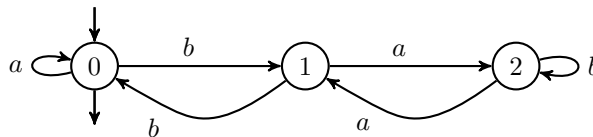


- (0) Décrire brièvement le langage accepté par l'automate en question.  
 (1) Cet automate est-il déterministe ? Si non, le déterminer.  
 (2) Minimiser l'automate déterminisé (on doit trouver un DFA ayant quatre états).  
 Décrire brièvement la signification de ces quatre états, de façon à vérifier qu'il accepte le même langage que décrit en (0).  
 (3) Éliminer les états de l'automate d'origine de façon à obtenir une expression régulière dénotant le langage reconnu par le langage décrit en (0).

**Exercice 4.169.** — Soit  $\Sigma = \{a, b\}$ .

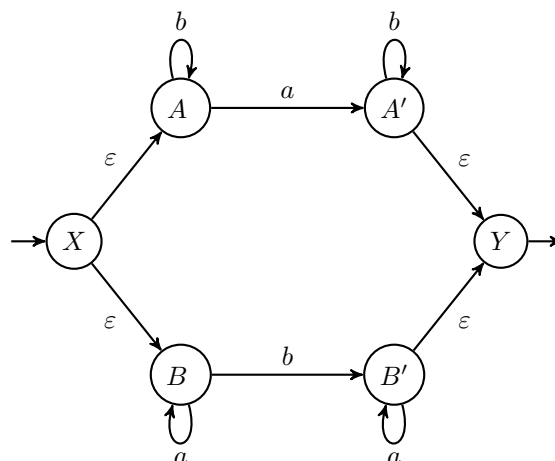
- (1) Tracer l'automate de Thompson de l'expression régulière  $a^*(baa^*)^*$ . Cet automate est-il déterministe ?  
 (2) En éliminer les transitions spontanées.  
 (3) Déterminiser l'automate obtenu (on demande un automate complet).  
 (4) Minimiser l'automate obtenu (on demande un automate complet).  
 (5) Vérifier le résultat en décrivant en français le langage dénoté par l'expression régulière initiale et reconnu par l'automate finalement obtenu.

**Exercice 4.170.** — Donner plusieurs (au moins trois) expressions régulières équivalentes dénotant le langage reconnu par l'automate suivant sur l'alphabet  $\Sigma = \{a, b\}$  :



(On pourra considérer les ordres suivants d'élimination des états : (A) 2, 1, 0, ensuite (B) 1, 2, 0 et enfin (C) 0, 2, 1.)

**Exercice 4.171.** — On considère l'automate fini  $M$  sur l'alphabet  $\Sigma = \{a, b\}$  représenté par la figure suivante :





(0) De quelle sorte d'automate s'agit-il? (Autrement dit : est-il déterministe ou non? avec transitions spontanées ou non?)

(1a) Décrire brièvement, en français, le langage  $L$  reconnu (=accepté) par l'automate  $M$ , puis donner une expression régulière qui le dénote. (On pourra préférer traiter la question (1b) d'abord.)

(1b) Pour chacun des mots suivants, dire s'ils sont dans  $L$  ou non :  $\varepsilon$ ,  $a$ ,  $b$ ,  $ab$ ,  $aa$ ,  $aab$ ,  $aabb$ ,  $abab$ ,  $ababa$ . (Note : il est recommandé de réutiliser ces mots pour vérifier rapidement les réponses aux questions suivantes et ainsi détecter d'éventuelles erreurs lors des transformations des automates.)

(2) Éliminer les transitions spontanées de l'automate  $M$ . (On supprimera les états devenus inutiles.) On appellera  $M_2$  l'automate obtenu.

(3) Déterminer l'automate  $M_2$  obtenu en (2), si nécessaire. (On demande un automate déterministe complet.) On appellera  $M_3$  l'automate déterminisé.

Pour simplifier le travail du correcteur, on demande de représenter  $M_3$  de sorte que les transitions étiquetées par  $a$  soient, dans la mesure du possible, horizontales de la gauche vers la droite, et celles étiquetées par  $b$ , verticales du haut vers le bas.

(4) Minimiser l'automate  $M_3$  obtenu en (3), si nécessaire (justifier).

(5) Donner un automate (de n'importe quelle sorte) qui reconnaît le langage  $\bar{L} = \Sigma^* \setminus L$  complémentaire de  $L$ .

(6) Décrire brièvement, en français, ce langage complémentaire  $\bar{L}$ .

(7) (Question bonus, plus longue, à ne traiter qu'en dernier.) Calculer une expression régulière qui dénote ce langage complémentaire  $\bar{L}$ . (Ne pas hésiter à introduire des notations intermédiaires.)

**Exercice 4.172.** — Dans cet exercice, on pose  $\Sigma = \{a, b\}$ .

On considère l'expression régulière  $r$  suivante :  $(ab|ba)^*$  (sur l'alphabet  $\Sigma$ ). On appelle  $L := L(r)$  le langage qu'elle dénote.

(0) Donner quatre exemples de mots de  $L$  et quatre exemples de mots n'appartenant pas à  $L$ .

(1) Traiter l'une ou l'autre des questions suivantes : (i) construire l'automate de Glushkov  $\mathcal{A}_1$  de  $r$ ; (ii) construire l'automate de Thompson de  $r$ , puis éliminer les transitions spontanées (=  $\varepsilon$ -transitions) de ce dernier (on retirera les états devenus inutiles) : on appellera  $\mathcal{A}_1$  l'automate ainsi obtenu.

(2) À partir de  $\mathcal{A}_1$ , construire un automate fini déterministe complet reconnaissant  $L$ . On appellera  $\mathcal{A}_2$  l'automate en question.

(3) Minimiser l'automate  $\mathcal{A}_2$ . On appellera  $\mathcal{A}_3$  l'automate canonique ainsi obtenu. (On obtient un automate ayant 4 états.)

(4) Construire un automate  $\mathcal{A}_4$  reconnaissant le langage  $M := \Sigma^* \setminus L$  complémentaire de  $L$ .

(5) En appliquant à  $\mathcal{A}_4$  la méthode d'élimination des états, déterminer une expression régulière dénotant le langage  $M$ .



## CHAPITRE 5

### DÉCIDABILITÉ ET COMPLEXITÉ

#### 5.1. Décidabilité

Cette section suit le texte de [Mad23].

**5.1.1. Présentation générale.** — On s'intéresse ici à la question de savoir ce qu'un algorithme *peut* ou *ne peut pas faire*.

*Discussion préalable.* — Pour procéder de façon rigoureuse, il faudrait formaliser la notion d'algorithme (par exemple à travers le concept de machine de Turing) : on a préféré rester informel sur cette définition par exemple « un algorithme est une série d'instruction précises indiquant des calculs à effectuer étape par étape et qui ne manipulent, à tout moment, que des données finies » ou « un algorithme est quelque chose qu'on pourrait, en principe, implémenter sur un ordinateur » étant entendu que cette notion est déjà bien connue et comprise, au moins dans la pratique. Les démonstrations du fait que tel ou tel problème est décidable par un algorithme ou que telle ou telle fonction est calculable par un algorithme deviennent beaucoup moins lisibles quand on les formalise avec une définition rigoureuse d'algorithme (notamment, programmer une machine de Turing est encore plus fastidieux que programmer un ordinateur en assembleur, donc s'il s'agit d'exhiber un algorithme, c'est probablement une mauvaise idée de l'écrire sous forme de machine de Turing).

**Remarque 5.1.** — Néanmoins, il est essentiel de savoir que ces formalisations existent : on peut par exemple évoquer le paradigme du  $\lambda$ -calcul de Church (la première formalisation rigoureuse de la calculabilité), les fonctions générales récursives ( $=\mu$ -récursives) à la Herbrand-Gödel-Kleene, les machines de Turing (des machines à états finis capables de lire, d'écrire et de se déplacer sur un ruban infini contenant des symboles d'un alphabet fini dont à chaque instant tous sauf un nombre fini sont des blancs), les machines à registres, le langage « FlooP » de Hofstadter, etc. Toutes ces formalisations sont équivalentes (au sens où, par exemple, elles conduisent à la même notion de fonction calculable ou calculable partielle, définie ci-dessous). La thèse de Church-Turing affirme, au moins informellement, que tout ce qui est effectivement

calculable par un algorithme<sup>(1)</sup> est calculable par n'importe laquelle de ces notions formelles d'algorithmes, qu'on peut rassembler sous le nom commun de calculabilité au sens de Church-Turing, ou « calculabilité » tout court.

Notamment, quasiment tous les langages de programmation informatique<sup>(2)</sup>, au moins si on ignore les limites des implémentations et qu'on les suppose capables de manipuler des entiers, chaînes de caractère, tableaux, etc., de taille arbitraire (mais toujours finie)<sup>(3)</sup>, sont « Turing-complets », c'est-à-dire équivalents dans leur pouvoir de calcul à la calculabilité de Church-Turing. Pour imaginer intuitivement la calculabilité, on peut donc choisir le langage qu'on préfère et imaginer qu'on programme dedans. Dans la pratique, pour qu'un langage soit Turing-complet, il lui suffit d'être capable de manipuler des entiers de taille arbitraire, de les comparer et de calculer les opérations arithmétiques dessus, et d'effectuer des tests et des boucles.

Il faut souligner qu'on s'intéresse uniquement à la question de savoir ce qu'un algorithme peut ou ne peut pas faire (calculabilité), pas au temps ou aux autres ressources qu'il peut prendre pour le faire (complexité), et on ne cherche donc pas à rendre les algorithmes efficaces en quelque sens que ce soit. Par exemple, pour arguer qu'il existe un algorithme qui décide si un entier naturel  $n$  est premier ou non, il suffit de dire qu'on peut calculer tous les produits  $pq$  avec  $2 \leq p, q \leq n-1$  et tester si l'un d'eux est égal à  $n$ , peu importe que cet algorithme soit absurdement inefficace.

De même, nos algorithmes sont capables de manipuler des entiers arbitrairement grands : ceci permet de dire, par exemple, que toute chaîne binaire peut être considérée comme un entier, peu importe le fait que cet entier ait peut-être des milliards de chiffres ; dans les langages informatiques réels, on a rarement envie de considérer toute donnée comme un entier, mais en calculabilité on peut se permettre de le faire.

*Mise en place.* — Soulignons et développons le point esquissé au paragraphe précédent : plutôt que de travailler avec des « mots » (éléments de  $\Sigma^*$  avec  $\Sigma$  un alphabet fini) qui expriment les données d'une instance d'un problème, en calculabilité, il est possible, dès que cela est commode, de remplacer ceux-ci par des entiers naturels.

Il suffit pour cela de choisir un « codage », parfois appelé codage de Gödel permettant de représenter un élément de  $\Sigma^*$  par un entier naturel : la chose importante est que la conversion d'un mot en entier ou d'un entier en mot soit calculable par un algorithme (même très inefficace), si bien que toute manipulation algorithmique sur des mots puisse être convertie en manipulation sur des entiers.

**Définition 5.2.** — On appelle *codage de Gödel* toute fonction  $\mathbb{N} \rightarrow \Sigma^*$  algorithmiquement calculable et dont la réciproque l'est aussi.

Il est commode pour simplifier les raisonnements (quoique pas indispensable) de supposer que le codage est bijectif.

---

1. Voire, dans certaines variantes de la thèse, tout ce qui est physiquement calculable dans notre Univers (y compris par des processus quantiques).

2. C, C++, Java, Python, JavaScript, Lisp, OCaml, Haskell, Prolog, etc. Certains langages se sont même révélés Turing-complets alors que ce n'était peut-être pas voulu : par exemple, HTML+CSS.

3. Autre condition : ne pas utiliser de générateur aléatoire matériel.

Il existe toutes sortes de manières d'obtenir un tel codage. La plus simple est sans doute la suivante :

**Exemple 5.3.** — Une fois choisi un ordre total arbitraire sur l'alphabet (fini!)  $\Sigma$ , on peut énumérer les mots sur  $\Sigma$  par ordre de taille et, pour une taille donnée, par ordre lexicographique (par exemple :  $\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots$  sur l'alphabet  $\{a, b\}$ ) ; cette énumération est visiblement faisable algorithmiquement, et quitte à numérotter au fur et à mesure qu'on énumère, on obtient un codage comme souhaité (par exemple  $0 \leftrightarrow \varepsilon, 1 \leftrightarrow a, 2 \leftrightarrow b, 3 \leftrightarrow ab$ , etc.).

Insistons sur le fait que ce n'est là qu'une possibilité parmi d'autres et que les détails du codage sont sans importance tant qu'il est calculable ; l'important est que ce soit faisable en théorie et donc qu'on peut considérer qu'au lieu de mots on a affaire à des entiers naturels.

Il va de soi que la concaténation de deux mots, la longueur d'un mot, le miroir d'un mot, sont tous calculables algorithmiquement.

Grâce au codage de Gödel, on peut considérer que, dans cette partie, le terme de « langage » désigne non plus une partie de  $\Sigma^*$  mais une partie de  $\mathbb{N}$ .

**Remarque 5.4.** — Le remplacement des mots par des entiers naturels en utilisant un codage comme on vient de le dire est assez standard en calculabilité. Il ne doit pas dérouter : on peut imaginer qu'on a affaire à  $\Sigma^*$  à chaque fois qu'il est question de  $\mathbb{N}$  ci-dessous, et on a toujours affaire à la théorie des langages. Le point central est justement que cela n'a pas d'importance ; comme  $\mathbb{N}$  est un objet mathématiquement plus simple, c'est surtout pour cela qu'il est utilisé à la place.)

*Problèmes de décision.* — Informellement, un problème de décision est une question portant sur un énoncé mathématique dont la réponse est exclusivement soit OUI ou soit NON. En regard de la définition 1.5, il s'agit d'un problème où la valeur de retour est booléenne et où il n'y a pas d'effets sur l'environnement.

**Définition 5.5.** — On appelle *problème de décision* toute fonction  $\Sigma^* \rightarrow \{\text{oui}, \text{non}\}$ .

On peut alors s'intéresser à l'ensemble des mots qui décrivent une instance du problème pour laquelle la réponse est OUI. Formellement, un problème de décision n'est donc rien d'autre qu'un ensemble de mots sur un certain alphabet, autrement dit un langage .

Compte tenu de la remarque précédente, dès lors qu'un codage de Gödel est fixé et permet de convertir une instance d'un problème en un entier, on peut même proposer la définition suivante.

**Définition 5.6.** — On appelle encore *problème de décision* toute partie de  $\mathbb{N}$ .

*Terminaison des algorithmes.* — Un algorithme qui effectue un calcul utile doit certainement terminer en temps fini. Néanmoins, même si on voudrait ne s'intéresser qu'à ceux-ci, il n'est pas possible d'ignorer le « problème » des algorithmes qui ne terminent jamais (et ne fournissent donc aucun résultat). C'est le point central de la

calculabilité (et du théorème de Turing 5.21 ci-dessous) qu'on ne peut pas se débarrasser des algorithmes qui ne terminent pas : on ne peut pas, par exemple, formaliser une notion suffisante<sup>(4)</sup> de calculabilité dans laquelle tout algorithme termine toujours ; ni développer un langage de programmation suffisamment général dans lequel il est impossible qu'un programme « plante » indéfiniment ou parte en boucle infinie.

**Remarque 5.7.** — Cette subtilité est d'ailleurs sans doute en partie responsable de la difficulté historique à dégager la bonne notion d'« algorithme » : on a commencé par développer des notions d'algorithmes terminant forcément, comme les fonctions primitives récursives, et on se rendait bien compte que ces notions étaient forcément toujours incomplètes.)

**Définition 5.8.** — On dit qu'une fonction  $f: \mathbb{N} \rightarrow \mathbb{N}$  est *calculable* (ou « récursive ») lorsqu'il existe un algorithme qui prend en entrée  $n \in \mathbb{N}$ , termine toujours en temps fini, et calcule (renvoie)  $f(n)$ .

**Définition 5.9.** — On dit qu'un ensemble  $A \subseteq \mathbb{N}$  (un « langage », cf. ??) est *décidable* (ou « calculable » ou « récursif ») lorsque sa fonction indicatrice  $\mathbf{1}_A: \mathbb{N} \rightarrow \mathbb{N}$  (valant 1 sur  $A$  et 0 sur son complémentaire) est calculable. Autrement dit : lorsqu'il existe un algorithme qui prend en entrée  $n \in \mathbb{N}$ , termine toujours en temps fini, et renvoie « oui » (1) si  $n \in A$ , et « non » (0) si  $n \notin A$  (on dira que l'algorithme « décide »  $A$ ).

**Définition 5.10.** — On dit qu'un problème de décision  $\Pi$  est *décidable* lorsqu'il existe un algorithme qui prend en entrée une instance du problème  $\Pi$ , termine toujours en temps fini et renvoie « oui » (1) si l'instance admet oui comme réponse, et « non » (0) dans le cas contraire.

**Définition 5.11.** — On dit qu'une fonction partielle  $f: \mathbb{N} \dashrightarrow \mathbb{N}$  (c'est-à-dire une fonction définie sur une partie de  $\mathbb{N}$ , appelé ensemble de définition de  $f$ ) est *calculable partielle* (ou « récursive partielle ») lorsqu'il existe un algorithme qui prend en entrée  $n \in \mathbb{N}$ , termine en temps fini si et seulement si  $f(n)$  est définie, et dans ce cas calcule (renvoie)  $f(n)$ . (Une fonction calculable est donc simplement une fonction calculable partielle qui est toujours définie : on dira parfois « calculable totale » pour souligner ce fait.)

On utilisera la notation  $f(n) \downarrow$  pour signifier le fait que la fonction calculable partielle  $f$  est définie en  $n$ , c'est-à-dire, que l'algorithme en question termine.

**Définition 5.12.** — On dit qu'un ensemble  $A \subseteq \mathbb{N}$  est *semi-décidable* (ou « semi-calculable » ou « semi-récursif ») lorsque la fonction partielle  $\mathbb{N} \dashrightarrow \mathbb{N}$  définie exactement sur  $A$  et y valant 1, est calculable partielle. Autrement dit : lorsqu'il existe un

---

4. Tout dépend, évidemment, de ce qu'on appelle « suffisant » : il existe bien des notions de calculabilité, plus faibles que celle de Church-Turing, où tout calcul termine, voir par exemple la notion de fonction « primitive récursive » ou le langage « BlooP » de Hofstadter ; mais de telles notions ne peuvent pas disposer d'une machine universelle comme expliqué plus loin (en raison d'un argument diagonal), donc elles sont nécessairement incomplètes en un certain sens.

algorithme qui prend en entrée  $n \in \mathbb{N}$ , termine en temps fini si et seulement si  $n \in A$ , et renvoie « oui » (1) dans ce cas<sup>(5)</sup> (on dira que l'algorithme « semi-décide »  $A$ ).

**Définition 5.13.** — On dit qu'un problème de décision  $\Pi$  est *semi-décidable* lorsqu'il existe un algorithme qui prend en entrée une instance du problème  $\Pi$ , termine en temps fini si et seulement si cette instance admet comme réponse « oui », et renvoie « oui » (1) dans ce cas.

**Remarque 5.14.** — On s'est limité ici à des fonctions d'une seule variable (entière), mais il n'y a pas de difficulté à étendre ces notions à plusieurs variables, et de parler de fonction calculable  $\mathbb{N}^k \rightarrow \mathbb{N}$  (voire  $\mathbb{N}^* \rightarrow \mathbb{N}$  avec  $\mathbb{N}^*$  l'ensemble des suites finies d'entiers naturels) ou de fonction calculable partielle de même type : de toute manière, on peut « coder » un couple d'entiers naturels comme un seul entier naturel (par exemple par  $(m, n) \mapsto 2^m(2n+1)-1$ , qui définit une bijection calculable  $\mathbb{N}^2 \rightarrow \mathbb{N}$ ), ou bien sûr un nombre fini quelconque (même variable), ce qui permet de faire « comme si » on avait toujours affaire à un seul paramètre entier.

*Complément :* — Comme on n'a pas défini formellement la notion d'algorithme, il peut être utile de signaler explicitement les faits suivants (qui devraient être évidents sur toute notion raisonnable d'algorithme) : les fonctions constantes sont calculables ; les opérations arithmétiques usuelles sont calculables ; les projections  $(n_1, \dots, n_k) \mapsto n_i$  sont calculables, ainsi que la fonction qui à  $(m, n, p, q)$  associe  $p$  si  $m = n$  et  $q$  sinon ; toute composée de fonctions calculables (partielle ou totale) est calculable idem ; si  $\underline{m} \mapsto g(\underline{m})$  est calculable (partielle ou totale, où  $\underline{m}$  désigne une liste d'entiers) et que  $(\underline{m}, n, v) \mapsto h(\underline{m}, n, v)$  l'est, alors la fonction  $f$  définie par récurrence par  $f(\underline{m}, 0) = g(\underline{m})$  et  $f(\underline{m}, n+1) = h(\underline{m}, n, f(\underline{m}, n))$  est encore calculable idem (algorithmiquement, il s'agit juste de boucler  $n$  fois) ; et enfin, si  $(\underline{m}, n) \mapsto g(\underline{m}, n)$  est calculable partielle, alors la fonction  $f$  (aussi notée  $\mu_n g$ ) définie par  $f(\underline{m}) = \min\{n : g(\underline{m}, n) = 0 \wedge \forall n' < n (g(\underline{m}, n') \neq 0)\}$  (et non définie si ce min n'existe pas) est calculable partielle (algorithmiquement, on teste  $g(\underline{m}, 0), g(\underline{m}, 1), g(\underline{m}, 2) \dots$  jusqu'à tomber sur 0). Ces propriétés peuvent d'ailleurs servir à *définir* rigoureusement la notion de fonction calculable, c'est le modèle des fonctions « générales récursives ». (Dans ce qui précède, la notation  $\underline{m}$  signifie  $m_1, \dots, m_k$ .)

**Exemple 5.15.** — L'ensemble des nombres pairs, des carrés parfaits, des nombres premiers, sont décidables, c'est-à-dire qu'il est algorithmique de savoir si un nombre est pair, parfait, ou premier. Quitte éventuellement à coder les mots d'un alphabet fini comme des entiers naturels (cf. ??), tout langage rationnel est décidable (cf. 4.141 et ??).

On verra plus bas des exemples d'ensembles qui ne le sont pas, et qui sont ou ne sont pas semi-décidables.

5. En fait, la valeur renvoyée n'a pas d'importance ; on peut aussi définir un ensemble semi-décidable comme l'ensemble de définition d'une fonction calculable partielle.

Les deux propositions suivantes, outre leur intérêt intrinsèque, servent à donner des exemples du genre de manipulation qu'on peut faire avec la notion de calculabilité et d'algorithme :

**Proposition 5.16.** — *Un ensemble  $A \subseteq \mathbb{N}$  est décidable si et seulement si  $A$  et  $\mathbb{N} \setminus A$  sont tous les deux semi-décidables.*

*Démonstration.* — Il est évident qu'un ensemble décidable est semi-décidable (si un algorithme décide  $A$ , on peut l'exécuter puis effectuer une boucle infinie si la réponse est « non » pour obtenir un algorithme qui semi-décide  $A$ ) ; il est également évident que le complémentaire d'un ensemble décidable est décidable (quitte à échanger les réponses « oui » et « non » dans un algorithme qui le décide). Ceci montre qu'un ensemble décidable est semi-décidable de complémentaire semi-décidable, i.e., la partie « seulement si ». Montrons maintenant le « si » : si on dispose d'algorithmes  $T_1$  et  $T_2$  qui semi-décident respectivement  $A$  et son complémentaire, on peut lancer leur exécution en parallèle sur  $n \in \mathbb{N}$  (c'est-à-dire exécuter une étape de  $T_1$  puis une étape de  $T_2$ , puis de  $T_1$ , et ainsi de suite jusqu'à ce que l'un des deux termine) : comme il y en a toujours (exactement) un qui termine, selon lequel c'est, ceci permet de décider algorithmiquement si  $n \in A$  ou  $n \notin A$ .  $\square$

**Proposition 5.17.** — *Un ensemble  $A \subseteq \mathbb{N}$  non vide est semi-décidable si et seulement si il existe une fonction calculable  $f: \mathbb{N} \rightarrow \mathbb{N}$  dont l'image  $(f(\mathbb{N}))$  vaut  $A$  (on dit aussi que  $A$  est « calculablement énumérable » ou « récursivement énumérable »).*

*Démonstration.* — Montrons qu'un ensemble semi-décidable non vide est calculablement énumérable. Fixons  $n_0 \in A$  une fois pour toutes. Soit  $T$  un algorithme qui semi-décide  $A$ . On définit une fonction  $f: \mathbb{N}^2 \rightarrow \mathbb{N}$  de la façon suivante :  $f(m, n) = n$  lorsque l'algorithme  $T$ , exécuté sur l'entrée  $n$ , termine au plus  $m$  étapes ; sinon,  $f(m, n) = n_0$ . On a bien sûr  $f(m, n) \in A$  dans tous les cas ; par ailleurs, si  $n \in A$ , comme l'algorithme  $T$  appliqué à  $n$  doit terminer, on voit que pour  $m$  assez grand on a  $f(m, n) = n$ , donc  $n$  est bien dans l'image de  $f$ . Ceci montre que  $f(\mathbb{N}^2) = A$ . Passer à  $f: \mathbb{N} \rightarrow \mathbb{N}$  est alors facile en composant par une bijection calculable  $\mathbb{N} \rightarrow \mathbb{N}^2$  (par exemple la réciproque de  $(m, n) \mapsto 2^m(2n + 1) - 1$ ).

Réciproquement, si  $A$  est calculablement énumérable, disons  $A = f(\mathbb{N})$  avec  $f$  calculable, on obtient un algorithme qui semi-décide  $A$  en calculant successivement  $f(0)$ ,  $f(1)$ ,  $f(2)$ , etc., jusqu'à trouver un  $k$  tel que  $f(k) = n$  (où  $n$  est l'entrée proposée), auquel cas l'algorithme renvoie « oui » (et sinon, il ne termine jamais puisqu'il effectue une boucle infinie à la recherche d'un tel  $k$ ).  $\square$

*Éclaircissement :* — Les deux démonstrations ci-dessus font appel à la notion intuitive d'« étape » de l'exécution d'un algorithme. Un peu plus précisément, pour chaque entier  $m$  et chaque algorithme  $T$ , il est possible d'« exécuter au plus  $m$  étapes » de l'algorithme  $T$ , c'est-à-dire commencer l'exécution de celui-ci, et si elle n'est pas finie au bout de  $m$  étapes, s'arrêter (on n'aura pas le résultat de l'exécution de  $T$ , juste l'information « ce n'est pas encore fini » et d'éventuels résultats intermédiaires, mais on peut décider de faire autre chose, y compris reprendre l'exécution plus tard). La longueur d'une « étape » n'est pas spécifiée et n'a pas d'importance, les choses



qui importent sont que (A) le fait d'exécuter les  $m$  premières étapes de  $T$  termine toujours (c'est bien l'intérêt), et (B) si l'algorithme  $T$  termine effectivement, alors pour  $m$  suffisamment grand, exécuter au plus  $m$  étapes donne bien le résultat final de  $T$  comme résultat.

**Exercice 5.18.** — Un ensemble  $A \subseteq \mathbb{N}$  infini est décidable si et seulement si il existe une fonction calculable  $f: \mathbb{N} \rightarrow \mathbb{N}$  strictement croissante dont l'image vaut  $A$ .

*Esquisse de preuve.* — Si  $A$  est décidable, on peut trouver son  $n$ -ième élément par ordre croissant en testant l'appartenance à  $A$  de tous les entiers naturels dans l'ordre jusqu'à trouver le  $n$ -ième qui appartienne; réciproquement, si on a une telle fonction, on peut tester l'appartenance à  $A$  en calculant les valeurs de la fonction jusqu'à tomber sur l'entier à tester ou le dépasser.) En mettant ensemble ce fait et la proposition, on peut en déduire le fait suivant : tout ensemble semi-décidable infini a un sous-ensemble décidable infini (indication : prendre une fonction qui énumère l'ensemble et jeter toute valeur qui n'est pas strictement plus grande que toutes les précédentes). Cf. exercice 5.63.  $\square$

**5.1.2. Machine universelle, et problème de l'arrêt.** — **Codage et machine universelle.** Les algorithmes sont eux-mêmes représentables par des mots sur un alphabet fini donc, si on préfère (cf. ??), par des entiers naturels : on parle aussi de *codage de Gödel* des algorithmes/programmes par des entiers. On obtient donc une énumération  $\varphi_0, \varphi_1, \varphi_2, \varphi_3 \dots$  de toutes les fonctions calculables partielles (la fonction  $\varphi_e$  étant la fonction que calcule l'algorithme [codé par l'entier]  $e$ , avec la convention que si cet algorithme est syntaxiquement invalide ou erroné pour une raison quelconque, la fonction  $\varphi_e$  est simplement non-définie partout). Les détails de cette énumération dépendent de la formalisation utilisée pour la calculabilité et ne sont pas importants<sup>(6)</sup>.

On aura tendance, dans la suite, à écrire « le  $e$ -ième algorithme », voire « l'algorithme  $e$  » (ou « le programme  $e$  »), pour désigner l'algorithme codé par l'entier naturel  $e$  (c'est-à-dire qu'on identifie librement un programme et son codage de Gödel) : on peut donc écrire que  $\varphi_e(n)$  est le résultat de l'exécution du programme  $e$  quand on lui fournit  $n$  en entrée (ou non défini si cette exécution ne termine pas).

Un point crucial dans cette numérotation des algorithmes par des entiers naturels  $e$  est l'existence d'une *machine universelle*, c'est-à-dire d'un algorithme  $U$  qui prend en entrée un entier  $e$  (codant un algorithme  $T$ ) et un entier  $n$ , et effectue la même chose que  $T$  sur l'entrée  $n$  (i.e.,  $U$  termine sur les entrées  $e$  et  $n$  si et seulement si  $T$  termine sur l'entrée  $n$ , et, dans ce cas, renvoie la même valeur). Autrement dit, le calcul de  $\varphi_e(n)$  en fonction de  $e$  et  $n$  est lui-même algorithmique.

Informatiquement, ceci représente le fait que les programmes informatiques sont eux-mêmes représentables informatiquement : dans un langage de programmation Turing-complet, on peut écrire un *interpréteur*  $U$  pour le langage lui-même (ou pour

6. Par exemple, ceux qui aiment le langage Java peuvent considérer que  $\varphi_e$  désigne le résultat de l'exécution du programme Java représenté par le mot codé par  $e$ , si ce programme est valable (remplacer Java par tout autre langage préféré).

un autre langage Turing-complet), c'est-à-dire un programme qui prend en entrée la représentation  $e$  d'un autre programme et qui exécute ce programme (sur une entrée  $n$ ).

Mathématiquement, on peut le formuler comme le fait que la fonction (partielle)  $(e, n) \mapsto \varphi_e(n)$  (= résultat du  $e$ -ième algorithme appliqué sur l'entrée  $n$ ) est elle-même calculable partielle.

Philosophiquement, cela signifie que la notion d'exécution d'un algorithme est elle-même algorithmique : on peut écrire un algorithme qui, donnée une description (formelle !) d'un algorithme et une entrée à laquelle l'appliquer, effectue l'exécution de l'algorithme fourni sur l'entrée fournie.

On ne peut pas démontrer ce résultat ici faute d'une description rigoureuse d'un modèle de calcul précis, mais il n'a rien de conceptuellement difficile (même s'il peut être fastidieux à écrire dans les détails : écrire un interpréteur d'un langage de programmation demande un minimum d'efforts). La machine universelle dépend, bien sûr, du codage des algorithmes par des entiers naturels.

*Compléments* : — Les deux résultats classiques suivants sont pertinents en lien avec la numérotation des fonctions calculables partielles. • Le *théorème de la forme normale de Kleene* assure qu'il existe un ensemble décidable  $\mathcal{T} \subseteq \mathbb{N}^4$  tel que  $\varphi_e(n)$  soit défini si et seulement si il existe  $m, v$  tels que  $(e, n, m, v) \in \mathcal{T}$ , et dans ce cas  $\varphi_e(n) = v$  (pour s'en convaincre, il suffit de définir  $\mathcal{T}$  comme l'ensemble des  $(e, n, m, v)$  tels que le  $e$ -ième algorithme exécuté sur l'entrée  $n$  termine en au plus  $m$  étapes et renvoie le résultat  $v$  : le fait qu'on dispose d'une machine universelle et qu'on puisse exécuter  $m$  étapes d'un algorithme assure que cet ensemble est bien décidable il est même « primitif récursif »). • Le *théorème  $s$ - $m$ - $n$*  assure qu'il existe une fonction calculable  $s$  telle que  $\varphi_{s(e, \underline{m})}(\underline{n}) = \varphi_e(\underline{m}, \underline{n})$  (intuitivement, donné un algorithme qui prend plusieurs entrées et des valeurs  $\underline{m}$  de certaines de ces entrées, on peut fabriquer un nouvel algorithme dans lequel ces valeurs ont été fixées c'est à peu près trivial mais de plus, cette transformation est *elle-même algorithmique*, i.e., on peut algorithmiquement substituer des valeurs  $\underline{m}$  dans un programme [codé par l'entier]  $e$  : c'est intuitivement clair, mais cela ne peut pas se démontrer avec les seules explications données ci-dessus sur l'énumération des fonctions calculables partielles, il faut regarder précisément comment le codage standard est fait pour une formalisation de la calculabilité).

La machine universelle n'a rien de « magique » : elle se contente de suivre les instructions de l'algorithme  $T$  qu'on lui fournit, et termine si et seulement si  $T$  termine. Peut-on savoir à l'avance si  $T$  terminera ? C'est le fameux « problème de l'arrêt ».

Intuitivement, le « problème de l'arrêt » est la question « l'algorithme suivant termine-t-il sur l'entrée suivante » ?

**Définition 5.19.** — On appelle *problème de l'arrêt* (ou « langage de l'arrêt ») l'ensemble des couples  $(e, n)$  tels que le  $e$ -ième algorithme termine sur l'entrée  $n$ , i.e.,  $\{(e, n) \in \mathbb{N}^2 : \varphi_e(n) \downarrow\}$  (où la notation «  $\varphi_e(n) \downarrow$  » signifie que  $\varphi_e(n)$  est défini, i.e., l'algorithme termine). Quitte à coder les couples d'entiers naturels par des entiers naturels (par exemple par  $(e, n) \mapsto 2^e(2n + 1) - 1$ ), on peut voir le problème de

l'arrêt comme une partie de  $\mathbb{N}$ . On peut aussi préférer <sup>(7)</sup> définir le problème de l'arrêt comme  $\{e \in \mathbb{N} : \varphi_e(e) \downarrow\}$ , on va voir dans la démonstration ci-dessous que c'est cet ensemble-là qui la fait fonctionner.

**Remarque 5.20.** — On pourrait aussi définir le problème de l'arrêt comme  $\{e \in \mathbb{N} : \varphi_e(0) \downarrow\}$  si on voulait, ce serait moins pratique pour la démonstration, mais cela ne changerait rien au résultat comme on peut le voir en appliquant le théorème s-m-n.)

**Théorème 5.21 (Turing).** — *Le problème de l'arrêt est semi-décidable mais non décidable.*

*Démonstration.* — Le problème de l'arrêt est semi-décidable en vertu de l'existence d'une machine universelle : donnés  $e$  et  $n$ , on exécute le  $e$ -ième algorithme sur l'entrée  $n$  (c'est ce que fait la machine universelle), et s'il termine on renvoie « oui » (et s'il ne termine pas, bien sûr, la seule possibilité est de ne pas terminer).

Montrons par l'absurde que le problème de l'arrêt n'est pas décidable. S'il l'était, on pourrait définir un algorithme qui, donné un entier  $e$ , effectue les calculs suivants : (1°) utiliser le problème de l'arrêt (supposé décidable!) pour savoir, algorithmiquement en temps fini, si le  $e$ -ième algorithme termine quand on lui passe son propre numéro  $e$  en entrée, i.e., si  $\varphi_e(e) \downarrow$ , et ensuite (2°) si oui, effectuer une boucle infinie, et si non, terminer, en renvoyant, disons, 42. L'algorithme qui vient d'être décrit aurait un certain numéro, disons,  $p$ , et la description de l'algorithme fait que, quel que soit  $e$ , la valeur  $\varphi_p(e)$  est indéfinie si  $\varphi_e(e)$  est définie tandis que  $\varphi_p(e)$  est définie (de valeur 42) si  $\varphi_e(e)$  est indéfinie. En particulier, en prenant  $e = p$ , on voit que  $\varphi_p(p)$  devrait être défini si et seulement si  $\varphi_p(p)$  n'est pas défini, ce qui est une contradiction.  $\square$

La démonstration ci-dessus est une instance de l'« argument diagonal » de Cantor, qui apparaît souvent en mathématiques. (La « diagonale » en question étant le fait qu'on considère  $\varphi_e(e)$ , i.e., on passe le numéro  $e$  d'un algorithme en argument à cet algorithme lui-même, donc on regarde la diagonale de la fonction de deux variables  $(e, n) \mapsto \varphi_e(n)$ ; en modifiant les valeurs sur cette diagonale, on produit une fonction qui ne peut pas se trouver dans une ligne  $\varphi_p$ .) Une variante facile du même argument permet de fabriquer des ensembles non semi-décidables (voir 5.23 ci-dessous), ou bien on peut appliquer ce qui précède :

**Corollaire 5.22.** — *Le complémentaire du problème de l'arrêt n'est pas semi-décidable.*

*Démonstration.* — On a vu que le problème de l'arrêt n'est pas décidable, et qu'un ensemble est décidable si et seulement si il est semi-décidable et que son complémentaire l'est aussi : comme le problème de l'arrêt est bien semi-décidable, son complémentaire ne l'est pas.  $\square$

7. Même si au final c'est équivalent, c'est *a priori* plus fort de dire que  $\{e \in \mathbb{N} : \varphi_e(e) \downarrow\}$  n'est pas décidable que de dire que  $\{(e, n) \in \mathbb{N}^2 : \varphi_e(n) \downarrow\}$  ne l'est pas.

*Complément :* — L'argument diagonal est aussi au cur du (voire, équivalent au) *théorème de récursion de Kleene*, qui affirme que pour toute fonction calculable partielle  $h: \mathbb{N}^2 \dashrightarrow \mathbb{N}$ , il existe  $p$  tel que  $\varphi_p(n) = h(p, n)$  pour tout  $n$  (la signification intuitive de ce résultat est qu'on peut supposer qu'un programme a accès à son propre code source  $p$ , i.e., on peut programmer comme s'il recevait en entrée un entier  $p$  codant ce code source ; ceci permet par exemple de façon anecdotique mais amusante d'écrire des programmes, parfois appelés « quines », qui affichent leur propre code source sans aller le chercher sur disque ou autre tricherie). *Démonstration :* donné  $e \in \mathbb{N}$ , on considère  $s(e, m)$  tel que  $\varphi_{s(e, m)}(n) = \varphi_e(m, n)$  : le théorème s-m-n (cf. ci-dessus) assure qu'une telle fonction calculable  $(e, m) \mapsto s(e, m)$  existe, et  $(e, n) \mapsto h(s(e, e), n)$  est alors aussi calculable partielle ; il existe donc  $q$  tel que  $\varphi_q(e, n) = h(s(e, e), n)$  : on pose  $p = s(q, q)$ , et on a  $\varphi_p(n) = \varphi_q(q, n) = h(s(q, q), n) = h(p, n)$ , comme annoncé. La non-décidabilité du problème de l'arrêt s'obtient en appliquant (de nouveau par l'absurde) ce résultat à  $h(e, n)$  la fonction qui n'est pas définie si  $\varphi_e(n)$  l'est et qui vaut 42 si  $\varphi_e(n)$  n'est pas définie.

La non-décidabilité du problème de l'arrêt est un résultat fondamental, car très souvent les résultats de non-décidabilité soit sont démontrés sur un modèle semblable, soit s'y ramènent directement : pour montrer qu'un certain ensemble  $A$  (un « problème ») n'est pas décidable, on cherche souvent à montrer que si un algorithme décidant  $A$  existait, on pourrait s'en servir pour construire un algorithme résolvant le problème de l'arrêt.

**Exemple 5.23.** — L'ensemble des  $e \in \mathbb{N}$  tels que la fonction calculable partielle  $\varphi_e$  soit totale (i.e., définie sur tout  $\mathbb{N}$ ) n'est pas semi-décidable. En effet, s'il l'était, d'après ce qu'on a vu, il serait « calculablement énumérable », c'est-à-dire qu'il existerait une fonction calculable  $f: \mathbb{N} \rightarrow \mathbb{N}$  dont l'image soit exactement l'ensemble des  $e$  pour lesquels  $\varphi_e$  est totale, i.e., toute fonction calculable totale s'écrirait sous la forme  $\varphi_{f(k)}$  pour un certain  $k$ . Mais la fonction  $n \mapsto \varphi_{f(n)}(n) + 1$  est calculable totale, donc il devrait exister un  $m$  tel que cette fonction s'écrive  $\varphi_{f(m)}$ , c'est-à-dire  $\varphi_{f(m)}(n) = \varphi_{f(n)}(n) + 1$ , et on aurait alors en particulier  $\varphi_{f(m)}(m) = \varphi_{f(m)}(m) + 1$ , une contradiction. • Son complémentaire, c'est-à-dire l'ensemble des  $e \in \mathbb{N}$  tels que la fonction calculable partielle  $\varphi_e$  ne soit pas totale, n'est pas non plus semi-décidable. En effet, supposons qu'il existe un algorithme qui, donné  $e$ , termine si et seulement si  $\varphi_e$  n'est pas totale. Donnés  $e$  et  $m$ , considérons l'algorithme qui prend une entrée  $n$ , ignore celle-ci, et effectue le calcul  $\varphi_e(m)$  : ceci définit une fonction calculable partielle (soit totale et constante, soit définie nulle part !)  $\varphi_{s(e, m)}$  où  $s$  est calculable (on applique ici le théorème s-m-n) en appliquant à  $s(e, m)$  l'algorithme supposé semi-décider si une fonction récursive partielle est non-totale, on voit qu'ici il semi-décide si  $\varphi_e(m)$  est non-défini, autrement dit on semi-décide le complémentaire du problème de l'arrêt, et on a vu que ce n'était pas possible !

**Exercice 5.24.** — Considérons une fonction  $h$  qui à  $e$  associe un nombre au moins égal au nombre d'étapes (cf. ci-dessus) du calcul de  $\varphi_e(e)$ , si celui-ci termine, et une valeur quelconque si  $\varphi_e(e)$  n'est pas défini. Alors  $h$  n'est pas calculable. (Indication : si elle l'était, on pourrait décider si  $\varphi_e(e)$  est défini en exécutant son calcul pendant

$h(e)$  étapes.) On peut même montrer que  $H(n) := \max\{h(i) : i \leq n\}$  domine asymptotiquement n'importe quelle fonction calculable mais c'est un peu plus difficile.

**Application 5.25.** — Sans rentrer dans les détails de ce que signifie un « système formel », on peut esquisser, au moins informellement, les arguments suivants. Imaginons qu'on ait formalisé la notion de démonstration mathématique (c'est-à-dire qu'on les écrit comme des mots dans un alphabet indiquant quels axiomes et quelles règles logiques sont utilisées) : même sans savoir quelle est exactement la logique formelle, le fait de *vérifier* qu'une démonstration est correcte doit certainement être algorithmique (il s'agit simplement de vérifier que chaque règle a été correctement appliquée), autrement dit, l'ensemble des démonstrations est décidable. L'ensemble des théorèmes, lui, est semi-décidable (on a un algorithme qui semi-décide si un certain énoncé est un théorème en énumérant toutes les chaînes de caractères possibles et en cherchant s'il s'agit d'une démonstration valable dont la conclusion est l'énoncé recherché). Or l'ensemble des théorèmes n'est pas décidable : en effet, si on avait un algorithme qui permet de décider si un énoncé mathématique est un théorème, on pourrait appliquer cet algorithme à l'énoncé formel (\*) « le  $e$ -ième algorithme termine sur l'entrée  $e$  », en observant qu'un tel énoncé, s'il est vrai, est forcément démontrable (i.e., si l'algorithme termine, on peut *démontrer* ce fait en écrivant étape par étape l'exécution de l'algorithme pour constituer une démonstration qu'il a bien été appliqué jusqu'au bout et a terminé), et en espérant que s'il est démontrable alors il est vrai : on aurait alors une façon de décider le problème de l'arrêt, une contradiction. Mais du coup, l'ensemble des non-théorèmes ne peut pas être semi-décidable ; or comme l'ensemble des énoncés  $P$  tels que  $\neg P$  (« non- $P$  », la négation logique de  $P$ ) soit un théorème est semi-décidable (puisque l'ensemble des théorèmes l'est), ils ne peuvent pas coïncider. Ceci montre qu'il existe un énoncé tel que ni  $P$  ni  $\neg P$  ne sont des théorèmes : c'est une forme du *théorème de Gödel* que Turing cherchait à démontrer ; mieux : en appliquant aux énoncés du type (\*), on montre ainsi qu'il existe un algorithme qui *ne termine pas* mais dont la non-terminaison *n'est pas démontrable*. (Modulo quelques hypothèses qui n'ont pas été explicitées sur le système formel dans lequel on travaille.)

## 5.2. Classes de complexité

La section précédente ignorait les questions d'efficacité de calcul. Dans cette section, nous revenons à des considérations sur la complexité.

*Mise en garde importante.* — Tandis que jusqu'à présent, nous nous sommes intéressés à la complexité d'un algorithme donné, nous cherchons à analyser ici la *difficulté* intrinsèque d'un problème, indépendamment de l'algorithme que nous avons découvert pour le résoudre ou même indépendamment du fait que nous sachions réellement résoudre le problème par un algorithme précis.

Comme pour la décidabilité, pour procéder de façon rigoureuse, il nous faudrait formaliser totalement la notion d'algorithme. En général, on emploie des machines de Turing et on compte le nombre de transitions traversées durant l'exécution de l'algorithme. Néanmoins, dans la mesure où les opérations élémentaires introduites

en 1.3.1 sont elles aussi considérées comme s'exécutant en temps constant, y compris si on devait les encoder par des machines de Turing, ici encore, nous continuons de nous appuyer sur la notion de complexité d'un algorithme telle qu'elle a été introduite lors de la définition 1.27.

### 5.2.1. Les classes **P** et **NP**. —

**Définition 5.26.** — On appelle *classe **P*** l'ensemble des problèmes de décisions qui admettent un algorithme de décision dont la complexité en temps dans le pire des cas est polynomiale en la taille de l'instance.

Compte tenu des équivalences que nous avons observées dans la section précédente, nous pourrions aussi définir la classe **P** comme la classe des langages reconnus par une machine de Turing déterministe.

**Exemple 5.27.** — Décider si un nombre est premier appartient à la classe **P**. Le premier test de primalité déterministe est dû à Agrawal, Kayal et Saxena et date de 2002. Dans une version améliorée, il permet de savoir si un entier  $n$  est premier en temps  $O((\log n)^{6+o(1)})$ . Rappelons que pour représenter un entier  $n$ , il faut  $\Theta(\log n)$  bits. La complexité est ici quasi-sextique, ce qui est trop élevé pour que cet algorithme soit utilisé en pratique. Dans l'industrie, on lui préfère des variantes randomisés plus rapides.

**Exemple 5.28.** — Décider si un graphe non orienté est connexe est dans la classe **P**. En effet, il est possible de conclure en effectuant un parcours de graphe, ce qui se fait en temps  $O(n + m)$  où  $n$  et  $m$  désignent le cardinal de l'ensemble des sommets et le cardinal de l'ensemble des arêtes.

**Définition 5.29.** — On appelle *classe **NP*** la classe des problèmes de décision  $A$  que l'on peut vérifier en temps polynomial, c'est-à-dire, tels qu'il existe un algorithme  $\mathcal{A}$  qui prend en entrée une instance  $a$  du problème  $A$  et un certificat  $c$  et qui renvoie une réponse oui ou non (en temps polynomial en  $|a|$ ) de sorte que

- si  $a$  admet la réponse oui au problème  $A$ , alors il existe au moins un certificat  $c_0$  (de taille polynomiale en  $|a|$ ) tel que  $\mathcal{A}(a, c_0) = \text{oui}$ .
- si  $a$  admet la réponse non au problème  $A$ , alors pour tout  $c$  (de taille polynomiale en  $|a|$ ), on a  $\mathcal{A}(a, c) = \text{non}$ .

**Remarque 5.30.** — Le nom de la classe **NP** s'explique par le fait qu'il existe une définition équivalente : les problèmes de la classe **NP** sont aussi les problèmes qui peuvent être résolus par un algorithme polynomial non déterministe.

**Exemple 5.31 (Graphe hamiltonien).** — Étant donné un graphe non orienté  $G = (V, E)$ , le problème du graphe hamiltonien revient à décider si le graphe  $G$  contient un cycle élémentaire  $C$  passant par chaque sommet une et une seule fois. Décider le graphe hamiltonien appartient à la classe **NP**.

En effet, on peut contrôler la réponse oui en fournissant l'instance  $G$  et le certificat candidat  $C$  où  $C$  est une suite d'arête  $C$ . L'algorithme consiste à vérifier que  $C$  passe par chaque sommet une et une seule fois, ce qu'on peut faire en temps polynomial.

**Remarque 5.32.** — Dans la définition ci-dessus, on ne cherche qu'à certifier la réponse OUI. On pourrait également introduire la classe **co-NP** des problèmes de décisions dont le complémentaire appartient à **NP**, autrement dit les problèmes dont on peut certifier la réponse NON.

Noter bien la dissymétrie qu'il y a entre certifier la réponse OUI et certifier la réponse NON comme le montre l'exemple.

**Exemple 5.33.** — Le shérif de Nottingham s'en prend à tous les pauvres qu'il rencontre tandis que Robin des Bois attaque tous les riches qu'il croise. Convaincre le shérif que vous ne méritez pas la bastonnade est possible : il vous suffit de sortir une belle pièce d'or de votre poche ; convaincre Robin que vous ne méritez pas la bastonnade est impossible, quoi que vous disiez, on pourra toujours vous soupçonner de cacher un trésor quelque part.

**Proposition 5.34.** — On a l'inclusion

$$\mathbf{P} \subseteq \mathbf{NP}.$$

*Démonstration.* — Soit  $A$  un problème dans la classe **P**. Il existe donc un algorithme  $\mathcal{A}_0$  qui décide  $A$ , c'est-à-dire dont la spécification est comme suit :

---

**Algorithme 5.1 :**  $\mathcal{A}_0$  (décision de  $A$ )

---

**Entrées :** Instance  $a$  de  $A$

**Sorties :** Oui si l'instance  $a$  admet oui comme réponse au problème  $A$  ; Non sinon

---

Considérons le nouvel algorithme  $\mathcal{A}$

---

**Algorithme 5.2 :**  $\mathcal{A}$  (vérification de  $A$ )

---

**Entrées :** Instance  $a$  de  $A$ , certificat candidat  $c$  de  $a$

**Sorties :** Oui si l'instance  $a$  admet oui comme réponse au problème  $A$  ; non sinon (quel que soit le certificat)

1 retourner  $\mathcal{A}_0(a)$

---

Avec cet algorithme  $\mathcal{A}$ , nous avons bien construit un vérificateur pour lequel n'importe quelle entrée peut servir de certificat du problème  $A$ . Ceci prouver que  $A$  appartient à la classe **NP**.  $\square$

### 5.2.2. Comparer la difficulté. —

**Définition 5.35.** — Un problème de décision  $A$  est *polynomialement réductible* à un problème de décision  $B$  s'il existe un algorithme qui, pour toute instance  $a$  du problème  $A$ , transforme  $a$  en une instance  $b$  du problème  $B$  en temps polynomial et tel que  $a$  admet la réponse oui au problème  $A$  si et seulement si  $b$  admet la réponse oui au problème  $B$ .

On note dans ce cas  $A \prec B$ .

**Remarque 5.36.** — Il faut interpréter le fait que  $A \prec B$  comme «  $A$  est au plus aussi difficile que  $B$ , modulo des opérations d'exécution en temps polynomial ».

**Problème 5.37 (Ensemble indépendant).** — Étant donné un graphe  $G = (V, E)$  et un entier  $k$ , existe-t-il un sous-ensemble de sommets  $I \subseteq V$  de taille  $\geq k$  tel que deux sommets quelconques de  $I$  ne sont jamais reliés ?

**Problème 5.38 (Clique).** — Étant donné un graphe  $G = (V, E)$  et un entier  $k$ , existe-t-il un sous-ensemble de sommets  $C \subseteq V$  de taille  $\geq k$  tel que deux sommets quelconques de  $C$  sont toujours reliés ?

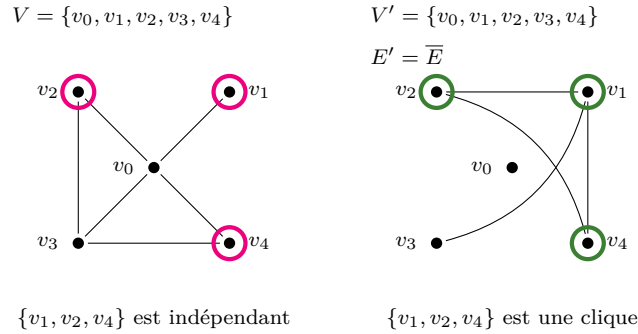
**Exemple 5.39.** — On a

$$\text{ENSEMBLE INDÉPENDANT} \prec \text{CLIQUE}$$

et

$$\text{CLIQUE} \prec \text{ENSEMBLE INDÉPENDANT}.$$

En effet, pour construire une réduction polynomiale, on peut le graphe complémentaire.



On remarque alors qu'un ensemble indépendant est une clique dans le graphe complémentaire.

**Problème 5.40 (Sommet-Couverture).** — Étant donné un graphe  $G = (V, E)$  et un entier  $k$ , existe-t-il un sous-ensemble de sommets  $C \subseteq V$  de taille  $\leq k$  tel que toute arête admet une extrémité dans  $C$  ?

**Exemple 5.41.** — On a

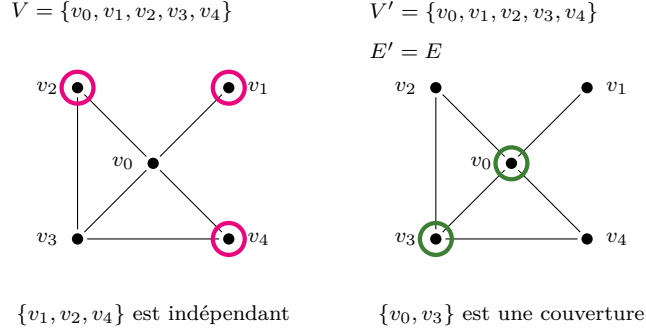
$$\text{ENSEMBLE INDÉPENDANT} \prec \text{SOMMET-COUVERTURE}$$

et

$$\text{SOMMET-COUVERTURE} \prec \text{ENSEMBLE INDÉPENDANT}.$$

En effet, pour construire une réduction polynomiale, on peut observer qu'un ensemble  $I$  est indépendant si et seulement si  $C = V \setminus I$  est une couverture.





### 5.3. Les problèmes NP-complets

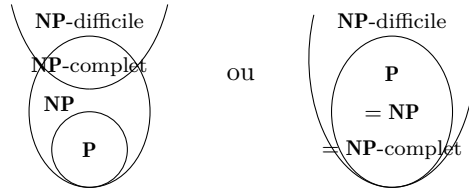
#### 5.3.1. Définition. —

**Définition 5.42.** — Un problème de décision  $A$  est **NP-difficile** si pour tout problème  $\Pi \in \mathbf{NP}$ , on a  $\Pi \prec A$ .

Un problème de décision  $A$  est **NP-complet** s'il est **NP-difficile** et appartient à **NP**.

Les problèmes **NP-complet** sont les plus difficiles parmi les problèmes de la classe **NP**.

**Remarque 5.43.** — Savoir si  $\mathbf{P} = \mathbf{NP}$  ou si l'on a une inclusion stricte est un problème ouvert. On peut toutefois affirmer que : s'il existe un problème **NP-complet** qui appartient à  $\mathbf{P}$ , alors  $\mathbf{P} = \mathbf{NP}$ . Inversement, l'inclusion est stricte, alors les problèmes **NP-complet** ne peuvent pas appartenir à  $\mathbf{P}$ .



**5.3.2. Un premier problème NP-complet.** — Les formules de logique ont été définies en 2.75. Nous précisons un format particulier de formules.

**Définition 5.44.** — Soient des variables booléennes  $\mathcal{V} = \{x_1, x_2, \dots, x_n\}$ , on définit :

- les *littéraux* comme les expressions suivantes  $x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$ ,
- les *clauses disjonctives* comme des disjonctions de littéraux de la forme  $\lambda_1 \vee \lambda_2 \vee \dots \vee \lambda_t$  (avec  $t$  littéraux distincts).

On parle alors de *formule de logique sous forme normale conjonctive* lorsque la formule se présente comme une conjonction de clauses disjonctives  $C_1 \wedge C_2 \wedge \dots \wedge C_s$ .

**Problème 5.45 (Sat).** — Le problème SAT (ou *satisfiabilité d'une formule de logique sous forme normale conjonctive*) est le problème de décision suivant : étant donnée une formule  $\Phi$  sous forme normale conjonctive, existe-t-il un modèle, c-à-d une valuation des variables (cf définition 2.105) telle que l'interprétation de  $\Phi$  soit la valeur de vérité « vrai » ?

**Exemple 5.46.** — La formule

$$\Phi = \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee \neg x_2 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_3} \wedge \underbrace{(x_1 \vee x_2 \vee x_4)}_{C_4}$$

est satisfiable. On peut utiliser la valuation

|       |     |
|-------|-----|
| $x_1$ | $V$ |
| $x_2$ | $F$ |
| $x_3$ | $V$ |
| $x_4$ | $V$ |

comme modèle.

**Théorème 5.47 (Cook - Levin).** — *Le problème SAT est NP-complet.*

*Esquisse de preuve.* — On commence par observer que SAT appartient à la classe **NP**. En effet, on peut utiliser une valuation de chaque variable à une valeur de vérité et contrôler en temps polynomial qu'il s'agit d'un modèle.

Soit  $A$  un problème dans la classe **NP** et  $\mathcal{A}(a, c)$  un algorithme qui vérifie le certificat  $c$  d'une instance  $a$ . Alors, on peut construire une formule de logique qui « simule »  $\mathcal{A}$  et est satisfaite ss'il existe un certificat  $c$  tel que  $\mathcal{A}(a, c)$  est vrai. On en déduit

$$A \preceq \text{SAT}$$

comme attendu. □

La preuve complète est beaucoup plus longue et demande plus d'efforts pour être exposée.

**Remarque 5.48.** — Tous les autres résultats de NP-complétude sont déduits du théorème de Cook-Levin.

**Problème 5.49 (3-Sat).** — Le problème SAT (ou *satisfiabilité d'une formule de logique sous forme normale conjonctive*) est le problème de décision suivant : étant donnée une formule  $\Phi$  sous forme normale conjonctive dont toutes les clauses ont exactement 3 littéraux, existe-t-il un modèle, c-à-d une valuation des variables telle que l'interprétation de  $\Phi$  soit la valeur de vérité « vrai » ?

**Lemme 5.50.** — *Le problème 3-SAT est NP-complet.*

*Démonstration.* — Nous nous proposons de démontrer que

$$\text{SAT} \preceq 3\text{-SAT}$$

Autrement dit, nous nous donnons une instance de SAT, c'est-à-dire une formule logique  $\Phi$  sous forme normale conjonctive, et nous devons construire en temps polynomial une nouvelle formule  $\Phi'$  qui est satisfiable si et seulement si  $\Phi$  est satisfiable.

Pour chaque clause  $C$ , de la forme  $C = \lambda_1 \vee \lambda_2 \vee \dots \vee \lambda_t$  de  $\Phi$ , nous procédons comme suit, en fonction du nombre de littéraux  $t$  que la clause contient.

Si  $t \geq 4$ , nous introduisons une variable booléenne fraîche, disons  $y$ , et nous remplaçons la clause  $C$  par les deux clauses

$$(\lambda_1 \vee \dots \vee \lambda_{t-2} \vee y) \wedge (\lambda_{t-1} \vee \lambda_t \vee \neg y)$$

et notons  $\Phi'$  la nouvelle formule. Si la formule  $\Phi$  admet un modèle  $\alpha : \mathcal{V} \rightarrow \mathbb{B}$ , alors, au moins un des littéral de  $C$  s'évalue à Vrai. S'il s'agit d'un littéral parmi  $\lambda_1, \dots, \lambda_{t-2}$ , on peut étendre  $\alpha$  en posant  $\alpha(y) = F$  et  $\alpha$  est encore un modèle de  $\Phi'$ . S'il s'agit d'un littéral parmi  $\lambda_{t-1}$  ou  $\lambda_t$ , on peut étendre  $\alpha$  en posant  $\alpha(y) = V$  et  $\alpha$  est encore un modèle de  $\Phi'$ . Inversement, si  $\alpha' : \mathcal{V} \cup \{y\} \rightarrow \mathbb{B}$  est un modèle de  $\Phi'$ , selon la valeur de  $\alpha'(y)$ , on peut établir que  $\lambda_1 \vee \dots \vee \lambda_{t-2}$  ou  $\lambda_{t-1} \vee \lambda_t$  est satisfait, ce qui montre que  $\alpha'$  est un modèle de  $\Phi$ . Si  $t \leq 2$ , nous introduisons une variable booléenne fraîche, disons encore  $y$ , et nous remplaçons la clause  $C$  par les deux clauses

$$(C \vee y) \wedge (C \vee \neg y)$$

Nous pourrions noter

$$v(\Phi) = \sum_{C \text{ clause de } \Phi} |\text{longueur}(C) - 3|.$$

Lorsque  $v(\Phi) = 0$ , la formule est ne possède que des clauses de longueur 3. Nous constatons que  $v(\Phi)$  est un entier qui décroît d'une unité à chaque transformation. Autrement dit, nous pouvons utiliser  $v(\Phi)$  comme variant qui montre que le processus de transformation des clauses se termine après  $v(\Phi)$  étapes. Chaque étape se calcule en temps linéaire. Ainsi, en temps polynomial, nous sommes capable de transformer  $\Phi$  en une instance de 3-SAT.  $\square$

**Théorème 5.51.** — *Le problème ENSEMBLE INDÉPENDANT (voir problème 5.37) est NP-complet.*

*Démonstration.* — Nous nous proposons de démontrer que

$$3\text{-SAT} \prec \text{ENSEMBLE INDÉPENDANT}$$

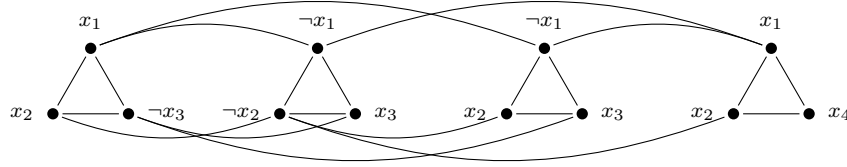
Autrement dit, nous nous donnons une instance de 3-SAT, c'est-à-dire une formule logique  $\Phi$  sous forme normale conjonctive dont les clauses sont de longueur 3, et nous devons construire en temps polynomial un graphe  $G$  et un seuil  $k$  tel que  $\Phi$  est satisfiable si et seulement si le graphe  $G$  possède un ensemble de sommets indépendants de cardinal  $\geq k$ .

Pour chaque clause  $\lambda_1 \vee \lambda_2 \vee \lambda_3$  de  $\Phi$ , nous construisons un triangle, c'est-à-dire trois sommets d'étiquette respective  $\lambda_1, \lambda_2$  et  $\lambda_3$  reliés entre eux par des arêtes. Ceci fait, nous ajoutons des arêtes pour relier tout couple de sommet d'étiquette ayant des littéraux opposés.

Par exemple, avec

$$\Phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4)$$

on introduit le graphe



Nous nous donnons comme seuil  $k$  le nombre de clause de  $\Phi$ .

Nous affirmons les deux faits suivants :

1. toute valuation avec  $\geq t$  clauses valides donne un ensemble indépendant de plus de  $t$  sommets et
2. tout ensemble indépendant de plus de  $t$  sommets mène à un modèle de la formule.

Pour le fait 1, soit  $\alpha : \mathcal{V} \rightarrow \mathbb{B}$  un modèle de  $\Phi$ . Pour chaque clause de  $\Phi$ , au moins un littéral est satisfait. On sélectionne ce littéral. On construit ainsi un ensemble  $I$  de  $t$  sommets. Comme il y a un sommet par triangle, aucune arête de triangle ne peut être couverte par deux sommets de  $I$ . Par ailleurs, par construction, aucune arête reliant des littéraux opposés ne peut être couverte par ses deux extrémités. Donc  $I$  est bien un ensemble indépendant.

Pour le fait 2, soit  $I$  un ensemble indépendant de plus de  $t$  sommets. À cause des triangles, il y a un seul sommet par triangle. À cause des arêtes reliant deux littéraux opposés,  $I$  ne contient pas de sommets ayant des étiquettes opposées. On fixe une valuation des littéraux qui apparaissent dans  $I$  de sorte que ceux-ci soient satisfaits. On complète cette valuation de manière quelconque. On a obtenu un modèle de  $\Phi$ .  $\square$

**5.3.3. Problèmes de décision vs problèmes d'optimisation.** — Les classes de complexité que nous avons vues sont réservées aux problèmes de décision.

**Définition 5.52.** — Un *problème d'optimisation* consiste à déterminer une solution réalisable d'une instance d'un problème qui maximise ou minimise une certaine fonction objectif.

Les problèmes suivants, que nous avons déjà rencontré dans ce cours, sont des problèmes d'optimisation.

**Problème 5.53 (Plus court chemin).** — Étant donné un graphe  $G = (V, E)$  pondéré, deux sommets  $s$  et  $t$  de  $V$ , déterminer le plus court chemin entre  $s$  et  $t$ .

**Problème 5.54 (Arbre couvrant de poids minimum)**

Étant donné un graphe  $G = (V, E)$  pondéré, trouver un arbre couvrant dont le poids est minimum.

**Problème 5.55 (Voyageur · euse de commerce).** — Étant donné un graphe  $G = (V, E)$  pondéré, trouver un tour hamiltonien de coût minimum.

**Problème 5.56 (Coupe maximum).** — Étant donné un graphe  $G = (V, E)$ , trouver une coupe  $(S, \bar{S})$  qui maximise le nombre d'arêtes coupées  $|E(S, \bar{S})|$ .

Tout problème d'optimisation  $A_{opt}$  peut être transformé en un problème de décision  $A_d$  comme suit.

|                                                                                                                                                                                                                |                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problème de minimisation <math>A_{opt}</math></b><br>Étant donné une instance $a$ d'un problème $A_{opt}$ , trouver $x$ parmi les solutions réalisables de $a$ qui minimise la fonction objectif $f_a(x)$ . | <b>Problème de décision <math>A_d</math> associé</b><br>Étant donné une instance $a$ d'un problème $A_{opt}$ et un seuil numérique $k$ , existe-t-il une solution réalisable $x$ tel que $f_a(x) \leq k$ . |
| <b>Problème de maximisation <math>A_{opt}</math></b><br>Étant donné une instance $a$ d'un problème $A_{opt}$ , trouver $x$ parmi les solutions réalisables de $a$ qui maximise la fonction objectif $f_a(x)$ . | <b>Problème de décision <math>A_d</math> associé</b><br>Étant donné une instance $a$ d'un problème $A_{opt}$ et un seuil numérique $k$ , existe-t-il une solution réalisable $x$ tel que $f_a(x) \geq k$ . |

**Exemple 5.57.** — Voici comment les problème ci-dessus se transforment en un problème de décision.

1. Étant donné un graphe  $G = (V, E)$  pondéré, deux sommets  $s$  et  $t$  de  $V$  et un seuil  $\ell$ , existe-t-il un chemin entre  $s$  et  $t$  de longueur  $\leq \ell$  ?
2. Étant donné un graphe  $G = (V, E)$  pondéré et un seuil  $p$ , existe-t-il un arbre couvrant de poids  $\leq p$  ?
3. Étant donné un graphe  $G = (V, E)$  pondéré et un seuil  $c$ , existe-t-il un tour hamiltonien de coût  $\leq c$  ?
4. Étant donné un graphe  $G = (V, E)$  et un seuil  $k$ , existe-t-il une coupe  $(S, \bar{S})$  qui coupe plus de  $k$  arêtes ?

Par abus de langage, on dit qu'un problème d'optimisation est **NP**-difficile si son problème de décision associé est lui-même **NP**-difficile.

#### 5.4. Exercices

**Exercice 5.58.** — On rappelle la définition du problème de l'arrêt : c'est l'ensemble  $H$  des couples <sup>(8)</sup>  $(e, x)$  formés d'un programme (=algorithme)  $e$  et d'une entrée  $x$ , tels que l'exécution du programme  $e$  sur l'entrée  $x$  termine en temps fini. On a vu en cours que  $H$  était semi-décidable mais non décidable <sup>(9)</sup>.

On considère l'ensemble  $H'$  des triplets  $(e_1, e_2, x)$  tels que  $(e_1, x) \in H$  ou bien  $(e_2, x) \in H$  (ou les deux). Autrement dit, au moins l'un des programmes  $e_i$  termine en temps fini quand on l'exécute sur l'entrée  $x$ .

- (1) L'ensemble  $H'$  est-il semi-décidable ?
- (2) L'ensemble  $H'$  est-il décidable ?

8. Pour être rigoureux, on a fixé un codage permettant de représenter les programmes  $e$ , les entrées  $x$  à ces programmes, et les couples  $(e, x)$ , comme des éléments de  $\mathbb{N}$  (ou bien de  $\Sigma^*$  sur un alphabet  $\Sigma \neq \emptyset$  arbitraire). Il n'est pas nécessaire de faire apparaître ce codage dans la description des algorithmes proposés, qui peut rester informelle.

9. Une variante consiste à considérer l'ensemble des  $e$  tels que  $e$  termine sur l'entrée  $e$  : l'ensemble  $H$  qu'on vient de définir s'y ramène facilement.

**Exercice 5.59.** — Soit  $\Sigma$  un alphabet (fini, non vide) fixé. Les questions suivantes sont indépendantes (mais on remarquera leur parallélisme). Ne pas hésiter à décrire les algorithmes de façon succincte et informelle.

(1) Expliquer, au moyen des résultats vus en cours, pourquoi il existe un algorithme  $A_1$  qui, étant donnée une expression rationnelle  $r$  sur  $\Sigma$ , décide si le langage  $L_r$  dénoté par  $r$  est différent du langage  $\Sigma^*$  de tous les mots sur  $\Sigma$ . (Autrement dit, l'algorithme  $A_1$  doit prendre en entrée une expression rationnelle  $r$ , terminer en temps fini, et répondre « vrai » s'il existe un mot  $w \in \Sigma^*$  tel que  $w \notin L_r$  et « faux » si  $L_r = \Sigma^*$ . On ne demande pas que l'algorithme soit efficace.)

(2) Expliquer pourquoi il existe un algorithme  $A_2$  comme suit : on lui fournit en entrée un algorithme  $T$  qui décide un langage  $L_T \subseteq \Sigma^*$  (c'est-à-dire que  $T$  termine toujours en temps fini quand on lui présente un mot sur  $\Sigma$ , et répond « vrai » ou « faux », et  $L_T$  est le langage des mots sur lesquels il répond « vrai »), et l'algorithme  $A_2$  doit semi-décider si  $L_T$  est différent de  $\Sigma^*$ . (C'est-à-dire que  $A_2$  doit terminer en répondant « vrai » s'il existe un mot  $w \in \Sigma^*$  tel que  $w \notin L_T$ , et ne pas terminer si  $L_T = \Sigma^*$ .)

(3) Expliquer pourquoi il *n'existe pas* d'algorithme  $A_3$  qui, dans les mêmes conditions que  $A_2$ , décide (au lieu de seulement semi-décider) si  $L_T$  est différent de  $\Sigma^*$ . (C'est-à-dire que  $A_3$  est censé terminer toujours, et répondre « vrai » s'il existe un mot  $w \in \Sigma^*$  tel que  $w \notin L_T$ , et « faux » si  $L_T = \Sigma^*$ .) Indication : expliquer comment on pourrait utiliser un tel  $A_3$  pour résoudre le problème de l'arrêt, en cherchant à fabriquer un  $T$  qui rejette un mot précisément si un programme donné s'arrête.

**Exercice 5.60.** — (1) Soit  $\Sigma$  un alphabet (i.e., un ensemble fini). L'ensemble  $L = \{u^k : u \in \Sigma^*, k \geq 2\}$  des mots qui sont une puissance  $k$ -ième pour un  $k \geq 2$  est-il décidable ? Semi-décidable ?

(2) L'ensemble des  $e \in \mathbb{N}$  tels que l'exécution du  $e$ -ième programme (ou, si on préfère, de la  $e$ -ième machine de Turing), exécuté sur l'entrée 42, termine en au plus  $10^{1000+e}$  étapes est-il décidable ? Semi-décidable ?

(3) L'ensemble des  $e \in \mathbb{N}$  tels que l'exécution du  $e$ -ième programme (ou, si on préfère, de la  $e$ -ième machine de Turing), exécuté sur l'entrée 42, termine en temps fini est-il décidable ? Semi-décidable ? (On pourra montrer qu'on peut y ramener le problème de l'arrêt.)

**Exercice 5.61.** — Soit  $\Sigma$  un alphabet (i.e., un ensemble fini). On s'intéresse à des langages sur  $\Sigma$ .

(A) Montrer que si deux langages  $L_1$  et  $L_2$  sont décidables, alors  $L_1 \cup L_2$  et  $L_1 \cap L_2$  et  $L_1 L_2$  sont décidables ; montrer que si un langage  $L$  est décidable alors  $L^*$  est décidable (pour ce dernier, on pourra commencer par chercher, si  $w \in \Sigma^*$  est un mot de longueur  $n$ , comment énumérer toutes les façons de le factoriser en mots de longueur non nulle).

(B) Montrer que si deux langages  $L_1$  et  $L_2$  sont semi-décidables, alors  $L_1 \cup L_2$  et  $L_1 \cap L_2$  et  $L_1 L_2$  sont semi-décidables ; montrer que si un langage  $L$  est semi-décidable alors  $L^*$  est semi-décidable.

**Exercice 5.62.** — On rappelle qu'une fonction  $f: \mathbb{N} \rightarrow \mathbb{N}$  est dite *calculable* lorsqu'il existe un algorithme (par exemple, un programme pour une machine de Turing) prenant en entrée un  $n \in \mathbb{N}$  qui termine toujours en temps fini et renvoie la valeur  $f(n)$ . On rappelle qu'une partie  $E$  de  $\mathbb{N}$  ou de  $\mathbb{N}^2$  est dite *décidable* lorsque sa fonction indicatrice est calculable, ou, ce qui revient au même, lorsqu'il existe un algorithme prenant en entrée un élément de  $\mathbb{N}$  ou de  $\mathbb{N}^2$  qui termine toujours en temps fini et renvoie vrai (1) ou faux (0) selon que l'élément fourni appartient ou non à  $E$ . On rappelle enfin qu'une partie  $E$  de  $\mathbb{N}$  ou de  $\mathbb{N}^2$  est dite *semi-décidable* lorsqu'il existe un algorithme prenant en entrée un élément de  $\mathbb{N}$  ou de  $\mathbb{N}^2$  qui termine toujours en temps fini et renvoie vrai (1) si l'élément fourni appartient à  $E$ , et sinon ne termine pas (on peut aussi accepter qu'il termine en renvoyant faux, cela ne change rien).

Soit  $f: \mathbb{N} \rightarrow \mathbb{N}$  : montrer qu'il y a équivalence entre les affirmations suivantes :

1. la fonction  $f$  est calculable,
2. le graphe  $\Gamma_f := \{(n, f(n)) : n \in \mathbb{N}\} = \{(n, p) \in \mathbb{N}^2 : p = f(n)\}$  de  $f$  est décidable,
3. le graphe  $\Gamma_f$  de  $f$  est semi-décidable.

(Montrer que (3) implique (1) est le plus difficile : on pourra commencer par s'entraîner en montrant que (2) implique (1). Pour montrer que (3) implique (2), on pourra chercher une façon de tester en parallèle un nombre croissant de valeurs de  $p$  de manière à s'arrêter si l'une quelconque convient.)

**Exercice 5.63.** — Soit  $A \subseteq \mathbb{N}$  un ensemble infini. Montrer qu'il y a équivalence entre :

- l'ensemble  $A$  est décidable,
- il existe une fonction calculable *strictement croissante*  $f: \mathbb{N} \rightarrow \mathbb{N}$  telle que  $f(\mathbb{N}) = A$ .

**Exercice 5.64.** — Soit  $S(e, n)$  le nombre d'étapes de l'exécution du  $e$ -ième programme (ou, si on préfère, de la  $e$ -ième machine de Turing) quand on lui fournit le nombre  $n$  en entrée, à supposer que cette exécution termine ; sinon,  $S(e, n)$  n'est pas défini.

Soit par ailleurs  $M(k)$  le maximum des  $S(e, n)$  pour  $0 \leq e \leq k$  et  $0 \leq n \leq k$  qui soient définis (et 0 si aucun d'eux n'est défini). Autrement dit, il s'agit du plus petit entier supérieur ou égal au nombre d'étapes de l'exécution de l'un des programmes  $0 \leq e \leq k$  sur l'un des entiers  $0 \leq n \leq k$  en entrée, lorsqu'ils terminent.

Montrer que la fonction  $M$  n'est pas calculable (i.e., n'est pas calculable par un algorithme) : on pourra pour cela montrer que la connaissance de  $M$  permet de résoudre le problème de l'arrêt. Montrer même qu'*aucune* fonction  $M'$  telle que  $M'(k) \geq M(k)$  pour tout  $k$  n'est calculable. Montrer que même si  $M'$  vérifie simplement  $M'(k) \geq M(k)$  pour  $k \geq k_0$ , alors  $M'$  n'est pas calculable.

*Remarque :* La fonction  $M$ , ou différentes variantes de celle-ci, s'appelle fonction du « castor affairé ». On peut montrer encore plus fort : si  $F$  est une fonction calculable quelconque, alors il existe  $k_0$  tel que  $M(k) \geq F(k)$  pour  $k \geq k_0$  (autrement dit, la fonction  $M$  finit par dépasser n'importe quelle fonction calculable : Radó, 1962, *On Non-Computable Functions*).

**Exercice 5.65.** — Dans cet exercice, on s'intéresse au langage  $L$  formé des programmes  $e$  (codés, dans une formalisation quelconque de la calculabilité<sup>(10)</sup>, comme des entiers naturels ou comme des mots sur un alphabet fixé sans importance) qui, quel que soit le paramètre  $n$  qu'on leur fournit en entrée, terminent en temps fini et retournent la valeur 0 : soit  $L = \{e : (\forall n)\varphi_e(n) \downarrow = 0\}$ . Si l'on préfère,  $L$  est l'ensemble de toutes les façons de coder la fonction constante égale à 0. On appellera aussi  $M$  le complémentaire de  $L$ . On se demande si  $L$  ou  $M$  sont semi-décidables.

(1) Thésée et Hippolyte se disputent pour savoir si  $L$  est semi-décidable. Thésée pense qu'il l'est, et il tient le raisonnement suivant pour l'expliquer :

Pour savoir si un programme  $e$  est dans l'ensemble  $L$ , il suffit de l'examiner pour vérifier que toutes les instructions qui mettent fin au programme renvoient la valeur 0 : si c'est le cas, on termine en répondant « oui » (c'est-à-dire  $e \in L$ ) ; sinon, on rentre dans une boucle infinie. Ceci fournit un algorithme qui semi-décide  $L$ .

Hippolyte, elle, pense que  $L$  n'est pas semi-décidable. Son argument est le suivant :

Si  $L$  était semi-décidable, je pourrais m'en servir pour résoudre le problème de l'arrêt. En effet, donné un programme  $e'$  et une entrée  $m$  sur laquelle je cherche à tester l'arrêt de  $e'$ , je peux fabriquer le programme  $e$  qui prend une entrée  $n$ , lance l'exécution de  $e'$  sur l'entrée  $m$  pendant au plus  $n$  étapes et à la fin renvoie 1 si ces  $n$  étapes ont suffi à terminer l'exécution de  $e'$ , et 0 sinon. Dire que  $e \in L$  signifie que  $e'$  ne termine jamais sur  $m$ , donc pouvoir semi-décider  $L$  permet de résoudre algorithmiquement le problème de l'arrêt, ce qui est impossible.

Qui a raison ? Expliquer précisément quelle est l'erreur (ou les erreurs) commise(s) par le raisonnement incorrect, et détailler les éventuels passages incomplets dans le raisonnement correct.

(2) Achille et Patrocle se disputent pour savoir si  $M$  (le complémentaire de  $L$ ) est semi-décidable. Achille pense qu'il l'est, et il tient le raisonnement suivant pour l'expliquer :

Pour savoir si un programme  $e$  est dans l'ensemble  $M$ , il suffit de tester successivement les valeurs  $\varphi_e(n)$  pour tous les  $n$  possibles : si l'on rencontre un  $n$  tel que  $\varphi_e(n)$  n'est pas 0, alors on termine en répondant « oui » (c'est-à-dire  $e \in M$ ) ; sinon, on ne va jamais terminer, et cela signifie que  $e \notin M$ . Ceci fournit un algorithme qui semi-décide  $M$ .

Patrocle, lui, pense que  $M$  n'est pas semi-décidable. Son argument est le suivant :

Si  $M$  était semi-décidable, je pourrais m'en servir pour résoudre le problème de l'arrêt. En effet, donné un programme  $e'$  et une entrée  $m$  sur laquelle je cherche à tester l'arrêt de  $e'$ , je peux fabriquer le programme  $e$  qui prend une entrée  $n$ , l'ignore purement et simplement, exécute  $e'$  sur l'entrée  $m$  et à la fin renvoie 0. Dire que  $e \in M$  signifie que  $e'$  ne termine pas sur  $m$ , donc pouvoir semi-décider  $M$  permet de résoudre algorithmiquement le problème de l'arrêt, ce qui est impossible.

10. Par exemple, un langage de programmation (Turing-complet) quelconque.



Qui a raison ? Expliquer précisément quelle est l'erreur (ou les erreurs) commise(s) par le raisonnement incorrect, et détailler les éventuels passages incomplets dans le raisonnement correct.

**Exercice 5.66.** — On rappelle qu'une *clique* dans un graphe  $G = (V, E)$  est un sous-ensemble  $S \subseteq V$  tel que  $\{(u, v) \in S \times S; u \neq v\} \subseteq E$ , i.e. le graphe restreint à  $S$  est complet. On parle de  $k$ -clique si  $|S| = k$ . Nous donnons les deux problèmes suivants :

**CLIQUE :**

**Entrée :** Un graphe non-orienté  $G$  et un entier  $k$ .

**Sortie :** Vrai si  $G$  contient une clique de taille  $\geq k$ , faux sinon.

**DEMICLIQUE :**

**Entrée :** Un graphe non-orienté  $G$ .

**Sortie :** Vrai si  $G$  contient une clique de taille  $\geq n/2$ , faux sinon.

Nous souhaitons démontrer que

$$\text{CLIQUE} \preceq_P \text{DEMICLIQUE}$$

c'est à dire trouver une réduction polynomiale qui transforme une entrée de CLIQUE en une entrée de DEMICLIQUE

1. Soit  $G = (V, E)$  un graphe et  $f_1(G)$  le graphe déduit de  $G$  par ajout d'un sommet connecté à tous les autres. Montrer que  $G$  contient une  $k$ -clique ssi  $f_1(G)$  contient une  $(k+1)$ -clique.
2. Soit  $a \in \mathbb{N}$ . Construire un graphe  $f_a(G)$  tel que  $G$  contient une  $k$ -clique ssi  $f_a(G)$  contient une  $(k+a)$ -clique.
3. Soit  $n = |V|$  l'ordre de  $G$ . Supposons que  $k \leq n/2$ . Trouver  $a$  tel que  $G$  contient une  $k$ -clique ssi  $G' = f_a(G)$  contient une  $n'/2$ -clique où  $n'$  est l'ordre de  $G'$ .
4. Supposons que  $k > n/2$ . Trouver un graphe  $G'$  tel que  $G$  contient une  $k$ -clique ssi  $G'$  contient une  $n'/2$ -clique.
5. En déduire que  $\text{CLIQUE} \preceq_P \text{DEMICLIQUE}$ .

**Exercice 5.67.** — Trouver une réduction polynômiale de CHEMINHAMILTONIEN vers COURTCHEMINELEMENTAIRE. Le problème CHEMINHAMILTONIEN étant NP-complet, qu'en déduisez-vous de COURTCHEMINELEMENTAIRE ?

**CHEMINHAMILTONIEN :**

**Entrée :** Un graphe non-orienté  $G$ .

**Sortie :** Vrai si'il existe un chemin passant une et une seule fois par chaque sommet. Faux sinon

**COURTCHEMINELEMENTAIRE :**

**Entrée :** Un graphe non-orienté  $G = (V, E)$  pondéré ( $p : E \rightarrow \mathbb{Q}$ ), deux sommets  $s$  et  $t$  et une longueur  $\ell$ .

**Sortie :** Vrai s'il existe un chemin élémentaire entre  $s$  et  $t$  de longueur  $\leq \ell$ . Faux sinon.

**Exercice 5.68.** — Le problème NAE-3-SAT est le problème suivant :

NAE-3-SAT :

Entrée : Une formule  $F = C_1 \wedge \dots \wedge C_t$  où  $C_j$  est la disjonction de 3 littéraux.

Sortie : Décide si  $F$  est NAE-satisfiable, c'est-à-dire s'il existe une valuation telle que toute clause  $C_j = \lambda \vee \mu \vee \nu$  de  $F$  est satisfaite et de plus les valeurs de  $\lambda, \mu, \nu$  apparaissant dans la clause ne sont pas toutes égales.

Le sigle NAE désigne en anglais la locution « not all equal ».

1. Donner l'ensemble des modèles de la formule

$$F_0 = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3),$$

c'est-à-dire l'ensemble de valuations des 3 variables telles que l'interprétation de  $F_0$  soit  $V$ . Indiquer quels modèles vérifient de plus le critère NAE.

2. Trouver une clause  $C_0$  à trois littéraux telle que

$$F_1 = F_0 \wedge C_0$$

soit satisfiable mais ne vérifie pas NAE-3-SAT.

3. Soit  $\Phi = C_1 \wedge \dots \wedge C_t$  une formule dont les clauses  $C_j$  comportent trois littéraux. On introduit  $t + 1$  nouvelles variables :  $(w_j)_{j \leq t}$  et  $z$  et on forme, pour  $C_j = \lambda \vee \mu \vee \nu$ , les clauses

$$C'_j = \lambda \vee \mu \vee w_j \quad \text{et} \quad C''_j = \nu \vee \neg w_j \vee z.$$

Montrer que la formule  $\Phi$  est satisfiable si et seulement si la formule

$$\Phi' = C'_1 \wedge C''_1 \wedge C'_2 \wedge C''_2 \wedge \dots \wedge C'_t \wedge C''_t$$

est NAE-satisfiable

4. Montrer que NAE-3-SAT est NP-complet.

**Exercice 5.69.** — Une coupe dans un graphe  $G = (V, E)$  est une partition de l'ensemble des sommets  $V = S \sqcup \bar{S}$ . Le problème COUPE est le problème de décision suivant :

COUPE :

Entrée : Un graphe  $G$  et un entier  $k$ .

Sortie : Décide si  $G$  admet une coupe  $S \sqcup \bar{S}$  telle qu'il y a plus de  $k$  arêtes entre  $S$  et  $\bar{S}$ .

1. Quel est le problème d'optimisation associé ?
2. Montrer que COUPE appartient à  $NP$ .
3. Soit  $F$  une formule de logique, instance de NAE-3-SAT (cf. exercice 5.68) sur les variables  $(x_i)_{i \leq n}$  possédant  $m$  clauses. On considère le multi-graphe  $G$  possédant  $2n$  sommets  $\{x_1, \neg x_1, \dots, x_n, \neg x_n\}$ . Pour chaque clause  $C = \alpha \vee \beta \vee \gamma$ , on ajoute le triangle  $[\alpha, \beta, \gamma]$  aux arêtes de  $G$ . Pour chaque  $i \leq n$ , soit  $n_i$  le nombre d'occurrence de  $x_i$  ou de  $\neg x_i$ . On ajoute  $n_i$  arêtes entre  $x_i$  et  $\neg x_i$ .
  - (a) Compter le nombre d'arêtes de  $G$ .
  - (b) Montrer que  $G$  possède une coupe de capacité  $\geq 5m$  ssi  $F$  est satisfaisable.
4. Montrer que  $\text{NAE-3-SAT} \leq_P \text{COUPE}$ .

5. Conclure.

**Exercice 5.70.** — Un ensemble indédendent dans un graphe  $G = (V, E)$  est un sous-ensemble de sommets  $S \subseteq V$  non reliés entre eux, *i.e.* tel que  $E \cap S \times S = \emptyset$ . Le problème ENSEMBLEINDÉPENDENT est le problème de décision suivant :

ENSEMBLEINDÉPENDENT :

Entrée : Un graphe  $G$  et un entier  $k$ .

Sortie : Décide si  $G$  contient un ensemble indépendant de cardinal  $\geq k$ .

1. Quel est le problème d'optimisation associé ?
2. Montrer que ENSEMBLEINDÉPENDENT appartient à  $NP$ .
3. Montrer que  $3 - \text{SAT} \preceq_P \text{ENSEMBLEINDÉPENDENT}$ .
4. Conclure.

**Exercice 5.71.** — Une clique dans un graphe  $G = (V, E)$  est un sous-ensemble de sommets  $S \subseteq V$  tous reliés entre eux. Le problème CLIQUE est le problème de décision suivant :

CLIQUE :

Entrée : Un graphe  $G$  et un entier  $k$ .

Sortie : Décide si  $G$  contient une clique de cardinal  $\geq k$ .

1. Quel est le problème d'optimisation associé ?
2. Montrer que CLIQUE appartient à  $NP$ .
3. Montrer que  $\text{ENSEMBLEINDÉPENDENT} \preceq_P \text{CLIQUE}$ .
4. Conclure.
- 5.

**Exercice 5.72.** — Une couverture par des sommets dans un graphe  $G = (V, E)$  est un sous-ensemble de sommets  $S \subseteq V$  tel que toute arête est incidente à l'un des sommets de  $S$ . Le problème SOMMETCOUVERTURE est le problème de décision suivant :

SOMMETCOUVERTURE :

Entrée : Un graphe  $G$  et un entier  $k$ .

Sortie : Décide si  $G$  contient une couverture par des sommets de cardinal  $\leq k$ .

1. Quel est le problème d'optimisation associé ?
2. Montrer que SOMMETCOUVERTURE appartient à  $NP$ .
3. Montrer que  $\text{ENSEMBLEINDÉPENDENT} \preceq_P \text{SOMMETCOUVERTURE}$ .
4. Conclure.

**Exercice 5.73.** — Le problème de la SOUSSOMME est le problème de décision suivant :

SOUSSOMME :

Entrée : Un entier  $p$ , une suite d'entiers  $A = (a_k)_{1 \leq k \leq p}$ , un entier  $T$ .

Sortie : Décide s'il existe un ensemble d'indices  $K \subseteq \llbracket 1, p \rrbracket$  tels que  $\sum_{k \in K} a_k = T$ .

1. Montrer que SOUSOMME appartient à  $NP$ .
2. Soit  $G = (V, E)$  un graphe et  $t$  un entier. Nous numérotons les sommets de  $V$  entre 1 et  $n$ . Nous numérotons les arêtes de  $E$  entre 1 et  $m$ . Nous définissons, pour tous  $i \in \llbracket 1, m \rrbracket$  et  $j \in \llbracket 1, n \rrbracket$ ,

$$b_{i,j} = \begin{cases} 1 & \text{si l'arête } i \text{ est incidente au sommet } j \\ 0 & \text{sinon} \end{cases}$$

Nous posons, pour tout  $j \in \llbracket 1, n \rrbracket$ ,

$$a_j = 4^m + \sum_{i=0}^{m-1} b_{i,j} 4^i$$

et pour tout  $i \in \llbracket 1, m \rrbracket$ ,

$$b_i = 4^i.$$

Nous posons  $A = \{a_j, j \in \llbracket 1, n \rrbracket\} \cup \{b_i, i \in \llbracket 1, m \rrbracket\}$  et

$$T = t \cdot 4^m + \sum_{i=0}^{m-1} 2 \cdot 4^i.$$

Montrer que  $\text{SOMMETCOUVERTURE} \preceq_P \text{SOUSOMME}$ .

3. Conclure.

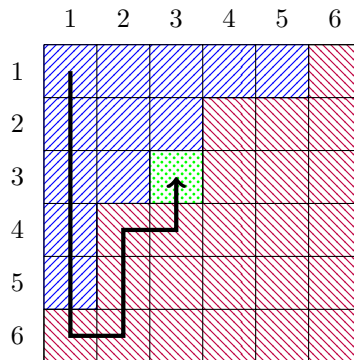
## CHAPITRE 6

### ÉLÉMENTS DE CORRIGÉ

La majorité des exercices dispose d'un corrigé, non imprimé ici, qui se trouve dans la version en ligne du polycopié (consulter le site Ecampus du cours).

#### 6.1. Concevoir un programme

**Solution 1.62.** On remarque que si  $a_i + a_j < x$ , alors  $a_{i'} + a_{j'} < x$  pour tout  $i' \leq i$  et  $j' \leq j$ . De même, si  $a_i + a_j > x$ , alors  $a_{i'} + a_{j'} > x$  pour tout  $i' \geq i$  et  $j' \geq j$ . Ainsi, pour tout couple  $(i, j)$  testé, toute une zone d'indices peut être éliminée. On peut représenter la situation comme suit ainsi que le parcours que l'on souhaite programmer parmi les indices :



On peut déjà déduire du dessin que le parcours a une longueur inférieure à  $3n$ .

On peut programmer les choses comme suit

```
1292 def RechercheCouple(S,x):
1293     S.sort()
1294     n = len(S)
1295     def test(i,j):
1296         return (S[i]+S[j] - x)
1297     if test(0,0) > 0 or test(n-1,n-1) < 0:
```

```

1298         return "Impossible"
1299     if test(0,0) == 0:
1300         return (0,0)
1301     i = 0
1302     j = 0
1303     def mouvement():
1304         nonlocal i,j
1305         if test(i,j) < 0 and i < n-1:
1306             i = i+1
1307             return True
1308         elif test(i,j) < 0 and j < n-1:
1309             j = j+1
1310             return True
1311         elif test(i,j) > 0 and i > 0 and test(i-1,j) >= 0:
1312             i = i-1
1313             return True
1314         elif test(i,j) > 0 and j < n-1:
1315             j = j+1
1316             return True
1317         return False
1318     while mouvement():
1319         if test(i,j) == 0:
1320             return (i,j)
1321 
```

L'algorithme est bien linéaire en temps, car dans la première boucle  $i \leq n$ , donc il y a au plus  $n$  étapes de coût  $O(1)$  par étape. Par la suite, dans chaque itération ou bien  $i$  ou bien  $j$  est modifiée avec  $0 \leq i \leq n$  et  $0 \leq j \leq n$ , ce qui fait  $2n$  itérations au plus.

**Solution 1.63.** On suppose que Roméo se déplace à vitesse 1. Tant que Roméo n'a pas trouvé Juliette, il peut explorer successivement chacune des directions sur une distance  $(t_i)_{i \geq 1}$  et revenir sur ses pas.

Pour que Roméo rencontre Juliette, il faut que la suite vérifie  $\sup_i t_i \geq d$ , autrement dit il faut que  $\limsup_{i \rightarrow +\infty} t_i = +\infty$ . Nous pouvons supposer sans perte de généralité que la suite est strictement croissante. Soit  $i$  tel que  $t_i \leq d < t_{i+1}$ , alors l'algorithme détecte donc Juliette en temps  $\leq 8 \sum_{k=1}^{i+1} t_k$ . Nous choisissons  $(t_i)$  tel que d'une part  $t_{i+1} = \Theta(t_i)$ , ce qui garantit que  $d$  et  $t_i$  soient du même ordre de grandeur, et  $t_{i+1} = \Theta\left(\sum_{k=1}^{i+1} t_k\right)$ , ce qui garantit que le temps reste linéaire. On peut par exemple prendre la suite  $t_i = 2^i$ .

**Solution 1.64.** À chaque question "X connaît-il Y ?" on reçoit la réponse oui ou non. Dans le premier cas, X n'est pas une célébrité, dans le second, Y n'en est pas une. On part de la liste des invités et on ne pose que des questions concernant les éléments de cette liste. Chaque question permet d'éliminer une personne de la liste. En  $n - 1$

étapes, il est possible de réduire l'ensemble des invités à une seule potentielle célébrité. Il reste à vérifier le statut de cette personne, ce qui coûte  $2(n-1)$  questions.

**Solution 1.65.** Supposons que l'on accède au cours de l'algorithme à la lecture du  $k$ -ième bit. S'il lit un 0, lire des bits avant le  $k$ -ième ne lui apportera aucune information supplémentaire. De même s'il lit un 1, lire des bits après le  $k$ -ième ne lui apportera aucune information supplémentaire. Tout algorithme optimal doit donc commencer par une suite de lecture de bit en position  $\kappa = (k_i)_{i \geq 1}$  où  $k_i$  est une suite croissante. Tant que l'algorithme lit des 0, il continue à utiliser la suite  $\kappa$  pour choisir le bit suivant à lire.

Dès lors que le bit n°  $k_{i+1}$  est un 1, retrouver  $n$  revient à rechercher un élément dans une liste finie triée et coûte au minimum  $\Omega(\log(k_{i+1} - k_i))$  (par une recherche dichotomique, dont on peut voir l'optimalité en dessinant un arbre). Le tout est donc de trouver une suite qui soit suffisamment croissante pour que trouver le premier 1 soit rapide, mais pas trop croissante, pour que repérer  $n$  entre  $k_i$  et  $k_{i+1}$  ne soit pas trop long.

Deux cas peuvent se produire.

- Si la suite vérifie  $k_{i+1} \leq C \cdot k_i$  pour une constant  $C$  fixée, alors  $k_i \leq C^i$ , si bien qu'il faut compter  $\Omega(\log n)$  pour dépasser  $n$  et ensuite  $\Omega(\log(k_{i+1} - k_i))$  pour retrouver  $n$ .
- Si  $\limsup k_{i+1}/k_i = +\infty$  (i.e. pour une sous-suite, on a  $\lim \frac{k_{i_j+1}}{k_{i_j}} = +\infty$ ), alors dépasser  $n$  peut être rapide, mais ensuite, pour le cas  $n = k_{i_j} + 1$ , il faut effectuer une recherche dichotomique qui coûte  $\Omega\left(\log\left(k_{i_j+1} \times \left(\frac{k_{i_j+1}}{k_{i_j}} - 1\right)\right)\right)$ , autrement dit, au moins  $\Omega(\log n)$ .

Dans tous les cas, la complexité est  $\Omega(\log n)$ . Mais si on choisit  $k_i = 2^i$ , alors  $k_{i+1} - k_i = 2^{i+1} - 2^i = k_i$  et donc la complexité reste en  $O(\log n)$ .

**Solution 1.66.** Il s'agit de la fonction nulle TODO : preuve de correction !

**Solution 1.67.** On a

$$\forall x \in \mathbb{N}, \quad f(x, y) = \begin{cases} x - y & \text{si } x \leq y \\ \infty & \text{sinon} \end{cases}$$

où le symbole  $\infty$  indique que le calcul ne s'arrête pas.

La fonction  $g$  renvoie le reste de la division euclidienne de  $x$  par  $y$ .

La fonction  $h$  renvoie le pgcd de  $x$  et de  $y$ .

TODO : preuve de correction !

**Solution 1.68.** Le programme `maccarthy91` calcule la fonction

$$\forall x \in \mathbb{N}, \quad f(x) = \begin{cases} 91 & \text{si } n \leq 101 \\ x - 10 & \text{si } n \geq 102 \end{cases}$$

Elle n'est pas définie ailleurs.

TODO : preuve de correction !

**Solution 1.69.** On peut considérer la fonction  $\mu : (x, y) \mapsto 0$ . Néanmoins, le programme `morris` ne s'arrête pas, car il se rappelle systématiquement avec les mêmes paramètres d'entrée.

**Solution 1.70.**

| $f(n)$       | $c$ | $f_c^*(n)$                      |
|--------------|-----|---------------------------------|
| $n-1$        | 0   | $n$                             |
| $n/2$        | 1   | $\lceil \log_2 n \rceil$        |
| $n/2$        | 2   | $\lceil \log_2 n \rceil - 1$    |
| $\sqrt{n}$   | 2   | $\lceil \log_2 \log_2 n \rceil$ |
| $\sqrt{n}$   | 1   | $+\infty$                       |
| $n^{1/3}$    | 2   | $\lceil \log_3 \log_2 n \rceil$ |
| $n/\log_2 n$ | 2   |                                 |

**Solution 1.71.** On obtient :

$$\begin{aligned}
 & 1/n, \underbrace{1, n^{1/\log_2 n} = 2, (\log_2 n)^{1/\log_2 n}, \ln \ln n, \sqrt{\log_2 n}, \ln n,}_{\log_2^2 n, 2^{\sqrt{2 \log_2 n}}, n/\log_2 n, n, \underbrace{n \log_2 n \sim \log_2(n!), n^2, n^3,}} \\
 & (\sqrt{2})^{\log_2 n}, (\log_2 n)!, 2^{\log_2 n}, 4^{\log_2 n}, \\
 & \underbrace{n^{\log_2 \log_2 n} \sim (\log_2 n)^{\log_2 n}, \left(\frac{3}{2}\right)^n, 2^n, n \cdot 2^n,}_{e^n, n!, (n+1)!, \underbrace{2^{2^n}, 2^{2^n+1}, 2^{2^{n+1}}}
 \end{aligned}$$

**Solution 1.72.** Le calcul donne les nombres suivants. À titre de comparaison, on estime à  $10^{80}$  le nombre d'atomes dans l'univers.

| $f \backslash t$ | 1 sec.                  | 1 min.                    | 1 h.                        | 1 jour                       |
|------------------|-------------------------|---------------------------|-----------------------------|------------------------------|
| $\log_2 n$       | $9.9 \cdot 10^{301029}$ | $5.5 \cdot 10^{18061799}$ | $2.4 \cdot 10^{1083707984}$ | $2.3 \cdot 10^{26008991625}$ |
| $\sqrt{n}$       | $1 \cdot 10^{12}$       | $3.6 \cdot 10^{15}$       | $1.3 \cdot 10^{19}$         | $7.5 \cdot 10^{21}$          |
| $n$              | $10^6$                  | $6 \cdot 10^7$            | $3,6 \cdot 10^9$            | $8,64 \cdot 10^{10}$         |
| $n \log_2 n$     | 62746                   | 2801417                   | 13378058                    | 2755147513                   |
| $n^2$            | $1 \cdot 10^3$          | $7.7 \cdot 10^3$          | $6 \cdot 10^4$              | $2.9 \cdot 10^5$             |
| $n^3$            | 100                     | 391                       | 1532                        | 4420                         |
| $2^n$            | 19                      | 25                        | 31                          | 36                           |
| $n!$             | 10                      | 12                        | 13                          | 14                           |

| $f \backslash t$ | 1 mois                        | 1 an                           | 1 siècle                         |
|------------------|-------------------------------|--------------------------------|----------------------------------|
| $\log_2 n$       | $1.1 \cdot 10^{780269748761}$ | $2.0 \cdot 10^{9493281943259}$ | $1.2 \cdot 10^{949328194325931}$ |
| $\sqrt{n}$       | $6.7 \cdot 10^{24}$           | $9.9 \cdot 10^{26}$            | $9.9 \cdot 10^{30}$              |
| $n$              | $2,60 \cdot 10^{12}$          | $3,15 \cdot 10^{13}$           | $3,15 \cdot 10^{15}$             |
| $n \log_2 n$     | $7.1 \cdot 10^{10}$           | $8.0 \cdot 10^{11}$            | $6.9 \cdot 10^{13}$              |
| $n^2$            | $1.6 \cdot 10^6$              | $5.6 \cdot 10^6$               | $5.6 \cdot 10^7$                 |
| $n^3$            | 13736                         | 31593                          | 146645                           |
| $2^n$            | 41                            | 44                             | 51                               |
| $n!$             | 16                            | 17                             | 18                               |



## 6.2. Structures de données

**Solution 2.173.** Sens direct. Supposons qu'il existe une suite infinie  $\mathbf{u} = (u_n)_{n \in \mathbb{N}} \subseteq E$  satisfaisant, pour tout  $n \in \mathbb{N}$ ,  $u_{n+1} < u_n$ . Alors l'ensemble  $A = \{u_n \in E, n \in \mathbb{N}\}$  est une partie non-vide de  $E$ . Par hypothèse sur  $E$ , elle admet un élément minimal, disons  $u_{n_0}$  pour un certain  $n_0 \in \mathbb{N}$ . Mais, on doit avoir  $u_{n_0+1} < u_{n_0}$ , ce qui contredit la minimalité de  $u_{n_0}$ .

Sens retour : Nous démontrons la contraposée. Soit  $A$  une partie non-vide de  $E$ . Montrons que si  $A$  ne contient pas d'élément minimal, alors on peut construire par récurrence une suite infinie  $\mathbf{u} = (u_n)_{n \in \mathbb{N}} \subseteq E$  de termes strictement décroissants. Comme  $A$  est non-vide, il existe un élément  $u_0$  dans  $A$ . De plus, si l'on a construit un certain élément  $u_n$ , comme  $u_n$  n'est pas minimal, il existe un élément strictement inférieure à  $u_n$  que nous appelons  $u_{n+1}$ .

**Solution 2.174.** Réponse : non. Dans cette définition, tous les sommets ont soit deux fils soit zéro fils (on parle d'*arbres pleins*). Si l'on reprend l'exemple du cours, les sommets d'étiquette « Esther » ou « Jakob » ne peuvent pas se représenter.

**Solution 2.175.** On propose le système de règles suivantes

$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (Ax)} \qquad \frac{w \in \mathcal{X}}{w\sigma_1\sigma_2 \in \mathcal{X}} \text{ (R}_{\sigma_1\sigma_2}\text{)}$$

où  $\sigma_1$  et  $\sigma_2$  parcourent  $\Sigma^2$ . Il y a au total  $1 + |\Sigma|^2$  règles.

Montrons par preuve par induction structurelle que  $\mathcal{X} \subseteq \mathcal{L}$ . Pour cela, nous constatons que le mot vide introduit par la règle (Ax) est de longueur paire. D'autre part, si  $\sigma_1$  et  $\sigma_2$  appartiennent à  $\Sigma^2$  et si  $w$  est un mot de longueur  $|w|$  paire, alors  $w' = w\sigma_1\sigma_2$  est un mot de longueur  $|w'| = |w| + 2$  qui est aussi paire. Par conséquent, tout mot de  $\mathcal{X}$  est de longueur paire.

Montrons par preuve par récurrence sur les entiers que  $\mathcal{L} \subseteq \mathcal{X}$ . Le prédicat  $\mathcal{P}(n)$  que nous montrons est le suivant : «  $\mathcal{P}(n)$  : tout mot de  $\mathcal{L}$  de longueur  $2n$  appartient à  $\mathcal{X}$  ». Pour  $n = 0$ , le seul mot de longueur nulle est le mot vide, qui est introduit par l'axiome (Ax). Donc  $\mathcal{P}(0)$  est satisfait. Supposons avoir démontré  $\mathcal{P}(n)$  pour un certain entier  $n \in \mathbb{N}$  et démontrons  $\mathcal{P}(n+1)$ . Soit  $w$  un mot de  $\mathcal{L}$  de longueur  $2n+2$ . Notons  $w'$  le préfixe de  $w$  de longueur  $2n$  d'une part et  $\sigma_1$  et  $\sigma_2$  les deux dernières lettres de  $w$  d'autre part. Comme  $w$  est de longueur  $2n$ ,  $w$  appartient à  $\mathcal{L}$ . Mais alors, à cause de la règle  $(R_{\sigma_1\sigma_2})$ ,  $w = w'\sigma_1\sigma_2$  appartient aussi à  $\mathcal{L}$ . En conclusion,  $\mathcal{P}(n)$  est vrai pour tout entier  $n$ .

Les deux inclusions montrent que  $\mathcal{L} = \mathcal{X}$ , ce que nous voulions démontrer.

**Solution 2.176.** On propose le système de règles

$$\frac{}{0 \in \mathcal{X}} \text{ (Ax)} \qquad \frac{w \in \mathcal{X}}{1w \in \mathcal{X}} \text{ (R1)} \qquad \frac{w \neq 0 \in \mathcal{X} \quad w' \in \mathcal{X}}{ww' \in \mathcal{X}} \text{ (Ext)}$$

TODO : démontrer que  $\mathcal{X} = \mathcal{R}$ .

**Solution 2.177.** Indications : Partir de

$$\frac{}{\varepsilon \in \mathcal{Y}} \text{ (Vide)} \qquad \frac{}{0 \in \mathcal{Y}} \text{ (Zéro)} \qquad \frac{w \in \mathcal{Y}}{w1 \in \mathcal{Y}} \text{ (Ext 1)} \qquad \frac{w \in \mathcal{Y}}{w10 \in \mathcal{Y}} \text{ (Ext 10)}$$

Montrer par induction structurelle que  $\mathcal{Y} \subseteq \mathcal{Z}$ . Montrer par récurrence sur la longueur des mots que  $\mathcal{Z} \subseteq \mathcal{Y}$ . Cette définition est non ambiguë.

Autre solution possible :

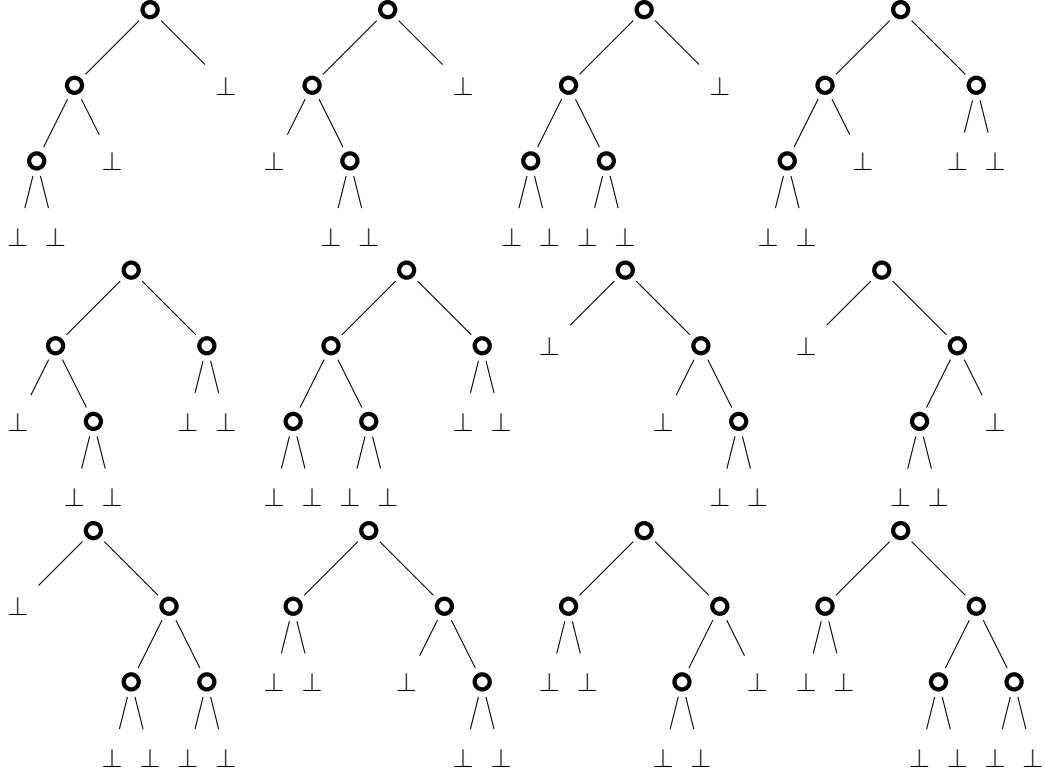
$$\frac{}{\varepsilon \in \mathcal{X}} \text{ (Vide)}$$

$$\frac{}{0 \in \mathcal{X}} \text{ (Zéro)}$$

$$\frac{w \in \mathcal{X} \quad w' \in \mathcal{X}}{w1w' \in \mathcal{X}} \text{ (Ext)}$$

Cette définition est ambiguë.

**Solution 2.179.** Voici tous les cas possibles :



**Solution 2.180.** On démontre le résultat par induction structurelle. Nous passons en revue les trois règles.

Règle (Z) : Si  $e = z$ , on a  $|z|_z = 1$ ,  $|z|_t = 0$  et  $|z|_b = 0$ . Ainsi

$$|z|_z = 2 \cdot |z|_t + |z|_b + 1.$$

Règle (B) : Soient  $w_1$  et  $w_2$  deux mots de  $\mathcal{G}$  tels que

$$\begin{cases} |w_1|_z &= 2 \cdot |w_1|_t + |w_1|_b + 1 \\ |w_2|_z &= 2 \cdot |w_2|_t + |w_2|_b + 1. \end{cases}$$

Appelons  $w$  le mot  $w = b(w_1 | w_2)$ . On a

$$\begin{aligned} |w|_z &= |w_1|_z + |w_2|_z \\ |w|_b &= 1 + |w_1|_b + |w_2|_b \\ |w|_t &= |w_1|_t + |w_2|_t \end{aligned}$$

Mais alors

$$\begin{aligned} |w|_z - 2 \cdot |w|_t - |w|_b - 1 &= |w_1|_z + |w_2|_z - 2(|w_1|_t + |w_2|_t) - (1 + |w_1|_b + |w_2|_b) - 1 \\ &= 0 \end{aligned}$$

Règle (Z) : Soient  $w_1$ ,  $w_2$  et  $w_3$  deux mots de  $\mathcal{G}$  tels que

$$\begin{cases} |w_1|_z &= 2 \cdot |w_1|_t + |w_1|_b + 1 \\ |w_2|_z &= 2 \cdot |w_2|_t + |w_2|_b + 1 \\ |w_3|_z &= 2 \cdot |w_3|_t + |w_3|_b + 1. \end{cases}$$

Appelons  $w$  le mot  $w = \mathbf{t}(w_1 | w_2 | w_3)$ . On a

$$\begin{aligned} |w|_z &= |w_1|_z + |w_2|_z + |w_3|_z \\ |w|_b &= |w_1|_b + |w_2|_b + |w_3|_b \\ |w|_t &= 1 + |w_1|_t + |w_2|_t + |w_3|_t \end{aligned}$$

Mais alors

$$\begin{aligned} |w|_z - 2 \cdot |w|_t - |w|_b - 1 &= |w_1|_z + |w_2|_z + |w_3|_z \\ &\quad - 2(1 + |w_1|_t + |w_2|_t + |w_3|_t) \\ &\quad - (|w_1|_b + |w_2|_b + |w_3|_b) - 1 \\ &= |w_1|_z - 2 \cdot |w_1|_t - |w_1|_b - 1 \\ &\quad + |w_2|_z - 2 \cdot |w_2|_t - |w_2|_b - 1 \\ &\quad + |w_3|_z - 2 \cdot |w_3|_t - |w_3|_b - 1 \\ &= 0 \end{aligned}$$

Comme l'égalité que nous voulons démontrer passe aux trois règles, d'après le théorème de démonstration par induction structurale, l'égalité est vraie pour tout mot de  $\mathcal{G}$ .

**Solution 2.181.** Nous fixons la liste  $\ell_2$  et nous raisonnons par induction structurale (sur la construction de la liste  $\ell_1$ ).

Cas où  $\ell_1 = \perp$  : On a alors  $\text{Concat}_{\ell_2}(\ell_1) = \ell_2$ . Donc  $\text{Long}(\text{Concat}_{\ell_2}(\ell_1)) = \text{Long}(\ell_2)$ . Or, par ailleurs,  $\text{Long}(\ell_1) = \text{Long}(\perp) = 0$ . Donc la relation est bien satisfaite.

Cas où  $\ell_1 = (e : q)$  : Notre hypothèse est que  $\text{Long}(\text{Concat}_{\ell_2}(q)) = \text{Long}(q) + \text{Long}(\ell_2)$ .

On a alors  $\text{Concat}_{\ell_2}(\ell_1) = (e : \text{Concat}_{\ell_2}(q))$ . Mais alors

$$\begin{aligned} \text{Long}(\text{Concat}_{\ell_2}(\ell_1)) &= \text{Long}((e : \text{Concat}_{\ell_2}(q))) \\ &= 1 + \text{Long}(\text{Concat}_{\ell_2}(q)) \\ &= 1 + \text{Long}(q) + \text{Long}(\ell_2) \\ &= \text{Long}(e : q) + \text{Long}(\ell_2) \\ &= \text{Long}(\ell_1) + \text{Long}(\ell_2) \end{aligned}$$

Les deux règles étant couvertes, nous pouvons conclure que la formule est vraie pour toute liste  $\ell_1$ .

**Solution 2.182.** Soient  $f(a)$  le nombre de feuille et  $n(a)$  le nombre de sommets internes. Nous affirmons que

$$f(a) \leq n(a) + 1.$$

Nous démontrons la relation par induction structurelle.

Dans le cas de l'arbre vide  $a = \perp$ , il n'y a rien à démontrer puisque  $n(a) = f(a) = 0$ .

Dans le cas où l'arbre  $a = (\perp | \perp)$  est une feuille, on a  $n(a) = 0$  et  $f(a) = 1$ . Donc  $f(a) \leq n(a) + 1$ .

Dans le cas où l'arbre  $a = (g | d)$  avec  $g \neq \perp$  ou  $d \neq \perp$ , on  $f(a) = f(g) + f(d)$  et  $n(a) = 1 + n(g) + n(d)$ . Donc

$$f(a) = f(g) + f(d) \leq n(g) + 1 + n(d) + 1 = n(a) + 1.$$

**Solution 2.183.** On définit le cardinal d'un arbre par les règles :

$$\begin{array}{c} \overline{\text{card}(\perp) = 0} \quad (\text{vide}) \\[1em] \overline{\begin{array}{cc} g \in \mathcal{A} & d \in \mathcal{A} \end{array}} \quad (\text{cons.}) \\ \text{card}((e | g | d)) = 1 + \text{card}(g) + \text{card}(d) \end{array}$$

On propose la relation

$$\text{haut}(a) + 1 \leq \text{card}(a) \leq 2^{\text{haut}(a)+1} - 1.$$

Nous démontrons cette relation en raisonnant par induction structurelle.

Cas de base :  $a = \perp$ . On a  $\text{haut}(a) = -1$  et  $\text{card} = 0$ . Vérifier la relation revient à constater que

$$0 = (-1) + 1 \leq 0 \leq 2^{-1+1} - 1 = 0.$$

Hérédité. Soient  $g$  et  $d$  deux arbres binaires tels que

$$\text{haut}(g) + 1 \leq \text{card}(g) \leq 2^{\text{haut}(g)+1} - 1$$

et

$$\text{haut}(d) + 1 \leq \text{card}(d) \leq 2^{\text{haut}(d)+1} - 1.$$

Appelons  $a$  l'arbre  $a = (g | d)$ . Par définition des deux fonctions, on a les relations.

$$\begin{cases} \text{haut}(a) &= \max(\text{haut}(g) + 1, \text{haut}(d) + 1) \\ \text{card}(a) &= 1 + \text{card}(g) + \text{card}(d) \end{cases}$$

Nous commençons par observer que

$$\begin{aligned} \text{haut}(a) + 1 &= \max(\underbrace{\text{haut}(g) + 1}_{\geq 0}, \underbrace{\text{haut}(d) + 1}_{\geq 0}) + 1 \\ &\leq 1 + \text{haut}(g) + 1 + \text{haut}(d) + 1 \\ &\leq 1 + \text{card}(g) + \text{card}(d) \\ &= \text{card}(a). \end{aligned}$$

Par ailleurs,

$$\begin{aligned} \text{card}(a) &= 1 + \text{card}(g) + \text{card}(d) \\ &\leq 1 + 2^{\text{haut}(g)+1} - 1 + 2^{\text{haut}(d)+1} - 1 \\ &= 2 \cdot (2^{\text{haut}(g)} + 2^{\text{haut}(d)}) - 1 \\ &\leq 2 \cdot (2 \cdot 2^{\max(\text{haut}(g), \text{haut}(d))}) - 1 \\ &= 2^{\text{haut}(a)+1} - 1. \end{aligned}$$

Ainsi, l'encadrement proposé est vrai pour l'axiome et pour la règle de construction. D'après le théorème de preuve par induction structurelle, la relation proposée est vraie pour n'importe quel arbre binaire.

**Solution 2.184.** Soit  $\rho(h)$  le nombre d'arbres binaires de hauteur  $h$ . Un arbre  $a$  de hauteur  $h+1$  est forcément de la forme  $a = (g|d)$  où  $g$  et  $d$  sont des arbres de hauteur  $h$ , ou bien où  $g$  est un arbre de hauteur  $h$  et  $d$  est de hauteur comprise entre  $-1$  et  $h-1$ , ou bien où  $d$  est un arbre de hauteur comprise entre  $-1$  et  $h-1$  et  $g$  est de hauteur  $h$ .

En conséquence

$$\rho(h) = \begin{cases} 1 & \text{si } h = -1 \\ \rho(h-1)^2 + 2 \sum_{t=-1}^{h-2} \rho(h)\rho(t) & \text{si } h \geq 0 \end{cases}$$

LISTING 6.1. Nombre d'arbres binaires de hauteur  $h$

```

1322 def treecount(h):
1323     T = [ 0 ] * (h+2)
1324     T[0] = 1
1325     for t in range(1,h+2):
1326         T[t] = T[t-1] * T[t-1]
1327         for i in range(t-1):
1328             T[t] += 2 * T[t-1] * T[i]
1329     return T

```

Voir aussi <https://oeis.org/A001699>.

**Solution 2.185.** Nous considérons l'alphabet formé des quatre syboles  $\Sigma = \{1, 2, 3, +\}$  et définissons  $\mathcal{S}$  comme une partie de  $\Sigma^*$  avec les règles suivantes :

$$\frac{}{1 \in \mathcal{S}} \text{ (Ax 1)} \quad \frac{}{2 \in \mathcal{S}} \text{ (Ax 2)} \quad \frac{}{3 \in \mathcal{S}} \text{ (Ax 3)} \quad \frac{y \in \mathcal{S} \quad z \in \mathcal{S}}{y+z \in \mathcal{S}} \text{ (Som)}$$

L'ambiguïté provient du faire que le mot  $w = 1+2+3$  admet plus d'une dérivation. En voici deux distinctes. D'une part :

$$\frac{\frac{}{1 \in \mathcal{S}} \text{ (Ax 1)} \quad \frac{\frac{}{2 \in \mathcal{S}} \text{ (Ax 2)}}{1+2 \in \mathcal{S}} \text{ (Som)}}{1+2+3 \in \mathcal{S}} \text{ (Ax 3)} \quad \frac{}{3 \in \mathcal{S}} \text{ (Som)}$$

D'autre part :

$$\frac{\frac{}{1 \in \mathcal{S}} \text{ (Ax 1)} \quad \frac{\frac{}{2+3 \in \mathcal{S}} \text{ (Som)}}{1+2+3 \in \mathcal{S}} \text{ (Ax 3)} \quad \frac{}{3 \in \mathcal{S}} \text{ (Som)}$$

**Solution 2.186.** 1. La liste contient [63, 70, 46, 28, 23, 34, 26]

2. La liste contient les éléments [91, 43, 40, 70, 27, 57, 24, 95]

3. Il y a un conflit de mémoire car le 7eme maillon ( $[24, 97]$ ) de la liste se trouve sur les cases 147 - 148 qui sont également utilisées par le tableau.

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 36  | 80  | 82  | 72  | 83  | 81  | 65  | 98  | 21  | 57  | 147 | 69  | 70  | 55  | 19  | 84  |
| 48  | 49  | 50  | 51  | 52  | 53  | 54  | 55  | 56  | 57  | 58  | 59  | 60  | 61  | 62  | 63  |
| 34  | 47  | 80  | 43  | 87  | 80  | 97  | 67  | 76  | 91  | 23  | 20  | 95  | 26  | 0   | 77  |
| 64  | 65  | 66  | 67  | 68  | 69  | 70  | 71  | 72  | 73  | 74  | 75  | 76  | 77  | 78  | 79  |
| 28  | 27  | 57  | 34  | 79  | 1   | 56  | 40  | 173 | 75  | 76  | 20  | 84  | 72  | 80  | 55  |
| 80  | 81  | 82  | 83  | 84  | 85  | 86  | 87  | 88  | 89  | 90  | 91  | 92  | 93  | 94  | 95  |
| 9   | 95  | 0   | 7   | 24  | 95  | 28  | 9   | 68  | 3   | 5   | 49  | 8   | 32  | 20  | 92  |
| 96  | 97  | 98  | 99  | 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 |
| 93  | 16  | 2   | 20  | 91  | 91  | 67  | 12  | 91  | 47  | 51  | 55  | 28  | 65  | 36  | 31  |
| 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 |
| 76  | 4   | 0   | 52  | 2   | 41  | 27  | 90  | 18  | 29  | 20  | 7   | 63  | 70  | 46  | 28  |
| 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 |
| 23  | 34  | 26  | 24  | 97  | 23  | 28  | 80  | 3   | 11  | 18  | 27  | 98  | 56  | 27  | 14  |
| 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 |
| 54  | 52  | 88  | 0   | 14  | 93  | 46  | 23  | 39  | 62  | 41  | 45  | 64  | 70  | 81  | 55  |
| 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 |

**Solution 2.188.** On considère deux pointeurs **lievre** et **tortue** qui parcourent les maillons de la liste à des vitesses différentes : **lievre** ← **lievre.suivant** à chaque étape et **tortue** ← **tortue.suivant** toutes les étapes paires. À chaque étape, on compare les deux pointeurs. S'ils pointent vers la même adresse, c'est que la liste se boucle sur elle-même. Si **lievre** pointe vers l'adresse vide, c'est que la liste s'arrête.

Ce programme fonctionne car si la liste est finie, **lievre** arrive à la fin avant **tortue**. Il n'y a pas d'erreur possible.

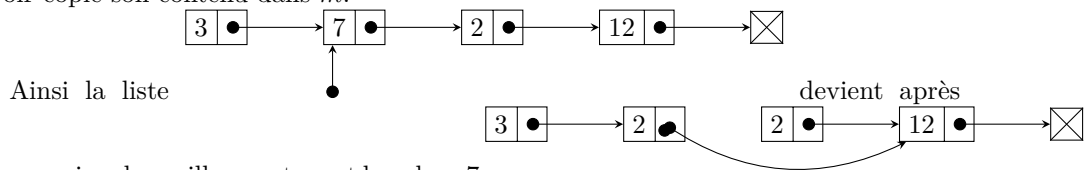
Preuve de correction. Si la liste est finie, le programme le détecte en  $n$  étapes. Si la liste boucle, notons  $p$  la longueur de la prépériode et  $q$  celle de la période ( $p + q = n$ ). Il faut  $2p$  étapes pour que le pointeur **tortue** entre dans le cycle. Si  $e$  désigne l'écart entre **lievre** et **tortue** au moment où le pointeur **tortue** entre dans le cycle, après  $2e$  étapes, **lievre** a rattrapé **tortue**, les deux pointeurs sont égaux et le programme s'arrête après  $2p + 2e$  étape. Comme  $p, q \leq n$ , sa complexité est  $O(n)$ .

**Solution 2.189.** Idée : On considère deux pointeurs **lent** et **rapide**. Le premier se déplace d'un élément par étape, le second de deux éléments par étapes. Lorsque

**rapide** atteint la fin de la liste, **lent** est au niveau du milieu de la liste.

**Solution 2.190.** On parcourt la liste en renversant les pointeurs comme dans l'exercice 2.187, puis on re-parcourt la liste à partir de la fin en remettant les pointeurs en place et en affichant les éléments rencontrés.

**Solution 2.191.** La solution ne fonctionne que si le maillon à supprimer  $m$  n'est pas le dernier maillon. Plutôt que de supprimer,  $m$ , on supprime le successeur de  $m$  après avoir copié son contenu dans  $m$ .



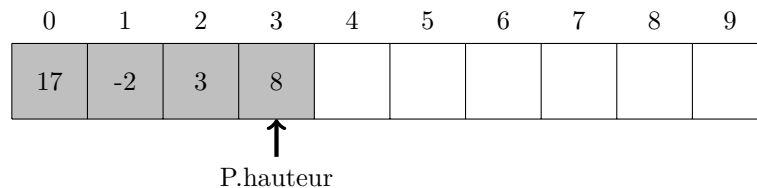
suppression du maillon contenant la valeur 7

**Solution 2.192.**

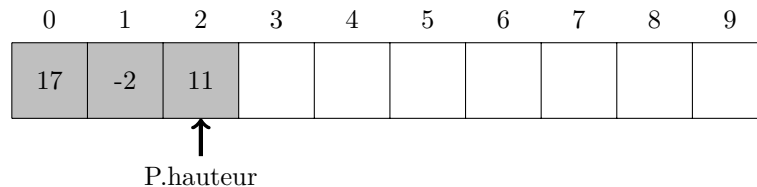
**Solution 2.193.** Soit  $\ell_1$  et  $\ell_2$  les longueurs des listes.

1. On parcourt les deux listes et on compare le dernier élément. Coût :  $\ell_1 + \ell_2$ .
2. Le problème est facile à résoudre si les deux listes ont même longueur : on avance simultanément d'un élément le long de chaque liste. Dès lors que deux maillons pointent vers le même successeur, on a trouvé le problème de fusion. On commence par calculer la longueur  $\ell_1$  et  $\ell_2$  de chaque liste. Supposons que  $\ell_1 > \ell_2$ , il suffit d'ignorer les  $\ell_1 - \ell_2$  premiers termes de la première liste et de procéder comme pour deux listes de même longueur. Coût :  $2(\ell_1 + \ell_2)$ .
3. On crée un maillon sentinelle spécial. On parcourt la première liste et on fait pointer chaque maillon vers le maillon sentinelle. Puis on parcourt la seconde liste : soit elle débouche sur le pointeur vide et les deux listes ne se joignent pas, soit elle débouche sur le maillon sentinelle et on a détecté le point de fusion.

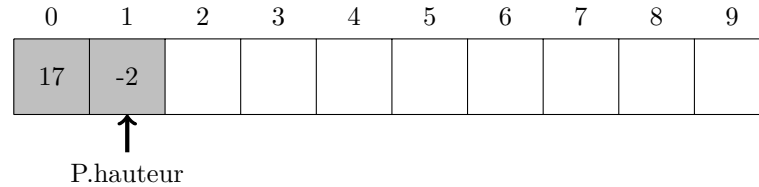
**Solution 2.194.** 1. Après le premier empilement,



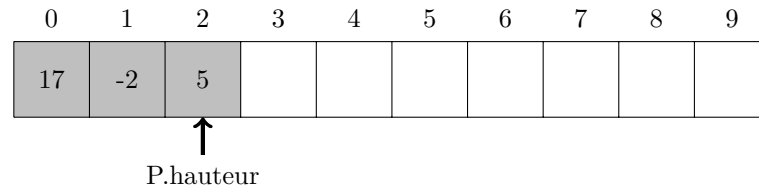
2. Comme  $8 + 3 = 11$



3. Ensuite



4. Enfin



**Solution 2.195.** On considère la file  $F$  formée de deux piles  $P_1$  et  $P_2$  mises bout à bout. On garantit par les algorithmes décrits ci-après que la pile  $P_1$  contient toujours des éléments enfilés avant ceux de  $P_2$ , que les éléments de  $P_1$  sont empilés du plus ancien en tête au plus récent en fond de pile et que les éléments de  $P_2$  sont empilés du plus récent en tête au plus ancien en fond de pile.

[Preuve de correction à rédiger]

Tout élément ne transite qu'une seule fois de  $P_2$  à  $P_1$ , si bien que le nombre total d'opération est dans le pire des cas le double du nombre d'opération sur une file classique. La complexité est bien conservée.

Remarque : on dit dans ce cas que la *complexité amortie* est  $O(1)$  par opération.

**Solution 2.196.** 1. On obtient 3, 4, 2, 5, 1.

2. (a) Push, Pop, Push, Pop, Push, Pop, Push, Push, Pop, Pop.  
 (b) Push, Push, Pop, Push, Pop, Push, Push, Pop, Pop, Pop.  
 (c) Impossible. Pour placer le 3, il faut Push,Push,Push,Pop, mais ensuite, comme 2,1 sont stockés, on est forcé de faire sortir 2 avant 1.

3. On remarque que quoi qu'il arrive, les entiers inscrits sur la pile le sont toujours en ordre décroissant. La condition de l'énoncé est donc effectivement impossible à réaliser car pour sortir  $p_i$  avant  $p_j$  et  $p_k$ , il faut avoir déjà empilé ces deux nombres et ceux-ci ne peuvent ressortir que dans l'ordre décroissant.

Soit  $(p_1, \dots, p_n)$  une permutation vérifiant les conditions de l'énoncé. Pour réaliser la permutation  $(p_1, \dots, p_n)$ , on peut procéder de manière systématique comme suit :

On remarque déjà que la boucle tant que ne peut pas être infinie car le compteur  $j$  est incrémenté à chaque passage dans la boucle.

Preuve de correction. Comme  $j$  est incrémenté après chaque Push, on est sûr de bien empiler successivement tous les entiers entre 1 et  $n$  comme souhaité.

Montrons par récurrence que le Pop du  $i$ -ième passage dans la boucle pour renvoie  $p_i$  et qu'à l'issue de ce passage,  $j = \max_{k \leq i} p_k$  et la pile contient dans l'ordre décroissant  $[1, j] \setminus \{p_1, \dots, p_i\}$ .



- Au premier passage, la boucle tant que incrémente  $j$  jusqu'à  $p_1$  et empile  $[1, p_1]$ . Donc Pop renvoie  $p_1$ ,  $j = p_1$  et la pile ne contient plus que  $[1, p_1 - 1]$  qui correspond au résultat attendu.
- Au  $i$ -ième passage, soit  $j > p_i$ , alors la boucle tant que ne fait rien,  $j$  ne bouge pas et reste égal à  $\max_{k \leq i} p_k$ . Alors Pop renvoie le plus grand élément parmi  $[1, j] \setminus \{p_1, \dots, p_{i-1}\}$ , que nous notons  $\theta$ . Montrons que  $\theta = p_i$ . Si  $p_i \neq \theta$ , alors comme  $p_i < j$  et que les  $(p_k)$  sont distincts,  $p_i < \theta$ . De plus, comme  $\theta$  n'a pas encore été utilisé, c'est qu'il correspond à un certain  $p_s$  avec  $s > i$ . Par ailleurs  $j = p_t$  pour un certain  $t < i$ . On a donc  $t < i < s$  et  $p_i < p_s = \theta < j = p_t$  ce qui est justement exclu. Donc Pop renvoie bien  $p_i$  et la pile contient ce qui était pr'evu.  
Soit  $p_{i-1} \leq p_i$ , la boucle empile  $[p_{i-1}, p_i]$ , incrémente  $j$  à  $p_i = \max_{k \leq i} p_k$ . Puis Pop renvoie  $p_i$  et il reste  $[1, p_i] \setminus \{p_1, \dots, p_i\}$  dans la pile comme voulu.

4. La seule permutation possible est l'identité.

**Solution 2.197.** Le danger est de croire que l'on peut se contenter de stocker le minimum courant de la pile. Que se passe-t-il alors quand on dépile le minimum ?

Il faut garder en mémoire également la suite des minima successifs. Ceci peut être accompli en gérant deux piles : une pile classique et une pile des minima. On peut résumer les choses dans des maillons chaînés avec deux types de successeurs : un pointeur vers le successeur dans la pile des éléments et un pointeur vers le successeur dans la pile des minima. Deux pointeurs servent à garder le début de la pile des éléments et le début de la pile des minima.

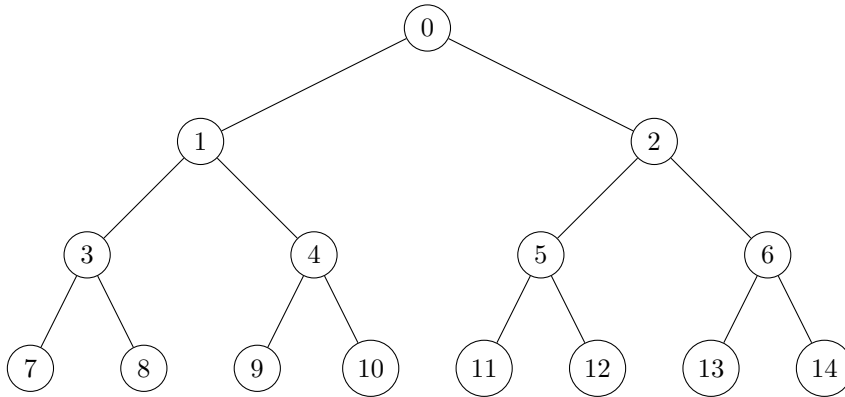
**Solution 2.198.** On simule la file  $F$  avec deux piles  $P_1$  et  $P_2$ . Quand on enfila un élément  $a$ , on empile  $(a, m)$  sur  $P_1$  où  $m$  est le minimum de  $P_1$  (qui peut être calculé en  $O(1)$  à partir de l'élément de tête  $(a', m')$  de  $P_1$  par la formule  $m = \min(a, m')$ ). Quand on défile de  $F$ , on dépile depuis  $P_2$ . Si  $P_2$  est vide, on renverse d'abord  $P_1$  dans  $P_2$ . On peut noter que si  $(a, m)$  est l'élément de tête de  $P_2$ ,  $m$  est encore le minimum de  $P_2$ . Donc, **get\_min** renvoie le minimum des minima des éléments de tête de  $P_1$  et  $P_2$  (coût  $O(1)$ ).

**Solution 2.199.** Evidemment, si on pouvait initialiser un vecteur booléen  $A$  de longueur  $n$ , il suffirait de passer à True la case  $A[i]$  pour insérer  $i$ .

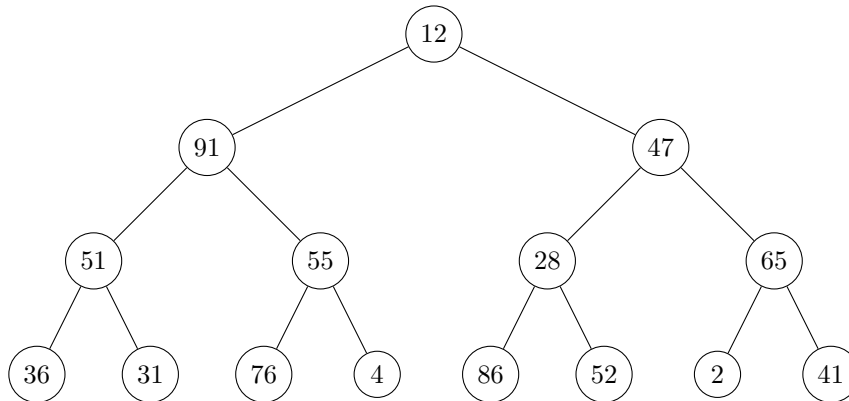
Nous considérons deux tableaux  $A$  de longueur  $n$  et  $B$  de longueur  $m$  et un compteur  $c$  initialisé à 0. Ces tableaux contiennent initialement des valeurs quelconques. Lors de l'insertion de l'entier  $i$ , nous effectuons  $A[i] \leftarrow c$ ,  $B[c] \leftarrow i$ ,  $c \leftarrow c + 1$ . En quelque sorte,  $c$  représente le numéro d'ordre de l'inscription de  $i$ . Pour vérifier la présence de  $i$ , nous pouvons contrôler d'abord que  $A[i]$  possède une valeur  $\leq c$  (valeur courante de  $c$ ) puis que  $B[A[i]] = i$ . Le premier test contrôle que  $B[A[i]]$  a été écrit par nous, le second qu'il s'agit bien de la valeur  $i$  que l'on a inséré à cette occasion. Pour supprimer un élément, on peut remplacer  $A[i]$  par  $m + 1$ .

Des variations sur cette idée permettent de stocker également des multi-ensembles.

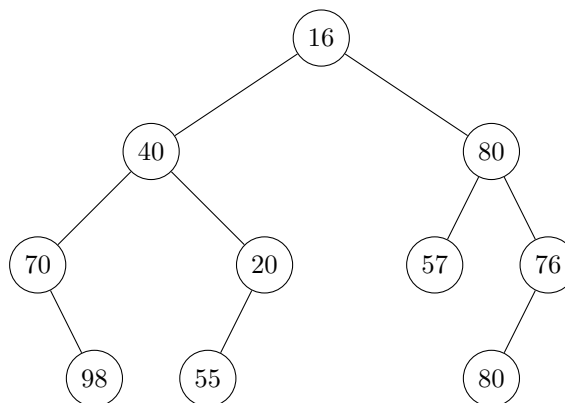
**Solution 2.200.** 1. Les indices dans le tableau correspondent aux nœuds suivants :



Par conséquent, l'arbre est



2. On a, en suivant les pointeurs,

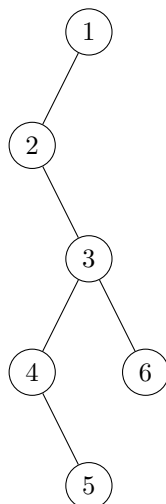


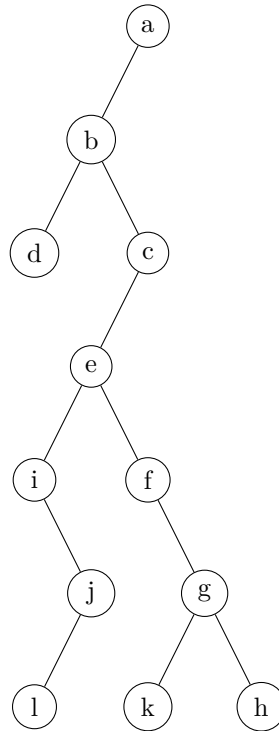
**Solution 2.201.** 1. Le rapport est de  $4/(4 \cdot 4) = 25\%$  par nœud, donc aussi de 25% pour tout l'arbre.

2. Dans un arbre binaire entier (i.e. si tous les nœuds internes ont deux fils), le nombre de feuille  $f$  et de nœud interne est relié par la relation  $f = n + 1$ , soit environ un nœud interne par feuille. On en déduit un rapport proche de  $4/(4 + 2 + 2) = 50\%$ . Si l'arbre n'est pas entier, on ne peut rien dire en général : cas extrême, une suite de  $n - 1$  nœuds internes et une seule feuille.

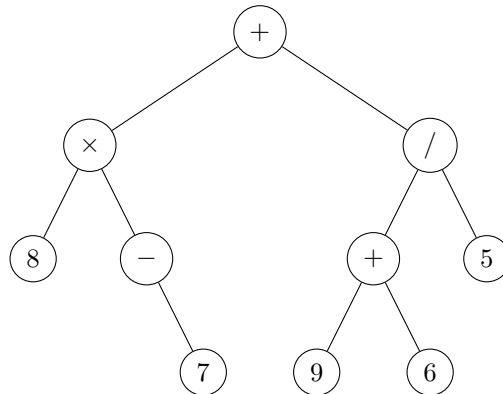
**Solution 2.202.** 1. Arbre 1. Préordre : 1 2 3 4 5 6. Postordre : 2 4 5 3 6 1. Arbre  
2. Préordre : a b d c e i j l f g k h. Post ordre d b i l j e f k g h c a

2. On obtient





**Solution 2.203.** On obtient :



**Solution 2.204.** 1. Préfixe : a b d e g h i c f j k l. Suffixe : d g i h e b j l k f c a.  
Infixe : d b g e i h a c j f k l.

**Solution 2.205.**

**Solution 2.206.** 1. Préfixe : I D A C B G E F H O M K J L N. Postfixe : B C A  
F E H G D J L K N M O I. Infixe : A B C D E F G H I J K L M N O.

2. Montrer que pour tout sommet  $x$ , les sommets du sous-arbre gauche portent une valeur inférieure à celle de  $x$  et les sommets du sous-arbre de droite portent

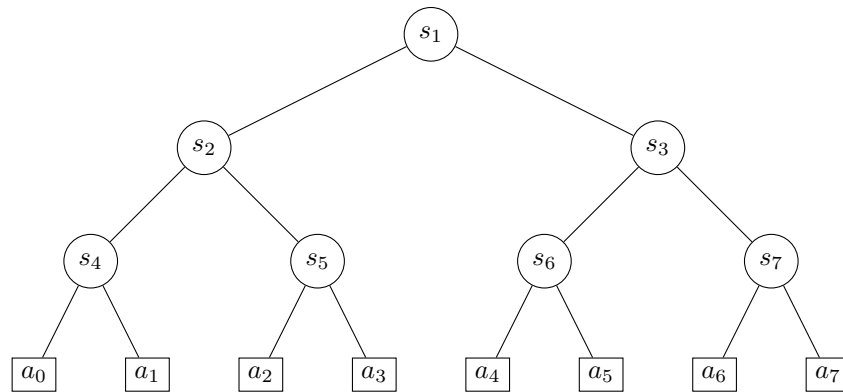
une valeur supérieure à celle de  $x$  revient à vérifier que les sommets forment une suite croissante dans l'ordre infixe, ce qui est le cas.

**Solution 2.207.** La plus petite hauteur est obtenue pour un arbre parfait :  $\lfloor \log_2 n \rfloor$ . La plus grande hauteur est obtenue pour un arbre en ligne  $n - 1$ .

**Solution 2.208.** Quitte à compléter la suite  $(a_i)_i$  par des zéros, on supposera que  $n$  est une puissance de 2 (au pire, on rajoute  $n - 1$  données, ceci ne change donc pas les ordres de grandeurs par rapport à  $n$ ).

On considère comme structure un arbre binaire complet, stocké sous la forme d'un tableau de taille  $2n - 1$ . On désigne par  $s_k$  la valeur du sommet  $k$ . Les nœuds de l'arbre sont numérotés niveaux par niveaux, la racine étant le nœud 1, les feuilles étant les nœuds  $n, n + 1, \dots, 2n - 1$ . Ainsi, le père du sommet  $k$  est le sommet  $\lfloor k/2 \rfloor$ , le fils gauche  $2k$  et le fils droit  $2k + 1$ ; il est donc très facile de déterminer le chemin dans l'arbre entre la racine et le sommet  $i$  en fonction de l'écriture binaire de  $i$ .

Les éléments  $a_0, \dots, a_{n-1}$  sont stockés au niveau des feuilles :  $s_{n+k} = a_k$ . Chaque nœud interne porte la somme des feuilles du sous-arbre dont il est la racine.



L'opération **add** demande de mettre à jour tous les nœuds entre le sommet et la feuille concernée : il y a  $\log_2 n + 1$  tels sommets et chaque mise à jour représente  $O(1)$  opérations élémentaires. L'opération **add** a une complexité  $O(\log n)$ .

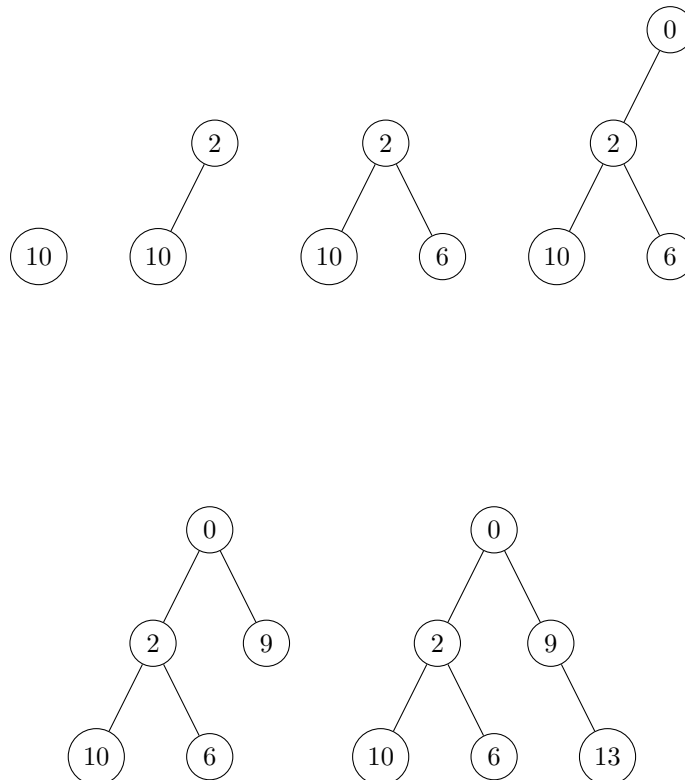
Pour l'opération **report**, on peut noter qu'il suffit de calculer les sommes  $\sum_{k=0}^j a_k$  et  $\sum_{k=0}^i a_k$  en temps  $O(\log n)$  et soustraire les résultats. Ceci peut être fait comme suit : on parcourt le chemin de la racine de l'arbre vers la feuille  $i$  en tenant un compteur initialisé à 0. À chaque étape, si l'on se déplace vers le fils gauche, on ne modifie pas le compteur, si l'on se déplace vers le fils droit, on ajoute la valeur du fils gauche au compteur. Lorsqu'on rencontre une feuille, on ajoute la valeur de la feuille. On voit par un raisonnement par récurrence, que l'algorithme calcule bien la somme  $\sum_{k=0}^i a_k$ . Comme le chemin parcouru est de longueur  $\log_2 n + 1$ , le coût demeure en  $O(\log n)$ .

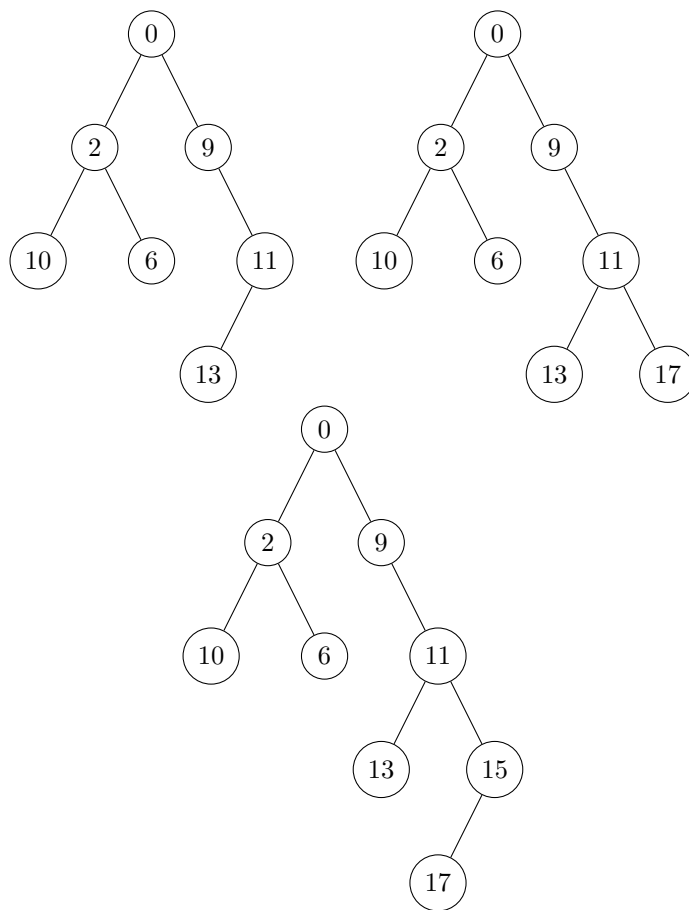
**Solution 2.209.** 1. On précalcule et stocke dans un tableau  $A[i, j]$  tous les résultats. Ce tableau peut être calculé en temps  $O(n^2)$  en commençant par initialiser la diagonale  $A[i, j] = x_i$  puis en notant que  $A[i, j] = \min A[i, j - 1], A[i + 1, j]$  par exemple.

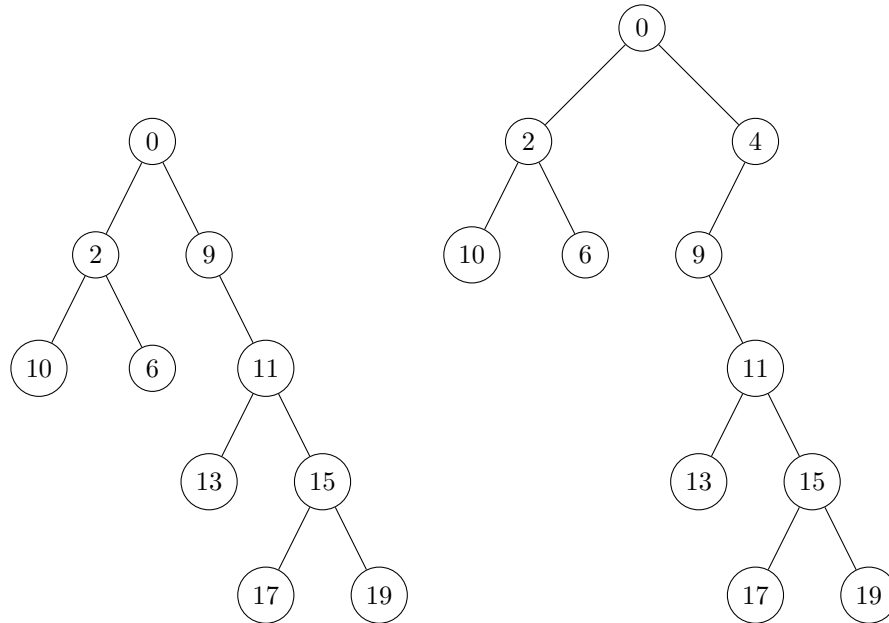
2. On précalcule l'*arbre cartésien* associé à la suite (coût  $O(n)$  en temps et en espace). À chaque requête, on recherche le plus proche ancêtre de  $x_i$  et  $x_j$  dans l'arbre, qui est aussi le minimum attendu (coût  $O(\log n)$ ).

L'arbre cartésien est l'arbre binaire dont chaque nœud est un élément  $x_i$ , dont le parcours dans l'ordre symétrique renvoie la suite initiale et ayant une structure de tas (i.e. tel que la clé d'un père est toujours inférieure à la clé de ses fils). Il est unique. On peut le construire par insertion successive des éléments de la liste. Afin d'insérer  $x_{i+1}$ , on remonte de  $x_{i+1}$  vers la racine jusqu'à arriver à un nœud  $y$  de clé inférieure à  $x_{i+1}$ . Un nouveau nœud  $x_{i+1}$  devient le fils droit de  $y$ , tandis que l'ancien fils droit de  $y$  devient le fils gauche de  $x_{i+1}$ .

Exemple : soit la suite 10, 2, 6, 0, 9, 13, 11, 17, 15, 19, 4. La construction de l'arbre se fait comme suit.







Pour trouver l'ancêtre commun le plus proche entre deux sommets  $s$  et  $t$ , on construit le chemin entre la racine  $r$  et chaque sommet. On reconnaît l'endroit où les chemins divergent. Le coût est de l'ordre de la hauteur de l'arbre construit, qui est en  $O(\log n)$  avec une grande probabilité.

**Solution 2.210.**

**Solution 2.211.** On construit un arbre 9-aire dont chaque sommet pointe vers une liste chaînée. La position du nœud dans l'arbre représente une suite de chiffres ; la liste chaînée contient les mots associés. Cette structure de donnée s'appelle *trie* (de *retrieval* en anglais).

**Solution 2.212.** L'exercice est subtil si on souhaite rester en complexité  $O(n)$ . En effet, on est tenté de commencer par chercher le père qui se trouve au milieu, puis agir par récurrence sur les deux moitiés de la liste. Mais chercher le milieu a un coût  $O(n)$  et cet algorithme nous conduit à un coût  $O(n \log n)$  qui est aussi mauvais que de retrier la liste complètement.

On peut faire mieux. On commence par calculer la longueur  $n$  de la liste. Coût  $O(n)$ . Muni de cette information, on effectue dans l'ordre :

- construction d'un ABR avec les  $\lceil n/2 \rceil - 1$  premiers éléments de la liste.
- construction du père
- construction d'un ABR avec les  $n - \lceil n/2 \rceil$  éléments restants de la liste.

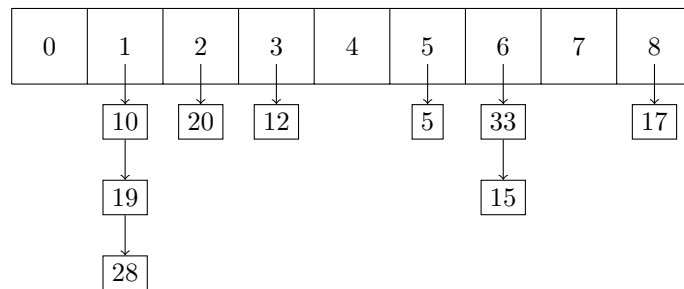
avec un algorithme `ListeTrieToABR( $p, \ell$ )` qui reçoit en entrée un pointeur  $p$  vers le premier maillon et la longueur  $\ell$  de la sous-liste à transformer et renvoie un pointeur vers la racine de l'ABR et un pointeur vers le prochain élément dans la liste. Cette



façon de faire est en  $O(n)$ .

**Solution 2.214.** On parcourt le tas en parcours préfixe, tout en tronquant l'exploration de l'arbre dès que l'on atteint un élément inférieur strictement à  $x$ . On compte les éléments rencontrés au fil du parcours. S'il y en a plus de  $k$ , on s'arrête. Au total, on ne parcourt pas plus de  $k$  nœuds, ce qui coûte au plus  $O(k)$  espace en mémoire. Par ailleurs, comme l'arbre est binaire, en chaque nœud parcouru, il y a au plus deux fils que l'on élimine. L'examen de chaque sommet ne prend donc que  $O(1)$  par sommet, soit  $O(k)$  pour l'ensemble du parcours.

**Solution 2.215.** On obtient :



**Solution 2.216.** Dans tous les cas, la première collision se produit après

|    |   |   |   |   |   |   |   |   |    |    |
|----|---|---|---|---|---|---|---|---|----|----|
| 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  | 10 |
| 22 |   |   |   | 4 |   |   |   |   | 31 | 10 |

Ensuite les choses diffèrent selon les cas.

1. (Sondage linéaire) On obtient :

|    |    |   |   |   |    |    |    |    |    |    |
|----|----|---|---|---|----|----|----|----|----|----|
| 0  | 1  | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 |
| 22 | 88 |   |   | 4 | 15 | 28 | 17 | 59 | 31 | 10 |

2. (Sondage quadratique) La suite  $i + 3i^2 \bmod 11$  prend les valeurs successives 0, 4, 3, 8, 8, 3, 4, 0, 2. On obtient :

|    |   |    |    |   |   |    |    |    |    |    |
|----|---|----|----|---|---|----|----|----|----|----|
| 0  | 1 | 2  | 3  | 4 | 5 | 6  | 7  | 8  | 9  | 10 |
| 22 |   | 88 | 17 | 4 |   | 28 | 59 | 15 | 31 | 10 |

Pour 15, on teste successivement les positions  $15 = 4 \bmod 11$  (occupée),  $15 + 1 + 3 \cdot 1^2 = 4 + 4 = 8$  (libre). Pour 17, on teste successivement les positions  $17 = 6 \bmod 11$  (occupée),  $6 + 4 = 10$  (occupée),  $6 + 3 = 9$  (occupée),  $6 + 8 = 3$

mod 11 (libre). Pour 88, on teste successivement les positions  $88 = 0 \pmod{11}$  (occupée),  $0+4 = 4 \pmod{11}$  (occupée),  $0+3 = 3 \pmod{11}$  (occupée),  $0+8 = 8 \pmod{11}$  (occupée),  $0+8 = 8 \pmod{11} \dots 0+8+3 \cdot 8^2 = 2 \pmod{11}$  (libre). Pour 59, on teste  $59 = 4 \pmod{11}$ ,  $59+4 = 8 \pmod{11}$ ,  $59+3 = 7 \pmod{11}$ .

3. (Sondage double) On obtient :

|    |    |    |    |   |    |    |   |   |    |    |
|----|----|----|----|---|----|----|---|---|----|----|
| 0  | 1  | 2  | 3  | 4 | 5  | 6  | 7 | 8 | 9  | 10 |
| 22 | 17 | 59 | 15 | 4 | 88 | 28 |   |   | 31 | 10 |

Pour placer 15 : 4 (occupé),  $4+1 \cdot 6 = 10$  (occupé),  $4+2 \cdot 6 = 3$  (libre). Pour placer 17 : 6 (occupé),  $4+1 \cdot 8 = 1$  (libre). Pour placer 88 : 0 (occupé),  $0+1 \cdot 9 = 9$  (occupé),  $0+2 \cdot 9 = 5$  (libre). Pour placer 59 : 4 (occupé),  $4+1 \cdot 10 = 3$  (occupé),  $4+2 \cdot 10 = 2$  (libre).

**Solution 2.217.** 1. On obtient

| Nom        | $h_1$ | $h_2$ | Nom          | $h_1$ | $h_2$ |
|------------|-------|-------|--------------|-------|-------|
| Alfio      | 0     | 5     | Jean-Marie   | 9     | 9     |
| Amel       | 0     | 4     | Joachim      | 9     | 9     |
| Aminata    | 0     | 8     | John         | 9     | 16    |
| Anthony    | 0     | 19    | Josée        | 9     | 1     |
| Antoinette | 0     | 19    | Klaus-Dieter | 10    | 10    |
| ...        |       |       |              |       |       |

2. Non. Il y a 35 noms à placer et 26 valeurs de clé possibles, ce qui ne suffit pas à stocker tous les noms.

3. Pour la fonction  $h_1$ , on obtient

| Valeur | Avec $h_1$                                                  | Avec $h_2$                                    |
|--------|-------------------------------------------------------------|-----------------------------------------------|
| 0      | Alfio, Amel, Aminata,<br>Anthony, Antoinette                |                                               |
| 1      | Bernardine, Boris                                           | Josée                                         |
| 2      | Charles                                                     | Charles                                       |
| 3      | Denis, Diego, Dominique                                     | Marco, Sylvaine                               |
| 4      | Erik, Eva                                                   | Amel, Eva                                     |
| 5      |                                                             | Alfio                                         |
| 6      | Gérard                                                      |                                               |
| 7      | Hichem                                                      | Diego, Otto, Thomas                           |
| 8      | Ian                                                         | Amin, Petra                                   |
| 9      | Jacques, Jean-François, Jean-Marie,<br>Joachim, John, Josée | Hichem, Jean-François,<br>Jean-Marie, Joachim |
| 10     | Klaus-Dieter                                                | Klaus-Dieter                                  |
| 11     |                                                             | Jacques                                       |
| 12     | Manuela, Marco, Michèle                                     | Erik                                          |
| 13     |                                                             |                                               |
| 14     | Otto                                                        | Michèle                                       |
| 15     | Petra, Philippe                                             | Dominique                                     |
| 16     |                                                             | Denise, John                                  |
| 17     | Roberta                                                     |                                               |
| 18     | Sacha, Stefan, Sylvain                                      | Bernard, Boris, Roberta                       |
| 19     | Tomas, Todor                                                | Anthony, Antoinette                           |
| 20     |                                                             | Sacha                                         |
| 21     |                                                             | Ian                                           |
| 22     |                                                             | Stefan, Todor                                 |
| 23     |                                                             | Gérard, Philippe                              |
| 24     |                                                             |                                               |
| 25     |                                                             | Manuela                                       |

4. La seconde table est préférable car les listes obtenues sont moins longues
5. La distribution de la première lettre d'un prénom est assez éloignée d'une distribution uniforme. Ajouter la 3-ième lettre rend le haché plus équiprobable dans l'ensemble  $\{0, \dots, 25\}$ .

**Solution 2.218.** Notons  $h$  la fonction de hachage. On raisonne par la contraposée. Si un tel ensemble  $E$  n'existe pas, c'est que pour tout  $1 \leq x \leq n$ ,  $|h^{-1}(x)| < t$ . Mais alors, comme  $U$  est l'union disjointe  $U = \bigsqcup_{x=1}^n h^{-1}(x)$ , le cardinal de  $U$  vérifie  $|U| < tn$ .

**Solution 2.219.** 1. On se trouve à la cellule  $k + \sum_{\ell=1}^i \ell = k + i(i+1)/2 \pmod n$

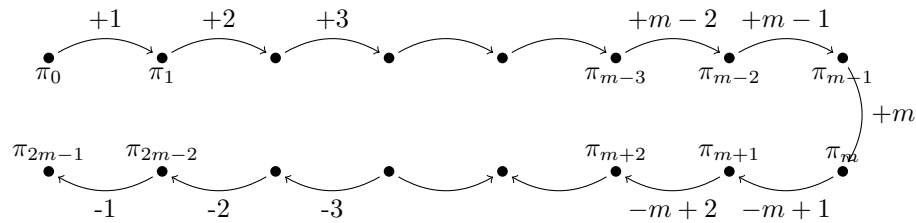
2. Notons  $\pi_i = \frac{i(i+1)}{2}$ . En fait, nous allons montrer que la suite

$$\sigma_n = (\pi_0 \pmod n, \pi_1 \pmod n, \dots, \pi_{n-1} \pmod n)$$

forme une permutation de l'intervalle entier  $[0, n-1]$ .

Nous procédons par récurrence. Pour  $m = 2$ , la suite est  $[0, 1]$  qui est la permutation identité.

Supposons avoir prouvé que la suite  $\sigma_m$  est une permutation de  $[0, m - 1]$  et montrons que la suite  $\sigma_{2m}$  est alors une permutation de  $[0, 2m - 1]$ . Nous remarquons que modulo  $2m$ , additionner  $m + i$  revient à additionner  $-(m - i)$ , si bien que les éléments de  $\sigma_{2m}$  s'obtiennent en procédant comme selon le schéma (modulo  $2m$ )



On remarque en particulier que  $\pi_i = \pi_{2m-1-i} + m \pmod{2m}$ . Comme  $\sigma_m$  est une permutation modulo  $m$ , il s'ensuit que la suite  $\sigma_{2m}$  est une permutation modulo  $2m$ , ce qui conclut la récurrence.

#### Solution 2.220.

**Solution 2.221.** On crée une file (FIFO)  $F$  pour stocker les commandes et une table de hachage  $T$  de taille  $m$  (avec adressage ouvert) pour stocker les quantités de produits en instance de commande.

- **Commande()** enfile les champs [produit, quantité, nom du client] à  $F$ ; vérifie si le produit  $p$  est dans  $T$ ; si oui, augmente la quantité de  $p$ , sinon ajoute  $p$  à  $T$  et retient la quantité commandée.
- **Livraison()** défile le sommet de  $S$  et fait décroître la quantité dans  $T$ , en supprimant  $p$  de  $T$  si elle passe à 0.
- **Achalandage( $p$ )** interroge  $T$  et renvoie la quantité stockée si  $p$  est dans  $T$ , 0 sinon.

Les temps sont tous en  $O(1)$ , car la table est de longueur  $m$ .

### 6.3. Algorithmes de graphes

**Solution 3.88.** 1. Pour le cube :

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Pour le second graphe :

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Pour le dernier graphe :

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

2.

**Solution 3.89.** Non.

**Solution 3.90.** Le nombre d'arête, qui est un entier, doit être  $nd/2$ . Comme  $n$  est impair,  $d$  est pair.

**Solution 3.91.** Pour compléter le vocabulaire, nous appellerons les sommets restants des pieds. Le point crucial est d'utiliser les caractérisations suivantes :

| Sommet | Arêtes présentes | Arêtes lacunaires |
|--------|------------------|-------------------|
| Dard   | 1                | $n - 2$           |
| Queue  | 2                | $n - 3$           |
| Pied   | de 1 à $n - 3$   | de $n - 2$ à 2    |
| Corps  | $n - 2$          | 1                 |

Par conséquent, commençons par quelques faits :

- Étant donné un sommet  $u$ , il est possible de calculer le degré de  $u$  en  $O(n)$  par l'addition des  $n$  coefficients de la ligne  $u$  de la matrice  $A$ .
- Avec  $O(1)$  requêtes sur la matrice d'adjacence, il est possible seulement d'exclure le fait qu'un sommet donné soit un dard, une queue ou un corps. Par contre,  $O(n)$  requêtes sont nécessaires pour affirmer connaître ou confirmer le statut d'un sommet.
- Étant donné un sommet  $u$  de degré 1 ou  $n - 2$ , il est possible en parcourant la ligne  $u$  dans la matrice de déterminer l'unique voisin de  $u$  (repéré par un 1) ou l'unique non-voisin de  $u$  (repéré par un 0).

- Étant donné un sommet  $u$  de degré 1 ou  $n-2$ , les algorithmes suivants permettent de décider en  $O(n)$  si  $G$  est un scorpion

---

**Algorithme 6.1 : ESTSCORPIONDEPUISDEGREEUN**


---

**Entrées :** Sommet  $u$   
**Sorties :** Si  $\deg u = 1$ , teste si  $G$  est un scorpion, sinon pas de réponse

```

1  $d_0 \leftarrow \text{degree}(u)$ 
2 si  $d_0 \neq 1$  alors
3    $\lfloor$  retourner Pas de réponse possible
4  $v \leftarrow \text{voisin}(u)$ 
5  $d \leftarrow \text{degree}(v)$ 
6 si  $d = n - 1$  alors
7    $\lfloor$  ESTSCORPIONDEPUISDEGREEUNMOINSUN( $v$ )
8 sinon si  $d = 2$  alors
9    $w \leftarrow \text{voisin}(v)$ 
10   $d' \leftarrow \text{degree}(w)$ 
11  si  $d' = n - 1$  alors
12     $\lfloor$  retourner  $G$  n'est pas un scorpion
13  sinon
14     $\lfloor$  retourner  $G$  n'est pas un scorpion
15 sinon
16  $\lfloor$  retourner  $G$  n'est pas un scorpion
```

---



---

**Algorithme 6.2 : ESTSCORPIONDEPUISDEGREEUNMOINSUN**


---

**Entrées :** Sommet  $u$   
**Sorties :** Si  $\deg u = n - 2$ , teste si  $G$  est un scorpion, sinon pas de réponse

```

1  $d_0 \leftarrow \text{degree}(u)$ 
2 si  $d_0 \neq n - 2$  alors
3    $\lfloor$  retourner Pas de réponse possible
4  $v \leftarrow \text{nonvoisin}(u)$ 
5  $d \leftarrow \text{degree}(v)$ 
6 si  $d = 1$  alors
7    $w \leftarrow \text{voisin}(v)$ 
8    $d' \leftarrow \text{degree}(w)$ 
9   si  $d' = 2$  alors
10     $\lfloor$  si  $u = \text{voisin}(w)$  alors
11       $\lfloor$  retourner  $G$  est un scorpion
12 retourner  $G$  n'est pas un scorpion
```

---

Principe de l'algorithme. Nous commençons par créer deux listes disjointes :  $V_1$  et  $V_2$  telles que  $V_1$  contient le corps et  $V_2$  contient le dard. Puis nous allons chercher

à réduire le cardinal de l'une des deux listes en temps  $O(1)$  tout en conservant les propriétés que  $V_1$  contient le corps et  $V_2$  contient le dard.

Cette première étape peut se faire en testant pour tout  $u$  deux arêtes  $(u, v)$ ,  $v$  quelconque ou bien en triant les sommets adjacents des sommets non-adjacents d'un sommet fixé quelconque, soit un coût initial en  $O(n)$ .

Si nous sélectionnons deux sommets de  $V_1$  et deux de  $V_2$  et que nous explorons entièrement leur configuration, nous sommes en mesure de détecter forcément au moins un sommet qui n'est pas un corps ou un dard, ce qui permet de réduire  $|V_1| + |V_2|$  en  $O(1)$  opérations. Décrivons plus précisément une suite constante de tests atteignant ce but. Dans chaque cas, on supprime une arête  $v$  de  $V_1$  si nous avons découvert que  $v$  ne peut pas être un corps ou de  $V_2$  si nous avons découvert que  $v$  ne peut pas être un dard.

---

**Algorithme 6.3 : REDUCTION**


---

**Entrées :**  $V_1$  liste de sommets sans dard,  $V_2$  liste de sommets sans corps  
**Résultat :**  $|V_1| + |V_2|$  réduit d'au moins 1

```

1  $u_1 \leftarrow \text{piocher}(V_1)$ 
2  $v_1 \leftarrow \text{piocher}(V_1)$ 
3 si  $A(u_1, v_1) = 0$  alors
4   |  $\text{supprimer}(u_1, V_1)$ 
5   |  $\text{supprimer}(v_1, V_1)$ 
6 sinon
7   |  $u_2 \leftarrow \text{piocher}(V_2)$ 
8   | si  $A(u_2, u_1) = 1$  et  $A(u_2, v_1) = 1$  alors
9     |  $\text{supprimer}(u_2, V_2)$ 
10  | sinon si  $A(u_2, u_1) = 1$  et  $A(u_2, v_1) = 0$  alors
11    |  $\text{supprimer}(v_1, V_1)$ 
12  | sinon si  $A(u_2, u_1) = 0$  et  $A(u_2, v_1) = 1$  alors
13    |  $\text{supprimer}(u_1, V_1)$ 
14  | sinon
15    |  $u_2 \leftarrow \text{piocher}(V_2)$ 
16    | si  $A(v_2, u_1) = 1$  et  $A(v_2, v_1) = 1$  alors
17      |  $\text{supprimer}(v_2, V_2)$ 
18    | sinon si  $A(v_2, u_1) = 1$  et  $A(v_2, v_1) = 0$  alors
19      |  $\text{supprimer}(v_1, V_1)$ 
20    | sinon si  $A(v_2, u_1) = 0$  et  $A(v_2, v_1) = 1$  alors
21      |  $\text{supprimer}(u_1, V_1)$ 
22    | sinon si  $A(v_2, u_1) = 0$  et  $A(v_2, v_1) = 0$  alors
23      |  $\text{supprimer}(u_1, V_1)$ 
24      |  $\text{supprimer}(v_1, V_1)$ 

```

---

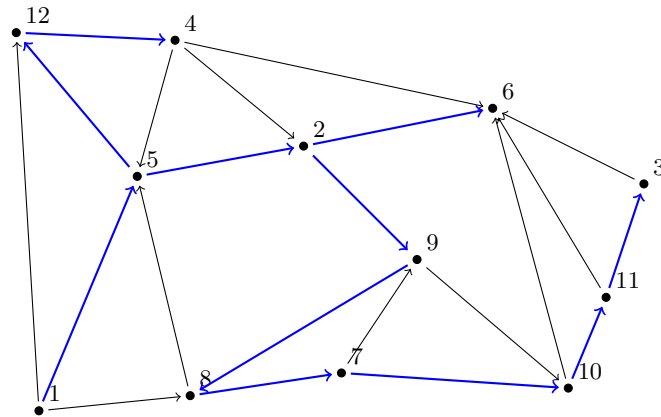
On peut appeler  $O(n)$  fois l'algorithme REDUCTION pour se ramener à une liste  $V_1$  ou  $V_2$  de cardinal 1 puis contrôler que  $G$  est un scorpion par les premiers algorithmes.

**6.3.0.0.1. Remarque :** — Une propriété de graphe est une propriété d'un graphe qui ne dépend pas de la numérotation de ses sommets (par exemple « être sans arête »). La conjecture d'Aanderaa-Rosenberg énonçait que vérifier une telle propriété demande  $\Omega(n^2)$  tests sur les arêtes. Le graphe du scorpion est un contre-exemple à cette conjecture. Sous de plus fortes conditions —la monotonie de la propriété—, la conjecture est vraie. Par ailleurs, on dit qu'une propriété est évasive s'il est nécessaire de tester les  $n(n-1)/2$  arêtes pour la déterminer ; on pense que toute propriété de graphe monotone et non-triviale est évasive.

**Solution 3.92.** 1. Le parcours DFS(1) visite les sommets (dans l'ordre de leur première rencontre, avec la marque  $\uparrow$  quand nous rebroussons chemin)

1, 5, 2, 6,  $\uparrow$ , 9, 8, 10, 11, 3,  $\uparrow^7$ , 12, 4

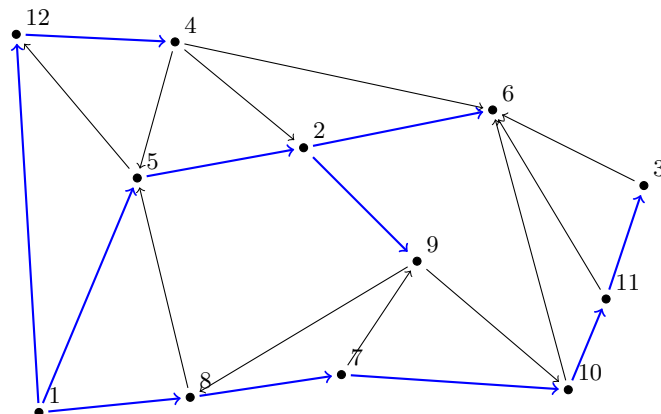
et conduit à l'arborescence



2. Le parcours BSF(1) visite les sommets

1, 5, 8, 12, 2, 7, 4, 6, 9, 10, 11, 3

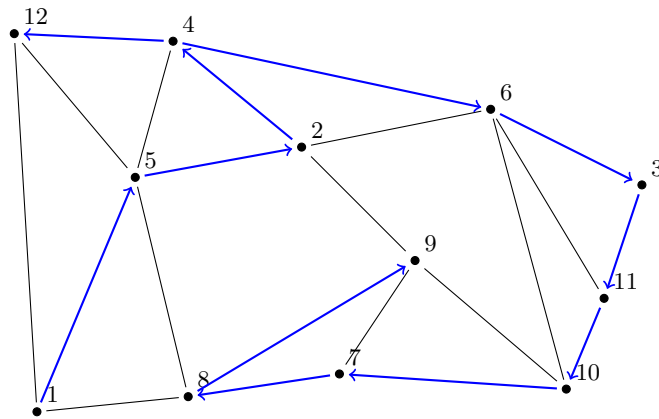
et donne lieu à l'arborescence





3. Le parcours DFS(1) visite

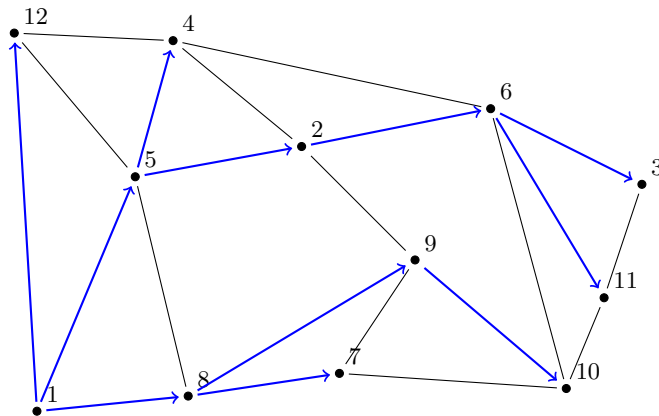
1, 5, 2, 4, 6, 3, 11, 10, 7, 8, 9, 12



Le parcours BFS(1) visite

1, 5, 8, 12, 2, 4, 7, 9, 6, 10, 3, 11

et



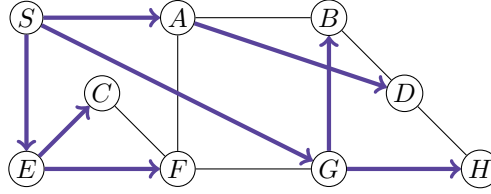
**Solution 3.93.** 1. Oui

2. Non

3. Non

4. Oui.

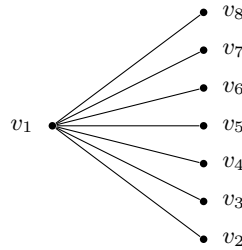
**Solution 3.94.** 1. On doit avoir  $S, G, E, A, H, B, F, C, D$



2. Non, car les deux sommets sont à distance 2 de  $S$ . Il faudrait qu'ils soient à distance  $d$  et  $d + 1$  pour un certain  $d$ .

**Solution 3.95.** Ce n'est pas possible. À cause de l'arborescence de gauche, on peut noter que tout couple de sommets se trouve à distance  $\leq 2$ . Dans l'arborescence de droite, la racine est à distance 3 de l'un des sommets.

**Solution 3.96.** Non, ce n'est pas garanti que cela se produise. On peut imaginer le contre-exemple suivant :



Avec  $r = v_1$  et  $s = v_5$ ,  $s$  est (probablement) le 5ième sommet rencontré depuis  $v_1$ . Par contre, en démarrant en  $s$ ,  $r$  est toujours le second.

**Solution 3.97.** Ce n'est pas possible. À cause de l'arborescence de gauche et en convenant que les sommets ont été vus de gauche à droite, on peut noter que tout sommet n'est pas voisin d'un des sommets à sa droite. Autrement dit  $G$  est égal à l'arborescence de gauche (sans les orientations). Dans l'arborescence de droite, il y a des chemins de longueur 3, ce qui n'est pas le cas dans  $G$ .

**Solution 3.98.** On ajoute à chaque sommet  $x$  une variable  $x.\text{dist}$ . Lorsqu'on traverse l'arête  $(x, y)$  et que  $y$  n'est pas encore marqué, on effectue  $y.\text{dist} \leftarrow x.\text{dist} + 1$ .

**Solution 3.99.** On initialise une variable à 0. À chaque marquage, on incrémente cette variable. Par exemple, sur le modèle de DFS :

---

**Algorithme 6.4 : DECOMPTEDFS**

---

**Entrées :** Sommet  $s$

**Sorties :** Nombre de sommets non marqués atteints depuis  $s$

```

1  $n \leftarrow 1$ 
2 marquer( $s$ )
3 pour  $v \in \text{voisins}(s)$  faire
4   si  $v$  non marqué alors
5      $n \leftarrow n + \text{DECOMPTEDFS}(v)$ 
6 retourner  $n$ 
```

---

**Solution 3.100.** Vous n'avez pas la réellement la possibilité d'utiliser un parcours en largeur, car cela vous demanderait de pouvoir vous téléporter instantanément d'une branche à l'autre ou tout du moins de faire beaucoup de backtracking (retour sur trace).

**Solution 3.101.** On effectue un tri topologique des sommets. Le tri que l'on obtient dépend nécessairement de l'ordre dans lequel on choisit d'entrer les sommets du graphe. On peut avoir par exemple : chaussettes, sous-vêtements, pantalons, chaussures, montre, chemise, ceinture, cravate, veste.

**Solution 3.102.** Le principe est la même que celui de l'exercice ?? : il faut ordonner dans l'ordre topologique les sommets du graphe. On effectue un parcours de graphe (par exemple depuis Main) et on numérote dans l'ordre post-fixe. La numérotation fournit directement l'ordre souhaité. Par exemple, Entities, Interactors, Database, Presenters, View, Controllers, Authorizer et finalement Main.

**Solution 3.103.** 1. Les données sont présentées sous forme de graphe acyclique. Il vaut mieux utiliser un tableau de listes de voisins que une matrice d'adjacence. En effet, on peut penser que nombre de pièces référencées est énorme et que peu d'entre elles sont nécessaires pour produire une nouvelle pièce.

2. C'est un problème proche du tri topologique dans un graphe orienté acyclique (cf. exercice ??). On peut explorer par un parcours en profondeur le graphe  $G^*$  dans lequel les arêtes ont été renversées.

On attribue à chaque sommet  $x$  un compteur  $x.quantite$  initialisé à 0. On initialise à 1 le sommet de l'objet que l'on veut produire. Puis, pour tout sommet  $s$  dans l'ordre décroissant de la numérotation postfixe, on effectue pour tout voisin  $p$  l'opération :  $s.quantite \leftarrow s.quantite + p.quantite \cdot capacite(s, p)$ .

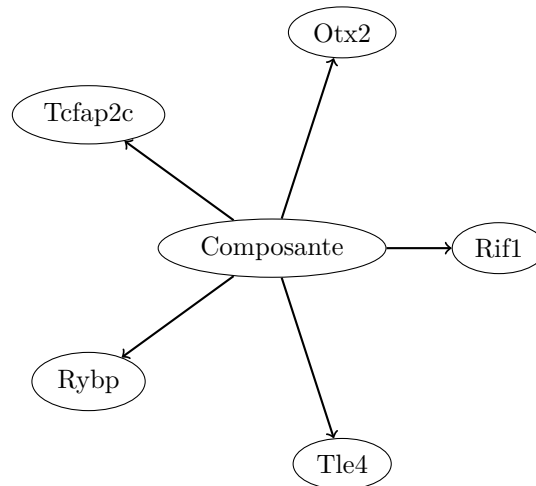
3. Pour un avion Airbus, il faut entre 3 et 4 millions de composants élémentaires.

**Solution 3.104.** Soit  $e$  une expression bien parenthésée. Nous construisons le tri topologique suivant :  $a_i$  reçoit comme numéro la position de la  $i$ ème parenthèse ouvrante de  $e$  et  $a_i$  reçoit comme numéro la position de la  $i$ ème parenthèse fermante de  $e$ . Par construction, les arcs  $(a_i, a_{i+1})$  et  $(b_i, b_{i+1})$  sont compatibles avec le tri proposé. Par ailleurs, comme  $e$  est bien parenthésé, les arcs  $(a_i, b_i)$  sont compatibles avec le tri proposé.

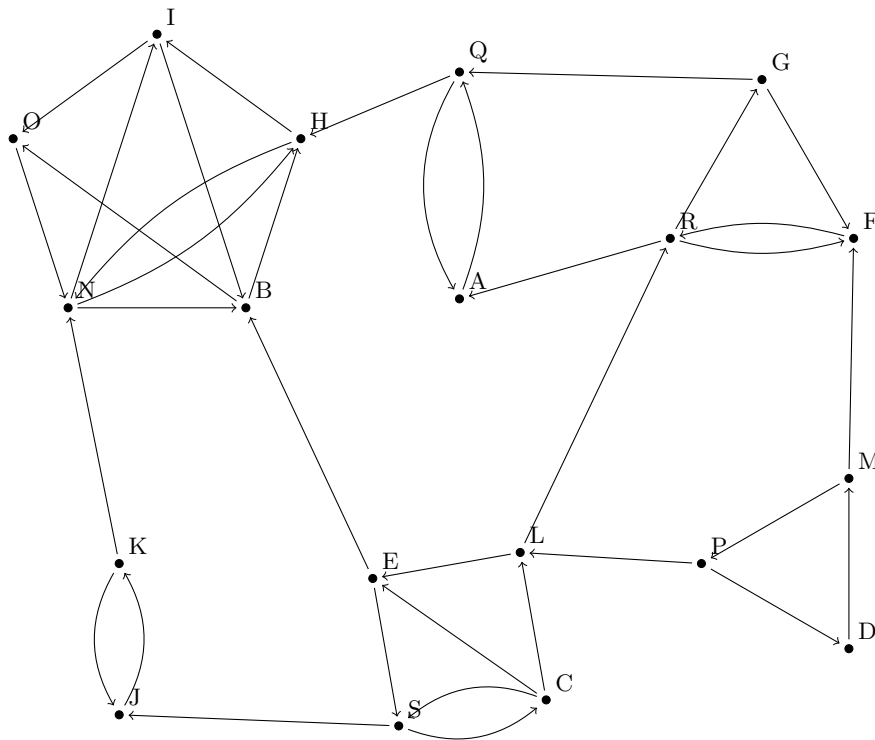
Réciproquement, un tri topologique étant donné, nous construisons une expression bien parenthésée  $e$  dont la  $j$ -ème lettre est '(' si le sommet de numéro  $j$  est l'un des  $a_i$  et dont la  $j$ -ème lettre est ')' si le sommet de numéro  $j$  est l'un des  $b_i$ . À cause de l'ordre entre les arcs  $(a_i, a_{i+1})$  et  $(b_i, b_{i+1})$ , les sommets  $a_i$  et  $b_i$  codent toujours la  $i$ ème parenthèse ouvrante ou fermante. De plus, à cause de l'arc  $(a_i, b_i)$ , la  $i$ ème parenthèse est fermée seulement après avoir été ouverte. Donc  $e$  est bien parenthésée.

Les deux fonctions sont réciproques l'une de l'autre et sont donc des bijections.

**Solution 3.105.** On obtient une grosse composante fortement connexe formée de Esrrb, Sox2, Tcf7, Oct4, Zfp219, Nr5a2, Dax1, Nanog, Etv5 et des composantes fortement connexes formées d'un seul élément : Rif1, Otx2, Tcfap2c, Rybp, Tle4.



**Solution 3.106.** Le graphe en décrit par le tableau admet la représentation suivante :



On applique l'algorithme vu en cours.

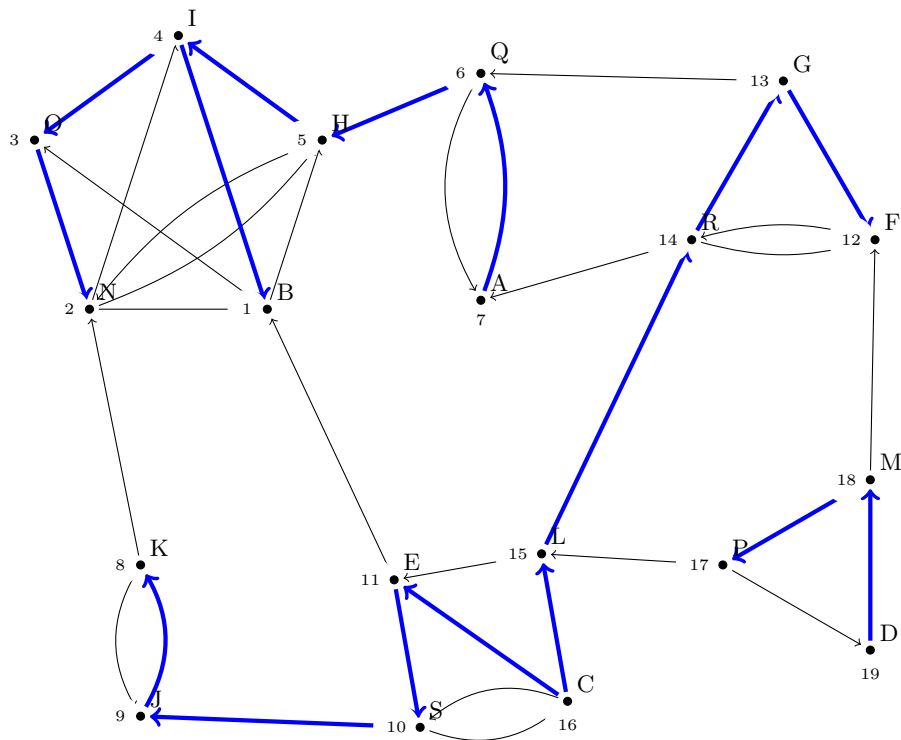
Phase 1 : Nous commençons énumérer tous les sommets par des parcours DFS et nous les numérotions en ordre postfixe. Les arbres que nous obtenons sont formés des

sommets

$A, Q, H, I, B, O, N$

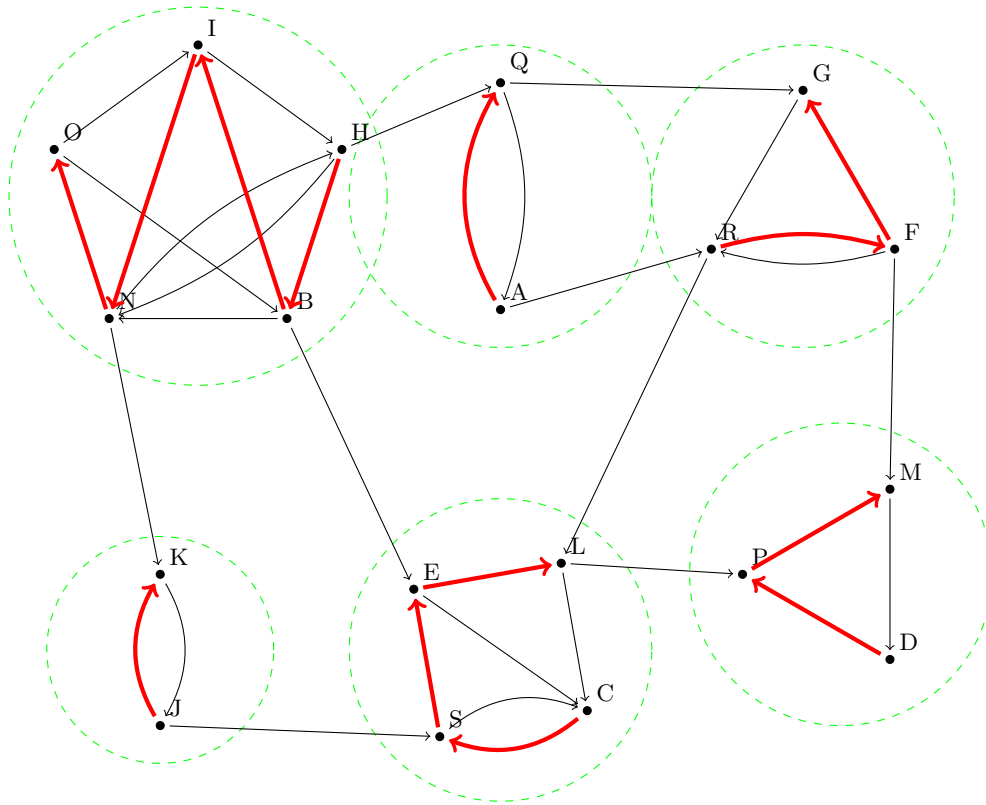
$C, E, S, J, K, L, R, G, F$

$D, M, P$

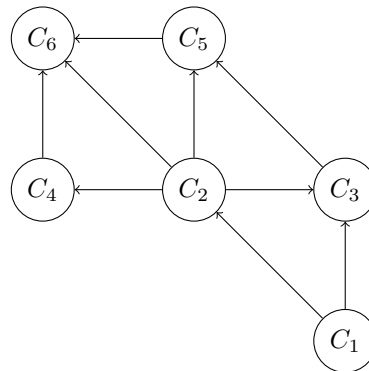


Phase 2 : Nous travaillons dans le graphe opposé et reprenons des parcours DFS à partir des sommets de plus grand numéro. Nous obtenons donc

| Composante | Sommet          |
|------------|-----------------|
| 1          | $D, P, M$       |
| 2          | $C, S, E, L$    |
| 3          | $R, F, G$       |
| 4          | $J, K$          |
| 5          | $A, Q$          |
| 6          | $H, B, I, N, O$ |



Enfin, les composantes fortement connexes s'organisent comme suit (noter qu'elles sont directement triées dans l'ordre topologique)



**Solution 3.107.** On construit un graphe dont les sommets sont les invités et les arêtes les conversations permettant de conclure. On étiquette chaque arête par  $+1$  si les opinions concordent et par  $-1$  si elles divergent. Si on représente le graphe par des listes d'adjacence, la construction coûte  $O(n + m)$  opérations. Ensuite on parcourt le

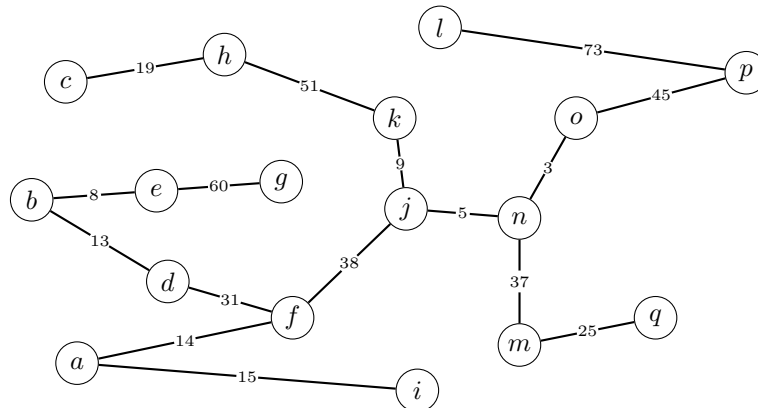
graphe en marquant chaque sommet par  $\pm 1$ . Chaque composante connexe du graphe est parcourue séparément. Initialement, le tampon utilisé pour le marquage vaut 1. À chaque traversée d'arête, on le multiplie par la valeur de l'arête. Lorsque le parcours débouche sur une arête déjà marquée, on vérifie que le tampon et la marque déjà faite sont concordantes. Si ce n'est pas le cas, c'est qu'il y a au moins trois opinions en présence. Dans le cas contraire, on a pu attribuer une des deux opinions à chaque invité. S'il n'y a qu'une seule composante connexe, on est sûr qu'il n'y a que deux opinions. Sinon, on ne peut pas le conclure avec certitude.

**Solution 3.108.** Soit  $n$  le nombre de sommet. Nous proposons la CNS « il existe un chemin de longueur  $n - 1$  ».

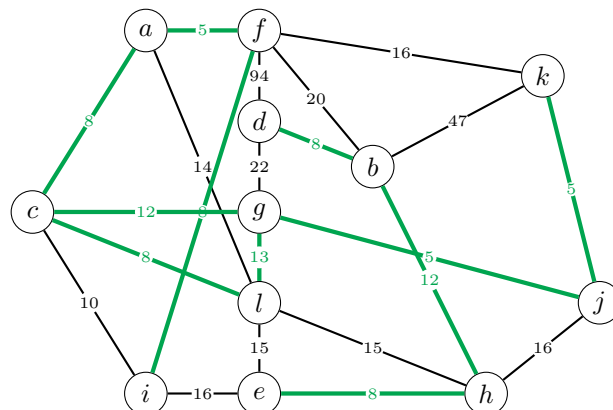
En effet, un chemin de longueur  $n - 1$  contient les  $n$  sommets, dont les numéros sont croissants et distincts.

Réciproquement, s'il existe deux numérotations topologiques  $\alpha$  et  $\beta$ , alors il existe deux sommets  $u$  et  $v$  tels que  $\alpha(u) < \alpha(v)$  et  $\beta(v) > \beta(u)$ . Mais alors, il n'existe pas de chemin entre  $u$  et  $v$ . Donc un chemin quelconque ne peut pas contenir  $u$  et  $v$  à la fois : il est forcément de longueur  $< n - 1$ .

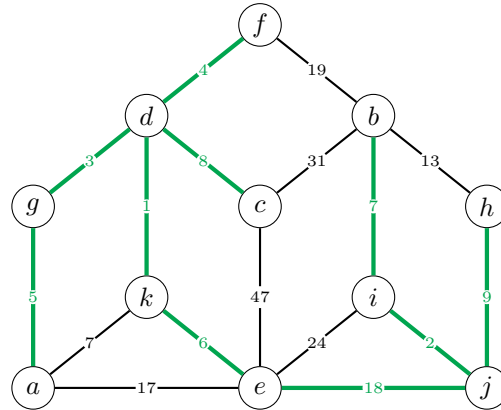
**Solution 3.109.** On trouve comme solution



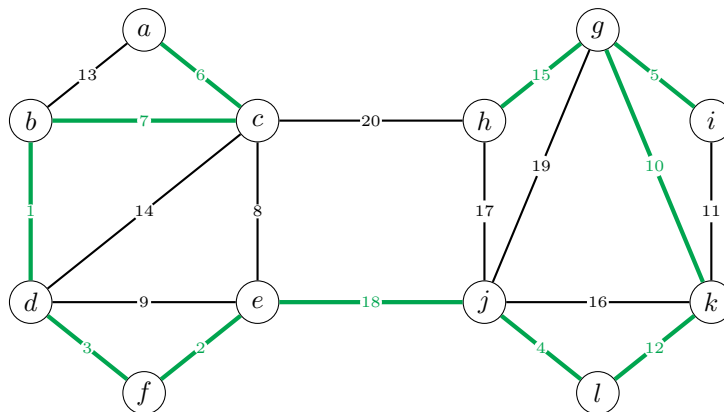
**Solution 3.110.** Une solution possible est



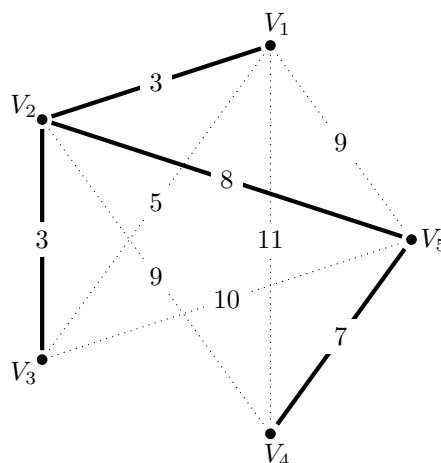
**Solution 3.111.** Une solution possible est



**Solution 3.112.** Une solution possible est

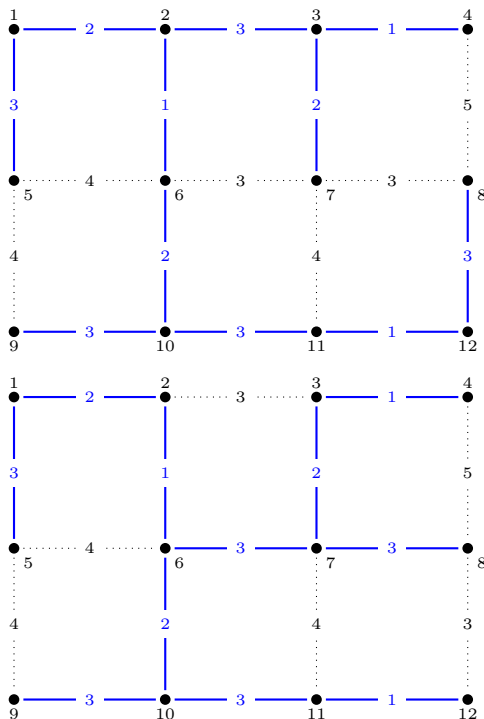


**Solution 3.113.** On construit l'arbre couvrant minimal suivant





**Solution 3.114.** Différentes solutions sont possibles, selon l'ordre par lequel on procède. En voici deux (il y en a d'autres)



- Solution 3.115.**
1. Comme tous les arbres ont  $n - 1$  arêtes, cela ajoute  $(n - 1)c$  au poids de tout arbre mais ne change pas lequel est minimal.
  2. Soit  $\delta > 0$  le montant de la diminution du poids. Le poids de  $T$  passe de disons  $P$  à  $P - \delta$ . Si un autre arbre  $T'$ , de poids  $P'$  devient l'arbre couvrant de poids minimal, alors  $P' \leq P - \delta < P$ , mais  $T'$  est initialement de poids  $P'$  également et  $P' < P$ , ce qui est exclu.

**Solution 3.116.** 1. Nous supposons  $G$  complet avec  $w(e) = +\infty$  s'il n'y a pas d'arête  $e$  initialement. Par récurrence sur  $n = |V|$ . Le cas de base ( $n = 2$ ) est évident.

Supposons avoir montré l'unicité d'un tel arbre pour tous les graphes à  $n - 1$  sommets. Un arbre couvrant minimal contient forcément l'unique arête de poids minimal  $e_0 = (u, v)$ . En effet, s'il existe un arbre  $T$  dépourvu de l'arête de poids minimal, on rajoute  $e_0$  à  $T$ , ce qui crée exactement un cycle  $C$ . On retire n'importe quelle arête de  $C$  différente de  $e_0$  et on a encore un arbre, dont le poids total est strictement inférieur au poids de  $T$ .

Soit à présent le graphe  $G' = (V', E')$  dans lequel les sommets  $u$  et  $v$  ont été fusionnés en un nouveau sommet  $\bar{u} + \bar{v}$ . Les sommets sont  $V' = V \setminus \{u, v\} \cup \{\bar{u} + \bar{v}\}$  et les poids des arêtes sont  $w'(e) = w(e)$  si  $e$  ne contient pas le sommet  $u$  ou  $v$  et  $w'(\bar{u} + \bar{v}, z) = \min(w(u, z), w(v, z))$ . Un arbre couvrant minimal  $T'$  de  $G'$  correspond bijectivement à un arbre couvrant minimal  $T$  de  $G$  (on ajoute

l'arête  $e_0$  et une arête  $(u + v, z)$  est remplacée par l'unique arête  $(u, z)$  ou  $(v, z)$  qui minimise le poids). Par hypothèse de récurrence, il n'y a qu'un seul arbre minimal dans  $G'$  donc dans  $G$  aussi.

2. Soit  $m = |E|$  et  $A \in \mathbb{N}$  tel que  $A > \sum_{i=1}^m i$ . On numérote les arêtes. On remplace tous les poids  $w(e_i)$  par  $w'(e) = A \cdot w(e) + i$ . Les poids sont désormais tous distincts car distincts modulo  $A$ . De plus, pour tout arbre  $S$ ,

$$A \cdot w(S) < w'(S) < A \cdot w(S) + A.$$

et par suite pour deux arbres  $S$  et  $S'$ ,  $w(S) = w(S')$  équivaut à

$$A \cdot w(S) < w'(S') < A \cdot w(S) + A.$$

Soient  $P = w(T)$  et  $P' = w(T')$  les poids totaux d'arbres couvrants  $T$  et  $T'$  minimaux pour  $w$  et  $w'$ . On a alors d'une part  $w(T) \leq w(T')$  (minimalité de  $T$ ) ce qui implique  $Aw(T) \leq Aw(T') < w'(T')$ . On a d'autre part  $w'(T') \leq w'(T)$ , ce qui implique  $w'(T') \leq w'(T) < Aw(T) + A$ . Ensemble on obtient :

$$Aw(T) < w'(T') < Aw(T) + A$$

d'où  $w(T) = w(T')$  comme voulu.

**Solution 3.117.** Soit  $T$  les sommets contenus dans  $S$ . On peut calculer les composantes connexes  $C_0$  de  $S$ , puis appliquer l'algorithme de Prim à  $C = C_0 \cup (V \setminus T)$  au départ d'un élément quelconque. On maintient une distance

---

**Algorithme 6.5 : PRIMPRIME**

---

**Entrées :**  $G = (V, E)$  graphe,  $S$  liste d'arêtes sans cycle

**Résultat :** arbre couvrant minimal contenant  $S$  enraciné en  $u_0$

1  $C_0 \leftarrow ComposantesConnexes(S)$

2  $T \leftarrow Sommets(S)$

3  $C \leftarrow (V \setminus T) \cup C_0$

4 **pour**  $v \in C$  **faire**

5      $d(v) = +\infty$

6  $v_0 = C[1]$

---

**Solution 3.118.** On construit l'arbre couvrant de poids minimal  $T$  du graphe complet sur  $V$  pondéré par  $d$ . On retire les  $k - 1$  arêtes les plus lourdes de  $T$ . On obtient une forêt de  $k$  arbres :  $A_1, A_2 \dots A_k$ , qui sont les  $k$  composantes connexes. On appelle  $(V_i)$  l'ensemble des sommets de  $A_i$ .

Preuve de correction : Il est facile de voir que l'espacement  $e$  du clustering obtenu est égal au poids de la dernière arête retirée, i.e. la plus légère. Notons  $e_{\max} \geq e$  l'espacement du clustering optimal.

Supposons que  $u$  et  $v$  sont deux objets reliées par une arête dans un certain arbre  $A_i$ , alors par construction  $d(u, v) \leq e$ . Mais si  $u$  et  $v$  se trouvent dans deux clusters séparés dans un clustering optimal, alors,  $e_{\max} \leq d(u, v)$ , ce qui montre que ou bien

notre clustering est optimal, ou  $u$  et  $v$  sont aussi dans le même cluster dans un clustering optimal. Mais les arbres  $A_i$  sont connectés. Nous avons montré que ou notre clustering est optimal, ou le clustering optimal lui est identique, ce qui revient au même !

**Solution 3.119.** Soit  $T = (V, E, p)$  un arbre pondéré à  $n$  sommets. Soit  $P$  son poids total et  $\epsilon > 0$ . On construit le graphe complet sur  $V$ , pondéré par  $w(e) = p(e)$  si  $e \in E$  et  $w(e) = P + \epsilon$  sinon. Alors l'arbre couvrant et l'arborescence au départ de n'importe quel sommet coïncident.

**Solution 3.120.** 1. Soit  $T$  l'arbre couvrant  $G$  de poids minimal. Comme les poids sont tous distincts, l'arbre  $T$  est unique (cf. exercice 3.116). Comme  $T$  est un arbre, rappelons qu'il existe un unique chaîne élémentaire entre deux sommets donnés.

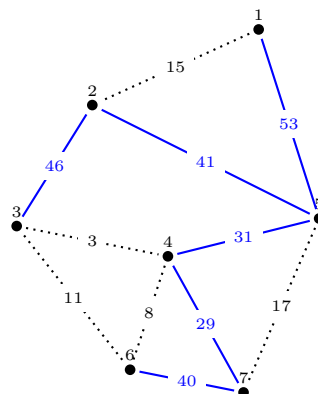
Appelons  $C_m$  un chemin d'étranglement minimal entre  $s$  et  $t$  et  $C_T$  l'unique chaîne élémentaire  $C_T$  entre  $s$  et  $t$  dans l'arbre  $T$ . Sans perte de généralité, nous pouvons supposer que  $C_m$  est élémentaire. Montrons que  $C_m = C_T$ .

Supposons que  $C_m$  contient une arête  $e$  qui n'est pas contenue dans  $C_T$ . Alors  $T \cup \{e\}$  contient un cycle  $\Sigma$  utilisant  $e$ . Comme  $T$  est un arbre minimal, toutes les arêtes de  $\Sigma$  sont de poids strictement inférieur à  $p(e)$ . Mais alors, remplacer  $e$  par  $\Sigma \setminus \{e\}$  dans  $C_m$  rendrait strictement inférieur la longueur d'étranglement, ce qui n'est pas possible. Donc  $C_m \subseteq T$ . Mais il n'existe qu'un seul chemin entre  $s$  et  $t$  dans  $T$ , donc  $C_m = C_T$ .

Un algorithme commencera par construire l'arbre couvrant de poids minimal. Si on utilise PRIM, on pourra partir de  $s$  et s'arrêter dès que  $t$  est touché. Puis, l'algorithme recherche un chemin de  $s$  à  $t$ , chose que l'on peut faire par un simple parcours du graphe  $T$  au départ de  $s$ , tout en indiquant pour chaque nouveau sommet découvert sa distance d'étranglement.

2. Dans le problème modélisé, on cherche un chemin entre deux routeurs dont le poids minimal des arêtes est aussi grand que possible. C'est donc un problème de distance d'étranglement dans lequel les comparaisons ont toutes été opposées.

Dans l'exemple, l'arbre couvrant de poids maximum est



la bande passante optimale entre 3 et 6 est 29 en utilisant les sommets 2, 5, 4, 7 comme relais.

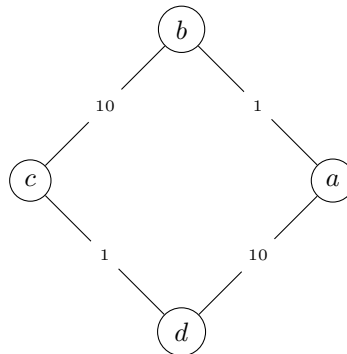
**Solution 3.121.** Appliquons l'algorithme de Prim en partant du sommet  $v_1$ . À l'étape  $k$ , l'algorithme ajoute le sommet  $v_{k+1}$  qui se trouve à distance  $k + 1$  de la composante en train de grossir. Ainsi le poids total de l'arbre construit est-il

$$\sum_{k=1}^{n-1} (k+1) = 1000 \cdot 1001/2 - 1 = 500499.$$

De cette manière, l'algorithme de Prim peut construire au moins  $999!$  arbres distincts. Pour voir qu'il n'y a pas d'autres arbres possibles, on peut démontrer que la restriction d'un arbre optimal  $T$  aux  $t$  premiers sommet est encore un arbre (i.e. il reste connecté). Si ce n'était pas le cas, on trouverait une arête  $e = (v_i, v_j)$  ( $i < j$ ) entre deux composantes de  $T \cap \llbracket 1, t \rrbracket$  mais alors en ajoutant  $e$  à  $T$  on créerait un cycle n'ayant pas que des arêtes de poids  $\leq t$ , ce qui prouverait que  $T$  n'est pas optimal.

**Solution 3.122.** On raisonne en imaginant le déroulement de l'algorithme de Kruskal : d'abord les 500 premiers sommets sont reliés entre eux, pour un coût de 499, puis les 500 derniers sommets sont reliés entre eux, pour un coût de  $2 \cdot 499$ , enfin, la composant des 500 premiers sommets est reliée à la composante des 500 derniers sommets pour un coût de 3. Le coût total est de 1500.

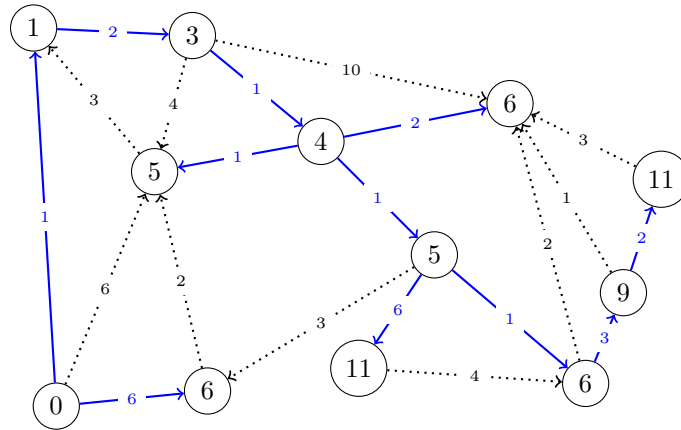
**Solution 3.123.** Il est erroné. On peut considérer le graphe à 4 sommets



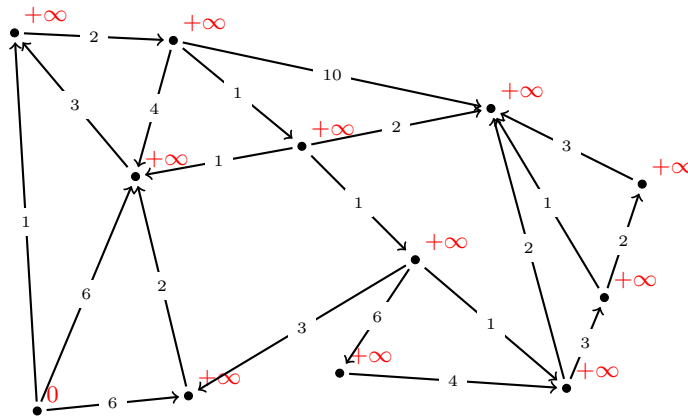
Ou coupe le graphe avec  $V_1 = \{a, b\}$  et  $V_2 = \{c, d\}$ . L'arbre obtenu est de poids 21 alors l'arbre minimum a pour poids 12.

**Solution 3.124.** 1. Algorithme de Dijkstra pour le graphe orienté

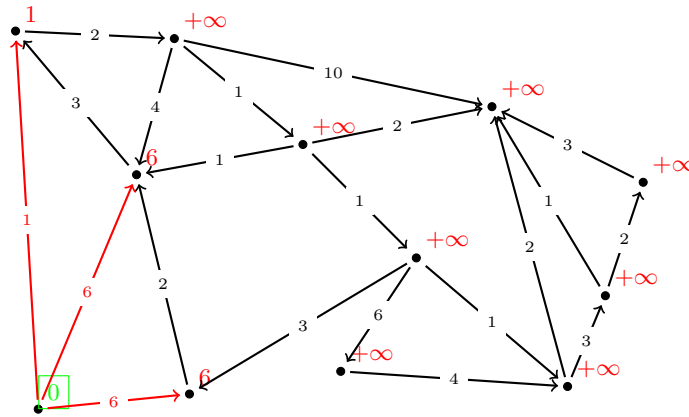
On obtient les plus courtes distances et l'arborescence suivantes :



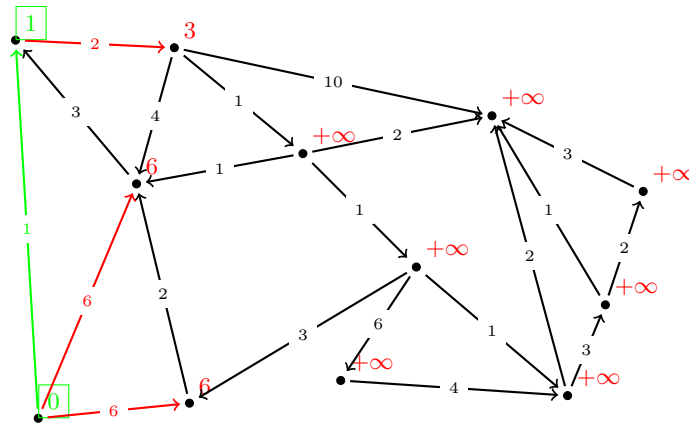
Voici les détails de l'exécution de l'algorithme. Nous représentons en rouge les mentions (distances et pères) provisoires et en vert les mentions définitives. À l'étape 0, nous commençons par :



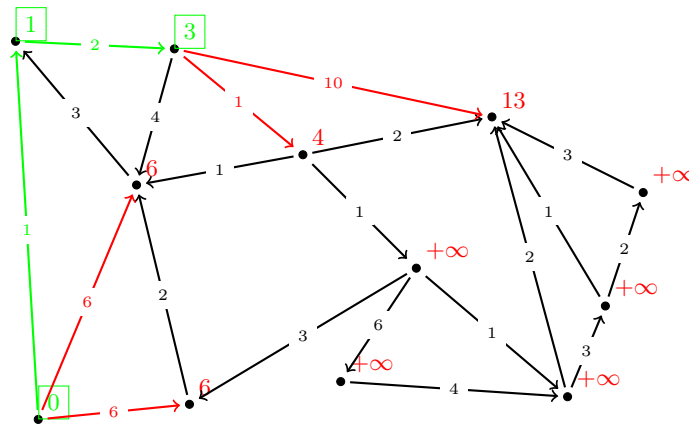
À l'étape 1 :



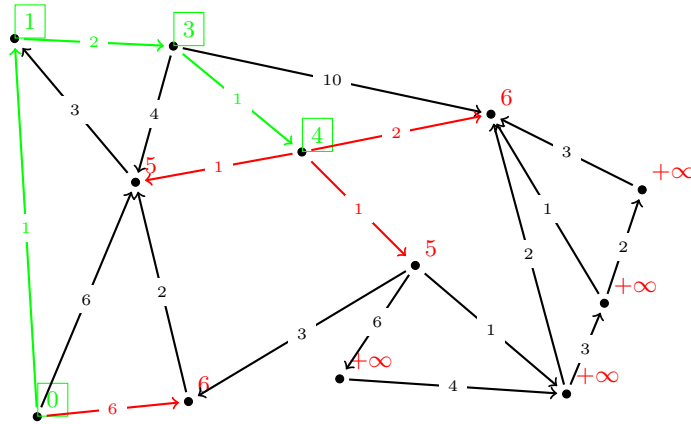
À l'étape 2 :



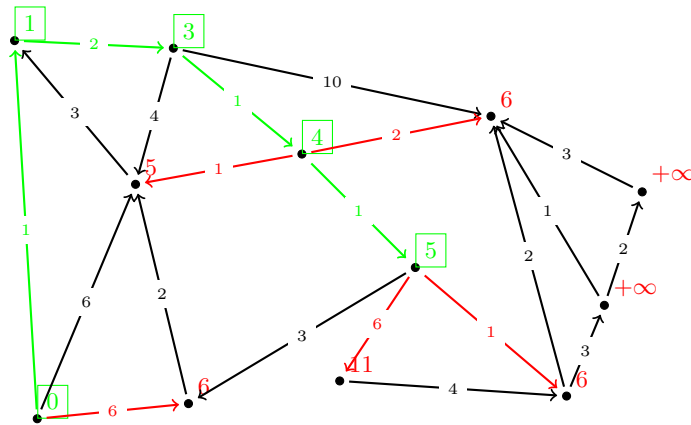
À l'étape 3 :



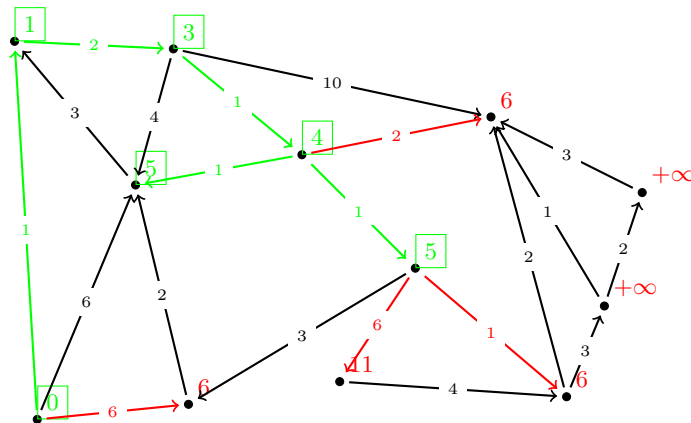
À l'étape 4 :



À l'étape 5 (nous aurions le choix de prendre le sommet no 5 plutôt que le no 9 comme pivot ) :



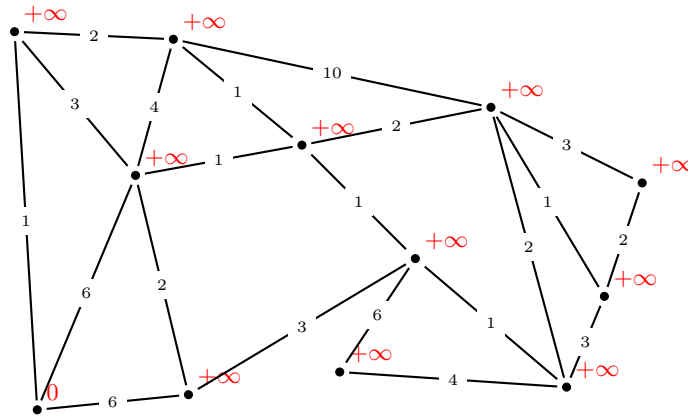
À l'étape 6 :



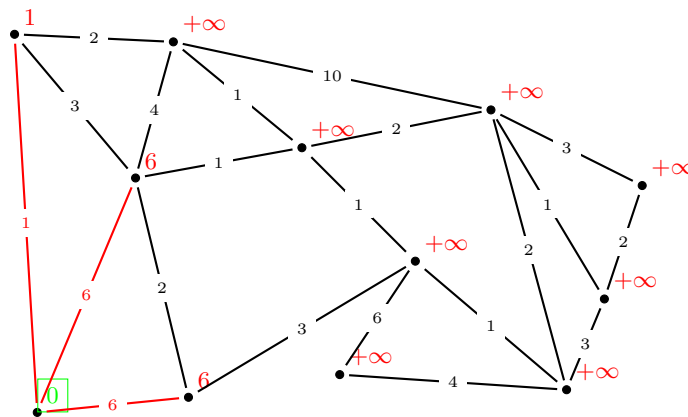
et ainsi de suite jusqu'au résultat final.

## 2. Algorithme de Dijkstra pour le graphe non-orienté

Nous représentons toujours en rouge les mentions (distances et pères) provisoires et en vert les mentions définitives. À l'étape 0, nous commençons par :

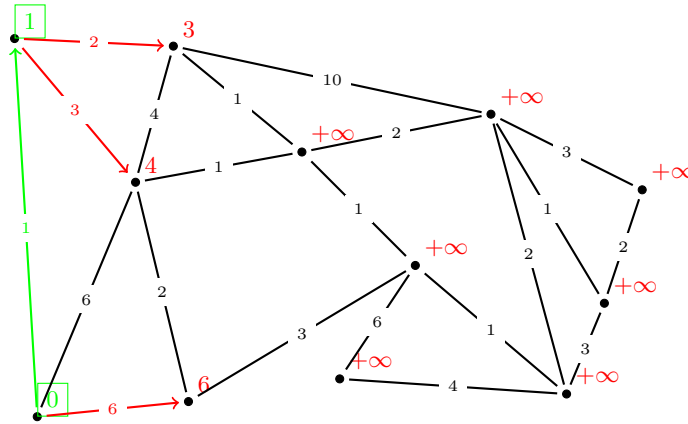


À l'étape 1 :

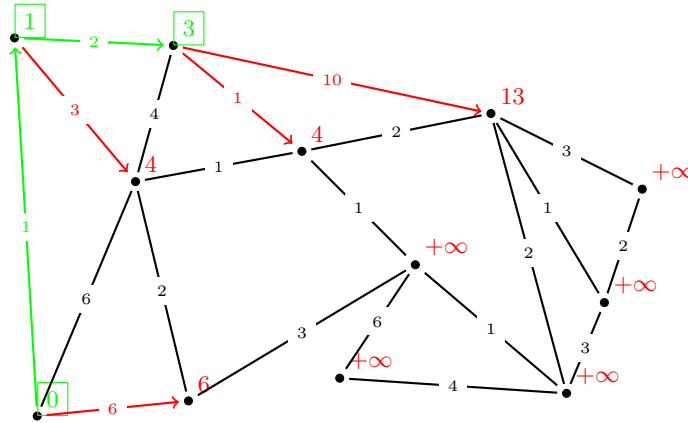


À l'étape 2 (nous observons un premier changement avec le cas orienté) :

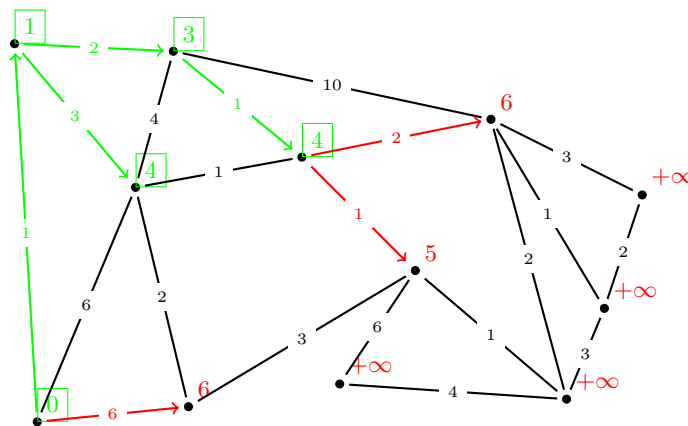




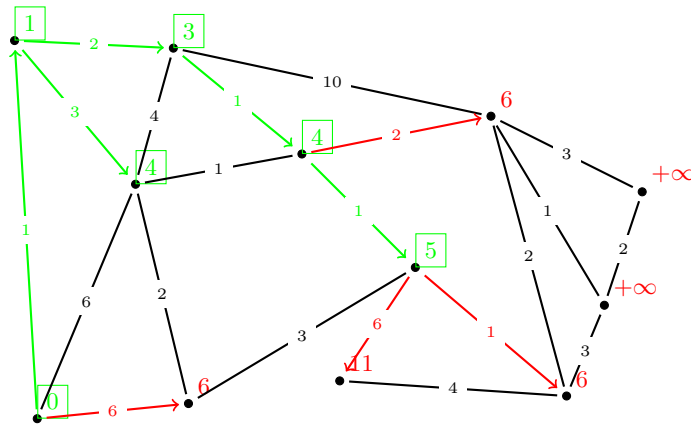
À l'étape 3 :



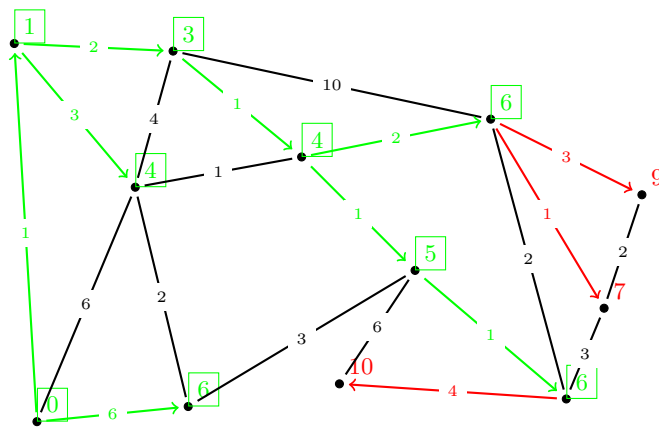
À l'étape 5 (nous laissons au lecteur le soin de représenter l'étape 4) :



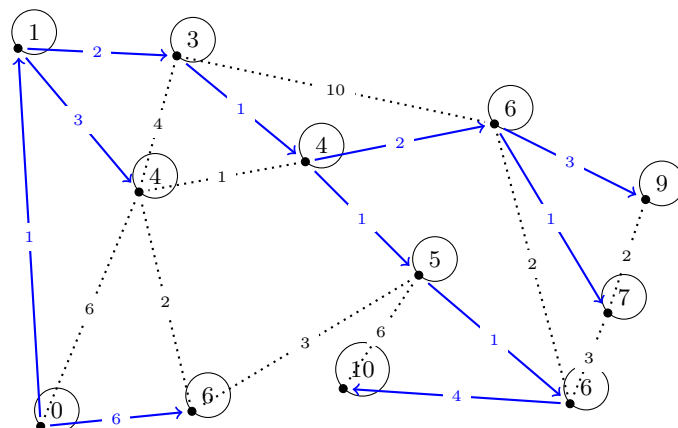
À l'étape 6 :



À l'étape 9 :



Finalement on obtient les plus courts chemins suivants :



3. Application de l'algorithme de Ford au graphe orienté. Nous notons  $\ell^{(k)}(v)$  la distance du sommet 1 au sommet  $v$  en moins de  $k$  arcs.

L'initialisation est la suivante :

| $v$             | 1 | 2         | 3         | 4         | 5         | 6         | 7         | 8         | 9         | 10        | 11        | 12        |
|-----------------|---|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $\ell^{(0)}(v)$ | 0 | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |

Rappelons qu'ensuite nous devons calculer  $\ell^{(k)}$  par la formule

$$\ell^{(k+1)}(v) = \min\{\ell^{(k)}(v)\} \cup \{\ell^{(k)}(u) + c(u, v); u \in V \text{ t.q. } (u, v) \in E\}$$

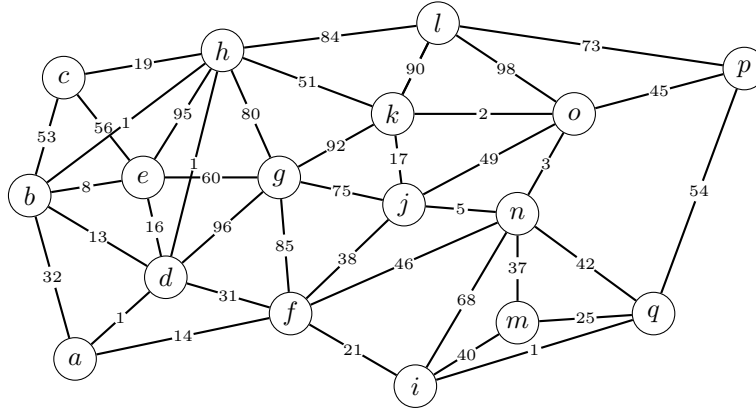
Autrement dit, ici, nous avons à appliquer les formules

$$\left\{ \begin{array}{ll} \ell^{(k+1)}(1) &= \ell^{(k)}(1) \\ \ell^{(k+1)}(2) &= \min\{\ell^{(k)}(2), \ell^{(k)}(4) + 1\} \\ \ell^{(k+1)}(3) &= \min\{\ell^{(k)}(3), \ell^{(k)}(11) + 2\} \\ \ell^{(k+1)}(4) &= \min\{\ell^{(k)}(4), \ell^{(k)}(12) + 2\} \\ \ell^{(k+1)}(5) &= \min\{\ell^{(k)}(5), \ell^{(k)}(1) + 6, \ell^{(k)}(8) + 2, \ell^{(k)}(2) + 1\} \\ \ell^{(k+1)}(6) &= \min\{\ell^{(k)}(6), \ell^{(k)}(4) + 10, \ell^{(k)}(2) + 2, \ell^{(k)}(10) + 2, \ell^{(k)}(11) + 1, \ell^{(k)}(3) + 2\} \\ \ell^{(k+1)}(7) &= \min\{\ell^{(k)}(7), \ell^{(k)}(9) + 6\} \\ \ell^{(k+1)}(8) &= \min\{\ell^{(k)}(8), \ell^{(k)}(1) + 6, \ell^{(k)}(9) + 3\} \\ \ell^{(k+1)}(9) &= \min\{\ell^{(k)}(9), \ell^{(k)}(2) + 1\} \\ \ell^{(k+1)}(10) &= \min\{\ell^{(k)}(10), \ell^{(k)}(7) + 4, \ell^{(k)}(9) + 1\} \\ \ell^{(k+1)}(11) &= \min\{\ell^{(k)}(11), \ell^{(k)}(10) + 3\} \\ \ell^{(k+1)}(12) &= \min\{\ell^{(k)}(12), \ell^{(k)}(1) + 1\} \end{array} \right.$$

Nous obtenons par calcul successif :

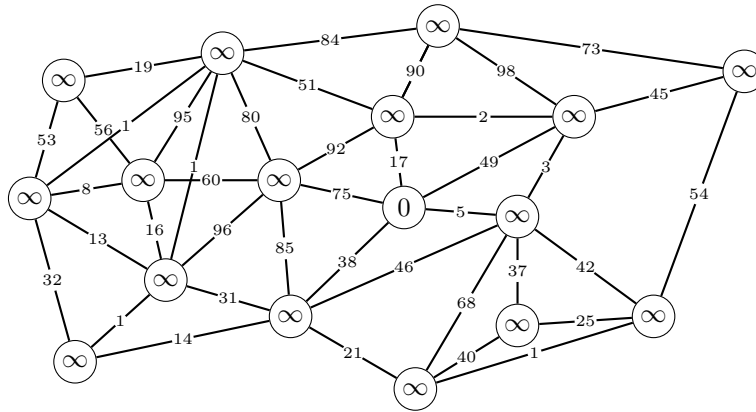
| $v$             | 1 | 2         | 3         | 4         | 5         | 6         | 7         | 8         | 9         | 10        | 11        | 12        |
|-----------------|---|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $\ell^{(0)}(v)$ | 0 | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| $\ell^{(1)}(v)$ | 0 | $+\infty$ | $+\infty$ | $+\infty$ | 6         | $+\infty$ | $+\infty$ | 6         | $+\infty$ | $+\infty$ | $+\infty$ | 1         |
| $\ell^{(2)}(v)$ | 0 | $+\infty$ | $+\infty$ | 3         | 6         | $+\infty$ | $+\infty$ | 6         | $+\infty$ | $+\infty$ | $+\infty$ | 1         |
| $\ell^{(3)}(v)$ | 0 | 2         | $+\infty$ | 3         | 6         | 13        | $+\infty$ | 6         | $+\infty$ | $+\infty$ | $+\infty$ | 1         |
| $\ell^{(4)}(v)$ | 0 | 2         | $+\infty$ | 3         | 5         | 6         | $+\infty$ | 6         | 5         | $+\infty$ | $+\infty$ | 1         |
| $\ell^{(5)}(v)$ | 0 | 2         | $+\infty$ | 3         | 5         | 6         | 11        | 6         | 5         | 6         | $+\infty$ | 1         |
| $\ell^{(6)}(v)$ | 0 | 2         | $+\infty$ | 3         | 5         | 6         | 11        | 6         | 5         | 6         | 9         | 1         |
| $\ell^{(6)}(v)$ | 0 | 2         | 11        | 3         | 5         | 6         | 11        | 6         | 5         | 6         | 9         | 1         |

**Solution 3.125.** Le graphe à étudier est le suivant

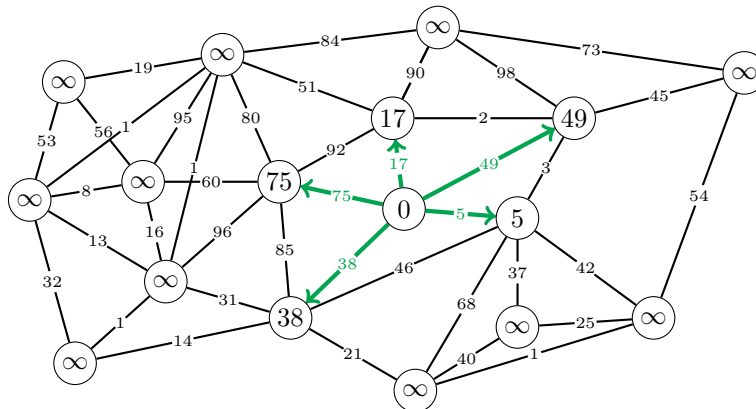


Voici dans le détail comment l'algorithme de Dijkstra procède.

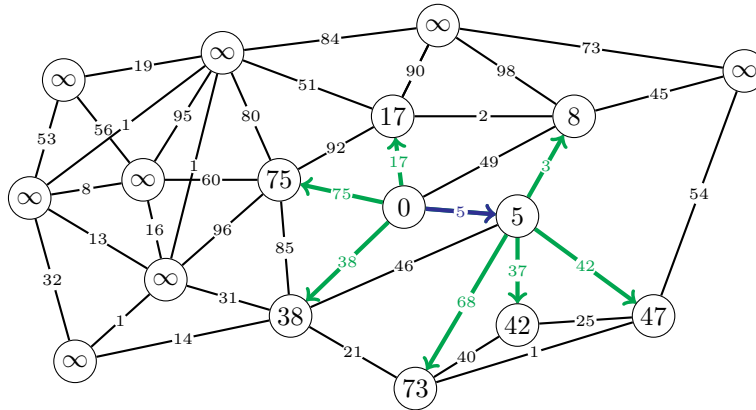
ÉTAPE 0 (DIJKSTRA) : À l'initialisation, les attributs de chaque sommet du graphe sont les suivants.



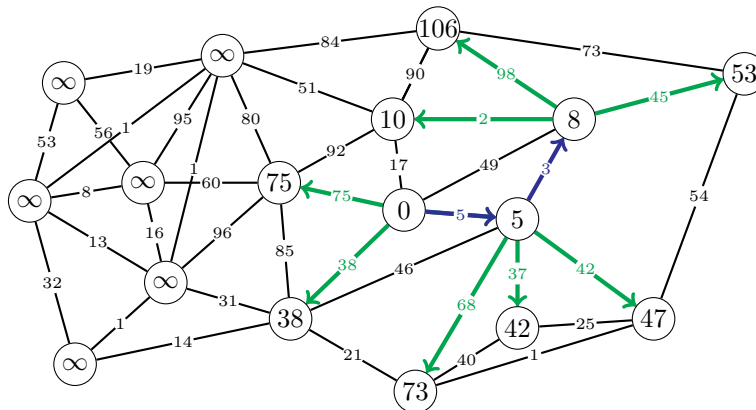
ÉTAPE 1 (DIJKSTRA) : On marque le sommet  $j$  ; on étudie les voisins de  $j$ . Il en résulte



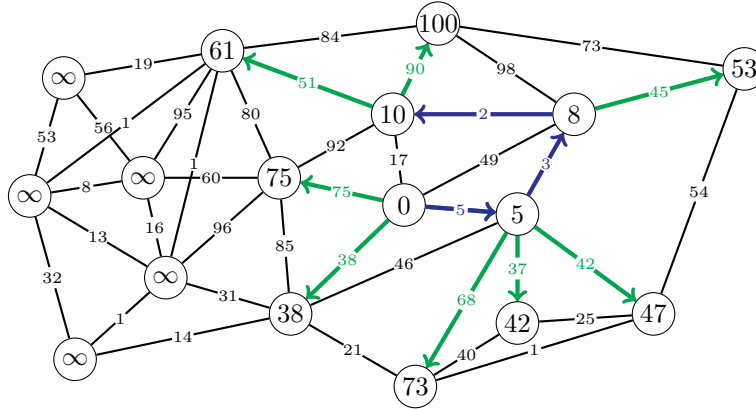
ÉTAPE 2 (DIJKSTRA) : On marque le sommet  $n$  ; on étudie les voisins de  $n$ . Il en résulte



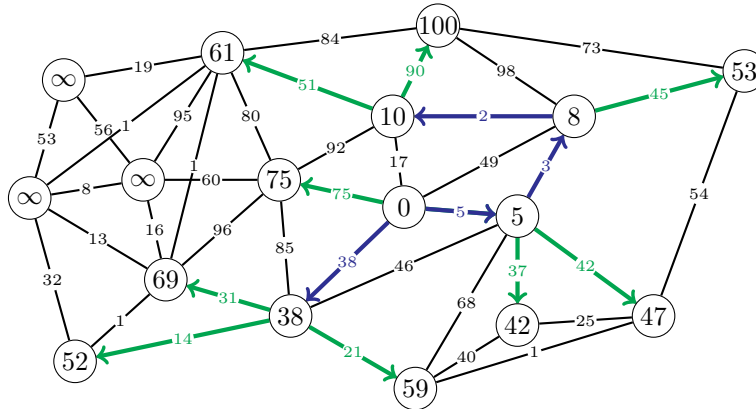
ÉTAPE 3 (DIJKSTRA) : On marque le sommet  $o$  ; on étudie les voisins de  $o$ . Il en résulte



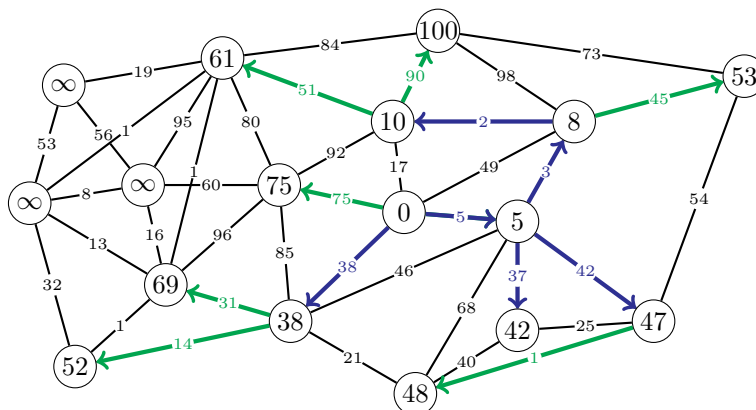
ÉTAPE 4 (DIJKSTRA) : On marque le sommet  $k$  ; on étudie les voisins de  $k$ . Il en résulte



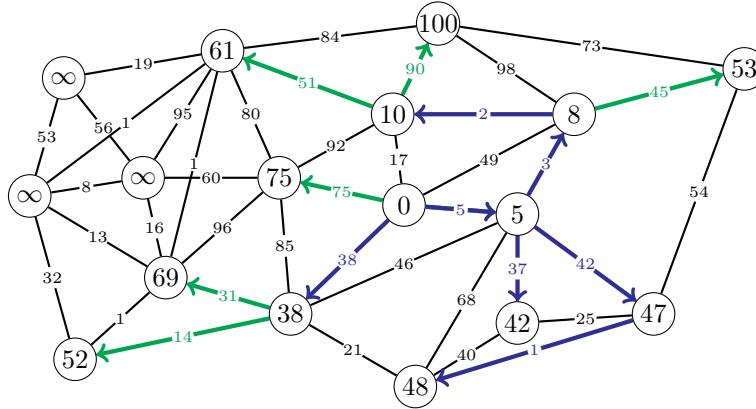
ÉTAPE 5 (DIJKSTRA) : On marque le sommet  $f$  ; on étudie les voisins de  $f$ . Il en résulte



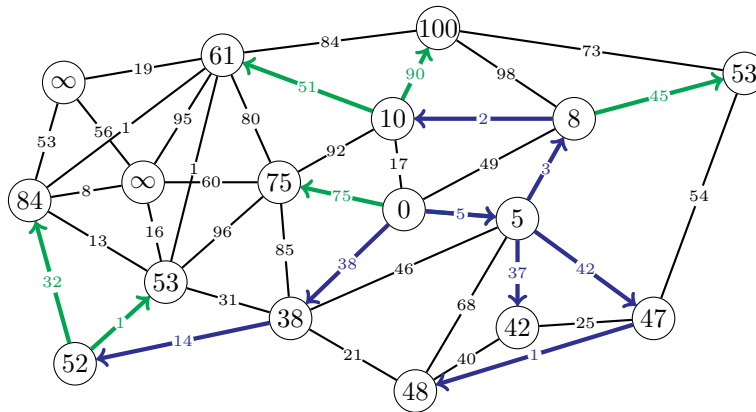
ÉTAPE 6 (DIJKSTRA) : On marque le sommet  $m$  et le sommet  $q$ . Il en résulte, sans grands changements,



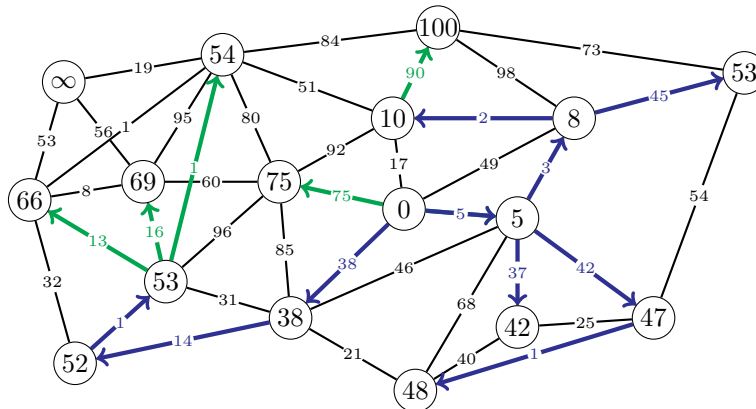
ÉTAPE 7 (DIJKSTRA) : On marque le sommet  $i$ . Il en résulte, sans changements,



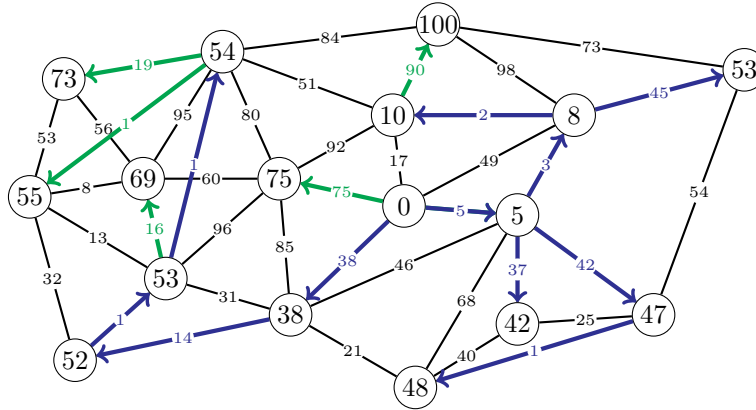
ÉTAPE 8 (DIJKSTRA) : On marque le sommet  $a$  et on étudie ses voisins non encore marqués. Il en résulte



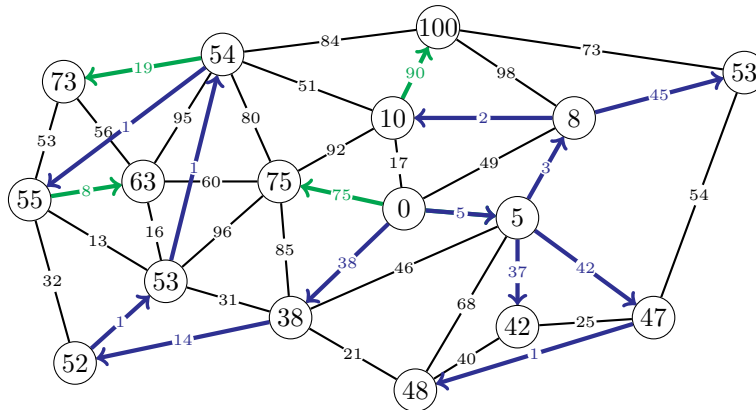
ÉTAPE 9 (DIJKSTRA) : On marque le sommet  $p$  (rien ne change). On marque le sommet  $d$  et on étudie ses voisins non encore marqués. Il en résulte



ÉTAPE 10 (DIJKSTRA) : On marque le sommet  $h$  et on étudie ses voisins non encore marqués. Il en résulte

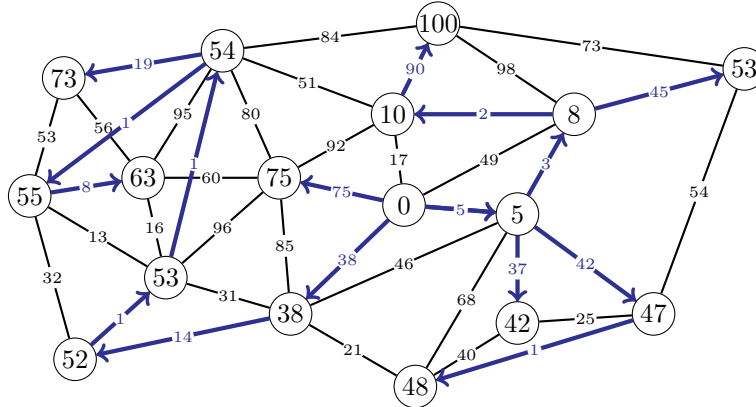


ÉTAPE 11 (DIJKSTRA) : On marque le sommet  $b$  et on étudie ses voisins non encore marqués. Il en résulte



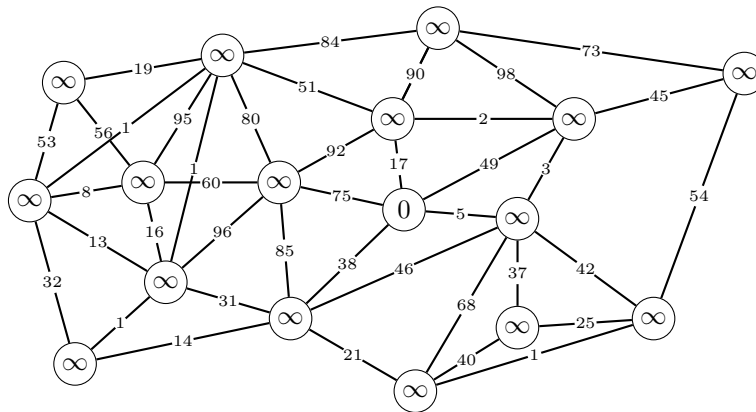
ÉTAPES RESTANTES (DIJKSTRA) : Les sommets à distance 63 (sommet  $e$ ), 73 (sommet  $c$ ), 75 (sommet  $g$ ) et 100 (sommet  $l$ ) sont successivement marqué dans cet ordre, sans que les distances provisoires ne changent plus.



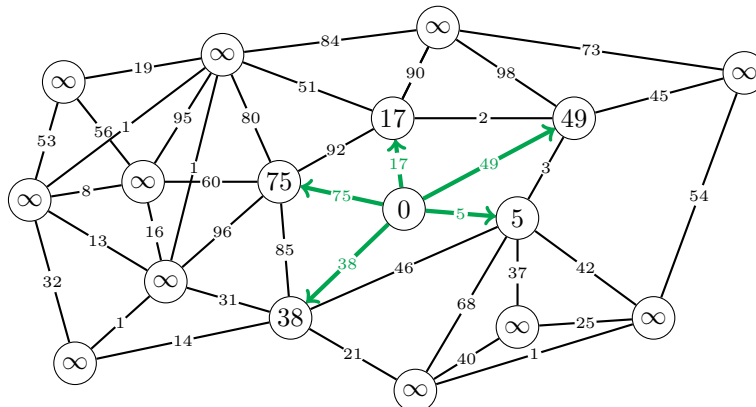


Voici dans le détail comment l'algorithme de Bellman-Ford procède.

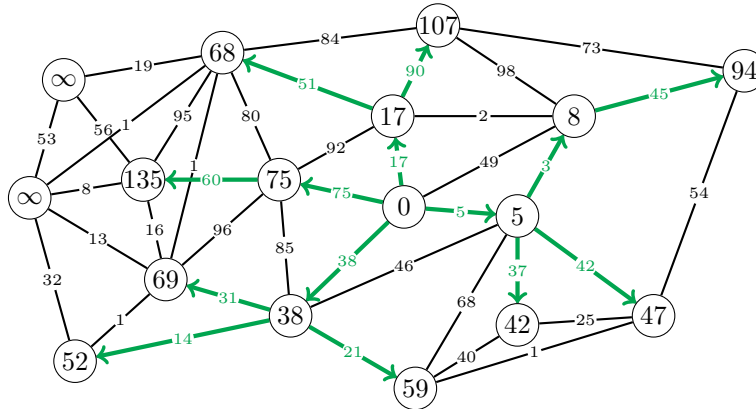
ÉTAPE 0 (BELLMAN-FORD) : À l'initialisation, les attributs de chaque sommet du graphe sont les suivants.



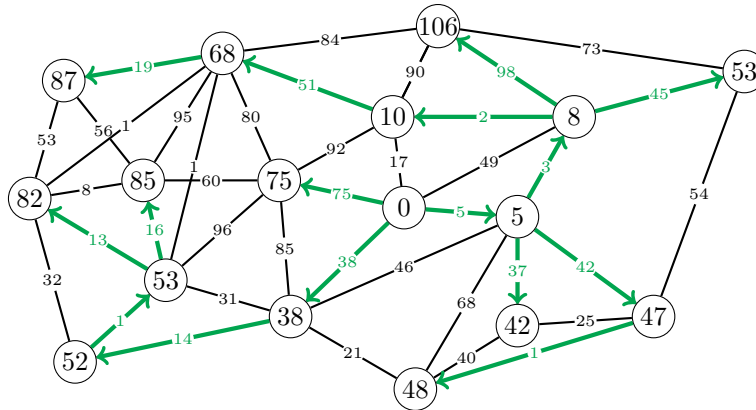
ÉTAPE 1 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 0.



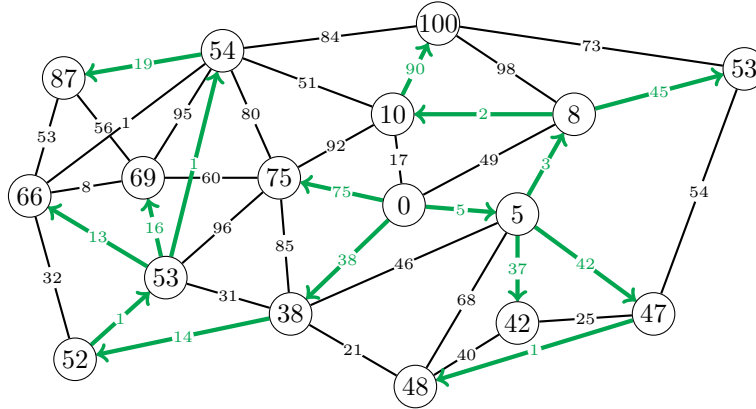
ÉTAPE 2 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 1.



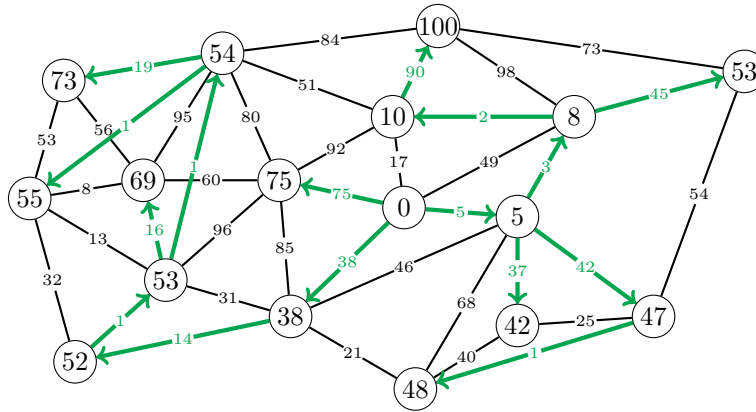
ÉTAPE 3 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 2.



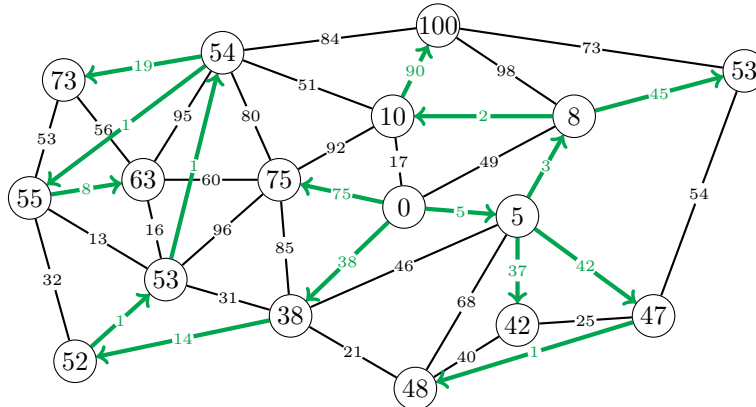
ÉTAPE 4 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 3.



ÉTAPE 5 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 4.



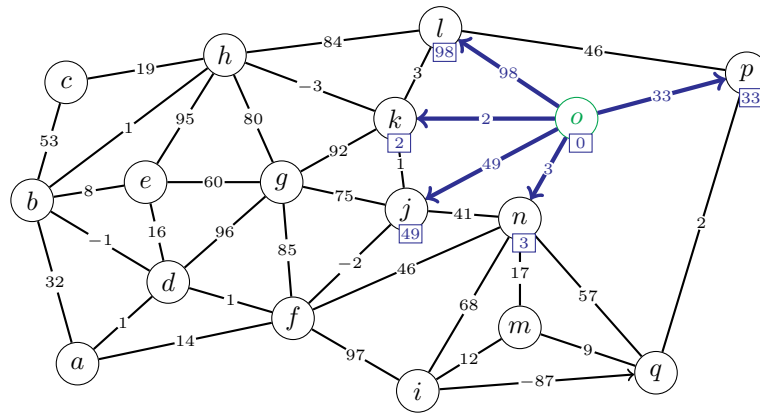
ÉTAPE 6 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 5.



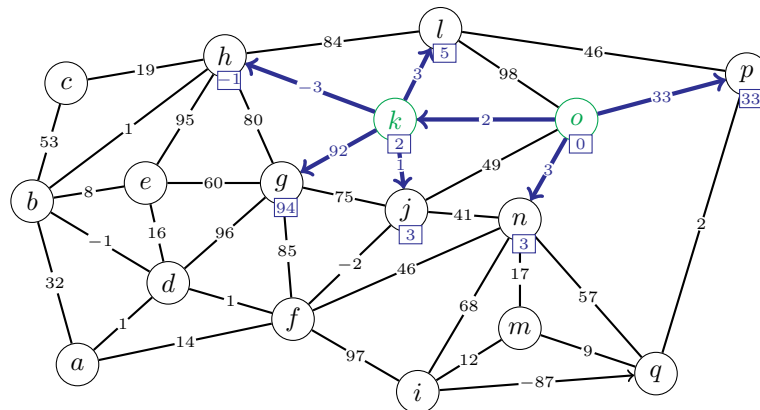
ÉTAPE 7 (BELLMAN-FORD) : Pour chacun des sommets, on essaye d'améliorer la distance en utilisant un prédécesseur  $z$  et la distance calculée de  $j$  à  $z$  à l'étape 6. Il n'y a plus de changements. L'algorithme est terminé.

**Solution 3.126.** 1. On applique l'algorithme de Dijkstra au départ du sommet  $o$ . Comme certains coûts sont négatifs, on n'a aucune garantie que sur le résultat de l'algorithme.

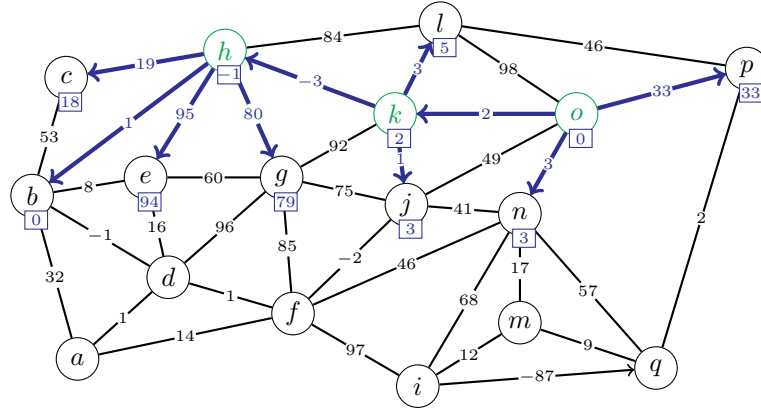
ÉTAPE 1 : On étudie les voisins de  $o$



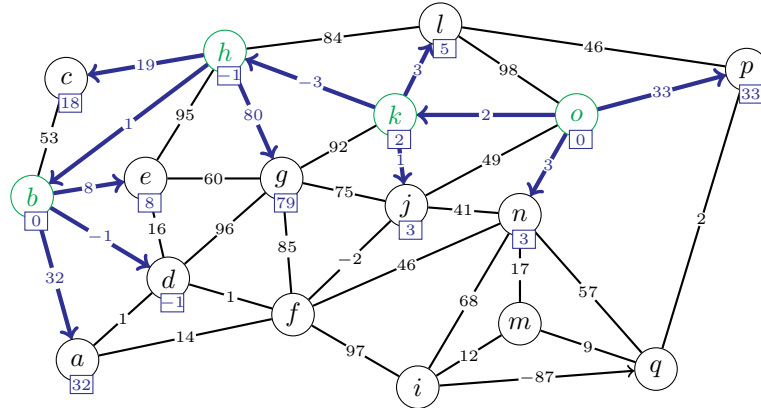
ÉTAPE 2 : On étudie les voisins de  $k$



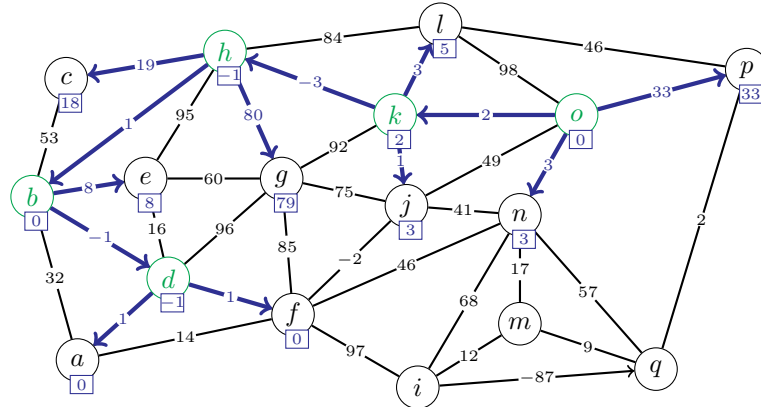
ÉTAPE 3 : On étudie les voisins de  $h$



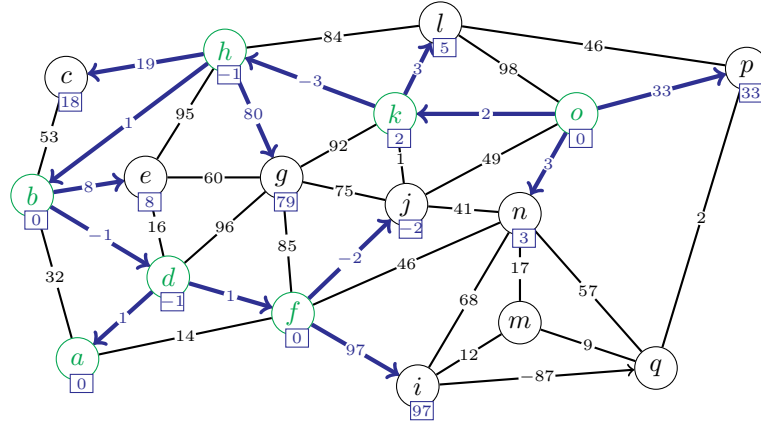
ÉTAPE 4 : On étudie les voisins de  $b$



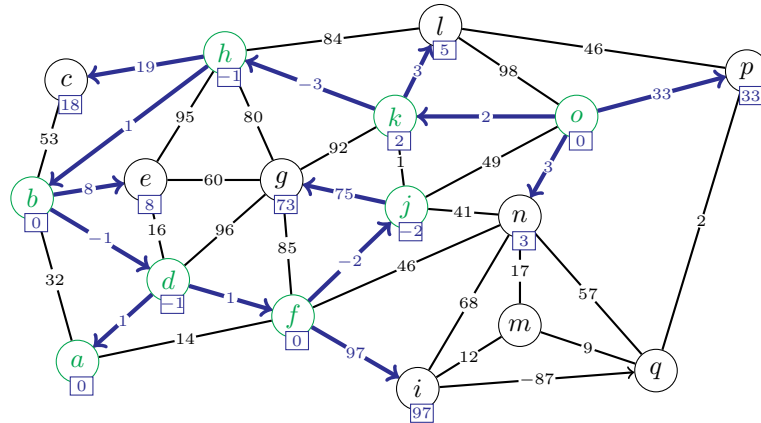
ÉTAPE 5 : On étudie les voisins de  $d$



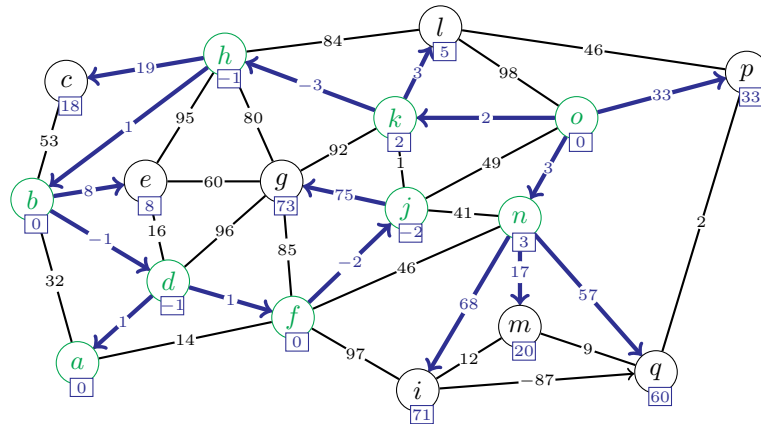
ÉTAPE 6 : On étudie les voisins de  $a$ , il ne se passe rien. On étudie les voisins de  $f$



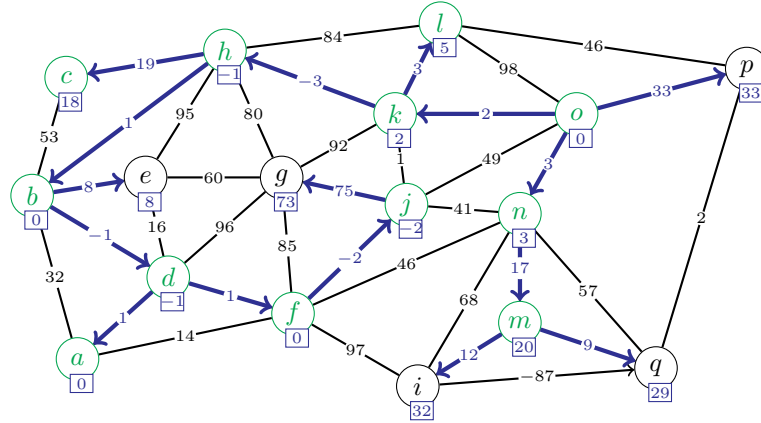
ÉTAPE 7 : On étudie les voisins de  $j$



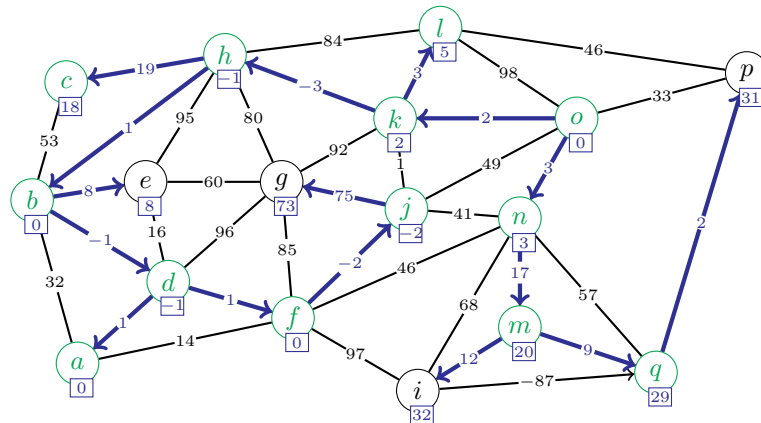
ÉTAPE 8 : On étudie les voisins de  $n$



ÉTAPE 9 : On étudie les voisins de  $l$ , de  $c$  et de  $m$



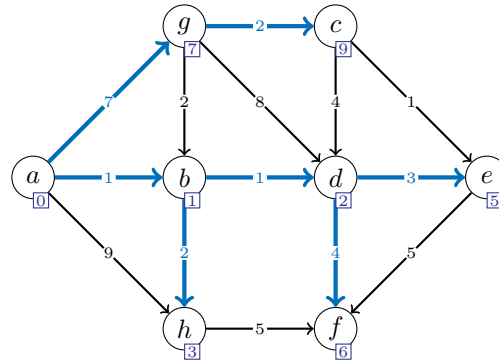
ÉTAPE 10 : On étudie les voisins  $q$



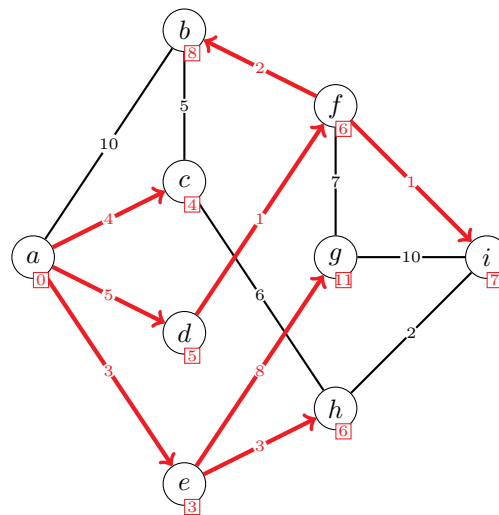
À ce stade, l'arborescence ne change plus. L'algorithme se termine avec les mêmes inscriptions.

2. On note la présence d'un cycle absorbant :  $b - h - k - j - f - d$ . Comme tous les sommets sont connectés à ce cycle, la distance de  $o$  à n'importe quel sommet est  $-\infty$ .
3. L'algorithme de Ford-Dantzig pourrait repérer l'arc  $(i, q)$  et mettre à jour la distance de  $q$  ainsi que ses descendants (ici seulement  $p$ ).
4. L'algorithme de Ford-Dantzig pourrait repérer l'arc  $(j, k)$  et détecter ainsi un cycle absorbant.

**Solution 3.127.** On a comme solution



**Solution 3.128.** On a comme solution



**Solution 3.129.** On applique l'algorithme aux poids suivants

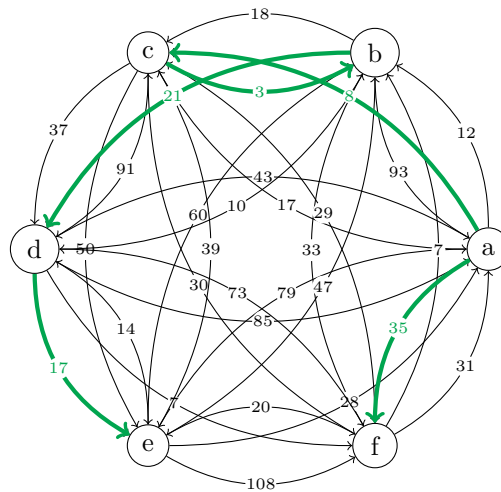
|          | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> |
|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | 0        | 12       | 8        | 43       | 79       | 35       |
| <i>b</i> | 93       | 0        | 18       | 21       | 60       | 33       |
| <i>c</i> | 17       | 3        | 0        | 37       | 50       | 30       |
| <i>d</i> | 85       | 10       | 91       | 0        | 17       | 7        |
| <i>e</i> | 28       | 47       | 39       | 14       | 0        | 108      |
| <i>f</i> | 31       | 7        | 29       | 73       | 20       | 0        |



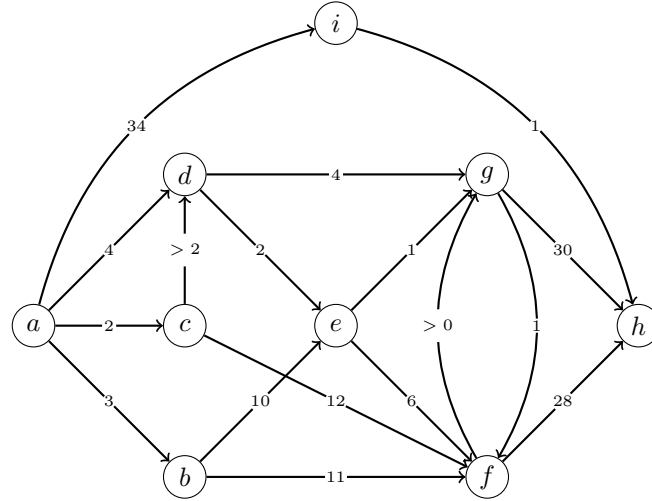
On obtient les itérations suivantes :

| sommet   | $\pi(a)$<br>père(a) | $\pi(b)$<br>père(b) | $\pi(c)$<br>père(c) | $\pi(d)$<br>père(d) | $\pi(e)$<br>père(e) | $\pi(f)$<br>père(f) |
|----------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| <i>a</i> | 0                   | 12                  | 8                   | 43                  | 79                  | 35                  |
|          |                     | <i>a</i>            | <i>a</i>            | <i>a</i>            | <i>a</i>            | <i>a</i>            |
| <i>c</i> |                     | 11                  |                     | 43                  | 58                  | 35                  |
|          |                     | <i>c</i>            |                     | <i>a</i>            | <i>c</i>            | <i>a</i>            |
| <i>b</i> |                     |                     |                     | 32                  | 58                  | 35                  |
|          |                     |                     |                     | <i>b</i>            | <i>c</i>            | <i>a</i>            |
| <i>d</i> |                     |                     |                     |                     | 49                  | 35                  |
|          |                     |                     |                     |                     | <i>d</i>            | <i>a</i>            |
| <i>f</i> |                     |                     |                     |                     | 49                  |                     |
|          |                     |                     |                     |                     | <i>d</i>            |                     |

Ou encore graphiquement



**Solution 3.130.** On obtient :



**Solution 3.131.** On peut répondre directement par la suite

$$(0, 0) \rightarrow (5, 0) \rightarrow (2, 3) \rightarrow (2, 0) \rightarrow (0, 2) \rightarrow (5, 2) \rightarrow (4, 3) \rightarrow (4, 0).$$

On peut aussi modéliser la situation par un graphe orienté dont les sommets représentent les états possibles des seaux et les arêtes les actions possibles. Une solution optimale est un *plus court chemin* entre le nœud  $(0, 0)$  et le nœud  $(4, 0)$ , que l'on calcule par l'algorithme de Dijkstra.

Plus précisément, soit  $V$  l'ensemble des sommets

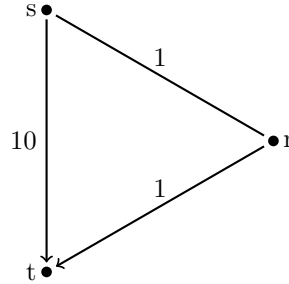
$$V = \{S_{u,v}; u \in [0, 5], v \in [0, 3]\}.$$

Nous pouvons nous restreindre à des valeurs entières puisque toute opération conserve cette propriété (dans un cas général, nous pourrions uniquement considérer les multiples du pgcd de 3 et 5, qui ici vaut 1). Les opérations possibles sont :

- vider un seau : arête  $S_{u,v} \rightarrow S_{u,0}$  et  $S_{u,v} \rightarrow S_{0,v}$ .
- remplir un seau complètement : arête  $S_{u,v} \rightarrow S_{u,3}$  et  $S_{u,v} \rightarrow S_{5,v}$ .
- renverser le contenu d'un seau dans un autre : arête  $S_{u,v} \rightarrow S_{0,u+v}$  si  $u+v \leq 3$  ou arête  $S_{u,v} \rightarrow S_{u+v-3,3}$  si  $u+v > 3$  et arête  $S_{u,v} \rightarrow S_{u+v,0}$  si  $u+v \leq 5$  ou arête  $S_{u,v} \rightarrow S_{5,u+v-5}$  si  $u+v > 5$ .

**Solution 3.132.** 1. Oui, sous réserve que la constante soit positive. Si elle est nulle, tout chemin convient. Si elle est négative, il se peut que des circuits absorbants soient créés.

2. Non. Voici un contre-exemple avec  $c = 1000$  où  $V = \{s, t, r\}$ .



**Solution 3.133.** Pour un graphe avec  $m = \Omega(n)$ , l'algorithme de Dijkstra coûte moins que celui de Bellman-Ford : le premier est en  $O(n^2)$  au pire et le second en  $O(nm)$ . De plus, si au cours de l'algorithme de Dijkstra, on arrive à choisir  $t$  comme pivot, on peut terminer l'algorithme. Avec l'algorithme de Bellman-Ford, on doit nécessairement attendre la fin du calcul. Dans le cas du graphe complet  $K_n$  dont tous poids sont à 1, on peut finir Dijkstra en  $O(1)$  tandis que le coût reste en  $O(n^3)$  pour Bellman-Ford.

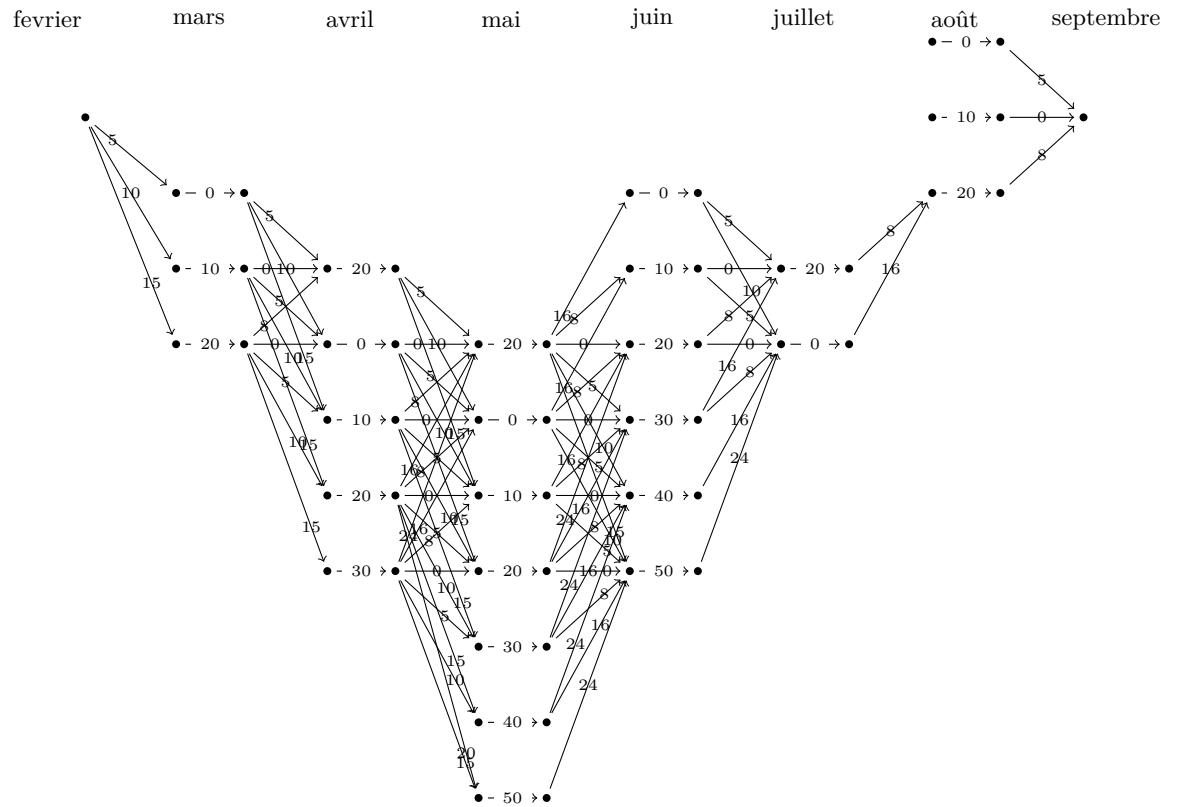
- Solution 3.134.**
1. Les sous-suites communes sont  $AB$  et  $AC$ .
  2. On attribue le poids 1 aux arêtes horizontales et verticales et 0 aux diagonales.
  3. En lisant les sommets lignes par ligne ou colonne par colonne, on obtient un tri topologique du graphe. L'algorithme le plus approprié est celui de Bellman. Comme chaque sommet est de degré entrant inférieur à 3, il fonctionne en  $O(mn)$ .

**Solution 3.135.** Nous considérons un ensemble de sommets  $V \subseteq \{(m, n); m \text{ mois}, n \in \mathbb{N}\}$ . Les mois  $m$  sont entre février et août ;  $m + 1$  désigne le mois suivant. Limitons à priori les valeurs possibles de  $n$ . Le nombre  $n$  est majoré par  $n \leq 6$  en mars,  $n \leq 9$  en avril,  $n \leq 12$  en mai, etc. Il est aussi majoré par la contrainte de renvoi, soit par 4 en août, 6 en juillet, etc. Il est minoré pour chaque mois par  $4/5$  de la capacité indiquée par le tableau. On obtient comme borne les nombres suivants

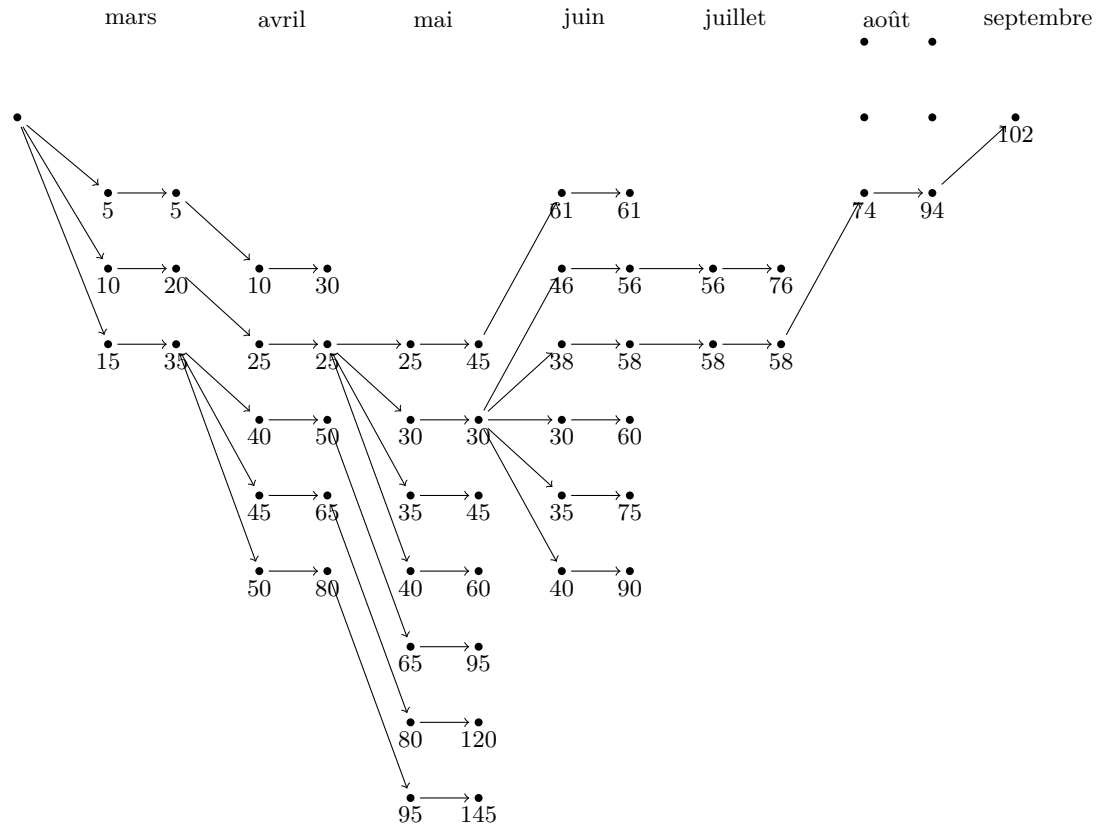
| Mois       | Mars  | Avril | Mai    | Juin  | Juillet | Août  |
|------------|-------|-------|--------|-------|---------|-------|
| Inf        | 3,2   | 4,8   | 5,6    | 3,2   | 4,8     | 1,6   |
| Sup        | 6     | 9     | 12     | 15    | 18      | 21    |
| Sup        | 28    | 19    | 13     | 9     | 6       | 4     |
| Intervalle | [4,6] | [5,9] | [6,12] | [4,9] | [5,6]   | [2,4] |

Pour  $-n/3 \leq k \leq 3$ , on place un arc entre  $(m, n)$  et  $(m + 1, n + k)$  de poids 0 si  $k = 0$ , de poids  $5k$  si  $k > 0$  et  $8k$  si  $k < 0$ .

Un sommet en déficit, respectivement en excès, de  $k$  ouvriers est divisé en deux demi-sommets reliés par un arc de poids  $20k$ , respectivement de poids  $10k$ . Le demi-sommet initial conserve toutes les arêtes entrantes, le demi-sommet final les arêtes sortantes.

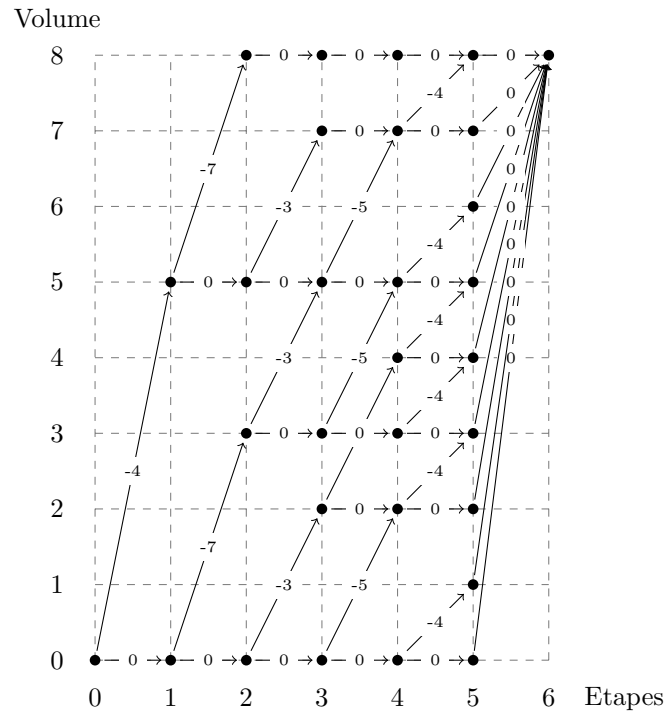


L'algorithme de fournit l'arborescence suivante :

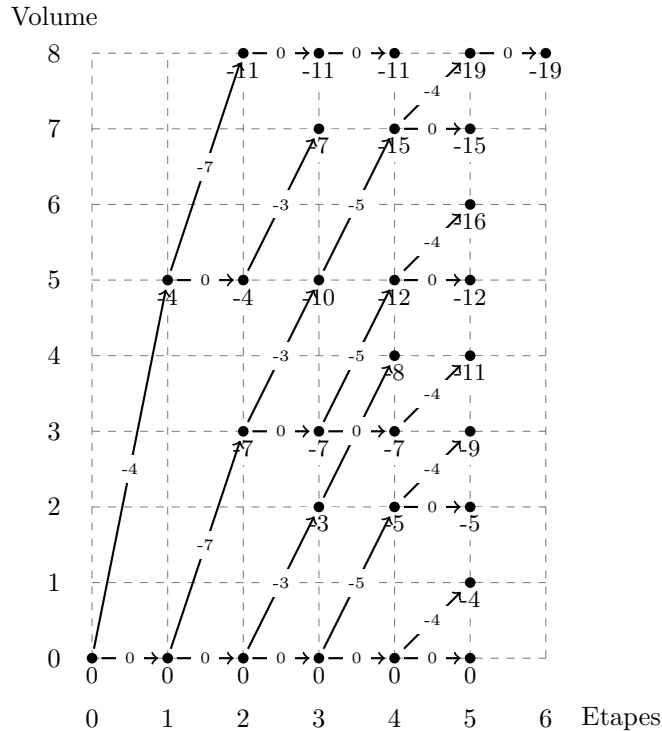


**Solution 2.** 1. Un chemin entre  $(0, 0)$  et  $(k + 1, C)$  représente un choix d'objets sélectionnés de volume inférieur à la capacité  $C$ , la longueur d'un chemin étant l'opposé du volume des objets emportés. Un chemin minimal est correspond à un choix d'objet maximal.

2. Nous ne dessinons que les sommets accessibles depuis  $(0, 0)$



On obtient l'arborescence de plus courts chemins suivante :



qui suggère de prendre les 4 derniers objets.

**Solution 3.137.** On remplace les probabilités  $p$  par les poids  $-\ln p$ , de sorte que désormais, un chemin de poids minimal est un chemin qui maximise le produit des probabilités rencontrées. L'algorithme de Dijkstra renvoie le chemin

$$0 - 24 - 25 - 26 - 38 - 39 - 40 - 42 - 43$$

et il a pour probabilité .0297.

**Solution 3.138.** 1. On cherche un cycle absorbant dans le graphe orienté à  $n$  sommets dont l'arc de  $i$  à  $j$  est pondéré par  $\ln(n_j/n_i)$ .

2. On cherche un plus court chemin dans le graphe non-orienté à  $n$  sommets dont les arêtes sont de poids 1 et représentent les échanges possibles.

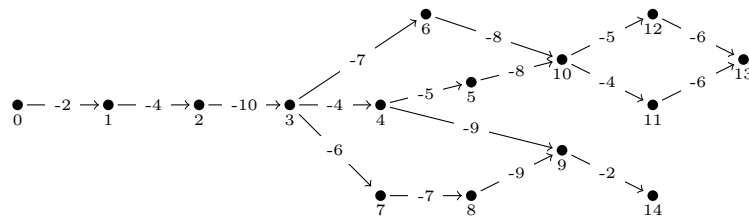
**Solution 3.139.** Nous construisons un graphe à 15 sommets dont les sommets sont soit les 14 tâches à effectuer, soit le sommet "Début du chantier des travaux" (de numéro 0) et dont les arcs représentent les relations d'antériorité. En particulier, toute tâche doit être effectuée après "début du chantier".

Si nous donnons à chaque arc comme poids le temps de complétion de son extrémité finale, nous constatons que le temps minimum correspond au chemin le plus long!

Justification. Fait 1 : Tout chemin dans le graphe représente une suite de tâches qui doivent se suivre. Il est donc impossible d'utiliser moins de temps. Fait 2 : Si on programme toute tâche aussi tôt que possible, c'est-à-dire dès que toutes ses tâches

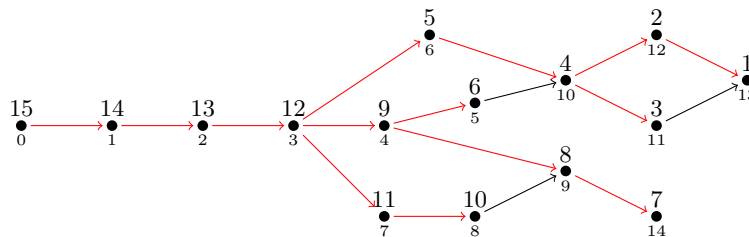
requisies sont achevées, on peut construire une suite de tâches consécutives (en partant de la dernière à s'achever et en remontant de proche en proche à celle dont on attendait la fin) dont les temps d'exécution additionnés correspondent au temps total de cette programmation. Par conséquent, le temps minimum correspond bien au chemin le plus long.

À fin de nous ramener à un problème de plus court chemin, on pouvons utiliser l'opposé des temps de complétion comme poids du graphe. On obtient donc le graphe (dans lequel nous omettons les arcs de 0 vers  $i \geq 2$ ) :

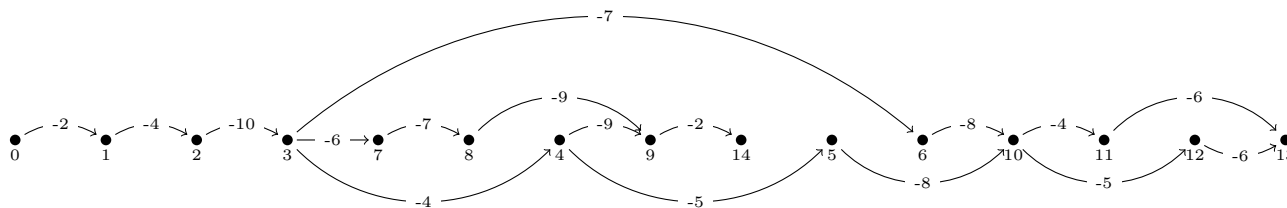


Comme il s'agit d'un graphe acyclique et que nous pouvons toujours supposer étendre un chemin en le commençant en 0, nous pouvons utiliser l'algorithme de Bellman pour calculer les chemins les plus courts au départ de 0 uniquement.

Calcul d'un tri topologique. Nous effectuons un parcours en profondeur et employons la numérotation postfixe.



Ceci nous permet de réordonner les sommets comme suit :



Le calcul des distances minimales se fait de gauche à droite :

| $v$       | 0 | 1  | 2  | 3   | 7   | 8   | 4  | 9  | 14 | 5  | 6  | 10 | 11 | 12 | 13 |
|-----------|---|----|----|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| $\ell(v)$ | 0 | -2 | -4 | -16 | -22 | -29 | 20 | 38 | 40 | 25 | 23 | 33 | 37 | 38 | 44 |

La durée minimale est de 44 unités de temps.



**Solution 3.140.** On calcule les distances  $(\ell^{(k)}(v))_{1 \leq v \leq 8}$  qui représentent la plus courte distance de 1 à  $v$  en utilisant un chemin de moins de  $k$  arcs. Ces valeurs se calculent par récurrence. L'initialisation est faite avec

$$\ell^{(0)} = [0, +\infty, +\infty, +\infty, +\infty, +\infty, +\infty, +\infty]$$

Puis, nous pouvons calculer  $\ell^{(k)}$  par la formule

$$\ell^{(k+1)}(v) = \min\{\ell^{(k)}(v)\} \cup \{\ell^{(k)}(u) + c(u, v); u \in V \text{ t.q. } (u, v) \in E\}$$

$$\ell^{(1)} = [0, 4, +\infty, +\infty, +\infty, +\infty, 5, +\infty]$$

$$\ell^{(2)} = [0, 4, 7, 5, 6, +\infty, 5, 6]$$

$$\ell^{(3)} = [0, 4, 7, 5, 6, 7, 5, 6]$$

$$\ell^{(4)} = [0, 4, 7, 4, 6, 7, 5, 6]$$

$$\ell^{(5)} = [0, 4, 7, 4, 5, 7, 5, 6]$$

$$\ell^{(6)} = [0, 4, 7, 4, 5, 6, 5, 6]$$

$$\ell^{(7)} = [0, 4, 7, 3, 5, 6, 5, 6]$$

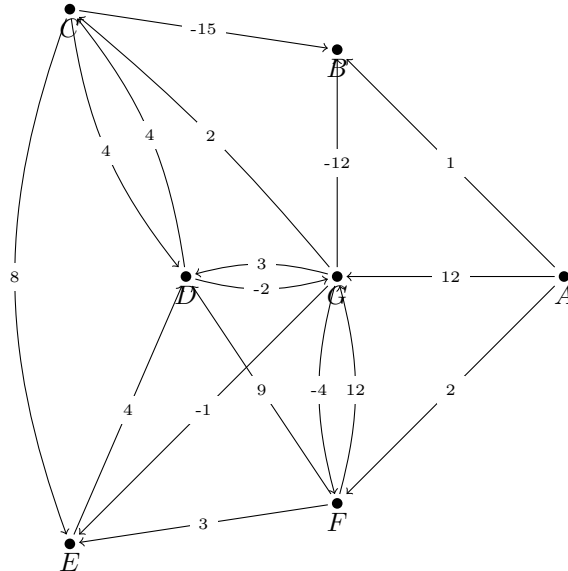
$$\ell^{(8)} = [0, 4, 7, 3, 4, 6, 5, 6]$$

$$\ell^{(9)} = [0, 4, 7, 3, 4, 5, 5, 6]$$

$$\ell^{(10)} = [0, 4, 7, 2, 4, 5, 5, 6]$$

La suite  $\ell^{(k)}$  ne se stabilise pas pour  $k \geq 8$ , c'est le signe que le graphe contient un cycle absorbant. On ne peut pas en déduire de distances minimales (cf. ce qui se passe au sommet 8) !

**Solution 3.141.** Voici le graphe.



Il possède des cycles (donc nous ne pouvons pas utiliser l'algorithme de Bellman) et des valuations négatives (donc nous ne pouvons pas utiliser l'algorithme de Dijkstra). On utilise par conséquent l'algorithme de Ford.

Nous calculons les distances  $(\ell^{(k)}(S))_{1 \leq v \leq 8}$  qui représentent la plus courte distance du sommet  $A$  au sommet  $S$  en utilisant un chemin de moins de  $k$  arcs. Ces valeurs se calculent par récurrence. L'initialisation est faite avec

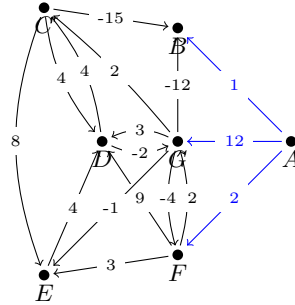
$$\ell^{(0)} = [0, +\infty, +\infty, +\infty, +\infty, +\infty, +\infty, +\infty]$$

Puis, nous pouvons calculer  $\ell^{(k)}$  par la formule

$$\ell^{(k+1)}(v) = \min\{\ell^{(k)}(v)\} \cup \{\ell^{(k)}(u) + c(u, v); u \in V \text{ t.q. } (u, v) \in E\}$$

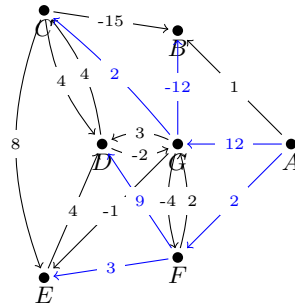
où  $E$  est l'ensemble des arcs. Nous notons aussi en bleu les pères provisoires :

$$\ell^{(1)} = [0, 1, +\infty, +\infty, +\infty, 2, 12]$$



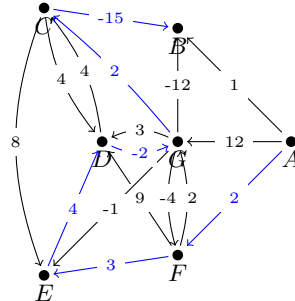
Puis au deuxième tour,

$$\ell^{(2)} = [0, 0, 14, 11, 5, 2, 12]$$



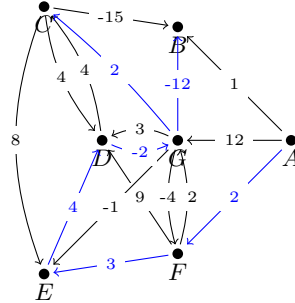
Puis au troisième tour,

$$\ell^{(3)} = [0, -1, 14, 9, 5, 2, 9]$$



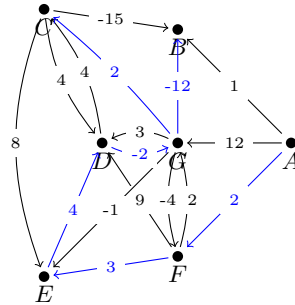
Puis au quatrième tour,

$$\ell^{(4)} = [0, -3, 11, 9, 5, 2, 7]$$



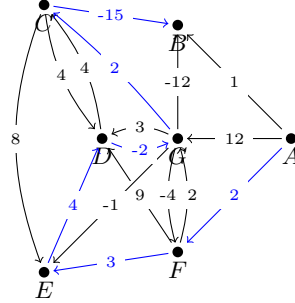
Puis au cinquième tour,

$$\ell^{(5)} = [0, -5, 9, 9, 5, 2, 7]$$



Puis au sixième tour,

$$\ell^{(6)} = [0, -6, 9, 9, 5, 2, 7]$$

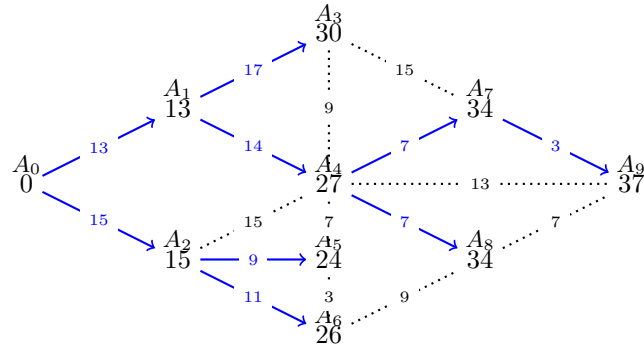


Au septième tour, plus aucune des valeurs de  $\ell$  ne change. Comme il nous a fallu moins de 6 tours pour stabiliser ces valeurs, nous avons obtenu les plus courts chemins.

$$\ell^{(7)} = [0, -6, 9, 9, 5, 2, 7]$$

NB : il est intéressant d'observer qu'en cours d'exécution de l'algorithme, l'arborescence temporairement stockée ne correspond pas aux chemins en moins de  $k$  arcs donnant la valeur  $\ell^{(k)}(v)$ .

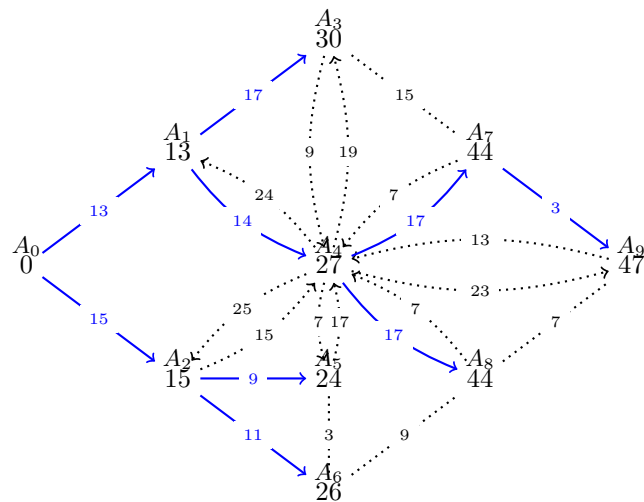
**Solution 3.142.** 1. L'arborescence issue de l'application de l'algorithme de Dijkstra est la suivante.



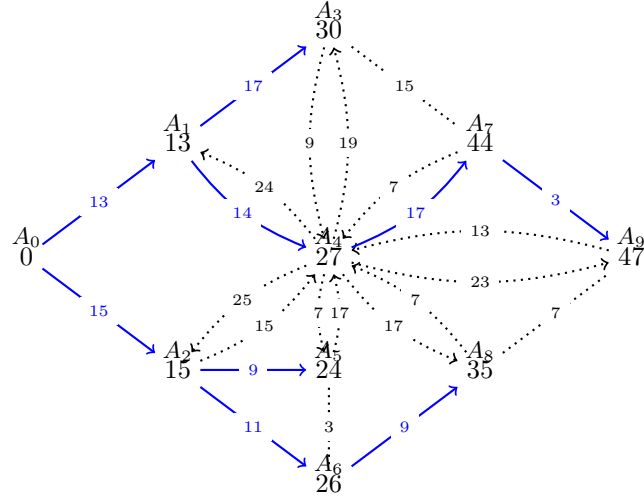
2. NB :  $60\text{km/h} = 1\text{km} / \text{min}$ . Il est donc inutile de convertir des unités. Pour tenir compte de l'accident, nous décidons de passer à un graphe dans lequel les arêtes sont orientées. Pour les arrêtes qui sortent de  $A_4$ , nous augmentons leur poids de 10.

Nous disposons d'ores et déjà d'une arborescence. C'est le cas idéal pour appliquer l'algorithme de Ford-Dantzig.

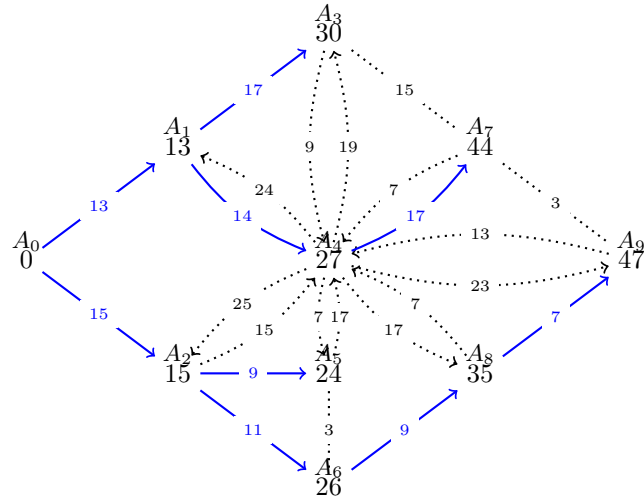
On ajoute 10 dans l'arborescence à l'arc qui arrive dans la ville  $A_4$  et on continue l'algorithme. Il s'avère que l'arborescence reste la même.



L'algorithme de Ford-Dantzig consiste à identifier un arc qui améliore l'arborescence. Ici, on peut sélectionner l'arc  $(A_6, A_8)$ , qui permet de passer à la nouvelle arborescence



Ensuite, on peut sélectionner l'arc  $(A_8, A_9)$ , qui permet de passer à la nouvelle arborescence.

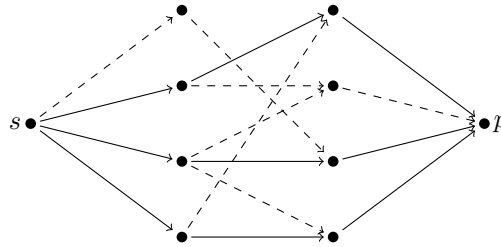


À ce stade, il n'est plus possible d'améliorer l'arborescence. L'algorithme de Ford-Dantzig s'arrête.

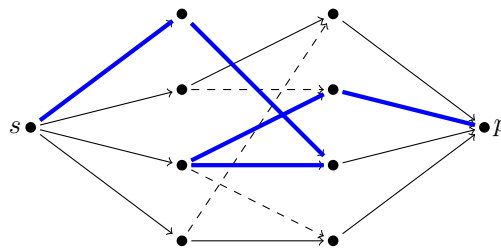
**Solution 3.146.** Un graphe orienté acyclique admet un tri topologique dans lequel un arc est de la forme  $(u, v)$  avec  $1 \leq u < v \leq n$ . En conséquence, il y a au plus  $\frac{n(n-1)}{2}$  arcs. Par ailleurs, cette borne peut être atteinte si l'on prend  $E = \{(u, v) \text{ avec } 1 \leq u < v \leq n\}$ .

**Solution 3.147.** Le couplage est maximal. On ne peut pas lui ajouter d'arête sans briser la propriété d'appariement.

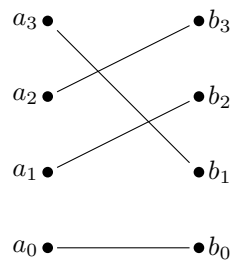
Il n'est pas maximum. On applique la méthode de détermination des couplages par des flots. Cela revient à dire que dans le réseau suivant, dont toutes les arêtes ont capacité 1, nous considérons le flot suivant :



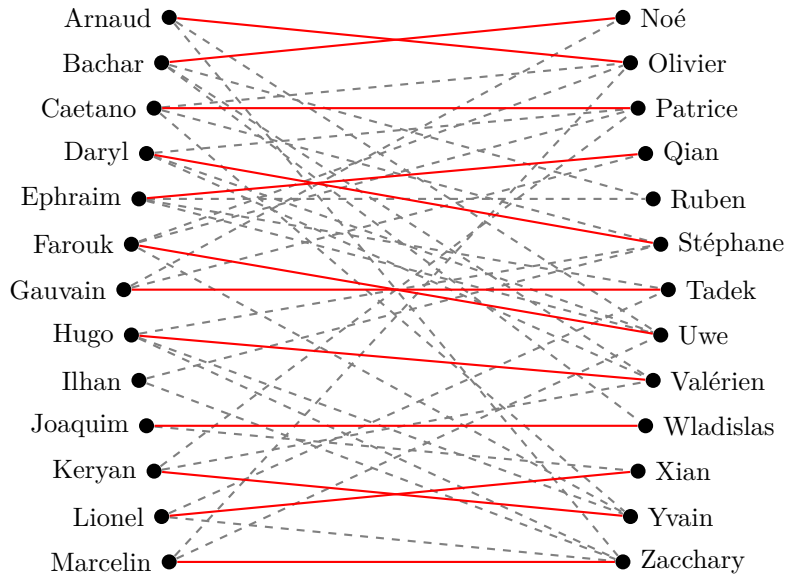
Cependant, il existe une chaîne augmentante, comme présenté ainsi :



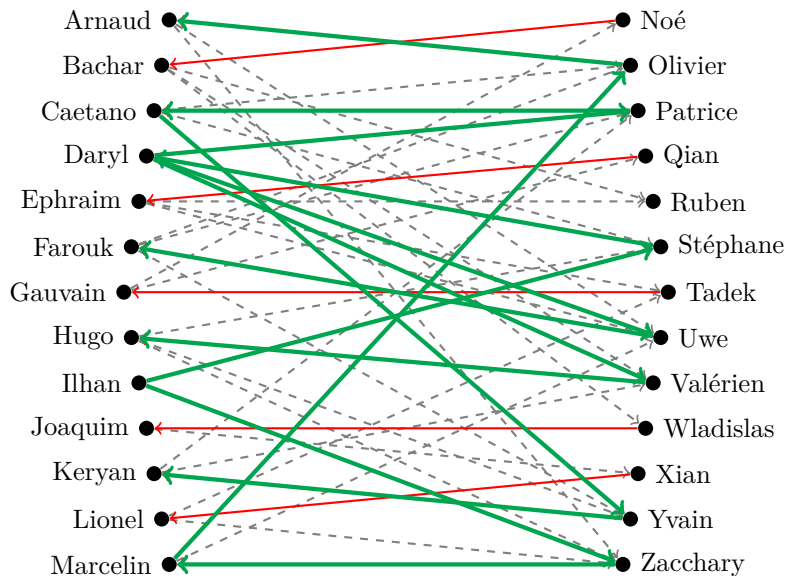
Elle permet d'aboutir au couplage suivant, qui est optimal.



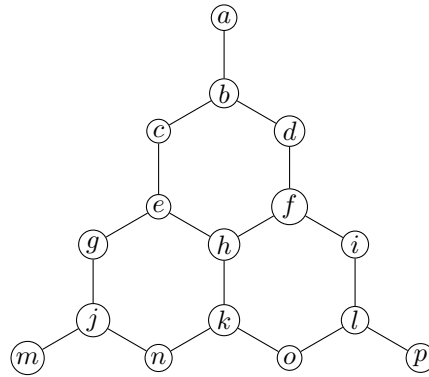
**Solution 3.148.** On peut obtenir un couplage avec 12 arêtes. Ce couplage peut s'obtenir par 12 itérations de l'algorithme de marquage de Ford et Fulkerson ou bien en jouant aux devinettes.



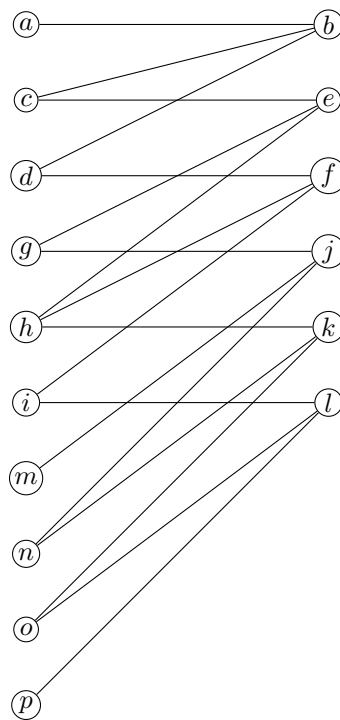
On remarque que dans le graphe les huit sommets A, C, D, F, H, I, K, M ne sont reliés qu'aux 7 sommets O, P, S, U, V, Y, Z. On peut appliquer l'algorithme de marquage de Ford et Fulkerson : on observe qu'il n'existe pas de chemin depuis la source vers le puit



**Solution 3.149.** Nous commençons par nommer les sommets du graphe de la manière suivante :



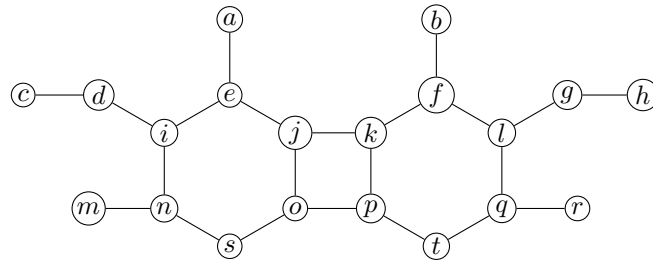
Puis nous réorganisons les sommets comme suit



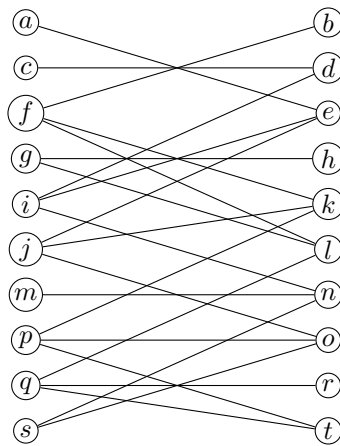
On peut ensuite facilement voir qu'un couplage maximum est donné par les couples face à face verticalement :  $(a, b)$ ,  $(c, e)$ ,  $(d, f)$ ,  $(g, j)$ ,  $(h, k)$ ,  $(i, l)$ .

**Solution 3.150.** On commence par nommer les sommets de la manière suivante :

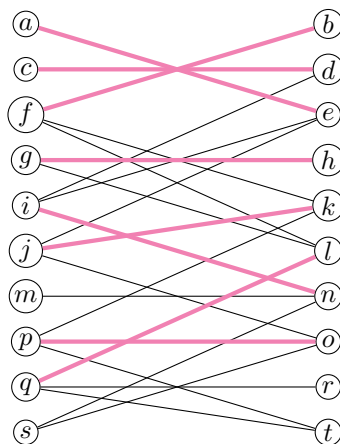




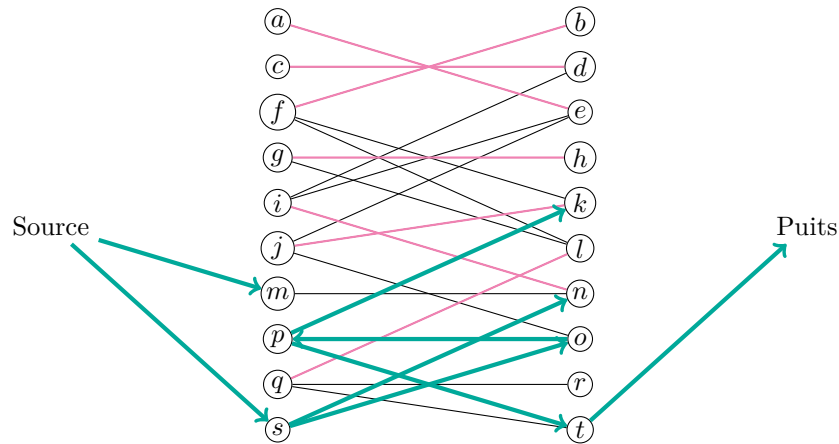
On réécrit le graphe pour faire apparaître la bipartition



On cherche un début de solution par une attribution gloutonne d'arêtes, ce qui fournit déjà un couplage avec 8 arêtes.

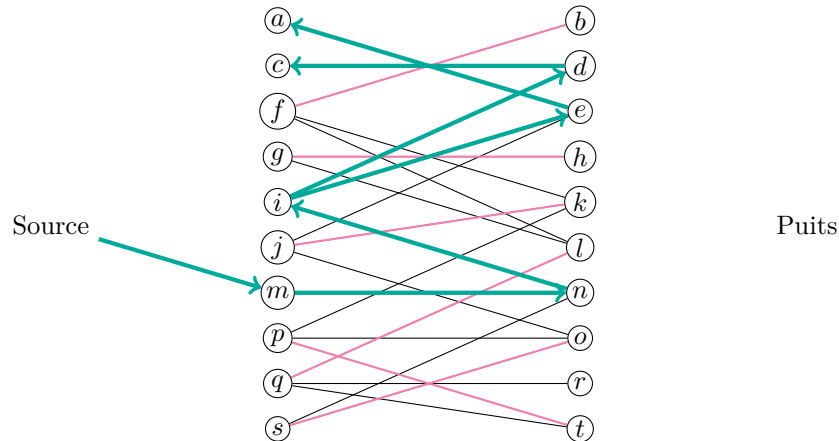


Pour continuer, on applique l'algorithme de marquage de Ford et Fulkerson.

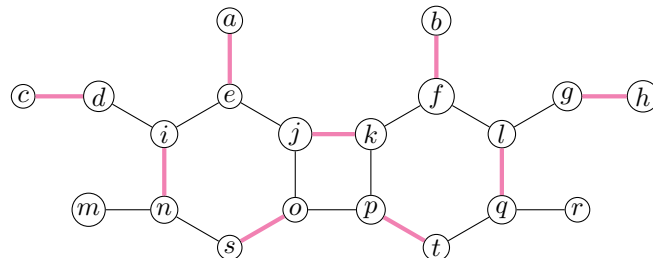


ce qui suggère d'inverser le rôle des arêtes  $(s, o)$ ,  $(o, p)$  et  $(p, t)$  pour obtenir un nouveau couplage qui implique 9 arêtes.

On applique l'algorithme de marquage de Ford et Fulkerson une seconde fois.



Il n'y a plus de chemin augmentant entre la source et le puits. On a donc obtenu un couplage maximum.

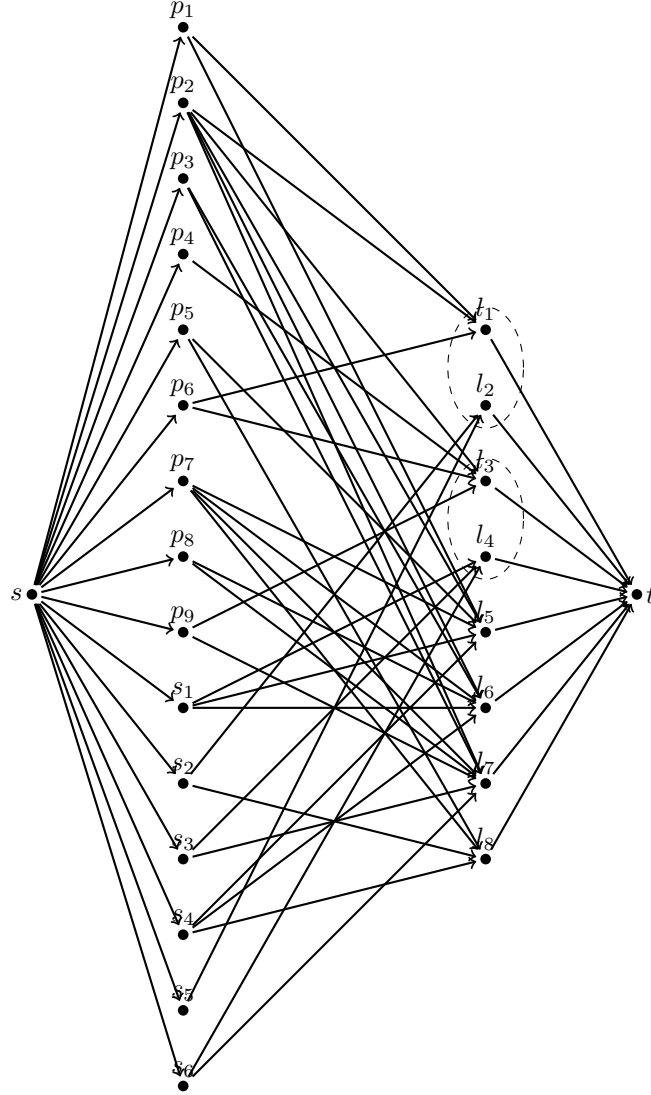


**Solution 3.151.** 1. (a) La condition est évidente : il faut au moins  $r$  filles distinctes pour marier  $r$  garçons.

- (b) Si  $S = \{s\}$ , il est clair que  $c(S, \overline{S}) \leq n$ . Comme les capacités entre les sommets de  $V_1$  et  $V_2$  sont infinies et qu'une coupe minimale sature les arêtes, si  $v \in V_1$  et  $v \in S$ , alors les voisins  $N(v) \subseteq V_2$  sont aussi dans  $S$ . Donc  $N(X) \subseteq S \cap V_2$ . Pour calculer la capacité de la coupe, il y a au moins les arcs de  $s$  vers les sommets de  $V_1 \setminus X$  et les arcs entre les voisins de  $N(X)$  et  $t$ , ce qui au total donne au moins  $n - |X| + |N(X)| = n$  arcs de capacité 1. Par conséquent  $c(S, \overline{S}) = n$ , donc un couplage parfait est possible.
2. On considère le graphe biparti  $G = (V = V_1 \sqcup V_2, E)$  où  $V_1$  est l'ensemble des tas,  $V_2$  l'ensemble des valeurs des cartes et une arête relie  $u$  à  $v$  si la carte de type  $v$  est contenue dans le tas  $u$ . Si l'équation 3 était contredit, alors par le principe des tiroirs, il faudrait qu'une valeur de carte existe 5 fois au moins, mais il n'y a que quatre couleurs. Donc par le lemme des mariages, il est possible de trouver la suite complète comme voulu.

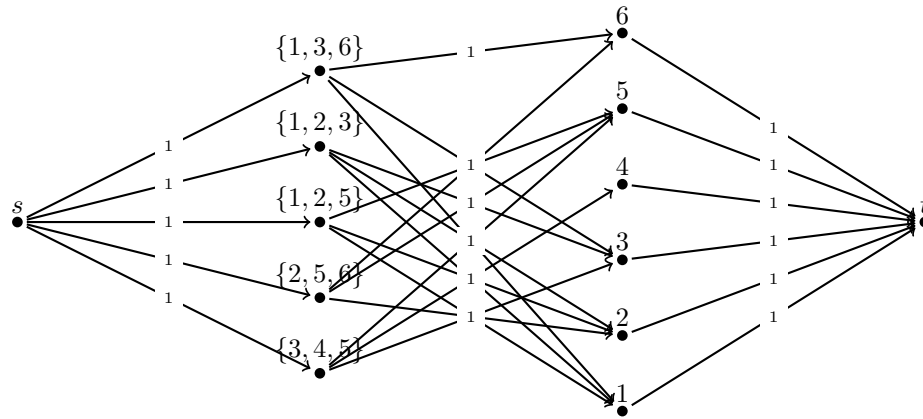
**Solution 3.152.** On construit le réseau  $(G, c, s, t)$  où  $G$  contient un sommet pour chaque individu, un sommet pour chaque lit et des sommets  $s$  et  $t$ . La chambre  $C_1$  contient le lit  $(L_1, L_2)$ ; Les arcs de  $G$  sont les arcs qui relient la source  $s$  aux individus, les lits au puits  $p$  et les élèves aux lits des chambres souhaitées. Pour les chambres doubles, on distingue un lit sortilège et un lit potion. Les arcs sont tous de capacité 1.

Un flot à valeurs entières  $f$  est une affectation possible, sa valeur  $|f|$  est alors le nombre d'élèves et de lits alloués.

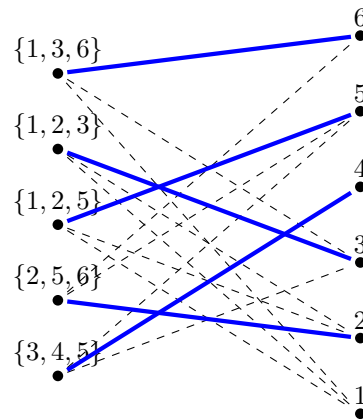


**Solution 3.153.** Pour répondre au problème, on considère le réseau dont les sommets distincts sont une source  $s$ , un puit  $t$ , les indices  $I$  de la famille  $\mathcal{A}$  et les éléments des  $A_i$  :  $A = \bigcup_{i \in I} A_i$ . On place un arc de capacité 1 de la source à  $I$ , de  $A$  à  $t$  et entre  $i \in I$  et  $a \in A$  si  $a \in A_i$ . Un SDR existe ssi la valeur de flot maximum est  $n$ .

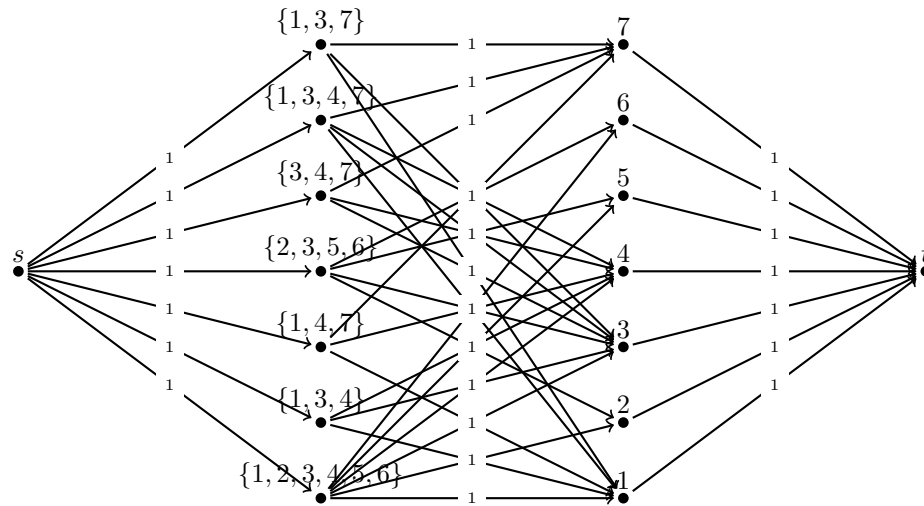
1. Dans le premier cas, on obtient le réseau



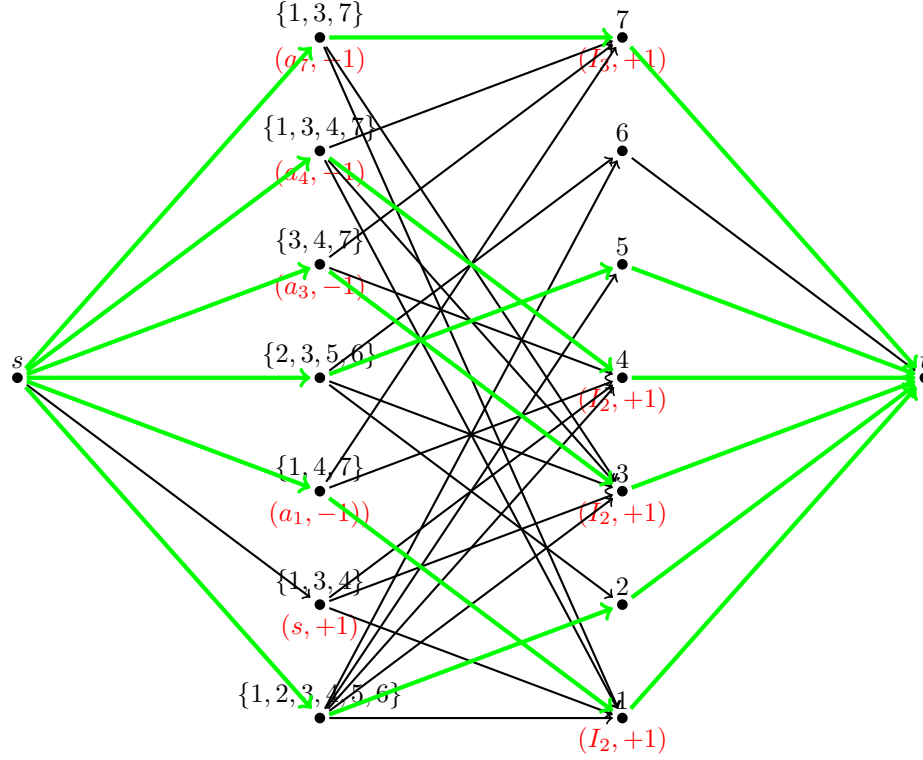
Nous appliquons l'algorithme de Ford et Fulkerson et obtenons un flot maximum à valeur entière. Nous en déduisons le SDR qui suit.



2. Nous obtenons ici



Avec l'algorithme de Ford et Fulkerson nous parvenons à construire un flot maximum de valeur 6. Nous représentons encore le dernier marquage, qui montre que l'on ne peut pas construire un flot de valeur plus grande.



Par conséquent, il n'existe pas de SDR.

**Solution 3.154.** Soit  $G$  un graphe biparti  $k$  régulier sur  $2n$  sommets. On construit le réseau vu en cours pour déterminer un couplage maximal, dont la source est notée  $s$  et le puits  $t$ . On définit le flux  $f$  par  $f(s, u) = f(v, t) = 1$  pour tous  $u \in V_1$ ,  $v \in V_2$  et  $f(u, v) = 1/k$  si  $(u, v)$  est une arête du graphe original. Notons que la loi des nœuds est bien respectée car le graphe est  $k$ -régulier. Ce flux est de valeur  $|f| = n$ , donc maximal. Or le flux maximal peut être atteint par des flots à valeurs entières. Donc il existe un couplage de taille  $n$ , i.e. un couplage parfait.

**Solution 3.155.** Montrons que  $A$  possède une stratégie gagnante ssi  $G$  n'admet pas de couplage parfait et  $B$  possède une stratégie gagnante ssi  $G$  admet un couplage parfait.

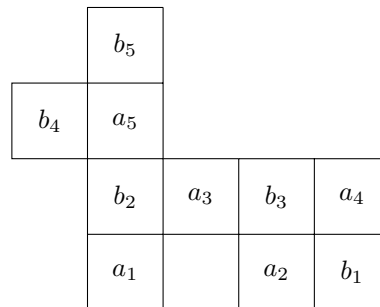
Dans le premier cas, s'il existe un couplage parfait,  $B$  le détermine. Cela lui fournit une bijection  $\pi : V_1 \rightarrow V_2$  telle que  $\pi(v) \in N(v)$  pour tout  $v \in V_1$  où  $N(v)$  est l'ensemble des voisins du sommet  $v$ . Il lui suffit de toujours jouer  $y_k = \pi(x_k) \in V_2$ . Le jeu s'arrête après que tous les  $2n$  sommets sont joués :  $A$  est perdant.

Dans le second cas, s'il n'existe pas de couplage parfait, on peut avant tout et sans perte de généralité se placer dans le graphe biparti restant (i.e. dans le graphe débarrassé des sommets déjà joués) et décrire le premier coup de  $A$ . Le joueur  $A$  peut soit trouver un sommet  $v$  qui n'a pas de voisin dans le graphe restant, mais alors  $B$

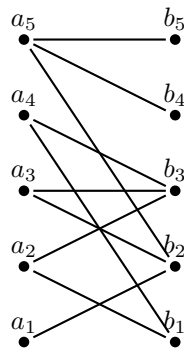
perd immédiatement. Ou sinon, le joueur  $A$  peut sélectionner un sommet  $x_1$  tel que tel que pour  $y \in N(x_1)$ ,  $\deg(y) \neq 1$ . Dans ce cas,  $B$  est assuré de continuer à jouer au tour suivant. De plus un tel sommet  $x_1$  existe car sinon pour tout  $x \in V_1$ , il existe  $y_x \in V_2$  tel que  $\deg y_x = 1$ . Mais alors, on a défini une injection  $\{y_x, x \in V_1\} \rightarrow V_1$ . Comme  $V_1$  et  $V_2$  sont finis de même cardinal, c'est une bijection. Mais alors  $(x, y_x)_{x \in V_1}$  est un couplage parfait ce qui est exclu. D'autre part,  $A$  gagne nécessairement, sinon, la partie se déroule jusqu'à ce que les  $2n$  sommets sont joués et l'on a un couplage parfait de  $G$ .

Comme déterminer l'existence d'un couplage parfait peut se faire avec Ford et Fulkerson en temps polynomial, l'exercice est résolu.

- Solution 3.156.**
1. Les parités sont opposées : l'une des sommes est paire, l'autre est impaire.
  2. Soit  $G$  le graphe dont les sommets sont les centres des carrés et les arêtes sont les sommets de carrés adjacents. La question précédente montre que ce graphe est biparti. Trouver un pavage revient à trouver un couplage parfait.
  3. On utilise la méthode vue en cours.
  4. Fixons les notations



Nous obtenons le graphe biparti suivant :



qui n'admet pas de couplage parfait.

**Solution 3.157.** Soient  $C_1$  et  $C_2$  deux couplages. Quitte à supprimer des arêtes, on peut supposer que les extrémités de  $C_1$  dans  $R_1$  sont exactement  $R_1$  et que les extrémités de  $C_2$  dans  $R_2$  sont exactement  $R_2$ .





|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> |          |          |          |          |          |          |          |          |          |          |          |          |

On décale : dans le tableau ci-dessous les lettres soulignées ne sont pas re-comparées.

|          |          |          |          |                 |                 |                 |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|-----------------|-----------------|-----------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i>        | <i>a</i>        | <i>a</i>        | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          | <u><i>b</i></u> | <u><i>a</i></u> | <u><i>a</i></u> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> |          |          |          |          |          |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> |          |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |                 |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|-----------------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i>        | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <u><i>b</i></u> | <i>a</i> |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |

Il y a 27 comparaisons au total.

3. On a

|          |               |          |           |            |             |              |               |                |
|----------|---------------|----------|-----------|------------|-------------|--------------|---------------|----------------|
| <i>w</i> | $\varepsilon$ | <i>b</i> | <i>ba</i> | <i>baa</i> | <i>baab</i> | <i>baaba</i> | <i>baabaa</i> | <i>baabaaa</i> |
| $r_p$    | -1            | 0        | 0         | -1         | 0           | 0            | 3             | -1             |

4. Voici les positionnement successifs

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
| <i>b</i> |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> |          |          |          |          |          |          |          |          |          |          |          |          |

|          |          |          |          |                 |                 |                 |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|-----------------|-----------------|-----------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i>        | <i>a</i>        | <i>a</i>        | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          | <u><i>b</i></u> | <u><i>a</i></u> | <u><i>a</i></u> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> |          |          |          |          |          |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> |          |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |          |

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>a</i> | <i>a</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> | <i>b</i> |
|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          | <i>b</i> | <i>a</i> |

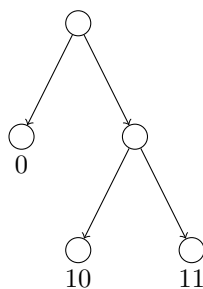
Il y a 26 comparaisons au total, soit une comparaison de moins.

**Solution 4.158.** 1. On sait que les deux lettres suivantes sont distinctes.

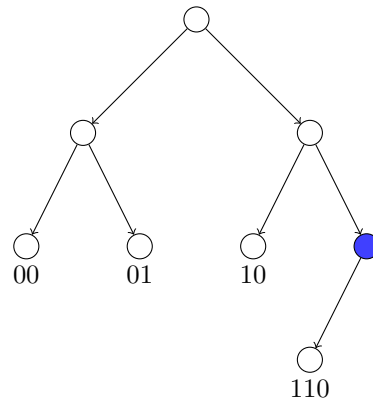
2. Mais comme l'alphabet est binaire, l'un des deux lettres doit coïncider avec la lettre de la chaîne  $s$  dans laquelle on recherche le motif  $p$ . On peut donc gagner une comparaison dans ce cas.

**Solution 4.159.** Les trois codes sont préfixes et se prêtent au dessin des arbres suivants

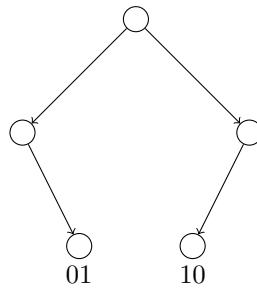
- Le code  $(0, 10, 11)$  est un code pour l'alphabet  $\{a, b, c\}$  et les fréquences  $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$  comme le montre l'arbre :



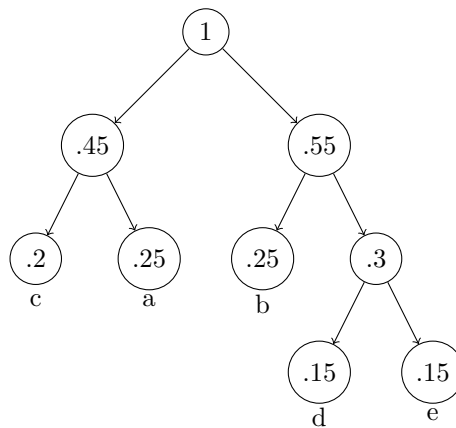
- Le second code ne peut être un code de Huffman car, l'arbre n'est pas complet



— Le dernier code ne peut être un code de Huffman car, l'arbre n'est pas complet



**Solution 4.160.** 1. On obtient l'arbre

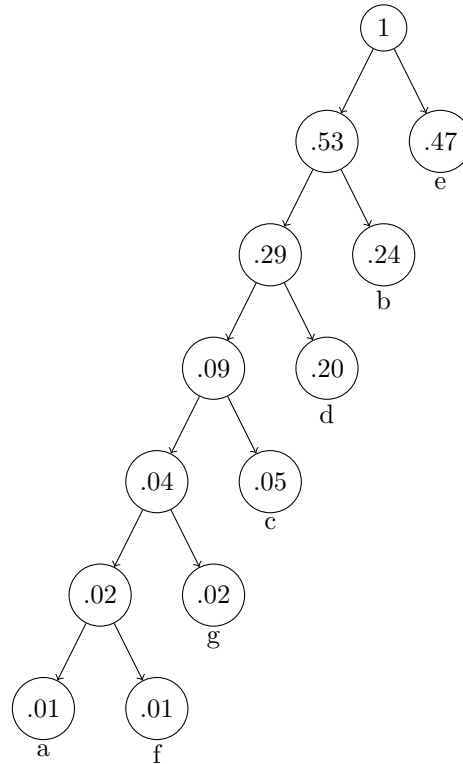


et le codage

| <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> |
|----------|----------|----------|----------|----------|
| 01       | .10      | 00       | 110      | 111      |

On passe de 7 bits par caractère à  $.25 \cdot 2 + .25 \cdot 2 + .2 \cdot 2 + .15 \cdot 3 + .15 \cdot 3 = 1.49$  bits par caractère en moyenne, soit un taux de compression de 21 %.

2. On obtient

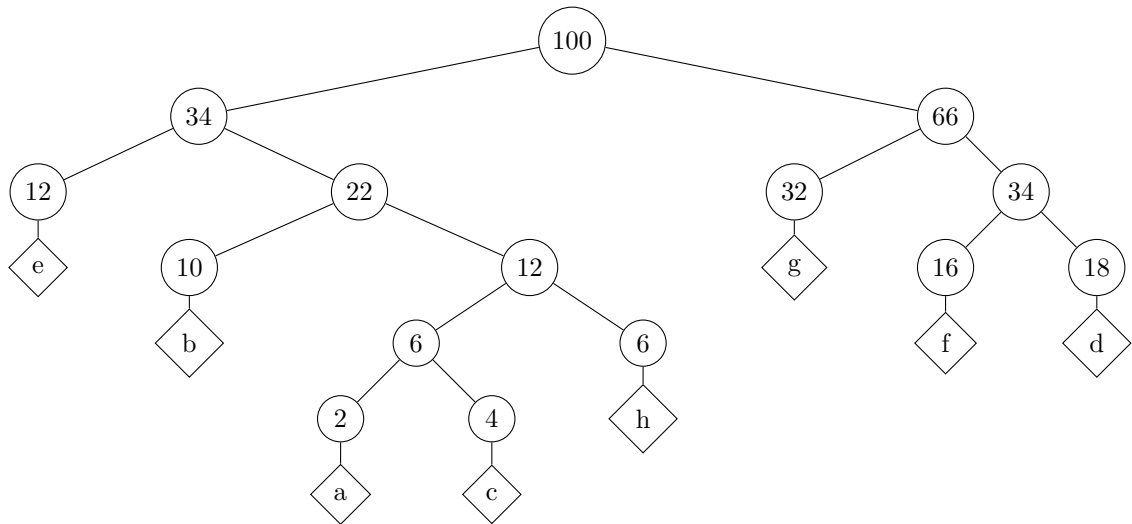


Le code est

| <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> | <i>g</i> |
|----------|----------|----------|----------|----------|----------|----------|
| 000000   | 01       | 0001     | 001      | 1        | 000001   | 00001    |

On passe de 7 bits par caractère à  $.01 \cdot 6 + .24 \cdot 2 + .05 \cdot 4 + .20 \cdot 3 + .47 \cdot 1 + .01 \cdot 6 + .02 \cdot 5 = 1.97$  bits par caractère en moyenne, soit un taux de compression de 28 %.

**Solution 4.161.** Voici une solution possible



On peut en déduire le code suivant :

| Symbole | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> | <i>g</i> | <i>h</i> |
|---------|----------|----------|----------|----------|----------|----------|----------|----------|
| Code    | 01100    | 010      | 01101    | 111      | 000      | 110      | 10       | 0111     |

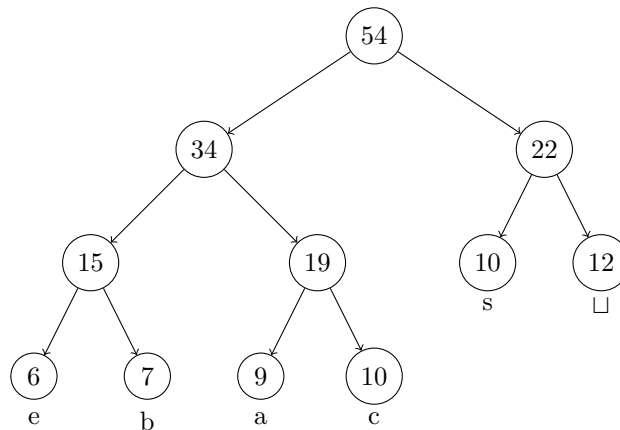
Avec un code à longueur fixe, il faut toujours 3 bits par symbole, donc aussi 3 bits par symbole en moyenne sur l'ensemble du texte. Avec le codage de Huffman, on obtient un taux de compression de 2,86 bits par symbole :

$$\frac{1}{100} (2 \cdot 5 + 10 \cdot 3 + 4 \cdot 5 + 18 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 32 \cdot 2 + 6 \cdot 4) = \frac{286}{100}.$$

**Solution 4.162.** On commence par évaluer les fréquences de chaque lettre

| □  | <i>a</i> | <i>b</i> | <i>c</i> | <i>e</i> | <i>s</i> |
|----|----------|----------|----------|----------|----------|
| 12 | 9        | 7        | 10       | 6        | 10       |

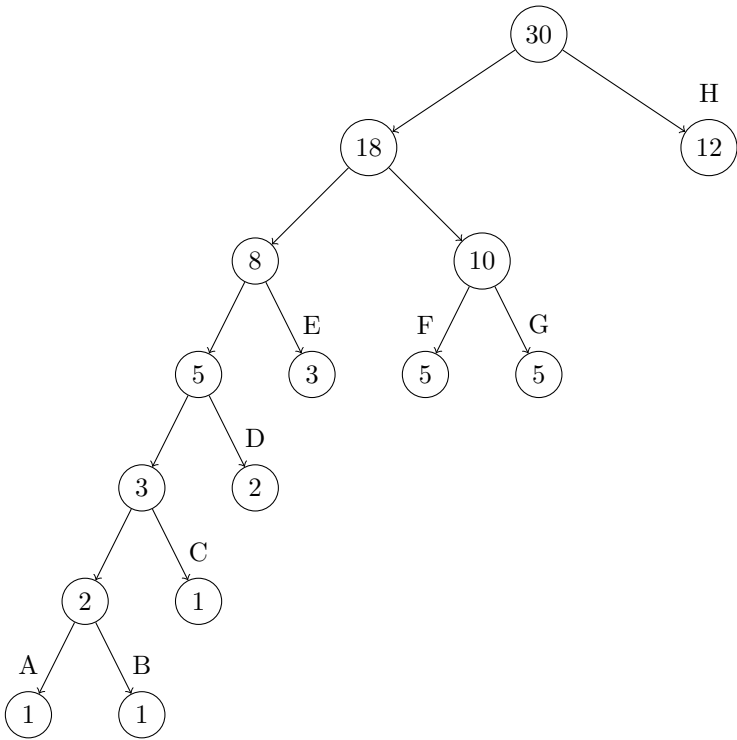
On en déduit l'arbre



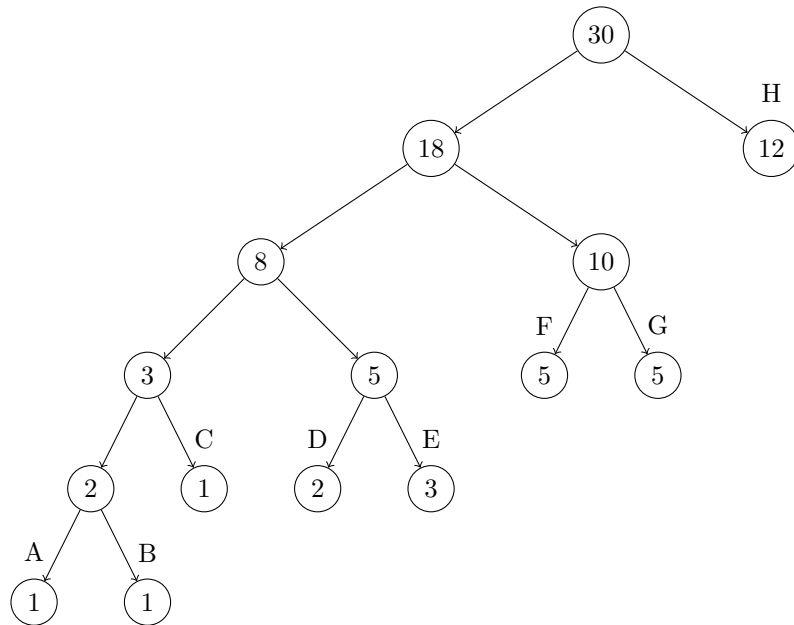
et le code de chaque symbole

| $\square$ | $a$ | $b$ | $c$ | $e$ | $s$ |
|-----------|-----|-----|-----|-----|-----|
| 11        | 010 | 001 | 011 | 000 | 10  |

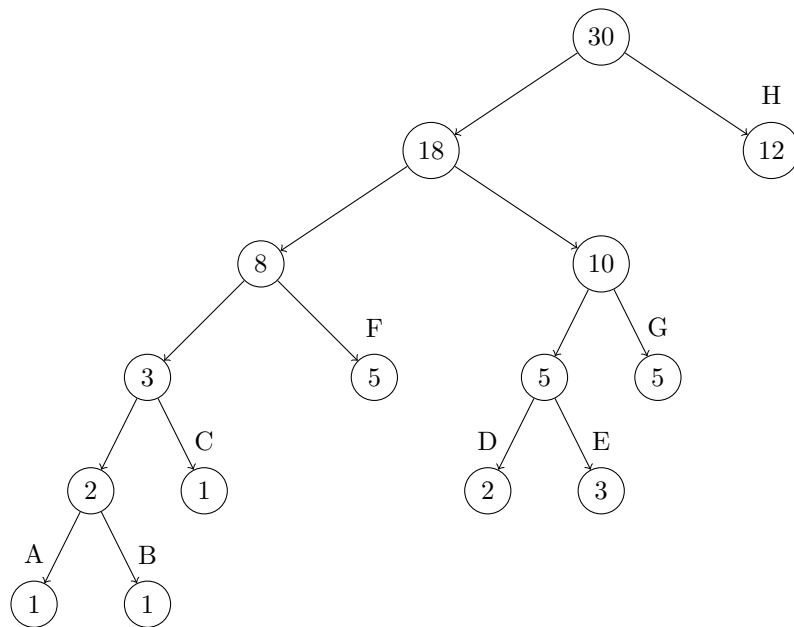
**Solution 4.163.** On peut obtenir :



ou

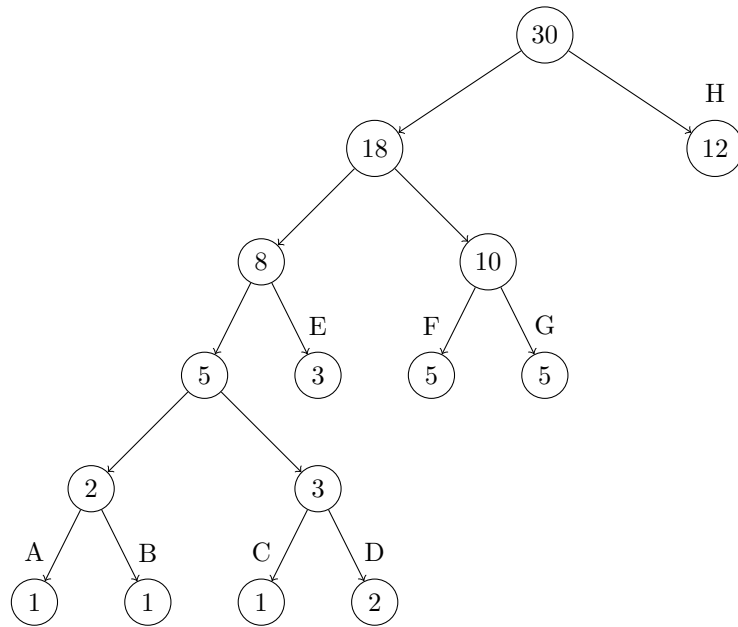


ou

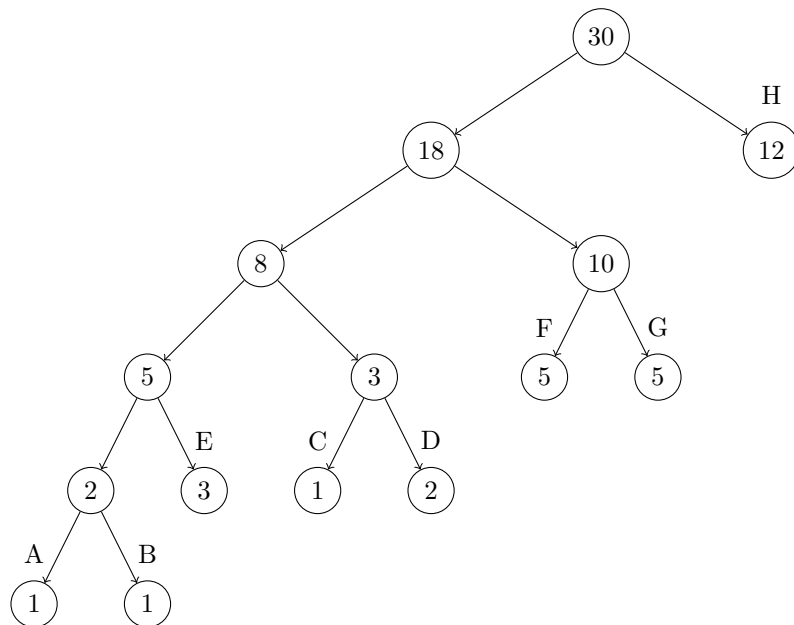


ou





ou

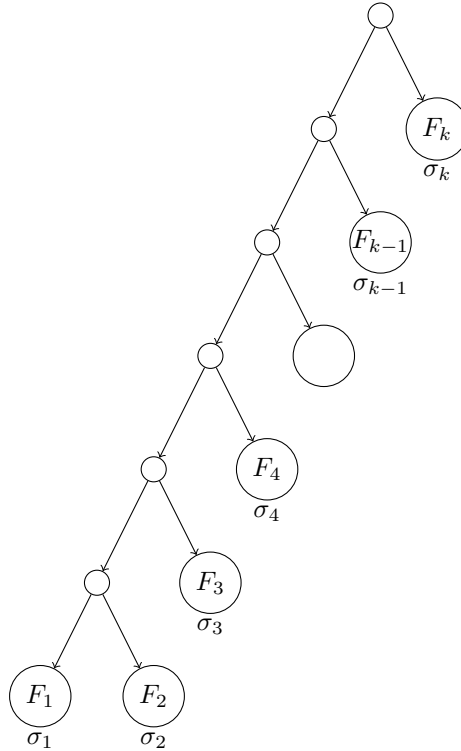


et ainsi de suite. D'autres variations sont encore possibles. Dans tous les cas on trouve 76 comme longueur.

**Solution 4.164.** On démontre par récurrence que pour tout entier  $n$ ,

$$\sum_{i=1}^n F_i = F_{n+1} - 1$$

Ceci a pour conséquence que l'algorithme de Huffman groupe les lettres sous la forme d'un arbre "peigne"



Le codage est donc  $\sigma_1 \mapsto 0^{k+1}$  et  $\sigma_n \mapsto 0^{k-n}1$  pour  $n \geq 2$ .

**Solution 4.165.** Un codage préfixe optimal se présente sous la forme d'un arbre binaire complet possédant exactement  $k$  feuilles et dont chaque feuille porte une lettre distincte de l'alphabet. Il nous suffit donc de transmettre la forme de l'arbre et la valeur portée par chaque feuille.

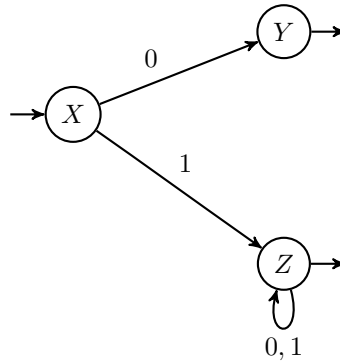
Observons qu'un arbre complet à  $k$  feuilles compte nécessairement  $2k - 1$  nœuds. En effet, un arbre à une feuille possède un seul sommet ; si un arbre compte  $k > 2$  feuilles, son sous-arbre gauche disons  $i$  feuilles et son sous-arbre droit  $k - i$  feuilles, alors par récurrence cet arbre comporte  $1 + (2i - 1) + (2(k - i) - 1) = 2k - i$  sommets.

Pour coder l'arbre, nous parcourons les  $2k - 1$  sommets dans l'ordre préfixe et portons un 0 si nous rencontrons un nœud interne ou un 1 si nous rencontrons une feuille. Cette suite de  $2k - 1$  bits suffit à reconstruire l'arbre. En effet, soit je lis un zéro dans la suite de bits, je place un sommet interne et je sais que le nœud suivant

sera le fils gauche, soit je lis un 1, je place une feuille, je peux remonter au dernier sommet construit de degré 1 et le nœud suivant sera son fils droit.

Finalement pour représenter le codage préfixe, nous représentons la forme de l'arbre par  $2k - 1$  bits. Puis nous indiquons le numéro des feuilles en les citant dans l'ordre préfixe aussi. Chaque numéro nécessite  $\lceil \log_2 k \rceil$  bits, soit  $k \cdot \lceil \log_2 k \rceil$  en tout. On retrouve la complexité souhaitée.

**Solution 4.3.** (1) On peut écrire  $L_n = L_{0|1(0|1)^*}$ , langage dénoté par l'expression régulière  $0|1(0|1)^*$ . Ce langage est reconnu, par exemple, par le DFAI suivant :



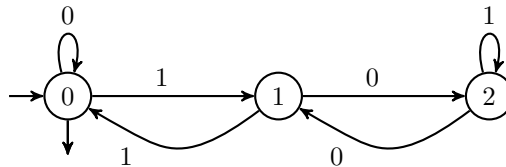
(2) (a) Le langage  $L_a$  est rationnel car il s'agit du langage des mots binaires qui soit sont le mot vide soit finissent par 0 : il est dénoté par l'expression régulière  $\varepsilon|(0|1)^*0$ .

(b) On a  $L_b = L_a \cap L_n$  et on a vu que  $L_a$  et  $L_n$  sont rationnels, donc  $L_b$  l'est aussi (on peut aussi exhiber une expression régulière qui le dénote :  $0|1(0|1)^*0$ ).

(c) Ajouter un 0 ou un 1 à la fin d'un mot binaire de valeur numérique  $n$  le transforme en un mot de valeur numérique  $2n+x$  où  $x$  est le chiffre affixé. Considérons les six combinaisons entre les trois cas possibles de la valeur numérique  $n$  modulo 3 et les deux cas possibles de la valeur de  $x$  :

| $n \equiv ? \pmod{3}$ | $x = ?$ | $2n + x \equiv ? \pmod{3}$ |
|-----------------------|---------|----------------------------|
| 0                     | 0       | 0                          |
| 0                     | 1       | 1                          |
| 1                     | 0       | 2                          |
| 1                     | 1       | 0                          |
| 2                     | 0       | 1                          |
| 2                     | 1       | 2                          |

Ceci définit un DFA dont les trois états correspondent aux trois valeurs possibles de  $n$  modulo 3, la transition  $n \rightarrow n'$  étiquetée par  $x$  correspond au passage de  $n$  à  $2n+x$  modulo 3, c'est-à-dire :



(On a marqué l'état 0 comme initial car le mot vide a une valeur numérique congrue à 0 modulo 3, et seul 0 comme final car on veut reconnaître les multiples de 3.)

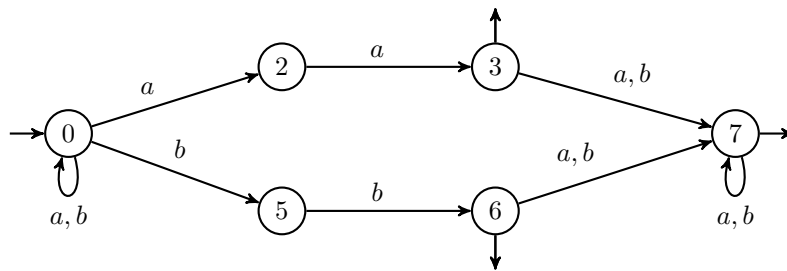
(d) Le langage  $L_d$  n'est pas rationnel (on pourrait le démontrer à l'aide du lemme de pompage, mais ce n'est pas très facile).

(e) Le langage  $L_e$  est rationnel car il s'agit du langage dénoté par l'expression régulière  $0*10*$ .

**Solution 4.167.** Supposons par l'absurde que  $L$  soit rationnel. D'après le lemme de pompage, il existe un certain  $k$  tel que tout mot de  $L$  de longueur  $\geq k$  se factorise sous la forme  $uvw$  avec (i)  $|v| \geq 1$ , (ii)  $|uv| \leq k$  et (iii)  $uv^i w \in L$  pour tout  $i \geq 0$ . Soit  $p$  un nombre premier supérieur ou égal à  $k$  (qui existe car l'ensemble des nombres premiers est infini) : le mot  $a^p \in L$  admet une factorisation comme on vient de dire. Posons  $|u| = m$  et  $|v| = n$ , si bien que  $|w| = p - m - n$ . On a alors  $n \geq 1$  d'après (i), et  $|uv^i w| = m + in + (p - m - n) = p + (i - 1)n$  est premier pour tout  $i \geq 0$  d'après (iii). En particulier pour  $i = p + 1$  on voit que  $p + pn = p(n + 1)$  est premier, ce qui contredit le fait qu'il s'agit d'un multiple non-trivial ( $n + 1 \geq 2$ ) de  $p$ .

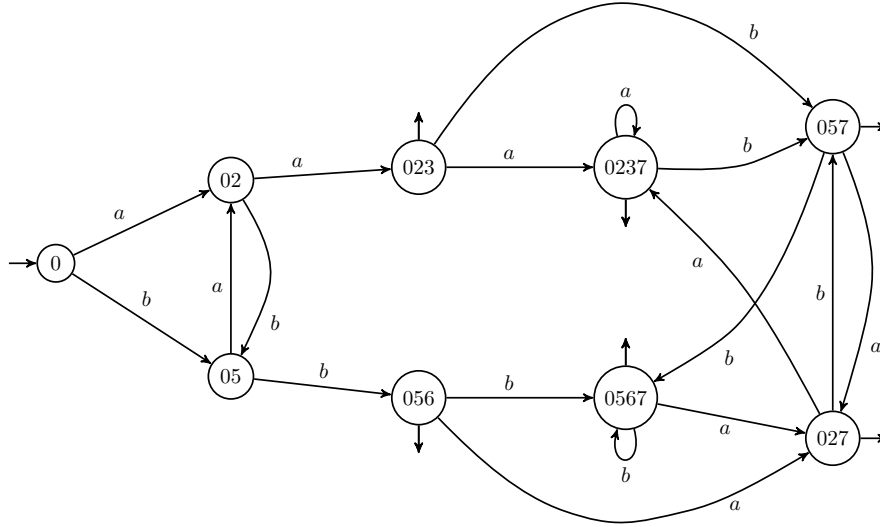
**Solution 4.168.** (0) L'automate proposé accepte les mots ayant soit deux  $a$  consécutifs (en passant par le chemin  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7$ ) soit deux  $b$  consécutifs (en passant par le chemin  $0 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ ).

(1) L'automate ayant des  $\epsilon$ -transitions, il ne peut pas être déterministe : on a affaire à un NFA. Avant de le déterminer, on élimine ses  $\epsilon$ -transitions : la  $\epsilon$ -fermeture de 0 est  $\{0, 1, 4\}$ , celle de 3 est  $\{3, 7\}$  et celle de 6 est  $\{6, 7\}$  ; on est amené au NFA suivant :



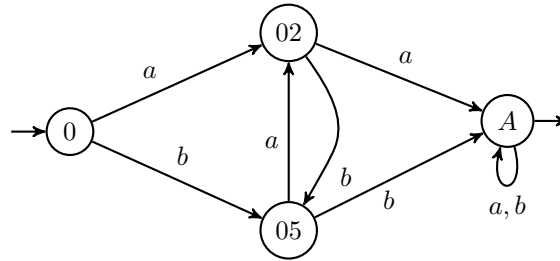
(Les états 1 et 4 étant inaccessibles, ils ont été retirés. Il ne faut pas oublier que 3 et 6 sont finaux puisqu'ils ont l'état final 7 dans leur  $\epsilon$ -fermeture.)

On peut maintenant procéder à la déterminisation. Pour abrégier les noms des états, on note, par exemple, 023 pour  $\{0, 2, 3\}$ . En construisant de proche en proche, on obtient le DFA suivant :



(À titre d'exemple, la transition étiquetée  $b$  partant de l'état 0237 conduit à l'état 057 car les transitions étiquetées  $b$  dans le NFA précédent et partant des états parmi  $\{0, 2, 3, 7\}$  sont  $0 \rightarrow 0$ ,  $0 \rightarrow 5$ ,  $3 \rightarrow 7$  et  $7 \rightarrow 7$ . Tous les états contenant l'un des symboles 3, 6, 7 sont finaux.)

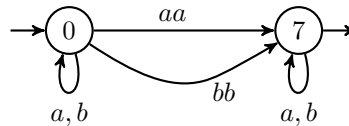
(2) On a affaire à un DFA (sous-entendu : *complet*) dont tous les états sont accessibles, on peut donc appliquer directement l'algorithme de Moore. Une première partition sépare les états finaux, soit 023, 0237, 027, 056, 0567, 057 des non-finaux, soit 0, 02, 05. L'étape suivante distingue 02 parce que sa  $a$ -transition conduit à une classe différente que celles de 0 et 05, et 05 parce que sa  $b$ -transition conduit à une classe différente de 0 et 02. Les étapes suivantes ne changent rien. Finalement, on arrive à un automate à quatre états :



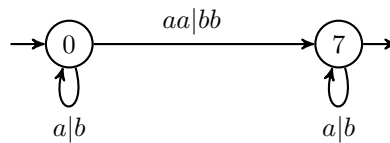
où  $A$  représente la classe de tous les états finaux de l'automate précédent.

La signification des quatre états est la suivante : l'état 0 signifie que l'automate n'a encore rien lu, l'état 02 signifie que l'automate vient de lire un  $a$ , le 05 signifie qu'il vient de lire un  $b$ , le  $A$  signifie qu'il a lu deux  $a$  consécutifs ou bien deux  $b$  consécutifs. Sur cette description, il est clair que l'automate accepte les mots contenant deux  $a$  consécutifs ou bien deux  $b$  consécutifs.

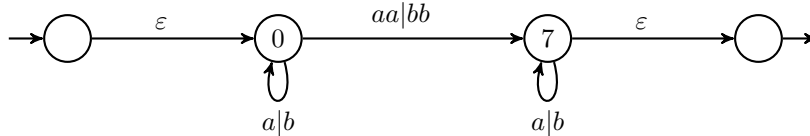
(3) L'élimination des états 1 à 6 peut se faire dans un ordre quelconque et conduit à l'automate (à transitions étiquetées par des expressions régulières) suivant :



Il y a plusieurs flèches entre les mêmes états : quitte à les remplacer par des disjonctions, on obtient :

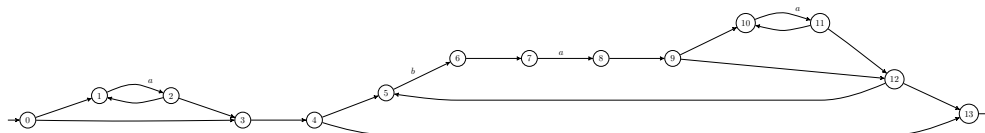


Enfin, on doit éliminer les états 0 et 7 eux-mêmes : pour cela, on ajoute un nouvel état initial qui ne soit la cible d'aucune flèche et un nouvel état final d'où ne part aucune flèche,



Et l'élimination des états 0 et 7 dans un ordre quelconque conduit finalement à l'expression régulière  $(a|b)^*(aa|bb)(a|b)^*$ .

**Solution 4.169.** (1) L'automate de Thompson de  $a^*(baa^*)^*$  doit comporter 14 états puisque cette expression régulière contient 7 symboles parenthèses non comptées. Il est le suivant (où on a omis les  $\varepsilon$  sur les transitions spontanées) :

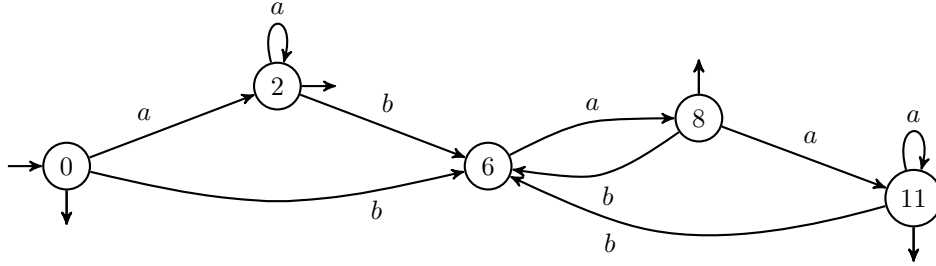


Cet automate n'est pas déterministe : un automate comportant des  $\varepsilon$ -transitions est forcément non-déterministe.

(2) Tous les états autres que 0 (car il est initial) et 2, 6, 8, 11 (car des transitions non spontanées y aboutissent) vont disparaître ; les  $\varepsilon$ -fermetures  $C(q)$  de ces états sont les suivantes :

| $q$ | $\varepsilon$ -fermeture $C(q)$ |
|-----|---------------------------------|
| 0   | $\{0, 1, 3, 4, 5, 13\}$         |
| 2   | $\{1, 2, 3, 4, 5, 13\}$         |
| 6   | $\{6, 7\}$                      |
| 8   | $\{5, 8, 9, 10, 12, 13\}$       |
| 11  | $\{5, 10, 11, 12, 13\}$         |

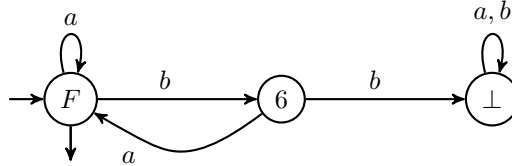
En remplaçant chaque transition  $q^\# \rightarrow q'$  étiquetée d'un  $x \in \Sigma$  dans l'automate par une transition  $q \rightarrow q'$  pour chaque état  $q$  tel que  $q^\# \in C(q)$ , on obtient le NFA suivant :



Les états 0, 2, 8, 11 sont finaux car ce sont eux qui ont 13 dans leur -fermeture.

(3) L'automate ainsi obtenu est déjà déterministe incomplet ; pour le déterminer-compléter, il n'y a qu'à ajouter un puits  $\perp$  avec la seule transition qui manque, c'est-à-dire une transition étiquetée par  $b$  depuis l'état 6 (et des transitions de  $\perp$  vers lui-même étiquetées  $a$  et  $b$ ). Nous ne représentons pas l'automate à 6 états ainsi fabriqué.

(4) On part de l'algorithme déterministe complet obtenu à la question (3), et on lui applique l'algorithme de minimisation. On sépare d'abord ses états en deux classes, les finaux  $\{0, 2, 8, 11\}$  et les non-finaux  $\{6, \perp\}$ . La transition étiquetée  $a$  sépare les états 6 et  $\perp$  car le premier aboutit dans la classe  $\{0, 2, 8, 11\}$  tandis que le second aboutit dans la classe  $\{6, \perp\}$ . On vérifie ensuite qu'aucune transition ne sépare des états. L'automate minimal est donc le suivant :



où l'état  $F$  représente la classe  $0 \equiv 2 \equiv 8 \equiv 11$ .

(5) Il s'agit du langage constitué des mots n'ayant jamais deux  $b$  consécutifs ni de  $b$  final, c'est-à-dire des mots dans lesquels chaque  $b$  est suivi d'au moins un  $a$  : l'expression régulière initiale le présente comme le langage constitué des mots formés d'un nombre quelconque de  $a$  puis d'un nombre quelconque de répétitions d'un  $b$  suivi d'au moins un  $a$ . L'automate final interdit les suites de deux  $b$  consécutifs comme ceci : l'état  $F$  correspond à la situation où on ne vient pas de rencontrer un  $b$  (=la lettre précédente était un  $a$  ou bien on vient de commencer le mot) et on n'en a jamais rencontré deux, l'état 6 à la situation où on vient de rencontrer un  $b$  et on n'en a jamais rencontré deux, et l'état  $\perp$  à la situation où on a rencontré au moins une fois deux  $b$  consécutifs. Avec cette description, il est clair que l'automate fait ce qui était demandé.

**Solution 4.170.** (A) Si on commence par éliminer l'état 2 (en considérant l'automate comme un automate à transitions étiquetées par des expressions régulières), l'état 1 reçoit une transition vers lui-même étiquetée  $ab*a$ . Si on élimine l'état 1, l'état 0 reçoit à la place une transition vers lui-même étiquetée par  $b(ab*a)*b$ , qu'on peut fusionner avec la transition vers lui-même déjà existante étiquetée par  $a$  pour une seule étiquetée par  $a|b(ab*a)*b$ . Quitte éventuellement à ajouter un nouvel état initial (conduisant à 0 par une transition spontanée) et un nouvel état final (vers lequel 0 conduit par une transition spontanée) et à éliminer l'état 0, on obtient finalement l'expression régulière

$$(a|b(ab*a)*b)*$$

(B) Si on commence par éliminer l'état 1, il apparaît une transition  $0 \rightarrow 2$  étiquetée  $ba$  et une  $2 \rightarrow 0$  étiquetée  $ab$  (si on veut appliquer l'algorithme de façon purement mécanique, l'état 1 n'a pas de transition vers lui-même, c'est-à-dire qu'on pourrait l'étiqueter par  $\perp$ , symbole d'expression régulière qui dénote le langage vide, et l'expression régulière  $\perp*$  est équivalente à  $\varepsilon$ ); mais il ne faut pas oublier que l'état 2 reçoit lui aussi une transition vers lui-même (en passant par 1) étiquetée  $aa$ , qu'on peut fusionner avec la transition vers lui-même déjà existante étiquetée par  $b$  pour obtenir une transition étiquetée  $b|aa$ ; de même, l'état 0 reçoit une transition étiquetée  $bb$ , qu'on peut fusionner avec celle existante pour obtenir  $a|bb$ . L'élimination de l'état 2 fait alors apparaître une transition de 0 vers lui-même étiquetée  $ba(b|aa)*ab$ , qu'on peut fusionner avec la transition vers lui-même déjà étiquetée par  $a|bb$  pour une seule étiquetée par  $a|bb|ba(b|aa)*ab$ . On obtient finalement

$$(a|bb|ba(b|aa)*ab)*$$

(en particulier, cette expression est équivalente à celle obtenue précédemment).

(C) Si on préfère commencer par éliminer l'état 0, il faut au préalable ajouter un nouvel état initial  $I$  (conduisant à 0 par une transition spontanée) et un nouvel état final  $F$  (vers lequel 0 conduit par une transition spontanée). L'élimination de l'état 0 fait apparaître une transition de  $I$  vers 1 étiquetée  $a*b$ , une transition 1 vers  $F$  étiquetée  $ba*$ , et une transition de l'état 1 vers lui-même étiquetée  $ba*b$ , et enfin une transition de  $I$  vers  $F$  étiquetée  $a*$ . L'élimination de l'état 2 fait apparaître une transition de 1 vers lui-même étiquetée  $ab*a$ , qu'on peut fusionner avec celle déjà existante étiquetée  $ba*b$  pour obtenir une transition  $(ab*a|ba*b)$ . Finalement, l'élimination de l'état 1 donne l'expression régulière

$$a*|a*b(ab*a|ba*b)*ba*$$

(toujours équivalente aux précédentes).

**Solution 4.171.** (0) L'automate  $M$  est un automate fini non-déterministe à transitions spontanées, ou -NFA (le concept d'« automate déterministe à transitions spontanées » n'aurait tout simplement pas de sens).

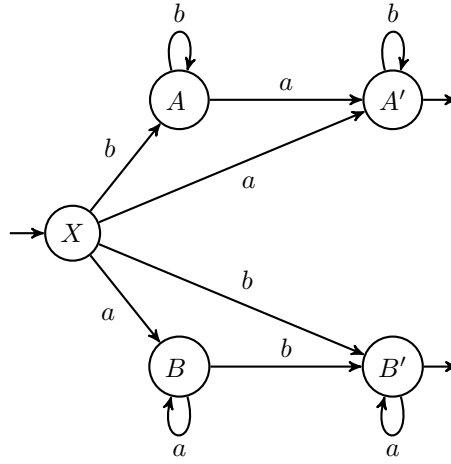
(1a) Le chemin par les états  $X, A, A', Y$  accepte les mots exactement un  $a$ , c'est-à-dire le langage dénoté par  $b*ab*$ . Le chemin par les états  $X, B, B', Y$  accepte les mots comportant exactement un  $b$ , c'est-à-dire le langage dénoté par  $a*ba*$ . L'automate  $M$  dans son ensemble accepte les mots comportant exactement un  $a$  ou (inclusif)



exactement un  $b$  (i.e.  $L = \{w \in \Sigma^* : |w|_a = 1 \text{ ou } |w|_b = 1\}$  si  $|w|_x$  désigne le nombre total d'occurrences de la lettre  $x$  dans le mot  $w$ ). C'est le langage dénoté par l'expression régulière  $b*ab*|a*ba*$  (nous notons ici et ailleurs  $|$  pour la disjonction, qu'on peut aussi noter  $+$ ).

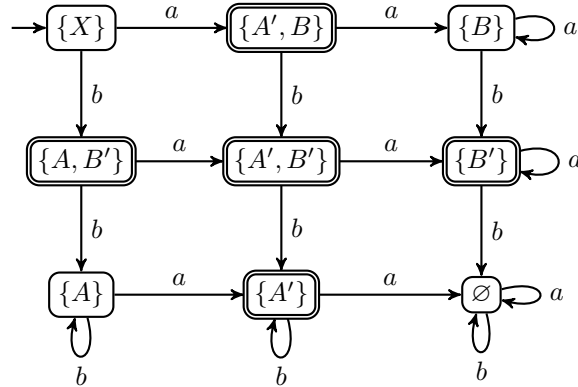
(1b) Parmi les mots proposés,  $a$ ,  $b$ ,  $ab$  et  $aab$  appartiennent à  $L$ , tandis que  $\varepsilon$ ,  $aa$ ,  $aabb$ ,  $abab$  et  $ababa$  n'y appartiennent pas.

(2) La  $-$ fermeture (arrière) de l'état  $X$  est  $\{X, A, B\}$ ; la  $-$ fermeture de l'état  $A'$  est  $\{A', Y\}$  et celle de l'état  $B'$  est  $\{B', Y\}$ ; les autres états sont leur propre  $-$ fermeture (i.e., celle-ci est un singleton). L'élimination des transitions spontanées conduit donc à l'automate  $M_2$  suivant :

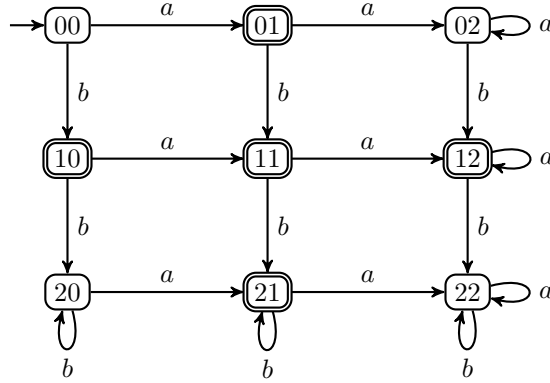


(On a supprimé l'état  $Y$  qui est devenu inutile car aucune transition non-spontanée n'y conduit.)

(3) L'algorithme de déterminisation conduit à l'automate  $M_3$  suivant où, pour plus de lisibilité, les états finaux ont été marqués en les entourant deux fois plutôt que par une flèche sortante :



Pour la commodité de la suite de la correction, on renomme les états de cet automate  $M_3$  de la façon suivante :



Ici, l'état  $0\bullet$  signifie que l'automate n'a pas rencontré de  $a$ , l'état  $1\bullet$  qu'il en a rencontré exactement un, et l'état  $2\bullet$  qu'il en a rencontré au moins deux ; les états  $\bullet 0$ ,  $\bullet 1$  et  $\bullet 2$  ont la même signification pour la lettre  $b$ .

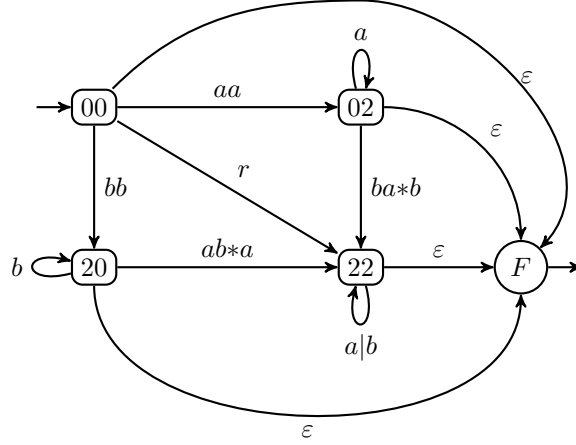
(4) L'automate  $M_3$  est déjà minimal. En effet, l'algorithme de minimisation commence par séparer les classes  $\{01, 10, 11, 12, 21\}$  (états finaux) et  $\{00, 02, 20, 22\}$  (états non-finaux) ; ensuite, la transition étiquetée par  $a$  sépare la classe  $\{01, 10, 11, 12, 21\}$  en  $\{01, 21\}$  (qui vont vers un état non-final) et  $\{10, 11, 12\}$  (qui vont vers un état final), et la classe  $\{00, 02, 20, 22\}$  en  $\{00, 20\}$  (qui vont vers un état final) et  $\{02, 22\}$  (qui vont vers un non-final). La transition étiquetée par  $b$  sépare ensuite en deux chacune des trois classes  $\{00, 20\}$ ,  $\{01, 21\}$  et  $\{02, 22\}$  (car le premier élément va dans la classe  $\{10, 11, 12\}$  tandis que le second reste dans la même classe) et sépare en trois la classe  $\{10, 11, 12\}$ . On a donc séparé chacun des états.

(5) Pour reconnaître le complémentaire du langage reconnu par un automate fini déterministe complet, il suffit d'échanger états finaux et non-finaux : on peut donc prendre l'automate dessiné en (3) avec, cette fois, la convention que les états simplement entourés sont finaux (et les doublement entourés sont non-finaux). Appelons-le  $M_5$ .

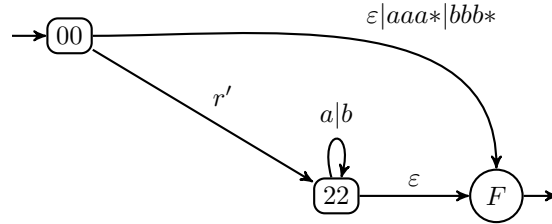
*Attention* : échanger états finaux et non-finaux ne marche pas pour reconnaître le complémentaire du langage reconnu par un automate non-déterministe ou incomplet (car la négation de « il existe un chemin qui va vers un état final » est « aucun chemin ne va vers un état final » et pas « il existe un chemin qui va vers un état non-final »).

(6) Puisque  $L$  est le langage formé des mots comportant exactement un  $a$  ou (inclusif) exactement un  $b$ , son complémentaire  $\bar{L}$  est formé des mots ayant un nombre différent de 1 de  $a$  et un nombre différent de 1 de  $b$  ; si on préfère, il s'agit du langage comportant (0 ou au moins 2 fois la lettre  $a$ ) et (0 ou au moins 2 fois la lettre  $b$ ).

(7) L'élimination des états n'est pas trop complexe car l'automate  $M_5$  a très peu de boucles. Éliminons simultanément tous les états non-finaux (01, 10, 11, 12 et 21), et profitons-en pour créer un nouvel (et unique) état final  $F$  :



où  $r := abaa*b | abbb*a | baaa*b | babb*a$  (correspondant aux quatre façons de passer de 00 à 22 dans le graphe ci-dessus). Éliminons l'état 02 et l'état 20 :



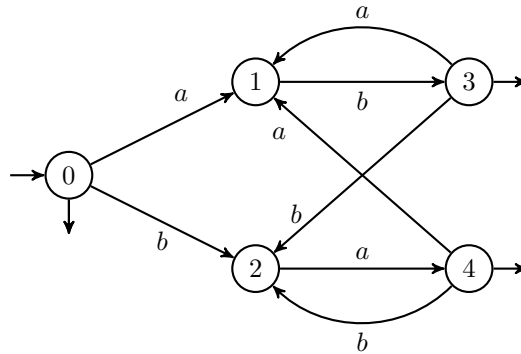
où  $r' := r | aaa*ba*b | bbb*ab*a = abaa*b | abbb*a | baaa*b | babb*a | aaa*ba*b | bbb*ab*a$ . On obtient finalement l'expression régulière suivante pour  $\bar{L}$  :

$$\varepsilon | aaa* | bbb* | (abaa*b | abbb*a | baaa*b | babb*a | aaa*ba*b | bbb*ab*a)(a|b)*$$

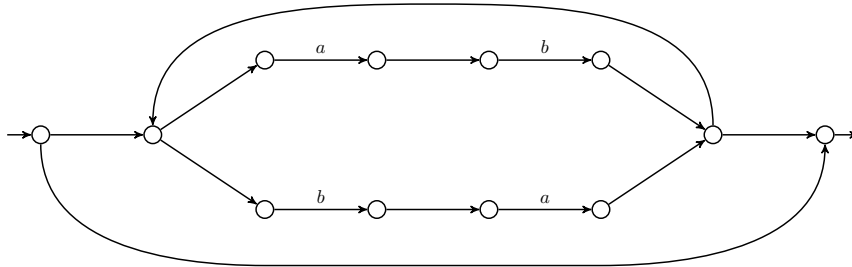
Pour comprendre cette expression régulière, la disjonction de plus haut niveau correspond aux quatre possibilités : (i) 0 fois la lettre  $a$  et 0 fois la lettre  $b$ , (ii) au moins 2 fois la lettre  $a$  et 0 fois la lettre  $b$ , (iii) 0 fois la lettre  $a$  et au moins 2 fois la lettre  $b$ , et (iv) au moins 2 fois la lettre  $a$  et au moins 2 fois la lettre  $b$ . Pour mieux comprendre l'expression du cas (iv), on peut remarquer que  $abaa*b | baaa*b | aaa*ba*b$  dénote le langage formé des mots comportant au moins deux  $a$  et exactement deux  $b$  et qui finissent par un  $b$ , et symétriquement  $abbb*a | babb*a | bbb*ab*a$  dénote le langage formé des mots comportant au moins deux  $b$  et exactement deux  $a$  et qui finissent par un  $a$  : l'expression du cas (iv) correspond donc à écrire un mot ayant au moins deux  $a$  et au moins deux  $b$  comme le premier préfixe qui vérifie cette propriété suivi d'un suffixe quelconque. (On pouvait utiliser directement ce raisonnement pour produire l'expression.)

**Solution 4.172.** Par exemple, les mots  $\varepsilon$ ,  $ab$ ,  $ba$  et  $abba$  appartiennent à  $L$ . Les mots  $a$ ,  $b$ ,  $aa$  et  $aba$  n'appartiennent pas à  $L$ .

L'automate  $\mathcal{A}_1$  obtenu est le suivant :

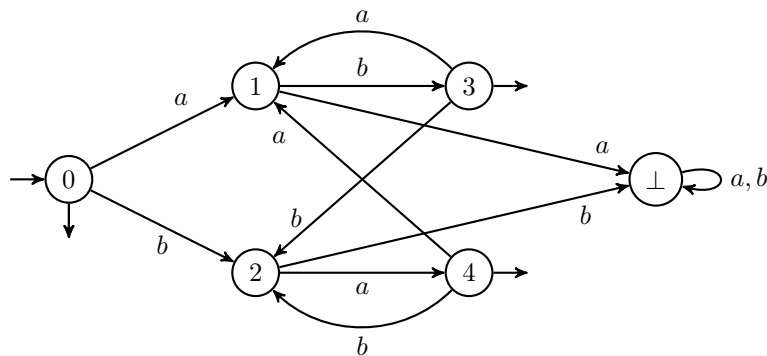


C'est l'automate de Glushkov. Si on commence par construire l'automate de Thompson, on obtient le suivant (à 12 états) :

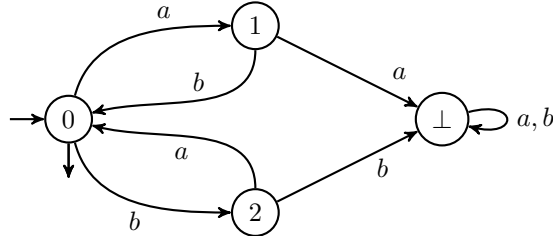


(où toutes les flèches non étiquetées sont des transitions spontanées, c'est-à-dire qu'il faut les imaginer étiquetées par  $\varepsilon$ ) qui par élimination des transitions spontanées donne celui  $\mathcal{A}_1$  qu'on a tracé au-dessus (les seuls états qui subsistent après élimination des transitions spontanées sont l'état initial et les quatre états auxquels aboutissent une transition non spontanée).

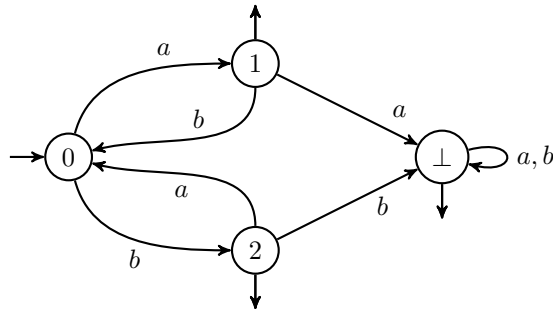
L'automate  $\mathcal{A}_1$  est déterministe incomplet : il s'agit donc simplement de lui ajouter un état « puits » pour le rendre complet, ce qui donne l'automate  $\mathcal{A}_2$  suivant :



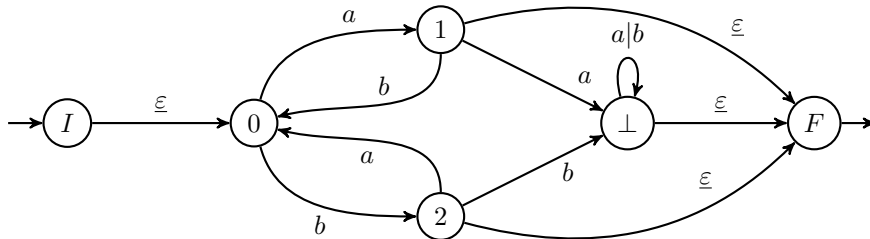
L'algorithme de minimisation identifie les trois états finaux (0, 3, 4) de l'automate  $\mathcal{A}_2$ , ce qui donne l'automate  $\mathcal{A}_3$  suivant (on note 0 pour l'état résultant de l'identification de 0, 3, 4) :



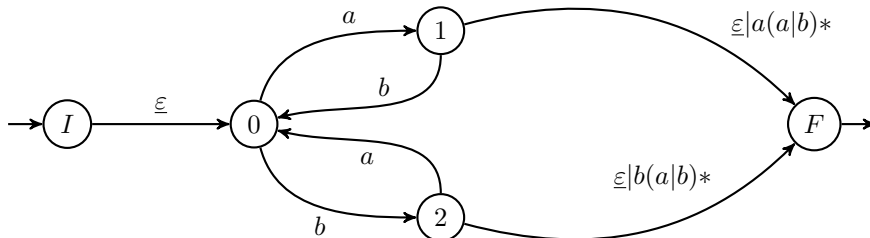
L'automate  $\mathcal{A}_3$  étant un automate fini déterministe complet reconnaissant  $L$ , il suffit de marquer finaux les états non-finaux et vice versa pour obtenir l'automate  $\mathcal{A}_4$  suivant :



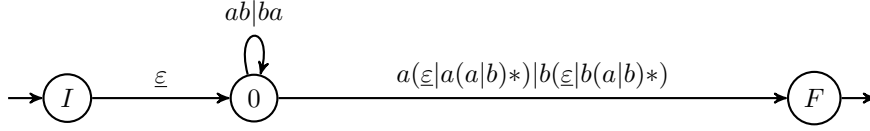
Pour appliquer la méthode d'élimination des états, on commence par modifier l'automate pour avoir un unique état initial  $I$ , qui ne soit pas final, et qui n'ait aucune transition qui y aboutisse, et un unique état final  $F$ , qui ne soit pas initial, et sans aucune transition qui en part :



On élimine ensuite l'état  $\perp$  :



Puis les états 1 et 2 peuvent être éliminés simultanément :



Et l'expression régulière finalement obtenue est la suivante :

$$(ab|ba)^*(a(\varepsilon|a(a|b)^*)|b(\varepsilon|b(a|b)^*))$$

On pouvait aussi donner, entre autres choses :

$$(ab|ba)^*(a|aa(a|b)^*|b|bb(a|b)^*)$$

(obtenue en éliminant les états 1 et 2 avant  $\perp$ ).

### 6.5. Décidabilité et complexité

**Solution 5.58.** Nous allons montrer que  $H'$  est semi-décidable.

Considérons l'algorithme suivant : donné en entrée un triplet  $(e_1, e_2, x)$ , on exécute en parallèle, en fournissant à chacun l'entrée  $x$ , les programmes (codés par)  $e_1$  et  $e_2$  (au moyen d'une machine universelle), et en s'arrêtant immédiatement si l'un des deux s'arrête, et on renvoie alors « vrai ». Manifestement, cet algorithme termine (en renvoyant « vrai ») si et seulement si  $(e_1, e_2, x) \in H'$ , ce qui montre que  $H'$  est semi-décidable.

**Solution 5.58.** Nous allons montrer que  $H'$  n'est pas décidable.

Supposons par l'absurde qu'il existe un algorithme  $T$  qui décide  $H'$ . Soit  $e'$  (le code d')un programme quelconque qui effectue une boucle infinie (quelle que soit son entrée) : comme  $e'$  ne termine jamais, un triplet  $(e, e', x)$  appartient à  $H'$  si et seulement si  $(e, x)$  appartient à  $H$ . Considérons maintenant l'algorithme suivant : donné en entrée un couple  $(e, x)$ , on applique l'algorithme  $T$  supposé exister pour décider si  $(e, e', x) \in H'$  ; comme on vient de l'expliquer, ceci équivaut à  $(e, x) \in H$ . On a donc fabriqué un algorithme qui décide  $H$ , ce qui est impossible, d'où la contradiction annoncée.

**Solution 5.59.** (1) Donnée une expression rationnelle  $r$ , on sait qu'on peut algorithmiquement fabriquer un automate fini non-déterministe à transitions spontanées qui reconnaît exactement le langage  $L_r$  dénoté par  $r$ , et ensuite éliminer les transitions spontanées et déterminer l'automate pour obtenir un automate fini déterministe complet reconnaissant  $L_r$ . Sur un tel automate, savoir si  $L_r \neq \Sigma^*$  est trivial : dès lors qu'il existe un état  $q$  non-final accessible, il existe un mot rejeté par l'automate (i.e., n'appartenant pas à  $L_r$ ), à savoir le mot lu en suivant les étiquettes d'un chemin quelconque de l'état initial  $q_0$  jusqu'à  $q$ , et inversement, si tous les états sont finaux, il est trivial que l'automate accepte tous les mots. (On pouvait aussi minimiser l'automate et le comparer à l'automate minimal trivial qui reconnaît le langage  $\Sigma^*$ .)

(2) Par hypothèse on dispose d'un algorithme  $T$  qui, donné un mot  $w$ , décide si  $w \in L_T$ . Pour semi-décider s'il existe un  $w$  tel que  $w \notin L_T$ , il suffit de tester tous les mots possibles : plus exactement, on construit un algorithme  $A_2$  qui effectue une

boucle infinie sur tous les  $w \in \Sigma^*$  et, pour chacun, teste si  $w \in L_T$  (en lançant l'algorithme  $T$  qui, par hypothèse, termine toujours), et si ce n'est pas le cas, termine immédiatement en répondant « vrai » (on a trouvé un  $w$  n'appartenant pas à  $L_T$ ), tandis que si c'est le cas, l'algorithme  $A_2$  ne terminera jamais.

(4) Supposons par l'absurde qu'on dispose d'un algorithme  $A_3$  comme on vient de dire, et montrons pour arriver à une contradiction qu'on peut s'en servir pour résoudre le problème de l'arrêt. On se donne donc un algorithme  $S$  et une entrée  $x$  de  $S$  et on cherche à savoir (en utilisant  $A_3$ ) si  $S$  termine sur l'entrée  $x$ . Pour cela, on va construire un  $T$  auquel appliquer  $A_3$ .

Voici une solution possible : donné un mot  $w \in \Sigma^*$ , le programme  $T$  ne considère que la longueur  $|w|$  de  $w$ , et lance (=simule) l'exécution de  $S$  sur l'entrée  $x$  pour au plus  $|w|$  étapes : si l'exécution termine dans le temps imparti, alors  $T$  rejette le mot  $w$ , sinon, il l'accepte (dans tous les cas,  $T$  termine et répond « vrai » ou « faux », donc il est une entrée légitime à  $A_3$ ). Cette construction fait que  $L_T$  rejette au moins un mot précisément lorsque  $S$  termine sur  $x$  : si au contraire  $S$  ne termine pas sur  $x$ , alors  $L_T = \Sigma^*$ . L'utilisation de  $A_3$  sur  $T$  permet donc de savoir algorithmiquement si  $S$  termine sur  $x$ , ce qui contredit l'indécidabilité du problème de l'arrêt.

*Variante de la même idée* : on appelle « trace d'exécution » de  $S$  sur  $x$  un mot  $w$  qui code le calcul complet de l'exécution de  $S$  sur  $x$  (par exemple, si on voit  $S$  comme une machine de Turing, l'état courant et le contenu du ruban à chaque étape), du début à l'arrêt. Une telle trace d'exécution existe donc précisément si  $S$  termine sur  $x$ . Or il est visiblement décidable de savoir si un mot  $w$  donné est une trace d'exécution (il suffit de vérifier qu'à chaque étape la machine a bien fait ce qu'elle devait faire). On peut donc écrire un algorithme  $T$  qui termine toujours et accepte précisément les mots qui *ne sont pas* une trace d'exécution de  $S$  sur  $x$ . Le fait que  $L_T$  soit différent de  $\Sigma^*$  signifie alors exactement qu'une trace d'exécution existe, donc que  $S$  termine sur  $x$ . Ainsi l'utilisation de  $A_3$  permet de savoir algorithmiquement si  $S$  termine sur  $x$ , ce qui contredit l'indécidabilité du problème de l'arrêt.

**Solution 5.60.** (1) Si  $w = u^k$  pour un certain  $u \neq \varepsilon$ , alors nécessairement  $k \leq |w|$  puisque  $|w| = k \cdot |u|$ . On dispose donc de l'algorithme suivant pour décider si  $w \in L$  : si  $w = \varepsilon$ , retourner vrai immédiatement ; sinon, pour  $k$  allant de 2 à  $|w|$  et qui divise  $|w|$ , considérer les  $k$  facteurs successifs de  $w$  de longueur  $|w|/k$  (c'est-à-dire, pour  $0 \leq i < k$ , le facteur de  $w$  de longueur  $\ell := |w|/k$  commençant à la position  $i\ell$ ) : s'ils sont tous égaux, renvoyer vrai ; si la boucle termine sans avoir trouvé de  $k$  qui convienne, renvoyer faux. Le langage proposé est donc décidable (et *a fortiori* semi-décidable).

(2) Donné un  $e \in \mathbb{N}$ , la fonction  $10^{1000+e}$  est évidemment calculable. On peut ensuite lancer l'exécution du  $e$ -ième programme, sur l'entrée 42, pour au plus ce nombre d'étapes (en utilisant la machine universelle, c'est-à-dire, par exemple, en simulant la  $e$ -ième machine de Turing sur une machine de Turing). Si l'exécution termine en le temps imparti, on renvoie vrai, sinon, on renvoie faux : ceci montre que l'ensemble proposé est bien décidable (et *a fortiori* semi-décidable).

(3) L'ensemble  $A$  proposé est « presque » le problème de l'arrêt. La différence est que le problème de l'arrêt est l'ensemble des couples  $(e, n)$  tels que le  $e$ -ième programme termine sur l'entrée  $n$  alors qu'ici on a fixé l'entrée à 42. Il s'agit donc de montrer que cette limitation ne rend pas pour autant calculable l'ensemble considéré. Or donnés deux entiers  $(e, n)$ , on peut fabriquer un programme  $e'$  qui prend en entrée une valeur, ignore cette valeur, et exécute le  $e$ -ième programme sur l'entrée  $n$ ; de plus un tel  $e'$  se calcule algorithmiquement<sup>(1)</sup> à partir de  $e$  et  $n$ . L'exécution du programme  $e'$  sur l'entrée 42 (ou n'importe quelle autre entrée) se comporte donc comme l'exécution du programme  $e$  sur l'entrée  $n$ , et notamment, termine si et seulement si elle termine. Autrement dit,  $e'$  appartient à l'ensemble  $A$  considéré dans cette question si et seulement si  $(e, n)$  appartient au problème de l'arrêt. Comme on vient de dire qu'on peut calculer  $e'$  algorithmiquement à partir de  $(e, n)$ , si l'ensemble  $A$  était décidable, le problème de l'arrêt le serait, ce qui n'est pas le cas. Donc  $A$  n'est pas décidable. En revanche,  $A$  est semi-décidable : il suffit de lancer l'exécution du programme  $e$  sur l'entrée 42 et renvoyer vrai si elle termine (si elle ne termine pas, on ne termine pas).

**Solution 5.61.** (A) Supposons qu'on dispose d'algorithmes  $T_1$  et  $T_2$  qui décident  $L_1$  et  $L_2$  respectivement (i.e., donné  $w \in \Sigma^*$ , l'algorithme  $T_i$  termine toujours en temps fini, en répondant oui si  $w \in L_i$  et non si  $w \notin L_i$ ).

Pour faire un algorithme qui décide  $L_1 \cup L_2$ , donné un mot  $w \in \Sigma^*$ , il suffit de lancer successivement  $T_1$  et  $T_2$  : si l'un des deux répond oui, on répond oui, sinon on répond non (autrement dit, on calcule les valeurs de vérité de  $w \in L_1$  et  $w \in L_2$  au moyen de  $T_1$  et  $T_2$ , et on calcule ensuite leur « ou » logique). De même pour décider  $L_1 \cap L_2$ , il suffit de lancer successivement  $T_1$  et  $T_2$ , si les deux répondent oui on répond oui, sinon on répond non (i.e., on calcule les valeurs de vérité de  $w \in L_1$  et  $w \in L_2$  au moyen de  $T_1$  et  $T_2$ , et on calcule ensuite leur « et » logique).

Pour décider  $L_1 L_2$ , on effectue une boucle sur toutes les factorisations  $w = uv$  de  $w$ , c'est-à-dire, une boucle sur toutes les longueurs  $0 \leq i \leq |w|$  en appelant à chaque fois  $u$  le préfixe de  $w$  de longueur  $i$  et  $v$  le suffixe de  $w$  de longueur  $|w| - i$ , et pour chaque paire  $(u, v)$  ainsi trouvée, on utilise  $T_1$  et  $T_2$  pour tester si  $u \in L_1$  et  $v \in L_2$  : si c'est le cas, on termine l'algorithme en répondant oui (on a  $w = uv \in L_1 L_2$ ) ; si aucune paire ne convient, on répond non.

L'algorithme pour décider  $L^*$  est semblable : il s'agit de tester toutes les manières de factoriser un mot  $w \in \Sigma^*$  en facteurs de longueur non nulle. (On peut d'ores et déjà exclure  $w = \varepsilon$  car le mot vide appartient de toute façon à  $L^*$ .) Si  $n = |w| > 0$ , on peut effectuer une boucle pour un nombre de facteurs  $k$  allant de 1 à  $n$ , et, pour chaque  $k$ , effectuer  $k$  boucles emboîtées pour déterminer les limites des facteurs  $u_1, \dots, u_k \in \Sigma^+$  tels que  $w = u_1 \cdots u_k$  (il suffit par exemple de faire boucler  $i_1, \dots, i_k$  chacun de 1 à  $n$ , et lorsque  $i_1 + \dots + i_k = n$ , appeler  $u_j$  le facteur de  $w$  de longueur  $i_j$  commençant à la position  $i_1 + \dots + i_{j-1}$ ). Pour chaque factorisation comme on vient de le dire,

1. Techniquement, on invoque ici le théorème s-m-n ; mais dans les faits, il s'agit essentiellement d'ajouter une ligne « `let n=(représentation décimale de n)` » au début d'un programme, ce qui est certainement faisable.



on teste si tous les  $u_i$  appartiennent à  $L$ , et si c'est le cas on renvoie vrai (le mot  $w$  appartient à  $L^*$ ) ; si aucune factorisation ne convient, on renvoie faux.

(Dans l'algorithme qui précède, on a écarté les factorisations faisant intervenir le mot vide, car si  $w$  est factorisable en mots de  $L$  en faisant intervenir le mot vide, quitte à retirer celui-ci, il est encore factorisable en mots non vides de  $L$ .)

(B) Les algorithmes sont très semblables à ceux de la partie (A) si ce n'est qu'il faut tenir compte de la possibilité qu'ils puissent ne pas terminer. À part pour l'intersection, on doit donc les lancer « en parallèle » et pas « en série » : lorsqu'on dira qu'on lance deux algorithmes  $T$  et  $T'$  « en parallèle », cela signifie qu'on exécute une étape du calcul de  $T$ , puis une étape de  $T'$ , puis de nouveau une de  $T$ , et ainsi de suite en alternant entre les deux, jusqu'à ce que l'un termine et renvoie vrai.

Si  $L_1$  et  $L_2$  sont semi-décidables et si  $T_1$  et  $T_2$  sont des algorithmes qui les « semi-décident » (i.e.,  $T_i$  termine en temps fini et répond oui si  $w \in L_i$ , et ne termine pas sinon), pour semi-décider  $L_1 \cup L_2$ , on lance les deux algorithmes  $T_1$  et  $T_2$  en parallèle sur le même mot  $w$  : si l'un d'eux termine, on termine en renvoyant vrai (sinon, bien sûr, on ne termine pas).

Pour semi-décider  $L_1 \cap L_2$ , en revanche, il n'y a pas de raison de procéder en parallèle : on lance d'abord  $T_1$  sur le mot  $w$  à tester : si  $T_1$  termine, on lance ensuite  $T_2$  sur le même mot : si  $T_2$  termine et renvoie vrai, on renvoie vrai ; si l'un des deux algorithmes  $T_i$  lancés séquentiellement ne termine pas, bien sûr, le calcul dans son ensemble ne terminera pas.

Pour semi-décider  $L_1 L_2$  ou  $L^*$ , on procède comme dans le cas (A) en lançant en parallèle les algorithmes pour tester toutes les différentes factorisations possibles  $w = uv$  ou bien  $w = u_1 \cdots u_k$  (en mots non vides) du mot  $w$ .

**Solution 5.62.** Montrons que (1) implique (2) : si on dispose d'un algorithme capable de calculer  $f(n)$  en fonction de  $n$ , alors il est facile d'écrire un algorithme capable de décider si  $p = f(n)$  (il suffit de calculer  $f(n)$  avec l'algorithme supposé exister, de comparer avec la valeur de  $p$  fournie, et de renvoyer vrai/1 si elles sont égales, et faux/0 sinon).

Le fait que (2) implique (3) est évident car tout ensemble décidable est semi-décidable.

Montrons que (2) implique (1) même si ce ne sera au final pas utile : supposons qu'on ait un algorithme  $T$  qui décide  $\Gamma_f$  (i.e., donnés  $(n, p)$ , termine toujours en temps fini, en répondant oui si  $p = f(n)$  et non si  $p \neq f(n)$ ), et on cherche à écrire un algorithme qui calcule  $f(n)$ . Pour cela, donné un  $n$ , il suffit de lancer l'algorithme  $T$  successivement sur les valeurs  $(n, 0)$  puis  $(n, 1)$  puis  $(n, 2)$  et ainsi de suite (c'est-à-dire faire une boucle infinie sur  $p$  et lancer  $T$  sur chaque couple  $(n, p)$ ) jusqu'à trouver un  $p$  pour lequel  $T$  réponde vrai : on termine alors en renvoyant la valeur  $p$  qu'on a trouvée, qui vérifie  $p = f(n)$  par définition de  $T$ .

Reste à montrer que (3) implique (1) : supposons qu'on ait un algorithme  $T$  qui « semi-décide »  $\Gamma_f$  (i.e., donnés  $(n, p)$ , termine en temps fini et répond oui si  $p = f(n)$ , et ne termine pas sinon), et on cherche à écrire un algorithme qui calcule  $f(n)$ . Pour cela, on va tester les valeurs  $0 \leq p \leq M$  chacune pour  $M$  étapes et faire tendre  $M$  vers l'infini : plus exactement, on utilise l'algorithme  $U$  suivant :

- pour  $M$  allant de 0 à l'infini,
  - pour  $p$  allant de 0 à  $M$ ,
    - exécuter l'algorithme  $T$  sur l'entrée  $(n, p)$  pendant au plus  $M$  étapes,
    - s'il termine en renvoyant vrai (1), terminer et renvoyer  $p$  (sinon, continuer les boucles).

(Intuitivement,  $U$  essaie de lancer l'algorithme  $T$  sur un nombre de valeurs de  $p$  de plus en plus grand et en attendant de plus en plus longtemps pour voir si l'une d'elles termine.)

Si l'algorithme  $U$  défini ci-dessus termine, il renvoie forcément  $f(n)$  (puisque l'algorithme  $T$  a répondu vrai, c'est que  $p = f(n)$ , et on renvoie la valeur en question) ; il reste à expliquer pourquoi  $U$  termine toujours. Mais la valeur  $f(n)$  existe (même si on ne la connaît pas) car la fonction  $f$  était supposée définie partout, et lorsque l'algorithme  $T$  est lancé sur  $(n, f(n))$  il est donc censé terminer en un certain nombre (fini !) d'étapes : si  $M$  est supérieur à la fois à  $f(n)$  et à ce nombre d'étapes, la valeur  $f(n)$  va être prise par  $p$  dans la boucle intérieure, et pour cette valeur, l'algorithme  $T$  va terminer sur l'entrée  $(n, p)$  en au plus  $M$  étapes, ce qui assure que  $U$  termine effectivement.

L'algorithme  $U$  calcule donc bien la fonction  $f$  demandée, ce qui prouve (1).

**Solution 5.63.** Supposons  $A$  décidable : on va construire  $f$  comme indiqué. Plus exactement, on va appeler  $f(n)$  le  $n$ -ième élément de  $A$  par ordre croissant (c'est-à-dire que  $f(0)$  est le plus petit élément de  $A$ , et  $f(1)$  le suivant par ordre de taille, et ainsi de suite ; noter que  $A$  est infini donc cette fonction est bien définie). Montrons que  $f$  est calculable : donné un entier  $n$ , on teste successivement si  $0 \in A$  puis  $1 \in A$  puis  $2 \in A$  et ainsi de suite, à chaque fois en utilisant un algorithme décidant  $A$  (qui est censé exister par hypothèse) jusqu'à obtenir  $n$  fois la réponse « oui » ; plus exactement :

- initialiser  $m \leftarrow 0$ ,
- pour  $k$  allant de 0 à l'infini,
  - interroger l'algorithme qui décide si  $k \in A$ ,
  - s'il répond « oui » :
    - si  $m = n$ , terminer et renvoyer  $k$ ,
    - sinon, incrémenter  $m$  (c'est-à-dire faire  $m \leftarrow m + 1$ ).

La boucle termine car  $A$  est infini.

Réciproquement, supposons  $f$  strictement croissante calculable et posons  $A = f(\mathbb{N})$  : on veut montrer que  $A$  est décidable. Or pour décider si  $k \in A$ , il suffit de calculer successivement  $f(0)$ ,  $f(1)$ ,  $f(2)$  et ainsi de suite, et de terminer si  $f(n)$  atteint ou dépasse le  $k$  fixé : s'il l'atteint, on renvoie vrai (on a trouvé  $n$  tel que  $f(n) = k$ ), sinon, on renvoie faux (la valeur  $k$  a été sautée par la fonction  $f$  et ne sera donc jamais atteinte). L'algorithme est donc explicitement :

- pour  $n$  allant de 0 à l'infini,
  - calculer  $f(n)$ ,

- si  $f(n) = k$ , renvoyer vrai,
- si  $f(n) > k$ , renvoyer faux.

La boucle termine car toute fonction strictement croissante  $\mathbb{N} \rightarrow \mathbb{N}$  est de limite  $+\infty$  en l'infini (donc  $f(n)$  finit forcément par atteindre ou dépasser  $k$ ).

**Solution 5.64.** Supposons que  $M$  soit calculable. On peut alors résoudre le problème de l'arrêt de la manière suivante : donné un algorithme  $T$ , de numéro  $e$ , et une entrée  $n$  à fournir à cet algorithme, pour savoir si  $T$  s'arrête, on calcule  $M(k)$  où  $k = \max(e, n)$ , on exécute ensuite l'algorithme  $T$  pendant au plus  $M(k)$  étapes : s'il termine dans le temps imparti, on répond vrai (il a terminé), sinon, on répond faux (il ne terminera jamais). Cette résolution du problème de l'arrêt est correcte, car si  $T$  termine sur l'entrée  $n$ , il prendra par définition  $S(e, n)$  étapes, avec  $0 \leq e \leq k$  et  $0 \leq n \leq k$  par définition de  $k$ , donc  $S(e, n) \leq M(k)$  par définition de  $M(k)$  : ceci signifie précisément que si  $T$  n'a pas terminé en  $M(k)$  étapes, il ne terminera jamais.

Exactement le même argument montre que  $M'$  n'est pas calculable sous l'hypothèse que  $M'(k) \geq M(k)$  pour tout  $k$  : s'il l'était, on pourrait exécuter l'algorithme  $T$  pendant au plus  $M'(k)$  étapes, et comme on a  $S(e, n) \leq M(k) \leq M'(k)$ , la même démonstration convient.

Enfin, si on suppose seulement  $M'(k) \geq M(k)$  pour  $k \geq k_0$ , la fonction  $M'$  n'est toujours pas calculable : en effet, si on suppose par l'absurde qu'elle l'est, la fonction  $M''$  qui à  $k$  associe  $M'(k)$  si  $k \geq k_0$  et  $M(k)$  sinon, serait encore calculable puisqu'elle ne diffère de  $M'$  qu'en un nombre fini de valeurs, or changer la valeur en un point d'une fonction calculable donne toujours une fonction calculable (même si on « ne connaît pas » la valeur à changer, elle existe, donc l'algorithme modifié existe). Mais d'après le paragraphe précédent,  $M''$  n'est pas calculable puisqu'elle est partout supérieure ou égale à  $M$ .

**Solution 5.65.** C'est Hippolyte qui a raison ( $L$  n'est pas semi-décidable).

L'argument de Thésée est stupide : une instruction mettant fin au programme avec une valeur autre que 0 pourrait ne jamais être atteinte, et réciproquement, même si toutes les instructions mettant fin au programme renvoyaient 0, il pourrait aussi ne jamais terminer. (Au passage, l'argument de Thésée semblerait même montrer que  $L$  est décidable, pas juste semi-décidable.)

L'argument d'Hippolyte, lui, est correct : il montre que si  $L$  était semi-décidable, le complémentaire du problème de l'arrêt (l'ensemble des  $e'$  qui ne terminent jamais) serait semi-décidable, ce qui n'est pas le cas. (Le complémentaire du problème de l'arrêt n'est pas semi-décidable, car s'il l'était, le problème de l'arrêt serait décidable, vu qu'il est déjà semi-décidable.) Parmi les points qui méritent éventuellement d'être précisés, on peut mentionner : le fait que  $e$  se déduise algorithmiquement de  $e'$  et  $m$  ; et le fait qu'il est algorithmiquement possible de lancer l'exécution d'un programme sur  $n$  étapes (peu importe la définition exacte de « étape » tant que chacune termine à coup sûr en temps fini) et tester si elle est bien finie au bout de ce temps.

**Solution 5.65.** C'est Patrocle qui a raison ( $M$  n'est pas semi-décidable).

Le raisonnement d'Achille se fonde sur l'idée que le complémentaire de « l'ensemble des programmes qui renvoient toujours la valeur 0 » est « l'ensemble des programmes qui renvoient parfois une valeur autre que 0 », ce qui oublie la possibilité que le

programme ne termine pas. C'est là la principale erreur (le reste est globalement correct).

L'argument de Patrocle, lui, est correct : il montre que si  $M$  était semi-décidable, le complémentaire du problème de l'arrêt (l'ensemble des  $e'$  qui ne terminent jamais) serait semi-décidable, ce qui n'est pas le cas. (Le complémentaire du problème de l'arrêt n'est pas semi-décidable, car s'il l'était, le problème de l'arrêt serait décidable, vu qu'il est déjà semi-décidable.) On fait appel à des fonctions constantes, et qui ne peuvent renvoyer que 0 ou ne pas terminer, mais ce n'est pas spécialement problématique. Parmi les points qui méritent éventuellement d'être précisés, on peut mentionner le fait que  $e$  se déduise algorithmiquement de  $e'$  et  $m$ .

- Solution 5.66.**
1. Nous appelons  $a$  le sommet auxillière ajouté à  $G$  dans  $f_1(G)$ . Supposons que  $G$  contienne une  $k$ -clique. Soit  $S$  une telle clique. Alors  $S' = S \cup \{a\}$  est une  $k+1$ -clique de  $f_1(G)$ . Réciproquement, soit  $S'$  une  $(k+1)$ -clique de  $f_1(G)$ . Si  $a \in S'$ , alors  $S = S' \setminus \{a\}$  est encore une  $k$ -clique de  $G$ . Sinon, n'importe quel  $S \subseteq S'$  est une  $k$ -clique de  $G$ .
  2. On considère  $f_a(G) = f_1 \circ \dots \circ f_1(G)$  (composition  $a$  fois).
  3. Il nous faut résoudre  $k + a = (n + a)/2$  ce qui équivaut à  $a = n - 2k$ .
  4. Nous notons  $g_1(G)$  le graphe obtenu à partir de  $G$  en ajoutant un sommet isolé (relié à aucun autre sommet). Il est alors clair que  $G$  contient une  $k$ -clique ssi  $g_1(G)$  contient une  $k$ -clique. Nous posons  $g_a(G) = g_1 \circ \dots \circ g_1(G)$  ( $a$  fois). En particulier  $G$  contient une  $k$ -clique ssi  $g_{2k-n}(G)$  contient une demiclique.
  5. Nous construisons désormais la réduction suivante. Soit  $I = (G, k)$  une instance de CLIQUE. Si  $k \leq n/2$ , soit  $I'$  l'instance  $I' = (f_{2n-k}(G))$  de DEMICLIQUE, sinon soit  $I' = g_{2k-n}(G)$ . Nous remarquons que la taille de  $I'$  est polynomiale en la taille de  $I$ , d'autre part la construction de  $I'$  nécessite un temps polynomial en la taille de  $I$ . Enfin

$$\text{CLIQUE}(I) = \text{DEMICLIQUE}(I').$$

Donc CLIQUE se réduit à DEMICLIQUE, ce qui signifie que DEMICLIQUE n'est pas un problème plus facile que CLIQUE.

**Solution 5.67.** Un chemin hamiltonien dans  $G = (V, E)$  est un chemin élémentaire de longueur  $n - 1$  où  $n = |V|$ , i.e. de longueur la plus grande possible.

Étant donné une instance  $G_0 = (V_0, E_0)$  un graphe, que l'on voit comme une instance de CHEMINHAMILTONIEN, on construit l'instance  $(G, p, s, t, \ell)$  suivante de COURTCHEMINÉLEMENTAIRE. D'abord, nous créons  $s$  et  $t$  deux nouveaux sommets. Nous posons  $G = (V_0 \cup \{s, t\}, E)$  avec  $E = E_0 \cup \{(s, u), (u, t); u \in V_0\}$ . Enfin, nous choisissons la fonction poids  $p(e) = -1$  pour tout  $e \in E$  et  $\ell = n + 1$ .

Montrons que CHEMINHAMILTONIEN( $G_0$ ) équivaut à COURTCHEMINÉLEMENTAIRE( $G, p, s, t, \ell$ ). Condition nécessaire. Si  $G_0$  possède un chemin hamiltonien, disons

$$v_{i_1} - v_{i_2} - \dots - v_{i_n},$$

alors, on peut créer dans  $G$  le chemin élémentaire

$$s - v_{i_1} - v_{i_2} - \dots - v_{i_n} - t,$$

entre  $s$  et  $t$  qui est de longueur  $-(n+1)$ , cqfd.

Condition suffisante. Si  $G$  possède un chemin de longueur  $\leq \ell$  entre  $s$  et  $t$ , disons

$$s - v_{j_1} - v_{j_2} - \cdots - v_{j_k} - t,$$

comme les poids sont tous  $-1$ , ce chemin est de longueur  $-k-1$ . Donc  $-k-1 \leq -n-1$ . Mais ce chemin est élémentaire, donc  $k \leq n$ . Ainsi  $k = n$  est

$$v_{j_1} - v_{j_2} - \cdots - v_{j_n}$$

est un chemin hamiltonien de  $G_0$ , cqfd.

Par ailleurs, la nouvelle instance se calcule en temps linéaire par rapport à la taille de la première instance. Donc notre réduction est bien polynomiale.

On peut en déduire que COURTCHEMINELEMENTAIRE est NP-complet.

Remarque : on sait que la recherche d'un plus court chemin dans un graphe dont les poids sont positifs est réalisable en temps polynomial avec l'algorithme de Dijkstra. Il est donc nécessaire de passer par des poids négatifs pour réaliser cette réduction.

**Solution 5.68.** 1. On peut écrire la table de vérité de chaque clause

| $x_1$ | $x_2$ | $x_3$ | $\neg x_1 \vee x_2 \vee x_3$ | $x_1 \vee \neg x_2 \vee x_3$ | $x_1 \vee x_2 \vee \neg x_3$ | $F_0$ |
|-------|-------|-------|------------------------------|------------------------------|------------------------------|-------|
| $V$   | $V$   | $V$   | $V$                          | $V$                          | $V$                          | $V$   |
| $F$   | $V$   | $V$   | $V^*$                        | $V$                          | $V$                          | $V$   |
| $V$   | $F$   | $V$   | $V$                          | $V^*$                        | $V$                          | $V$   |
| $F$   | $F$   | $V$   | $V$                          | $V$                          | $F$                          | $F$   |
| $V$   | $V$   | $F$   | $V$                          | $V$                          | $V^*$                        | $V$   |
| $F$   | $V$   | $F$   | $V$                          | $F$                          | $V$                          | $F$   |
| $V$   | $F$   | $F$   | $F$                          | $V$                          | $V$                          | $F$   |
| $F$   | $F$   | $F$   | $V$                          | $V$                          | $V$                          | $V$   |

Dans le tableau, on marque par une  $*$  les clauses dont tous les littéraux prennent la valeur de vérité  $V$ .

On constate que la formule  $F_0$  admet 5 modèles, mais seulement deux d'entre eux sont NAE :  $\{x_1 = x_2 = x_3 = V\}$  et  $\{x_1 = x_2 = x_3 = F\}$ . Les 3 autres modèles ne sont pas NAE.

2. Prenons  $C_0 = x_1 \vee x_2 \vee x_3$  et réécrivons la table de vérité.

| $x_1$ | $x_2$ | $x_3$ | $F_0$ | $C_0$ | $F_1$ |
|-------|-------|-------|-------|-------|-------|
| $V$   | $V$   | $V$   | $V$   | $V$   | $V$   |
| $F$   | $V$   | $V$   | $V$   | $V$   | $V$   |
| $V$   | $F$   | $V$   | $V$   | $V$   | $V$   |
| $F$   | $F$   | $V$   | $F$   | $V$   | $F$   |
| $V$   | $V$   | $F$   | $V$   | $V$   | $V$   |
| $F$   | $V$   | $F$   | $F$   | $V$   | $F$   |
| $V$   | $F$   | $F$   | $F$   | $V$   | $F$   |
| $F$   | $F$   | $F$   | $V$   | $F$   | $F$   |

On constate que  $F_1$  est toujours une formule satisfiable, cependant, plus aucun modèle n'est NAE.

Cette question permet de montrer que les notions de satisfiabilité et de NAE-satisfiabilité sont deux notions distinctes.

3. Supposons que la formule  $\Phi$  soit satisfiable et montrons que  $\Phi'$  est NAE satisfiable en cherchant un modèle NAE de  $\Phi'$ . Comme  $\Phi$  est satisfiable, il existe une valuation telle que l'interprétation de  $\Phi$  est  $V$ . Pour NAE-satisfaire  $\Phi'$ , nous gardons la même valuation que nous complétons comme suit. Nous posons  $z = F$ . Pour toute clause  $C_i = \lambda \vee \mu \vee \nu$ , nous posons  $w_i \leftarrow \nu \wedge \neg\lambda$ . Vérifions que  $\Phi'$  est bien NAE-satisfaite.
  - Si  $\nu = V$ , alors  $C_i''$  est bien satisfaite à cause de  $\nu$  et NAE puisque  $z = F$  tandis que  $\nu = V$ . Par ailleurs, dans ce cas, les littéraux  $\lambda$  et  $w_i$  prennent des valeurs opposées, donc  $C_i'$  est à la fois satisfaite et NAE.
  - Si  $\nu = F$ , alors  $w_i = F$  aussi. Donc  $\neg w_i = V$ . Ceci implique que  $C_i''$  est NAE-satisfaite. D'autre part, nous devons avoir  $\lambda = V$  ou  $\mu = V$  (sinon l'hypothèse  $\Phi$  satisfaite ne tiendrait pas). Donc  $C_j'$  est aussi NAE-satisfaite.

Supposons que la formule  $\Phi'$  soit NAE satisfiable et montrons que  $\Phi$  est satisfiable. Soit une valuation telle que l'interprétation de la formule  $\Phi'$  prend la valeur de vérité  $V$ . Quitte à changer la valuation pour la valuation opposée, nous pouvons toujours supposer que  $z = F$ . Soit  $C_i = \lambda \vee \mu \vee \nu$  l'une des clause de  $\Phi$ , montrons qu'elle est satisfaite.

- Si  $\nu = V$ , alors  $C_i$  est clairement satisfaite.
  - Si  $\nu = F$ , alors nous devons avoir  $\neg w_i = V$  pour que  $C_i''$  soit satisfaite. Donc  $w_i = F$ . Ceci implique que  $\lambda = V$  ou  $\mu = V$  afin que  $C_i''$  soit satisfaite. Mais alors  $C_i$  est aussi satisfaite.
4. Pour montrer que NAE-3-SAT est NP-complet, il suffit de vérifier que NAE-3-SAT est plus difficile qu'un problème NP-complet. On peut par exemple s'appuyer sur le problème 3-SAT, dont on a vu en cours qu'il est NP-complet. Or la question 3 fournit une réduction polynomiale de 3-SAT à NAE-3-SAT. Ainsi NAE-3-SAT est NP-complet.

**Solution 5.69.** 1. Maximiser le nombre d'arêtes coupées

2. Un certificat est donné par une coupe  $S$ ,  $\bar{S}$  et peut être vérifié en temps polynomial.
3. Il y a  $6m$  arêtes. S'il y a une coupe de capacité supérieure à  $5m$ , montrons qu'il existe alors une coupe de capacité supérieure à  $5m$  dans laquelle les variables sont séparées de leur opposé. Supposons que  $x_i$  et  $\neg x_i$  soient dans la même part de la coupe et notons  $n_i^+$  et  $n_i^-$  le nombre d'occurrence de  $x_i$  et  $\neg x_i$  dans  $F$ . Si  $n_i^+ \leq n_i/2$ , on envoie  $x_i$  dans la part complémentaire, sinon, on déplace  $\neg x_i$ . Ce faisant, on gagne  $n_i$  arêtes et on en perd au plus  $2n_i^\pm$ . Au total, la coupe gagne des arêtes.

À présent, donnons aux littéraux de  $S$  la valeur  $\top$ . Les arêtes entre littéraux opposés contribuent à  $3m$  dans la capacité de la coupe. Les  $2m$  restantes viennent donc des triangles. Chaque triangle contribue pour 0 (si les trois littéraux sont du

même côté) ou pour 2 (sinon). Comme il y a  $m$  triangles, aucun n'a de littéraux du même côté, donc tous possèdent un ou deux littéral à vrai. Autrement dit, la formule est NAE-satisfaisable.

Réciproquement, une NAE-satisfaction étant donnée, on met dans  $S$  les littéraux valant  $\top$  pour construire une coupe de capacité  $\geq 5m$ .

4. Immédiat. La réduction précédente est polynomiale.
5. COUPE est NP-complet.

**Solution 5.70.** 1. Entrée : graphe  $G$ . Sortie : trouver un ensemble indépendant de cardinal maximum.

2. On utilise comme certificat un ensemble indépendant. Il est polynomial de vérifier qu'un ensemble de sommets est indépendant.
3. Soit  $F = C_1 \wedge \dots \wedge C_t$  une instance de 3-SAT. Pour toute 3-clause  $C = \lambda_1 \wedge \lambda_2 \wedge \lambda_3$ , on construit trois sommets  $v_{C, \lambda_i}$ . On relie les littéraux complémentaires. Une attribution valable dans 3-SAT avec  $\geq t$  clauses satisfaites donne un ensemble indépendant avec  $\geq t$  sommets et vice-versa.
4. ENSEMBLEINDÉPENDENT est NP-complet.

**Solution 5.71.** 1. Entrée : graphe  $G$ . Sortie : trouver une clique de cardinal maximum.

2. On utilise comme certificat une clique. Il est polynomial de vérifier qu'un ensemble de sommets forme une clique.
3. Passer au complémentaire des arêtes.
4. CLIQUE est NP-complet.

**Solution 5.72.** 1. Entrée : graphe  $G$ . Sortie : trouver une couverture de cardinal minimum.

2. On utilise comme certificat une couverture. Il est polynomial de vérifier que toutes les arêtes du graphes ont une extrémité dans  $S$ .
3. Passer au complémentaire des sommets.
4. SOMMETCOUVERTURE est NP-complet.

**Solution 5.73.** 1. On utilise  $I$  comme certificat. Il est polynômial en  $n$  de tester que  $\sum_{i \in I} a_i = K$ .





## APPENDICE A

### AIDE À LA PROGRAMMATION EN PYTHON

#### A.1. Les objets en Python

**Définition A.1.** — En informatique, un *objet* est un conteneur symbolique et autonome qui contient des informations et des mécanismes concernant un certain sujet.

Un objet possède une collection d'*attributs* qui ont chacun une valeur. En Python, on peut déclarer et accéder aux attributs d'un objet `obj` par la notation pointée : `obj.attr` est la valeur de l'attribut `attr`.

**A.1.1. Création d'un objet à travers une classe.** — En général, on crée un objet à partir d'un modèle, qu'on appelle *classe*.

**Exemple A.2.** — Voici un exemple d'une classe `Point`, qui pour le moment, ne fait rien

```
1330 class Vector():
1331     pass
1332
1333 p = Vector()
1334 p.abscissa = 2
1335 p.ordinate = 3
1336 p.height = -4
```

Ici, on dit alors que la variable `p` est liée à une *instance de la classe* `Vector` ou tout simplement que `p` est un objet `Vector`. L'objet `p` possède à ce stade trois attributs, qui ont été utilisés « à la main » : un attribut `abscissa` qui vaut 2, un attribut `ordinate` qui vaut 3 et un attribut `height` qui vaut -4.

**A.1.2. Méthodes de la classe.** — L'objet hérite alors des comportements et des caractéristiques prévues par la classe. On peut définir des *variables de classes* (des constantes, des fonctions, etc.) c'est-à-dire des variables auxquelles tout objet de la classe a accès. Dans le contexte de la programmation orientée objet, on préfère utiliser

le terme *méthode* pour désigner une fonction qui s'applique à n'importe quelle instance de la classe.

En Python, un méthode se déclare en l'inscrivant dans le corps de la classe (c'est-à-dire en ajoutant une indentation pour signifier que la définition à lieu dans la classe). Par convention, le premier argument se nomme `self` et fait référence à l'objet.

**Exemple A.3.** — Voici un exemple.

```

1337 class Vector():
1338     scalars = "real field"
1339
1340     def norm(self):
1341         return max(self.abscissa, self.ordinate, self.height)
1342
1343 p = Vector()
1344 p.abscissa = 2
1345 p.ordinate = 3
1346 p.height = -4
1347
1348 p.norm()
```

Dans cet exemple, `p.norm()` s'évalue comme l'appel à la fonction `norm` écrite dans la classe `Vector` où l'argument `self` prend la valeur `p`. Par ailleurs, `p.scalars` renvoie la valeur commune `"real field"` préattribuée au niveau de la classe.

Ce mécanisme permet de réutiliser le même nom de variable (notamment de fonctions) au sein de classes différentes. À l'évaluation, Python est capable de retrouver la variable provenant de la classe de l'objet.

**Exemple A.4.** — Dans le code ci-dessous, `p.norm()` et `z.norm()` font référence à deux fonctions différentes : la fonction `norm` de la classe `Vector` dans le premier cas et la fonction `norm` de la classe `Complex` dans le second cas.

```

1349 class Vector():
1350     def norm(self):
1351         return max(self.abscissa, self.ordinate, self.height)
1352
1353 class Complex():
1354     def norm(self):
1355         return self.real**2 + self.imag**2
1356
1357 p = Vector()
1358 p.abscissa = 2
1359 p.ordinate = 3
1360 p.height = -4
1361
1362 z = Complex()
1363 z.real = 1
```

```

1364 z.imag = -1
1365
1366 p.norm()
1367 z.norm()

```

**A.1.3. Constructeurs de classe.** — En général, on souhaite construire des objets dont l'état initial est connu et garanti. Pour cela, on inclut une méthode au nom spécial `__init__()`. Cette méthode s'appelle le *constructeur*. Elle est appelée systématiquement à la construction d'un objet.

*Exemple A.5.* — Nous reprenons le cas vu ci-avant.

```

1368 class Vector():
1369     def __init__(self, argument1, argument2, argument3):
1370         self.abscissa = argument1
1371         self.ordinate = argument2
1372         self.height = argument3
1373
1374 p = Vector(2, 3, -4)

```

Dans l'exemple ci-dessus, la commande `p = Vector(2, 3, -4)` instancie un objet (dont le nom n'a pas d'importance), lui attache trois attributs, puis finalement le lie à la variable `p`.

Par une vilaine habitude, de nombreux programmeurs utilisent le même nom comme argument de `__init__()` et comme attributs des instances de la classe. Dans l'exemple, on écrirait plutôt

```

1375 class Vector():
1376     def __init__(self, abscissa, ordinate, height):
1377         self.abscissa = abscissa
1378         self.ordinate = ordinate
1379         self.height = height
1380
1381 p = Vector(2, 3, -4)

```

ce qui peut être perturbant quand on débute (puisque'il faut comprendre que la même syntaxe désigne des choses différentes).

Notez que le constructeur peut faire d'autres choses que des affectations. On pourrait avoir le code

```

1382 class Vector():
1383     def __init__(self, argument1, argument2, argument3):
1384         self.abscissa = argument1
1385         self.ordinate = argument2

```

```

1386         self.height = argument3
1387         self._euclidean_norm = argument1**2 + argument2**2 +
        ↪ argument3**2

```

dans lequel quatre attributs sont créés et initialisés à la construction.

**A.1.4. Noms réservés.** — La méthode `__init__()` n'est pas la seule méthode dont le nom est réservé par Python. De manière générale, il existe une série de noms réservés, encadrés par des doubles tirets bas, qui correspondent à des comportements particuliers de Python.

*Exemple A.6.* — La méthode `__add__()` permet d'indiquer le comportement de l'opérateur `+`; la méthode `__str__()` permet de fabriquer une description de l'objet conviviale pour l'utilisateur sous forme de chaîne de caractère. Dans le code suivant

```

1388 class Vector():
1389     def __init__(self, argument1, argument2, argument3):
1390         self.abscissa = argument1
1391         self.ordinate = argument2
1392         self.height = argument3
1393
1394     def __add__(self, other):
1395         return Vector(self.abscissa + other.abscissa,
1396                       self.ordinate + other.ordinate,
1397                       self.height + other.height)
1398
1399     def __str__(self):
1400         coords = map(str, [self.abscissa, self.ordinate,
1401                             ↪ self.height])
1402         return '(' + ', '.join(coords) + ')'
1403
1404 p1 = Vector(2, 3, -4)
1405 p2 = Vector(0, 5, 6)
1406 q = p1 + p2
1407 print(q)

```

deux phénomènes magiques se produisent. D'une part, Python est capable de calculer la somme `p+p` en invoquant en réalité `p1.__add__(p2)`. D'autre part, Python est capable d'afficher avec `print(q)` une version « jolie » du vecteur `q`.

Par ailleurs, un usage s'est installé parmi les programmeurs Python. Lorsqu'un attribut porte un nom préfixé par `_` (comme ci-dessus avec l'attribut `_euclidean_norm`), il est généralement considéré que l'intention du programmeur est de désigner une variable à usage interne. Certains langages de programmation (par exemple Java) sont plus stricts sur ce point : on peut ou on doit systématiquement déclarer si un attribut est privé (c'est-à-dire réservé à la cuisine interne de l'objet) ou public (c'est-à-dire accessible à l'utilisateur de l'objet).

## INDEX

- NP-complétude, 273
- accessible, 118
- algorithme, 9
- algorithme d'approximation, 161
- algorithme en ligne, 198
- algorithme Las Vegas, 32
- algorithme Monte Carlo, 32
- alphabet, 43
- arborescence, 121
- arbre, 120
- arbre binaire, 55
- arbre couvrant, 156
- arbre préfixe, 198
- arbre, graphe, 121
- automate fini déterministe (DFA), 217
- automate fini déterministe à spécification incomplète (DFAi), 220
- automates finis non-déterministes à transitions spontanées ( $\epsilon$ NFA), 228
- Bellman (algorithme de), 146
- Bellman Ford
  - algorithme, 153
- Bellman Ford (algorithme de), 152
- biparti (graphe), 162
- bord, 194
- calculable, 262
- calculable partielle, 262
- chaîne, 118
- chemin, 117
- chemin augmentant, 162
- classe **NP**, 270
- classe **P**, 270
- clique, 272
- codage de Gödel, 260
- complémentaire (langage), 244
- concaténation (de langages), 234
- connexité (graphe), 119
- Cook-Levin (théorème de), 274
- correction, 12
- coupe maximum, 276
- couplage, 161
- couverture, 165
- degré, 117
- Dijkstra (algorithme de), 149
- décidable, 262
- définition inductive, 47
- ensemble indépendant, 272
- expression régulière, 213
- facteur, 192
- feuille, 53
- Floyd Warshall (algorithme de), 154
- forte connexité, 135
- forêt, 121
- glouton, 24
- graphe hamiltonien, 270
- graphe non orienté, 118
- graphe orienté, 117
- Huffman (algorithme de), 209
- Kahn (algorithme de), 131
- Kleene (théorème de récursion de), 268
- Knuth Morris Pratt (algorithme de), 197
- Kosaraju Sharir (algorithme de), 138
- Kruskal
  - algorithme, 157
- Kruskal (algorithme de), 156
- König, Egervary (théorème de), 165
- langage, 44
- langage rationnel, 212

- liste, 49
- mot, 44
- NFA, *voir* automate fini non-déterministe
- numérotation topologique, 132
- objet, 285
- parcours en largeur, 141
- parcours en profondeur, 125
- plus court chemin, 145
- pompage (lemme de), 246
- preuve par induction structurelle, 63
- Prim (algorithme de), 160
- principe d'optimalité de Bellman, 29
- problème d'optimisation, 276
- problème de décision, 261
- problème de l'arrêt, 266
- préfixe, 191
- reconnaître, *voir* accepter
- règle d'inférence, 47
- s-m-n (théorème), 266
- Sat, 274
- semi-décidable, 262
- sommet interne, 53
- sommet-couverture, 272
- sous-mot, 192
- spécification, 10
- suffixe, 191
- terminaison, 12, 261
- thèse de Church-Turing, 259
- transition (relation de), 224
- tri fusion, 27
- tri topologique (algorithme de), 133
- trie, 198
- Turing (théorème de), 267
- union (de langages), 233
- étoile de Kleene, 235

## BIBLIOGRAPHIE

- [BCF<sup>+</sup>22] T. BALABONSKI, S. CONCHON, J.-C. FILLIÂTRE, K. NGUYEN & L. SARTRE – *Informatique - MP2I/MPI - CPGE 1re et 2e années - Cours et exercices corrigés*, Ellipses, 2022.
- [Bou23] O. BOURNEZ – *Fondements de l’informatique : logique, modèles et calculs. notes de cours (inf412)*, Polycopié École Polytechnique, 2023.
- [CH19] I. CHARON & O. HUDRY – *Introduction à l’optimisation continue et discrète*, Lavoisier, 2019.
- [CLRS09] T. H. CORMEN, C. E. LEISERSON, R. L. RIVEST & C. STEIN – *Introduction to algorithms, 3rd edition*, MIT Press, 2009.
- [Eri19] J. ERICKSON – *Algorithms*, 2019, <http://jeffe.cs.illinois.edu/teaching/algorithms/>.
- [Mad23] D. A. MADORE – *Théorie des langages, notes de cours (inf105)*, Polycopié Télécom Paris, 2023.